

第 3-15 章

纯 PyTorch 代码改写

《动手学深度学习（PyTorch 第二版）》代码整理版

来源：整理自《动手学深度学习（PyTorch 第二版）》第 3-15 章；原始示例来自 D2L 中文 PyTorch 版本。

改写说明：已移除 d2l 依赖，使用 PyTorch / torchvision 重新组织代码和结构图。

包含 13 个章节、95 个代码小节和共享工具模块；每节均标明章节、网络/流程类别，并配有结构图。

阅读提示：蓝色/绿色框表示网络结构；橙色框表示数据管线；紫色或青色框表示训练、工程或性能流程。

说明：本 PDF 聚焦代码；可运行源码同时保存在同目录的 ZIP 包中。

目录

目录	2
网络结构图总览	6
共享工具	14
chapters/common.py	14
第 03 章：线性神经网络	122
第 03 章 · 线性回归 · chapters/chapter_03_linear_networks/linear_regression.py	122
第 03 章 · 线性回归 · chapters/chapter_03_linear_networks/linear_regression_scratch.py	123
第 03 章 · 线性回归 · chapters/chapter_03_linear_networks/linear_regression_concise.py	125
第 03 章 · 图像分类数据管线 · chapters/chapter_03_linear_networks/image_classification_dataset.py	126
第 03 章 · Softmax 回归 · chapters/chapter_03_linear_networks/softmax_regression_scratch.py	128
第 03 章 · Softmax 回归 · chapters/chapter_03_linear_networks/softmax_regression_concise.py	131
第 04 章：多层感知机	132
第 04 章 · 多层感知机 (MLP) · chapters/chapter_04_multilayer_perceptrons/mlp.py	132
第 04 章 · 多层感知机 (MLP) · chapters/chapter_04_multilayer_perceptrons/mlp_scratch.py	133
第 04 章 · 多层感知机 (MLP) · chapters/chapter_04_multilayer_perceptrons/mlp_concise.py	134
第 04 章 · MLP 的泛化与正则化 · chapters/chapter_04_multilayer_perceptrons/underfit_overfit.py	135
第 04 章 · MLP 的泛化与正则化 · chapters/chapter_04_multilayer_perceptrons/weight_decay.py	136
第 04 章 · MLP 的泛化与正则化 · chapters/chapter_04_multilayer_perceptrons/dropout.py	138
第 04 章 · MLP 的泛化与正则化 · chapters/chapter_04_multilayer_perceptrons/numerical_stability_and_init.py	140
第 04 章 · MLP 回归 · chapters/chapter_04_multilayer_perceptrons/kaggle_house_price.py	141
第 05 章：深度学习计算	145
第 05 章 · nn.Module 组合模型 · chapters/chapter_05_deep_learning_computation/model_construction.py	145
第 05 章 · 参数管理 · chapters/chapter_05_deep_learning_computation/parameters.py	147
第 05 章 · 自定义层 MLP · chapters/chapter_05_deep_learning_computation/custom_layer.py	149
第 05 章 · 模型保存与加载 · chapters/chapter_05_deep_learning_computation/read_write.py	150
第 05 章 · GPU 训练流程 · chapters/chapter_05_deep_learning_computation/use_gpu.py	151
第 05 章 · LazyLinear MLP · chapters/chapter_05_deep_learning_computation/deferred_init.py	152
第 06 章：卷积神经网络	154
第 06 章 · 二维卷积层 · chapters/chapter_06_convolutional_neural_networks/conv_layer.py	154
第 06 章 · 卷积尺寸变换 · chapters/chapter_06_convolutional_neural_networks/padding_and_strides.py	155
第 06 章 · 多通道卷积 · chapters/chapter_06_convolutional_neural_networks/channels.py	156
第 06 章 · 池化层 · chapters/chapter_06_convolutional_neural_networks/pooling.py	157
第 06 章 · LeNet · chapters/chapter_06_convolutional_neural_networks/lenet.py	159
第 07 章：现代卷积神经网络	161
第 07 章 · AlexNet · chapters/chapter_07_modern_convolutional_networks/alexnet.py	161

第 07 章 · VGG · chapters/chapter_07_modern_convolutional_networks/vgg.py	162
第 07 章 · NiN (Network in Network) · chapters/chapter_07_modern_convolutional_networks/nin.py	163
第 07 章 · GoogLeNet / Inception · chapters/chapter_07_modern_convolutional_networks/googlenet.py	164
第 07 章 · 带 BatchNorm 的 CNN · chapters/chapter_07_modern_convolutional_networks/batch_norm.py	166
第 07 章 · ResNet · chapters/chapter_07_modern_convolutional_networks/resnet.py	168
第 07 章 · DenseNet · chapters/chapter_07_modern_convolutional_networks/densenet.py	169
第 08 章：循环神经网络	171
第 08 章 · 序列预测流程 · chapters/chapter_08_recurrent_neural_networks/sequence.py	171
第 08 章 · 语言模型数据管线 · chapters/chapter_08_recurrent_neural_networks/text_preprocessing.py	173
第 08 章 · 语言模型数据管线 · chapters/chapter_08_recurrent_neural_networks/language_models_and_dataset.py	175
第 08 章 · RNN 语言模型 · chapters/chapter_08_recurrent_neural_networks/rnn.py	177
第 08 章 · RNN 语言模型 · chapters/chapter_08_recurrent_neural_networks/rnn_scratch.py	177
第 08 章 · RNN 语言模型 · chapters/chapter_08_recurrent_neural_networks/rnn_concise.py	180
第 09 章：现代循环神经网络	182
第 09 章 · GRU 语言模型 · chapters/chapter_09_modern_recurrent_neural_networks/gru.py	182
第 09 章 · LSTM 语言模型 · chapters/chapter_09_modern_recurrent_neural_networks/lstm.py	183
第 09 章 · 深层 RNN · chapters/chapter_09_modern_recurrent_neural_networks/deep_rnn.py	185
第 09 章 · 双向 RNN · chapters/chapter_09_modern_recurrent_neural_networks/bi_rnn.py	186
第 09 章 · 机器翻译数据管线 · chapters/chapter_09_modern_recurrent_neural_networks/machine_translation_and_dataset.py	187
第 09 章 · Encoder-Decoder · chapters/chapter_09_modern_recurrent_neural_networks/encoder_decoder.py	189
第 09 章 · Seq2Seq · chapters/chapter_09_modern_recurrent_neural_networks/seq2seq.py	190
第 10 章：注意力机制	194
第 10 章 · 注意力机制 · chapters/chapter_10_attention_mechanisms/attention_cues.py	194
第 10 章 · Nadaraya-Watson 注意力 · chapters/chapter_10_attention_mechanisms/nadaraya_watson.py	195
第 10 章 · 注意力评分函数 · chapters/chapter_10_attention_mechanisms/attention_scoring_functions.py	197
第 10 章 · Bahdanau 加性注意力 · chapters/chapter_10_attention_mechanisms/bahdanau_attention.py	199
第 10 章 · Multi-Head Attention · chapters/chapter_10_attention_mechanisms/multihead_attention.py	201
第 10 章 · Self-Attention + 位置编码 · chapters/chapter_10_attention_mechanisms/self_attention_and_positional_encoding.py	202
第 10 章 · Transformer · chapters/chapter_10_attention_mechanisms/transformer.py	204
第 11 章：优化算法	208
第 11 章 · 优化算法训练流程 · chapters/chapter_11_optimization/optimization_intro.py	208
第 11 章 · 优化算法训练流程 · chapters/chapter_11_optimization/convexity.py	209
第 11 章 · 优化算法训练流程 · chapters/chapter_11_optimization/gd.py	210
第 11 章 · 优化算法训练流程 · chapters/chapter_11_optimization/sgd.py	212
第 11 章 · 优化算法训练流程 · chapters/chapter_11_optimization/minibatch_sgd.py	213
第 11 章 · 优化算法训练流程 · chapters/chapter_11_optimization/momentum.py	216

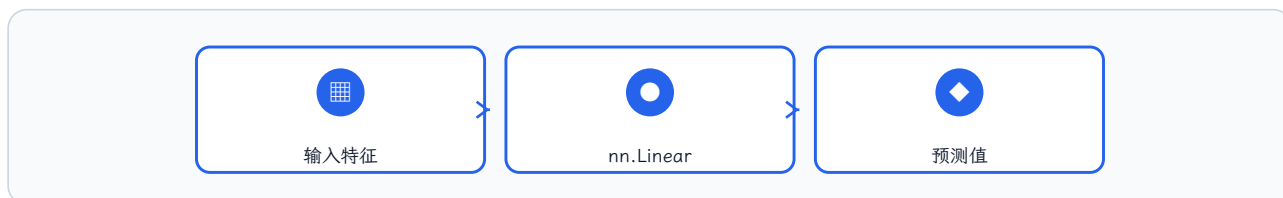
第 11 章 · 优化算法训练流程 · chapters/chapter_11_optimization/adagrad.py	217
第 11 章 · 优化算法训练流程 · chapters/chapter_11_optimization/rmsprop.py	218
第 11 章 · 优化算法训练流程 · chapters/chapter_11_optimization/adadelta.py	220
第 11 章 · 优化算法训练流程 · chapters/chapter_11_optimization/adam.py	221
第 11 章 · 优化算法训练流程 · chapters/chapter_11_optimization/lr_scheduler.py	222
第 12 章：计算性能	225
第 12 章 · 计算图/执行流程 · chapters/chapter_12_computational_performance/hybridize.py	225
第 12 章 · 异步计算流程 · chapters/chapter_12_computational_performance/async_computation.py	227
第 12 章 · 自动并行流程 · chapters/chapter_12_computational_performance/auto_parallelism.py	228
第 12 章 · 多 GPU 数据并行 · chapters/chapter_12_computational_performance/multiple_gpus.py	229
第 12 章 · 多 GPU 数据并行 · chapters/chapter_12_computational_performance/multiple_gpus_concise.py	231
第 13 章：计算机视觉	233
第 13 章 · 图像增强管线 · chapters/chapter_13_computer_vision/image_augmentation.py	233
第 13 章 · 预训练模型微调 · chapters/chapter_13_computer_vision/fine_tuning.py	235
第 13 章 · 目标检测数据与锚框 · chapters/chapter_13_computer_vision/bounding_box.py	237
第 13 章 · 目标检测数据与锚框 · chapters/chapter_13_computer_vision/anchor.py	238
第 13 章 · 目标检测数据与锚框 · chapters/chapter_13_computer_vision/multiscale_object_detection.py	242
第 13 章 · 目标检测数据与锚框 · chapters/chapter_13_computer_vision/object_detection_dataset.py	243
第 13 章 · SSD · chapters/chapter_13_computer_vision/ssd.py	244
第 13 章 · R-CNN · chapters/chapter_13_computer_vision/rcnn.py	248
第 13 章 · 语义分割数据管线 · chapters/chapter_13_computer_vision/semantic_segmentation_and_dataset.py	248
第 13 章 · 转置卷积 · chapters/chapter_13_computer_vision/transposed_conv.py	251
第 13 章 · FCN (全卷积网络) · chapters/chapter_13_computer_vision/fcn.py	252
第 13 章 · 神经风格迁移 · chapters/chapter_13_computer_vision/neural_style.py	254
第 13 章 · 图像分类训练管线 · chapters/chapter_13_computer_vision/kaggle_cifar10.py	256
第 13 章 · 图像分类训练管线 · chapters/chapter_13_computer_vision/kaggle_dog.py	259
第 14 章：自然语言处理：预训练	262
第 14 章 · 词嵌入数据管线 · chapters/chapter_14_nlp_pretraining/word_embedding_dataset.py	262
第 14 章 · Word2Vec / Skip-gram · chapters/chapter_14_nlp_pretraining/word2vec_pretraining.py	265
第 14 章 · 子词嵌入 · chapters/chapter_14_nlp_pretraining/subword_embedding.py	267
第 14 章 · 词向量相似度/类比 · chapters/chapter_14_nlp_pretraining/similarity_analogy.py	268
第 14 章 · BERT Encoder · chapters/chapter_14_nlp_pretraining/bert.py	270
第 14 章 · BERT 预训练数据管线 · chapters/chapter_14_nlp_pretraining/bert_dataset.py	272
第 14 章 · BERT 预训练 · chapters/chapter_14_nlp_pretraining/bert_pretraining.py	275
第 15 章：自然语言处理：应用	278
第 15 章 · 情感分析数据管线 · chapters/chapter_15_nlp_applications/sentiment_analysis_and_dataset.py	278
第 15 章 · BiRNN 情感分类器 · chapters/chapter_15_nlp_applications/sentiment_analysis_rnn.py	279

第 15 章 • TextCNN 情感分类器 • chapters/chapter_15_nlp_applications/sentiment_analysis_cnn.py	281
第 15 章 • 自然语言推断数据管线 • chapters/chapter_15_nlp_applications/natural_language_inference_and_dataset.py	
第 15 章 • Attention NLI • chapters/chapter_15_nlp_applications/natural_language_inference_attention.py . . .	285
第 15 章 • BERT NLI • chapters/chapter_15_nlp_applications/natural_language_inference_bert.py	287

网络结构图总览

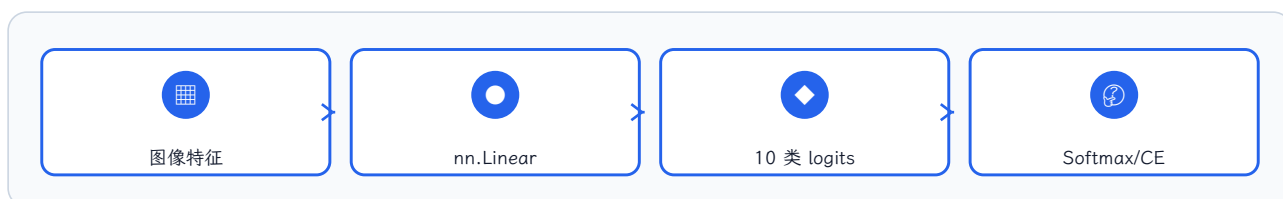
以下按章节集中展示本书中的主要网络。每张图上方都写明“第几章·网络名称”，方便先看结构、再回到对应代码。优化算法、数据集和设备管理等非网络内容仍保留在各章节中，并明确标注为流程。

第 03 章·线性回归



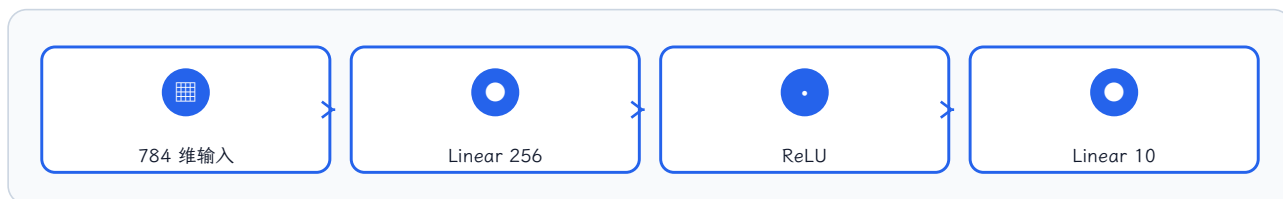
单层线性映射；训练目标是最小化均方误差。

第 03 章·Softmax 回归



单层多类分类器；输出类别 logits 后交给交叉熵损失。

第 04 章·多层感知机 (MLP)



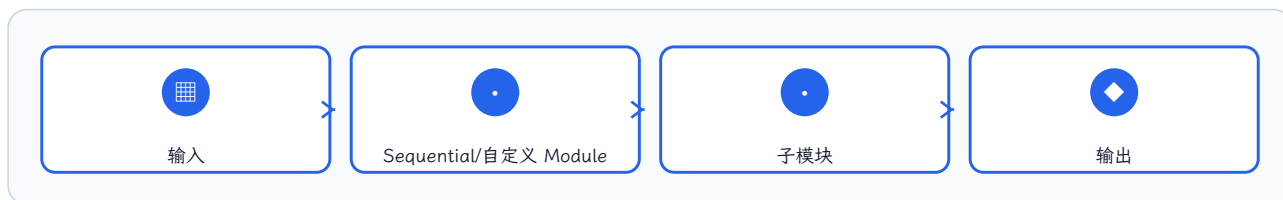
全连接分类网络；本章用它从线性模型过渡到深层网络。

第 04 章·MLP 回归



面向连续值房价预测的全连接回归网络。

第 05 章·nn.Module 组合模型



展示 PyTorch 如何用模块嵌套表达网络结构。

第 05 章 · 自定义层 MLP



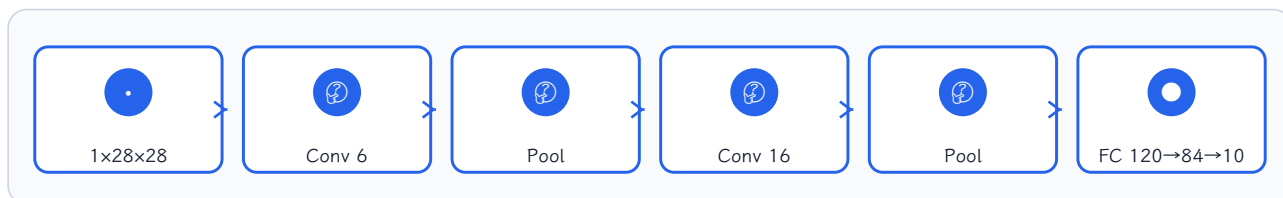
把自定义层组合到 PyTorch 网络中。

第 05 章 · LazyLinear MLP



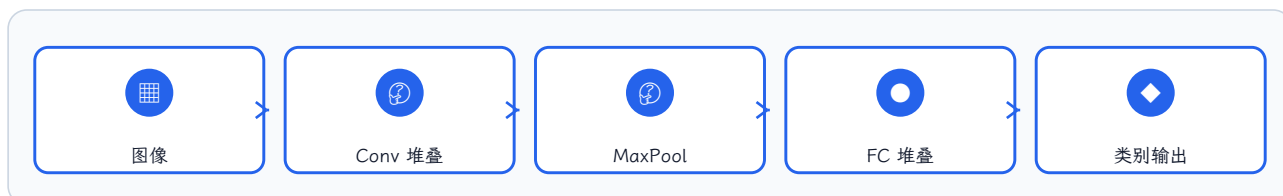
延后推断输入维度的 MLP。

第 06 章 · LeNet



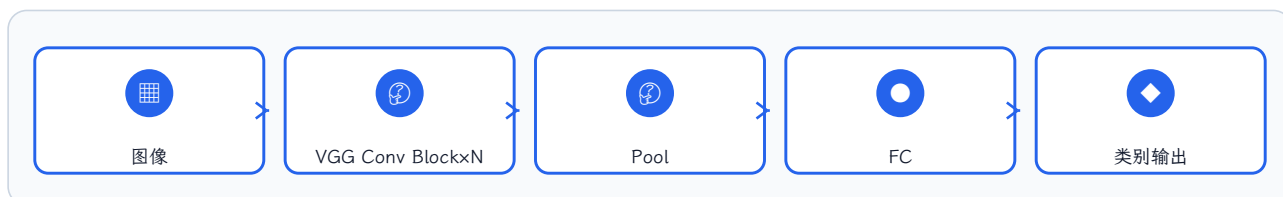
第 6 章经典卷积神经网络。

第 07 章 · AlexNet



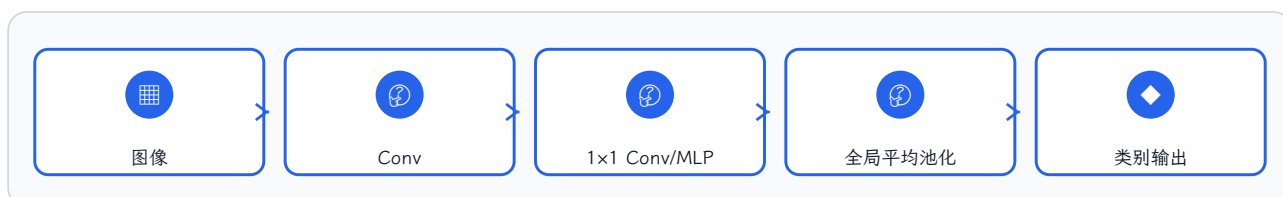
第 7 章深层卷积网络代表。

第 07 章 · VGG



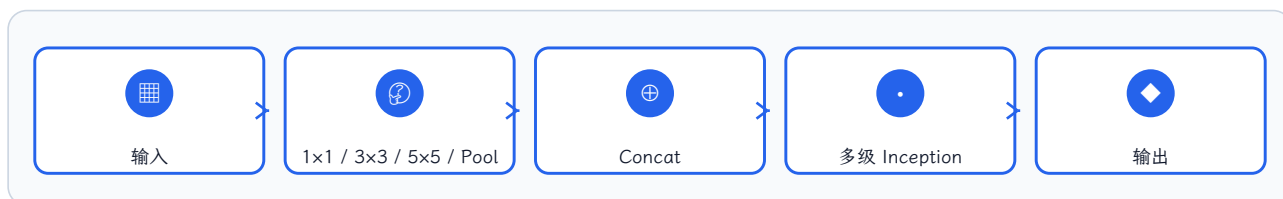
第 7 章使用重复卷积块构建深度网络。

第 07 章 · NiN (Network in Network)



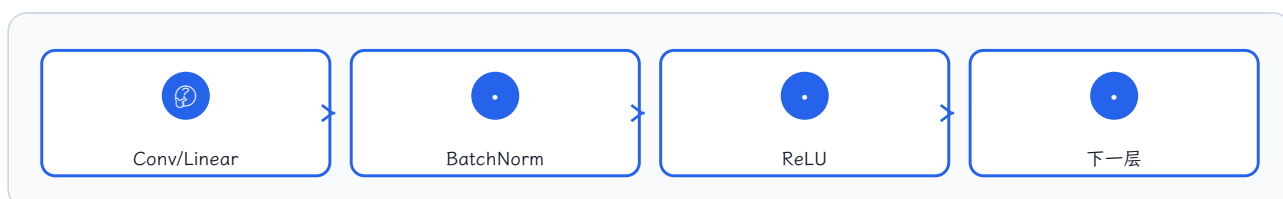
第 7 章用 1x1 卷积替代部分全连接表达。

第 07 章 · GoogLeNet / Inception



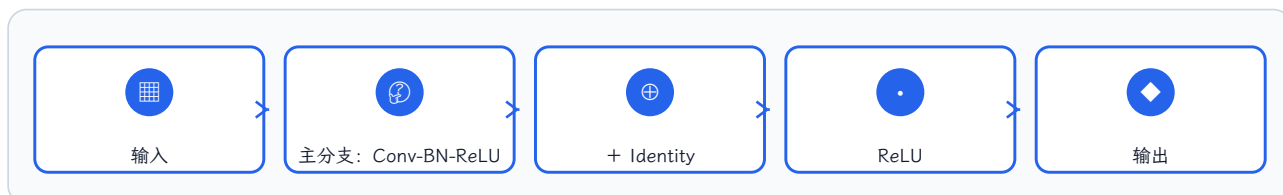
第 7 章并行多尺度卷积结构。

第 07 章 · 带 BatchNorm 的 CNN



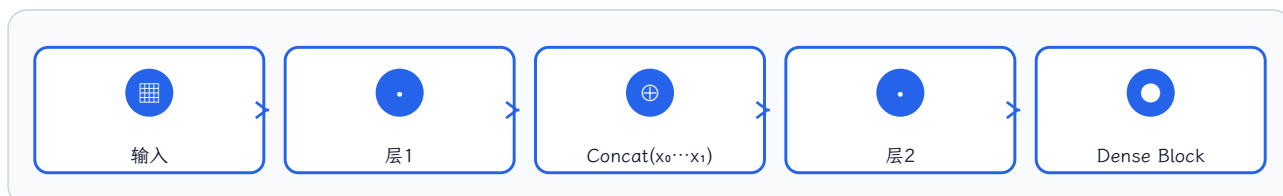
第 7 章把归一化层插入卷积或全连接块。

第 07 章 · ResNet



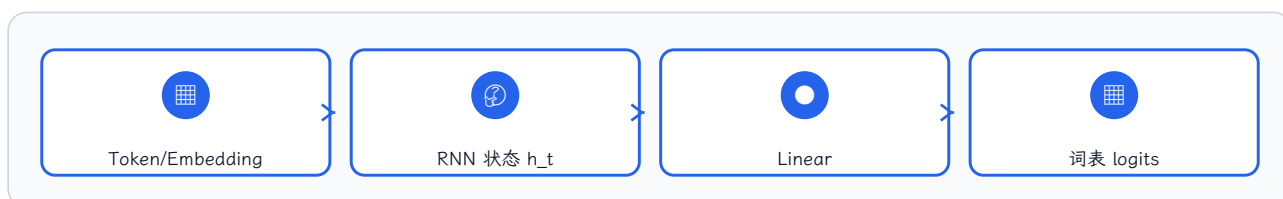
第 7 章残差连接网络。

第 07 章 · DenseNet



第 7 章密集连接网络。

第 08 章 · RNN 语言模型



第 8 章基础循环神经网络语言模型。

第 09 章 · GRU 语言模型



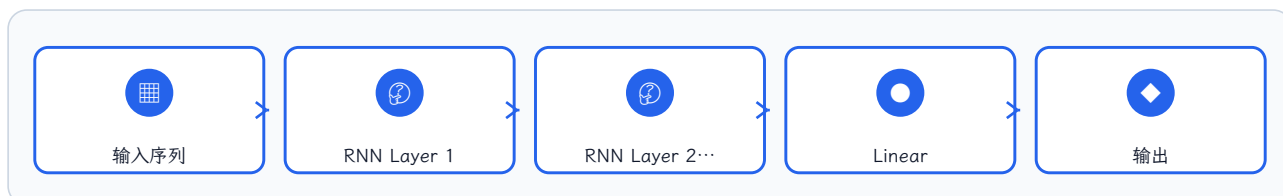
第 9 章门控循环单元。

第 09 章 · LSTM 语言模型



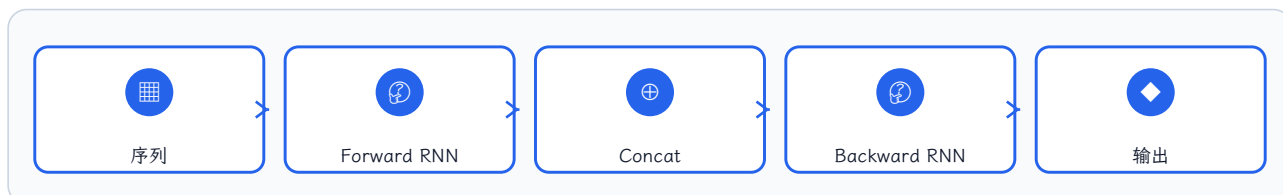
第 9 章长短期记忆网络。

第 09 章 · 深层 RNN



第 9 章堆叠多层循环网络。

第 09 章 · 双向 RNN



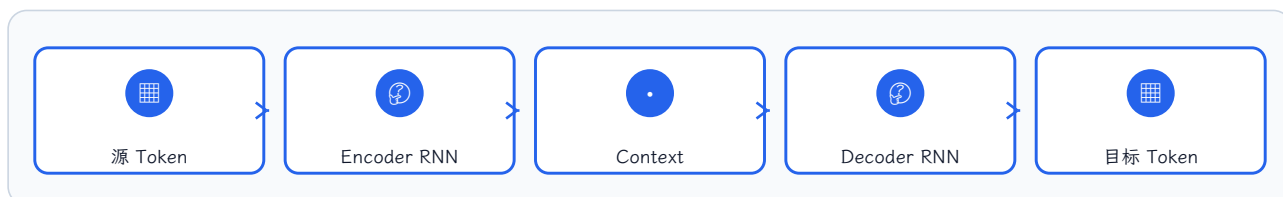
第 9 章双向循环网络。

第 09 章 · Encoder-Decoder



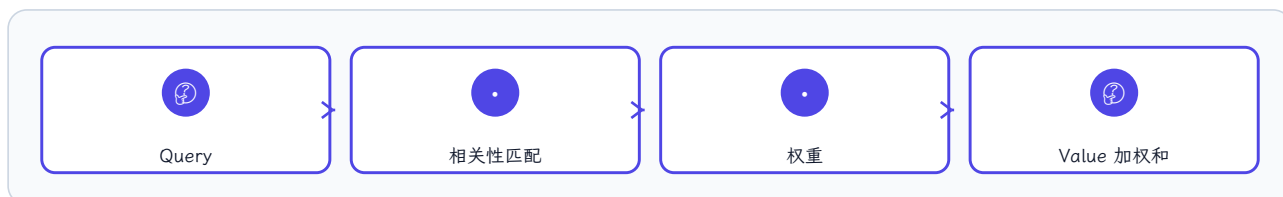
第 9 章序列到序列框架。

第 09 章 · Seq2Seq



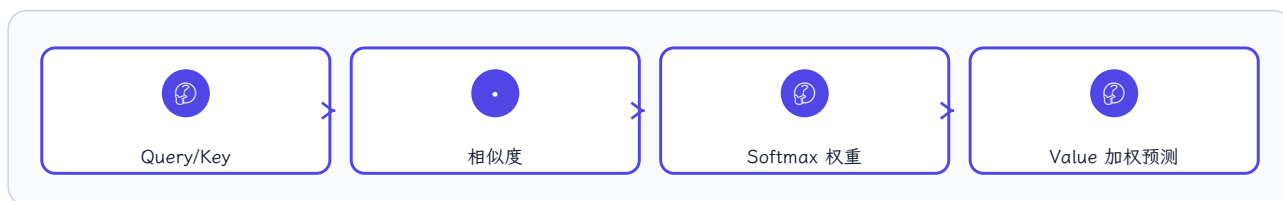
第 9 章机器翻译基础网络。

第 10 章 · 注意力机制



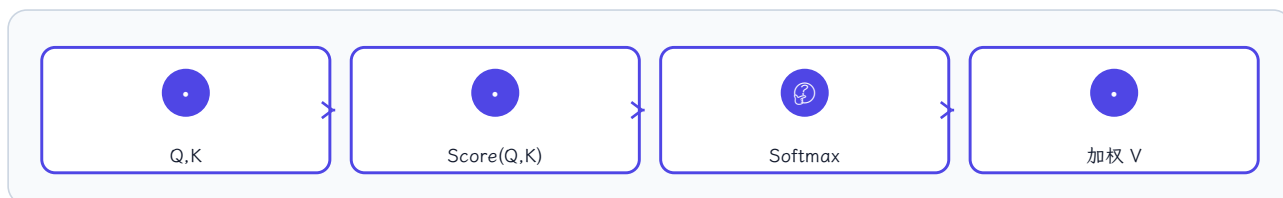
第 10 章注意力的基本抽象。

第 10 章 · Nadaraya-Watson 注意力



第 10 章注意力加权回归。

第 10 章 · 注意力评分函数



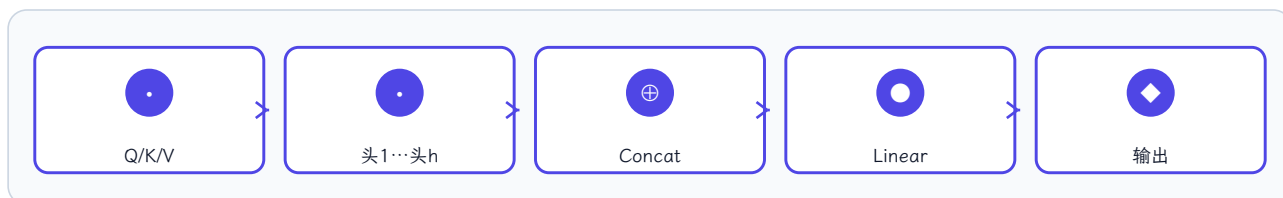
第 10 章点积与加性评分函数。

第 10 章 · Bahdanau 加性注意力



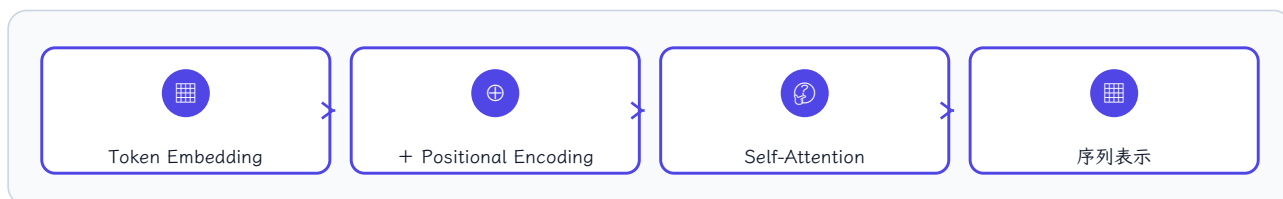
第 10 章序列到序列中的加性注意力。

第 10 章 · Multi-Head Attention



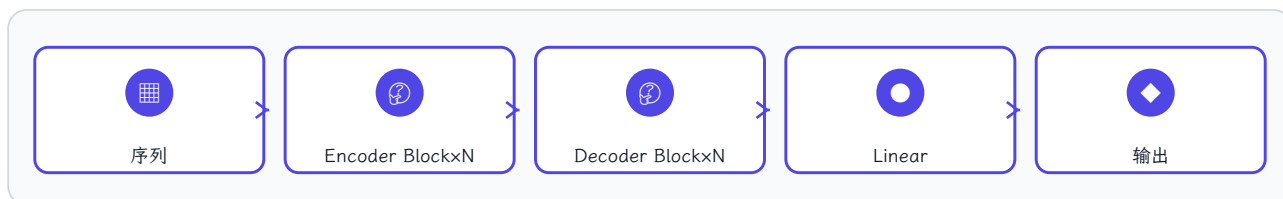
第 10 章多头注意力。

第 10 章 · Self-Attention + 位置编码



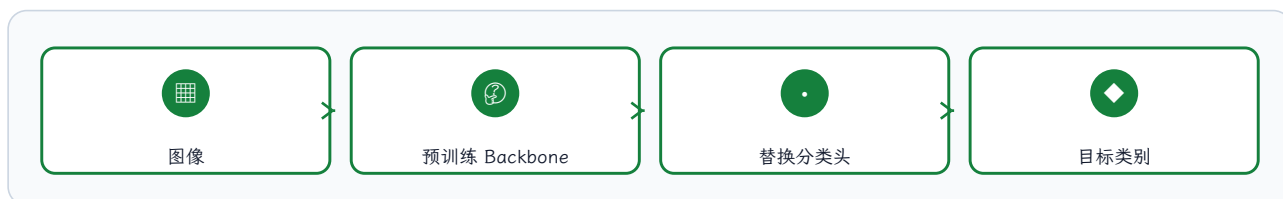
第 10 章 Transformer 的核心输入表示。

第 10 章 · Transformer



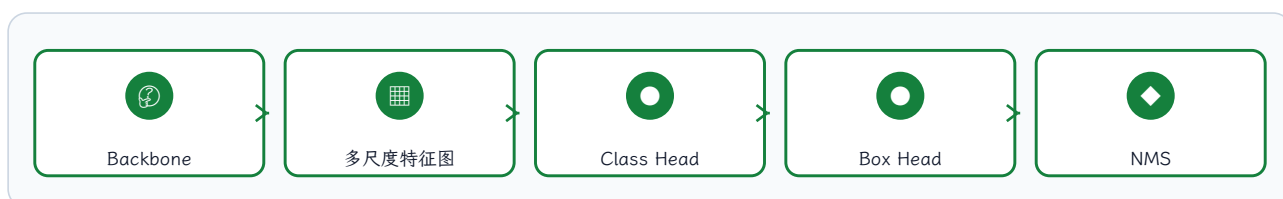
第 10 章基于注意力的编码器-解码器网络。

第 13 章 · 预训练模型微调



第 13 章迁移学习/微调结构。

第 13 章 · SSD



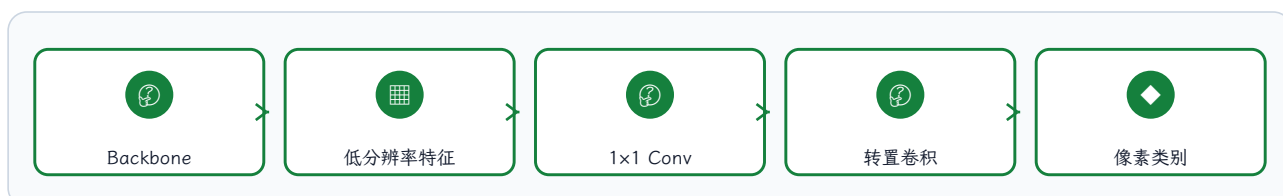
第 13 章单阶段多尺度目标检测网络。

第 13 章 · R-CNN



第 13 章候选区域式目标检测流程。

第 13 章 · FCN (全卷积网络)



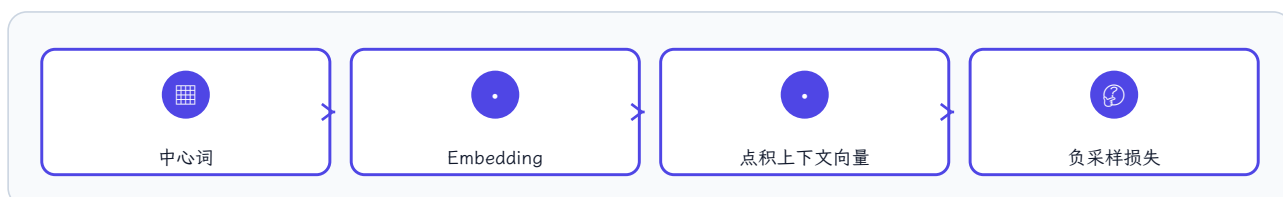
第 13 章像素级语义分割网络。

第 13 章 · 神经风格迁移



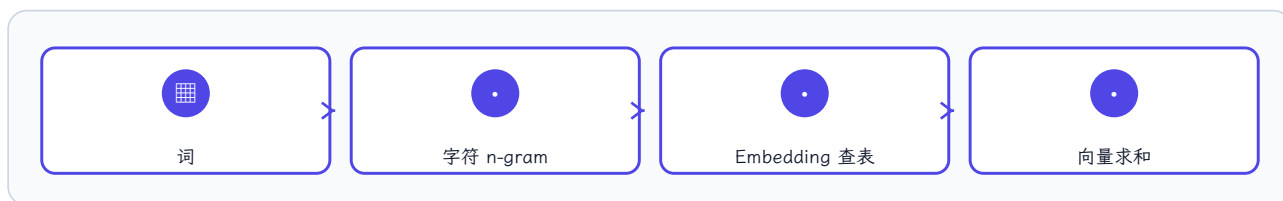
第 13 章以特征提取器和损失函数驱动图像生成。

第 14 章 · Word2Vec / Skip-gram



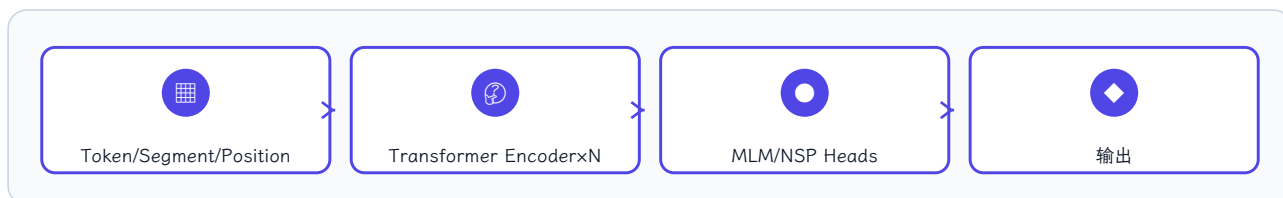
第 14 章经典词向量预训练模型。

第 14 章 · 子词嵌入



第 14 章利用子词信息构造词表示。

第 14 章 · BERT Encoder



第 14 章双向 Transformer 预训练编码器。

第 14 章 · BERT 预训练



第 14 章 BERT 的预训练任务与损失。

第 15 章 · BiRNN 情感分类器



第 15 章基于双向循环网络的情感分类。

第 15 章 · TextCNN 情感分类器



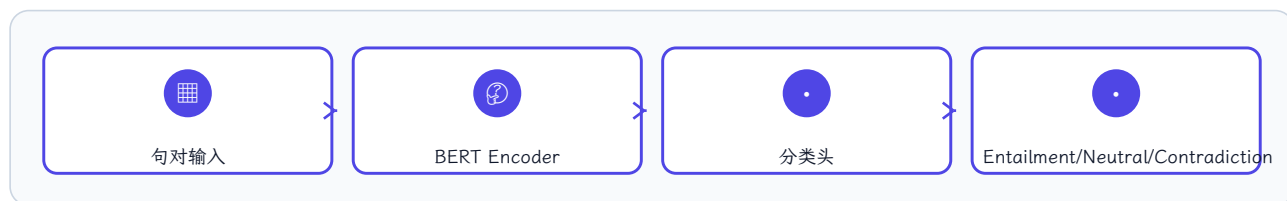
第 15 章基于卷积的文本分类。

第 15 章 · Attention NLI



第 15 章基于注意力的自然语言推断。

第 15 章 · BERT NLI



第 15 章用预训练 BERT 完成自然语言推断。

共享工具

chapters/common.py

原生 PyTorch 数据、训练、绘图和评估工具；共享工具不属于某一个具体网络。

```
"""Shared native PyTorch utilities for the chapter 3+ rewrite.

This module contains only PyTorch/torchvision-based support code. It replaces
the book's original helper namespace so each generated chapter is standalone.
"""
DATA_HUB = dict()
# Public datasets used by the book's examples.
DATA_URL = 'https://d2l-data.s3-accelerate.amazonaws.com/'

import numpy as np
import torch
import torchvision
from PIL import Image
from torch import nn
from torch.nn import functional as F
from torch.utils import data
from torchvision import transforms

nn_Module = nn.Module

##### WARNING #####
# The below part is generated automatically through:
#   nativebook build lib
# Don't edit it directly

import collections
import hashlib
import inspect
import math
import os
import random
import re
import shutil
import sys
import tarfile
import time
import zipfile
from collections import defaultdict
import pandas as pd
import requests
from IPython import display
from matplotlib import pyplot as plt
from matplotlib_inline import backend_inline

native = sys.modules[__name__]

import numpy as np
import torch
import torchvision
from PIL import Image
from scipy.spatial import distance_matrix
from torch import nn
from torch.nn import functional as F
from torchvision import transforms

def use_svg_display():
    """Use the svg format to display a plot in Jupyter.

    Defined in :numref:`sec_calculus`"""
    backend_inline.set_matplotlib_formats('svg')

def set_figsize(figsize=(3.5, 2.5)):
    """Set the figure size for matplotlib.

    Defined in :numref:`sec_calculus`"""
    use_svg_display()
    native.plt.rcParams['figure.figsize'] = figsize
```

```

def set_axes(axes, xlabel, ylabel, xlim, ylim, xscale, yscale, legend):
    """Set the axes for matplotlib.

    Defined in :numref:`sec_calculus`"""
    axes.set_xlabel(xlabel), axes.set_ylabel(ylabel)
    axes.set_xscale(xscale), axes.set_yscale(yscale)
    axes.set_xlim(xlim), axes.set_ylim(ylim)
    if legend:
        axes.legend(legend)
    axes.grid()

def plot(X, Y=None, xlabel=None, ylabel=None, legend=[], xlim=None,
        ylim=None, xscale='linear', yscale='linear',
        fmts=('-', 'm--', 'g-', 'r:'), figsize=(3.5, 2.5), axes=None):
    """Plot data points.

    Defined in :numref:`sec_calculus`"""

    def has_one_axis(X): # True if X (tensor or list) has 1 axis
        return (hasattr(X, "ndim") and X.ndim == 1 or isinstance(X, list)
                and not hasattr(X[0], "__len__"))

    if has_one_axis(X): X = [X]
    if Y is None:
        X, Y = [[]] * len(X), X
    elif has_one_axis(Y):
        Y = [Y]
    if len(X) != len(Y):
        X = X * len(Y)

    set_figsize(figsize)
    if axes is None:
        axes = native.plt.gca()
    axes.cla()
    for x, y, fmt in zip(X, Y, fmts):
        axes.plot(x,y,fmt) if len(x) else axes.plot(y,fmt)
    set_axes(axes, xlabel, ylabel, xlim, ylim, xscale, yscale, legend)

def add_to_class(Class):
    """Register functions as methods in created class.

    Defined in :numref:`sec_oo-design`"""
    def wrapper(obj):
        setattr(Class, obj.__name__, obj)
    return wrapper

class HyperParameters:
    """The base class of hyperparameters."""
    def save_hyperparameters(self, ignore=[]):
        """Defined in :numref:`sec_oo-design`"""
        raise NotImplemented

def save_hyperparameters(self, ignore=[]):
    """Save function arguments into class attributes.

    Defined in :numref:`sec_utils`"""
    frame = inspect.currentframe().f_back
    _, _, _, local_vars = inspect.getargvalues(frame)
    self.hparams = {k:v for k, v in local_vars.items()
                    if k not in set(ignore+['self']) and not k.startswith('_')}
    for k, v in self.hparams.items():
        setattr(self, k, v)

class ProgressBoard(native.HyperParameters):
    """The board that plots data points in animation.

    Defined in :numref:`sec_oo-design`"""
    def __init__(self, xlabel=None, ylabel=None, xlim=None,
                 ylim=None, xscale='linear', yscale='linear',
                 ls=['-', '--', '-.', ':'], colors=['C0', 'C1', 'C2', 'C3'],
                 fig=None, axes=None, figsize=(3.5, 2.5), display=True):
        self.save_hyperparameters()

    def draw(self, x, y, label, every_n=1):
        raise NotImplemented

    def draw(self, x, y, label, every_n=1):

```

```

"""Defined in :numref:`sec_utils`"""
Point = collections.namedtuple('Point', ['x', 'y'])
if not hasattr(self, 'raw_points'):
    self.raw_points = collections.OrderedDict()
    self.data = collections.OrderedDict()
if label not in self.raw_points:
    self.raw_points[label] = []
    self.data[label] = []
points = self.raw_points[label]
line = self.data[label]
points.append(Point(x, y))
if len(points) != every_n:
    return
mean = lambda x: sum(x) / len(x)
line.append(Point(mean([p.x for p in points]),
                    mean([p.y for p in points])))
points.clear()
if not self.display:
    return
native.use_svg_display()
if self.fig is None:
    self.fig = native.plt.figure(figsize=self.figsize)
plt_lines, labels = [], []
for (k, v), ls, color in zip(self.data.items(), self.ls, self.colors):
    plt_lines.append(native.plt.plot([p.x for p in v], [p.y for p in v],
                                     linestyle=ls, color=color)[0])
    labels.append(k)
axes = self.axes if self.axes else native.plt.gca()
if self.xlim: axes.set_xlim(self.xlim)
if self.ylim: axes.set_ylim(self.ylim)
if not self.xlabel: self.xlabel = self.x
axes.set_xlabel(self.xlabel)
axes.set_ylabel(self.ylabel)
axes.set_xscale(self.xscale)
axes.set_yscale(self.yscale)
axes.legend(plt_lines, labels)
display.display(self.fig)
display.clear_output(wait=True)

```

```

class Module(native.nn.Module, native.HyperParameters):
    """The base class of models.

    Defined in :numref:`sec_oo-design`"""
    def __init__(self, plot_train_per_epoch=2, plot_valid_per_epoch=1):
        super().__init__()
        self.save_hyperparameters()
        self.board = ProgressBoard()

    def loss(self, y_hat, y):
        raise NotImplementedError

    def forward(self, X):
        assert hasattr(self, 'net'), 'Neural network is defined'
        return self.net(X)

    def plot(self, key, value, train):
        """Plot a point in animation."""
        assert hasattr(self, 'trainer'), 'Trainer is not initied'
        self.board.xlabel = 'epoch'
        if train:
            x = self.trainer.train_batch_idx / \
                self.trainer.num_train_batches
            n = self.trainer.num_train_batches / \
                self.plot_train_per_epoch
        else:
            x = self.trainer.epoch + 1
            n = self.trainer.num_val_batches / \
                self.plot_valid_per_epoch
        self.board.draw(x, native.numpy(native.to(value, native.cpu()))),
            ('train_' if train else 'val_') + key,
            every_n=int(n))

    def training_step(self, batch):
        l = self.loss(self(*batch[:-1]), batch[-1])
        self.plot('loss', l, train=True)
        return l

```

```

def validation_step(self, batch):
    l = self.loss(self(*batch[:-1]), batch[-1])
    self.plot('loss', l, train=False)

def configure_optimizers(self):
    raise NotImplementedError

def configure_optimizers(self):
    """Defined in :numref:`sec_classification`"""
    return torch.optim.SGD(self.parameters(), lr=self.lr)

def apply_init(self, inputs, init=None):
    """Defined in :numref:`sec_lazy_init`"""
    self.forward(*inputs)
    if init is not None:
        self.net.apply(init)

class DataModule(native.HyperParameters):
    """The base class of data.

    Defined in :numref:`subsec_oo-design-models`"""
    def __init__(self, root='./data', num_workers=4):
        self.save_hyperparameters()

    def get_dataloader(self, train):
        raise NotImplementedError

    def train_dataloader(self):
        return self.get_dataloader(train=True)

    def val_dataloader(self):
        return self.get_dataloader(train=False)

    def get_tensorloader(self, tensors, train, indices=slice(0, None)):
        """Defined in :numref:`sec_synthetic-regression-data`"""
        tensors = tuple(a[indices] for a in tensors)
        dataset = torch.utils.data.TensorDataset(*tensors)
        return torch.utils.data.DataLoader(dataset, self.batch_size,
                                           shuffle=train)

class Trainer(native.HyperParameters):
    """The base class for training models with data.

    Defined in :numref:`subsec_oo-design-models`"""
    def __init__(self, max_epochs, num_gpus=0, gradient_clip_val=0):
        self.save_hyperparameters()
        assert num_gpus == 0, 'No GPU support yet'

    def prepare_data(self, data):
        self.train_dataloader = data.train_dataloader()
        self.val_dataloader = data.val_dataloader()
        self.num_train_batches = len(self.train_dataloader)
        self.num_val_batches = (len(self.val_dataloader)
                                if self.val_dataloader is not None else 0)

    def prepare_model(self, model):
        model.trainer = self
        model.board.xlim = [0, self.max_epochs]
        self.model = model

    def fit(self, model, data):
        self.prepare_data(data)
        self.prepare_model(model)
        self.optim = model.configure_optimizers()
        self.epoch = 0
        self.train_batch_idx = 0
        self.val_batch_idx = 0
        for self.epoch in range(self.max_epochs):
            self.fit_epoch()

    def fit_epoch(self):
        raise NotImplementedError

    def prepare_batch(self, batch):
        """Defined in :numref:`sec_linear_scratch`"""
        return batch

```

```

def fit_epoch(self):
    """Defined in :numref:`sec_linear_scratch`"""
    self.model.train()
    for batch in self.train_dataloader:
        loss = self.model.training_step(self.prepare_batch(batch))
        self.optim.zero_grad()
        with torch.no_grad():
            loss.backward()
            if self.gradient_clip_val > 0: # To be discussed later
                self.clip_gradients(self.gradient_clip_val, self.model)
            self.optim.step()
        self.train_batch_idx += 1
    if self.val_dataloader is None:
        return
    self.model.eval()
    for batch in self.val_dataloader:
        with torch.no_grad():
            self.model.validation_step(self.prepare_batch(batch))
        self.val_batch_idx += 1

def __init__(self, max_epochs, num_gpus=0, gradient_clip_val=0):
    """Defined in :numref:`sec_use_gpu`"""
    self.save_hyperparameters()
    self.gpus = [native.gpu(i) for i in range(min(num_gpus, native.num_gpus()))]

def prepare_batch(self, batch):
    """Defined in :numref:`sec_use_gpu`"""
    if self.gpus:
        batch = [native.to(a, self.gpus[0]) for a in batch]
    return batch

def prepare_model(self, model):
    """Defined in :numref:`sec_use_gpu`"""
    model.trainer = self
    model.board.xlim = [0, self.max_epochs]
    if self.gpus:
        model.to(self.gpus[0])
    self.model = model

def clip_gradients(self, grad_clip_val, model):
    """Defined in :numref:`sec_rnn-scratch`"""
    params = [p for p in model.parameters() if p.requires_grad]
    norm = torch.sqrt(sum(torch.sum((p.grad ** 2)) for p in params))
    if norm > grad_clip_val:
        for param in params:
            param.grad[:] *= grad_clip_val / norm

class SyntheticRegressionData(native.DataModule):
    """Synthetic data for linear regression.

    Defined in :numref:`sec_synthetic-regression-data`"""
    def __init__(self, w, b, noise=0.01, num_train=1000, num_val=1000,
                 batch_size=32):
        super().__init__()
        self.save_hyperparameters()
        n = num_train + num_val
        self.X = native.randn(n, len(w))
        noise = native.randn(n, 1) * noise
        self.y = native.matmul(self.X, native.reshape(w, (-1, 1))) + b + noise

def get_dataloader(self, train):
    """Defined in :numref:`sec_synthetic-regression-data`"""
    i = slice(0, self.num_train) if train else slice(self.num_train, None)
    return self.get_tensorloader((self.X, self.y), train, i)

class LinearRegressionScratch(native.Module):
    """The linear regression model implemented from scratch.

    Defined in :numref:`sec_linear_scratch`"""
    def __init__(self, num_inputs, lr, sigma=0.01):
        super().__init__()
        self.save_hyperparameters()
        self.w = native.normal(0, sigma, (num_inputs, 1), requires_grad=True)
        self.b = native.zeros(1, requires_grad=True)

```

```

def forward(self, X):
    """Defined in :numref:`sec_linear_scratch`"""
    return native.matmul(X, self.w) + self.b

def loss(self, y_hat, y):
    """Defined in :numref:`sec_linear_scratch`"""
    l = (y_hat - y) ** 2 / 2
    return native.reduce_mean(l)

def configure_optimizers(self):
    """Defined in :numref:`sec_linear_scratch`"""
    return SGD([self.w, self.b], self.lr)

class SGD(native.HyperParameters):
    """Minibatch stochastic gradient descent.

    Defined in :numref:`sec_linear_scratch`"""
    def __init__(self, params, lr):
        self.save_hyperparameters()

    def step(self):
        for param in self.params:
            param -= self.lr * param.grad

    def zero_grad(self):
        for param in self.params:
            if param.grad is not None:
                param.grad.zero_()

class LinearRegression(native.Module):
    """The linear regression model implemented with high-level APIs.

    Defined in :numref:`sec_linear_concise`"""
    def __init__(self, lr):
        super().__init__()
        self.save_hyperparameters()
        self.net = nn.LazyLinear(1)
        self.net.weight.data.normal_(0, 0.01)
        self.net.bias.data.fill_(0)

    def forward(self, X):
        """Defined in :numref:`sec_linear_concise`"""
        return self.net(X)

def loss(self, y_hat, y):
    """Defined in :numref:`sec_linear_concise`"""
    fn = nn.MSELoss()
    return fn(y_hat, y)

def configure_optimizers(self):
    """Defined in :numref:`sec_linear_concise`"""
    return torch.optim.SGD(self.parameters(), self.lr)

def get_w_b(self):
    """Defined in :numref:`sec_linear_concise`"""
    return (self.net.weight.data, self.net.bias.data)

class FashionMNIST(native.DataModule):
    """The Fashion-MNIST dataset.

    Defined in :numref:`sec_fashion_mnist`"""
    def __init__(self, batch_size=64, resize=(28, 28)):
        super().__init__()
        self.save_hyperparameters()
        trans = transforms.Compose([transforms.Resize(resize),
                                     transforms.ToTensor()])
        self.train = torchvision.datasets.FashionMNIST(
            root=self.root, train=True, transform=trans, download=True)
        self.val = torchvision.datasets.FashionMNIST(
            root=self.root, train=False, transform=trans, download=True)

    def text_labels(self, indices):
        """Return text labels.

        Defined in :numref:`sec_fashion_mnist`"""
        labels = ['t-shirt', 'trouser', 'pullover', 'dress', 'coat',

```

```

        'sandal', 'shirt', 'sneaker', 'bag', 'ankle boot']
    return [labels[int(i)] for i in indices]

def get_dataloader(self, train):
    """Defined in :numref:`sec_fashion_mnist`"""
    data = self.train if train else self.val
    return torch.utils.data.DataLoader(data, self.batch_size, shuffle=train,
                                       num_workers=self.num_workers)

def visualize(self, batch, nrow=1, ncol=8, labels=[]):
    """Defined in :numref:`sec_fashion_mnist`"""
    X, y = batch
    if not labels:
        labels = self.text_labels(y)
    native.show_images(X.squeeze(1), nrow, ncol, titles=labels)

def show_images(imgs, num_rows, num_cols, titles=None, scale=1.5):
    """Plot a list of images.

    Defined in :numref:`sec_fashion_mnist`"""
    raise NotImplementedError

class Classifier(native.Module):
    """The base class of classification models.

    Defined in :numref:`sec_classification`"""
    def validation_step(self, batch):
        Y_hat = self(*batch[:-1])

        self.plot('loss', self.loss(Y_hat, batch[-1]), train=False)
        self.plot('acc', self.accuracy(Y_hat, batch[-1]), train=False)

    def accuracy(self, Y_hat, Y, averaged=True):
        """Compute the number of correct predictions.

        Defined in :numref:`sec_classification`"""
        Y_hat = native.reshape(Y_hat, (-1, Y_hat.shape[-1]))
        preds = native.astype(native.argmax(Y_hat, axis=1), Y.dtype)
        compare = native.astype(preds == native.reshape(Y, -1), native.float32)
        return native.reduce_mean(compare) if averaged else compare

    def loss(self, Y_hat, Y, averaged=True):
        """Defined in :numref:`sec_softmax_concise`"""
        Y_hat = native.reshape(Y_hat, (-1, Y_hat.shape[-1]))
        Y = native.reshape(Y, (-1,))
        return F.cross_entropy(
            Y_hat, Y, reduction='mean' if averaged else 'none')

    def layer_summary(self, X_shape):
        """Defined in :numref:`sec_lenet`"""
        X = native.randn(*X_shape)
        for layer in self.net:
            X = layer(X)
            print(layer.__class__.__name__, 'output shape:\t', X.shape)

class SoftmaxRegression(native.Classifier):
    """The softmax regression model.

    Defined in :numref:`sec_softmax_concise`"""
    def __init__(self, num_outputs, lr):
        super().__init__()
        self.save_hyperparameters()
        self.net = nn.Sequential(nn.Flatten(),
                                nn.Linear(num_outputs))

    def forward(self, X):
        return self.net(X)

def cpu():
    """Get the CPU device.

    Defined in :numref:`sec_use_gpu`"""
    return torch.device('cpu')

def gpu(i=0):
    """Get a GPU device.

    Defined in :numref:`sec_use_gpu`"""

```

```

    return torch.device(f'cuda:{i}')

def num_gpus():
    """Get the number of available GPUs.

    Defined in :numref:`sec_use_gpu`"""
    return torch.cuda.device_count()

def try_gpu(i=0):
    """Return gpu(i) if exists, otherwise return cpu().

    Defined in :numref:`sec_use_gpu`"""
    if num_gpus() >= i + 1:
        return gpu(i)
    return cpu()

def try_all_gpus():
    """Return all available GPUs, or [cpu()], if no GPU exists.

    Defined in :numref:`sec_use_gpu`"""
    return [gpu(i) for i in range(num_gpus())]

def corr2d(X, K):
    """Compute 2D cross-correlation.

    Defined in :numref:`sec_conv_layer`"""
    h, w = K.shape
    Y = native.zeros((X.shape[0] - h + 1, X.shape[1] - w + 1))
    for i in range(Y.shape[0]):
        for j in range(Y.shape[1]):
            Y[i, j] = native.reduce_sum((X[i: i + h, j: j + w] * K))
    return Y

def init_cnn(module):
    """Initialize weights for CNNs.

    Defined in :numref:`sec_lenet`"""
    if type(module) == nn.Linear or type(module) == nn.Conv2d:
        nn.init.xavier_uniform_(module.weight)

class LeNet(native.Classifier):
    """The LeNet-5 model.

    Defined in :numref:`sec_lenet`"""
    def __init__(self, lr=0.1, num_classes=10):
        super().__init__()
        self.save_hyperparameters()
        self.net = nn.Sequential(
            nn.LazyConv2d(6, kernel_size=5, padding=2), nn.Sigmoid(),
            nn.AvgPool2d(kernel_size=2, stride=2),
            nn.LazyConv2d(16, kernel_size=5), nn.Sigmoid(),
            nn.AvgPool2d(kernel_size=2, stride=2),
            nn.Flatten(),
            nn.LazyLinear(120), nn.Sigmoid(),
            nn.LazyLinear(84), nn.Sigmoid(),
            nn.LazyLinear(num_classes))

class Residual(nn.Module):
    """The Residual block of ResNet models.

    Defined in :numref:`sec_resnet`"""
    def __init__(self, num_channels, use_1x1conv=False, strides=1):
        super().__init__()
        self.conv1 = nn.LazyConv2d(num_channels, kernel_size=3, padding=1,
                                   stride=strides)
        self.conv2 = nn.LazyConv2d(num_channels, kernel_size=3, padding=1)
        if use_1x1conv:
            self.conv3 = nn.LazyConv2d(num_channels, kernel_size=1,
                                       stride=strides)
        else:
            self.conv3 = None

        self.bn1 = nn.LazyBatchNorm2d()
        self.bn2 = nn.LazyBatchNorm2d()

    def forward(self, X):
        Y = F.relu(self.bn1(self.conv1(X)))

```

```

        Y = self.bn2(self.conv2(Y))
        if self.conv3:
            X = self.conv3(X)
        Y += X
        return F.relu(Y)

class ResNeXtBlock(nn.Module):
    """The ResNeXt block.

    Defined in :numref:`subsec_residual-blks`"""
    def __init__(self, num_channels, groups, bot_mul, use_1x1conv=False,
                 strides=1):
        super().__init__()
        bot_channels = int(round(num_channels * bot_mul))
        self.conv1 = nn.LazyConv2d(bot_channels, kernel_size=1, stride=1)
        self.conv2 = nn.LazyConv2d(bot_channels, kernel_size=3,
                                   stride=strides, padding=1,
                                   groups=bot_channels//groups)
        self.conv3 = nn.LazyConv2d(num_channels, kernel_size=1, stride=1)
        self.bn1 = nn.LazyBatchNorm2d()
        self.bn2 = nn.LazyBatchNorm2d()
        self.bn3 = nn.LazyBatchNorm2d()
        if use_1x1conv:
            self.conv4 = nn.LazyConv2d(num_channels, kernel_size=1,
                                       stride=strides)
            self.bn4 = nn.LazyBatchNorm2d()
        else:
            self.conv4 = None

    def forward(self, X):
        Y = F.relu(self.bn1(self.conv1(X)))
        Y = F.relu(self.bn2(self.conv2(Y)))
        Y = self.bn3(self.conv3(Y))
        if self.conv4:
            X = self.bn4(self.conv4(X))
        return F.relu(Y + X)

class TimeMachine(native.DataModule):
    """The Time Machine dataset.

    Defined in :numref:`sec_text-sequence`"""
    def _download(self):
        fname = native.download(native.DATA_URL + 'timemachine.txt', self.root,
                               '090b5e7e70c295757f55df93cb0a180b9691891a')
        with open(fname) as f:
            return f.read()

    def _preprocess(self, text):
        """Defined in :numref:`sec_text-sequence`"""
        return re.sub('[^A-Za-z]+', ' ', text).lower()

    def _tokenize(self, text):
        """Defined in :numref:`sec_text-sequence`"""
        return list(text)

    def build(self, raw_text, vocab=None):
        """Defined in :numref:`sec_text-sequence`"""
        tokens = self._tokenize(self._preprocess(raw_text))
        if vocab is None: vocab = Vocab(tokens)
        corpus = [vocab[token] for token in tokens]
        return corpus, vocab

    def __init__(self, batch_size, num_steps, num_train=10000, num_val=5000):
        """Defined in :numref:`sec_language-model`"""
        super(native.TimeMachine, self).__init__()
        self.save_hyperparameters()
        corpus, self.vocab = self.build(self._download())
        array = native.tensor([corpus[i:i+num_steps+1]
                               for i in range(len(corpus)-num_steps)])
        self.X, self.Y = array[:, :-1], array[:, 1:]

    def get_dataloader(self, train):
        """Defined in :numref:`subsec_partitioning-seqs`"""
        idx = slice(0, self.num_train) if train else slice(
            self.num_train, self.num_train + self.num_val)
        return self.get_tensorloader([self.X, self.Y], train, idx)

```

```

class Vocab:
    """Vocabulary for text."""
    def __init__(self, tokens=[], min_freq=0, reserved_tokens=[]):
        """Defined in :numref:`sec_text-sequence`"""
        # Flatten a 2D list if needed
        if tokens and isinstance(tokens[0], list):
            tokens = [token for line in tokens for token in line]
        # Count token frequencies
        counter = collections.Counter(tokens)
        self.token_freqs = sorted(counter.items(), key=lambda x: x[1],
                                  reverse=True)

        # The list of unique tokens
        self.idx_to_token = list(sorted(set(['<unk>'] + reserved_tokens + [
            token for token, freq in self.token_freqs if freq >= min_freq])))
        self.token_to_idx = {token: idx
                              for idx, token in enumerate(self.idx_to_token)}

    def __len__(self):
        return len(self.idx_to_token)

    def __getitem__(self, tokens):
        if not isinstance(tokens, (list, tuple)):
            return self.token_to_idx.get(tokens, self.unk)
        return [self.__getitem__(token) for token in tokens]

    def to_tokens(self, indices):
        if hasattr(indices, '__len__') and len(indices) > 1:
            return [self.idx_to_token[int(index)] for index in indices]
        return self.idx_to_token[indices]

    @property
    def unk(self): # Index for the unknown token
        return self.token_to_idx['<unk>']

class RNNScratch(native.Module):
    """The RNN model implemented from scratch.

    Defined in :numref:`sec_rnn-scratch`"""
    def __init__(self, num_inputs, num_hiddens, sigma=0.01):
        super().__init__()
        self.save_hyperparameters()
        self.W_xh = nn.Parameter(
            native.randn(num_inputs, num_hiddens) * sigma)
        self.W_hh = nn.Parameter(
            native.randn(num_hiddens, num_hiddens) * sigma)
        self.b_h = nn.Parameter(native.zeros(num_hiddens))

    def forward(self, inputs, state=None):
        """Defined in :numref:`sec_rnn-scratch`"""
        if state is None:
            # Initial state with shape: (batch_size, num_hiddens)
            state = native.zeros((inputs.shape[1], self.num_hiddens),
                                  device=inputs.device)
        else:
            state, = state
        outputs = []
        for X in inputs: # Shape of inputs: (num_steps, batch_size, num_inputs)
            state = native.tanh(native.matmul(X, self.W_xh) +
                                native.matmul(state, self.W_hh) + self.b_h)
            outputs.append(state)
        return outputs, state

def check_len(a, n):
    """Check the length of a list.

    Defined in :numref:`sec_rnn-scratch`"""
    assert len(a) == n, f'list\'s length {len(a)} != expected length {n}'

def check_shape(a, shape):
    """Check the shape of a tensor.

    Defined in :numref:`sec_rnn-scratch`"""
    assert a.shape == shape, \
        f'tensor\'s shape {a.shape} != expected shape {shape}'

class RNNLMScratch(native.Classifier):
    """The RNN-based language model implemented from scratch.

```

```

Defined in :numref:`sec_rnn-scratch`"""
def __init__(self, rnn, vocab_size, lr=0.01):
    super().__init__()
    self.save_hyperparameters()
    self.init_params()

def init_params(self):
    self.W_hq = nn.Parameter(
        native.randn(
            self.rnn.num_hiddens, self.vocab_size) * self.rnn.sigma)
    self.b_q = nn.Parameter(native.zeros(self.vocab_size))

def training_step(self, batch):
    l = self.loss(self(*batch[:-1]), batch[-1])
    self.plot('ppl', native.exp(l), train=True)
    return l

def validation_step(self, batch):
    l = self.loss(self(*batch[:-1]), batch[-1])
    self.plot('ppl', native.exp(l), train=False)

```

```

def one_hot(self, X):
    """Defined in :numref:`sec_rnn-scratch`"""
    # Output shape: (num_steps, batch_size, vocab_size)
    return F.one_hot(X.T, self.vocab_size).type(torch.float32)

def output_layer(self, rnn_outputs):
    """Defined in :numref:`sec_rnn-scratch`"""
    outputs = [native.matmul(H, self.W_hq) + self.b_q for H in rnn_outputs]
    return native.stack(outputs, 1)

def forward(self, X, state=None):
    """Defined in :numref:`sec_rnn-scratch`"""
    embs = self.one_hot(X)
    rnn_outputs, _ = self.rnn(embs, state)
    return self.output_layer(rnn_outputs)

def predict(self, prefix, num_preds, vocab, device=None):
    """Defined in :numref:`sec_rnn-scratch`"""
    state, outputs = None, [vocab[prefix[0]]]
    for i in range(len(prefix) + num_preds - 1):
        X = native.tensor([outputs[-1]], device=device)
        embs = self.one_hot(X)
        rnn_outputs, state = self.rnn(embs, state)
        if i < len(prefix) - 1: # Warm-up period
            outputs.append(vocab[prefix[i + 1]])
        else: # Predict num_preds steps
            Y = self.output_layer(rnn_outputs)
            outputs.append(int(native.reshape(native.argmax(Y, axis=2), 1)))
    return ''.join([vocab.idx_to_token[i] for i in outputs])

```

```

class RNN(native.Module):
    """The RNN model implemented with high-level APIs.

    Defined in :numref:`sec_rnn-concise`"""
    def __init__(self, num_inputs, num_hiddens):
        super().__init__()
        self.save_hyperparameters()
        self.rnn = nn.RNN(num_inputs, num_hiddens)

    def forward(self, inputs, H=None):
        return self.rnn(inputs, H)

class RNNLM(native.RNNLMScratch):
    """The RNN-based language model implemented with high-level APIs.

    Defined in :numref:`sec_rnn-concise`"""
    def init_params(self):
        self.linear = nn.LazyLinear(self.vocab_size)

    def output_layer(self, hiddens):
        return native.swapaxes(self.linear(hiddens), 0, 1)

class GRU(native.RNN):
    """The multilayer GRU model.

```

```

Defined in :numref:`sec_deep_rnn`"""
def __init__(self, num_inputs, num_hiddens, num_layers, dropout=0):
    native.Module.__init__(self)
    self.save_hyperparameters()
    self.rnn = nn.GRU(num_inputs, num_hiddens, num_layers,
                      dropout=dropout)

class MTFraEng(native.DataModule):
    """The English-French dataset.

    Defined in :numref:`sec_machine_translation`"""
    def _download(self):
        native.extract(native.download(
            native.DATA_URL+'fra-eng.zip', self.root,
            '94646ad1522d915e7b0f9296181140edcf86a4f5'))
        with open(self.root + '/fra-eng/fra.txt', encoding='utf-8') as f:
            return f.read()

    def _preprocess(self, text):
        """Defined in :numref:`sec_machine_translation`"""
        # Replace non-breaking space with space
        text = text.replace('\u202f', ' ').replace('\xa0', ' ')
        # Insert space between words and punctuation marks
        no_space = lambda char, prev_char: char in ',.!? ' and prev_char != ' '
        out = [' ' + char if i > 0 and no_space(char, text[i - 1]) else char
               for i, char in enumerate(text.lower())]
        return ''.join(out)

    def _tokenize(self, text, max_examples=None):
        """Defined in :numref:`sec_machine_translation`"""
        src, tgt = [], []
        for i, line in enumerate(text.split('\n')):
            if max_examples and i > max_examples: break
            parts = line.split('\t')
            if len(parts) == 2:
                # Skip empty tokens
                src.append([t for t in f'{parts[0]} <eos>'.split(' ') if t])
                tgt.append([t for t in f'{parts[1]} <eos>'.split(' ') if t])
        return src, tgt

    def __init__(self, batch_size, num_steps=9, num_train=512, num_val=128):
        """Defined in :numref:`sec_machine_translation`"""
        super(MTFraEng, self).__init__()
        self.save_hyperparameters()
        self.arrays, self.src_vocab, self.tgt_vocab = self._build_arrays(
            self._download())

    def _build_arrays(self, raw_text, src_vocab=None, tgt_vocab=None):
        """Defined in :numref:`subsec_loading-seq-fixed-len`"""
        def _build_array(sentences, vocab, is_tgt=False):
            pad_or_trim = lambda seq, t: (
                seq[:t] if len(seq) > t else seq + ['<pad>'] * (t - len(seq)))
            sentences = [pad_or_trim(s, self.num_steps) for s in sentences]
            if is_tgt:
                sentences = [['<bos>'] + s for s in sentences]
            if vocab is None:
                vocab = native.Vocab(sentences, min_freq=2)
            array = native.tensor([vocab[s] for s in sentences])
            valid_len = native.reduce_sum(
                native.astype(array != vocab['<pad>'], native.int32), 1)
            return array, vocab, valid_len
        src, tgt = self._tokenize(self._preprocess(raw_text),
                                self.num_train + self.num_val)

        src_array, src_vocab, src_valid_len = _build_array(src, src_vocab)
        tgt_array, tgt_vocab, _ = _build_array(tgt, tgt_vocab, True)
        return ((src_array, tgt_array[:,-1], src_valid_len, tgt_array[:,1:]),
                src_vocab, tgt_vocab)

    def get_dataloader(self, train):
        """Defined in :numref:`subsec_loading-seq-fixed-len`"""
        idx = slice(0, self.num_train) if train else slice(self.num_train, None)
        return self.get_tensorloader(self.arrays, train, idx)

    def build(self, src_sentences, tgt_sentences):
        """Defined in :numref:`subsec_loading-seq-fixed-len`"""

```

```

        raw_text = '\n'.join([src + '\t' + tgt for src, tgt in zip(
            src_sentences, tgt_sentences)])
        arrays, _, _ = self._build_arrays(
            raw_text, self.src_vocab, self.tgt_vocab)
        return arrays

def show_list_len_pair_hist(legend, xlabel, ylabel, xlist, ylist):
    """Plot the histogram for list length pairs.

    Defined in :numref:`sec_machine_translation`"""
    native.set_figsize()
    _, _, patches = native.plt.hist(
        [[len(l) for l in xlist], [len(l) for l in ylist]])
    native.plt.xlabel(xlabel)
    native.plt.ylabel(ylabel)
    for patch in patches[1].patches:
        patch.set_hatch('/')
    native.plt.legend(legend)

class Encoder(nn.Module):
    """The base encoder interface for the encoder--decoder architecture.

    Defined in :numref:`sec_encoder-decoder`"""
    def __init__(self):
        super().__init__()

    # Later there can be additional arguments (e.g., length excluding padding)
    def forward(self, X, *args):
        raise NotImplementedError

class Decoder(nn.Module):
    """The base decoder interface for the encoder--decoder architecture.

    Defined in :numref:`sec_encoder-decoder`"""
    def __init__(self):
        super().__init__()

    # Later there can be additional arguments (e.g., length excluding padding)
    def init_state(self, enc_all_outputs, *args):
        raise NotImplementedError

    def forward(self, X, state):
        raise NotImplementedError

class EncoderDecoder(native.Classifier):
    """The base class for the encoder--decoder architecture.

    Defined in :numref:`sec_encoder-decoder`"""
    def __init__(self, encoder, decoder):
        super().__init__()
        self.encoder = encoder
        self.decoder = decoder

    def forward(self, enc_X, dec_X, *args):
        enc_all_outputs = self.encoder(enc_X, *args)
        dec_state = self.decoder.init_state(enc_all_outputs, *args)
        # Return decoder output only
        return self.decoder(dec_X, dec_state)[0]

    def predict_step(self, batch, device, num_steps,
                     save_attention_weights=False):
        """Defined in :numref:`sec_seq2seq_training`"""
        batch = [native.to(a, device) for a in batch]
        src, tgt, src_valid_len, _ = batch
        enc_all_outputs = self.encoder(src, src_valid_len)
        dec_state = self.decoder.init_state(enc_all_outputs, src_valid_len)
        outputs, attention_weights = [native.expand_dims(tgt[:, 0], 1), ], []
        for _ in range(num_steps):
            Y, dec_state = self.decoder(outputs[-1], dec_state)
            outputs.append(native.argmax(Y, 2))
            # Save attention weights (to be covered later)
            if save_attention_weights:
                attention_weights.append(self.decoder.attention_weights)
        return native.concat(outputs[1:], 1), attention_weights

def init_seq2seq(module):
    """Initialize weights for sequence-to-sequence learning.

```

```

Defined in :numref:`sec_seq2seq`"""
if type(module) == nn.Linear:
    nn.init.xavier_uniform_(module.weight)
if type(module) == nn.GRU:
    for param in module._flat_weights_names:
        if "weight" in param:
            nn.init.xavier_uniform_(module._parameters[param])

class Seq2SeqEncoder(native.Encoder):
    """The RNN encoder for sequence-to-sequence learning.

    Defined in :numref:`sec_seq2seq`"""
    def __init__(self, vocab_size, embed_size, num_hiddens, num_layers,
                 dropout=0):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_size)
        self.rnn = native.GRU(embed_size, num_hiddens, num_layers, dropout)
        self.apply(init_seq2seq)

    def forward(self, X, *args):
        # X shape: (batch_size, num_steps)
        embs = self.embedding(native.astype(native.transpose(X), native.int64))
        # embs shape: (num_steps, batch_size, embed_size)
        outputs, state = self.rnn(embs)
        # outputs shape: (num_steps, batch_size, num_hiddens)
        # state shape: (num_layers, batch_size, num_hiddens)
        return outputs, state

class Seq2Seq(native.EncoderDecoder):
    """The RNN encoder--decoder for sequence to sequence learning.

    Defined in :numref:`sec_seq2seq_decoder`"""
    def __init__(self, encoder, decoder, tgt_pad, lr):
        super().__init__(encoder, decoder)
        self.save_hyperparameters()

    def validation_step(self, batch):
        Y_hat = self(*batch[:-1])
        self.plot('loss', self.loss(Y_hat, batch[-1]), train=False)

    def configure_optimizers(self):
        # Adam optimizer is used here
        return torch.optim.Adam(self.parameters(), lr=self.lr)

def bleu(pred_seq, label_seq, k):
    """Compute the BLEU.

    Defined in :numref:`sec_seq2seq_training`"""
    pred_tokens, label_tokens = pred_seq.split(' '), label_seq.split(' ')
    len_pred, len_label = len(pred_tokens), len(label_tokens)
    score = math.exp(min(0, 1 - len_label / len_pred))
    for n in range(1, min(k, len_pred) + 1):
        num_matches, label_subs = 0, collections.defaultdict(int)
        for i in range(len_label - n + 1):
            label_subs[' '.join(label_tokens[i: i + n])] += 1
        for i in range(len_pred - n + 1):
            if label_subs[' '.join(pred_tokens[i: i + n])] > 0:
                num_matches += 1
                label_subs[' '.join(pred_tokens[i: i + n])] -= 1
        score *= math.pow(num_matches / (len_pred - n + 1), math.pow(0.5, n))
    return score

def show_heatmaps(matrices, xlabel, ylabel, titles=None, figsize=(2.5, 2.5),
                  cmap='Reds'):
    """Show heatmaps of matrices.

    Defined in :numref:`sec_queries-keys-values`"""
    native.use_svg_display()
    num_rows, num_cols, _, _ = matrices.shape
    fig, axes = native.plt.subplots(num_rows, num_cols, figsize=figsize,
                                   sharex=True, sharey=True, squeeze=False)
    for i, (row_axes, row_matrices) in enumerate(zip(axes, matrices)):
        for j, (ax, matrix) in enumerate(zip(row_axes, row_matrices)):
            pcm = ax.imshow(native.numpy(matrix), cmap=cmap)
            if i == num_rows - 1:
                ax.set_xlabel(xlabel)

```

```

        if j == 0:
            ax.set_ylabel(ylabel)
        if titles:
            ax.set_title(titles[j])
    fig.colorbar(pcm, ax=axes, shrink=0.6);

def masked_softmax(X, valid_lens):
    """Perform softmax operation by masking elements on the last axis.

    Defined in :numref:`sec_attention-scoring-functions`"""
    # X: 3D tensor, valid_lens: 1D or 2D tensor
    def _sequence_mask(X, valid_len, value=0):
        maxlen = X.size(1)
        mask = torch.arange((maxlen), dtype=torch.float32,
                             device=X.device)[None, :] < valid_len[:, None]
        X[~mask] = value
        return X

    if valid_lens is None:
        return nn.functional.softmax(X, dim=-1)
    else:
        shape = X.shape
        if valid_lens.dim() == 1:
            valid_lens = torch.repeat_interleave(valid_lens, shape[1])
        else:
            valid_lens = valid_lens.reshape(-1)
        # On the last axis, replace masked elements with a very large negative
        # value, whose exponentiation outputs 0
        X = _sequence_mask(X.reshape(-1, shape[-1]), valid_lens, value=-1e6)
        return nn.functional.softmax(X.reshape(shape), dim=-1)

class DotProductAttention(nn.Module):
    """Scaled dot product attention.

    Defined in :numref:`subsec_batch_dot`"""
    def __init__(self, dropout):
        super().__init__()
        self.dropout = nn.Dropout(dropout)

    # Shape of queries: (batch_size, no. of queries, d)
    # Shape of keys: (batch_size, no. of key-value pairs, d)
    # Shape of values: (batch_size, no. of key-value pairs, value dimension)
    # Shape of valid_lens: (batch_size,) or (batch_size, no. of queries)
    def forward(self, queries, keys, values, valid_lens=None):
        d = queries.shape[-1]
        # Swap the last two dimensions of keys with keys.transpose(1, 2)
        scores = torch.bmm(queries, keys.transpose(1, 2)) / math.sqrt(d)
        self.attention_weights = masked_softmax(scores, valid_lens)
        return torch.bmm(self.dropout(self.attention_weights), values)

class AdditiveAttention(nn.Module):
    """Additive attention.

    Defined in :numref:`subsec_batch_dot`"""
    def __init__(self, num_hiddens, dropout, **kwargs):
        super(AdditiveAttention, self).__init__(**kwargs)
        self.W_k = nn.LazyLinear(num_hiddens, bias=False)
        self.W_q = nn.LazyLinear(num_hiddens, bias=False)
        self.w_v = nn.LazyLinear(1, bias=False)
        self.dropout = nn.Dropout(dropout)

    def forward(self, queries, keys, values, valid_lens):
        queries, keys = self.W_q(queries), self.W_k(keys)
        # After dimension expansion, shape of queries: (batch_size, no. of
        # queries, 1, num_hiddens) and shape of keys: (batch_size, 1, no. of
        # key-value pairs, num_hiddens). Sum them up with broadcasting
        features = queries.unsqueeze(2) + keys.unsqueeze(1)
        features = torch.tanh(features)
        # There is only one output of self.w_v, so we remove the last
        # one-dimensional entry from the shape. Shape of scores: (batch_size,
        # no. of queries, no. of key-value pairs)
        scores = self.w_v(features).squeeze(-1)
        self.attention_weights = masked_softmax(scores, valid_lens)
        # Shape of values: (batch_size, no. of key-value pairs, value
        # dimension)
        return torch.bmm(self.dropout(self.attention_weights), values)

```

```

class AttentionDecoder(native.Decoder):
    """The base attention-based decoder interface.

    Defined in :numref:`sec_seq2seq_attention`"""
    def __init__(self):
        super().__init__()

    @property
    def attention_weights(self):
        raise NotImplementedError

class MultiHeadAttention(native.Module):
    """Multi-head attention.

    Defined in :numref:`sec_multihead-attention`"""
    def __init__(self, num_hiddens, num_heads, dropout, bias=False, **kwargs):
        super().__init__()
        self.num_heads = num_heads
        self.attention = native.DotProductAttention(dropout)
        self.W_q = nn.LazyLinear(num_hiddens, bias=bias)
        self.W_k = nn.LazyLinear(num_hiddens, bias=bias)
        self.W_v = nn.LazyLinear(num_hiddens, bias=bias)
        self.W_o = nn.LazyLinear(num_hiddens, bias=bias)

    def forward(self, queries, keys, values, valid_lens):
        # Shape of queries, keys, or values:
        # (batch_size, no. of queries or key-value pairs, num_hiddens)
        # Shape of valid_lens: (batch_size,) or (batch_size, no. of queries)
        # After transposing, shape of output queries, keys, or values:
        # (batch_size * num_heads, no. of queries or key-value pairs,
        # num_hiddens / num_heads)
        queries = self.transpose_qkv(self.W_q(queries))
        keys = self.transpose_qkv(self.W_k(keys))
        values = self.transpose_qkv(self.W_v(values))

        if valid_lens is not None:
            # On axis 0, copy the first item (scalar or vector) for num_heads
            # times, then copy the next item, and so on
            valid_lens = torch.repeat_interleave(
                valid_lens, repeats=self.num_heads, dim=0)

        # Shape of output: (batch_size * num_heads, no. of queries,
        # num_hiddens / num_heads)
        output = self.attention(queries, keys, values, valid_lens)
        # Shape of output_concat: (batch_size, no. of queries, num_hiddens)
        output_concat = self.transpose_output(output)
        return self.W_o(output_concat)

    def transpose_qkv(self, X):
        """Transposition for parallel computation of multiple attention heads.

        Defined in :numref:`sec_multihead-attention`"""
        # Shape of input X: (batch_size, no. of queries or key-value pairs,
        # num_hiddens). Shape of output X: (batch_size, no. of queries or
        # key-value pairs, num_heads, num_hiddens / num_heads)
        X = X.reshape(X.shape[0], X.shape[1], self.num_heads, -1)
        # Shape of output X: (batch_size, num_heads, no. of queries or key-value
        # pairs, num_hiddens / num_heads)
        X = X.permute(0, 2, 1, 3)
        # Shape of output: (batch_size * num_heads, no. of queries or key-value
        # pairs, num_hiddens / num_heads)
        return X.reshape(-1, X.shape[2], X.shape[3])

    def transpose_output(self, X):
        """Reverse the operation of transpose_qkv.

        Defined in :numref:`sec_multihead-attention`"""
        X = X.reshape(-1, self.num_heads, X.shape[1], X.shape[2])
        X = X.permute(0, 2, 1, 3)
        return X.reshape(X.shape[0], X.shape[1], -1)

class PositionalEncoding(nn.Module):
    """Positional encoding.

    Defined in :numref:`sec_self-attention-and-positional-encoding`"""

```

```

def __init__(self, num_hiddens, dropout, max_len=1000):
    super().__init__()
    self.dropout = nn.Dropout(dropout)
    # Create a long enough P
    self.P = native.zeros((1, max_len, num_hiddens))
    X = native.arange(max_len, dtype=torch.float32).reshape(
        -1, 1) / torch.pow(10000, torch.arange(
            0, num_hiddens, 2, dtype=torch.float32) / num_hiddens)
    self.P[:, :, 0::2] = torch.sin(X)
    self.P[:, :, 1::2] = torch.cos(X)

def forward(self, X):
    X = X + self.P[:, :X.shape[1], :].to(X.device)
    return self.dropout(X)

class PositionWiseFFN(nn.Module):
    """The positionwise feed-forward network.

    Defined in :numref:`sec_transformer`"""
    def __init__(self, ffn_num_hiddens, ffn_num_outputs):
        super().__init__()
        self.dense1 = nn.LazyLinear(ffn_num_hiddens)
        self.relu = nn.ReLU()
        self.dense2 = nn.LazyLinear(ffn_num_outputs)

    def forward(self, X):
        return self.dense2(self.relu(self.dense1(X)))

class AddNorm(nn.Module):
    """The residual connection followed by layer normalization.

    Defined in :numref:`subsec_positionwise-ffn`"""
    def __init__(self, norm_shape, dropout):
        super().__init__()
        self.dropout = nn.Dropout(dropout)
        self.ln = nn.LayerNorm(norm_shape)

    def forward(self, X, Y):
        return self.ln(self.dropout(Y) + X)

class TransformerEncoderBlock(nn.Module):
    """The Transformer encoder block.

    Defined in :numref:`subsec_positionwise-ffn`"""
    def __init__(self, num_hiddens, ffn_num_hiddens, num_heads, dropout,
        use_bias=False):
        super().__init__()
        self.attention = native.MultiHeadAttention(num_hiddens, num_heads,
            dropout, use_bias)
        self.addnorm1 = AddNorm(num_hiddens, dropout)
        self.ffn = PositionWiseFFN(ffn_num_hiddens, num_hiddens)
        self.addnorm2 = AddNorm(num_hiddens, dropout)

    def forward(self, X, valid_lens):
        Y = self.addnorm1(X, self.attention(X, X, X, valid_lens))
        return self.addnorm2(Y, self.ffn(Y))

class TransformerEncoder(native.Encoder):
    """The Transformer encoder.

    Defined in :numref:`subsec_transformer-encoder`"""
    def __init__(self, vocab_size, num_hiddens, ffn_num_hiddens,
        num_heads, num_blks, dropout, use_bias=False):
        super().__init__()
        self.num_hiddens = num_hiddens
        self.embedding = nn.Embedding(vocab_size, num_hiddens)
        self.pos_encoding = native.PositionalEncoding(num_hiddens, dropout)
        self.blks = nn.Sequential()
        for i in range(num_blks):
            self.blks.add_module("block"+str(i), TransformerEncoderBlock(
                num_hiddens, ffn_num_hiddens, num_heads, dropout, use_bias))

    def forward(self, X, valid_lens):
        # Since positional encoding values are between -1 and 1, the embedding
        # values are multiplied by the square root of the embedding dimension
        # to rescale before they are summed up
        X = self.pos_encoding(self.embedding(X) * math.sqrt(self.num_hiddens))

```

```

        self.attention_weights = [None] * len(self.blks)
        for i, blk in enumerate(self.blks):
            X = blk(X, valid_lens)
            self.attention_weights[
                i] = blk.attention.attention.attention_weights
        return X

def annotate(text, xy, xytext):
    """Defined in :numref:`sec_optimization-intro`"""
    native.plt.gca().annotate(text, xy=xy, xytext=xytext,
                              arrowprops=dict(arrowstyle='->'))

def train_2d(trainer, steps=20, f_grad=None):
    """Optimize a 2D objective function with a customized trainer.

    Defined in :numref:`subsec_gd-learningrate`"""
    # `s1` and `s2` are internal state variables that will be used in Momentum, adagrad, RMSProp
    x1, x2, s1, s2 = -5, -2, 0, 0
    results = [(x1, x2)]
    for i in range(steps):
        if f_grad:
            x1, x2, s1, s2 = trainer(x1, x2, s1, s2, f_grad)
        else:
            x1, x2, s1, s2 = trainer(x1, x2, s1, s2)
        results.append((x1, x2))
    print(f'epoch {i + 1}, x1: {float(x1):f}, x2: {float(x2):f}')
    return results

def show_trace_2d(f, results):
    """Show the trace of 2D variables during optimization.

    Defined in :numref:`subsec_gd-learningrate`"""
    native.set_figsize()
    native.plt.plot(*zip(*results), '-o', color='#ff7f0e')
    x1, x2 = native.meshgrid(native.arange(-5.5, 1.0, 0.1),
                              native.arange(-3.0, 1.0, 0.1), indexing='ij')
    native.plt.contour(x1, x2, f(x1, x2), colors='#1f77b4')
    native.plt.xlabel('x1')
    native.plt.ylabel('x2')

class Timer:
    """Record multiple running times."""
    def __init__(self):
        """Defined in :numref:`sec_minibatch_sgd`"""
        self.times = []
        self.start()

    def start(self):
        """Start the timer."""
        self.tik = time.time()

    def stop(self):
        """Stop the timer and record the time in a list."""
        self.times.append(time.time() - self.tik)
        return self.times[-1]

    def avg(self):
        """Return the average time."""
        return sum(self.times) / len(self.times)

    def sum(self):
        """Return the sum of time."""
        return sum(self.times)

    def cumsum(self):
        """Return the accumulated time."""
        return np.array(self.times).cumsum().tolist()

native.DATA_HUB['airfoil'] = (native.DATA_URL + 'airfoil_self_noise.dat',
                              '76e5be1548fd8222e5074cf0faae75edff8cf93f')

def get_data_ch11(batch_size=10, n=1500):
    """Defined in :numref:`sec_minibatches`"""
    data = np.genfromtxt(native.download('airfoil'),
                          dtype=np.float32, delimiter='\\t')
    data = torch.from_numpy((data - data.mean(axis=0)) / data.std(axis=0))
    data_iter = native.load_array((data[:n, :-1], data[:n, -1]),

```

```

        batch_size, is_train=True)
    return data_iter, data.shape[1]-1

def train_ch11(trainer_fn, states, hyperparams, data_iter,
               feature_dim, num_epochs=2):
    """Defined in :numref:`sec_minibatches`"""
    # Initialization
    w = torch.normal(mean=0.0, std=0.01, size=(feature_dim, 1),
                     requires_grad=True)
    b = torch.zeros((1), requires_grad=True)
    net, loss = lambda X: native.linreg(X, w, b), native.squared_loss
    # Train
    animator = native.Animator(xlabel='epoch', ylabel='loss',
                              xlim=[0, num_epochs], ylim=[0.22, 0.35])
    n, timer = 0, native.Timer()
    for _ in range(num_epochs):
        for X, y in data_iter:
            l = loss(net(X), y).mean()
            l.backward()
            trainer_fn([w, b], states, hyperparams)
            n += X.shape[0]
            if n % 200 == 0:
                timer.stop()
                animator.add(n/X.shape[0]/len(data_iter),
                             (native.evaluate_loss(net, data_iter, loss),))
                timer.start()
    print(f'loss: {animator.Y[0][-1]:.3f}, {timer.sum()/num_epochs:.3f} sec/epoch')
    return timer.cumsum(), animator.Y[0]

def train_concise_ch11(trainer_fn, hyperparams, data_iter, num_epochs=4):
    """Defined in :numref:`sec_minibatches`"""
    # Initialization
    net = nn.Sequential(nn.Linear(5, 1))
    def init_weights(module):
        if type(module) == nn.Linear:
            torch.nn.init.normal_(module.weight, std=0.01)
    net.apply(init_weights)

    optimizer = trainer_fn(net.parameters(), **hyperparams)
    loss = nn.MSELoss(reduction='none')
    animator = native.Animator(xlabel='epoch', ylabel='loss',
                              xlim=[0, num_epochs], ylim=[0.22, 0.35])
    n, timer = 0, native.Timer()
    for _ in range(num_epochs):
        for X, y in data_iter:
            optimizer.zero_grad()
            out = net(X)
            y = y.reshape(out.shape)
            l = loss(out, y)
            l.mean().backward()
            optimizer.step()
            n += X.shape[0]
            if n % 200 == 0:
                timer.stop()
                # `MSELoss` computes squared error without the 1/2 factor
                animator.add(n/X.shape[0]/len(data_iter),
                             (native.evaluate_loss(net, data_iter, loss) / 2,))
                timer.start()
    print(f'loss: {animator.Y[0][-1]:.3f}, {timer.sum()/num_epochs:.3f} sec/epoch')

class Benchmark:
    """For measuring running time."""
    def __init__(self, description='Done'):
        """Defined in :numref:`sec_hybridize`"""
        self.description = description

    def __enter__(self):
        self.timer = native.Timer()
        return self

    def __exit__(self, *args):
        print(f'{self.description}: {self.timer.stop():.4f} sec')

def split_batch(X, y, devices):
    """Split `X` and `y` into multiple devices.

```

```

Defined in :numref:`sec_multi_gpu`"""
assert X.shape[0] == y.shape[0]
return (nn.parallel.scatter(X, devices),
        nn.parallel.scatter(y, devices))

def resnet18(num_classes, in_channels=1):
    """A slightly modified ResNet-18 model.

    Defined in :numref:`sec_multi_gpu_concise`"""
    def resnet_block(in_channels, out_channels, num_residuals,
                     first_block=False):
        blk = []
        for i in range(num_residuals):
            if i == 0 and not first_block:
                blk.append(native.Residual(out_channels, use_1x1conv=True,
                                           strides=2))
            else:
                blk.append(native.Residual(out_channels))
        return nn.Sequential(*blk)

    # This model uses a smaller convolution kernel, stride, and padding and
    # removes the max-pooling layer
    net = nn.Sequential(
        nn.Conv2d(in_channels, 64, kernel_size=3, stride=1, padding=1),
        nn.BatchNorm2d(64),
        nn.ReLU())
    net.add_module("resnet_block1", resnet_block(64, 64, 2, first_block=True))
    net.add_module("resnet_block2", resnet_block(64, 128, 2))
    net.add_module("resnet_block3", resnet_block(128, 256, 2))
    net.add_module("resnet_block4", resnet_block(256, 512, 2))
    net.add_module("global_avg_pool", nn.AdaptiveAvgPool2d((1,1)))
    net.add_module("fc", nn.Sequential(nn.Flatten(),
                                       nn.Linear(512, num_classes)))

    return net

def train_batch_ch13(net, X, y, loss, trainer, devices):
    """Train for a minibatch with multiple GPUs (defined in Chapter 13).

    Defined in :numref:`sec_image_augmentation`"""
    if isinstance(X, list):
        # Required for BERT fine-tuning (to be covered later)
        X = [x.to(devices[0]) for x in X]
    else:
        X = X.to(devices[0])
    y = y.to(devices[0])
    net.train()
    trainer.zero_grad()
    pred = net(X)
    l = loss(pred, y)
    l.sum().backward()
    trainer.step()

    train_loss_sum = l.sum()
    train_acc_sum = native.accuracy(pred, y)
    return train_loss_sum, train_acc_sum

def train_ch13(net, train_iter, test_iter, loss, trainer, num_epochs,
               devices=native.try_all_gpus()):
    """Train a model with multiple GPUs (defined in Chapter 13).

    Defined in :numref:`sec_image_augmentation`"""
    timer, num_batches = native.Timer(), len(train_iter)
    animator = native.Animator(xlabel='epoch', xlim=[1, num_epochs], ylim=[0, 1],
                              legend=['train loss', 'train acc', 'test acc'])
    net = nn.DataParallel(net, device_ids=devices).to(devices[0])
    for epoch in range(num_epochs):
        # Sum of training loss, sum of training accuracy, no. of examples,
        # no. of predictions
        metric = native.Accumulator(4)
        for i, (features, labels) in enumerate(train_iter):
            timer.start()
            l, acc = train_batch_ch13(
                net, features, labels, loss, trainer, devices)
            metric.add(l, acc, labels.shape[0], labels.numel())
            timer.stop()
            if (i + 1) % (num_batches // 5) == 0 or i == num_batches - 1:
                animator.add(epoch + (i + 1) / num_batches,
                             (metric[0] / metric[2], metric[1] / metric[3],

```

```

        None))
    test_acc = native.evaluate_accuracy_gpu(net, test_iter)
    animator.add(epoch + 1, (None, None, test_acc))
    print(f'loss {metric[0] / metric[2]:.3f}, train acc '
          f'{metric[1] / metric[3]:.3f}, test acc {test_acc:.3f}')
    print(f'{metric[2] * num_epochs / timer.sum():.1f} examples/sec on '
          f'{str(devices)}')

native.DATA_HUB['hotdog'] = (native.DATA_URL + 'hotdog.zip',
                             'fba480ffa8aa7e0febbb511d181409f899b9baa5')

def box_corner_to_center(bboxes):
    """Convert from (upper-left, lower-right) to (center, width, height).

    Defined in :numref:`sec_bbox`"""
    x1, y1, x2, y2 = bboxes[:, 0], bboxes[:, 1], bboxes[:, 2], bboxes[:, 3]
    cx = (x1 + x2) / 2
    cy = (y1 + y2) / 2
    w = x2 - x1
    h = y2 - y1
    bboxes = native.stack((cx, cy, w, h), axis=-1)
    return bboxes

def box_center_to_corner(bboxes):
    """Convert from (center, width, height) to (upper-left, lower-right).

    Defined in :numref:`sec_bbox`"""
    cx, cy, w, h = bboxes[:, 0], bboxes[:, 1], bboxes[:, 2], bboxes[:, 3]
    x1 = cx - 0.5 * w
    y1 = cy - 0.5 * h
    x2 = cx + 0.5 * w
    y2 = cy + 0.5 * h
    bboxes = native.stack((x1, y1, x2, y2), axis=-1)
    return bboxes

def bbox_to_rect(bbox, color):
    """Convert bounding box to matplotlib format.

    Defined in :numref:`sec_bbox`"""
    # Convert the bounding box (upper-left x, upper-left y, lower-right x,
    # lower-right y) format to the matplotlib format: ((upper-left x,
    # upper-left y), width, height)
    return native.plt.Rectangle(
        xy=(bbox[0], bbox[1]), width=bbox[2]-bbox[0], height=bbox[3]-bbox[1],
        fill=False, edgecolor=color, linewidth=2)

def multibox_prior(data, sizes, ratios):
    """Generate anchor boxes with different shapes centered on each pixel.

    Defined in :numref:`sec_anchor`"""
    in_height, in_width = data.shape[-2:]
    device, num_sizes, num_ratios = data.device, len(sizes), len(ratios)
    boxes_per_pixel = (num_sizes + num_ratios - 1)
    size_tensor = native.tensor(sizes, device=device)
    ratio_tensor = native.tensor(ratios, device=device)
    # Offsets are required to move the anchor to the center of a pixel. Since
    # a pixel has height=1 and width=1, we choose to offset our centers by 0.5
    offset_h, offset_w = 0.5, 0.5
    steps_h = 1.0 / in_height # Scaled steps in y axis
    steps_w = 1.0 / in_width # Scaled steps in x axis

    # Generate all center points for the anchor boxes
    center_h = (torch.arange(in_height, device=device) + offset_h) * steps_h
    center_w = (torch.arange(in_width, device=device) + offset_w) * steps_w
    shift_y, shift_x = torch.meshgrid(center_h, center_w, indexing='ij')
    shift_y, shift_x = shift_y.reshape(-1), shift_x.reshape(-1)

    # Generate `boxes_per_pixel` number of heights and widths that are later
    # used to create anchor box corner coordinates (xmin, xmax, ymin, ymax)
    w = torch.cat((size_tensor * torch.sqrt(ratio_tensor[0]),
                   sizes[0] * torch.sqrt(ratio_tensor[1]))\
                  * in_height / in_width # Handle rectangular inputs
    h = torch.cat((size_tensor / torch.sqrt(ratio_tensor[0]),
                   sizes[0] / torch.sqrt(ratio_tensor[1]))
    # Divide by 2 to get half height and half width
    anchor_manipulations = torch.stack((-w, -h, w, h)).T.repeat(
        in_height * in_width, 1) / 2

```

```

# Each center point will have `boxes_per_pixel` number of anchor boxes, so
# generate a grid of all anchor box centers with `boxes_per_pixel` repeats
out_grid = torch.stack([shift_x, shift_y, shift_x, shift_y],
                        dim=1).repeat_interleave(boxes_per_pixel, dim=0)
output = out_grid + anchor_manipulations
return output.unsqueeze(0)

def show_bboxes(axes, bboxes, labels=None, colors=None):
    """Show bounding boxes.

    Defined in :numref:`sec_anchor`"""

    def make_list(obj, default_values=None):
        if obj is None:
            obj = default_values
        elif not isinstance(obj, (list, tuple)):
            obj = [obj]
        return obj

    labels = make_list(labels)
    colors = make_list(colors, ['b', 'g', 'r', 'm', 'c'])
    for i, bbox in enumerate(bboxes):
        color = colors[i % len(colors)]
        rect = native.bbox_to_rect(native.numpy(bbox), color)
        axes.add_patch(rect)
        if labels and len(labels) > i:
            text_color = 'k' if color == 'w' else 'w'
            axes.text(rect.xy[0], rect.xy[1], labels[i],
                      va='center', ha='center', fontsize=9, color=text_color,
                      bbox=dict(facecolor=color, lw=0))

def box_iou(boxes1, boxes2):
    """Compute pairwise IoU across two lists of anchor or bounding boxes.

    Defined in :numref:`sec_anchor`"""
    box_area = lambda boxes: ((boxes[:, 2] - boxes[:, 0]) *
                               (boxes[:, 3] - boxes[:, 1]))
    # Shape of `boxes1`, `boxes2`, `areas1`, `areas2`: (no. of boxes1, 4),
    # (no. of boxes2, 4), (no. of boxes1,), (no. of boxes2,)
    areas1 = box_area(boxes1)
    areas2 = box_area(boxes2)
    # Shape of `inter_upperlefts`, `inter_lowerrights`, `inters`: (no. of
    # boxes1, no. of boxes2, 2)
    inter_upperlefts = torch.max(boxes1[:, None, :2], boxes2[:, :2])
    inter_lowerrights = torch.min(boxes1[:, None, 2:], boxes2[:, 2:])
    inters = (inter_lowerrights - inter_upperlefts).clamp(min=0)
    # Shape of `inter_areas` and `union_areas`: (no. of boxes1, no. of boxes2)
    inter_areas = inters[:, :, 0] * inters[:, :, 1]
    union_areas = areas1[:, None] + areas2 - inter_areas
    return inter_areas / union_areas

def assign_anchor_to_bbox(ground_truth, anchors, device, iou_threshold=0.5):
    """Assign closest ground-truth bounding boxes to anchor boxes.

    Defined in :numref:`sec_anchor`"""
    num_anchors, num_gt_boxes = anchors.shape[0], ground_truth.shape[0]
    # Element x_ij in the i-th row and j-th column is the IoU of the anchor
    # box i and the ground-truth bounding box j
    jaccard = box_iou(anchors, ground_truth)
    # Initialize the tensor to hold the assigned ground-truth bounding box for
    # each anchor
    anchors_bbox_map = torch.full((num_anchors,), -1, dtype=torch.long,
                                  device=device)
    # Assign ground-truth bounding boxes according to the threshold
    max_iou, indices = torch.max(jaccard, dim=1)
    anc_i = torch.nonzero(max_iou >= iou_threshold).reshape(-1)
    box_j = indices[max_iou >= iou_threshold]
    anchors_bbox_map[anc_i] = box_j
    col_discard = torch.full((num_anchors,), -1)
    row_discard = torch.full((num_gt_boxes,), -1)
    for _ in range(num_gt_boxes):
        max_idx = torch.argmax(jaccard) # Find the largest IoU
        box_idx = (max_idx % num_gt_boxes).long()
        anc_idx = (max_idx / num_gt_boxes).long()
        anchors_bbox_map[anc_idx] = box_idx
        jaccard[:, box_idx] = col_discard

```

```

        jaccard[anc_idx, :] = row_discard
    return anchors_bbox_map

def offset_boxes(anchors, assigned_bb, eps=1e-6):
    """Transform for anchor box offsets.

    Defined in :numref:`subsec_labeling-anchor-boxes`"""
    c_anc = native.box_corner_to_center(anchors)
    c_assigned_bb = native.box_corner_to_center(assigned_bb)
    offset_xy = 10 * (c_assigned_bb[:, :2] - c_anc[:, :2]) / c_anc[:, :2]
    offset_wh = 5 * native.log(eps + c_assigned_bb[:, 2:] / c_anc[:, 2:])
    offset = native.concat([offset_xy, offset_wh], axis=1)
    return offset

def multibox_target(anchors, labels):
    """Label anchor boxes using ground-truth bounding boxes.

    Defined in :numref:`subsec_labeling-anchor-boxes`"""
    batch_size, anchors = labels.shape[0], anchors.squeeze(0)
    batch_offset, batch_mask, batch_class_labels = [], [], []
    device, num_anchors = anchors.device, anchors.shape[0]
    for i in range(batch_size):
        label = labels[i, :, :]
        anchors_bbox_map = assign_anchor_to_bbox(
            label[:, 1:], anchors, device)
        bbox_mask = ((anchors_bbox_map >= 0).float().unsqueeze(-1)).repeat(
            1, 4)
        # Initialize class labels and assigned bounding box coordinates with
        # zeros
        class_labels = torch.zeros(num_anchors, dtype=torch.long,
                                   device=device)
        assigned_bb = torch.zeros((num_anchors, 4), dtype=torch.float32,
                                  device=device)
        # Label classes of anchor boxes using their assigned ground-truth
        # bounding boxes. If an anchor box is not assigned any, we label its
        # class as background (the value remains zero)
        indices_true = torch.nonzero(anchors_bbox_map >= 0)
        bb_idx = anchors_bbox_map[indices_true]
        class_labels[indices_true] = label[bb_idx, 0].long() + 1
        assigned_bb[indices_true] = label[bb_idx, 1:]
        # Offset transformation
        offset = offset_boxes(anchors, assigned_bb) * bbox_mask
        batch_offset.append(offset.reshape(-1))
        batch_mask.append(bbox_mask.reshape(-1))
        batch_class_labels.append(class_labels)
    bbox_offset = torch.stack(batch_offset)
    bbox_mask = torch.stack(batch_mask)
    class_labels = torch.stack(batch_class_labels)
    return (bbox_offset, bbox_mask, class_labels)

def offset_inverse(anchors, offset_preds):
    """Predict bounding boxes based on anchor boxes with predicted offsets.

    Defined in :numref:`subsec_labeling-anchor-boxes`"""
    anc = native.box_corner_to_center(anchors)
    pred_bbox_xy = (offset_preds[:, :2] * anc[:, :2] / 10) + anc[:, :2]
    pred_bbox_wh = native.exp(offset_preds[:, 2:] / 5) * anc[:, 2:]
    pred_bbox = native.concat((pred_bbox_xy, pred_bbox_wh), axis=1)
    predicted_bbox = native.box_center_to_corner(pred_bbox)
    return predicted_bbox

def nms(bboxes, scores, iou_threshold):
    """Sort confidence scores of predicted bounding boxes.

    Defined in :numref:`subsec_predicting-bounding-boxes-nms`"""
    B = torch.argsort(scores, dim=-1, descending=True)
    keep = [] # Indices of predicted bounding boxes that will be kept
    while B.numel() > 0:
        i = B[0]
        keep.append(i)
        if B.numel() == 1: break
        iou = box_iou(bboxes[i, :].reshape(-1, 4),
                      bboxes[B[1:], :].reshape(-1, 4)).reshape(-1)
        inds = torch.nonzero(iou <= iou_threshold).reshape(-1)
        B = B[inds + 1]
    return native.tensor(keep, device=bboxes.device)

```

```

def multibox_detection(cls_probs, offset_preds, anchors, nms_threshold=0.5,
                      pos_threshold=0.009999999):
    """Predict bounding boxes using non-maximum suppression.

    Defined in :numref:`subsec_predicting-bounding-boxes-nms`"""
    device, batch_size = cls_probs.device, cls_probs.shape[0]
    anchors = anchors.squeeze(0)
    num_classes, num_anchors = cls_probs.shape[1], cls_probs.shape[2]
    out = []
    for i in range(batch_size):
        cls_prob, offset_pred = cls_probs[i], offset_preds[i].reshape(-1, 4)
        conf, class_id = torch.max(cls_prob[1:], 0)
        predicted_bb = offset_inverse(anchors, offset_pred)
        keep = nms(predicted_bb, conf, nms_threshold)
        # Find all non-`keep` indices and set the class to background
        all_idx = torch.arange(num_anchors, dtype=torch.long, device=device)
        combined = torch.cat((keep, all_idx))
        uniques, counts = combined.unique(return_counts=True)
        non_keep = uniques[counts == 1]
        all_id_sorted = torch.cat((keep, non_keep))
        class_id[non_keep] = -1
        class_id = class_id[all_id_sorted]
        conf, predicted_bb = conf[all_id_sorted], predicted_bb[all_id_sorted]
        # Here `pos_threshold` is a threshold for positive (non-background)
        # predictions
        below_min_idx = (conf < pos_threshold)
        class_id[below_min_idx] = -1
        conf[below_min_idx] = 1 - conf[below_min_idx]
        pred_info = torch.cat((class_id.unsqueeze(1),
                              conf.unsqueeze(1),
                              predicted_bb), dim=1)

        out.append(pred_info)
    return native.stack(out)

native.DATA_HUB['banana-detection'] = (
    native.DATA_URL + 'banana-detection.zip',
    '5de26c8fce5ccdea9f91267273464dc968d20d72')

def read_data_bananas(is_train=True):
    """Read the banana detection dataset images and labels.

    Defined in :numref:`sec_object-detection-dataset`"""
    data_dir = native.download_extract('banana-detection')
    csv_fname = os.path.join(data_dir, 'bananas_train' if is_train
                             else 'bananas_val', 'label.csv')
    csv_data = pd.read_csv(csv_fname)
    csv_data = csv_data.set_index('img_name')
    images, targets = [], []
    for img_name, target in csv_data.iterrows():
        images.append(torchvision.io.read_image(
            os.path.join(data_dir, 'bananas_train' if is_train else
                          'bananas_val', 'images', f'{img_name}')))
        # Here `target` contains (class, upper-left x, upper-left y,
        # lower-right x, lower-right y), where all the images have the same
        # banana class (index 0)
        targets.append(list(target))
    return images, torch.tensor(targets).unsqueeze(1) / 256

class BananasDataset(torch.utils.data.Dataset):
    """A customized dataset to load the banana detection dataset.

    Defined in :numref:`sec_object-detection-dataset`"""
    def __init__(self, is_train):
        self.features, self.labels = read_data_bananas(is_train)
        print('read ' + str(len(self.features)) + (f' training examples' if
                                                  is_train else f' validation examples'))

    def __getitem__(self, idx):
        return (self.features[idx].float(), self.labels[idx])

    def __len__(self):
        return len(self.features)

def load_data_bananas(batch_size):
    """Load the banana detection dataset.

```

```

Defined in :numref:`sec_object-detection-dataset`"""
train_iter = torch.utils.data.DataLoader(BananasDataset(is_train=True),
                                          batch_size, shuffle=True)
val_iter = torch.utils.data.DataLoader(BananasDataset(is_train=False),
                                       batch_size)
return train_iter, val_iter

native.DATA_HUB['voc2012'] = (native.DATA_URL + 'VOCtrainval_11-May-2012.tar',
                             '4e443f8a2eca6b1dac8a6c57641b67dd40621a49')

def read_voc_images(voc_dir, is_train=True):
    """Read all VOC feature and label images.

    Defined in :numref:`sec_semantic_segmentation`"""
    txt_fname = os.path.join(voc_dir, 'ImageSets', 'Segmentation',
                             'train.txt' if is_train else 'val.txt')
    mode = torchvision.io.image.ImageReadMode.RGB
    with open(txt_fname, 'r') as f:
        images = f.read().split()
    features, labels = [], []
    for i, fname in enumerate(images):
        features.append(torchvision.io.read_image(os.path.join(
            voc_dir, 'JPEGImages', f'{fname}.jpg')))
        labels.append(torchvision.io.read_image(os.path.join(
            voc_dir, 'SegmentationClass', f'{fname}.png'), mode))
    return features, labels

VOC_COLORMAP = [[0, 0, 0], [128, 0, 0], [0, 128, 0], [128, 128, 0],
                [0, 0, 128], [128, 0, 128], [0, 128, 128], [128, 128, 128],
                [64, 0, 0], [192, 0, 0], [64, 128, 0], [192, 128, 0],
                [64, 0, 128], [192, 0, 128], [64, 128, 128], [192, 128, 128],
                [0, 64, 0], [128, 64, 0], [0, 192, 0], [128, 192, 0],
                [0, 64, 128]]

VOC_CLASSES = ['background', 'aeroplane', 'bicycle', 'bird', 'boat',
               'bottle', 'bus', 'car', 'cat', 'chair', 'cow',
               'diningtable', 'dog', 'horse', 'motorbike', 'person',
               'potted plant', 'sheep', 'sofa', 'train', 'tv/monitor']

def voc_colormap2label():
    """Build the mapping from RGB to class indices for VOC labels.

    Defined in :numref:`sec_semantic_segmentation`"""
    colormap2label = torch.zeros(256 * 3, dtype=torch.long)
    for i, colormap in enumerate(VOC_COLORMAP):
        colormap2label[
            (colormap[0] * 256 + colormap[1]) * 256 + colormap[2]] = i
    return colormap2label

def voc_label_indices(colormap, colormap2label):
    """Map any RGB values in VOC labels to their class indices.

    Defined in :numref:`sec_semantic_segmentation`"""
    colormap = colormap.permute(1, 2, 0).numpy().astype('int32')
    idx = ((colormap[:, :, 0] * 256 + colormap[:, :, 1]) * 256
           + colormap[:, :, 2])
    return colormap2label[idx]

def voc_rand_crop(feature, label, height, width):
    """Randomly crop both feature and label images.

    Defined in :numref:`sec_semantic_segmentation`"""
    rect = torchvision.transforms.RandomCrop.get_params(
        feature, (height, width))
    feature = torchvision.transforms.functional.crop(feature, *rect)
    label = torchvision.transforms.functional.crop(label, *rect)
    return feature, label

class VOCSegDataset(torch.utils.data.Dataset):
    """A customized dataset to load the VOC dataset.

    Defined in :numref:`sec_semantic_segmentation`"""
    def __init__(self, is_train, crop_size, voc_dir):
        self.transform = torchvision.transforms.Normalize(
            mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
        self.crop_size = crop_size

```

```

        features, labels = read_voc_images(voc_dir, is_train=is_train)
        self.features = [self.normalize_image(feature)
                        for feature in self.filter(features)]
        self.labels = self.filter(labels)
        self.colormap2label = voc_colormap2label()
        print('read ' + str(len(self.features)) + ' examples')

def normalize_image(self, img):
    return self.transform(img.float() / 255)

def filter(self, imgs):
    return [img for img in imgs if (
        img.shape[1] >= self.crop_size[0] and
        img.shape[2] >= self.crop_size[1])]

def __getitem__(self, idx):
    feature, label = voc_rand_crop(self.features[idx], self.labels[idx],
                                *self.crop_size)
    return (feature, voc_label_indices(label, self.colormap2label))

def __len__(self):
    return len(self.features)

def load_data_voc(batch_size, crop_size):
    """Load the VOC semantic segmentation dataset.

    Defined in :numref:`sec_semantic_segmentation`"""
    voc_dir = native.download_extract('voc2012', os.path.join(
        'VOCdevkit', 'VOC2012'))
    num_workers = native.get_data_loader_workers()
    train_iter = torch.utils.data.DataLoader(
        VOCSegDataset(True, crop_size, voc_dir), batch_size,
        shuffle=True, drop_last=True, num_workers=num_workers)
    test_iter = torch.utils.data.DataLoader(
        VOCSegDataset(False, crop_size, voc_dir), batch_size,
        drop_last=True, num_workers=num_workers)
    return train_iter, test_iter

native.DATA_HUB['cifar10_tiny'] = (native.DATA_URL + 'kaggle_cifar10_tiny.zip',
                                '2068874e4b9a9f0fb07ebe0ad2b29754449ccacd')

def read_csv_labels(fname):
    """Read `fname` to return a filename to label dictionary.

    Defined in :numref:`sec_kaggle_cifar10`"""
    with open(fname, 'r') as f:
        # Skip the file header line (column name)
        lines = f.readlines()[1:]
    tokens = [l.rstrip().split(',') for l in lines]
    return dict((name, label) for name, label in tokens)

def copyfile(filename, target_dir):
    """Copy a file into a target directory.

    Defined in :numref:`sec_kaggle_cifar10`"""
    os.makedirs(target_dir, exist_ok=True)
    shutil.copy(filename, target_dir)

def reorg_train_valid(data_dir, labels, valid_ratio):
    """Split the validation set out of the original training set.

    Defined in :numref:`sec_kaggle_cifar10`"""
    # The number of examples of the class that has the fewest examples in the
    # training dataset
    n = collections.Counter(labels.values()).most_common()[-1][1]
    # The number of examples per class for the validation set
    n_valid_per_label = max(1, math.floor(n * valid_ratio))
    label_count = {}
    for train_file in os.listdir(os.path.join(data_dir, 'train')):
        label = labels[train_file.split('.')[0]]
        fname = os.path.join(data_dir, 'train', train_file)
        copyfile(fname, os.path.join(data_dir, 'train_valid_test',
                                     'train_valid', label))
        if label not in label_count or label_count[label] < n_valid_per_label:
            copyfile(fname, os.path.join(data_dir, 'train_valid_test',
                                         'valid', label))

```

```

        label_count[label] = label_count.get(label, 0) + 1
    else:
        copyfile(fname, os.path.join(data_dir, 'train_valid_test',
                                      'train', label))
    return n_valid_per_label

def reorg_test(data_dir):
    """Organize the testing set for data loading during prediction.

    Defined in :numref:`sec_kaggle_cifar10`"""
    for test_file in os.listdir(os.path.join(data_dir, 'test')):
        copyfile(os.path.join(data_dir, 'test', test_file),
                  os.path.join(data_dir, 'train_valid_test', 'test',
                              'unknown'))

native.DATA_HUB['dog_tiny'] = (native.DATA_URL + 'kaggle_dog_tiny.zip',
                              '0cb91d09b814ecdc07b50f31f8dcad3e81d6a86d')

native.DATA_HUB['ptb'] = (native.DATA_URL + 'ptb.zip',
                          '319d85e578af0cdc590547f26231e4e31cdf1e42')

def read_ptb():
    """Load the PTB dataset into a list of text lines.

    Defined in :numref:`sec_word2vec_data`"""
    data_dir = native.download_extract('ptb')
    # Read the training set
    with open(os.path.join(data_dir, 'ptb.train.txt')) as f:
        raw_text = f.read()
    return [line.split() for line in raw_text.split('\n')]

def subsample(sentences, vocab):
    """Subsample high-frequency words.

    Defined in :numref:`sec_word2vec_data`"""
    # Exclude unknown tokens ('<unk>')
    sentences = [[token for token in line if vocab[token] != vocab.unk]
                  for line in sentences]
    counter = collections.Counter([
        token for line in sentences for token in line])
    num_tokens = sum(counter.values())

    # Return True if `token` is kept during subsampling
    def keep(token):
        return (random.uniform(0, 1) <
                math.sqrt(1e-4 / counter[token] * num_tokens))

    return ([[token for token in line if keep(token)] for line in sentences],
            counter)

def get_centers_and_contexts(corpus, max_window_size):
    """Return center words and context words in skip-gram.

    Defined in :numref:`sec_word2vec_data`"""
    centers, contexts = [], []
    for line in corpus:
        # To form a "center word--context word" pair, each sentence needs to
        # have at least 2 words
        if len(line) < 2:
            continue
        centers += line
        for i in range(len(line)): # Context window centered at `i`
            window_size = random.randint(1, max_window_size)
            indices = list(range(max(0, i - window_size),
                                  min(len(line), i + 1 + window_size)))
            # Exclude the center word from the context words
            indices.remove(i)
            contexts.append([line[idx] for idx in indices])
    return centers, contexts

class RandomGenerator:
    """Randomly draw among {1, ..., n} according to n sampling weights."""
    def __init__(self, sampling_weights):
        """Defined in :numref:`sec_word2vec_data`"""
        # Exclude
        self.population = list(range(1, len(sampling_weights) + 1))
        self.sampling_weights = sampling_weights

```

```

        self.candidates = []
        self.i = 0

    def draw(self):
        if self.i == len(self.candidates):
            # Cache `k` random sampling results
            self.candidates = random.choices(
                self.population, self.sampling_weights, k=10000)
            self.i = 0
        self.i += 1
        return self.candidates[self.i - 1]

def get_negatives(all_contexts, vocab, counter, K):
    """Return noise words in negative sampling.

    Defined in :numref:`sec_word2vec_data`"""
    # Sampling weights for words with indices 1, 2, ... (index 0 is the
    # excluded unknown token) in the vocabulary
    sampling_weights = [counter[vocab.to_tokens(i)]**0.75
                        for i in range(1, len(vocab))]
    all_negatives, generator = [], RandomGenerator(sampling_weights)
    for contexts in all_contexts:
        negatives = []
        while len(negatives) < len(contexts) * K:
            neg = generator.draw()
            # Noise words cannot be context words
            if neg not in contexts:
                negatives.append(neg)
        all_negatives.append(negatives)
    return all_negatives

def batchify(data):
    """Return a minibatch of examples for skip-gram with negative sampling.

    Defined in :numref:`sec_word2vec_data`"""
    max_len = max(len(c) + len(n) for _, c, n in data)
    centers, contexts_negatives, masks, labels = [], [], [], []

    for center, context, negative in data:
        cur_len = len(context) + len(negative)
        centers += [center]
        contexts_negatives += [context + negative + [0] * (max_len - cur_len)]
        masks += [[1] * cur_len + [0] * (max_len - cur_len)]
        labels += [[1] * len(context) + [0] * (max_len - len(context))]
    return (native.reshape(native.tensor(centers), (-1, 1)), native.tensor(
        contexts_negatives), native.tensor(masks), native.tensor(labels))

def load_data_ptb(batch_size, max_window_size, num_noise_words):
    """Download the PTB dataset and then load it into memory.

    Defined in :numref:`subsec_word2vec-minibatch-loading`"""
    num_workers = native.get_data_loader_workers()
    sentences = read_ptb()
    vocab = native.Vocab(sentences, min_freq=10)
    subsampled, counter = subsample(sentences, vocab)
    corpus = [vocab[line] for line in subsampled]
    all_centers, all_contexts = get_centers_and_contexts(
        corpus, max_window_size)
    all_negatives = get_negatives(
        all_contexts, vocab, counter, num_noise_words)

    class PTBDataset(torch.utils.data.Dataset):
        def __init__(self, centers, contexts, negatives):
            assert len(centers) == len(contexts) == len(negatives)
            self.centers = centers
            self.contexts = contexts
            self.negatives = negatives

        def __getitem__(self, index):
            return (self.centers[index], self.contexts[index],
                    self.negatives[index])

        def __len__(self):
            return len(self.centers)

    dataset = PTBDataset(all_centers, all_contexts, all_negatives)
    data_iter = torch.utils.data.DataLoader(dataset, batch_size, shuffle=True,

```

```

        collate_fn=batchify,
        num_workers=num_workers)

    return data_iter, vocab

native.DATA_HUB['glove.6b.50d'] = (native.DATA_URL + 'glove.6B.50d.zip',
    '0b8703943ccdb6eb788e6f091b8946e82231bc4d')

native.DATA_HUB['glove.6b.100d'] = (native.DATA_URL + 'glove.6B.100d.zip',
    'cd43bfb07e44e6f27cbcc7bc9ae3d80284fdaf5a')

native.DATA_HUB['glove.42b.300d'] = (native.DATA_URL + 'glove.42B.300d.zip',
    'b5116e234e9eb9076672cfeabf5469f3eec904fa')

native.DATA_HUB['wiki.en'] = (native.DATA_URL + 'wiki.en.zip',
    'c1816da3821ae9f43899be655002f6c723e91b88')

class TokenEmbedding:
    """Token Embedding."""
    def __init__(self, embedding_name):
        """Defined in :numref:`sec_synonyms`"""
        self.idx_to_token, self.idx_to_vec = self._load_embedding(
            embedding_name)
        self.unknown_idx = 0
        self.token_to_idx = {token: idx for idx, token in
            enumerate(self.idx_to_token)}

    def _load_embedding(self, embedding_name):
        idx_to_token, idx_to_vec = ['<unk>'], []
        data_dir = native.download_extract(embedding_name)
        # GloVe website: https://nlp.stanford.edu/projects/glove/
        # fastText website: https://fasttext.cc/
        with open(os.path.join(data_dir, 'vec.txt'), 'r') as f:
            for line in f:
                elems = line.rstrip().split(' ')
                token, elems = elems[0], [float(elem) for elem in elems[1:]]
                # Skip header information, such as the top row in fastText
                if len(elems) > 1:
                    idx_to_token.append(token)
                    idx_to_vec.append(elems)
        idx_to_vec = [[0] * len(idx_to_vec[0])] + idx_to_vec
        return idx_to_token, native.tensor(idx_to_vec)

    def __getitem__(self, tokens):
        indices = [self.token_to_idx.get(token, self.unknown_idx)
            for token in tokens]
        vecs = self.idx_to_vec[native.tensor(indices)]
        return vecs

    def __len__(self):
        return len(self.idx_to_token)

def get_tokens_and_segments(tokens_a, tokens_b=None):
    """Get tokens of the BERT input sequence and their segment IDs.

    Defined in :numref:`sec_bert`"""
    tokens = ['<cls>'] + tokens_a + ['<sep>']
    # 0 and 1 are marking segment A and B, respectively
    segments = [0] * (len(tokens_a) + 2)
    if tokens_b is not None:
        tokens += tokens_b + ['<sep>']
        segments += [1] * (len(tokens_b) + 1)
    return tokens, segments

class BERTEncoder(nn.Module):
    """BERT encoder.

    Defined in :numref:`subsec_bert_input_rep`"""
    def __init__(self, vocab_size, num_hiddens, ffn_num_hiddens, num_heads,
        num_blks, dropout, max_len=1000, **kwargs):
        super(BERTEncoder, self).__init__(**kwargs)
        self.token_embedding = nn.Embedding(vocab_size, num_hiddens)
        self.segment_embedding = nn.Embedding(2, num_hiddens)
        self.blks = nn.Sequential()
        for i in range(num_blks):
            self.blks.add_module(f"{i}", native.TransformerEncoderBlock(
                num_hiddens, ffn_num_hiddens, num_heads, dropout, True))
        # In BERT, positional embeddings are learnable, thus we create a

```

```

        # parameter of positional embeddings that are long enough
        self.pos_embedding = nn.Parameter(torch.randn(1, max_len,
                                                    num_hiddens))

def forward(self, tokens, segments, valid_lens):
    # Shape of `X` remains unchanged in the following code snippet:
    # (batch size, max sequence length, `num_hiddens`)
    X = self.token_embedding(tokens) + self.segment_embedding(segments)
    X = X + self.pos_embedding[:, :X.shape[1], :]
    for blk in self.blks:
        X = blk(X, valid_lens)
    return X

class MaskLM(nn.Module):
    """The masked language model task of BERT.

    Defined in :numref:`subsec_bert_input_rep`"""
    def __init__(self, vocab_size, num_hiddens, **kwargs):
        super(MaskLM, self).__init__(**kwargs)
        self.mlp = nn.Sequential(nn.LazyLinear(num_hiddens),
                                nn.ReLU(),
                                nn.LayerNorm(num_hiddens),
                                nn.LazyLinear(vocab_size))

    def forward(self, X, pred_positions):
        num_pred_positions = pred_positions.shape[1]
        pred_positions = pred_positions.reshape(-1)
        batch_size = X.shape[0]
        batch_idx = torch.arange(0, batch_size)
        # Suppose that `batch_size` = 2, `num_pred_positions` = 3, then
        # `batch_idx` is `torch.tensor([0, 0, 0, 1, 1, 1])`
        batch_idx = torch.repeat_interleave(batch_idx, num_pred_positions)
        masked_X = X[batch_idx, pred_positions]
        masked_X = masked_X.reshape((batch_size, num_pred_positions, -1))
        mlm_Y_hat = self.mlp(masked_X)
        return mlm_Y_hat

class NextSentencePred(nn.Module):
    """The next sentence prediction task of BERT.

    Defined in :numref:`subsec_mlm`"""
    def __init__(self, **kwargs):
        super(NextSentencePred, self).__init__(**kwargs)
        self.output = nn.LazyLinear(2)

    def forward(self, X):
        # `X` shape: (batch size, `num_hiddens`)
        return self.output(X)

class BERTModel(nn.Module):
    """The BERT model.

    Defined in :numref:`subsec_nsp`"""
    def __init__(self, vocab_size, num_hiddens, ffn_num_hiddens,
                 num_heads, num_blks, dropout, max_len=1000):
        super(BERTModel, self).__init__()
        self.encoder = BERTEncoder(vocab_size, num_hiddens, ffn_num_hiddens,
                                   num_heads, num_blks, dropout,
                                   max_len=max_len)
        self.hidden = nn.Sequential(nn.LazyLinear(num_hiddens),
                                    nn.Tanh())
        self.mlm = MaskLM(vocab_size, num_hiddens)
        self.nsp = NextSentencePred()

    def forward(self, tokens, segments, valid_lens=None, pred_positions=None):
        encoded_X = self.encoder(tokens, segments, valid_lens)
        if pred_positions is not None:
            mlm_Y_hat = self.mlm(encoded_X, pred_positions)
        else:
            mlm_Y_hat = None
        # The hidden layer of the MLP classifier for next sentence prediction.
        # 0 is the index of the '<cls>' token
        nsp_Y_hat = self.nsp(self.hidden(encoded_X[:, 0, :]))
        return encoded_X, mlm_Y_hat, nsp_Y_hat

native.DATA_HUB['wikitext-2'] = (

```

```

'https://s3.amazonaws.com/research.metamind.io/wikitext/'
'wikitext-2-v1.zip', '3c914d17d80b1459be871a5039ac23e752a53cbe')

def _read_wiki(data_dir):
    """Defined in :numref:`sec_bert-dataset`"""
    file_name = os.path.join(data_dir, 'wiki.train.tokens')
    with open(file_name, 'r') as f:
        lines = f.readlines()
    # Uppercase letters are converted to lowercase ones
    paragraphs = [line.strip().lower().split(' . ')]
    for line in lines if len(line.split(' . ')) >= 2]
    random.shuffle(paragraphs)
    return paragraphs

def _get_next_sentence(sentence, next_sentence, paragraphs):
    """Defined in :numref:`sec_bert-dataset`"""
    if random.random() < 0.5:
        is_next = True
    else:
        # `paragraphs` is a list of lists of lists
        next_sentence = random.choice(random.choice(paragraphs))
        is_next = False
    return sentence, next_sentence, is_next

def _get_nsp_data_from_paragraph(paragraph, paragraphs, vocab, max_len):
    """Defined in :numref:`sec_bert-dataset`"""
    nsp_data_from_paragraph = []
    for i in range(len(paragraph) - 1):
        tokens_a, tokens_b, is_next = _get_next_sentence(
            paragraph[i], paragraph[i + 1], paragraphs)
        # Consider 1 '<cls>' token and 2 '<sep>' tokens
        if len(tokens_a) + len(tokens_b) + 3 > max_len:
            continue
        tokens, segments = native.get_tokens_and_segments(tokens_a, tokens_b)
        nsp_data_from_paragraph.append((tokens, segments, is_next))
    return nsp_data_from_paragraph

def _replace_mlm_tokens(tokens, candidate_pred_positions, num_mlm_preds,
                        vocab):
    """Defined in :numref:`sec_bert-dataset`"""
    # For the input of a masked language model, make a new copy of tokens and
    # replace some of them by '<mask>' or random tokens
    mlm_input_tokens = [token for token in tokens]
    pred_positions_and_labels = []
    # Shuffle for getting 15% random tokens for prediction in the masked
    # language modeling task
    random.shuffle(candidate_pred_positions)
    for mlm_pred_position in candidate_pred_positions:
        if len(pred_positions_and_labels) >= num_mlm_preds:
            break
        masked_token = None
        # 80% of the time: replace the word with the '<mask>' token
        if random.random() < 0.8:
            masked_token = '<mask>'
        else:
            # 10% of the time: keep the word unchanged
            if random.random() < 0.5:
                masked_token = tokens[mlm_pred_position]
            # 10% of the time: replace the word with a random word
            else:
                masked_token = random.choice(vocab.idx_to_token)
        mlm_input_tokens[mlm_pred_position] = masked_token
        pred_positions_and_labels.append(
            (mlm_pred_position, tokens[mlm_pred_position]))
    return mlm_input_tokens, pred_positions_and_labels

def _get_mlm_data_from_tokens(tokens, vocab):
    """Defined in :numref:`subsec_prepare_mlm_data`"""
    candidate_pred_positions = []
    # `tokens` is a list of strings
    for i, token in enumerate(tokens):
        # Special tokens are not predicted in the masked language modeling
        # task
        if token in ['<cls>', '<sep>']:
            continue
        candidate_pred_positions.append(i)
    # 15% of random tokens are predicted in the masked language modeling task

```

```

num_mlm_preds = max(1, round(len(tokens) * 0.15))
mlm_input_tokens, pred_positions_and_labels = _replace_mlm_tokens(
    tokens, candidate_pred_positions, num_mlm_preds, vocab)
pred_positions_and_labels = sorted(pred_positions_and_labels,
                                   key=lambda x: x[0])
pred_positions = [v[0] for v in pred_positions_and_labels]
mlm_pred_labels = [v[1] for v in pred_positions_and_labels]
return vocab[mlm_input_tokens], pred_positions, vocab[mlm_pred_labels]

def _pad_bert_inputs(examples, max_len, vocab):
    """Defined in :numref:`subsec_prepare_mlm_data`"""
    max_num_mlm_preds = round(max_len * 0.15)
    all_token_ids, all_segments, valid_lens, = [], [], []
    all_pred_positions, all_mlm_weights, all_mlm_labels = [], [], []
    nsp_labels = []
    for (token_ids, pred_positions, mlm_pred_label_ids, segments,
         is_next) in examples:
        all_token_ids.append(torch.tensor(token_ids + [vocab['<pad>']] * (
            max_len - len(token_ids)), dtype=torch.long))
        all_segments.append(torch.tensor(segments + [0] * (
            max_len - len(segments)), dtype=torch.long))
        # `valid_lens` excludes count of '<pad>' tokens
        valid_lens.append(torch.tensor(len(token_ids), dtype=torch.float32))
        all_pred_positions.append(torch.tensor(pred_positions + [0] * (
            max_num_mlm_preds - len(pred_positions)), dtype=torch.long))
        # Predictions of padded tokens will be filtered out in the loss via
        # multiplication of 0 weights
        all_mlm_weights.append(
            torch.tensor([1.0] * len(mlm_pred_label_ids) + [0.0] * (
                max_num_mlm_preds - len(pred_positions)),
                dtype=torch.float32))
        all_mlm_labels.append(torch.tensor(mlm_pred_label_ids + [0] * (
            max_num_mlm_preds - len(mlm_pred_label_ids)), dtype=torch.long))
        nsp_labels.append(torch.tensor(is_next, dtype=torch.long))
    return (all_token_ids, all_segments, valid_lens, all_pred_positions,
            all_mlm_weights, all_mlm_labels, nsp_labels)

class WikiTextDataset(torch.utils.data.Dataset):
    """Defined in :numref:`subsec_prepare_mlm_data`"""
    def __init__(self, paragraphs, max_len):
        # Input `paragraphs[i]` is a list of sentence strings representing a
        # paragraph; while output `paragraphs[i]` is a list of sentences
        # representing a paragraph, where each sentence is a list of tokens
        paragraphs = [native.tokenize(
            paragraph, token='word') for paragraph in paragraphs]
        sentences = [sentence for paragraph in paragraphs
                     for sentence in paragraph]
        self.vocab = native.Vocab(sentences, min_freq=5, reserved_tokens=[
            '<pad>', '<mask>', '<cls>', '<sep>'])
        # Get data for the next sentence prediction task
        examples = []
        for paragraph in paragraphs:
            examples.extend(_get_nsp_data_from_paragraph(
                paragraph, paragraphs, self.vocab, max_len))
        # Get data for the masked language model task
        examples = [(_get_mlm_data_from_tokens(tokens, self.vocab)
                     + (segments, is_next))
                    for tokens, segments, is_next in examples]
        # Pad inputs
        (self.all_token_ids, self.all_segments, self.valid_lens,
         self.all_pred_positions, self.all_mlm_weights,
         self.all_mlm_labels, self.nsp_labels) = _pad_bert_inputs(
            examples, max_len, self.vocab)

    def __getitem__(self, idx):
        return (self.all_token_ids[idx], self.all_segments[idx],
                self.valid_lens[idx], self.all_pred_positions[idx],
                self.all_mlm_weights[idx], self.all_mlm_labels[idx],
                self.nsp_labels[idx])

    def __len__(self):
        return len(self.all_token_ids)

def load_data_wiki(batch_size, max_len):
    """Load the WikiText-2 dataset.

    Defined in :numref:`subsec_prepare_mlm_data`"""

```

```

num_workers = native.get_dataloader_workers()
data_dir = native.download_extract('wikitext-2', 'wikitext-2')
paragraphs = _read_wiki(data_dir)
train_set = _WikiTextDataset(paragraphs, max_len)
train_iter = torch.utils.data.DataLoader(train_set, batch_size,
                                         shuffle=True, num_workers=num_workers)

return train_iter, train_set.vocab

def _get_batch_loss_bert(net, loss, vocab_size, tokens_X,
                        segments_X, valid_lens_x,
                        pred_positions_X, mlm_weights_X,
                        mlm_Y, nsp_y):
    """Defined in :numref:`sec_bert-pretraining`"""

    # Forward pass
    _, mlm_Y_hat, nsp_Y_hat = net(tokens_X, segments_X,
                                  valid_lens_x.reshape(-1),
                                  pred_positions_X)

    # Compute masked language model loss
    mlm_l = loss(mlm_Y_hat.reshape(-1, vocab_size), mlm_Y.reshape(-1)) * \
        mlm_weights_X.reshape(-1, 1)
    mlm_l = mlm_l.sum() / (mlm_weights_X.sum() + 1e-8)
    # Compute next sentence prediction loss
    nsp_l = loss(nsp_Y_hat, nsp_y)
    l = mlm_l + nsp_l
    return mlm_l, nsp_l, l

native.DATA_HUB['aclImdb'] = (native.DATA_URL + 'aclImdb_v1.tar.gz',
                             '01ada507287d82875905620988597833ad4e0903')

def read_imdb(data_dir, is_train):
    """Read the IMDb review dataset text sequences and labels.

    Defined in :numref:`sec_sentiment`"""
    data, labels = [], []
    for label in ('pos', 'neg'):
        folder_name = os.path.join(data_dir, 'train' if is_train else 'test',
                                    label)
        for file in os.listdir(folder_name):
            with open(os.path.join(folder_name, file), 'rb') as f:
                review = f.read().decode('utf-8').replace('\n', '')
                data.append(review)
                labels.append(1 if label == 'pos' else 0)
    return data, labels

def load_data_imdb(batch_size, num_steps=500):
    """Return data iterators and the vocabulary of the IMDb review dataset.

    Defined in :numref:`sec_sentiment`"""
    data_dir = native.download_extract('aclImdb', 'aclImdb')
    train_data = read_imdb(data_dir, True)
    test_data = read_imdb(data_dir, False)
    train_tokens = native.tokenize(train_data[0], token='word')
    test_tokens = native.tokenize(test_data[0], token='word')
    vocab = native.Vocab(train_tokens, min_freq=5)
    train_features = torch.tensor([native.truncate_pad(
        vocab[line], num_steps, vocab['<pad>']) for line in train_tokens])
    test_features = torch.tensor([native.truncate_pad(
        vocab[line], num_steps, vocab['<pad>']) for line in test_tokens])
    train_iter = native.load_array((train_features, torch.tensor(train_data[1])),
                                   batch_size)
    test_iter = native.load_array((test_features, torch.tensor(test_data[1])),
                                   batch_size,
                                   is_train=False)
    return train_iter, test_iter, vocab

def predict_sentiment(net, vocab, sequence):
    """Predict the sentiment of a text sequence.

    Defined in :numref:`sec_sentiment_rnn`"""
    sequence = torch.tensor(vocab[sequence.split()], device=native.try_gpu())
    label = torch.argmax(net(sequence.reshape(1, -1)), dim=1)
    return 'positive' if label == 1 else 'negative'

native.DATA_HUB['SNLI'] = (
    'https://nlp.stanford.edu/projects/snli/snli_1.0.zip',
    '9fcdc07509c7e87ec61c640c1b2753d9041758e4')

```

```

def read_snli(data_dir, is_train):
    """Read the SNLI dataset into premises, hypotheses, and labels.

    Defined in :numref:`sec_natural-language-inference-and-dataset`"""
    def extract_text(s):
        # Remove information that will not be used by us
        s = re.sub('\\(', ' ', s)
        s = re.sub('\\)', ' ', s)
        # Substitute two or more consecutive whitespace with space
        s = re.sub('\\s{2,}', ' ', s)
        return s.strip()
    label_set = {'entailment': 0, 'contradiction': 1, 'neutral': 2}
    file_name = os.path.join(data_dir, 'snli_1.0_train.txt'
                             if is_train else 'snli_1.0_test.txt')
    with open(file_name, 'r') as f:
        rows = [row.split('\\t') for row in f.readlines()[1:]]
    premises = [extract_text(row[1]) for row in rows if row[0] in label_set]
    hypotheses = [extract_text(row[2]) for row in rows if row[0] in label_set]
    labels = [label_set[row[0]] for row in rows if row[0] in label_set]
    return premises, hypotheses, labels

class SNLIDataset(torch.utils.data.Dataset):
    """A customized dataset to load the SNLI dataset.

    Defined in :numref:`sec_natural-language-inference-and-dataset`"""
    def __init__(self, dataset, num_steps, vocab=None):
        self.num_steps = num_steps
        all_premise_tokens = native.tokenize(dataset[0])
        all_hypothesis_tokens = native.tokenize(dataset[1])
        if vocab is None:
            self.vocab = native.Vocab(all_premise_tokens + all_hypothesis_tokens,
                                      min_freq=5, reserved_tokens=['<pad>'])
        else:
            self.vocab = vocab
        self.premises = self._pad(all_premise_tokens)
        self.hypotheses = self._pad(all_hypothesis_tokens)
        self.labels = torch.tensor(dataset[2])
        print('read ' + str(len(self.premises)) + ' examples')

    def _pad(self, lines):
        return torch.tensor([native.truncate_pad(
            self.vocab[line], self.num_steps, self.vocab['<pad>'])
            for line in lines])

    def __getitem__(self, idx):
        return (self.premises[idx], self.hypotheses[idx]), self.labels[idx]

    def __len__(self):
        return len(self.premises)

def load_data_snli(batch_size, num_steps=50):
    """Download the SNLI dataset and return data iterators and vocabulary.

    Defined in :numref:`sec_natural-language-inference-and-dataset`"""
    num_workers = native.get_data_loader_workers()
    data_dir = native.download_extract('SNLI')

    train_data = read_snli(data_dir, True)
    test_data = read_snli(data_dir, False)
    train_set = SNLIDataset(train_data, num_steps)
    test_set = SNLIDataset(test_data, num_steps, train_set.vocab)
    train_iter = torch.utils.data.DataLoader(train_set, batch_size,
                                             shuffle=True,
                                             num_workers=num_workers)
    test_iter = torch.utils.data.DataLoader(test_set, batch_size,
                                             shuffle=False,
                                             num_workers=num_workers)
    return train_iter, test_iter, train_set.vocab

def predict_snli(net, vocab, premise, hypothesis):
    """Predict the logical relationship between the premise and hypothesis.

    Defined in :numref:`sec_natural-language-inference-attention`"""
    net.eval()
    premise = torch.tensor(vocab[premise], device=native.try_gpu())
    hypothesis = torch.tensor(vocab[hypothesis], device=native.try_gpu())
    label = torch.argmax(net([premise.reshape((1, -1)),

```

```

        hypothesis.reshape((1, -1)])), dim=1)
    return 'entailment' if label == 0 else 'contradiction' if label == 1 \
        else 'neutral'

def rbfkernel(x1, x2, ls=4.):
    dist = distance_matrix(np.expand_dims(x1, 1), np.expand_dims(x2, 1))
    return np.exp(-(1. / ls / 2) * (dist ** 2))

class HPOTrainer(native.Trainer):
    """Defined in :numref:`sec_definition_hpo`"""
    def validation_error(self):
        self.model.eval()
        accuracy = 0
        val_batch_idx = 0
        for batch in self.val_dataloader:
            with torch.no_grad():
                x, y = self.prepare_batch(batch)
                y_hat = self.model(x)
                accuracy += self.model.accuracy(y_hat, y)
            val_batch_idx += 1
        return 1 - accuracy / val_batch_idx

class HP0Searcher(native.HyperParameters):
    """Defined in :numref:`sec_api_hpo`"""
    def sample_configuration() -> dict:
        raise NotImplementedError

    def update(self, config: dict, error: float, additional_info=None):
        pass

class RandomSearcher(HP0Searcher):
    """Defined in :numref:`sec_api_hpo`"""
    def __init__(self, config_space: dict, initial_config=None):
        self.save_hyperparameters()

    def sample_configuration(self) -> dict:
        if self.initial_config is not None:
            result = self.initial_config
            self.initial_config = None
        else:
            result = {
                name: domain.rvs()
                for name, domain in self.config_space.items()
            }
        return result

class HP0Scheduler(native.HyperParameters):
    """Defined in :numref:`sec_api_hpo`"""
    def suggest(self) -> dict:
        raise NotImplementedError

    def update(self, config: dict, error: float, info=None):
        raise NotImplementedError

class BasicScheduler(HP0Scheduler):
    """Defined in :numref:`sec_api_hpo`"""
    def __init__(self, searcher: HP0Searcher):
        self.save_hyperparameters()

    def suggest(self) -> dict:
        return self.searcher.sample_configuration()

    def update(self, config: dict, error: float, info=None):
        self.searcher.update(config, error, additional_info=info)

class HP0Tuner(native.HyperParameters):
    """Defined in :numref:`sec_api_hpo`"""
    def __init__(self, scheduler: HP0Scheduler, objective: callable):
        self.save_hyperparameters()
        # Bookkeeping results for plotting
        self.incumbent = None
        self.incumbent_error = None
        self.incumbent_trajectory = []
        self.cumulative_runtime = []
        self.current_runtime = 0
        self.records = []

```

```

def run(self, number_of_trials):
    for i in range(number_of_trials):
        start_time = time.time()
        config = self.scheduler.suggest()
        print(f"Trial {i}: config = {config}")
        error = self.objective(**config)
        error = float(native.numpy(error.cpu()))
        self.scheduler.update(config, error)
        runtime = time.time() - start_time
        self.bookkeeping(config, error, runtime)
        print(f"    error = {error}, runtime = {runtime}")

def bookkeeping(self, config: dict, error: float, runtime: float):
    """Defined in :numref:`sec_api_hpo`"""
    self.records.append({"config": config, "error": error, "runtime": runtime})
    # Check if the last hyperparameter configuration performs better
    # than the incumbent
    if self.incumbent is None or self.incumbent_error > error:
        self.incumbent = config
        self.incumbent_error = error
    # Add current best observed performance to the optimization trajectory
    self.incumbent_trajectory.append(self.incumbent_error)
    # Update runtime

    self.current_runtime += runtime
    self.cumulative_runtime.append(self.current_runtime)

def hpo_objective_lenet(learning_rate, batch_size, max_epochs=10):
    """Defined in :numref:`sec_api_hpo`"""
    model = native.LeNet(lr=learning_rate, num_classes=10)
    trainer = native.HPOTrainer(max_epochs=max_epochs, num_gpus=1)
    data = native.FashionMNIST(batch_size=batch_size)
    model.apply_init([next(iter(data.get_dataloader(True)))[0]], native.init_cnn)
    trainer.fit(model=model, data=data)
    validation_error = trainer.validation_error()
    return validation_error

class SuccessiveHalvingScheduler(native.HPOScheduler):
    """Defined in :numref:`sec_mf_hpo`"""
    def __init__(self, searcher, eta, r_min, r_max, prefact=1):
        self.save_hyperparameters()
        # Compute K, which is later used to determine the number of configurations
        self.K = int(np.log(r_max / r_min) / np.log(eta))
        # Define the rungs
        self.rung_levels = [r_min * eta ** k for k in range(self.K + 1)]
        if r_max not in self.rung_levels:
            # The final rung should be r_max
            self.rung_levels.append(r_max)
            self.K += 1
        # Bookkeeping
        self.observed_error_at_rungs = defaultdict(list)
        self.all_observed_error_at_rungs = defaultdict(list)
        # Our processing queue
        self.queue = []

    def suggest(self):
        """Defined in :numref:`sec_mf_hpo_sh`"""
        if len(self.queue) == 0:
            # Start a new round of successive halving
            # Number of configurations for the first rung:
            n0 = int(self.prefact * self.eta ** self.K)
            for _ in range(n0):
                config = self.searcher.sample_configuration()
                config["max_epochs"] = self.r_min # Set r = r_min
                self.queue.append(config)
            # Return an element from the queue
            return self.queue.pop()

    def update(self, config: dict, error: float, info=None):
        """Defined in :numref:`sec_mf_hpo_sh`"""
        ri = int(config["max_epochs"]) # Rung r_i
        # Update our searcher, e.g if we use Bayesian optimization later
        self.searcher.update(config, error, additional_info=info)
        self.all_observed_error_at_rungs[ri].append((config, error))
        if ri < self.r_max:
            # Bookkeeping
            self.observed_error_at_rungs[ri].append((config, error))
            # Determine how many configurations should be evaluated on this rung

```

```

        ki = self.K - self.rung_levels.index(ri)
        ni = int(self.prefact * self.eta ** ki)
        # If we observed all configuration on this rung r_i, we estimate the
        # top 1 / eta configuration, add them to queue and promote them for
        # the next rung r_{i+1}
        if len(self.observed_error_at_rungs[ri]) >= ni:
            kiplus1 = ki - 1
            niplus1 = int(self.prefact * self.eta ** kiplus1)
            best_performing_configurations = self.get_top_n_configurations(
                rung_level=ri, n=niplus1
            )
            riplus1 = self.rung_levels[self.K - kiplus1] # r_{i+1}
            # Queue may not be empty: insert new entries at the beginning
            self.queue = [
                dict(config, max_epochs=riplus1)
                for config in best_performing_configurations
            ] + self.queue
            self.observed_error_at_rungs[ri] = [] # Reset

def get_top_n_configurations(self, rung_level, n):
    """Defined in :numref:`sec_mf_hpo_sh`"""
    rung = self.observed_error_at_rungs[rung_level]
    if not rung:
        return []
    sorted_rung = sorted(rung, key=lambda x: x[1])
    return [x[0] for x in sorted_rung[:n]]

def update_D(X, Z, net_D, net_G, loss, trainer_D):
    """Update discriminator.

    Defined in :numref:`sec_basic_gan`"""
    batch_size = X.shape[0]
    ones = torch.ones((batch_size,), device=X.device)
    zeros = torch.zeros((batch_size,), device=X.device)
    trainer_D.zero_grad()
    real_Y = net_D(X)
    fake_X = net_G(Z)
    # Do not need to compute gradient for `net_G`, detach it from
    # computing gradients.
    fake_Y = net_D(fake_X.detach())
    loss_D = (loss(real_Y, ones.reshape(real_Y.shape)) +
              loss(fake_Y, zeros.reshape(fake_Y.shape))) / 2
    loss_D.backward()
    trainer_D.step()
    return loss_D

def update_G(Z, net_D, net_G, loss, trainer_G):
    """Update generator.

    Defined in :numref:`sec_basic_gan`"""
    batch_size = Z.shape[0]
    ones = torch.ones((batch_size,), device=Z.device)
    trainer_G.zero_grad()
    # We could reuse `fake_X` from `update_D` to save computation
    fake_X = net_G(Z)
    # Recomputing `fake_Y` is needed since `net_D` is changed
    fake_Y = net_D(fake_X)
    loss_G = loss(fake_Y, ones.reshape(fake_Y.shape))
    loss_G.backward()
    trainer_G.step()
    return loss_G

native.DATA_HUB['pokemon'] = (native.DATA_URL + 'pokemon.zip',
                              'c065c0e2593b8b161a2d7873e42418bf6a21106c')

def frozen_lake(seed):
    """Defined in :numref:`sec_utils`"""
    # See https://www.gymnasium.dev/environments/toy\_text/frozen\_lake/ to learn more about this env
    # How to process env.P.items is adapted from https://sites.google.com/view/deep-rl-bootcamp/labs
    import gym

    env = gym.make('FrozenLake-v1', is_slippery=False)
    env.seed(seed)
    env.action_space.seed(seed)
    env.action_space.seed(seed)
    env_info = {}

```

```

env_info['desc'] = env.desc # 2D array specifying what each grid item means
env_info['num_states'] = env.nS # Number of observations/states or obs/state dim
env_info['num_actions'] = env.nA # Number of actions or action dim
# Define indices for (transition probability, nextstate, reward, done) tuple
env_info['trans_prob_idx'] = 0 # Index of transition probability entry
env_info['nextstate_idx'] = 1 # Index of next state entry
env_info['reward_idx'] = 2 # Index of reward entry
env_info['done_idx'] = 3 # Index of done entry
env_info['mdp'] = {}
env_info['env'] = env

for (s, others) in env.P.items():
    # others(s) = {a0: [ (p(s'|s,a0), s', reward, done),...], a1:[...], ...}

    for (a, pxrds) in others.items():
        # pxrds is [(p1,next1,r1,d1),(p2,next2,r2,d2),...].
        # e.g. [(0.3, 0, 0, False), (0.3, 0, 0, False), (0.3, 4, 1, False)]
        env_info['mdp'][(s,a)] = pxrds

return env_info

def make_env(name='', seed=0):
    """Defined in :numref:`sec_utils`"""
    # Input parameters:
    # name: specifies a gym environment.
    # For Value iteration, only FrozenLake-v1 is supported.
    if name == 'FrozenLake-v1':
        return frozen_lake(seed)

    else:
        raise ValueError("%s env is not supported in this Notebook")

def show_value_function_progress(env_desc, V, pi):
    """Defined in :numref:`sec_utils`"""
    # This function visualizes how value and policy changes over time.
    # V: [num_iters, num_states]
    # pi: [num_iters, num_states]
    # How to visualize value function is adapted (but changed) from: https://sites.google.com/view/deep-rl-bootcamp/labs

    num_iters = V.shape[0]
    fig, ax = plt.subplots(figsize=(15, 15))

    for k in range(V.shape[0]):
        plt.subplot(4, 4, k + 1)
        plt.imshow(V[k].reshape(4,4), cmap="bone")
        ax = plt.gca()
        ax.set_xticks(np.arange(0, 5)-.5, minor=True)
        ax.set_yticks(np.arange(0, 5)-.5, minor=True)
        ax.grid(which="minor", color="w", linestyle='-', linewidth=3)
        ax.tick_params(which="minor", bottom=False, left=False)

        ax.set_xticks([])
        ax.set_yticks([])

        # LEFT action: 0, DOWN action: 1
        # RIGHT action: 2, UP action: 3
        action2xdy = {0:(-.25, 0),1: (0, .25),
                      2:(0.25, 0),3: (-.25, 0)}

        for y in range(4):
            for x in range(4):
                action = pi[k].reshape(4,4)[y, x]
                dx, dy = action2xdy[action]

                if env_desc[y,x].decode() == 'H':
                    ax.text(x, y, str(env_desc[y,x].decode()),
                            ha="center", va="center", color="y",
                            size=20, fontweight='bold')

                elif env_desc[y,x].decode() == 'G':
                    ax.text(x, y, str(env_desc[y,x].decode()),
                            ha="center", va="center", color="w",
                            size=20, fontweight='bold')

                else:
                    ax.text(x, y, str(env_desc[y,x].decode()),
                            ha="center", va="center", color="g",

```

```

        size=15, fontweight='bold')

    # No arrow for cells with G and H labels
    if env_desc[y,x].decode() != 'G' and env_desc[y,x].decode() != 'H':
        ax.arrow(x, y, dx, dy, color='r', head_width=0.2, head_length=0.15)

    ax.set_title("Step = " + str(k + 1), fontsize=20)

fig.tight_layout()
plt.show()

def show_Q_function_progress(env_desc, V_all, pi_all):
    """Defined in :numref:`sec_utils`"""
    # This function visualizes how value and policy changes over time.
    # V: [num_iters, num_states]
    # pi: [num_iters, num_states]

    # We want to only shows few values
    num_iters_all = V_all.shape[0]
    num_iters = num_iters_all // 10

    vis_indx = np.arange(0, num_iters_all, num_iters).tolist()
    vis_indx.append(num_iters_all - 1)
    V = np.zeros((len(vis_indx), V_all.shape[1]))
    pi = np.zeros((len(vis_indx), V_all.shape[1]))

    for c, i in enumerate(vis_indx):
        V[c] = V_all[i]
        pi[c] = pi_all[i]

    num_iters = V.shape[0]
    fig, ax = plt.subplots(figsize=(15, 15))

    for k in range(V.shape[0]):
        plt.subplot(4, 4, k + 1)
        plt.imshow(V[k].reshape(4,4), cmap="bone")
        ax = plt.gca()
        ax.set_xticks(np.arange(0, 5)-.5, minor=True)
        ax.set_yticks(np.arange(0, 5)-.5, minor=True)
        ax.grid(which="minor", color="w", linestyle='-', linewidth=3)
        ax.tick_params(which="minor", bottom=False, left=False)
        ax.set_xticks([])
        ax.set_yticks([])

        # LEFT action: 0, DOWN action: 1
        # RIGHT action: 2, UP action: 3
        action2dxdy = {0:(-.25, 0),1:(0, .25),
                       2:(0.25, 0),3:(-.25, 0)}

        for y in range(4):
            for x in range(4):
                action = pi[k].reshape(4,4)[y, x]
                dx, dy = action2dxdy[action]

                if env_desc[y,x].decode() == 'H':
                    ax.text(x, y, str(env_desc[y,x].decode()),
                           ha="center", va="center", color="y",
                           size=20, fontweight='bold')

                elif env_desc[y,x].decode() == 'G':
                    ax.text(x, y, str(env_desc[y,x].decode()),
                           ha="center", va="center", color="w",
                           size=20, fontweight='bold')

                else:
                    ax.text(x, y, str(env_desc[y,x].decode()),
                           ha="center", va="center", color="g",
                           size=15, fontweight='bold')

                # No arrow for cells with G and H labels
                if env_desc[y,x].decode() != 'G' and env_desc[y,x].decode() != 'H':
                    ax.arrow(x, y, dx, dy, color='r', head_width=0.2, head_length=0.15)

        ax.set_title("Step = " + str(vis_indx[k] + 1), fontsize=20)

fig.tight_layout()
plt.show()

```

```

def load_array(data_arrays, batch_size, is_train=True):
    """Construct a PyTorch data iterator.

    Defined in :numref:`sec_utils`"""
    dataset = torch.utils.data.TensorDataset(*data_arrays)
    return torch.utils.data.DataLoader(dataset, batch_size, shuffle=is_train)

def synthetic_data(w, b, num_examples):
    """Generate  $y = Xw + b + \text{noise}$ .

    Defined in :numref:`sec_utils`"""
    X = native.normal(0, 1, (num_examples, len(w)))
    y = native.matmul(X, w) + b
    y += native.normal(0, 0.01, y.shape)
    return X, native.reshape(y, (-1, 1))

def sgd(params, lr, batch_size):
    """Minibatch stochastic gradient descent.

    Defined in :numref:`sec_utils`"""
    with torch.no_grad():
        for param in params:
            param -= lr * param.grad / batch_size
            param.grad.zero_()

def get_dataloader_workers():
    """Use 4 processes to read the data.

    Defined in :numref:`sec_utils`"""
    return 4

def load_data_fashion_mnist(batch_size, resize=None):
    """Download the Fashion-MNIST dataset and then load it into memory.

    Defined in :numref:`sec_utils`"""
    trans = [transforms.ToTensor()]
    if resize:
        trans.insert(0, transforms.Resize(resize))
    trans = transforms.Compose(trans)
    mnist_train = torchvision.datasets.FashionMNIST(
        root="../data", train=True, transform=trans, download=True)
    mnist_test = torchvision.datasets.FashionMNIST(
        root="../data", train=False, transform=trans, download=True)
    return (torch.utils.data.DataLoader(mnist_train, batch_size, shuffle=True,
                                         num_workers=get_dataloader_workers()),
            torch.utils.data.DataLoader(mnist_test, batch_size, shuffle=False,
                                         num_workers=get_dataloader_workers()))

def evaluate_accuracy_gpu(net, data_iter, device=None):
    """Compute the accuracy for a model on a dataset using a GPU.

    Defined in :numref:`sec_utils`"""
    if isinstance(net, nn.Module):
        net.eval() # Set the model to evaluation mode
        if not device:
            device = next(iter(net.parameters())).device
    # No. of correct predictions, no. of predictions
    metric = native.Accumulator(2)

    with torch.no_grad():
        for X, y in data_iter:
            if isinstance(X, list):
                # Required for BERT Fine-tuning (to be covered later)
                X = [x.to(device) for x in X]
            else:
                X = X.to(device)
            y = y.to(device)
            metric.add(native.accuracy(net(X), y), native.size(y))
    return metric[0] / metric[1]

def train_ch6(net, train_iter, test_iter, num_epochs, lr, device):
    """Train a model with a GPU (defined in Chapter 6).

    Defined in :numref:`sec_utils`"""
    def init_weights(m):

```

```

        if type(m) == nn.Linear or type(m) == nn.Conv2d:
            nn.init.xavier_uniform_(m.weight)
    net.apply(init_weights)
    print('training on', device)
    net.to(device)
    optimizer = torch.optim.SGD(net.parameters(), lr=lr)
    loss = nn.CrossEntropyLoss()
    animator = native.Animator(xlabel='epoch', xlim=[1, num_epochs],
                               legend=['train loss', 'train acc', 'test acc'])
    timer, num_batches = native.Timer(), len(train_iter)
    for epoch in range(num_epochs):
        # Sum of training loss, sum of training accuracy, no. of examples
        metric = native.Accumulator(3)
        net.train()
        for i, (X, y) in enumerate(train_iter):
            timer.start()
            optimizer.zero_grad()
            X, y = X.to(device), y.to(device)
            y_hat = net(X)
            l = loss(y_hat, y)
            l.backward()
            optimizer.step()
            with torch.no_grad():
                metric.add(l * X.shape[0], native.accuracy(y_hat, y), X.shape[0])
            timer.stop()
            train_l = metric[0] / metric[2]
            train_acc = metric[1] / metric[2]
            if (i + 1) % (num_batches // 5) == 0 or i == num_batches - 1:
                animator.add(epoch + (i + 1) / num_batches,
                             (train_l, train_acc, None))
            test_acc = evaluate_accuracy_gpu(net, test_iter)
            animator.add(epoch + 1, (None, None, test_acc))
    print(f'loss {train_l:.3f}, train acc {train_acc:.3f}, '
          f'test acc {test_acc:.3f}')
    print(f'{metric[2] * num_epochs / timer.sum():.1f} examples/sec '
          f'on {str(device)}')

def show_images(imgs, num_rows, num_cols, titles=None, scale=1.5):
    """Plot a list of images.

    Defined in :numref:`sec_utils`"""
    figsize = (num_cols * scale, num_rows * scale)
    _, axes = native.plt.subplots(num_rows, num_cols, figsize=figsize)
    axes = axes.flatten()
    for i, (ax, img) in enumerate(zip(axes, imgs)):
        try:
            img = native.numpy(img)
        except:
            pass
        ax.imshow(img)
        ax.axes.get_xaxis().set_visible(False)
        ax.axes.get_yaxis().set_visible(False)
        if titles:
            ax.set_title(titles[i])
    return axes

def linreg(X, w, b):
    """The linear regression model.

    Defined in :numref:`sec_utils`"""
    return native.matmul(X, w) + b

def squared_loss(y_hat, y):
    """Squared loss.

    Defined in :numref:`sec_utils`"""
    return (y_hat - native.reshape(y, y_hat.shape)) ** 2 / 2

def get_fashion_mnist_labels(labels):
    """Return text labels for the Fashion-MNIST dataset.

    Defined in :numref:`sec_utils`"""
    text_labels = ['t-shirt', 'trouser', 'pullover', 'dress', 'coat',
                   'sandal', 'shirt', 'sneaker', 'bag', 'ankle boot']
    return [text_labels[int(i)] for i in labels]

```

```

class Animator:
    """For plotting data in animation."""
    def __init__(self, xlabel=None, ylabel=None, legend=None, xlim=None,
                  ylim=None, xscale='linear', yscale='linear',
                  fmts=('-', 'm--', 'g-', 'r:'), nrows=1, ncols=1,
                  figsize=(3.5, 2.5)):
        """Defined in :numref:`sec_utils`"""
        # Incrementally plot multiple lines
        if legend is None:
            legend = []
        native.use_svg_display()
        self.fig, self.axes = native.plt.subplots(nrows, ncols, figsize=figsize)
        if nrows * ncols == 1:
            self.axes = [self.axes, ]
        # Use a lambda function to capture arguments
        self.config_axes = lambda: native.set_axes(
            self.axes[0], xlabel, ylabel, xlim, ylim, xscale, yscale, legend)
        self.X, self.Y, self.fmts = None, None, fmts

    def add(self, x, y):
        # Add multiple data points into the figure
        if not hasattr(y, "__len__"):
            y = [y]
        n = len(y)
        if not hasattr(x, "__len__"):
            x = [x] * n
        if not self.X:
            self.X = [[] for _ in range(n)]
        if not self.Y:
            self.Y = [[] for _ in range(n)]
        for i, (a, b) in enumerate(zip(x, y)):
            if a is not None and b is not None:
                self.X[i].append(a)
                self.Y[i].append(b)
        self.axes[0].cla()
        for x, y, fmt in zip(self.X, self.Y, self.fmts):
            self.axes[0].plot(x, y, fmt)
        self.config_axes()
        display.display(self.fig)
        display.clear_output(wait=True)

class Accumulator:
    """For accumulating sums over `n` variables."""
    def __init__(self, n):
        """Defined in :numref:`sec_utils`"""
        self.data = [0.0] * n

    def add(self, *args):
        self.data = [a + float(b) for a, b in zip(self.data, args)]

    def reset(self):
        self.data = [0.0] * len(self.data)

    def __getitem__(self, idx):
        return self.data[idx]

def accuracy(y_hat, y):
    """Compute the number of correct predictions.

    Defined in :numref:`sec_utils`"""
    if len(y_hat.shape) > 1 and y_hat.shape[1] > 1:
        y_hat = native.argmax(y_hat, axis=1)
    cmp = native.astype(y_hat, y.dtype) == y
    return float(native.reduce_sum(native.astype(cmp, y.dtype)))

def download(url, folder='./data', sha1_hash=None):
    """Download a file to folder and return the local filepath.

    Defined in :numref:`sec_utils`"""
    if not url.startswith('http'):
        # For back compatability
        url, sha1_hash = DATA_HUB[url]
    os.makedirs(folder, exist_ok=True)
    fname = os.path.join(folder, url.split('/')[-1])
    # Check if hit cache
    if os.path.exists(fname) and sha1_hash:

```

```

        sha1 = hashlib.sha1()
        with open(fname, 'rb') as f:
            while True:
                data = f.read(1048576)
                if not data:
                    break
                sha1.update(data)
            if sha1.hexdigest() == sha1_hash:
                return fname
# Download
print(f'Downloading {fname} from {url}...')
r = requests.get(url, stream=True, verify=True)
with open(fname, 'wb') as f:
    f.write(r.content)
return fname

def extract(filename, folder=None):
    """Extract a zip/tar file into folder.

    Defined in :numref:`sec_utils`"""
    base_dir = os.path.dirname(filename)
    _, ext = os.path.splitext(filename)
    assert ext in ('.zip', '.tar', '.gz'), 'Only support zip/tar files.'
    if ext == '.zip':
        fp = zipfile.ZipFile(filename, 'r')
    else:
        fp = tarfile.open(filename, 'r')

    if folder is None:
        folder = base_dir
    fp.extractall(folder)

def download_extract(name, folder=None):
    """Download and extract a zip/tar file.

    Defined in :numref:`sec_utils`"""
    fname = download(name)
    base_dir = os.path.dirname(fname)
    data_dir, ext = os.path.splitext(fname)
    if ext == '.zip':
        fp = zipfile.ZipFile(fname, 'r')
    elif ext in ('.tar', '.gz'):
        fp = tarfile.open(fname, 'r')
    else:
        assert False, 'Only zip/tar files can be extracted.'
    fp.extractall(base_dir)
    return os.path.join(base_dir, folder) if folder else data_dir

def tokenize(lines, token='word'):
    """Split text lines into word or character tokens.

    Defined in :numref:`sec_utils`"""
    assert token in ('word', 'char'), 'Unknown token type: ' + token
    return [line.split() if token == 'word' else list(line) for line in lines]

def evaluate_loss(net, data_iter, loss):
    """Evaluate the loss of a model on the given dataset.

    Defined in :numref:`sec_utils`"""
    metric = native.Accumulator(2) # Sum of losses, no. of examples
    for X, y in data_iter:
        out = net(X)
        y = native.reshape(y, out.shape)
        l = loss(out, y)
        metric.add(native.reduce_sum(l), native.size(l))
    return metric[0] / metric[1]

def grad_clipping(net, theta):
    """Clip the gradient.

    Defined in :numref:`sec_utils`"""
    if isinstance(net, nn.Module):
        params = [p for p in net.parameters() if p.requires_grad]
    else:
        params = net.params
    norm = torch.sqrt(sum(torch.sum((p.grad ** 2)) for p in params))
    if norm > theta:

```

```

        for param in params:
            param.grad[:] *= theta / norm

native.DATA_HUB['fra-eng'] = (native.DATA_URL + 'fra-eng.zip',
                              '94646ad1522d915e7b0f9296181140edcf86a4f5')

def read_data_nmt():
    """Load the English-French dataset.

    Defined in :numref:`sec_utils`"""
    data_dir = native.download_extract('fra-eng')
    with open(os.path.join(data_dir, 'fra.txt'), 'r', encoding='utf-8') as f:
        return f.read()

def preprocess_nmt(text):
    """Preprocess the English-French dataset.

    Defined in :numref:`sec_utils`"""
    def no_space(char, prev_char):
        return char in set(',.!?') and prev_char != ' '

    # Replace non-breaking space with space, and convert uppercase letters to
    # lowercase ones
    text = text.replace('\u202f', ' ').replace('\xa0', ' ').lower()
    # Insert space between words and punctuation marks
    out = [' ' + char if i > 0 and no_space(char, text[i - 1]) else char
            for i, char in enumerate(text)]
    return ''.join(out)

def tokenize_nmt(text, num_examples=None):
    """Tokenize the English-French dataset.

    Defined in :numref:`sec_utils`"""
    source, target = [], []
    for i, line in enumerate(text.split('\n')):
        if num_examples and i > num_examples:
            break
        parts = line.split('\t')
        if len(parts) == 2:
            source.append(parts[0].split(' '))
            target.append(parts[1].split(' '))
    return source, target

def truncate_pad(line, num_steps, padding_token):
    """Truncate or pad sequences.

    Defined in :numref:`sec_utils`"""
    if len(line) > num_steps:
        return line[:num_steps] # Truncate
    return line + [padding_token] * (num_steps - len(line)) # Pad

def build_array_nmt(lines, vocab, num_steps):
    """Transform text sequences of machine translation into minibatches.

    Defined in :numref:`sec_utils`"""
    lines = [vocab[l] for l in lines]
    lines = [l + [vocab['<eos>']] for l in lines]
    array = native.tensor([truncate_pad(
        l, num_steps, vocab['<pad>']) for l in lines])
    valid_len = native.reduce_sum(
        native.astype(array != vocab['<pad>'], native.int32), 1)
    return array, valid_len

def load_data_nmt(batch_size, num_steps, num_examples=600):
    """Return the iterator and the vocabularies of the translation dataset.

    Defined in :numref:`sec_utils`"""
    text = preprocess_nmt(read_data_nmt())
    source, target = tokenize_nmt(text, num_examples)
    src_vocab = native.Vocab(source, min_freq=2,
                             reserved_tokens=['<pad>', '<bos>', '<eos>'])
    tgt_vocab = native.Vocab(target, min_freq=2,
                              reserved_tokens=['<pad>', '<bos>', '<eos>'])

```

```

src_array, src_valid_len = build_array_nmt(source, src_vocab, num_steps)
tgt_array, tgt_valid_len = build_array_nmt(target, tgt_vocab, num_steps)
data_arrays = (src_array, src_valid_len, tgt_array, tgt_valid_len)
data_iter = native.load_array(data_arrays, batch_size)
return data_iter, src_vocab, tgt_vocab

def sequence_mask(X, valid_len, value=0):
    """Mask irrelevant entries in sequences.

    Defined in :numref:`sec_utils`"""
    maxlen = X.size(1)
    mask = torch.arange((maxlen), dtype=torch.float32,
                        device=X.device)[None, :] < valid_len[:, None]
    X[~mask] = value
    return X

class MaskedSoftmaxCELoss(nn.CrossEntropyLoss):
    """The softmax cross-entropy loss with masks.

    Defined in :numref:`sec_utils`"""
    # `pred` shape: (`batch_size`, `num_steps`, `vocab_size`)
    # `label` shape: (`batch_size`, `num_steps`)
    # `valid_len` shape: (`batch_size`,)
    def forward(self, pred, label, valid_len):
        weights = torch.ones_like(label)
        weights = sequence_mask(weights, valid_len)
        self.reduction='none'
        unweighted_loss = super(MaskedSoftmaxCELoss, self).forward(
            pred.permute(0, 2, 1), label)
        weighted_loss = (unweighted_loss * weights).mean(dim=1)
        return weighted_loss

def train_seq2seq(net, data_iter, lr, num_epochs, tgt_vocab, device):
    """Train a model for sequence to sequence.

    Defined in :numref:`sec_utils`"""
    def xavier_init_weights(m):
        if type(m) == nn.Linear:
            nn.init.xavier_uniform_(m.weight)
        if type(m) == nn.GRU:
            for param in m._flat_weights_names:
                if "weight" in param:
                    nn.init.xavier_uniform_(m._parameters[param])
    net.apply(xavier_init_weights)
    net.to(device)
    optimizer = torch.optim.Adam(net.parameters(), lr=lr)
    loss = MaskedSoftmaxCELoss()
    net.train()
    animator = native.Animator(xlabel='epoch', ylabel='loss',
                              xlim=[10, num_epochs])
    for epoch in range(num_epochs):
        timer = native.Timer()
        metric = native.Accumulator(2) # Sum of training loss, no. of tokens

        for batch in data_iter:
            optimizer.zero_grad()
            X, X_valid_len, Y, Y_valid_len = [x.to(device) for x in batch]
            bos = torch.tensor([tgt_vocab['<bos>']] * Y.shape[0],
                               device=device).reshape(-1, 1)
            dec_input = native.concat([bos, Y[:, :-1]], 1) # Teacher forcing
            Y_hat, _ = net(X, dec_input, X_valid_len)
            l = loss(Y_hat, Y, Y_valid_len)
            l.sum().backward() # Make the loss scalar for `backward`
            native.grad_clipping(net, 1)
            num_tokens = Y_valid_len.sum()
            optimizer.step()
            with torch.no_grad():
                metric.add(l.sum(), num_tokens)
        if (epoch + 1) % 10 == 0:
            animator.add(epoch + 1, (metric[0] / metric[1],))
    print(f'loss {metric[0] / metric[1]:.3f}, {metric[1] / timer.stop():.1f} '
          f'tokens/sec on {str(device)}')

def predict_seq2seq(net, src_sentence, src_vocab, tgt_vocab, num_steps,
                    device, save_attention_weights=False):
    """Predict for sequence to sequence.

```

```

Defined in :numref:`sec_utils`"""
# Set `net` to eval mode for inference
net.eval()
src_tokens = src_vocab[src_sentence.lower().split(' ')] + [
    src_vocab['<eos>']]
enc_valid_len = torch.tensor([len(src_tokens)], device=device)
src_tokens = native.truncate_pad(src_tokens, num_steps, src_vocab['<pad>'])
# Add the batch axis
enc_X = torch.unsqueeze(
    torch.tensor(src_tokens, dtype=torch.long, device=device), dim=0)
enc_outputs = net.encoder(enc_X, enc_valid_len)
dec_state = net.decoder.init_state(enc_outputs, enc_valid_len)
# Add the batch axis
dec_X = torch.unsqueeze(torch.tensor(
    [tgt_vocab['<bos>']], dtype=torch.long, device=device), dim=0)
output_seq, attention_weight_seq = [], []
for _ in range(num_steps):
    Y, dec_state = net.decoder(dec_X, dec_state)
    # We use the token with the highest prediction likelihood as input
    # of the decoder at the next time step
    dec_X = Y.argmax(dim=2)
    pred = dec_X.squeeze(dim=0).type(torch.int32).item()
    # Save attention weights (to be covered later)
    if save_attention_weights:
        attention_weight_seq.append(net.decoder.attention_weights)
    # Once the end-of-sequence token is predicted, the generation of the
    # output sequence is complete
    if pred == tgt_vocab['<eos>']:
        break
    output_seq.append(pred)
return ' '.join(tgt_vocab.to_tokens(output_seq)), attention_weight_seq

```

```

# Alias defined in config.ini
nn_Module = nn.Module

```

```

ones_like = torch.ones_like
ones = torch.ones
zeros_like = torch.zeros_like
zeros = torch.zeros
tensor = torch.tensor
arange = torch.arange
meshgrid = torch.meshgrid
sin = torch.sin
sinh = torch.sinh
cos = torch.cos
cosh = torch.cosh
tanh = torch.tanh
linspace = torch.linspace
exp = torch.exp
log = torch.log
normal = torch.normal
rand = torch.rand
randn = torch.randn
matmul = torch.matmul
int32 = torch.int32
int64 = torch.int64
float32 = torch.float32
concat = torch.cat
stack = torch.stack
abs = torch.abs
eye = torch.eye
sigmoid = torch.sigmoid
batch_matmul = torch.bmm
numpy = lambda x, *args, **kwargs: x.detach().numpy(*args, **kwargs)
size = lambda x, *args, **kwargs: x.numel(*args, **kwargs)
reshape = lambda x, *args, **kwargs: x.reshape(*args, **kwargs)
to = lambda x, *args, **kwargs: x.to(*args, **kwargs)
reduce_sum = lambda x, *args, **kwargs: x.sum(*args, **kwargs)
argmax = lambda x, *args, **kwargs: x.argmax(*args, **kwargs)
astype = lambda x, *args, **kwargs: x.type(*args, **kwargs)
transpose = lambda x, *args, **kwargs: x.t(*args, **kwargs)
reduce_mean = lambda x, *args, **kwargs: x.mean(*args, **kwargs)
expand_dims = lambda x, *args, **kwargs: x.unsqueeze(*args, **kwargs)
swapaxes = lambda x, *args, **kwargs: x.swapaxes(*args, **kwargs)
repeat = lambda x, *args, **kwargs: x.repeat(*args, **kwargs)

```

```
# Topic-specific implementations originally supplied by the book are kept here
# as ordinary PyTorch helpers so every section can import them directly.
```

```
# From chapter_03_linear_networks/image_classification_dataset.py
def get_fashion_mnist_labels(labels): #@save
    """返回Fashion-MNIST数据集的文本标签"""
    text_labels = ['t-shirt', 'trouser', 'pullover', 'dress', 'coat',
                   'sandal', 'shirt', 'sneaker', 'bag', 'ankle boot']
    return [text_labels[int(i)] for i in labels]
```

```
# From chapter_03_linear_networks/image_classification_dataset.py
def show_images(imgs, num_rows, num_cols, titles=None, scale=1.5): #@save
    """绘制图像列表"""
    figsize = (num_cols * scale, num_rows * scale)
    _, axes = native.plt.subplots(num_rows, num_cols, figsize=figsize)
    axes = axes.flatten()
```

```
    for i, (ax, img) in enumerate(zip(axes, imgs)):
        if torch.is_tensor(img):
            # 图片张量
            ax.imshow(img.numpy())
        else:
            # PIL图片
            ax.imshow(img)
            ax.axes.get_xaxis().set_visible(False)
            ax.axes.get_yaxis().set_visible(False)
        if titles:
            ax.set_title(titles[i])
    return axes
```

```
# From chapter_03_linear_networks/image_classification_dataset.py
def get_dataloader_workers(): #@save
    """使用4个进程来读取数据"""
    return 4
```

```
# From chapter_03_linear_networks/image_classification_dataset.py
def load_data_fashion_mnist(batch_size, resize=None): #@save
    """下载Fashion-MNIST数据集，然后将其加载到内存中"""
    trans = [transforms.ToTensor()]
    if resize:
        trans.insert(0, transforms.Resize(resize))
    trans = transforms.Compose(trans)
    mnist_train = torchvision.datasets.FashionMNIST(
        root="../data", train=True, transform=trans, download=True)
    mnist_test = torchvision.datasets.FashionMNIST(
        root="../data", train=False, transform=trans, download=True)
    return (data.DataLoader(mnist_train, batch_size, shuffle=True,
                            num_workers=get_dataloader_workers()),
            data.DataLoader(mnist_test, batch_size, shuffle=False,
                            num_workers=get_dataloader_workers()))
```

```
# From chapter_03_linear_networks/linear_regression.py
class Timer: #@save
    """记录多次运行时间"""
    def __init__(self):
        self.times = []
        self.start()

    def start(self):
        """启动计时器"""
        self.tik = time.time()

    def stop(self):
        """停止计时器并将时间记录在列表中"""
        self.times.append(time.time() - self.tik)
        return self.times[-1]

    def avg(self):
        """返回平均时间"""
        return sum(self.times) / len(self.times)

    def sum(self):
        """返回时间总和"""
        return sum(self.times)
```

```

def cumsum(self):
    """返回累计时间"""
    return np.array(self.times).cumsum().tolist()

# From chapter_03_linear_networks/linear_regression.py
def normal(x, mu, sigma):
    p = 1 / math.sqrt(2 * math.pi * sigma**2)
    return p * np.exp(-0.5 / sigma**2 * (x - mu)**2)

# From chapter_03_linear_networks/linear_regression_concise.py
def load_array(data_arrays, batch_size, is_train=True): #@save
    """构造一个PyTorch数据迭代器"""
    dataset = data.TensorDataset(*data_arrays)
    return data.DataLoader(dataset, batch_size, shuffle=is_train)

# From chapter_03_linear_networks/linear_regression_scratch.py
def synthetic_data(w, b, num_examples): #@save
    """生成y=Xw+b+噪声"""
    X = native.normal(0, 1, (num_examples, len(w)))
    y = native.matmul(X, w) + b
    y += native.normal(0, 0.01, y.shape)
    return X, native.reshape(y, (-1, 1))

# From chapter_03_linear_networks/linear_regression_scratch.py
def data_iter(batch_size, features, labels):
    num_examples = len(features)
    indices = list(range(num_examples))
    # 这些样本是随机读取的，没有特定的顺序
    random.shuffle(indices)
    for i in range(0, num_examples, batch_size):
        batch_indices = native.tensor(
            indices[i: min(i + batch_size, num_examples)])
        yield features[batch_indices], labels[batch_indices]

# From chapter_03_linear_networks/linear_regression_scratch.py
def linreg(X, w, b): #@save
    """线性回归模型"""
    return native.matmul(X, w) + b

# From chapter_03_linear_networks/linear_regression_scratch.py
def squared_loss(y_hat, y): #@save
    """均方损失"""
    return (y_hat - native.reshape(y, y_hat.shape)) ** 2 / 2

# From chapter_03_linear_networks/linear_regression_scratch.py
def sgd(params, lr, batch_size): #@save
    """小批量随机梯度下降"""
    with torch.no_grad():
        for param in params:
            param -= lr * param.grad / batch_size
            param.grad.zero_()

# From chapter_03_linear_networks/softmax_regression_concise.py
def init_weights(m):
    if type(m) == nn.Linear:
        nn.init.normal_(m.weight, std=0.01)

# From chapter_03_linear_networks/softmax_regression_scratch.py
def softmax(X):
    X_exp = native.exp(X)
    partition = native.reduce_sum(X_exp, 1, keepdim=True)
    return X_exp / partition

# From chapter_03_linear_networks/softmax_regression_scratch.py
def net(X):
    return softmax(native.matmul(native.reshape(X, (-1, W.shape[0])), W) + b)

# From chapter_03_linear_networks/softmax_regression_scratch.py
def cross_entropy(y_hat, y):
    return - native.log(y_hat[range(len(y_hat)), y])

# From chapter_03_linear_networks/softmax_regression_scratch.py
def accuracy(y_hat, y): #@save
    """计算预测正确的数量"""
    if len(y_hat.shape) > 1 and y_hat.shape[1] > 1:

```

```

        y_hat = native.argmax(y_hat, axis=1)
        cmp = native.astype(y_hat, y.dtype) == y
        return float(native.reduce_sum(native.astype(cmp, y.dtype)))

# From chapter_03_linear_networks/softmax_regression_scratch.py
def evaluate_accuracy(net, data_iter): #@save
    """计算在指定数据集上模型的精度"""
    if isinstance(net, torch.nn.Module):
        net.eval() # 将模型设置为评估模式
    metric = Accumulator(2) # 正确预测数、预测总数
    with torch.no_grad():
        for X, y in data_iter:
            metric.add(accuracy(net(X), y), native.size(y))
    return metric[0] / metric[1]

# From chapter_03_linear_networks/softmax_regression_scratch.py
class Accumulator: #@save
    """在n个变量上累加"""
    def __init__(self, n):
        self.data = [0.0] * n

    def add(self, *args):
        self.data = [a + float(b) for a, b in zip(self.data, args)]

    def reset(self):
        self.data = [0.0] * len(self.data)

    def __getitem__(self, idx):
        return self.data[idx]

# From chapter_03_linear_networks/softmax_regression_scratch.py
def train_epoch_ch3(net, train_iter, loss, updater): #@save
    """训练模型一个迭代周期（定义见第3章）"""
    # 将模型设置为训练模式
    if isinstance(net, torch.nn.Module):
        net.train()
    # 训练损失总和、训练准确度总和、样本数
    metric = Accumulator(3)
    for X, y in train_iter:
        # 计算梯度并更新参数
        y_hat = net(X)
        l = loss(y_hat, y)
        if isinstance(updater, torch.optim.Optimizer):
            # 使用PyTorch内置的优化器和损失函数
            updater.zero_grad()
            l.mean().backward()
            updater.step()
        else:
            # 使用定制的优化器和损失函数
            l.sum().backward()
            updater(X.shape[0])
        metric.add(float(l.sum()), accuracy(y_hat, y), y.numel())
    # 返回训练损失和训练精度
    return metric[0] / metric[2], metric[1] / metric[2]

# From chapter_03_linear_networks/softmax_regression_scratch.py
class Animator: #@save
    """在动画中绘制数据"""
    def __init__(self, xlabel=None, ylabel=None, legend=None, xlim=None,
                 ylim=None, xscale='linear', yscale='linear',
                 fmts=('-', 'm--', 'g-', 'r:'), nrows=1, ncols=1,
                 figsize=(3.5, 2.5)):
        # 增量地绘制多条线
        if legend is None:
            legend = []
        native.use_svg_display()
        self.fig, self.axes = native.plt.subplots(nrows, ncols, figsize=figsize)
        if nrows * ncols == 1:
            self.axes = [self.axes, ]
        # 使用lambda函数捕获参数
        self.config_axes = lambda: native.set_axes(
            self.axes[0], xlabel, ylabel, xlim, ylim, xscale, yscale, legend)
        self.X, self.Y, self.fmts = None, None, fmts

    def add(self, x, y):
        # 向图表中添加多个数据点
        if not hasattr(y, "__len__"):

```

```

        y = [y]
        n = len(y)
        if not hasattr(x, "__len__"):
            x = [x] * n
        if not self.X:
            self.X = [[] for _ in range(n)]
        if not self.Y:
            self.Y = [[] for _ in range(n)]
        for i, (a, b) in enumerate(zip(x, y)):
            if a is not None and b is not None:
                self.X[i].append(a)
                self.Y[i].append(b)
        self.axes[0].cla()
        for x, y, fmt in zip(self.X, self.Y, self.fmts):
            self.axes[0].plot(x, y, fmt)
        self.config_axes()
        display.display(self.fig)
        display.clear_output(wait=True)

# From chapter_03_linear_networks/softmax_regression_scratch.py
def train_ch3(net, train_iter, test_iter, loss, num_epochs, updater): #@save
    """训练模型（定义见第3章）"""
    animator = Animator(xlabel='epoch', xlim=[1, num_epochs], ylim=[0.3, 0.9],
                        legend=['train loss', 'train acc', 'test acc'])
    for epoch in range(num_epochs):
        train_metrics = train_epoch_ch3(net, train_iter, loss, updater)
        test_acc = evaluate_accuracy(net, test_iter)
        animator.add(epoch + 1, train_metrics + (test_acc,))
    train_loss, train_acc = train_metrics
    assert train_loss < 0.5, train_loss

    assert train_acc <= 1 and train_acc > 0.7, train_acc
    assert test_acc <= 1 and test_acc > 0.7, test_acc

# From chapter_03_linear_networks/softmax_regression_scratch.py
def updater(batch_size):
    return native.sgd([W, b], lr, batch_size)

# From chapter_03_linear_networks/softmax_regression_scratch.py
def predict_ch3(net, test_iter, n=6): #@save
    """预测标签（定义见第3章）"""
    for X, y in test_iter:
        break
    trues = native.get_fashion_mnist_labels(y)
    preds = native.get_fashion_mnist_labels(native.argmax(net(X), axis=1))
    titles = [true + '\n' + pred for true, pred in zip(trues, preds)]
    native.show_images(
        native.reshape(X[0:n], (n, 28, 28)), 1, n, titles=titles[0:n])

# From chapter_04_multilayer_perceptrons/dropout.py
def dropout_layer(X, dropout):
    assert 0 <= dropout <= 1
    # 在本情况中，所有元素都被丢弃
    if dropout == 1:
        return torch.zeros_like(X)
    # 在本情况中，所有元素都被保留
    if dropout == 0:
        return X
    mask = (torch.rand(X.shape) > dropout).float()
    return mask * X / (1.0 - dropout)

# From chapter_04_multilayer_perceptrons/dropout.py
class Net(nn.Module):
    def __init__(self, num_inputs, num_outputs, num_hiddens1, num_hiddens2,
                 is_training = True):
        super(Net, self).__init__()
        self.num_inputs = num_inputs
        self.training = is_training
        self.lin1 = nn.Linear(num_inputs, num_hiddens1)
        self.lin2 = nn.Linear(num_hiddens1, num_hiddens2)
        self.lin3 = nn.Linear(num_hiddens2, num_outputs)
        self.relu = nn.ReLU()

    def forward(self, X):
        H1 = self.relu(self.lin1(X.reshape((-1, self.num_inputs))))
        # 只有在训练模型时才使用dropout
        if self.training == True:
            # 在第一个全连接层之后添加一个dropout层

```

```

        H1 = dropout_layer(H1, dropout1)
        H2 = self.relu(self.lin2(H1))
        if self.training == True:
            # 在第二个全连接层之后添加一个dropout层
            H2 = dropout_layer(H2, dropout2)
        out = self.lin3(H2)
        return out

# From chapter_04_multilayer_perceptrons/dropout.py
def init_weights(m):
    if type(m) == nn.Linear:
        nn.init.normal_(m.weight, std=0.01)

# From chapter_04_multilayer_perceptrons/kaggle_house_price.py
def download(name, cache_dir=os.path.join('..', 'data')): #@save
    """下载一个DATA_HUB中的文件, 返回本地文件名"""
    assert name in DATA_HUB, f'{name} 不存在于 {DATA_HUB}'
    url, sha1_hash = DATA_HUB[name]
    os.makedirs(cache_dir, exist_ok=True)
    fname = os.path.join(cache_dir, url.split('/')[-1])
    if os.path.exists(fname):
        sha1 = hashlib.sha1()
        with open(fname, 'rb') as f:
            while True:
                data = f.read(1048576)
                if not data:
                    break
                sha1.update(data)
        if sha1.hexdigest() == sha1_hash:
            return fname # 命中缓存
    print(f'正在从{url}下载{fname}...')
    r = requests.get(url, stream=True, verify=True)
    with open(fname, 'wb') as f:
        f.write(r.content)
    return fname

# From chapter_04_multilayer_perceptrons/kaggle_house_price.py
def download_extract(name, folder=None): #@save
    """下载并解压zip/tar文件"""
    fname = download(name)
    base_dir = os.path.dirname(fname)
    data_dir, ext = os.path.splitext(fname)
    if ext == '.zip':
        fp = zipfile.ZipFile(fname, 'r')
    elif ext in ('.tar', '.gz'):
        fp = tarfile.open(fname, 'r')
    else:
        assert False, '只有zip/tar文件可以被解压缩'
    fp.extractall(base_dir)
    return os.path.join(base_dir, folder) if folder else data_dir

# From chapter_04_multilayer_perceptrons/kaggle_house_price.py
def download_all(): #@save
    """下载DATA_HUB中的所有文件"""
    for name in DATA_HUB:
        download(name)

# From chapter_04_multilayer_perceptrons/kaggle_house_price.py
def get_net():
    net = nn.Sequential(nn.Linear(in_features, 1))
    return net

# From chapter_04_multilayer_perceptrons/kaggle_house_price.py
def log_rmse(net, features, labels):
    # 为了在取对数时进一步稳定该值, 将小于1的值设置为1
    clipped_preds = torch.clamp(net(features), 1, float('inf'))
    rmse = torch.sqrt(loss(torch.log(clipped_preds),
                           torch.log(labels)))
    return rmse.item()

# From chapter_04_multilayer_perceptrons/kaggle_house_price.py
def train(net, train_features, train_labels, test_features, test_labels,
          num_epochs, learning_rate, weight_decay, batch_size):
    train_ls, test_ls = [], []
    train_iter = native.load_array((train_features, train_labels), batch_size)
    # 这里使用的是Adam优化算法

```

```

optimizer = torch.optim.Adam(net.parameters(),
                              lr = learning_rate,
                              weight_decay = weight_decay)
for epoch in range(num_epochs):
    for X, y in train_iter:
        optimizer.zero_grad()
        l = loss(net(X), y)
        l.backward()
        optimizer.step()
    train_ls.append(log_rmse(net, train_features, train_labels))
    if test_labels is not None:
        test_ls.append(log_rmse(net, test_features, test_labels))
return train_ls, test_ls

# From chapter_04_multilayer_perceptrons/kaggle_house_price.py
def get_k_fold_data(k, i, X, y):
    assert k > 1
    fold_size = X.shape[0] // k
    X_train, y_train = None, None
    for j in range(k):
        idx = slice(j * fold_size, (j + 1) * fold_size)
        X_part, y_part = X[idx, :], y[idx]
        if j == i:
            X_valid, y_valid = X_part, y_part
        elif X_train is None:
            X_train, y_train = X_part, y_part
        else:
            X_train = native.concat([X_train, X_part], 0)
            y_train = native.concat([y_train, y_part], 0)
    return X_train, y_train, X_valid, y_valid

# From chapter_04_multilayer_perceptrons/kaggle_house_price.py
def k_fold(k, X_train, y_train, num_epochs, learning_rate, weight_decay,
          batch_size):
    train_l_sum, valid_l_sum = 0, 0
    for i in range(k):
        data = get_k_fold_data(k, i, X_train, y_train)
        net = get_net()
        train_ls, valid_ls = train(net, *data, num_epochs, learning_rate,
                                   weight_decay, batch_size)
        train_l_sum += train_ls[-1]
        valid_l_sum += valid_ls[-1]
        if i == 0:
            native.plot(list(range(1, num_epochs + 1)), [train_ls, valid_ls],
                        xlabel='epoch', ylabel='rmse', xlim=[1, num_epochs],
                        legend=['train', 'valid'], yscale='log')
    print(f'折{i + 1}, 训练log rmse{float(train_ls[-1]):f}, '
          f'验证log rmse{float(valid_ls[-1]):f}')
    return train_l_sum / k, valid_l_sum / k

# From chapter_04_multilayer_perceptrons/kaggle_house_price.py
def train_and_pred(train_features, test_features, train_labels, test_data,
                  num_epochs, lr, weight_decay, batch_size):
    net = get_net()
    train_ls, _ = train(net, train_features, train_labels, None, None,
                        num_epochs, lr, weight_decay, batch_size)
    native.plot(np.arange(1, num_epochs + 1), [train_ls], xlabel='epoch',
                ylabel='log rmse', xlim=[1, num_epochs], yscale='log')
    print(f'训练log rmse: {float(train_ls[-1]):f}')
    # 将网络应用于测试集。
    preds = native.numpy(net(test_features))
    # 将其重新格式化以导出到Kaggle
    test_data['SalePrice'] = pd.Series(preds.reshape(1, -1)[0])
    submission = pd.concat([test_data['Id'], test_data['SalePrice']], axis=1)
    submission.to_csv('submission.csv', index=False)

# From chapter_04_multilayer_perceptrons/mlp_concise.py
def init_weights(m):
    if type(m) == nn.Linear:
        nn.init.normal_(m.weight, std=0.01)

# From chapter_04_multilayer_perceptrons/mlp_scratch.py
def relu(X):
    a = torch.zeros_like(X)
    return torch.max(X, a)

# From chapter_04_multilayer_perceptrons/mlp_scratch.py

```

```

def net(X):
    X = native.reshape(X, (-1, num_inputs))
    H = relu(X@W1 + b1) # 这里"@"代表矩阵乘法
    return (H@W2 + b2)

# From chapter_04_multilayer_perceptrons/underfit_overfit.py
def evaluate_loss(net, data_iter, loss): # @save
    """评估给定数据集上模型的损失"""
    metric = native.Accumulator(2) # 损失的总和,样本数量
    for X, y in data_iter:
        out = net(X)
        y = native.reshape(y, out.shape)
        l = loss(out, y)
        metric.add(native.reduce_sum(l), native.size(l))
    return metric[0] / metric[1]

# From chapter_04_multilayer_perceptrons/underfit_overfit.py
def train(train_features, test_features, train_labels, test_labels,
          num_epochs=400):
    loss = nn.MSELoss(reduction='none')
    input_shape = train_features.shape[-1]
    # 不设置偏置, 因为我们已经在多项式中实现了它
    net = nn.Sequential(nn.Linear(input_shape, 1, bias=False))
    batch_size = min(10, train_labels.shape[0])
    train_iter = native.load_array((train_features, train_labels.reshape(-1,1)),
                                   batch_size)
    test_iter = native.load_array((test_features, test_labels.reshape(-1,1)),
                                   batch_size, is_train=False)
    trainer = torch.optim.SGD(net.parameters(), lr=0.01)
    animator = native.Animator(xlabel='epoch', ylabel='loss', yscale='log',
                               xlim=[1, num_epochs], ylim=[1e-3, 1e2],
                               legend=['train', 'test'])
    for epoch in range(num_epochs):
        native.train_epoch_ch3(net, train_iter, loss, trainer)
        if epoch == 0 or (epoch + 1) % 20 == 0:
            animator.add(epoch + 1, (evaluate_loss(net, train_iter, loss),
                                     evaluate_loss(net, test_iter, loss)))
    print('weight:', net[0].weight.data.numpy())

# From chapter_04_multilayer_perceptrons/weight_decay.py
def init_params():
    w = torch.normal(0, 1, size=(num_inputs, 1), requires_grad=True)
    b = torch.zeros(1, requires_grad=True)
    return [w, b]

# From chapter_04_multilayer_perceptrons/weight_decay.py
def l2_penalty(w):
    return torch.sum(w.pow(2)) / 2

# From chapter_04_multilayer_perceptrons/weight_decay.py
def train(lambd):
    w, b = init_params()
    net, loss = lambda X: native.linreg(X, w, b), native.squared_loss
    num_epochs, lr = 100, 0.003
    animator = native.Animator(xlabel='epochs', ylabel='loss', yscale='log',
                               xlim=[5, num_epochs], legend=['train', 'test'])
    for epoch in range(num_epochs):
        for X, y in train_iter:
            # 增加了L2范数惩罚项,
            # 广播机制使l2_penalty(w)成为一个长度为batch_size的向量
            l = loss(net(X), y) + lambd * l2_penalty(w)
            l.sum().backward()
            native.sgd([w, b], lr, batch_size)
        if (epoch + 1) % 5 == 0:
            animator.add(epoch + 1, (native.evaluate_loss(net, train_iter, loss),
                                     native.evaluate_loss(net, test_iter, loss)))
    print('w的L2范数是: ', torch.norm(w).item())

# From chapter_04_multilayer_perceptrons/weight_decay.py
def train_concise(wd):
    net = nn.Sequential(nn.Linear(num_inputs, 1))
    for param in net.parameters():
        param.data.normal_()
    loss = nn.MSELoss(reduction='none')
    num_epochs, lr = 100, 0.003
    # 偏置参数没有衰减
    trainer = torch.optim.SGD([

```

```

        {"params":net[0].weight,'weight_decay': wd},
        {"params":net[0].bias}], lr=lr)
animator = native.Animator(xlabel='epochs', ylabel='loss', yscale='log',
                           xlim=[5, num_epochs], legend=['train', 'test'])
for epoch in range(num_epochs):
    for X, y in train_iter:
        trainer.zero_grad()
        l = loss(net(X), y)
        l.mean().backward()
        trainer.step()
    if (epoch + 1) % 5 == 0:
        animator.add(epoch + 1,
                      (native.evaluate_loss(net, train_iter, loss),
                       native.evaluate_loss(net, test_iter, loss)))
print('w的L2范数: ', net[0].weight.norm().item())

# From chapter_05_deep_learning_computation/custom_layer.py
class CenteredLayer(nn.Module):
    def __init__(self):
        super().__init__()

    def forward(self, X):
        return X - X.mean()

# From chapter_05_deep_learning_computation/custom_layer.py
class MyLinear(nn.Module):
    def __init__(self, in_units, units):
        super().__init__()
        self.weight = nn.Parameter(torch.randn(in_units, units))
        self.bias = nn.Parameter(torch.randn(units,))
    def forward(self, X):
        linear = torch.matmul(X, self.weight.data) + self.bias.data
        return F.relu(linear)

# From chapter_05_deep_learning_computation/deferred_init.py
def get_net():
    return nn.Sequential(nn.LazyLinear(256), nn.ReLU(), nn.LazyLinear(10))

# From chapter_05_deep_learning_computation/model_construction.py
class MLP(nn.Module):
    # 用模型参数声明层。这里，我们声明两个全连接的层
    def __init__(self):
        # 调用MLP的父类Module的构造函数来执行必要的初始化。
        # 这样，在类实例化时也可以指定其他函数参数，例如模型参数params（稍后将介绍）
        super().__init__()
        self.hidden = nn.Linear(20, 256) # 隐藏层
        self.out = nn.Linear(256, 10) # 输出层

    # 定义模型的前向传播，即如何根据输入X返回所需的模型输出
    def forward(self, X):
        # 注意，这里我们使用ReLU的函数版本，其在nn.functional模块中定义。
        return self.out(F.relu(self.hidden(X)))

# From chapter_05_deep_learning_computation/model_construction.py
class MySequential(nn.Module):
    def __init__(self, *args):
        super().__init__()
        for idx, module in enumerate(args):
            # 这里，module是Module子类的一个实例。我们把它保存在'Module'类的成员
            # 变量_modules中。_module的类型是OrderedDict
            self._modules[str(idx)] = module

    def forward(self, X):
        # OrderedDict保证了按照成员添加的顺序遍历它们
        for block in self._modules.values():
            X = block(X)
        return X

# From chapter_05_deep_learning_computation/model_construction.py
class FixedHiddenMLP(nn.Module):
    def __init__(self):
        super().__init__()
        # 不计算梯度的随机权重参数。因此其在训练期间保持不变
        self.rand_weight = torch.rand((20, 20), requires_grad=False)
        self.linear = nn.Linear(20, 20)

    def forward(self, X):

```

```

        X = self.linear(X)
    # 使用创建的常量参数以及relu和mm函数
    X = F.relu(torch.mm(X, self.rand_weight) + 1)
    # 复用全连接层。这相当于两个全连接层共享参数
    X = self.linear(X)
    # 控制流
    while X.abs().sum() > 1:
        X /= 2
    return X.sum()

# From chapter_05_deep_learning_computation/model_construction.py
class NestMLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(20, 64), nn.ReLU(),
                                   nn.Linear(64, 32), nn.ReLU())
        self.linear = nn.Linear(32, 16)

    def forward(self, X):
        return self.linear(self.net(X))

# From chapter_05_deep_learning_computation/parameters.py
def block1():
    return nn.Sequential(nn.Linear(4, 8), nn.ReLU(),
                          nn.Linear(8, 4), nn.ReLU())

# From chapter_05_deep_learning_computation/parameters.py
def block2():
    net = nn.Sequential()
    for i in range(4):
        # 在这里嵌套
        net.add_module(f'block {i}', block1())
    return net

# From chapter_05_deep_learning_computation/parameters.py
def init_normal(m):
    if type(m) == nn.Linear:
        nn.init.normal_(m.weight, mean=0, std=0.01)
        nn.init.zeros_(m.bias)

# From chapter_05_deep_learning_computation/parameters.py
def init_constant(m):
    if type(m) == nn.Linear:
        nn.init.constant_(m.weight, 1)
        nn.init.zeros_(m.bias)

# From chapter_05_deep_learning_computation/parameters.py
def init_xavier(m):
    if type(m) == nn.Linear:
        nn.init.xavier_uniform_(m.weight)

# From chapter_05_deep_learning_computation/parameters.py
def init_42(m):
    if type(m) == nn.Linear:
        nn.init.constant_(m.weight, 42)

# From chapter_05_deep_learning_computation/parameters.py
def my_init(m):
    if type(m) == nn.Linear:
        print("Init", *[(name, param.shape)
                        for name, param in m.named_parameters()][0])
        nn.init.uniform_(m.weight, -10, 10)
        m.weight.data *= m.weight.data.abs() >= 5

# From chapter_05_deep_learning_computation/read_write.py
class MLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.hidden = nn.Linear(20, 256)
        self.output = nn.Linear(256, 10)

    def forward(self, x):
        return self.output(F.relu(self.hidden(x)))

# From chapter_05_deep_learning_computation/use_gpu.py
def try_gpu(i=0): #@save

```

```

"""如果存在, 则返回gpu(i), 否则返回cpu()"""
if torch.cuda.device_count() >= i + 1:
    return torch.device(f'cuda:{i}')
return torch.device('cpu')

# From chapter_05_deep_learning_computation/use_gpu.py
def try_all_gpus(): # @save
    """返回所有可用的GPU, 如果没有GPU, 则返回[cpu().]"""
    devices = [torch.device(f'cuda:{i}')]
    for i in range(torch.cuda.device_count()):
        return devices if devices else [torch.device('cpu')]

# From chapter_06_convolutional_neural_networks/channels.py
def corr2d_multi_in(X, K):
    # 先遍历“X”和“K”的第0个维度(通道维度), 再把它们加在一起
    return sum(native.corr2d(x, k) for x, k in zip(X, K))

# From chapter_06_convolutional_neural_networks/channels.py
def corr2d_multi_in_out(X, K):
    # 迭代“K”的第0个维度, 每次都对输入“X”执行互相关运算。
    # 最后将所有结果都叠加在一起
    return native.stack([corr2d_multi_in(X, k) for k in K], 0)

# From chapter_06_convolutional_neural_networks/channels.py
def corr2d_multi_in_out_1x1(X, K):
    c_i, h, w = X.shape
    c_o = K.shape[0]
    X = native.reshape(X, (c_i, h * w))
    K = native.reshape(K, (c_o, c_i))
    # 全连接层中的矩阵乘法
    Y = native.matmul(K, X)
    return native.reshape(Y, (c_o, h, w))

# From chapter_06_convolutional_neural_networks/conv_layer.py
def corr2d(X, K): # @save
    """计算二维互相关运算"""
    h, w = K.shape
    Y = native.zeros((X.shape[0] - h + 1, X.shape[1] - w + 1))
    for i in range(Y.shape[0]):
        for j in range(Y.shape[1]):
            Y[i, j] = native.reduce_sum((X[i: i + h, j: j + w] * K))
    return Y

# From chapter_06_convolutional_neural_networks/conv_layer.py
class Conv2D(nn.Module):
    def __init__(self, kernel_size):
        super().__init__()
        self.weight = nn.Parameter(torch.rand(kernel_size))
        self.bias = nn.Parameter(torch.zeros(1))

    def forward(self, x):
        return corr2d(x, self.weight) + self.bias

# From chapter_06_convolutional_neural_networks/lenet.py
def evaluate_accuracy_gpu(net, data_iter, device=None): # @save
    """使用GPU计算模型在数据集上的精度"""
    if isinstance(net, nn.Module):
        net.eval() # 设置为评估模式
        if not device:
            device = next(iter(net.parameters())).device
    # 正确预测的数量, 总预测的数量
    metric = native.Accumulator(2)
    with torch.no_grad():
        for X, y in data_iter:
            if isinstance(X, list):
                # BERT微调所需的(之后将介绍)
                X = [x.to(device) for x in X]
            else:
                X = X.to(device)
            y = y.to(device)
            metric.add(native.accuracy(net(X), y), native.size(y))
    return metric[0] / metric[1]

# From chapter_06_convolutional_neural_networks/lenet.py
def train_ch6(net, train_iter, test_iter, num_epochs, lr, device):
    """用GPU训练模型(在第六章定义)"""
    def init_weights(m):

```

```

        if type(m) == nn.Linear or type(m) == nn.Conv2d:
            nn.init.xavier_uniform_(m.weight)
    net.apply(init_weights)
    print('training on', device)
    net.to(device)
    optimizer = torch.optim.SGD(net.parameters(), lr=lr)
    loss = nn.CrossEntropyLoss()
    animator = native.Animator(xlabel='epoch', xlim=[1, num_epochs],
                               legend=['train loss', 'train acc', 'test acc'])
    timer, num_batches = native.Timer(), len(train_iter)
    for epoch in range(num_epochs):
        # 训练损失之和, 训练准确率之和, 样本数
        metric = native.Accumulator(3)
        net.train()
        for i, (X, y) in enumerate(train_iter):
            timer.start()
            optimizer.zero_grad()
            X, y = X.to(device), y.to(device)
            y_hat = net(X)
            l = loss(y_hat, y)
            l.backward()
            optimizer.step()
            with torch.no_grad():
                metric.add(l * X.shape[0], native.accuracy(y_hat, y), X.shape[0])
            timer.stop()
            train_l = metric[0] / metric[2]
            train_acc = metric[1] / metric[2]
            if (i + 1) % (num_batches // 5) == 0 or i == num_batches - 1:
                animator.add(epoch + (i + 1) / num_batches,
                             (train_l, train_acc, None))
            test_acc = evaluate_accuracy_gpu(net, test_iter)
            animator.add(epoch + 1, (None, None, test_acc))
    print(f'loss {train_l:.3f}, train acc {train_acc:.3f}, '
          f'test acc {test_acc:.3f}')
    print(f'{metric[2] * num_epochs / timer.sum():.1f} examples/sec '
          f'on {str(device)}')

```

From chapter_06_convolutional_neural_networks/padding_and_strides.py

```

def comp_conv2d(conv2d, X):
    # 这里的 (1, 1) 表示批量大小和通道数都是1
    X = X.reshape((1, 1) + X.shape)
    Y = conv2d(X)
    # 省略前两个维度: 批量大小和通道
    return Y.reshape(Y.shape[2:])

```

From chapter_06_convolutional_neural_networks/pooling.py

```

def pool2d(X, pool_size, mode='max'):
    p_h, p_w = pool_size
    Y = native.zeros((X.shape[0] - p_h + 1, X.shape[1] - p_w + 1))
    for i in range(Y.shape[0]):
        for j in range(Y.shape[1]):
            if mode == 'max':
                Y[i, j] = X[i: i + p_h, j: j + p_w].max()
            elif mode == 'avg':
                Y[i, j] = X[i: i + p_h, j: j + p_w].mean()
    return Y

```

From chapter_07_modern_convolutional_networks/batch_norm.py

```

def batch_norm(X, gamma, beta, moving_mean, moving_var, eps, momentum):
    # 通过is_grad_enabled来判断当前模式是训练模式还是预测模式
    if not torch.is_grad_enabled():
        # 如果是在预测模式下, 直接使用传入的移动平均所得的均值和方差
        X_hat = (X - moving_mean) / torch.sqrt(moving_var + eps)
    else:
        assert len(X.shape) in (2, 4)
        if len(X.shape) == 2:
            # 使用全连接层的情况, 计算特征维上的均值和方差
            mean = X.mean(dim=0)
            var = ((X - mean) ** 2).mean(dim=0)
        else:
            # 使用二维卷积层的情况, 计算通道维上 (axis=1) 的均值和方差。
            # 这里我们需要保持X的形状以便后面可以做广播运算
            mean = X.mean(dim=(0, 2, 3), keepdim=True)
            var = ((X - mean) ** 2).mean(dim=(0, 2, 3), keepdim=True)
        # 训练模式下, 用当前的均值和方差做标准化
        X_hat = (X - mean) / torch.sqrt(var + eps)
        # 更新移动平均的均值和方差

```

```

        moving_mean = momentum * moving_mean + (1.0 - momentum) * mean
        moving_var = momentum * moving_var + (1.0 - momentum) * var
    Y = gamma * X_hat + beta # 缩放和移位
    return Y, moving_mean.data, moving_var.data

# From chapter_07_modern_convolutional_networks/batch_norm.py
class BatchNorm(nn.Module):
    # num_features: 完全连接层的输出数量或卷积层的输出通道数。
    # num_dims: 2表示完全连接层, 4表示卷积层
    def __init__(self, num_features, num_dims):
        super().__init__()
        if num_dims == 2:
            shape = (1, num_features)
        else:
            shape = (1, num_features, 1, 1)
        # 参与求梯度和迭代的拉伸和偏移参数, 分别初始化成1和0
        self.gamma = nn.Parameter(torch.ones(shape))
        self.beta = nn.Parameter(torch.zeros(shape))
        # 非模型参数的变量初始化为0和1
        self.moving_mean = torch.zeros(shape)
        self.moving_var = torch.ones(shape)

    def forward(self, X):
        # 如果X不在内存上, 将moving_mean和moving_var
        # 复制到X所在显存上
        if self.moving_mean.device != X.device:
            self.moving_mean = self.moving_mean.to(X.device)
            self.moving_var = self.moving_var.to(X.device)
        # 保存更新过的moving_mean和moving_var
        Y, self.moving_mean, self.moving_var = batch_norm(
            X, self.gamma, self.beta, self.moving_mean,
            self.moving_var, eps=1e-5, momentum=0.9)
        return Y

# From chapter_07_modern_convolutional_networks/densenet.py
def conv_block(input_channels, num_channels):
    return nn.Sequential(
        nn.BatchNorm2d(input_channels), nn.ReLU(),
        nn.Conv2d(input_channels, num_channels, kernel_size=3, padding=1))

# From chapter_07_modern_convolutional_networks/densenet.py
class DenseBlock(nn.Module):
    def __init__(self, num_convs, input_channels, num_channels):
        super(DenseBlock, self).__init__()
        layer = []
        for i in range(num_convs):
            layer.append(conv_block(
                num_channels * i + input_channels, num_channels))
        self.net = nn.Sequential(*layer)

    def forward(self, X):
        for blk in self.net:
            Y = blk(X)
        # 连接通道维度上每个块的输入和输出
        X = torch.cat((X, Y), dim=1)
        return X

# From chapter_07_modern_convolutional_networks/densenet.py
def transition_block(input_channels, num_channels):
    return nn.Sequential(
        nn.BatchNorm2d(input_channels), nn.ReLU(),
        nn.Conv2d(input_channels, num_channels, kernel_size=1),
        nn.AvgPool2d(kernel_size=2, stride=2))

# From chapter_07_modern_convolutional_networks/googlenet.py
class Inception(nn.Module):
    # c1--c4是每条路径的输出通道数
    def __init__(self, in_channels, c1, c2, c3, c4, **kwargs):
        super(Inception, self).__init__(**kwargs)
        # 线路1, 单1x1卷积层
        self.p1_1 = nn.Conv2d(in_channels, c1, kernel_size=1)
        # 线路2, 1x1卷积层后接3x3卷积层
        self.p2_1 = nn.Conv2d(in_channels, c2[0], kernel_size=1)
        self.p2_2 = nn.Conv2d(c2[0], c2[1], kernel_size=3, padding=1)
        # 线路3, 1x1卷积层后接5x5卷积层
        self.p3_1 = nn.Conv2d(in_channels, c3[0], kernel_size=1)

```

```

        self.p3_2 = nn.Conv2d(c3[0], c3[1], kernel_size=5, padding=2)
# 线路4, 3x3最大汇聚层后接1x1卷积层
        self.p4_1 = nn.MaxPool2d(kernel_size=3, stride=1, padding=1)
        self.p4_2 = nn.Conv2d(in_channels, c4, kernel_size=1)

    def forward(self, x):
        p1 = F.relu(self.p1_1(x))
        p2 = F.relu(self.p2_2(F.relu(self.p2_1(x))))
        p3 = F.relu(self.p3_2(F.relu(self.p3_1(x))))
        p4 = F.relu(self.p4_2(self.p4_1(x)))
# 在通道维度上连接输出
        return torch.cat((p1, p2, p3, p4), dim=1)

# From chapter_07_modern_convolutional_networks/nin.py
def nin_block(in_channels, out_channels, kernel_size, strides, padding):
    return nn.Sequential(
        nn.Conv2d(in_channels, out_channels, kernel_size, strides, padding),
        nn.ReLU(),
        nn.Conv2d(out_channels, out_channels, kernel_size=1, nn.ReLU(),
        nn.Conv2d(out_channels, out_channels, kernel_size=1, nn.ReLU())

# From chapter_07_modern_convolutional_networks/resnet.py
class Residual(nn.Module): #@save
    def __init__(self, input_channels, num_channels,
                 use_1x1conv=False, strides=1):
        super().__init__()
        self.conv1 = nn.Conv2d(input_channels, num_channels,
                                kernel_size=3, padding=1, stride=strides)
        self.conv2 = nn.Conv2d(num_channels, num_channels,
                                kernel_size=3, padding=1)

        if use_1x1conv:
            self.conv3 = nn.Conv2d(input_channels, num_channels,
                                    kernel_size=1, stride=strides)
        else:
            self.conv3 = None
        self.bn1 = nn.BatchNorm2d(num_channels)
        self.bn2 = nn.BatchNorm2d(num_channels)

    def forward(self, X):
        Y = F.relu(self.bn1(self.conv1(X)))
        Y = self.bn2(self.conv2(Y))
        if self.conv3:
            X = self.conv3(X)
        Y += X
        return F.relu(Y)

# From chapter_07_modern_convolutional_networks/resnet.py
def resnet_block(input_channels, num_channels, num_residuals,
                 first_block=False):
    blk = []
    for i in range(num_residuals):
        if i == 0 and not first_block:
            blk.append(Residual(input_channels, num_channels,
                                use_1x1conv=True, strides=2))
        else:
            blk.append(Residual(num_channels, num_channels))
    return blk

# From chapter_07_modern_convolutional_networks/vgg.py
def vgg_block(num_convs, in_channels, out_channels):
    layers = []
    for _ in range(num_convs):
        layers.append(nn.Conv2d(in_channels, out_channels,
                                kernel_size=3, padding=1))
        layers.append(nn.ReLU())
        in_channels = out_channels
    layers.append(nn.MaxPool2d(kernel_size=2, stride=2))
    return nn.Sequential(*layers)

# From chapter_07_modern_convolutional_networks/vgg.py
def vgg(conv_arch):
    conv_blks = []
    in_channels = 1
# 卷积层部分
    for (num_convs, out_channels) in conv_arch:
        conv_blks.append(vgg_block(num_convs, in_channels, out_channels))
        in_channels = out_channels

```

```

return nn.Sequential(
    *conv_blks, nn.Flatten(),
    # 全连接层部分
    nn.Linear(out_channels * 7 * 7, 4096), nn.ReLU(), nn.Dropout(0.5),
    nn.Linear(4096, 4096), nn.ReLU(), nn.Dropout(0.5),
    nn.Linear(4096, 10))

# From chapter_08_recurrent_neural_networks/language_models_and_dataset.py
def seq_data_iter_random(corpus, batch_size, num_steps): #@save
    """使用随机抽样生成一个小批量子序列"""
    # 从随机偏移量开始对序列进行分区，随机范围包括num_steps-1
    corpus = corpus[random.randint(0, num_steps - 1):]
    # 减去1，是因为我们需要考虑标签
    num_subseqs = (len(corpus) - 1) // num_steps
    # 长度为num_steps的子序列的起始索引
    initial_indices = list(range(0, num_subseqs * num_steps, num_steps))
    # 在随机抽样的迭代过程中，
    # 来自两个相邻的、随机的、小批量中的子序列不一定在原始序列上相邻
    random.shuffle(initial_indices)

    def data(pos):
        # 返回从pos位置开始的长度为num_steps的序列
        return corpus[pos: pos + num_steps]

    num_batches = num_subseqs // batch_size
    for i in range(0, batch_size * num_batches, batch_size):
        # 在这里，initial_indices包含子序列的随机起始索引
        initial_indices_per_batch = initial_indices[i: i + batch_size]
        X = [data(j) for j in initial_indices_per_batch]
        Y = [data(j + 1) for j in initial_indices_per_batch]
        yield native.tensor(X), native.tensor(Y)

# From chapter_08_recurrent_neural_networks/language_models_and_dataset.py
def seq_data_iter_sequential(corpus, batch_size, num_steps): #@save
    """使用顺序分区生成一个小批量子序列"""
    # 从随机偏移量开始划分序列
    offset = random.randint(0, num_steps)
    num_tokens = ((len(corpus) - offset - 1) // batch_size) * batch_size
    Xs = native.tensor(corpus[offset: offset + num_tokens])
    Ys = native.tensor(corpus[offset + 1: offset + 1 + num_tokens])
    Xs, Ys = Xs.reshape(batch_size, -1), Ys.reshape(batch_size, -1)
    num_batches = Xs.shape[1] // num_steps

    for i in range(0, num_steps * num_batches, num_steps):
        X = Xs[:, i: i + num_steps]
        Y = Ys[:, i: i + num_steps]
        yield X, Y

# From chapter_08_recurrent_neural_networks/language_models_and_dataset.py
class SeqDataLoader: #@save
    """加载序列数据的迭代器"""
    def __init__(self, batch_size, num_steps, use_random_iter, max_tokens):
        if use_random_iter:
            self.data_iter_fn = native.seq_data_iter_random
        else:
            self.data_iter_fn = native.seq_data_iter_sequential
        self.corpus, self.vocab = native.load_corpus_time_machine(max_tokens)
        self.batch_size, self.num_steps = batch_size, num_steps

    def __iter__(self):
        return self.data_iter_fn(self.corpus, self.batch_size, self.num_steps)

# From chapter_08_recurrent_neural_networks/language_models_and_dataset.py
def load_data_time_machine(batch_size, num_steps, #@save
                           use_random_iter=False, max_tokens=10000):
    """返回时光机器数据集的迭代器和词表"""
    data_iter = SeqDataLoader(
        batch_size, num_steps, use_random_iter, max_tokens)
    return data_iter, data_iter.vocab

# From chapter_08_recurrent_neural_networks/rnn_concise.py
class RNNModel(nn.Module):
    """神经网络模型"""
    def __init__(self, rnn_layer, vocab_size, **kwargs):
        super(RNNModel, self).__init__(**kwargs)
        self.rnn = rnn_layer
        self.vocab_size = vocab_size

```

```

        self.num_hiddens = self.rnn.hidden_size
# 如果RNN是双向的（之后将介绍），num_directions应该是2，否则应该是1
        if not self.rnn.bidirectional:
            self.num_directions = 1
            self.linear = nn.Linear(self.num_hiddens, self.vocab_size)
        else:
            self.num_directions = 2
            self.linear = nn.Linear(self.num_hiddens * 2, self.vocab_size)

    def forward(self, inputs, state):
        X = F.one_hot(inputs.T.long(), self.vocab_size)
        X = X.to(torch.float32)
        Y, state = self.rnn(X, state)
# 全连接层首先将Y的形状改为(时间步数*批量大小,隐藏单元数)
# 它的输出形状是(时间步数*批量大小,词表大小)。
        output = self.linear(Y.reshape((-1, Y.shape[-1])))
        return output, state

    def begin_state(self, device, batch_size=1):
        if not isinstance(self.rnn, nn.LSTM):
# nn.GRU以张量作为隐状态
            return torch.zeros((self.num_directions * self.rnn.num_layers,
                                batch_size, self.num_hiddens),
                                device=device)
        else:
# nn.LSTM以元组作为隐状态
            return (torch.zeros((
                self.num_directions * self.rnn.num_layers,
                batch_size, self.num_hiddens), device=device),
                    torch.zeros((
                        self.num_directions * self.rnn.num_layers,
                        batch_size, self.num_hiddens), device=device))

# From chapter_08_recurrent_neural_networks/rnn_scratch.py
def get_params(vocab_size, num_hiddens, device):
    num_inputs = num_outputs = vocab_size

    def normal(shape):
        return torch.randn(size=shape, device=device) * 0.01

# 隐藏层参数
    W_xh = normal((num_inputs, num_hiddens))
    W_hh = normal((num_hiddens, num_hiddens))
    b_h = native.zeros(num_hiddens, device=device)
# 输出层参数
    W_hq = normal((num_hiddens, num_outputs))
    b_q = native.zeros(num_outputs, device=device)
# 附加梯度
    params = [W_xh, W_hh, b_h, W_hq, b_q]
    for param in params:
        param.requires_grad_(True)
    return params

# From chapter_08_recurrent_neural_networks/rnn_scratch.py
def init_rnn_state(batch_size, num_hiddens, device):
    return (native.zeros((batch_size, num_hiddens), device=device), )

# From chapter_08_recurrent_neural_networks/rnn_scratch.py
def rnn(inputs, state, params):
# inputs的形状: (时间步数量, 批量大小, 词表大小)
    W_xh, W_hh, b_h, W_hq, b_q = params
    H, = state
    outputs = []
# X的形状: (批量大小, 词表大小)
    for X in inputs:
        H = torch.tanh(torch.mm(X, W_xh) + torch.mm(H, W_hh) + b_h)
        Y = torch.mm(H, W_hq) + b_q
        outputs.append(Y)
    return torch.cat(outputs, dim=0), (H,)

# From chapter_08_recurrent_neural_networks/rnn_scratch.py
class RNNModelScratch: #@save
    """从零开始实现的循环神经网络模型"""
    def __init__(self, vocab_size, num_hiddens, device,
                 get_params, init_state, forward_fn):
        self.vocab_size, self.num_hiddens = vocab_size, num_hiddens
        self.params = get_params(vocab_size, num_hiddens, device)

```

```

        self.init_state, self.forward_fn = init_state, forward_fn

    def __call__(self, X, state):
        X = F.one_hot(X.T, self.vocab_size).type(torch.float32)
        return self.forward_fn(X, state, self.params)

    def begin_state(self, batch_size, device):
        return self.init_state(batch_size, self.num_hiddens, device)

# From chapter_08_recurrent_neural_networks/rnn_scratch.py
def predict_ch8(prefix, num_preds, net, vocab, device): #@save
    """在prefix后面生成新字符"""
    state = net.begin_state(batch_size=1, device=device)
    outputs = [vocab[prefix[0]]]
    get_input = lambda: native.reshape(native.tensor(
        [outputs[-1]], device=device), (1, 1))
    for y in prefix[1:]: # 预热期
        _, state = net(get_input(), state)
        outputs.append(vocab[y])
    for _ in range(num_preds): # 预测num_preds步
        y, state = net(get_input(), state)
        outputs.append(int(y.argmax(dim=1).reshape(1)))
    return ''.join([vocab.idx_to_token[i] for i in outputs])

# From chapter_08_recurrent_neural_networks/rnn_scratch.py
def grad_clipping(net, theta): #@save
    """裁剪梯度"""
    if isinstance(net, nn.Module):
        params = [p for p in net.parameters() if p.requires_grad]
    else:
        params = net.params
    norm = torch.sqrt(sum(torch.sum((p.grad ** 2)) for p in params))
    if norm > theta:
        for param in params:
            param.grad[:] *= theta / norm

# From chapter_08_recurrent_neural_networks/rnn_scratch.py
def train_epoch_ch8(net, train_iter, loss, updater, device, use_random_iter):
    """训练网络一个迭代周期（定义见第8章）"""
    state, timer = None, native.Timer()
    metric = native.Accumulator(2) # 训练损失之和, 词元数量
    for X, Y in train_iter:
        if state is None or use_random_iter:
            # 在第一次迭代或使用随机抽样时初始化state
            state = net.begin_state(batch_size=X.shape[0], device=device)
        else:
            if isinstance(net, nn.Module) and not isinstance(state, tuple):
                # state对于nn.GRU是个张量
                state.detach_()
            else:
                # state对于nn.LSTM或对于我们从零开始实现的模型是个张量
                for s in state:
                    s.detach_()
        y = Y.T.reshape(-1)
        X, y = X.to(device), y.to(device)
        y_hat, state = net(X, state)
        l = loss(y_hat, y.long()).mean()
        if isinstance(updater, torch.optim.Optimizer):
            updater.zero_grad()
            l.backward()
            grad_clipping(net, 1)
            updater.step()
        else:
            l.backward()
            grad_clipping(net, 1)
        # 因为已经调用了mean函数
        updater(batch_size=1)
        metric.add(l * native.size(y), native.size(y))
    return math.exp(metric[0] / metric[1]), metric[1] / timer.stop()

# From chapter_08_recurrent_neural_networks/rnn_scratch.py
def train_ch8(net, train_iter, vocab, lr, num_epochs, device,
              use_random_iter=False):
    """训练模型（定义见第8章）"""
    loss = nn.CrossEntropyLoss()
    animator = native.Animator(xlabel='epoch', ylabel='perplexity',

```

```

        legend=['train'], xlim=[10, num_epochs])

# 初始化
if isinstance(net, nn.Module):
    updater = torch.optim.SGD(net.parameters(), lr)
else:
    updater = lambda batch_size: native.sgd(net.params, lr, batch_size)
predict = lambda prefix: predict_ch8(prefix, 50, net, vocab, device)

# 训练和预测
for epoch in range(num_epochs):
    ppl, speed = train_epoch_ch8(
        net, train_iter, loss, updater, device, use_random_iter)
    if (epoch + 1) % 10 == 0:
        print(predict('time traveller'))
        animator.add(epoch + 1, [ppl])
print(f'困惑度 {ppl:.1f}, {speed:.1f} 词元/秒 {str(device)}')
print(predict('time traveller'))
print(predict('traveller'))

# From chapter_08_recurrent_neural_networks/sequence.py
def init_weights(m):
    if type(m) == nn.Linear:
        nn.init.xavier_uniform_(m.weight)

# From chapter_08_recurrent_neural_networks/sequence.py
def get_net():
    net = nn.Sequential(nn.Linear(4, 10),
                        nn.ReLU(),
                        nn.Linear(10, 1))
    net.apply(init_weights)
    return net

# From chapter_08_recurrent_neural_networks/sequence.py
def train(net, train_iter, loss, epochs, lr):
    trainer = torch.optim.Adam(net.parameters(), lr)
    for epoch in range(epochs):
        for X, y in train_iter:
            trainer.zero_grad()
            l = loss(net(X), y)
            l.sum().backward()
            trainer.step()
        print(f'epoch {epoch + 1}, '
              f'loss: {native.evaluate_loss(net, train_iter, loss):f}')

# From chapter_08_recurrent_neural_networks/text_preprocessing.py
def read_time_machine(): #@save
    """将时间机器数据集加载到文本行的列表中"""
    with open(native.download('time_machine'), 'r') as f:
        lines = f.readlines()
    return [re.sub('[^A-Za-z]+', ' ', line).strip().lower() for line in lines]

# From chapter_08_recurrent_neural_networks/text_preprocessing.py
def tokenize(lines, token='word'): #@save
    """将文本行拆分为单词或字符词元"""
    if token == 'word':
        return [line.split() for line in lines]
    elif token == 'char':
        return [list(line) for line in lines]
    else:
        print('错误: 未知词元类型: ' + token)

# From chapter_08_recurrent_neural_networks/text_preprocessing.py
class Vocab: #@save
    """文本词表"""
    def __init__(self, tokens=None, min_freq=0, reserved_tokens=None):
        if tokens is None:
            tokens = []
        if reserved_tokens is None:
            reserved_tokens = []
        # 按出现频率排序
        counter = count_corpus(tokens)
        self._token_freqs = sorted(counter.items(), key=lambda x: x[1],
                                   reverse=True)
        # 未知词元的索引为0
        self.idx_to_token = ['<unk>'] + reserved_tokens
        self.token_to_idx = {token: idx
                              for idx, token in enumerate(self.idx_to_token)}
        for token, freq in self._token_freqs:

```

```

        if freq < min_freq:
            break
        if token not in self.token_to_idx:
            self.idx_to_token.append(token)
            self.token_to_idx[token] = len(self.idx_to_token) - 1

    def __len__(self):
        return len(self.idx_to_token)

    def __getitem__(self, tokens):
        if not isinstance(tokens, (list, tuple)):
            return self.token_to_idx.get(tokens, self.unk)
        return [self.__getitem__(token) for token in tokens]

    def to_tokens(self, indices):
        if not isinstance(indices, (list, tuple)):
            return self.idx_to_token[indices]
        return [self.idx_to_token[index] for index in indices]

    @property
    def unk(self): # 未知词元的索引为0
        return 0

    @property
    def token_freqs(self):
        return self._token_freqs

# From chapter_08_recurrent_neural_networks/text_preprocessing.py
def count_corpus(tokens): # @save
    """统计词元的频率"""
    # 这里的tokens是1D列表或2D列表
    if len(tokens) == 0 or isinstance(tokens[0], list):
        # 将词元列表展平成一个列表
        tokens = [token for line in tokens for token in line]
    return collections.Counter(tokens)

# From chapter_08_recurrent_neural_networks/text_preprocessing.py
def load_corpus_time_machine(max_tokens=-1): # @save
    """返回时光机器数据集的词元索引列表和词表"""
    lines = read_time_machine()
    tokens = tokenize(lines, 'char')
    vocab = Vocab(tokens)
    # 因为时光机器数据集集中的每个文本行不一定是一个句子或一个段落,
    # 所以将所有文本行展平到一个列表中
    corpus = [vocab[token] for line in tokens for token in line]
    if max_tokens > 0:
        corpus = corpus[:max_tokens]
    return corpus, vocab

# From chapter_09_modern_recurrent_neural_networks/encoder_decoder.py
class Encoder(nn.Module):
    """编码器-解码器架构的基本编码器接口"""
    def __init__(self, **kwargs):
        super(Encoder, self).__init__(**kwargs)

    def forward(self, X, *args):
        raise NotImplementedError

# From chapter_09_modern_recurrent_neural_networks/encoder_decoder.py
class Decoder(nn.Module):
    """编码器-解码器架构的基本解码器接口"""
    def __init__(self, **kwargs):
        super(Decoder, self).__init__(**kwargs)

    def init_state(self, enc_outputs, *args):
        raise NotImplementedError

    def forward(self, X, state):
        raise NotImplementedError

# From chapter_09_modern_recurrent_neural_networks/encoder_decoder.py
class EncoderDecoder(nn.Module):
    """编码器-解码器架构的基类"""
    def __init__(self, encoder, decoder, **kwargs):
        super(EncoderDecoder, self).__init__(**kwargs)
        self.encoder = encoder
        self.decoder = decoder

```

```

def forward(self, enc_X, dec_X, *args):
    enc_outputs = self.encoder(enc_X, *args)
    dec_state = self.decoder.init_state(enc_outputs, *args)
    return self.decoder(dec_X, dec_state)

# From chapter_09_modern_recurrent_neural_networks/gru.py
def get_params(vocab_size, num_hiddens, device):
    num_inputs = num_outputs = vocab_size

    def normal(shape):
        return torch.randn(size=shape, device=device)*0.01

    def three():
        return (normal((num_inputs, num_hiddens)),
                normal((num_hiddens, num_hiddens)),
                native.zeros(num_hiddens, device=device))

    W_xz, W_hz, b_z = three() # 更新门参数
    W_xr, W_hr, b_r = three() # 重置门参数
    W_xh, W_hh, b_h = three() # 候选隐状态参数
    # 输出层参数
    W_hq = normal((num_hiddens, num_outputs))
    b_q = native.zeros(num_outputs, device=device)
    # 附加梯度
    params = [W_xz, W_hz, b_z, W_xr, W_hr, b_r, W_xh, W_hh, b_h, W_hq, b_q]
    for param in params:
        param.requires_grad_(True)
    return params

# From chapter_09_modern_recurrent_neural_networks/gru.py
def init_gru_state(batch_size, num_hiddens, device):
    return (torch.zeros((batch_size, num_hiddens), device=device), )

# From chapter_09_modern_recurrent_neural_networks/gru.py
def gru(inputs, state, params):
    W_xz, W_hz, b_z, W_xr, W_hr, b_r, W_xh, W_hh, b_h, W_hq, b_q = params
    H, = state
    outputs = []
    for X in inputs:
        Z = torch.sigmoid((X @ W_xz) + (H @ W_hz) + b_z)
        R = torch.sigmoid((X @ W_xr) + (H @ W_hr) + b_r)
        H_tilda = torch.tanh((X @ W_xh) + ((R * H) @ W_hh) + b_h)
        H = Z * H + (1 - Z) * H_tilda
        Y = H @ W_hq + b_q
        outputs.append(Y)
    return torch.cat(outputs, dim=0), (H,)

# From chapter_09_modern_recurrent_neural_networks/lstm.py
def get_lstm_params(vocab_size, num_hiddens, device):
    num_inputs = num_outputs = vocab_size

    def normal(shape):
        return torch.randn(size=shape, device=device)*0.01

    def three():
        return (normal((num_inputs, num_hiddens)),
                normal((num_hiddens, num_hiddens)),
                native.zeros(num_hiddens, device=device))

    W_xi, W_hi, b_i = three() # 输入门参数
    W_xf, W_hf, b_f = three() # 遗忘门参数
    W_xo, W_ho, b_o = three() # 输出门参数
    W_xc, W_hc, b_c = three() # 候选记忆元参数
    # 输出层参数
    W_hq = normal((num_hiddens, num_outputs))
    b_q = native.zeros(num_outputs, device=device)
    # 附加梯度
    params = [W_xi, W_hi, b_i, W_xf, W_hf, b_f, W_xo, W_ho, b_o, W_xc, W_hc,
              b_c, W_hq, b_q]
    for param in params:
        param.requires_grad_(True)
    return params

# From chapter_09_modern_recurrent_neural_networks/lstm.py
def init_lstm_state(batch_size, num_hiddens, device):
    return (torch.zeros((batch_size, num_hiddens), device=device),

```

```

torch.zeros((batch_size, num_hiddens), device=device))

# From chapter_09_modern_recurrent_neural_networks/lstm.py
def lstm(inputs, state, params):
    [W_xi, W_hi, b_i, W_xf, W_hf, b_f, W_xo, W_ho, b_o, W_xc, W_hc, b_c,
     W_hq, b_q] = params
    (H, C) = state
    outputs = []
    for X in inputs:
        I = torch.sigmoid((X @ W_xi) + (H @ W_hi) + b_i)
        F = torch.sigmoid((X @ W_xf) + (H @ W_hf) + b_f)
        O = torch.sigmoid((X @ W_xo) + (H @ W_ho) + b_o)
        C_tilda = torch.tanh((X @ W_xc) + (H @ W_hc) + b_c)
        C = F * C + I * C_tilda
        H = O * torch.tanh(C)
        Y = (H @ W_hq) + b_q
        outputs.append(Y)
    return torch.cat(outputs, dim=0), (H, C)

# From chapter_09_modern_recurrent_neural_networks/machine_translation_and_dataset.py
def read_data_nmt():
    """载入“英语—法语”数据集"""
    data_dir = native.download_extract('fra-eng')
    with open(os.path.join(data_dir, 'fra.txt'), 'r',
              encoding='utf-8') as f:
        return f.read()

# From chapter_09_modern_recurrent_neural_networks/machine_translation_and_dataset.py
def preprocess_nmt(text):
    """预处理“英语—法语”数据集"""
    def no_space(char, prev_char):
        return char in set(',.!?') and prev_char != ' '

    # 使用空格替换不间断空格
    # 使用小写字母替换大写字母
    text = text.replace('\u202f', ' ').replace('\xa0', ' ').lower()
    # 在单词和标点符号之间插入空格
    out = [' ' + char if i > 0 and no_space(char, text[i - 1]) else char
           for i, char in enumerate(text)]
    return ''.join(out)

# From chapter_09_modern_recurrent_neural_networks/machine_translation_and_dataset.py
def tokenize_nmt(text, num_examples=None):
    """词元化“英语—法语”数据集"""
    source, target = [], []
    for i, line in enumerate(text.split('\n')):
        if num_examples and i > num_examples:
            break
        parts = line.split('\t')
        if len(parts) == 2:
            source.append(parts[0].split(' '))
            target.append(parts[1].split(' '))
    return source, target

# From chapter_09_modern_recurrent_neural_networks/machine_translation_and_dataset.py
def show_list_len_pair_hist(legend, xlabel, ylabel, xlist, ylist):
    """绘制列表长度对的直方图"""
    native.set_figsize()
    _, _, patches = native.plt.hist(
        [[len(l) for l in xlist], [len(l) for l in ylist]])
    native.plt.xlabel(xlabel)
    native.plt.ylabel(ylabel)

    for patch in patches[1].patches:
        patch.set_hatch('/')
    native.plt.legend(legend)

# From chapter_09_modern_recurrent_neural_networks/machine_translation_and_dataset.py
def truncate_pad(line, num_steps, padding_token):
    """截断或填充文本序列"""
    if len(line) > num_steps:
        return line[:num_steps] # 截断
    return line + [padding_token] * (num_steps - len(line))

# From chapter_09_modern_recurrent_neural_networks/machine_translation_and_dataset.py
def build_array_nmt(lines, vocab, num_steps):
    """将机器翻译的文本序列转换成小批量"""

```

```

lines = [vocab[l] for l in lines]
lines = [l + [vocab['<eos>']] for l in lines]
array = native.tensor([truncate_pad(
    l, num_steps, vocab['<pad>']) for l in lines])
valid_len = native.reduce_sum(
    native.astype(array != vocab['<pad>'], native.int32), 1)
return array, valid_len

# From chapter_09_modern_recurrent_neural_networks/machine_translation_and_dataset.py
def load_data_nmt(batch_size, num_steps, num_examples=600):
    """返回翻译数据集的迭代器和词表"""
    text = preprocess_nmt(read_data_nmt())
    source, target = tokenize_nmt(text, num_examples)
    src_vocab = native.Vocab(source, min_freq=2,
        reserved_tokens=['<pad>', '<bos>', '<eos>'])
    tgt_vocab = native.Vocab(target, min_freq=2,
        reserved_tokens=['<pad>', '<bos>', '<eos>'])
    src_array, src_valid_len = build_array_nmt(source, src_vocab, num_steps)
    tgt_array, tgt_valid_len = build_array_nmt(target, tgt_vocab, num_steps)
    data_arrays = (src_array, src_valid_len, tgt_array, tgt_valid_len)
    data_iter = native.load_array(data_arrays, batch_size)
    return data_iter, src_vocab, tgt_vocab

# From chapter_09_modern_recurrent_neural_networks/seq2seq.py
class Seq2SeqEncoder(native.Encoder):
    """用于序列到序列学习的循环神经网络编码器"""
    def __init__(self, vocab_size, embed_size, num_hiddens, num_layers,
        dropout=0, **kwargs):
        super(Seq2SeqEncoder, self).__init__(**kwargs)
        # 嵌入层
        self.embedding = nn.Embedding(vocab_size, embed_size)
        self.rnn = nn.GRU(embed_size, num_hiddens, num_layers,
            dropout=dropout)

    def forward(self, X, *args):
        # 输出'X'的形状: (batch_size,num_steps,embed_size)
        X = self.embedding(X)
        # 在循环神经网络模型中, 第一个轴对应于时间步
        X = X.permute(1, 0, 2)
        # 如果未提及状态, 则默认为0
        output, state = self.rnn(X)
        # output的形状:(num_steps,batch_size,num_hiddens)
        # state的形状:(num_layers,batch_size,num_hiddens)
        return output, state

# From chapter_09_modern_recurrent_neural_networks/seq2seq.py
class Seq2SeqDecoder(native.Decoder):
    """用于序列到序列学习的循环神经网络解码器"""
    def __init__(self, vocab_size, embed_size, num_hiddens, num_layers,
        dropout=0, **kwargs):
        super(Seq2SeqDecoder, self).__init__(**kwargs)
        self.embedding = nn.Embedding(vocab_size, embed_size)
        self.rnn = nn.GRU(embed_size + num_hiddens, num_hiddens, num_layers,
            dropout=dropout)
        self.dense = nn.Linear(num_hiddens, vocab_size)

    def init_state(self, enc_outputs, *args):
        return enc_outputs[1]

    def forward(self, X, state):
        # 输出'X'的形状: (batch_size,num_steps,embed_size)
        X = self.embedding(X).permute(1, 0, 2)
        # 广播context, 使其具有与X相同的num_steps
        context = state[-1].repeat(X.shape[0], 1, 1)
        X_and_context = native.concat((X, context), 2)
        output, state = self.rnn(X_and_context, state)
        output = self.dense(output).permute(1, 0, 2)
        # output的形状:(batch_size,num_steps,vocab_size)
        # state的形状:(num_layers,batch_size,num_hiddens)
        return output, state

# From chapter_09_modern_recurrent_neural_networks/seq2seq.py
def sequence_mask(X, valid_len, value=0):
    """在序列中屏蔽不相关的项"""
    maxlen = X.size(1)
    mask = torch.arange((maxlen), dtype=torch.float32,
        device=X.device)[None, :] < valid_len[:, None]

```

```

X[~mask] = value
return X

# From chapter_09_modern_recurrent_neural_networks/seq2seq.py
class MaskedSoftmaxCELoss(nn.CrossEntropyLoss):
    """带遮蔽的softmax交叉熵损失函数"""
    # pred的形状: (batch_size,num_steps,vocab_size)
    # label的形状: (batch_size,num_steps)
    # valid_len的形状: (batch_size,)
    def forward(self, pred, label, valid_len):
        weights = torch.ones_like(label)
        weights = sequence_mask(weights, valid_len)
        self.reduction='none'
        unweighted_loss = super(MaskedSoftmaxCELoss, self).forward(
            pred.permute(0, 2, 1), label)
        weighted_loss = (unweighted_loss * weights).mean(dim=1)
        return weighted_loss

# From chapter_09_modern_recurrent_neural_networks/seq2seq.py
def train_seq2seq(net, data_iter, lr, num_epochs, tgt_vocab, device):
    """训练序列到序列模型"""
    def xavier_init_weights(m):
        if type(m) == nn.Linear:
            nn.init.xavier_uniform_(m.weight)
        if type(m) == nn.GRU:
            for param in m._flat_weights_names:
                if "weight" in param:
                    nn.init.xavier_uniform_(m._parameters[param])

    net.apply(xavier_init_weights)
    net.to(device)
    optimizer = torch.optim.Adam(net.parameters(), lr=lr)
    loss = MaskedSoftmaxCELoss()
    net.train()
    animator = native.Animator(xlabel='epoch', ylabel='loss',
                               xlim=[10, num_epochs])
    for epoch in range(num_epochs):
        timer = native.Timer()
        metric = native.Accumulator(2) # 训练损失总和, 词元数量
        for batch in data_iter:
            optimizer.zero_grad()
            X, X_valid_len, Y, Y_valid_len = [x.to(device) for x in batch]
            bos = torch.tensor([tgt_vocab['<bos>']], device=device).reshape(-1, 1)
            dec_input = torch.cat([bos, Y[:, :-1]], 1) # 强制教学
            Y_hat, _ = net(X, dec_input, X_valid_len)
            l = loss(Y_hat, Y, Y_valid_len)
            l.sum().backward() # 损失函数的标量进行“反向传播”
            native.grad_clipping(net, 1)
            num_tokens = Y_valid_len.sum()
            optimizer.step()
            with torch.no_grad():
                metric.add(l.sum(), num_tokens)
            if (epoch + 1) % 10 == 0:
                animator.add(epoch + 1, (metric[0] / metric[1],))
        print(f'loss {metric[0] / metric[1]:.3f}, {metric[1] / timer.stop():.1f} '
              f'tokens/sec on {str(device)}')

# From chapter_09_modern_recurrent_neural_networks/seq2seq.py
def predict_seq2seq(net, src_sentence, src_vocab, tgt_vocab, num_steps,
                    device, save_attention_weights=False):
    """序列到序列模型的预测"""
    # 在预测时将net设置为评估模式
    net.eval()
    src_tokens = src_vocab[src_sentence.lower().split(' ')] + [
        src_vocab['<eos>']]
    enc_valid_len = torch.tensor([len(src_tokens)], device=device)
    src_tokens = native.truncate_pad(src_tokens, num_steps, src_vocab['<pad>'])
    # 添加批量轴
    enc_X = torch.unsqueeze(
        torch.tensor(src_tokens, dtype=torch.long, device=device), dim=0)
    enc_outputs = net.encoder(enc_X, enc_valid_len)
    dec_state = net.decoder.init_state(enc_outputs, enc_valid_len)
    # 添加批量轴
    dec_X = torch.unsqueeze(torch.tensor(
        [tgt_vocab['<bos>']], dtype=torch.long, device=device), dim=0)
    output_seq, attention_weight_seq = [], []

```

```

for _ in range(num_steps):
    Y, dec_state = net.decoder(dec_X, dec_state)
    # 我们使用具有预测最高可能性的词元，作为解码器在下一时间步的输入
    dec_X = Y.argmax(dim=2)
    pred = dec_X.squeeze(dim=0).type(torch.int32).item()
    # 保存注意力权重（稍后讨论）
    if save_attention_weights:
        attention_weight_seq.append(net.decoder.attention_weights)
    # 一旦序列结束词元被预测，输出序列的生成就完成了
    if pred == tgt_vocab['<eos>']:
        break
    output_seq.append(pred)

return ' '.join(tgt_vocab.to_tokens(output_seq)), attention_weight_seq

# From chapter_09_modern_recurrent_neural_networks/seq2seq.py
def bleu(pred_seq, label_seq, k): #@save
    """计算BLEU"""
    pred_tokens, label_tokens = pred_seq.split(' '), label_seq.split(' ')
    len_pred, len_label = len(pred_tokens), len(label_tokens)
    score = math.exp(min(0, 1 - len_label / len_pred))
    for n in range(1, k + 1):
        num_matches, label_subs = 0, collections.defaultdict(int)
        for i in range(len_label - n + 1):
            label_subs[' '.join(label_tokens[i: i + n])] += 1
        for i in range(len_pred - n + 1):
            if label_subs[' '.join(pred_tokens[i: i + n])] > 0:
                num_matches += 1
                label_subs[' '.join(pred_tokens[i: i + n])] -= 1
        score *= math.pow(num_matches / (len_pred - n + 1), math.pow(0.5, n))
    return score

# From chapter_10_attention_mechanisms/attention_cues.py
def show_heatmaps(matrices, xlabel, ylabel, titles=None, figsize=(2.5, 2.5),
                  cmap='Reds'):
    """显示矩阵热图"""
    native.use_svg_display()
    num_rows, num_cols = matrices.shape[0], matrices.shape[1]
    fig, axes = native.plt.subplots(num_rows, num_cols, figsize=figsize,
                                    sharex=True, sharey=True, squeeze=False)
    for i, (row_axes, row_matrices) in enumerate(zip(axes, matrices)):
        for j, (ax, matrix) in enumerate(zip(row_axes, row_matrices)):
            pcm = ax.imshow(native.numpy(matrix), cmap=cmap)
            if i == num_rows - 1:
                ax.set_xlabel(xlabel)
            if j == 0:
                ax.set_ylabel(ylabel)
            if titles:
                ax.set_title(titles[j])
    fig.colorbar(pcm, ax=axes, shrink=0.6);

# From chapter_10_attention_mechanisms/attention_scoring_functions.py
def masked_softmax(X, valid_lens):
    """通过在最后一个轴上掩蔽元素来执行softmax操作"""
    # X:3D张量, valid_lens:1D或2D张量
    if valid_lens is None:
        return nn.functional.softmax(X, dim=-1)
    else:
        shape = X.shape
        if valid_lens.dim() == 1:
            valid_lens = torch.repeat_interleave(valid_lens, shape[1])
        else:
            valid_lens = valid_lens.reshape(-1)
        # 最后一轴上被掩蔽的元素使用一个非常大的负值替换，从而其softmax输出为0
        X = native.sequence_mask(X.reshape(-1, shape[-1]), valid_lens,
                                value=-1e6)
        return nn.functional.softmax(X.reshape(shape), dim=-1)

# From chapter_10_attention_mechanisms/attention_scoring_functions.py
class AdditiveAttention(nn.Module):
    """加性注意力"""
    def __init__(self, key_size, query_size, num_hiddens, dropout, **kwargs):
        super(AdditiveAttention, self).__init__(**kwargs)

        self.W_k = nn.Linear(key_size, num_hiddens, bias=False)
        self.W_q = nn.Linear(query_size, num_hiddens, bias=False)
        self.w_v = nn.Linear(num_hiddens, 1, bias=False)
        self.dropout = nn.Dropout(dropout)

```

```

def forward(self, queries, keys, values, valid_lens):
    queries, keys = self.W_q(queries), self.W_k(keys)
    # 在维度扩展后,
    # queries的形状: (batch_size, 查询的个数, 1, num_hidden)
    # key的形状: (batch_size, 1, "键-值"对的个数, num_hiddens)
    # 使用广播方式进行求和
    features = queries.unsqueeze(2) + keys.unsqueeze(1)
    features = torch.tanh(features)
    # self.w_v仅有一个输出, 因此从形状中移除最后那个维度。
    # scores的形状: (batch_size, 查询的个数, "键-值"对的个数)
    scores = self.w_v(features).squeeze(-1)
    self.attention_weights = masked_softmax(scores, valid_lens)
    # values的形状: (batch_size, "键-值"对的个数, 值的维度)
    return torch.bmm(self.dropout(self.attention_weights), values)

# From chapter_10_attention_mechanisms/attention_scoring_functions.py
class DotProductAttention(nn.Module):
    """缩放点积注意力"""
    def __init__(self, dropout, **kwargs):
        super(DotProductAttention, self).__init__(**kwargs)
        self.dropout = nn.Dropout(dropout)

    # queries的形状: (batch_size, 查询的个数, d)
    # keys的形状: (batch_size, "键-值"对的个数, d)
    # values的形状: (batch_size, "键-值"对的个数, 值的维度)
    # valid_lens的形状:(batch_size, )或者(batch_size, 查询的个数)
    def forward(self, queries, keys, values, valid_lens=None):
        d = queries.shape[-1]
        # 设置transpose_b=True为了交换keys的最后两个维度
        scores = torch.bmm(queries, keys.transpose(1,2)) / math.sqrt(d)
        self.attention_weights = masked_softmax(scores, valid_lens)
        return torch.bmm(self.dropout(self.attention_weights), values)

# From chapter_10_attention_mechanisms/bahdanau_attention.py
class AttentionDecoder(native.Decoder):
    """带有注意力机制解码器的基本接口"""
    def __init__(self, **kwargs):
        super(AttentionDecoder, self).__init__(**kwargs)

    @property
    def attention_weights(self):
        raise NotImplementedError

# From chapter_10_attention_mechanisms/bahdanau_attention.py
class Seq2SeqAttentionDecoder(AttentionDecoder):
    def __init__(self, vocab_size, embed_size, num_hiddens, num_layers,
                 dropout=0, **kwargs):
        super(Seq2SeqAttentionDecoder, self).__init__(**kwargs)
        self.attention = native.AdditiveAttention(
            num_hiddens, num_hiddens, num_hiddens, dropout)
        self.embedding = nn.Embedding(vocab_size, embed_size)
        self.rnn = nn.GRU(
            embed_size + num_hiddens, num_hiddens, num_layers,
            dropout=dropout)
        self.dense = nn.Linear(num_hiddens, vocab_size)

```

```

def init_state(self, enc_outputs, enc_valid_lens, *args):
    # outputs的形状为(batch_size, num_steps, num_hiddens).
    # hidden_state的形状为(num_layers, batch_size, num_hiddens)
    outputs, hidden_state = enc_outputs
    return (outputs.permute(1, 0, 2), hidden_state, enc_valid_lens)

def forward(self, X, state):
    # enc_outputs的形状为(batch_size,num_steps,num_hiddens).
    # hidden_state的形状为(num_layers,batch_size,
    # num_hiddens)
    enc_outputs, hidden_state, enc_valid_lens = state
    # 输出X的形状为(num_steps,batch_size,embed_size)
    X = self.embedding(X).permute(1, 0, 2)
    outputs, self._attention_weights = [], []
    for x in X:
        # query的形状为(batch_size,1,num_hiddens)
        query = torch.unsqueeze(hidden_state[-1], dim=1)
        # context的形状为(batch_size,1,num_hiddens)
        context = self.attention(
            query, enc_outputs, enc_valid_lens)

```

```

# 在特征维度上连接
x = torch.cat((context, torch.unsqueeze(x, dim=1)), dim=-1)
# 将x变形为(1,batch_size,embed_size+num_hiddens)
out, hidden_state = self.rnn(x.permute(1, 0, 2), hidden_state)
outputs.append(out)
self._attention_weights.append(self.attention.attention_weights)
# 全连接层变换后, outputs的形状为
# (num_steps,batch_size,vocab_size)
outputs = self.dense(torch.cat(outputs, dim=0))
return outputs.permute(1, 0, 2), [enc_outputs, hidden_state,
                                enc_valid_lens]

@property
def attention_weights(self):
    return self._attention_weights

# From chapter_10_attention_mechanisms/multihead_attention.py
class MultiHeadAttention(nn.Module):
    """多头注意力"""
    def __init__(self, key_size, query_size, value_size, num_hiddens,
                 num_heads, dropout, bias=False, **kwargs):
        super(MultiHeadAttention, self).__init__(**kwargs)
        self.num_heads = num_heads
        self.attention = native.DotProductAttention(dropout)
        self.W_q = nn.Linear(query_size, num_hiddens, bias=bias)
        self.W_k = nn.Linear(key_size, num_hiddens, bias=bias)
        self.W_v = nn.Linear(value_size, num_hiddens, bias=bias)
        self.W_o = nn.Linear(num_hiddens, num_hiddens, bias=bias)

    def forward(self, queries, keys, values, valid_lens):
        # queries, keys, values的形状:
        # (batch_size, 查询或者“键-值”对的个数, num_hiddens)
        # valid_lens 的形状:
        # (batch_size, )或(batch_size, 查询的个数)
        # 经过变换后, 输出的queries, keys, values 的形状:
        # (batch_size*num_heads, 查询或者“键-值”对的个数,
        #   num_hiddens/num_heads)
        queries = transpose_qkv(self.W_q(queries), self.num_heads)
        keys = transpose_qkv(self.W_k(keys), self.num_heads)
        values = transpose_qkv(self.W_v(values), self.num_heads)

        if valid_lens is not None:
            # 在轴0, 将第一项 (标量或者向量) 复制num_heads次,
            # 然后如此复制第二项, 然后诸如此类。
            valid_lens = torch.repeat_interleave(
                valid_lens, repeats=self.num_heads, dim=0)

        # output的形状:(batch_size*num_heads, 查询的个数,
        #   num_hiddens/num_heads)
        output = self.attention(queries, keys, values, valid_lens)

        # output_concat的形状:(batch_size, 查询的个数, num_hiddens)
        output_concat = transpose_output(output, self.num_heads)
        return self.W_o(output_concat)

# From chapter_10_attention_mechanisms/multihead_attention.py
def transpose_qkv(X, num_heads):
    """为了多注意力头的并行计算而变换形状"""
    # 输入X的形状:(batch_size, 查询或者“键-值”对的个数, num_hiddens)
    # 输出X的形状:(batch_size, 查询或者“键-值”对的个数, num_heads,
    #   num_hiddens/num_heads)
    X = X.reshape(X.shape[0], X.shape[1], num_heads, -1)

    # 输出X的形状:(batch_size, num_heads, 查询或者“键-值”对的个数,
    #   num_hiddens/num_heads)
    X = X.permute(0, 2, 1, 3)

    # 最终输出的形状:(batch_size*num_heads,查询或者“键-值”对的个数,
    #   num_hiddens/num_heads)
    return X.reshape(-1, X.shape[2], X.shape[3])

# From chapter_10_attention_mechanisms/multihead_attention.py
def transpose_output(X, num_heads):
    """逆转transpose_qkv函数的操作"""
    X = X.reshape(-1, num_heads, X.shape[1], X.shape[2])
    X = X.permute(0, 2, 1, 3)
    return X.reshape(X.shape[0], X.shape[1], -1)

```

```

# From chapter_10_attention_mechanisms/nadaraya_waston.py
def f(x):
    return 2 * native.sin(x) + x**0.8

# From chapter_10_attention_mechanisms/nadaraya_waston.py
def plot_kernel_reg(y_hat):
    native.plot(x_test, [y_truth, y_hat], 'x', 'y', legend=['Truth', 'Pred'],
                xlim=[0, 5], ylim=[-1, 5])
    native.plt.plot(x_train, y_train, 'o', alpha=0.5);

# From chapter_10_attention_mechanisms/nadaraya_waston.py
class NWKernelRegression(nn.Module):
    def __init__(self, **kwargs):
        super().__init__(**kwargs)
        self.w = nn.Parameter(torch.rand((1,), requires_grad=True))

    def forward(self, queries, keys, values):
        # queries和attention_weights的形状为(查询个数, "键-值"对个数)
        queries = native.reshape(
            queries.repeat_interleave(keys.shape[1]), (-1, keys.shape[1]))
        self.attention_weights = nn.functional.softmax(
            -((queries - keys) * self.w)**2 / 2, dim=1)
        # values的形状为(查询个数, "键-值"对个数)
        return torch.bmm(self.attention_weights.unsqueeze(1),
                        values.unsqueeze(-1)).reshape(-1)

# From chapter_10_attention_mechanisms/self_attention_and_positional_encoding.py
class PositionalEncoding(nn.Module):
    """位置编码"""
    def __init__(self, num_hiddens, dropout, max_len=1000):
        super(PositionalEncoding, self).__init__()
        self.dropout = nn.Dropout(dropout)
        # 创建一个足够长的P
        self.P = native.zeros((1, max_len, num_hiddens))
        X = native.arange(max_len, dtype=torch.float32).reshape(
            -1, 1) / torch.pow(10000, torch.arange(
                0, num_hiddens, 2, dtype=torch.float32) / num_hiddens)
        self.P[:, :, 0::2] = torch.sin(X)
        self.P[:, :, 1::2] = torch.cos(X)

    def forward(self, X):
        X = X + self.P[:, :X.shape[1], :].to(X.device)
        return self.dropout(X)

# From chapter_10_attention_mechanisms/transformer.py
class PositionWiseFFN(nn.Module):
    """基于位置的前馈网络"""
    def __init__(self, ffn_num_input, ffn_num_hiddens, ffn_num_outputs,
                 **kwargs):
        super(PositionWiseFFN, self).__init__(**kwargs)
        self.dense1 = nn.Linear(ffn_num_input, ffn_num_hiddens)
        self.relu = nn.ReLU()
        self.dense2 = nn.Linear(ffn_num_hiddens, ffn_num_outputs)

    def forward(self, X):
        return self.dense2(self.relu(self.dense1(X)))

# From chapter_10_attention_mechanisms/transformer.py
class AddNorm(nn.Module):
    """残差连接后进行层规范化"""
    def __init__(self, normalized_shape, dropout, **kwargs):
        super(AddNorm, self).__init__(**kwargs)
        self.dropout = nn.Dropout(dropout)
        self.ln = nn.LayerNorm(normalized_shape)

    def forward(self, X, Y):
        return self.ln(self.dropout(Y) + X)

# From chapter_10_attention_mechanisms/transformer.py
class EncoderBlock(nn.Module):
    """Transformer编码器块"""
    def __init__(self, key_size, query_size, value_size, num_hiddens,
                 norm_shape, ffn_num_input, ffn_num_hiddens, num_heads,
                 dropout, use_bias=False, **kwargs):
        super(EncoderBlock, self).__init__(**kwargs)
        self.attention = native.MultiHeadAttention(

```

```

        key_size, query_size, value_size, num_hiddens, num_heads, dropout,
        use_bias)
    self.addnorm1 = AddNorm(norm_shape, dropout)
    self.ffn = PositionWiseFFN(
        ffn_num_input, ffn_num_hiddens, num_hiddens)

    self.addnorm2 = AddNorm(norm_shape, dropout)

def forward(self, X, valid_lens):
    Y = self.addnorm1(X, self.attention(X, X, X, valid_lens))
    return self.addnorm2(Y, self.ffn(Y))

# From chapter_10_attention_mechanisms/transformer.py
class TransformerEncoder(native.Encoder):
    """Transformer编码器"""
    def __init__(self, vocab_size, key_size, query_size, value_size,
                  num_hiddens, norm_shape, ffn_num_input, ffn_num_hiddens,
                  num_heads, num_layers, dropout, use_bias=False, **kwargs):
        super(TransformerEncoder, self).__init__(**kwargs)
        self.num_hiddens = num_hiddens
        self.embedding = nn.Embedding(vocab_size, num_hiddens)
        self.pos_encoding = native.PositionalEncoding(num_hiddens, dropout)
        self.blks = nn.Sequential()
        for i in range(num_layers):
            self.blks.add_module("block"+str(i),
                                  EncoderBlock(key_size, query_size, value_size, num_hiddens,
                                                norm_shape, ffn_num_input, ffn_num_hiddens,
                                                num_heads, dropout, use_bias))

def forward(self, X, valid_lens, *args):
    # 因为位置编码值在-1和1之间,
    # 因此嵌入值乘以嵌入维度的平方根进行缩放,
    # 然后再与位置编码相加。
    X = self.pos_encoding(self.embedding(X) * math.sqrt(self.num_hiddens))
    self.attention_weights = [None] * len(self.blks)
    for i, blk in enumerate(self.blks):
        X = blk(X, valid_lens)
        self.attention_weights[
            i] = blk.attention.attention.attention_weights
    return X

# From chapter_10_attention_mechanisms/transformer.py
class DecoderBlock(nn.Module):
    """解码器中第i个块"""
    def __init__(self, key_size, query_size, value_size, num_hiddens,
                  norm_shape, ffn_num_input, ffn_num_hiddens, num_heads,
                  dropout, i, **kwargs):
        super(DecoderBlock, self).__init__(**kwargs)
        self.i = i
        self.attention1 = native.MultiHeadAttention(
            key_size, query_size, value_size, num_hiddens, num_heads, dropout)
        self.addnorm1 = AddNorm(norm_shape, dropout)
        self.attention2 = native.MultiHeadAttention(
            key_size, query_size, value_size, num_hiddens, num_heads, dropout)
        self.addnorm2 = AddNorm(norm_shape, dropout)
        self.ffn = PositionWiseFFN(ffn_num_input, ffn_num_hiddens,
                                    num_hiddens)
        self.addnorm3 = AddNorm(norm_shape, dropout)

def forward(self, X, state):
    enc_outputs, enc_valid_lens = state[0], state[1]
    # 训练阶段, 输出序列的所有词元都在同一时间处理,
    # 因此state[2][self.i]初始化为None。
    # 预测阶段, 输出序列是通过词元一个接着一个解码的,
    # 因此state[2][self.i]包含着直到当前时间步第i个块解码的输出表示
    if state[2][self.i] is None:
        key_values = X
    else:
        key_values = torch.cat((state[2][self.i], X), axis=1)
    state[2][self.i] = key_values
    if self.training:
        batch_size, num_steps, _ = X.shape
        # dec_valid_lens的开头:(batch_size,num_steps),
        # 其中每一行是[1,2,...,num_steps]
        dec_valid_lens = torch.arange(
            1, num_steps + 1, device=X.device).repeat(batch_size, 1)
    else:

```

```

        dec_valid_lens = None

# 自注意力
    X2 = self.attention1(X, key_values, key_values, dec_valid_lens)
    Y = self.addnorm1(X, X2)
# 编码器-解码器注意力。
# enc_outputs的开头:(batch_size,num_steps,num_hiddens)
    Y2 = self.attention2(Y, enc_outputs, enc_outputs, enc_valid_lens)
    Z = self.addnorm2(Y, Y2)
    return self.addnorm3(Z, self.ffn(Z)), state

# From chapter_10_attention_mechanisms/transformer.py
class TransformerDecoder(native.AttentionDecoder):
    def __init__(self, vocab_size, key_size, query_size, value_size,
                  num_hiddens, norm_shape, ffn_num_input, ffn_num_hiddens,
                  num_heads, num_layers, dropout, **kwargs):
        super(TransformerDecoder, self).__init__(**kwargs)
        self.num_hiddens = num_hiddens
        self.num_layers = num_layers
        self.embedding = nn.Embedding(vocab_size, num_hiddens)
        self.pos_encoding = native.PositionalEncoding(num_hiddens, dropout)
        self.blks = nn.Sequential()
        for i in range(num_layers):
            self.blks.add_module("block"+str(i),
                                  DecoderBlock(key_size, query_size, value_size, num_hiddens,
                                                  norm_shape, ffn_num_input, ffn_num_hiddens,
                                                  num_heads, dropout, i))
        self.dense = nn.Linear(num_hiddens, vocab_size)

    def init_state(self, enc_outputs, enc_valid_lens, *args):
        return [enc_outputs, enc_valid_lens, [None] * self.num_layers]

    def forward(self, X, state):
        X = self.pos_encoding(self.embedding(X) * math.sqrt(self.num_hiddens))
        self._attention_weights = [[None] * len(self.blks) for _ in range(2)]
        for i, blk in enumerate(self.blks):
            X, state = blk(X, state)
# 解码器自注意力权重
            self._attention_weights[0][
                i] = blk.attention1.attention.attention_weights
# “编码器-解码器”自注意力权重
            self._attention_weights[1][
                i] = blk.attention2.attention.attention_weights
        return self.dense(X), state

@property
    def attention_weights(self):
        return self._attention_weights

# From chapter_11_optimization/adadelatapy
def init_adadelat_states(feature_dim):
    s_w, s_b = native.zeros((feature_dim, 1)), native.zeros(1)
    delta_w, delta_b = native.zeros((feature_dim, 1)), native.zeros(1)
    return ((s_w, delta_w), (s_b, delta_b))

# From chapter_11_optimization/adadelatapy
def adadelat(params, states, hyperparams):
    rho, eps = hyperparams['rho'], 1e-5
    for p, (s, delta) in zip(params, states):
        with torch.no_grad():
            # In-place updates via [:]
            s[:] = rho * s + (1 - rho) * torch.square(p.grad)
            g = (torch.sqrt(delta + eps) / torch.sqrt(s + eps)) * p.grad
            p[:] -= g
            delta[:] = rho * delta + (1 - rho) * g * g
        p.grad.data.zero_()

# From chapter_11_optimization/adagrad.py
def adagrad_2d(x1, x2, s1, s2):
    eps = 1e-6
    g1, g2 = 0.2 * x1, 4 * x2
    s1 += g1 ** 2
    s2 += g2 ** 2
    x1 -= eta / math.sqrt(s1 + eps) * g1
    x2 -= eta / math.sqrt(s2 + eps) * g2
    return x1, x2, s1, s2

```

```

# From chapter_11_optimization/adagrad.py
def f_2d(x1, x2):
    return 0.1 * x1 ** 2 + 2 * x2 ** 2

# From chapter_11_optimization/adagrad.py
def init_adagrad_states(feature_dim):
    s_w = native.zeros((feature_dim, 1))
    s_b = native.zeros(1)
    return (s_w, s_b)

# From chapter_11_optimization/adagrad.py
def adagrad(params, states, hyperparams):
    eps = 1e-6
    for p, s in zip(params, states):
        with torch.no_grad():
            s[:] += torch.square(p.grad)
            p[:] -= hyperparams['lr'] * p.grad / torch.sqrt(s + eps)
        p.grad.data.zero_()

# From chapter_11_optimization/adam.py
def init_adam_states(feature_dim):
    v_w, v_b = native.zeros((feature_dim, 1)), native.zeros(1)
    s_w, s_b = native.zeros((feature_dim, 1)), native.zeros(1)
    return ((v_w, s_w), (v_b, s_b))

# From chapter_11_optimization/adam.py
def adam(params, states, hyperparams):
    beta1, beta2, eps = 0.9, 0.999, 1e-6
    for p, (v, s) in zip(params, states):
        with torch.no_grad():
            v[:] = beta1 * v + (1 - beta1) * p.grad
            s[:] = beta2 * s + (1 - beta2) * torch.square(p.grad)

            v_bias_corr = v / (1 - beta1 ** hyperparams['t'])
            s_bias_corr = s / (1 - beta2 ** hyperparams['t'])
            p[:] -= hyperparams['lr'] * v_bias_corr / (torch.sqrt(s_bias_corr)
                                                    + eps)

        p.grad.data.zero_()
        hyperparams['t'] += 1

# From chapter_11_optimization/adam.py
def yogi(params, states, hyperparams):
    beta1, beta2, eps = 0.9, 0.999, 1e-3
    for p, (v, s) in zip(params, states):
        with torch.no_grad():
            v[:] = beta1 * v + (1 - beta1) * p.grad
            s[:] = s + (1 - beta2) * torch.sign(
                torch.square(p.grad) - s) * torch.square(p.grad)
            v_bias_corr = v / (1 - beta1 ** hyperparams['t'])
            s_bias_corr = s / (1 - beta2 ** hyperparams['t'])
            p[:] -= hyperparams['lr'] * v_bias_corr / (torch.sqrt(s_bias_corr)
                                                    + eps)

        p.grad.data.zero_()
        hyperparams['t'] += 1

# From chapter_11_optimization/gd.py
def f(x): # 目标函数
    return x ** 2

# From chapter_11_optimization/gd.py
def f_grad(x): # 目标函数的梯度(导数)
    return 2 * x

# From chapter_11_optimization/gd.py
def gd(eta, f_grad):
    x = 10.0
    results = [x]
    for i in range(10):
        x -= eta * f_grad(x)
        results.append(float(x))
    print(f'epoch 10, x: {x:f}')
    return results

# From chapter_11_optimization/gd.py
def show_trace(results, f):
    n = max(abs(min(results)), abs(max(results)))
    f_line = native.arange(-n, n, 0.01)
    native.set_figsize()

```

```

    native.plot([f_line, results], [[f(x) for x in f_line], [
        f(x) for x in results]], 'x', 'f(x)', fmts=['-', '-o'])

# From chapter_11_optimization/gd.py
def f(x): # 目标函数
    return x * native.cos(c * x)

# From chapter_11_optimization/gd.py
def f_grad(x): # 目标函数的梯度
    return native.cos(c * x) - c * x * native.sin(c * x)

# From chapter_11_optimization/gd.py
def train_2d(trainer, steps=20, f_grad=None): # @save
    """用定制的训练机优化2D目标函数"""
    # s1和s2是稍后将使用的内部状态变量
    x1, x2, s1, s2 = -5, -2, 0, 0
    results = [(x1, x2)]
    for i in range(steps):
        if f_grad:
            x1, x2, s1, s2 = trainer(x1, x2, s1, s2, f_grad)
        else:
            x1, x2, s1, s2 = trainer(x1, x2, s1, s2)
        results.append((x1, x2))
    print(f'epoch {i + 1}, x1: {float(x1):f}, x2: {float(x2):f}')
    return results

# From chapter_11_optimization/gd.py
def show_trace_2d(f, results): # @save
    """显示优化过程中2D变量的轨迹"""
    native.set_figsize()
    native.plt.plot(*zip(*results), '-o', color='#ff7f0e')
    x1, x2 = native.meshgrid(native.arange(-5.5, 1.0, 0.1),
                             native.arange(-3.0, 1.0, 0.1))
    native.plt.contour(x1, x2, f(x1, x2), colors='#1f77b4')
    native.plt.xlabel('x1')
    native.plt.ylabel('x2')

# From chapter_11_optimization/gd.py
def f_2d(x1, x2): # 目标函数
    return x1 ** 2 + 2 * x2 ** 2

# From chapter_11_optimization/gd.py
def f_2d_grad(x1, x2): # 目标函数的梯度
    return (2 * x1, 4 * x2)

# From chapter_11_optimization/gd.py
def gd_2d(x1, x2, s1, s2, f_grad):
    g1, g2 = f_grad(x1, x2)
    return (x1 - eta * g1, x2 - eta * g2, 0, 0)

# From chapter_11_optimization/gd.py
def f(x): # 0目标函数
    return native.cosh(c * x)

# From chapter_11_optimization/gd.py
def f_grad(x): # 目标函数的梯度
    return c * native.sinh(c * x)

# From chapter_11_optimization/gd.py
def f_hess(x): # 目标函数的Hessian
    return c**2 * native.cosh(c * x)

# From chapter_11_optimization/gd.py
def newton(eta=1):
    x = 10.0
    results = [x]
    for i in range(10):
        x -= eta * f_grad(x) / f_hess(x)
        results.append(float(x))
    print('epoch 10, x:', x)
    return results

# From chapter_11_optimization/gd.py
def f(x): # 目标函数
    return x * native.cos(c * x)

```

```

# From chapter_11_optimization/gd.py
def f_grad(x): # 目标函数的梯度
    return native.cos(c * x) - c * x * native.sin(c * x)

# From chapter_11_optimization/gd.py
def f_hess(x): # 目标函数的Hessian
    return - 2 * c * native.sin(c * x) - x * c**2 * native.cos(c * x)

# From chapter_11_optimization/lr_scheduler.py
def net_fn():
    model = nn.Sequential(
        nn.Conv2d(1, 6, kernel_size=5, padding=2), nn.ReLU(),
        nn.MaxPool2d(kernel_size=2, stride=2),
        nn.Conv2d(6, 16, kernel_size=5), nn.ReLU(),
        nn.MaxPool2d(kernel_size=2, stride=2),
        nn.Flatten(),
        nn.Linear(16 * 5 * 5, 120), nn.ReLU(),
        nn.Linear(120, 84), nn.ReLU(),
        nn.Linear(84, 10))

    return model

# From chapter_11_optimization/lr_scheduler.py
def train(net, train_iter, test_iter, num_epochs, loss, trainer, device,
        scheduler=None):
    net.to(device)
    animator = native.Animator(xlabel='epoch', xlim=[0, num_epochs],
        legend=['train loss', 'train acc', 'test acc'])

    for epoch in range(num_epochs):
        metric = native.Accumulator(3) # train_loss, train_acc, num_examples
        for i, (X, y) in enumerate(train_iter):
            net.train()
            trainer.zero_grad()
            X, y = X.to(device), y.to(device)
            y_hat = net(X)
            l = loss(y_hat, y)
            l.backward()
            trainer.step()
            with torch.no_grad():
                metric.add(l * X.shape[0], native.accuracy(y_hat, y), X.shape[0])
            train_loss = metric[0] / metric[2]
            train_acc = metric[1] / metric[2]
            if (i + 1) % 50 == 0:
                animator.add(epoch + i / len(train_iter),
                    (train_loss, train_acc, None))

        test_acc = native.evaluate_accuracy_gpu(net, test_iter)
        animator.add(epoch+1, (None, None, test_acc))

        if scheduler:
            if scheduler.__module__ == lr_scheduler.__name__:
                # UsingPyTorchIn-BuiltScheduler
                scheduler.step()
            else:
                # Usingcustomdefinedscheduler
                for param_group in trainer.param_groups:
                    param_group['lr'] = scheduler(epoch)

    print(f'train loss {train_loss:.3f}, train acc {train_acc:.3f}, '
        f'test acc {test_acc:.3f}')

# From chapter_11_optimization/lr_scheduler.py
class SquareRootScheduler:
    def __init__(self, lr=0.1):
        self.lr = lr

    def __call__(self, num_update):
        return self.lr * pow(num_update + 1.0, -0.5)

# From chapter_11_optimization/lr_scheduler.py
class FactorScheduler:
    def __init__(self, factor=1, stop_factor_lr=1e-7, base_lr=0.1):
        self.factor = factor
        self.stop_factor_lr = stop_factor_lr
        self.base_lr = base_lr

```

```

def __call__(self, num_update):
    self.base_lr = max(self.stop_factor_lr, self.base_lr * self.factor)
    return self.base_lr

# From chapter_11_optimization/lr_scheduler.py
def get_lr(trainer, scheduler):
    lr = scheduler.get_last_lr()[0]
    trainer.step()
    scheduler.step()
    return lr

# From chapter_11_optimization/lr_scheduler.py
class CosineScheduler:
    def __init__(self, max_update, base_lr=0.01, final_lr=0,
                 warmup_steps=0, warmup_begin_lr=0):
        self.base_lr_orig = base_lr
        self.max_update = max_update
        self.final_lr = final_lr
        self.warmup_steps = warmup_steps
        self.warmup_begin_lr = warmup_begin_lr
        self.max_steps = self.max_update - self.warmup_steps

    def get_warmup_lr(self, epoch):
        increase = (self.base_lr_orig - self.warmup_begin_lr) \
            * float(epoch) / float(self.warmup_steps)
        return self.warmup_begin_lr + increase

    def __call__(self, epoch):
        if epoch < self.warmup_steps:
            return self.get_warmup_lr(epoch)
        if epoch <= self.max_update:
            self.base_lr = self.final_lr + (
                self.base_lr_orig - self.final_lr) * (1 + math.cos(
                    math.pi * (epoch - self.warmup_steps) / self.max_steps)) / 2
            return self.base_lr

# From chapter_11_optimization/minibatch_sgd.py
def get_data_ch11(batch_size=10, n=1500):
    data = np.genfromtxt(native.download('airfoil'),
                        dtype=np.float32, delimiter='\t')
    data = torch.from_numpy((data - data.mean(axis=0)) / data.std(axis=0))
    data_iter = native.load_array((data[:n, :-1], data[:n, -1]),
                                   batch_size, is_train=True)
    return data_iter, data.shape[1]-1

# From chapter_11_optimization/minibatch_sgd.py
def sgd(params, states, hyperparams):
    for p in params:
        p.data.sub_(hyperparams['lr'] * p.grad)
        p.grad.data.zero_()

# From chapter_11_optimization/minibatch_sgd.py
def train_ch11(trainer_fn, states, hyperparams, data_iter,
               feature_dim, num_epochs=2):
    # 初始化模型
    w = torch.normal(mean=0.0, std=0.01, size=(feature_dim, 1),
                     requires_grad=True)
    b = torch.zeros((1), requires_grad=True)
    net, loss = lambda X: native.linreg(X, w, b), native.squared_loss
    # 训练模型
    animator = native.Animator(xlabel='epoch', ylabel='loss',
                              xlim=[0, num_epochs], ylim=[0.22, 0.35])
    n, timer = 0, native.Timer()
    for _ in range(num_epochs):
        for X, y in data_iter:
            l = loss(net(X), y).mean()
            l.backward()
            trainer_fn([w, b], states, hyperparams)
            n += X.shape[0]
            if n % 200 == 0:
                timer.stop()
                animator.add(n/X.shape[0]/len(data_iter),
                             (native.evaluate_loss(net, data_iter, loss),))
                timer.start()
    print(f'loss: {animator.Y[0][-1]:.3f}, {timer.avg():.3f} sec/epoch')
    return timer.cumsum(), animator.Y[0]

```

```

# From chapter_11_optimization/minibatch_sgd.py
def train_sgd(lr, batch_size, num_epochs=2):
    data_iter, feature_dim = get_data_ch11(batch_size)
    return train_ch11(
        sgd, None, {'lr': lr}, data_iter, feature_dim, num_epochs)

# From chapter_11_optimization/minibatch_sgd.py
def train_concise_ch11(trainer_fn, hyperparams, data_iter, num_epochs=4):
    # 初始化模型
    net = nn.Sequential(nn.Linear(5, 1))
    def init_weights(m):
        if type(m) == nn.Linear:
            torch.nn.init.normal_(m.weight, std=0.01)
    net.apply(init_weights)

    optimizer = trainer_fn(net.parameters(), **hyperparams)
    loss = nn.MSELoss(reduction='none')
    animator = native.Animator(xlabel='epoch', ylabel='loss',
                               xlim=[0, num_epochs], ylim=[0.22, 0.35])
    n, timer = 0, native.Timer()
    for _ in range(num_epochs):
        for X, y in data_iter:
            optimizer.zero_grad()
            out = net(X)
            y = y.reshape(out.shape)

            l = loss(out, y)
            l.mean().backward()
            optimizer.step()
            n += X.shape[0]
            if n % 200 == 0:
                timer.stop()
        # MSELoss计算平方误差时不带系数1/2
        animator.add(n/X.shape[0]/len(data_iter),
                    (native.evaluate_loss(net, data_iter, loss) / 2,))
        timer.start()
    print(f'loss: {animator.Y[0][-1]:.3f}, {timer.avg():.3f} sec/epoch')

# From chapter_11_optimization/momentum.py
def f_2d(x1, x2):
    return 0.1 * x1 ** 2 + 2 * x2 ** 2

# From chapter_11_optimization/momentum.py
def gd_2d(x1, x2, s1, s2):
    return (x1 - eta * 0.2 * x1, x2 - eta * 4 * x2, 0, 0)

# From chapter_11_optimization/momentum.py
def momentum_2d(x1, x2, v1, v2):
    v1 = beta * v1 + 0.2 * x1
    v2 = beta * v2 + 4 * x2
    return x1 - eta * v1, x2 - eta * v2, v1, v2

# From chapter_11_optimization/momentum.py
def init_momentum_states(feature_dim):
    v_w = native.zeros((feature_dim, 1))
    v_b = native.zeros(1)
    return (v_w, v_b)

# From chapter_11_optimization/momentum.py
def sgd_momentum(params, states, hyperparams):
    for p, v in zip(params, states):
        with torch.no_grad():
            v[:] = hyperparams['momentum'] * v + p.grad
            p[:] -= hyperparams['lr'] * v
        p.grad.data.zero_()

# From chapter_11_optimization/momentum.py
def train_momentum(lr, momentum, num_epochs=2):
    native.train_ch11(sgd_momentum, init_momentum_states(feature_dim),
                      {'lr': lr, 'momentum': momentum}, data_iter,
                      feature_dim, num_epochs)

# From chapter_11_optimization/optimization_intro.py
def f(x):
    return x * native.cos(np.pi * x)

# From chapter_11_optimization/optimization_intro.py
def g(x):

```

```

    return f(x) + 0.2 * native.cos(5 * np.pi * x)

# From chapter_11_optimization/optimization_intro.py
def annotate(text, xy, xytext): #@save
    native.plt.gca().annotate(text, xy=xy, xytext=xytext,
                              arrowprops=dict(arrowstyle='->'))

# From chapter_11_optimization/rmsprop.py
def rmsprop_2d(x1, x2, s1, s2):
    g1, g2, eps = 0.2 * x1, 4 * x2, 1e-6
    s1 = gamma * s1 + (1 - gamma) * g1 ** 2
    s2 = gamma * s2 + (1 - gamma) * g2 ** 2
    x1 -= eta / math.sqrt(s1 + eps) * g1
    x2 -= eta / math.sqrt(s2 + eps) * g2
    return x1, x2, s1, s2

# From chapter_11_optimization/rmsprop.py
def f_2d(x1, x2):
    return 0.1 * x1 ** 2 + 2 * x2 ** 2

# From chapter_11_optimization/rmsprop.py
def init_rmsprop_states(feature_dim):
    s_w = native.zeros((feature_dim, 1))
    s_b = native.zeros(1)
    return (s_w, s_b)

# From chapter_11_optimization/rmsprop.py
def rmsprop(params, states, hyperparams):
    gamma, eps = hyperparams['gamma'], 1e-6
    for p, s in zip(params, states):
        with torch.no_grad():
            s[:] = gamma * s + (1 - gamma) * torch.square(p.grad)
            p[:] -= hyperparams['lr'] * p.grad / torch.sqrt(s + eps)
            p.grad.data.zero_()

# From chapter_11_optimization/sgd.py
def f(x1, x2): # 目标函数
    return x1 ** 2 + 2 * x2 ** 2

# From chapter_11_optimization/sgd.py
def f_grad(x1, x2): # 目标函数的梯度
    return 2 * x1, 4 * x2

# From chapter_11_optimization/sgd.py
def sgd(x1, x2, s1, s2, f_grad):
    g1, g2 = f_grad(x1, x2)
    # 模拟有噪声的梯度
    g1 += native.normal(0.0, 1, (1,))
    g2 += native.normal(0.0, 1, (1,))
    eta_t = eta * lr()
    return (x1 - eta_t * g1, x2 - eta_t * g2, 0, 0)

# From chapter_11_optimization/sgd.py
def constant_lr():
    return 1

# From chapter_11_optimization/sgd.py
def exponential_lr():
    # 在函数外部定义，而在内部更新的全局变量
    global t
    t += 1
    return math.exp(-0.1 * t)

# From chapter_11_optimization/sgd.py
def polynomial_lr():
    # 在函数外部定义，而在内部更新的全局变量
    global t
    t += 1

    return (1 + 0.1 * t) ** (-0.5)

# From chapter_12_computational_performance/auto_parallelism.py
def run(x):
    return [x.mm(x) for _ in range(50)]

# From chapter_12_computational_performance/auto_parallelism.py
def copy_to_cpu(x, non_blocking=False):

```

```

        return [y.to('cpu', non_blocking=non_blocking) for y in x]

# From chapter_12_computational_performance/hybridize.py
def add(a, b):
    return a + b

# From chapter_12_computational_performance/hybridize.py
def fancy_func(a, b, c, d):
    e = add(a, b)
    f = add(c, d)
    g = add(e, f)
    return g

# From chapter_12_computational_performance/hybridize.py
def add_():
    return ''
def add(a, b):
    return a + b
...

# From chapter_12_computational_performance/hybridize.py
def fancy_func_():
    return ''
def fancy_func(a, b, c, d):
    e = add(a, b)
    f = add(c, d)
    g = add(e, f)
    return g
...

# From chapter_12_computational_performance/hybridize.py
def evoke_():
    return add_() + fancy_func_() + 'print(fancy_func(1, 2, 3, 4))'

# From chapter_12_computational_performance/hybridize.py
def get_net():
    net = nn.Sequential(nn.Linear(512, 256),
                        nn.ReLU(),
                        nn.Linear(256, 128),
                        nn.ReLU(),
                        nn.Linear(128, 2))
    return net

# From chapter_12_computational_performance/hybridize.py
class Benchmark:
    """用于测量运行时间"""
    def __init__(self, description='Done'):
        self.description = description

    def __enter__(self):
        self.timer = native.Timer()
        return self

    def __exit__(self, *args):
        print(f'{self.description}: {self.timer.stop():.4f} sec')

# From chapter_12_computational_performance/multiple_gpus.py
def lenet(X, params):
    h1_conv = F.conv2d(input=X, weight=params[0], bias=params[1])
    h1_activation = F.relu(h1_conv)
    h1 = F.avg_pool2d(input=h1_activation, kernel_size=(2, 2), stride=(2, 2))
    h2_conv = F.conv2d(input=h1, weight=params[2], bias=params[3])
    h2_activation = F.relu(h2_conv)
    h2 = F.avg_pool2d(input=h2_activation, kernel_size=(2, 2), stride=(2, 2))
    h2 = h2.reshape(h2.shape[0], -1)
    h3_linear = torch.mm(h2, params[4]) + params[5]
    h3 = F.relu(h3_linear)
    y_hat = torch.mm(h3, params[6]) + params[7]
    return y_hat

# From chapter_12_computational_performance/multiple_gpus.py
def get_params(params, device):
    new_params = [p.to(device) for p in params]
    for p in new_params:
        p.requires_grad_()
    return new_params

```

```

# From chapter_12_computational_performance/multiple_gpus.py
def allreduce(data):
    for i in range(1, len(data)):
        data[0][:] += data[i].to(data[0].device)
    for i in range(1, len(data)):
        data[i][:] = data[0].to(data[i].device)

# From chapter_12_computational_performance/multiple_gpus.py
def split_batch(X, y, devices):
    """将X和y拆分到多个设备上"""
    assert X.shape[0] == y.shape[0]
    return (nn.parallel.scatter(X, devices),
            nn.parallel.scatter(y, devices))

# From chapter_12_computational_performance/multiple_gpus.py
def train_batch(X, y, device_params, devices, lr):
    X_shards, y_shards = split_batch(X, y, devices)
    # 在每个GPU上分别计算损失
    ls = [loss(lenet(X_shard, device_W), y_shard).sum()
           for X_shard, y_shard, device_W in zip(
               X_shards, y_shards, device_params)]
    for l in ls: # 反向传播在每个GPU上分别执行
        l.backward()
    # 将每个GPU的所有梯度相加, 并将其广播到所有GPU
    with torch.no_grad():
        for i in range(len(device_params[0])):
            allreduce(
                [device_params[c][i].grad for c in range(len(devices))])
    # 在每个GPU上分别更新模型参数
    for param in device_params:
        native.sgd(param, lr, X.shape[0])

# From chapter_12_computational_performance/multiple_gpus.py
def train(num_gpus, batch_size, lr):
    train_iter, test_iter = native.load_data_fashion_mnist(batch_size)
    devices = [native.try_gpu(i) for i in range(num_gpus)]
    # 将模型参数复制到num_gpus个GPU
    device_params = [get_params(params, d) for d in devices]
    num_epochs = 10
    animator = native.Animator('epoch', 'test acc', xlim=[1, num_epochs])
    timer = native.Timer()
    for epoch in range(num_epochs):
        timer.start()
        for X, y in train_iter:
            # 为单个小批量执行多GPU训练
            train_batch(X, y, device_params, devices, lr)
            torch.cuda.synchronize()
        timer.stop()
        # 在GPU0上评估模型
        animator.add(epoch + 1, (native.evaluate_accuracy_gpu(
            lambda x: lenet(x, device_params[0]), test_iter, devices[0]),))
    print(f'测试精度: {animator.Y[0][-1]:.2f}, {timer.avg():.1f}秒/轮, '
          f'在{str(devices)}')

# From chapter_12_computational_performance/multiple_gpus_concise.py
def resnet18(num_classes, in_channels=1):
    """稍加修改的ResNet-18模型"""
    def resnet_block(in_channels, out_channels, num_residuals,
                     first_block=False):
        blk = []
        for i in range(num_residuals):
            if i == 0 and not first_block:
                blk.append(native.Residual(in_channels, out_channels,
                                           use_1x1conv=True, strides=2))
            else:
                blk.append(native.Residual(out_channels, out_channels))
        return nn.Sequential(*blk)

    # 该模型使用了更小的卷积核、步长和填充, 而且删除了最大汇聚层
    net = nn.Sequential(
        nn.Conv2d(in_channels, 64, kernel_size=3, stride=1, padding=1),
        nn.BatchNorm2d(64),
        nn.ReLU())
    net.add_module("resnet_block1", resnet_block(
        64, 64, 2, first_block=True))
    net.add_module("resnet_block2", resnet_block(64, 128, 2))
    net.add_module("resnet_block3", resnet_block(128, 256, 2))

```

```

net.add_module("resnet_block4", resnet_block(256, 512, 2))
net.add_module("global_avg_pool", nn.AdaptiveAvgPool2d((1,1)))
net.add_module("fc", nn.Sequential(nn.Flatten(),
                                   nn.Linear(512, num_classes)))

return net

# From chapter_12_computational_performance/multiple_gpus_concise.py
def train(net, num_gpus, batch_size, lr):
    train_iter, test_iter = native.load_data_fashion_mnist(batch_size)
    devices = [native.try_gpu(i) for i in range(num_gpus)]
    def init_weights(m):
        if type(m) in [nn.Linear, nn.Conv2d]:
            nn.init.normal_(m.weight, std=0.01)
    net.apply(init_weights)
    # 在多个GPU上设置模型
    net = nn.DataParallel(net, device_ids=devices)
    trainer = torch.optim.SGD(net.parameters(), lr)
    loss = nn.CrossEntropyLoss()

    timer, num_epochs = native.Timer(), 10
    animator = native.Animator('epoch', 'test acc', xlim=[1, num_epochs])
    for epoch in range(num_epochs):
        net.train()
        timer.start()
        for X, y in train_iter:
            trainer.zero_grad()
            X, y = X.to(devices[0]), y.to(devices[0])
            l = loss(net(X), y)
            l.backward()
            trainer.step()
        timer.stop()
        animator.add(epoch + 1, (native.evaluate_accuracy_gpu(net, test_iter),))
    print(f'测试精度: {animator.Y[0][-1]:.2f}, {timer.avg():.1f}秒/轮, '
          f'在{str(devices)}')

# From chapter_13_computer_vision/anchor.py
def multibox_prior(data, sizes, ratios):
    """生成以每个像素为中心具有不同形状的锚框"""
    in_height, in_width = data.shape[-2:]
    device, num_sizes, num_ratios = data.device, len(sizes), len(ratios)
    boxes_per_pixel = (num_sizes + num_ratios - 1)
    size_tensor = native.tensor(sizes, device=device)
    ratio_tensor = native.tensor(ratios, device=device)

    # 为了将锚点移动到像素的中心，需要设置偏移量。
    # 因为一个像素的高为1且宽为1，我们选择偏移我们的中心0.5
    offset_h, offset_w = 0.5, 0.5
    steps_h = 1.0 / in_height # 在y轴上缩放步长
    steps_w = 1.0 / in_width # 在x轴上缩放步长

    # 生成锚框的所有中心点
    center_h = (torch.arange(in_height, device=device) + offset_h) * steps_h
    center_w = (torch.arange(in_width, device=device) + offset_w) * steps_w
    shift_y, shift_x = torch.meshgrid(center_h, center_w, indexing='ij')
    shift_y, shift_x = shift_y.reshape(-1), shift_x.reshape(-1)

    # 生成“boxes_per_pixel”个高和宽，
    # 之后用于创建锚框的四角坐标(xmin,xmax,ymin,ymax)
    w = torch.cat((size_tensor * torch.sqrt(ratio_tensor[0]),
                  sizes[0] * torch.sqrt(ratio_tensor[1:]))\
                  * in_height / in_width # 处理矩形输入
    h = torch.cat((size_tensor / torch.sqrt(ratio_tensor[0]),
                  sizes[0] / torch.sqrt(ratio_tensor[1:]))
    # 除以2来获得半高和半宽
    anchor_manipulations = torch.stack((-w, -h, w, h)).T.repeat(
        in_height * in_width, 1) / 2

    # 每个中心点都将有“boxes_per_pixel”个锚框，
    # 所以生成含所有锚框中心的网格，重复了“boxes_per_pixel”次
    out_grid = torch.stack([shift_x, shift_y, shift_x, shift_y],
                          dim=1).repeat_interleave(boxes_per_pixel, dim=0)
    output = out_grid + anchor_manipulations
    return output.unsqueeze(0)

# From chapter_13_computer_vision/anchor.py
def show_bboxes(axes, bboxes, labels=None, colors=None):
    """显示所有边界框"""
    def _make_list(obj, default_values=None):

```

```

    if obj is None:
        obj = default_values
    elif not isinstance(obj, (list, tuple)):
        obj = [obj]
    return obj

labels = _make_list(labels)
colors = _make_list(colors, ['b', 'g', 'r', 'm', 'c'])
for i, bbox in enumerate(bboxes):
    color = colors[i % len(colors)]
    rect = native.bbox_to_rect(native.numpy(bbox), color)
    axes.add_patch(rect)
    if labels and len(labels) > i:
        text_color = 'k' if color == 'w' else 'w'
        axes.text(rect.xy[0], rect.xy[1], labels[i],
                  va='center', ha='center', fontsize=9, color=text_color,
                  bbox=dict(facecolor=color, lw=0))

# From chapter_13_computer_vision/anchor.py
def box_iou(boxes1, boxes2):
    """计算两个锚框或边界框列表中成对的交并比"""
    box_area = lambda boxes: ((boxes[:, 2] - boxes[:, 0]) *
                               (boxes[:, 3] - boxes[:, 1]))

    # boxes1, boxes2, areas1, areas2的形状:
    # boxes1: (boxes1的数量, 4),
    # boxes2: (boxes2的数量, 4),
    # areas1: (boxes1的数量, ),
    # areas2: (boxes2的数量, )
    areas1 = box_area(boxes1)
    areas2 = box_area(boxes2)

    # inter_upperlefts, inter_lowerrights, inters的形状:
    # (boxes1的数量, boxes2的数量, 2)
    inter_upperlefts = torch.max(boxes1[:, None, :2], boxes2[:, :2])
    inter_lowerrights = torch.min(boxes1[:, None, 2:], boxes2[:, 2:])
    inters = (inter_lowerrights - inter_upperlefts).clamp(min=0)

    # inter_areas and union_areas的形状: (boxes1的数量, boxes2的数量)
    inter_areas = inters[:, :, 0] * inters[:, :, 1]
    union_areas = areas1[:, None] + areas2 - inter_areas
    return inter_areas / union_areas

# From chapter_13_computer_vision/anchor.py
def assign_anchor_to_bbox(ground_truth, anchors, device, iou_threshold=0.5):
    """将最近的真实边界框分配给锚框"""
    num_anchors, num_gt_boxes = anchors.shape[0], ground_truth.shape[0]
    # 位于第i行和第j列的元素x_ij是锚框i和真实边界框j的IoU
    jaccard = box_iou(anchors, ground_truth)
    # 对于每个锚框, 分配的真实边界框的张量
    anchors_bbox_map = torch.full((num_anchors,), -1, dtype=torch.long,
                                  device=device)

    # 根据阈值, 决定是否分配真实边界框
    max_iou, indices = torch.max(jaccard, dim=1)
    anc_i = torch.nonzero(max_iou >= iou_threshold).reshape(-1)
    box_j = indices[max_iou >= iou_threshold]
    anchors_bbox_map[anc_i] = box_j
    col_discard = torch.full((num_anchors,), -1)
    row_discard = torch.full((num_gt_boxes,), -1)
    for _ in range(num_gt_boxes):
        max_idx = torch.argmax(jaccard)
        box_idx = (max_idx % num_gt_boxes).long()
        anc_idx = (max_idx / num_gt_boxes).long()
        anchors_bbox_map[anc_idx] = box_idx

    jaccard[:, box_idx] = col_discard
    jaccard[anc_idx, :] = row_discard
    return anchors_bbox_map

# From chapter_13_computer_vision/anchor.py
def offset_boxes(anchors, assigned_bb, eps=1e-6):
    """对锚框偏移量的转换"""
    c_anc = native.box_corner_to_center(anchors)
    c_assigned_bb = native.box_corner_to_center(assigned_bb)
    offset_xy = 10 * (c_assigned_bb[:, :2] - c_anc[:, :2]) / c_anc[:, 2:]
    offset_wh = 5 * native.log(eps + c_assigned_bb[:, 2:] / c_anc[:, 2:])
    offset = native.concat([offset_xy, offset_wh], axis=1)
    return offset

# From chapter_13_computer_vision/anchor.py

```

```

def multibox_target(anchors, labels):
    """使用真实边界框标记锚框"""
    batch_size, anchors = labels.shape[0], anchors.squeeze(0)
    batch_offset, batch_mask, batch_class_labels = [], [], []
    device, num_anchors = anchors.device, anchors.shape[0]
    for i in range(batch_size):
        label = labels[i, :, :]
        anchors_bbox_map = assign_anchor_to_bbox(
            label[:, 1:], anchors, device)
        bbox_mask = ((anchors_bbox_map >= 0).float().unsqueeze(-1)).repeat(
            1, 4)
        # 将类标签和分配的边界框坐标初始化为零
        class_labels = torch.zeros(num_anchors, dtype=torch.long,
                                   device=device)
        assigned_bb = torch.zeros((num_anchors, 4), dtype=torch.float32,
                                   device=device)
        # 使用真实边界框来标记锚框的类别。
        # 如果一个锚框没有被分配, 标记其为背景 (值为零)
        indices_true = torch.nonzero(anchors_bbox_map >= 0)
        bb_idx = anchors_bbox_map[indices_true]
        class_labels[indices_true] = label[bb_idx, 0].long() + 1
        assigned_bb[indices_true] = label[bb_idx, 1:]
        # 偏移量转换
        offset = offset_boxes(anchors, assigned_bb) * bbox_mask
        batch_offset.append(offset.reshape(-1))
        batch_mask.append(bbox_mask.reshape(-1))
        batch_class_labels.append(class_labels)
    bbox_offset = torch.stack(batch_offset)
    bbox_mask = torch.stack(batch_mask)
    class_labels = torch.stack(batch_class_labels)
    return (bbox_offset, bbox_mask, class_labels)

```

```

# From chapter_13_computer_vision/anchor.py
def offset_inverse(anchors, offset_preds):
    """根据带有预测偏移量的锚框来预测边界框"""
    anc = native.box_corner_to_center(anchors)
    pred_bbox_xy = (offset_preds[:, :2] * anc[:, 2:] / 10) + anc[:, :2]
    pred_bbox_wh = native.exp(offset_preds[:, 2:] / 5) * anc[:, 2:]
    pred_bbox = native.concat((pred_bbox_xy, pred_bbox_wh), axis=1)
    predicted_bbox = native.box_center_to_corner(pred_bbox)
    return predicted_bbox

```

```

# From chapter_13_computer_vision/anchor.py
def nms(boxes, scores, iou_threshold):
    """对预测边界框的置信度进行排序"""
    B = torch.argsort(scores, dim=-1, descending=True)
    keep = [] # 保留预测边界框的指标
    while B.numel() > 0:
        i = B[0]
        keep.append(i)
        if B.numel() == 1: break
        iou = box_iou(boxes[i, :].reshape(-1, 4),
                      boxes[B[1:], :].reshape(-1, 4)).reshape(-1)
        inds = torch.nonzero(iou <= iou_threshold).reshape(-1)
        B = B[inds + 1]
    return native.tensor(keep, device=boxes.device)

```

```

# From chapter_13_computer_vision/anchor.py
def multibox_detection(cls_probs, offset_preds, anchors, nms_threshold=0.5,
                      pos_threshold=0.009999999):
    """使用非极大值抑制来预测边界框"""
    device, batch_size = cls_probs.device, cls_probs.shape[0]
    anchors = anchors.squeeze(0)
    num_classes, num_anchors = cls_probs.shape[1], cls_probs.shape[2]
    out = []
    for i in range(batch_size):
        cls_prob, offset_pred = cls_probs[i], offset_preds[i].reshape(-1, 4)
        conf, class_id = torch.max(cls_prob[1:], 0)
        predicted_bb = offset_inverse(anchors, offset_pred)
        keep = nms(predicted_bb, conf, nms_threshold)

        # 找到所有的non_keep索引, 并将类设置为背景
        all_idx = torch.arange(num_anchors, dtype=torch.long, device=device)
        combined = torch.cat((keep, all_idx))
        uniques, counts = combined.unique(return_counts=True)
        non_keep = uniques[counts == 1]
        all_id_sorted = torch.cat((keep, non_keep))

```

```

        class_id[non_keep] = -1
        class_id = class_id[all_id_sorted]
        conf, predicted_bb = conf[all_id_sorted], predicted_bb[all_id_sorted]
# pos_threshold是一个用于非背景预测的阈值
        below_min_idx = (conf < pos_threshold)
        class_id[below_min_idx] = -1
        conf[below_min_idx] = 1 - conf[below_min_idx]
        pred_info = torch.cat((class_id.unsqueeze(1),
                               conf.unsqueeze(1),
                               predicted_bb), dim=1)

        out.append(pred_info)
    return native.stack(out)

# From chapter_13_computer_vision/bounding_box.py
def box_corner_to_center(boxes):
    """从(左上, 右下) 转换到 (中间, 宽度, 高度)"""
    x1, y1, x2, y2 = boxes[:, 0], boxes[:, 1], boxes[:, 2], boxes[:, 3]
    cx = (x1 + x2) / 2
    cy = (y1 + y2) / 2
    w = x2 - x1
    h = y2 - y1
    boxes = native.stack((cx, cy, w, h), axis=-1)
    return boxes

# From chapter_13_computer_vision/bounding_box.py
def box_center_to_corner(boxes):
    """从(中间, 宽度, 高度) 转换到 (左上, 右下)"""
    cx, cy, w, h = boxes[:, 0], boxes[:, 1], boxes[:, 2], boxes[:, 3]

    x1 = cx - 0.5 * w
    y1 = cy - 0.5 * h
    x2 = cx + 0.5 * w
    y2 = cy + 0.5 * h
    boxes = native.stack((x1, y1, x2, y2), axis=-1)
    return boxes

# From chapter_13_computer_vision/bounding_box.py
def bbox_to_rect(bbox, color):
    # 将边界框(左上x,左上y,右下x,右下y)格式转换成matplotlib格式:
    # ((左上x,左上y),宽,高)
    return native.plt.Rectangle(
        xy=(bbox[0], bbox[1]), width=bbox[2]-bbox[0], height=bbox[3]-bbox[1],
        fill=False, edgecolor=color, linewidth=2)

# From chapter_13_computer_vision/fcn.py
def bilinear_kernel(in_channels, out_channels, kernel_size):
    factor = (kernel_size + 1) // 2
    if kernel_size % 2 == 1:
        center = factor - 1
    else:
        center = factor - 0.5
    og = (torch.arange(kernel_size).reshape(-1, 1),
          torch.arange(kernel_size).reshape(1, -1))
    filt = (1 - torch.abs(og[0] - center) / factor) * \
           (1 - torch.abs(og[1] - center) / factor)
    weight = torch.zeros((in_channels, out_channels,
                          kernel_size, kernel_size))
    weight[range(in_channels), range(out_channels), :, :] = filt
    return weight

# From chapter_13_computer_vision/fcn.py
def loss(inputs, targets):
    return F.cross_entropy(inputs, targets, reduction='none').mean(1).mean(1)

# From chapter_13_computer_vision/fcn.py
def predict(img):
    X = test_iter.dataset.normalize_image(img).unsqueeze(0)
    pred = net(X.to(devices[0])).argmax(dim=1)
    return pred.reshape(pred.shape[1], pred.shape[2])

# From chapter_13_computer_vision/fcn.py
def label2image(pred):
    colormap = torch.tensor(native.VOC_COLORMAP, device=devices[0])
    X = pred.long()
    return colormap[X, :]

# From chapter_13_computer_vision/fine_tuning.py
def train_fine_tuning(net, learning_rate, batch_size=128, num_epochs=5,

```

```

        param_group=True):
train_iter = torch.utils.data.DataLoader(torchvision.datasets.ImageFolder(
    os.path.join(data_dir, 'train'), transform=train_augs),
    batch_size=batch_size, shuffle=True)
test_iter = torch.utils.data.DataLoader(torchvision.datasets.ImageFolder(
    os.path.join(data_dir, 'test'), transform=test_augs),
    batch_size=batch_size)
devices = native.try_all_gpus()
loss = nn.CrossEntropyLoss(reduction="none")
if param_group:
    params_1x = [param for name, param in net.named_parameters()]
    if name not in ["fc.weight", "fc.bias"]
    trainer = torch.optim.SGD([{'params': params_1x},
                                {'params': net.fc.parameters(),
                                 'lr': learning_rate * 10}],
                                lr=learning_rate, weight_decay=0.001)
else:
    trainer = torch.optim.SGD(net.parameters(), lr=learning_rate,
                                weight_decay=0.001)
native.train_ch13(net, train_iter, test_iter, loss, trainer, num_epochs,
    devices)

# From chapter_13_computer_vision/image_augmentation.py
def apply(img, aug, num_rows=2, num_cols=4, scale=1.5):
    Y = [aug(img) for _ in range(num_rows * num_cols)]
    native.show_images(Y, num_rows, num_cols, scale=scale)

# From chapter_13_computer_vision/image_augmentation.py
def load_cifar10(is_train, augs, batch_size):
    dataset = torchvision.datasets.CIFAR10(root="../data", train=is_train,
                                            transform=augs, download=True)
    dataloader = torch.utils.data.DataLoader(dataset, batch_size=batch_size,
                                            shuffle=is_train, num_workers=native.get_dataloader_workers())
    return dataloader

# From chapter_13_computer_vision/image_augmentation.py
def train_batch_ch13(net, X, y, loss, trainer, devices):
    """用多GPU进行小批量训练"""
    if isinstance(X, list):
        # 微调BERT中所需
        X = [x.to(devices[0]) for x in X]
    else:
        X = X.to(devices[0])
    y = y.to(devices[0])
    net.train()
    trainer.zero_grad()
    pred = net(X)
    l = loss(pred, y)
    l.sum().backward()
    trainer.step()
    train_loss_sum = l.sum()
    train_acc_sum = native.accuracy(pred, y)
    return train_loss_sum, train_acc_sum

# From chapter_13_computer_vision/image_augmentation.py
def train_ch13(net, train_iter, test_iter, loss, trainer, num_epochs,
    devices=native.try_all_gpus()):
    """用多GPU进行模型训练"""
    timer, num_batches = native.Timer(), len(train_iter)
    animator = native.Animator(xlabel='epoch', xlim=[1, num_epochs], ylim=[0, 1],
                                legend=['train loss', 'train acc', 'test acc'])
    net = nn.DataParallel(net, device_ids=devices).to(devices[0])
    for epoch in range(num_epochs):
        # 4个维度: 储存训练损失, 训练准确度, 实例数, 特点数
        metric = native.Accumulator(4)
        for i, (features, labels) in enumerate(train_iter):
            timer.start()
            l, acc = train_batch_ch13(
                net, features, labels, loss, trainer, devices)
            metric.add(l, acc, labels.shape[0], labels.numel())
            timer.stop()
            if (i + 1) % (num_batches // 5) == 0 or i == num_batches - 1:
                animator.add(epoch + (i + 1) / num_batches,
                    (metric[0] / metric[2], metric[1] / metric[3],
                     None))
        test_acc = native.evaluate_accuracy_gpu(net, test_iter)

```

```

        animator.add(epoch + 1, (None, None, test_acc))
    print(f'loss {metric[0] / metric[2]:.3f}, train acc '
          f'{metric[1] / metric[3]:.3f}, test acc {test_acc:.3f}')
    print(f'{metric[2] * num_epochs / timer.sum():.1f} examples/sec on '
          f'{str(devices)}')

# From chapter_13_computer_vision/image_augmentation.py
def init_weights(m):
    if type(m) in [nn.Linear, nn.Conv2d]:
        nn.init.xavier_uniform_(m.weight)

# From chapter_13_computer_vision/image_augmentation.py
def train_with_data_aug(train_augs, test_augs, net, lr=0.001):
    train_iter = load_cifar10(True, train_augs, batch_size)
    test_iter = load_cifar10(False, test_augs, batch_size)
    loss = nn.CrossEntropyLoss(reduction="none")
    trainer = torch.optim.Adam(net.parameters(), lr=lr)
    train_ch13(net, train_iter, test_iter, loss, trainer, 10, devices)

# From chapter_13_computer_vision/kaggle_cifar10.py
def read_csv_labels(fname):
    """读取fname来给标签字典返回一个文件名"""
    with open(fname, 'r') as f:
        # 跳过文件头行(列名)
        lines = f.readlines()[1:]
        tokens = [l.rstrip().split(',') for l in lines]
        return dict((name, label) for name, label in tokens)

# From chapter_13_computer_vision/kaggle_cifar10.py
def copyfile(filename, target_dir):
    """将文件复制到目标目录"""
    os.makedirs(target_dir, exist_ok=True)
    shutil.copy(filename, target_dir)

# From chapter_13_computer_vision/kaggle_cifar10.py
def reorg_train_valid(data_dir, labels, valid_ratio):
    """将验证集从原始的训练集中拆分出来"""
    # 训练数据集中样本最少的类别中的样本数
    n = collections.Counter(labels.values()).most_common()[-1][1]
    # 验证集中每个类别的样本数
    n_valid_per_label = max(1, math.floor(n * valid_ratio))
    label_count = {}
    for train_file in os.listdir(os.path.join(data_dir, 'train')):
        label = labels[train_file.split('.')[0]]
        fname = os.path.join(data_dir, 'train', train_file)
        copyfile(fname, os.path.join(data_dir, 'train_valid_test',
                                     'train_valid', label))
        if label not in label_count or label_count[label] < n_valid_per_label:
            copyfile(fname, os.path.join(data_dir, 'train_valid_test',
                                         'valid', label))
            label_count[label] = label_count.get(label, 0) + 1
        else:
            copyfile(fname, os.path.join(data_dir, 'train_valid_test',
                                         'train', label))
    return n_valid_per_label

# From chapter_13_computer_vision/kaggle_cifar10.py
def reorg_test(data_dir):
    """在预测期间整理测试集, 以方便读取"""
    for test_file in os.listdir(os.path.join(data_dir, 'test')):
        copyfile(os.path.join(data_dir, 'test', test_file),
                  os.path.join(data_dir, 'train_valid_test', 'test',
                              'unknown'))

# From chapter_13_computer_vision/kaggle_cifar10.py
def reorg_cifar10_data(data_dir, valid_ratio):
    labels = read_csv_labels(os.path.join(data_dir, 'trainLabels.csv'))
    reorg_train_valid(data_dir, labels, valid_ratio)
    reorg_test(data_dir)

# From chapter_13_computer_vision/kaggle_cifar10.py
def get_net():
    num_classes = 10
    net = native.resnet18(num_classes, 3)
    return net

# From chapter_13_computer_vision/kaggle_cifar10.py

```

```

def train(net, train_iter, valid_iter, num_epochs, lr, wd, devices, lr_period,
          lr_decay):
    trainer = torch.optim.SGD(net.parameters(), lr=lr, momentum=0.9,
                               weight_decay=wd)
    scheduler = torch.optim.lr_scheduler.StepLR(trainer, lr_period, lr_decay)
    num_batches, timer = len(train_iter), native.Timer()
    legend = ['train loss', 'train acc']
    if valid_iter is not None:
        legend.append('valid acc')
    animator = native.Animator(xlabel='epoch', xlim=[1, num_epochs],
                               legend=legend)
    net = nn.DataParallel(net, device_ids=devices).to(devices[0])
    for epoch in range(num_epochs):
        net.train()
        metric = native.Accumulator(3)
        for i, (features, labels) in enumerate(train_iter):
            timer.start()
            l, acc = native.train_batch_ch13(net, features, labels,
                                              loss, trainer, devices)
            metric.add(l, acc, labels.shape[0])
            timer.stop()
            if (i + 1) % (num_batches // 5) == 0 or i == num_batches - 1:
                animator.add(epoch + (i + 1) / num_batches,
                              (metric[0] / metric[2], metric[1] / metric[2],
                               None))
        if valid_iter is not None:
            valid_acc = native.evaluate_accuracy_gpu(net, valid_iter)
            animator.add(epoch + 1, (None, None, valid_acc))
        scheduler.step()
    measures = (f'train loss {metric[0] / metric[2]:.3f}',
                f'train acc {metric[1] / metric[2]:.3f}')
    if valid_iter is not None:
        measures += f', valid acc {valid_acc:.3f}'
    print(measures + f'\n{metric[2] * num_epochs / timer.sum():.1f}'
          f' examples/sec on {str(devices)}')

# From chapter_13_computer_vision/kaggle_dog.py
def reorg_dog_data(data_dir, valid_ratio):
    labels = native.read_csv_labels(os.path.join(data_dir, 'labels.csv'))
    native.reorg_train_valid(data_dir, labels, valid_ratio)
    native.reorg_test(data_dir)

# From chapter_13_computer_vision/kaggle_dog.py
def get_net(devices):
    finetune_net = nn.Sequential()
    finetune_net.features = torchvision.models.resnet34(pretrained=True)
    # 定义一个新的输出网络, 共有120个输出类别
    finetune_net.output_new = nn.Sequential(nn.Linear(1000, 256),
                                             nn.ReLU(),
                                             nn.Linear(256, 120))

    # 将模型参数分配给用于计算的CPU或GPU
    finetune_net = finetune_net.to(devices[0])
    # 冻结参数
    for param in finetune_net.features.parameters():
        param.requires_grad = False
    return finetune_net

# From chapter_13_computer_vision/kaggle_dog.py
def evaluate_loss(data_iter, net, devices):
    l_sum, n = 0.0, 0
    for features, labels in data_iter:
        features, labels = features.to(devices[0]), labels.to(devices[0])
        outputs = net(features)
        l = loss(outputs, labels)
        l_sum += l.sum()
        n += labels.numel()
    return (l_sum / n).to('cpu')

# From chapter_13_computer_vision/kaggle_dog.py
def train(net, train_iter, valid_iter, num_epochs, lr, wd, devices, lr_period,
          lr_decay):
    # 只训练小型自定义输出网络
    net = nn.DataParallel(net, device_ids=devices).to(devices[0])
    trainer = torch.optim.SGD((param for param in net.parameters()
                                if param.requires_grad), lr=lr,
                               momentum=0.9, weight_decay=wd)
    scheduler = torch.optim.lr_scheduler.StepLR(trainer, lr_period, lr_decay)

```

```

num_batches, timer = len(train_iter), native.Timer()
legend = ['train loss']
if valid_iter is not None:
    legend.append('valid loss')
animator = native.Animator(xlabel='epoch', xlim=[1, num_epochs],
                           legend=legend)
for epoch in range(num_epochs):
    metric = native.Accumulator(2)
    for i, (features, labels) in enumerate(train_iter):
        timer.start()
        features, labels = features.to(devices[0]), labels.to(devices[0])
        trainer.zero_grad()
        output = net(features)
        l = loss(output, labels).sum()
        l.backward()
        trainer.step()
        metric.add(l, labels.shape[0])
        timer.stop()
        if (i + 1) % (num_batches // 5) == 0 or i == num_batches - 1:
            animator.add(epoch + (i + 1) / num_batches,
                         (metric[0] / metric[1], None))

    measures = f'train loss {metric[0] / metric[1]:.3f}'
    if valid_iter is not None:
        valid_loss = evaluate_loss(valid_iter, net, devices)
        animator.add(epoch + 1, (None, valid_loss.detach().cpu()))
    scheduler.step()
if valid_iter is not None:
    measures += f', valid loss {valid_loss:.3f}'
print(measures + f'\n{metric[1] * num_epochs / timer.sum():.1f}'
      f' examples/sec on {str(devices)}')

# From chapter_13_computer_vision/multiscale_object_detection.py
def display_anchors(fmap_w, fmap_h, s):
    native.set_figsize()
    # 前两个维度上的值不影响输出
    fmap = native.zeros((1, 10, fmap_h, fmap_w))
    anchors = native.multibox_prior(fmap, sizes=s, ratios=[1, 2, 0.5])
    bbox_scale = native.tensor((w, h, w, h))
    native.show_bboxes(native.plt.imshow(img).axes,
                      anchors[0] * bbox_scale)

# From chapter_13_computer_vision/neural_style.py
def preprocess(img, image_shape):
    transforms = torchvision.transforms.Compose([
        torchvision.transforms.Resize(image_shape),
        torchvision.transforms.ToTensor(),
        torchvision.transforms.Normalize(mean=rgb_mean, std=rgb_std)])
    return transforms(img).unsqueeze(0)

# From chapter_13_computer_vision/neural_style.py
def postprocess(img):
    img = img[0].to(rgb_std.device)
    img = torch.clamp(img.permute(1, 2, 0) * rgb_std + rgb_mean, 0, 1)
    return torchvision.transforms.ToPILImage()(img.permute(2, 0, 1))

# From chapter_13_computer_vision/neural_style.py
def extract_features(X, content_layers, style_layers):
    contents = []
    styles = []
    for i in range(len(net)):
        X = net[i](X)
        if i in style_layers:
            styles.append(X)
        if i in content_layers:
            contents.append(X)
    return contents, styles

# From chapter_13_computer_vision/neural_style.py
def get_contents(image_shape, device):
    content_X = preprocess(content_img, image_shape).to(device)
    contents_Y, _ = extract_features(content_X, content_layers, style_layers)
    return content_X, contents_Y

# From chapter_13_computer_vision/neural_style.py
def get_styles(image_shape, device):
    style_X = preprocess(style_img, image_shape).to(device)
    _, styles_Y = extract_features(style_X, content_layers, style_layers)

```

```

    return style_X, styles_Y

# From chapter_13_computer_vision/neural_style.py
def content_loss(Y_hat, Y):
    # 我们从动态计算梯度的树中分离目标:
    # 这是一个规定的值, 而不是一个变量。
    return torch.square(Y_hat - Y.detach()).mean()

# From chapter_13_computer_vision/neural_style.py
def gram(X):
    num_channels, n = X.shape[1], native.size(X) // X.shape[1]
    X = native.reshape(X, (num_channels, n))
    return native.matmul(X, X.T) / (num_channels * n)

# From chapter_13_computer_vision/neural_style.py
def style_loss(Y_hat, gram_Y):
    return torch.square(gram(Y_hat) - gram_Y.detach()).mean()

# From chapter_13_computer_vision/neural_style.py
def tv_loss(Y_hat):
    return 0.5 * (native.abs(Y_hat[:, :, 1:, :] - Y_hat[:, :, :-1, :]).mean() +
                  native.abs(Y_hat[:, :, :, 1:] - Y_hat[:, :, :, :-1])).mean())

# From chapter_13_computer_vision/neural_style.py
def compute_loss(X, contents_Y_hat, styles_Y_hat, contents_Y, styles_Y_gram):
    # 分别计算内容损失、风格损失和全变分损失
    contents_l = [content_loss(Y_hat, Y) * content_weight for Y_hat, Y in zip(
        contents_Y_hat, contents_Y)]
    styles_l = [style_loss(Y_hat, Y) * style_weight for Y_hat, Y in zip(
        styles_Y_hat, styles_Y_gram)]
    tv_l = tv_loss(X) * tv_weight
    # 对所有损失求和
    l = sum(10 * styles_l + contents_l + [tv_l])
    return contents_l, styles_l, tv_l, l

# From chapter_13_computer_vision/neural_style.py
class SynthesizedImage(nn.Module):
    def __init__(self, img_shape, **kwargs):
        super(SynthesizedImage, self).__init__(**kwargs)
        self.weight = nn.Parameter(torch.rand(*img_shape))

    def forward(self):
        return self.weight

# From chapter_13_computer_vision/neural_style.py
def get_inits(X, device, lr, styles_Y):
    gen_img = SynthesizedImage(X.shape).to(device)
    gen_img.weight.data.copy_(X.data)
    trainer = torch.optim.Adam(gen_img.parameters(), lr=lr)
    styles_Y_gram = [gram(Y) for Y in styles_Y]
    return gen_img(), styles_Y_gram, trainer

# From chapter_13_computer_vision/neural_style.py
def train(X, contents_Y, styles_Y, device, lr, num_epochs, lr_decay_epoch):
    X, styles_Y_gram, trainer = get_inits(X, device, lr, styles_Y)
    scheduler = torch.optim.lr_scheduler.StepLR(trainer, lr_decay_epoch, 0.8)
    animator = native.Animator(xlabel='epoch', ylabel='loss',
                               xlim=[10, num_epochs],
                               legend=['content', 'style', 'TV'],
                               ncols=2, figsize=(7, 2.5))
    for epoch in range(num_epochs):
        trainer.zero_grad()
        contents_Y_hat, styles_Y_hat = extract_features(
            X, content_layers, style_layers)

        contents_l, styles_l, tv_l, l = compute_loss(
            X, contents_Y_hat, styles_Y_hat, contents_Y, styles_Y_gram)
        l.backward()
        trainer.step()
        scheduler.step()
        if (epoch + 1) % 10 == 0:
            animator.axes[1].imshow(postprocess(X))
            animator.add(epoch + 1, [float(sum(contents_l)),
                                     float(sum(styles_l)), float(tv_l)])
    return X

# From chapter_13_computer_vision/object_detection_dataset.py

```

```

def read_data_bananas(is_train=True):
    """读取香蕉检测数据集中的图像和标签"""
    data_dir = native.download_extract('banana-detection')
    csv_fname = os.path.join(data_dir, 'bananas_train' if is_train
                              else 'bananas_val', 'label.csv')
    csv_data = pd.read_csv(csv_fname)
    csv_data = csv_data.set_index('img_name')
    images, targets = [], []
    for img_name, target in csv_data.iterrows():
        images.append(torchvision.io.read_image(
            os.path.join(data_dir, 'bananas_train' if is_train else
                          'bananas_val', 'images', f'{img_name}'))))
    # 这里的target包含 (类别, 左上角x, 左上角y, 右下角x, 右下角y),
    # 其中所有图像都具有相同的香蕉类 (索引为0)
    targets.append(list(target))
    return images, torch.tensor(targets).unsqueeze(1) / 256

# From chapter_13_computer_vision/object_detection_dataset.py
class BananasDataset(torch.utils.data.Dataset):
    """一个用于加载香蕉检测数据集的自定义数据集"""
    def __init__(self, is_train):
        self.features, self.labels = read_data_bananas(is_train)
        print('read ' + str(len(self.features)) + (f' training examples' if
                                                    is_train else f' validation examples'))

    def __getitem__(self, idx):
        return (self.features[idx].float(), self.labels[idx])

    def __len__(self):
        return len(self.features)

# From chapter_13_computer_vision/object_detection_dataset.py
def load_data_bananas(batch_size):
    """加载香蕉检测数据集"""
    train_iter = torch.utils.data.DataLoader(BananasDataset(is_train=True),
                                              batch_size, shuffle=True)
    val_iter = torch.utils.data.DataLoader(BananasDataset(is_train=False),
                                           batch_size)
    return train_iter, val_iter

# From chapter_13_computer_vision/semantic_segmentation_and_dataset.py
def read_voc_images(voc_dir, is_train=True):
    """读取所有VOC图像并标注"""
    txt_fname = os.path.join(voc_dir, 'ImageSets', 'Segmentation',
                              'train.txt' if is_train else 'val.txt')
    mode = torchvision.io.image.ImageReadMode.RGB
    with open(txt_fname, 'r') as f:
        images = f.read().split()

    features, labels = [], []
    for i, fname in enumerate(images):
        features.append(torchvision.io.read_image(os.path.join(
            voc_dir, 'JPEGImages', f'{fname}.jpg')))
        labels.append(torchvision.io.read_image(os.path.join(
            voc_dir, 'SegmentationClass', f'{fname}.png'), mode))
    return features, labels

# From chapter_13_computer_vision/semantic_segmentation_and_dataset.py
def voc_colormap2label():
    """构建从RGB到VOC类别索引的映射"""
    colormap2label = torch.zeros(256 * 3, dtype=torch.long)
    for i, colormap in enumerate(VOC_COLORMAP):
        colormap2label[
            (colormap[0] * 256 + colormap[1]) * 256 + colormap[2]] = i
    return colormap2label

# From chapter_13_computer_vision/semantic_segmentation_and_dataset.py
def voc_label_indices(colormap, colormap2label):
    """将VOC标签中的RGB值映射到它们的类别索引"""
    colormap = colormap.permute(1, 2, 0).numpy().astype('int32')
    idx = ((colormap[:, :, 0] * 256 + colormap[:, :, 1]) * 256
           + colormap[:, :, 2])
    return colormap2label[idx]

# From chapter_13_computer_vision/semantic_segmentation_and_dataset.py
def voc_rand_crop(feature, label, height, width):
    """随机裁剪特征和标签图像"""
    rect = torchvision.transforms.RandomCrop.get_params(

```

```

        feature, (height, width))
    feature = torchvision.transforms.functional.crop(feature, *rect)
    label = torchvision.transforms.functional.crop(label, *rect)
    return feature, label

# From chapter_13_computer_vision/semantic_segmentation_and_dataset.py
class VOCSegDataset(torch.utils.data.Dataset):
    """一个用于加载VOC数据集的自定义数据集"""

    def __init__(self, is_train, crop_size, voc_dir):
        self.transform = torchvision.transforms.Normalize(
            mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
        self.crop_size = crop_size
        features, labels = read_voc_images(voc_dir, is_train=is_train)
        self.features = [self.normalize_image(feature)
                        for feature in self.filter(features)]
        self.labels = self.filter(labels)
        self.colormap2label = voc_colormap2label()
        print('read ' + str(len(self.features)) + ' examples')

    def normalize_image(self, img):
        return self.transform(img.float() / 255)

    def filter(self, imgs):
        return [img for img in imgs if (
            img.shape[1] >= self.crop_size[0] and
            img.shape[2] >= self.crop_size[1])]

    def __getitem__(self, idx):
        feature, label = voc_rand_crop(self.features[idx], self.labels[idx],
                                      *self.crop_size)

        return (feature, voc_label_indices(label, self.colormap2label))

    def __len__(self):
        return len(self.features)

# From chapter_13_computer_vision/semantic_segmentation_and_dataset.py
def load_data_voc(batch_size, crop_size):
    """加载VOC语义分割数据集"""
    voc_dir = native.download_extract('voc2012', os.path.join(
        'VOCdevkit', 'VOC2012'))
    num_workers = native.get_dataloader_workers()
    train_iter = torch.utils.data.DataLoader(
        VOCSegDataset(True, crop_size, voc_dir), batch_size,
        shuffle=True, drop_last=True, num_workers=num_workers)
    test_iter = torch.utils.data.DataLoader(
        VOCSegDataset(False, crop_size, voc_dir), batch_size,
        drop_last=True, num_workers=num_workers)
    return train_iter, test_iter

# From chapter_13_computer_vision/ssd.py
def cls_predictor(num_inputs, num_anchors, num_classes):
    return nn.Conv2d(num_inputs, num_anchors * (num_classes + 1),
                    kernel_size=3, padding=1)

# From chapter_13_computer_vision/ssd.py
def bbox_predictor(num_inputs, num_anchors):
    return nn.Conv2d(num_inputs, num_anchors * 4, kernel_size=3, padding=1)

# From chapter_13_computer_vision/ssd.py
def forward(x, block):
    return block(x)

# From chapter_13_computer_vision/ssd.py
def flatten_pred(pred):
    return torch.flatten(pred.permute(0, 2, 3, 1), start_dim=1)

# From chapter_13_computer_vision/ssd.py
def concat_preds(preds):
    return torch.cat([flatten_pred(p) for p in preds], dim=1)

# From chapter_13_computer_vision/ssd.py
def down_sample_blk(in_channels, out_channels):
    blk = []
    for _ in range(2):
        blk.append(nn.Conv2d(in_channels, out_channels,
                            kernel_size=3, padding=1))

```

```

        blk.append(nn.BatchNorm2d(out_channels))
        blk.append(nn.ReLU())
        in_channels = out_channels
        blk.append(nn.MaxPool2d(2))
        return nn.Sequential(*blk)

# From chapter_13_computer_vision/ssd.py
def base_net():
    blk = []
    num_filters = [3, 16, 32, 64]
    for i in range(len(num_filters) - 1):
        blk.append(down_sample_blk(num_filters[i], num_filters[i+1]))
    return nn.Sequential(*blk)

# From chapter_13_computer_vision/ssd.py
def get_blk(i):
    if i == 0:
        blk = base_net()
    elif i == 1:
        blk = down_sample_blk(64, 128)
    elif i == 4:
        blk = nn.AdaptiveMaxPool2d((1,1))
    else:
        blk = down_sample_blk(128, 128)
    return blk

# From chapter_13_computer_vision/ssd.py
def blk_forward(X, blk, size, ratio, cls_predictor, bbox_predictor):
    Y = blk(X)
    anchors = native.multibox_prior(Y, sizes=size, ratios=ratio)
    cls_preds = cls_predictor(Y)
    bbox_preds = bbox_predictor(Y)
    return (Y, anchors, cls_preds, bbox_preds)

# From chapter_13_computer_vision/ssd.py
class TinySSD(nn.Module):
    def __init__(self, num_classes, **kwargs):
        super(TinySSD, self).__init__(**kwargs)
        self.num_classes = num_classes
        idx_to_in_channels = [64, 128, 128, 128, 128]
        for i in range(5):
            # 即赋值语句self.blk_i=get_blk(i)
            setattr(self, f'blk_{i}', get_blk(i))
            setattr(self, f'cls_{i}', cls_predictor(idx_to_in_channels[i],
                                                    num_anchors, num_classes))
            setattr(self, f'bbox_{i}', bbox_predictor(idx_to_in_channels[i],
                                                    num_anchors))

    def forward(self, X):
        anchors, cls_preds, bbox_preds = [None] * 5, [None] * 5, [None] * 5
        for i in range(5):
            # getattr(self, 'blk_%d'%i)即访问self.blk_i
            X, anchors[i], cls_preds[i], bbox_preds[i] = blk_forward(
                X, getattr(self, f'blk_{i}'), sizes[i], ratios[i],
                getattr(self, f'cls_{i}'), getattr(self, f'bbox_{i}'))
            anchors = torch.cat(anchors, dim=1)
            cls_preds = concat_preds(cls_preds)
            cls_preds = cls_preds.reshape(
                cls_preds.shape[0], -1, self.num_classes + 1)
            bbox_preds = concat_preds(bbox_preds)
        return anchors, cls_preds, bbox_preds

# From chapter_13_computer_vision/ssd.py
def calc_loss(cls_preds, cls_labels, bbox_preds, bbox_labels, bbox_masks):
    batch_size, num_classes = cls_preds.shape[0], cls_preds.shape[2]
    cls = cls_loss(cls_preds.reshape(-1, num_classes),
                   cls_labels.reshape(-1)).reshape(batch_size, -1).mean(dim=1)
    bbox = bbox_loss(bbox_preds * bbox_masks,
                     bbox_labels * bbox_masks).mean(dim=1)
    return cls + bbox

# From chapter_13_computer_vision/ssd.py
def cls_eval(cls_preds, cls_labels):
    # 由于类别预测结果放在最后一维，argmax需要指定最后一维。
    return float((cls_preds.argmax(dim=-1).type(
        cls_labels.dtype) == cls_labels).sum())

```

```

# From chapter_13_computer_vision/ssd.py
def bbox_eval(bbox_preds, bbox_labels, bbox_masks):
    return float((torch.abs((bbox_labels - bbox_preds) * bbox_masks)).sum())

# From chapter_13_computer_vision/ssd.py
def predict(X):
    net.eval()
    anchors, cls_preds, bbox_preds = net(X.to(device))
    cls_probs = F.softmax(cls_preds, dim=2).permute(0, 2, 1)
    output = native.multibox_detection(cls_probs, bbox_preds, anchors)
    idx = [i for i, row in enumerate(output[0]) if row[0] != -1]
    return output[0, idx]

# From chapter_13_computer_vision/ssd.py
def display(img, output, threshold):
    native.set_figsize((5, 5))
    fig = native.plt.imshow(img)
    for row in output:
        score = float(row[1])
        if score < threshold:
            continue
        h, w = img.shape[0:2]
        bbox = [row[2:6] * torch.tensor((w, h, w, h), device=row.device)]
        native.show_bboxes(fig.axes, bbox, '%.2f' % score, 'w')

# From chapter_13_computer_vision/ssd.py
def smooth_l1(data, scalar):
    out = []
    for i in data:
        if abs(i) < 1 / (scalar ** 2):
            out.append(((scalar * i) ** 2) / 2)
        else:
            out.append(abs(i) - 0.5 / (scalar ** 2))
    return torch.tensor(out)

# From chapter_13_computer_vision/ssd.py
def focal_loss(gamma, x):
    return -(1 - x) ** gamma * torch.log(x)

# From chapter_13_computer_vision/transposed_conv.py
def trans_conv(X, K):
    h, w = K.shape
    Y = native.zeros((X.shape[0] + h - 1, X.shape[1] + w - 1))
    for i in range(X.shape[0]):
        for j in range(X.shape[1]):
            Y[i: i + h, j: j + w] += X[i, j] * K
    return Y

# From chapter_13_computer_vision/transposed_conv.py
def kernel2matrix(K):
    k, W = native.zeros(5), native.zeros((4, 9))
    k[2:], k[3:5] = K[0, :], K[1, :]
    W[0, :5], W[1, 1:6], W[2, 3:8], W[3, 4:] = k, k, k, k
    return W

# From chapter_14_nlp_pretraining/bert.py
def get_tokens_and_segments(tokens_a, tokens_b=None):
    """获取输入序列的词元及其片段索引"""
    tokens = ['<cls>'] + tokens_a + ['<sep>']
    # 0和1分别标记片段A和B
    segments = [0] * (len(tokens_a) + 2)
    if tokens_b is not None:
        tokens += tokens_b + ['<sep>']
        segments += [1] * (len(tokens_b) + 1)
    return tokens, segments

# From chapter_14_nlp_pretraining/bert.py
class BERTEncoder(nn.Module):
    """BERT编码器"""
    def __init__(self, vocab_size, num_hiddens, norm_shape, ffn_num_input,
                  ffn_num_hiddens, num_heads, num_layers, dropout,
                  max_len=1000, key_size=768, query_size=768, value_size=768,
                  **kwargs):
        super(BERTEncoder, self).__init__(**kwargs)
        self.token_embedding = nn.Embedding(vocab_size, num_hiddens)
        self.segment_embedding = nn.Embedding(2, num_hiddens)

```

```

self.blks = nn.Sequential()
for i in range(num_layers):
    self.blks.add_module(f"{i}", native.EncoderBlock(
        key_size, query_size, value_size, num_hiddens, norm_shape,
        ffn_num_input, ffn_num_hiddens, num_heads, dropout, True))
# 在BERT中, 位置嵌入是可学习的, 因此我们创建一个足够长的位置嵌入参数
self.pos_embedding = nn.Parameter(torch.randn(1, max_len,
                                                num_hiddens))

def forward(self, tokens, segments, valid_lens):
    # 在以下代码段中, X的形状保持不变: (批量大小, 最大序列长度, num_hiddens)
    X = self.token_embedding(tokens) + self.segment_embedding(segments)
    X = X + self.pos_embedding.data[:, :X.shape[1], :]
    for blk in self.blks:
        X = blk(X, valid_lens)
    return X

# From chapter_14_nlp_pretraining/bert.py
class MaskLM(nn.Module):
    """BERT的掩蔽语言模型任务"""
    def __init__(self, vocab_size, num_hiddens, num_inputs=768, **kwargs):
        super(MaskLM, self).__init__(**kwargs)
        self.mlp = nn.Sequential(nn.Linear(num_inputs, num_hiddens),
                                   nn.ReLU(),
                                   nn.LayerNorm(num_hiddens),
                                   nn.Linear(num_hiddens, vocab_size))

    def forward(self, X, pred_positions):
        num_pred_positions = pred_positions.shape[1]
        pred_positions = pred_positions.reshape(-1)
        batch_size = X.shape[0]
        batch_idx = torch.arange(0, batch_size)
        # 假设batch_size=2, num_pred_positions=3
        # 那么batch_idx是np.array ([0,0,0,1,1,1])
        batch_idx = torch.repeat_interleave(batch_idx, num_pred_positions)
        masked_X = X[batch_idx, pred_positions]
        masked_X = masked_X.reshape((batch_size, num_pred_positions, -1))
        mlm_Y_hat = self.mlp(masked_X)
        return mlm_Y_hat

# From chapter_14_nlp_pretraining/bert.py
class NextSentencePred(nn.Module):
    """BERT的下一句预测任务"""
    def __init__(self, num_inputs, **kwargs):
        super(NextSentencePred, self).__init__(**kwargs)
        self.output = nn.Linear(num_inputs, 2)

    def forward(self, X):
        # X的形状: (batchsize, num_hiddens)
        return self.output(X)

# From chapter_14_nlp_pretraining/bert.py
class BERTModel(nn.Module):
    """BERT模型"""
    def __init__(self, vocab_size, num_hiddens, norm_shape, ffn_num_input,
                  ffn_num_hiddens, num_heads, num_layers, dropout,
                  max_len=1000, key_size=768, query_size=768, value_size=768,
                  hid_in_features=768, mlm_in_features=768,
                  nsp_in_features=768):
        super(BERTModel, self).__init__()
        self.encoder = BERTEncoder(vocab_size, num_hiddens, norm_shape,
                                    ffn_num_input, ffn_num_hiddens, num_heads, num_layers,
                                    dropout, max_len=max_len, key_size=key_size,
                                    query_size=query_size, value_size=value_size)
        self.hidden = nn.Sequential(nn.Linear(hid_in_features, num_hiddens),
                                     nn.Tanh())
        self.mlm = MaskLM(vocab_size, num_hiddens, mlm_in_features)
        self.nsp = NextSentencePred(nsp_in_features)

    def forward(self, tokens, segments, valid_lens=None,
                  pred_positions=None):
        encoded_X = self.encoder(tokens, segments, valid_lens)
        if pred_positions is not None:
            mlm_Y_hat = self.mlm(encoded_X, pred_positions)
        else:
            mlm_Y_hat = None
        # 用于下一句预测的多层感知机分类器的隐藏层, 0是"<cls>"标记的索引

```

```

        nsp_Y_hat = self.nsp(self.hidden(encoded_X[:, 0, :]))
        return encoded_X, mlm_Y_hat, nsp_Y_hat

# From chapter_14_nlp_pretraining/bert_dataset.py
def _read_wiki(data_dir):
    file_name = os.path.join(data_dir, 'wiki.train.tokens')
    with open(file_name, 'r') as f:
        lines = f.readlines()
    # 大写字母转换为小写字母
    paragraphs = [line.strip().lower().split(' . ')]
    for line in lines if len(line.split(' . ')) >= 2]
    random.shuffle(paragraphs)
    return paragraphs

# From chapter_14_nlp_pretraining/bert_dataset.py
def _get_next_sentence(sentence, next_sentence, paragraphs):
    if random.random() < 0.5:
        is_next = True
    else:
        # paragraphs是三重列表的嵌套
        next_sentence = random.choice(random.choice(paragraphs))
        is_next = False
    return sentence, next_sentence, is_next

# From chapter_14_nlp_pretraining/bert_dataset.py
def _get_nsp_data_from_paragraph(paragraph, paragraphs, vocab, max_len):
    nsp_data_from_paragraph = []
    for i in range(len(paragraph) - 1):
        tokens_a, tokens_b, is_next = _get_next_sentence(
            paragraph[i], paragraph[i + 1], paragraphs)
        # 考虑1个'<cls>'词元和2个'<sep>'词元
        if len(tokens_a) + len(tokens_b) + 3 > max_len:
            continue
        tokens, segments = native.get_tokens_and_segments(tokens_a, tokens_b)
        nsp_data_from_paragraph.append((tokens, segments, is_next))
    return nsp_data_from_paragraph

# From chapter_14_nlp_pretraining/bert_dataset.py
def _replace_mlm_tokens(tokens, candidate_pred_positions, num_mlm_preds,
                        vocab):
    # 为遮蔽语言模型的输入创建新的词元副本，其中输入可能包含替换的"<mask>"或随机词元
    mlm_input_tokens = [token for token in tokens]
    pred_positions_and_labels = []
    # 打乱后用于在遮蔽语言模型任务中获取15%的随机词元进行预测
    random.shuffle(candidate_pred_positions)
    for mlm_pred_position in candidate_pred_positions:
        if len(pred_positions_and_labels) >= num_mlm_preds:
            break
        masked_token = None
        # 80%的时间：将词替换为"<mask>"词元
        if random.random() < 0.8:
            masked_token = '<mask>'
        else:
            # 10%的时间：保持词不变
            if random.random() < 0.5:
                masked_token = tokens[mlm_pred_position]
            # 10%的时间：用随机词替换该词
            else:
                masked_token = random.choice(vocab.idx_to_token)
        mlm_input_tokens[mlm_pred_position] = masked_token
        pred_positions_and_labels.append(
            (mlm_pred_position, tokens[mlm_pred_position]))
    return mlm_input_tokens, pred_positions_and_labels

# From chapter_14_nlp_pretraining/bert_dataset.py
def _get_mlm_data_from_tokens(tokens, vocab):
    candidate_pred_positions = []
    # tokens是一个字符串列表
    for i, token in enumerate(tokens):
        # 在遮蔽语言模型任务中不会预测特殊词元
        if token in ['<cls>', '<sep>']:
            continue
        candidate_pred_positions.append(i)
    # 遮蔽语言模型任务中预测15%的随机词元
    num_mlm_preds = max(1, round(len(tokens) * 0.15))
    mlm_input_tokens, pred_positions_and_labels = _replace_mlm_tokens(
        tokens, candidate_pred_positions, num_mlm_preds, vocab)

```

```

pred_positions_and_labels = sorted(pred_positions_and_labels,
                                   key=lambda x: x[0])
pred_positions = [v[0] for v in pred_positions_and_labels]
mlm_pred_labels = [v[1] for v in pred_positions_and_labels]
return vocab[mlm_input_tokens], pred_positions, vocab[mlm_pred_labels]

# From chapter_14_nlp_pretraining/bert_dataset.py
def _pad_bert_inputs(examples, max_len, vocab):
    max_num_mlm_preds = round(max_len * 0.15)
    all_token_ids, all_segments, valid_lens, = [], [], []
    all_pred_positions, all_mlm_weights, all_mlm_labels = [], [], []
    nsp_labels = []
    for (token_ids, pred_positions, mlm_pred_label_ids, segments,
         is_next) in examples:
        all_token_ids.append(torch.tensor(token_ids + [vocab['<pad>']] * (
            max_len - len(token_ids)), dtype=torch.long))
        all_segments.append(torch.tensor(segments + [0] * (
            max_len - len(segments)), dtype=torch.long))
    # valid_lens不包括'<pad>'的计数
    valid_lens.append(torch.tensor(len(token_ids), dtype=torch.float32))
    all_pred_positions.append(torch.tensor(pred_positions + [0] * (
        max_num_mlm_preds - len(pred_positions)), dtype=torch.long))
    # 填充词元的预测将通过乘以0权重在损失中过滤掉
    all_mlm_weights.append(
        torch.tensor([1.0] * len(mlm_pred_label_ids) + [0.0] * (
            max_num_mlm_preds - len(pred_positions)),
            dtype=torch.float32))
    all_mlm_labels.append(torch.tensor(mlm_pred_label_ids + [0] * (
        max_num_mlm_preds - len(mlm_pred_label_ids)), dtype=torch.long))
    nsp_labels.append(torch.tensor(is_next, dtype=torch.long))
    return (all_token_ids, all_segments, valid_lens, all_pred_positions,
            all_mlm_weights, all_mlm_labels, nsp_labels)

# From chapter_14_nlp_pretraining/bert_dataset.py
class WikiTextDataset(torch.utils.data.Dataset):
    def __init__(self, paragraphs, max_len):
        # 输入paragraphs[i]是代表段落的句子字符串列表;
        # 而输出paragraphs[i]是代表段落的句子列表, 其中每个句子都是词元列表
        paragraphs = [native.tokenize(
            paragraph, token='word') for paragraph in paragraphs]
        sentences = [sentence for paragraph in paragraphs
                     for sentence in paragraph]
        self.vocab = native.Vocab(sentences, min_freq=5, reserved_tokens=[
            '<pad>', '<mask>', '<cls>', '<sep>'])
    # 获取下一句子预测任务的数据
    examples = []
    for paragraph in paragraphs:
        examples.extend(_get_nsp_data_from_paragraph(
            paragraph, paragraphs, self.vocab, max_len))
    # 获取遮蔽语言模型任务的数据
    examples = [(_get_mlm_data_from_tokens(tokens, self.vocab)
                 + (segments, is_next))
                for tokens, segments, is_next in examples]
    # 填充输入
    (self.all_token_ids, self.all_segments, self.valid_lens,
     self.all_pred_positions, self.all_mlm_weights,
     self.all_mlm_labels, self.nsp_labels) = _pad_bert_inputs(
        examples, max_len, self.vocab)

    def __getitem__(self, idx):
        return (self.all_token_ids[idx], self.all_segments[idx],
                self.valid_lens[idx], self.all_pred_positions[idx],
                self.all_mlm_weights[idx], self.all_mlm_labels[idx],
                self.nsp_labels[idx])

    def __len__(self):
        return len(self.all_token_ids)

# From chapter_14_nlp_pretraining/bert_dataset.py
def load_data_wiki(batch_size, max_len):
    """加载WikiText-2数据集"""
    num_workers = native.get_data_loader_workers()
    data_dir = native.download_extract('wikitext-2', 'wikitext-2')
    paragraphs = _read_wiki(data_dir)
    train_set = WikiTextDataset(paragraphs, max_len)
    train_iter = torch.utils.data.DataLoader(train_set, batch_size,

```

```

        shuffle=True, num_workers=num_workers)
    return train_iter, train_set.vocab

# From chapter_14_nlp_pretraining/bert_pretraining.py
def _get_batch_loss_bert(net, loss, vocab_size, tokens_X,
                        segments_X, valid_lens_x,
                        pred_positions_X, mlm_weights_X,
                        mlm_Y, nsp_y):
    # 前向传播
    _, mlm_Y_hat, nsp_Y_hat = net(tokens_X, segments_X,
                                   valid_lens_x.reshape(-1),
                                   pred_positions_X)

    # 计算遮蔽语言模型损失
    mlm_l = loss(mlm_Y_hat.reshape(-1, vocab_size), mlm_Y.reshape(-1)) * \
            mlm_weights_X.reshape(-1, 1)
    mlm_l = mlm_l.sum() / (mlm_weights_X.sum() + 1e-8)
    # 计算下一句子预测任务的损失
    nsp_l = loss(nsp_Y_hat, nsp_y)
    l = mlm_l + nsp_l
    return mlm_l, nsp_l, l

# From chapter_14_nlp_pretraining/bert_pretraining.py
def train_bert(train_iter, net, loss, vocab_size, devices, num_steps):
    net = nn.DataParallel(net, device_ids=devices).to(devices[0])
    trainer = torch.optim.Adam(net.parameters(), lr=0.01)
    step, timer = 0, native.Timer()
    animator = native.Animator(xlabel='step', ylabel='loss',
                               xlim=[1, num_steps], legend=['mlm', 'nsp'])
    # 遮蔽语言模型损失的和, 下一句预测任务损失的和, 句子对的数量, 计数
    metric = native.Accumulator(4)
    num_steps_reached = False
    while step < num_steps and not num_steps_reached:
        for tokens_X, segments_X, valid_lens_x, pred_positions_X, \
            mlm_weights_X, mlm_Y, nsp_y in train_iter:
            tokens_X = tokens_X.to(devices[0])
            segments_X = segments_X.to(devices[0])
            valid_lens_x = valid_lens_x.to(devices[0])
            pred_positions_X = pred_positions_X.to(devices[0])
            mlm_weights_X = mlm_weights_X.to(devices[0])
            mlm_Y, nsp_y = mlm_Y.to(devices[0]), nsp_y.to(devices[0])
            trainer.zero_grad()
            timer.start()
            mlm_l, nsp_l, l = _get_batch_loss_bert(
                net, loss, vocab_size, tokens_X, segments_X, valid_lens_x,
                pred_positions_X, mlm_weights_X, mlm_Y, nsp_y)
            l.backward()
            trainer.step()
            metric.add(mlm_l, nsp_l, tokens_X.shape[0], 1)
            timer.stop()
            animator.add(step + 1,
                        (metric[0] / metric[3], metric[1] / metric[3]))

        step += 1
        if step == num_steps:
            num_steps_reached = True
            break

    print(f'MLM loss {metric[0] / metric[3]:.3f}, '
          f'NSP loss {metric[1] / metric[3]:.3f}')
    print(f'{metric[2] / timer.sum():.1f} sentence pairs/sec on '
          f'{str(devices)}')

# From chapter_14_nlp_pretraining/bert_pretraining.py
def get_bert_encoding(net, tokens_a, tokens_b=None):
    tokens, segments = native.get_tokens_and_segments(tokens_a, tokens_b)
    token_ids = torch.tensor(vocab[tokens], device=devices[0]).unsqueeze(0)
    segments = torch.tensor(segments, device=devices[0]).unsqueeze(0)
    valid_len = torch.tensor(len(tokens), device=devices[0]).unsqueeze(0)
    encoded_X, _, _ = net(token_ids, segments, valid_len)
    return encoded_X

# From chapter_14_nlp_pretraining/similarity_analogy.py
class TokenEmbedding:
    """GloVe嵌入"""
    def __init__(self, embedding_name):
        self.idx_to_token, self.idx_to_vec = self._load_embedding(
            embedding_name)
        self.unknown_idx = 0

```

```

        self.token_to_idx = {token: idx for idx, token in
                               enumerate(self.idx_to_token)}

def _load_embedding(self, embedding_name):
    idx_to_token, idx_to_vec = ['<unk>'], []
    data_dir = native.download_extract(embedding_name)
    # GloVe网站: https://nlp.stanford.edu/projects/glove/
    # fastText网站: https://fasttext.cc/
    with open(os.path.join(data_dir, 'vec.txt'), 'r') as f:
        for line in f:
            elems = line.rstrip().split(' ')
            token, elems = elems[0], [float(elem) for elem in elems[1:]]
            # 跳过标题信息, 例如fastText中的首行
            if len(elems) > 1:
                idx_to_token.append(token)
                idx_to_vec.append(elems)
    idx_to_vec = [[0] * len(idx_to_vec[0])] + idx_to_vec
    return idx_to_token, native.tensor(idx_to_vec)

def __getitem__(self, tokens):
    indices = [self.token_to_idx.get(token, self.unknown_idx)
                for token in tokens]
    vecs = self.idx_to_vec[native.tensor(indices)]
    return vecs

def __len__(self):
    return len(self.idx_to_token)

# From chapter_14_nlp_pretraining/similarity_analogy.py
def knn(W, x, k):
    # 增加1e-9以获得数值稳定性
    cos = torch.mv(W, x.reshape(-1,)) / (
        torch.sqrt(torch.sum(W * W, axis=1) + 1e-9) *
        torch.sqrt((x * x).sum()))
    _, topk = torch.topk(cos, k=k)
    return topk, [cos[int(i)] for i in topk]

# From chapter_14_nlp_pretraining/similarity_analogy.py
def get_similar_tokens(query_token, k, embed):
    topk, cos = knn(embed.idx_to_vec, embed[[query_token]], k + 1)
    for i, c in zip(topk[1:], cos[1:]): # 排除输入词
        print(f'{embed.idx_to_token[int(i)]}: cosine相似度={float(c):.3f}')

# From chapter_14_nlp_pretraining/similarity_analogy.py
def get_analogy(token_a, token_b, token_c, embed):
    vecs = embed[[token_a, token_b, token_c]]
    x = vecs[1] - vecs[0] + vecs[2]
    topk, cos = knn(embed.idx_to_vec, x, 1)
    return embed.idx_to_token[int(topk[0])]

# From chapter_14_nlp_pretraining/subword_embedding.py
def get_max_freq_pair(token_freqs):
    pairs = collections.defaultdict(int)
    for token, freq in token_freqs.items():
        symbols = token.split()
        for i in range(len(symbols) - 1):
            # "pairs"的键是两个连续符号的元组
            pairs[symbols[i], symbols[i + 1]] += freq
    return max(pairs, key=pairs.get)

# From chapter_14_nlp_pretraining/subword_embedding.py
def merge_symbols(max_freq_pair, token_freqs, symbols):
    symbols.append(''.join(max_freq_pair))
    new_token_freqs = dict()
    for token, freq in token_freqs.items():
        new_token = token.replace(' '.join(max_freq_pair),
                                     ''.join(max_freq_pair))
        new_token_freqs[new_token] = token_freqs[token]
    return new_token_freqs

# From chapter_14_nlp_pretraining/subword_embedding.py
def segment_BPE(tokens, symbols):
    outputs = []
    for token in tokens:
        start, end = 0, len(token)
        cur_output = []
        # 具有符号中可能最长子字的词元段

```

```

        while start < len(token) and start < end:
            if token[start: end] in symbols:
                cur_output.append(token[start: end])
                start = end
                end = len(token)
            else:
                end -= 1
        if start < len(token):
            cur_output.append('[UNK]')
        outputs.append(' '.join(cur_output))
    return outputs

# From chapter_14_nlp_pretraining/word2vec_pretraining.py
def skip_gram(center, contexts_and_negatives, embed_v, embed_u):
    v = embed_v(center)
    u = embed_u(contexts_and_negatives)
    pred = torch.bmm(v, u.permute(0, 2, 1))
    return pred

# From chapter_14_nlp_pretraining/word2vec_pretraining.py
class SigmoidBCELoss(nn.Module):
    # 带掩码的二元交叉熵损失
    def __init__(self):
        super().__init__()

    def forward(self, inputs, target, mask=None):
        out = nn.functional.binary_cross_entropy_with_logits(
            inputs, target, weight=mask, reduction="none")
        return out.mean(dim=1)

# From chapter_14_nlp_pretraining/word2vec_pretraining.py
def sigmd(x):
    return -math.log(1 / (1 + math.exp(-x)))

# From chapter_14_nlp_pretraining/word2vec_pretraining.py
def train(net, data_iter, lr, num_epochs, device=native.try_gpu()):
    def init_weights(m):
        if type(m) == nn.Embedding:
            nn.init.xavier_uniform_(m.weight)
    net.apply(init_weights)
    net = net.to(device)
    optimizer = torch.optim.Adam(net.parameters(), lr=lr)
    animator = native.Animator(xlabel='epoch', ylabel='loss',
                               xlim=[1, num_epochs])
    # 规范化的损失之和, 规范化的损失数
    metric = native.Accumulator(2)
    for epoch in range(num_epochs):
        timer, num_batches = native.Timer(), len(data_iter)
        for i, batch in enumerate(data_iter):
            optimizer.zero_grad()
            center, context_negative, mask, label = [
                data.to(device) for data in batch]

            pred = skip_gram(center, context_negative, net[0], net[1])
            l = (loss(pred.reshape(label.shape).float(), label.float(), mask)
                 / mask.sum(axis=1) * mask.shape[1])
            l.sum().backward()
            optimizer.step()
            metric.add(l.sum(), l.numel())
            if (i + 1) % (num_batches // 5) == 0 or i == num_batches - 1:
                animator.add(epoch + (i + 1) / num_batches,
                             (metric[0] / metric[1],))
        print(f'loss {metric[0] / metric[1]:.3f}, '
              f'{metric[1] / timer.stop():.1f} tokens/sec on {str(device)}')

# From chapter_14_nlp_pretraining/word2vec_pretraining.py
def get_similar_tokens(query_token, k, embed):
    W = embed.weight.data
    x = W[vocab[query_token]]
    # 计算余弦相似性。增加1e-9以获得数值稳定性
    cos = torch.mv(W, x) / torch.sqrt(torch.sum(W * W, dim=1) *
                                       torch.sum(x * x) + 1e-9)
    topk = torch.topk(cos, k=k+1)[1].cpu().numpy().astype('int32')
    for i in topk[1:]: # 删除输入词
        print(f'cosine sim={float(cos[i]):.3f}: {vocab.to_tokens(i)}')

# From chapter_14_nlp_pretraining/word_embedding_dataset.py

```

```

def read_ptb():
    """将PTB数据集加载到文本行的列表中"""
    data_dir = native.download_extract('ptb')
    # Read the training set.
    with open(os.path.join(data_dir, 'ptb.train.txt')) as f:
        raw_text = f.read()
    return [line.split() for line in raw_text.split('\n')]

# From chapter_14_nlp_pretraining/word_embedding_dataset.py
def subsample(sentences, vocab):
    """下采样高频词"""
    # 排除未知词元 '<unk>'
    sentences = [[token for token in line if vocab[token] != vocab.unk]
                  for line in sentences]
    counter = native.count_corpus(sentences)
    num_tokens = sum(counter.values())

    # 如果在下采样期间保留词元，则返回True
    def keep(token):
        return (random.uniform(0, 1) <
                math.sqrt(1e-4 / counter[token] * num_tokens))

    return ([[token for token in line if keep(token)] for line in sentences],
            counter)

# From chapter_14_nlp_pretraining/word_embedding_dataset.py
def compare_counts(token):
    return (f"{token}的数量: "
            f'之前={sum([l.count(token) for l in sentences])}, '
            f'之后={sum([l.count(token) for l in subsampled])}')

# From chapter_14_nlp_pretraining/word_embedding_dataset.py
def get_centers_and_contexts(corpus, max_window_size):
    """返回跳元模型中的中心词和上下文词"""
    centers, contexts = [], []
    for line in corpus:
        # 要形成“中心词-上下文词”对，每个句子至少需要有2个词
        if len(line) < 2:
            continue
        centers += line
        for i in range(len(line)): # 上下文窗口中间
            window_size = random.randint(1, max_window_size)
            indices = list(range(max(0, i - window_size),
                                   min(len(line), i + 1 + window_size)))

            # 从上下文词中排除中心词
            indices.remove(i)
            contexts.append([line[idx] for idx in indices])
    return centers, contexts

# From chapter_14_nlp_pretraining/word_embedding_dataset.py
class RandomGenerator:
    """根据n个采样权重在{1,...,n}中随机抽取"""
    def __init__(self, sampling_weights):
        # Exclude
        self.population = list(range(1, len(sampling_weights) + 1))
        self.sampling_weights = sampling_weights
        self.candidates = []
        self.i = 0

    def draw(self):
        if self.i == len(self.candidates):
            # 缓存k个随机采样结果
            self.candidates = random.choices(
                self.population, self.sampling_weights, k=10000)
            self.i = 0
        self.i += 1
        return self.candidates[self.i - 1]

# From chapter_14_nlp_pretraining/word_embedding_dataset.py
def get_negatives(all_contexts, vocab, counter, K):
    """返回负采样中的噪声词"""
    # 索引为1、2、... (索引0是词表中排除的未知标记)
    sampling_weights = [counter[vocab.to_tokens(i)]**0.75
                        for i in range(1, len(vocab))]
    all_negatives, generator = [], RandomGenerator(sampling_weights)
    for contexts in all_contexts:

```

```

        negatives = []
        while len(negatives) < len(contexts) * K:
            neg = generator.draw()
            # 噪声词不能是上下文词
            if neg not in contexts:
                negatives.append(neg)
            all_negatives.append(negatives)
        return all_negatives

# From chapter_14_nlp_pretraining/word_embedding_dataset.py
def batchify(data):
    """返回带有负采样的跳元模型的小批量样本"""
    max_len = max(len(c) + len(n) for _, c, n in data)
    centers, contexts_negatives, masks, labels = [], [], [], []
    for center, context, negative in data:
        cur_len = len(context) + len(negative)
        centers += [center]
        contexts_negatives += \
            [context + negative + [0] * (max_len - cur_len)]
        masks += [[1] * cur_len + [0] * (max_len - cur_len)]
        labels += [[1] * len(context) + [0] * (max_len - len(context))]
    return (native.reshape(native.tensor(centers), (-1, 1)), native.tensor(
        contexts_negatives), native.tensor(masks), native.tensor(labels))

# From chapter_14_nlp_pretraining/word_embedding_dataset.py
def load_data_ptb(batch_size, max_window_size, num_noise_words):
    """下载PTB数据集，然后将其加载到内存中"""
    num_workers = native.get_data_loader_workers()
    sentences = read_ptb()
    vocab = native.Vocab(sentences, min_freq=10)
    subsampled, counter = subsample(sentences, vocab)
    corpus = [vocab[line] for line in subsampled]
    all_centers, all_contexts = get_centers_and_contexts(
        corpus, max_window_size)
    all_negatives = get_negatives(
        all_contexts, vocab, counter, num_noise_words)

    class PTBDataset(torch.utils.data.Dataset):
        def __init__(self, centers, contexts, negatives):
            assert len(centers) == len(contexts) == len(negatives)
            self.centers = centers
            self.contexts = contexts
            self.negatives = negatives

        def __getitem__(self, index):
            return (self.centers[index], self.contexts[index],
                    self.negatives[index])

        def __len__(self):
            return len(self.centers)

    dataset = PTBDataset(all_centers, all_contexts, all_negatives)

    data_iter = torch.utils.data.DataLoader(
        dataset, batch_size, shuffle=True,
        collate_fn=batchify, num_workers=num_workers)
    return data_iter, vocab

# From chapter_15_nlp_applications/natural_language_inference_and_dataset.py
def read_snli(data_dir, is_train):
    """将SNLI数据集解析为前提、假设和标签"""
    def extract_text(s):
        # 删除我们不会使用的信息
        s = re.sub('\\(', ' ', s)
        s = re.sub('\\)', ' ', s)
        # 用一个空格替换两个或多个连续的空格
        s = re.sub('\\s{2,}', ' ', s)
        return s.strip()
    label_set = {'entailment': 0, 'contradiction': 1, 'neutral': 2}
    file_name = os.path.join(data_dir, 'snli_1.0_train.txt'
                              if is_train else 'snli_1.0_test.txt')
    with open(file_name, 'r') as f:
        rows = [row.split('\t') for row in f.readlines()[1:]]
        premises = [extract_text(row[1]) for row in rows if row[0] in label_set]
        hypotheses = [extract_text(row[2]) for row in rows if row[0] in label_set]
        labels = [label_set[row[0]] for row in rows if row[0] in label_set]

```

```

        return premises, hypotheses, labels

# From chapter_15_nlp_applications/natural_language_inference_and_dataset.py
class SNLIDataset(torch.utils.data.Dataset):
    """用于加载SNLI数据集的自定义数据集"""
    def __init__(self, dataset, num_steps, vocab=None):
        self.num_steps = num_steps
        all_premise_tokens = native.tokenize(dataset[0])
        all_hypothesis_tokens = native.tokenize(dataset[1])
        if vocab is None:
            self.vocab = native.Vocab(all_premise_tokens + \
                                     all_hypothesis_tokens, min_freq=5, reserved_tokens=['<pad>'])
        else:
            self.vocab = vocab
        self.premises = self._pad(all_premise_tokens)
        self.hypotheses = self._pad(all_hypothesis_tokens)
        self.labels = torch.tensor(dataset[2])
        print('read ' + str(len(self.premises)) + ' examples')

    def _pad(self, lines):
        return torch.tensor([native.truncate_pad(
            self.vocab[line], self.num_steps, self.vocab['<pad>'])
            for line in lines])

    def __getitem__(self, idx):
        return (self.premises[idx], self.hypotheses[idx]), self.labels[idx]

    def __len__(self):
        return len(self.premises)

# From chapter_15_nlp_applications/natural_language_inference_and_dataset.py
def load_data_snli(batch_size, num_steps=50):
    """下载SNLI数据集并返回数据迭代器和词表"""
    num_workers = native.get_data_loader_workers()
    data_dir = native.download_extract('SNLI')
    train_data = read_snli(data_dir, True)
    test_data = read_snli(data_dir, False)
    train_set = SNLIDataset(train_data, num_steps)
    test_set = SNLIDataset(test_data, num_steps, train_set.vocab)
    train_iter = torch.utils.data.DataLoader(train_set, batch_size,
                                             shuffle=True,
                                             num_workers=num_workers)
    test_iter = torch.utils.data.DataLoader(test_set, batch_size,
                                             shuffle=False,
                                             num_workers=num_workers)
    return train_iter, test_iter, train_set.vocab

# From chapter_15_nlp_applications/natural_language_inference_attention.py
def mlp(num_inputs, num_hiddens, flatten):
    net = []
    net.append(nn.Dropout(0.2))
    net.append(nn.Linear(num_inputs, num_hiddens))
    net.append(nn.ReLU())
    if flatten:
        net.append(nn.Flatten(start_dim=1))
    net.append(nn.Dropout(0.2))
    net.append(nn.Linear(num_hiddens, num_hiddens))
    net.append(nn.ReLU())
    if flatten:
        net.append(nn.Flatten(start_dim=1))
    return nn.Sequential(*net)

# From chapter_15_nlp_applications/natural_language_inference_attention.py
class Attend(nn.Module):
    def __init__(self, num_inputs, num_hiddens, **kwargs):
        super(Attend, self).__init__(**kwargs)
        self.f = mlp(num_inputs, num_hiddens, flatten=False)

    def forward(self, A, B):
        # A/B的形状: (批量大小, 序列A/B的词元数, embed_size)
        # f_A/f_B的形状: (批量大小, 序列A/B的词元数, num_hiddens)
        f_A = self.f(A)
        f_B = self.f(B)
        # e的形状: (批量大小, 序列A的词元数, 序列B的词元数)
        e = torch.bmm(f_A, f_B.permute(0, 2, 1))
        # beta的形状: (批量大小, 序列A的词元数, embed_size)
        # 意味着序列B被软对齐到序列A的每个词元(beta的第1个维度)

```

```

        beta = torch.bmm(F.softmax(e, dim=-1), B)
# beta的形状: (批量大小, 序列B的词元数, embed_size) .
# 意味着序列A被软对齐到序列B的每个词元(alpha的第1个维度)
        alpha = torch.bmm(F.softmax(e.permute(0, 2, 1), dim=-1), A)
        return beta, alpha

# From chapter_15_nlp_applications/natural_language_inference_attention.py
class Compare(nn.Module):
    def __init__(self, num_inputs, num_hiddens, **kwargs):
        super(Compare, self).__init__(**kwargs)
        self.g = mlp(num_inputs, num_hiddens, flatten=False)

    def forward(self, A, B, beta, alpha):
        V_A = self.g(torch.cat([A, beta], dim=2))
        V_B = self.g(torch.cat([B, alpha], dim=2))
        return V_A, V_B

# From chapter_15_nlp_applications/natural_language_inference_attention.py
class Aggregate(nn.Module):
    def __init__(self, num_inputs, num_hiddens, num_outputs, **kwargs):
        super(Aggregate, self).__init__(**kwargs)
        self.h = mlp(num_inputs, num_hiddens, flatten=True)
        self.linear = nn.Linear(num_hiddens, num_outputs)

    def forward(self, V_A, V_B):
# 对两组比较向量分别求和
        V_A = V_A.sum(dim=1)
        V_B = V_B.sum(dim=1)
# 将两个求和结果的连接送到多层感知机中
        Y_hat = self.linear(self.h(torch.cat([V_A, V_B], dim=1)))
        return Y_hat

# From chapter_15_nlp_applications/natural_language_inference_attention.py
class DecomposableAttention(nn.Module):
    def __init__(self, vocab, embed_size, num_hiddens, num_inputs_attend=100,
                 num_inputs_compare=200, num_inputs_agg=400, **kwargs):
        super(DecomposableAttention, self).__init__(**kwargs)
        self.embedding = nn.Embedding(len(vocab), embed_size)
        self.attend = Attend(num_inputs_attend, num_hiddens)
        self.compare = Compare(num_inputs_compare, num_hiddens)
# 有3种可能的输出: 蕴涵、矛盾和中性
        self.aggregate = Aggregate(num_inputs_agg, num_hiddens, num_outputs=3)

    def forward(self, X):
        premises, hypotheses = X
        A = self.embedding(premises)
        B = self.embedding(hypotheses)
        beta, alpha = self.attend(A, B)
        V_A, V_B = self.compare(A, B, beta, alpha)
        Y_hat = self.aggregate(V_A, V_B)
        return Y_hat

# From chapter_15_nlp_applications/natural_language_inference_attention.py
def predict_snli(net, vocab, premise, hypothesis):
    """预测前提和假设之间的逻辑关系"""
    net.eval()
    premise = torch.tensor(vocab[premise], device=native.try_gpu())
    hypothesis = torch.tensor(vocab[hypothesis], device=native.try_gpu())
    label = torch.argmax(net([premise.reshape((1, -1)),
                             hypothesis.reshape((1, -1))]), dim=1)
    return 'entailment' if label == 0 else 'contradiction' if label == 1 \
        else 'neutral'

# From chapter_15_nlp_applications/natural_language_inference_bert.py
def load_pretrained_model(pretrained_model, num_hiddens, ffn_num_hiddens,
                         num_heads, num_layers, dropout, max_len, devices):
    data_dir = native.download_extract(pretrained_model)
# 定义空词表以加载预定义词表
    vocab = native.Vocab()
    vocab.idx_to_token = json.load(open(os.path.join(data_dir,
        'vocab.json'))))
    vocab.token_to_idx = {token: idx for idx, token in enumerate(
        vocab.idx_to_token)}
    bert = native.BERTModel(len(vocab), num_hiddens, norm_shape=[256],
                            ffn_num_input=256, ffn_num_hiddens=ffn_num_hiddens,
                            num_heads=4, num_layers=2, dropout=0.2,

```

```

        max_len=max_len, key_size=256, query_size=256,
        value_size=256, hid_in_features=256,
        mlm_in_features=256, nsp_in_features=256)

# 加载预训练BERT参数
bert.load_state_dict(torch.load(os.path.join(data_dir,
                                              'pretrained.params')))

return bert, vocab

# From chapter_15_nlp_applications/natural_language_inference_bert.py
class SNLIBERTDataset(torch.utils.data.Dataset):
    def __init__(self, dataset, max_len, vocab=None):
        all_premise_hypothesis_tokens = [[
            p_tokens, h_tokens] for p_tokens, h_tokens in zip(
                *[native.tokenize([s.lower() for s in sentences])
                  for sentences in dataset[:2]])]

        self.labels = torch.tensor(dataset[2])
        self.vocab = vocab
        self.max_len = max_len
        (self.all_token_ids, self.all_segments,
         self.valid_lens) = self._preprocess(all_premise_hypothesis_tokens)
        print('read ' + str(len(self.all_token_ids)) + ' examples')

    def _preprocess(self, all_premise_hypothesis_tokens):
        pool = multiprocessing.Pool(4) # 使用4个进程
        out = pool.map(self._mp_worker, all_premise_hypothesis_tokens)
        all_token_ids = [
            token_ids for token_ids, segments, valid_len in out]
        all_segments = [segments for token_ids, segments, valid_len in out]
        valid_lens = [valid_len for token_ids, segments, valid_len in out]
        return (torch.tensor(all_token_ids, dtype=torch.long),
                torch.tensor(all_segments, dtype=torch.long),
                torch.tensor(valid_lens))

    def _mp_worker(self, premise_hypothesis_tokens):
        p_tokens, h_tokens = premise_hypothesis_tokens
        self._truncate_pair_of_tokens(p_tokens, h_tokens)
        tokens, segments = native.get_tokens_and_segments(p_tokens, h_tokens)
        token_ids = self.vocab[tokens] + [self.vocab['<pad>']] \
            * (self.max_len - len(tokens))
        segments = segments + [0] * (self.max_len - len(segments))
        valid_len = len(tokens)
        return token_ids, segments, valid_len

    def _truncate_pair_of_tokens(self, p_tokens, h_tokens):
        # 为BERT输入中的'<CLS>'、'<SEP>'和'<SEP>'词元保留位置
        while len(p_tokens) + len(h_tokens) > self.max_len - 3:
            if len(p_tokens) > len(h_tokens):
                p_tokens.pop()
            else:
                h_tokens.pop()

    def __getitem__(self, idx):
        return (self.all_token_ids[idx], self.all_segments[idx],
                self.valid_lens[idx]), self.labels[idx]

    def __len__(self):
        return len(self.all_token_ids)

# From chapter_15_nlp_applications/natural_language_inference_bert.py
class BERTClassifier(nn.Module):
    def __init__(self, bert):
        super(BERTClassifier, self).__init__()
        self.encoder = bert.encoder
        self.hidden = bert.hidden
        self.output = nn.Linear(256, 3)

    def forward(self, inputs):
        tokens_X, segments_X, valid_lens_x = inputs
        encoded_X = self.encoder(tokens_X, segments_X, valid_lens_x)
        return self.output(self.hidden(encoded_X[:, 0, :]))

# From chapter_15_nlp_applications/sentiment_analysis_and_dataset.py
def read_imdb(data_dir, is_train):
    """读取IMDb评论数据集文本序列和标签"""
    data, labels = [], []
    for label in ('pos', 'neg'):

```

```

        folder_name = os.path.join(data_dir, 'train' if is_train else 'test',
                                    label)
    for file in os.listdir(folder_name):
        with open(os.path.join(folder_name, file), 'rb') as f:
            review = f.read().decode('utf-8').replace('\n', '')
            data.append(review)
            labels.append(1 if label == 'pos' else 0)
    return data, labels

# From chapter_15_nlp_applications/sentiment_analysis_and_dataset.py
def load_data_imdb(batch_size, num_steps=500):
    """返回数据迭代器和IMDb评论数据集的词汇表"""
    data_dir = native.download_extract('aclImdb', 'aclImdb')
    train_data = read_imdb(data_dir, True)
    test_data = read_imdb(data_dir, False)
    train_tokens = native.tokenize(train_data[0], token='word')
    test_tokens = native.tokenize(test_data[0], token='word')
    vocab = native.Vocab(train_tokens, min_freq=5)
    train_features = torch.tensor([native.truncate_pad(
        vocab[line], num_steps, vocab['<pad>']) for line in train_tokens])
    test_features = torch.tensor([native.truncate_pad(
        vocab[line], num_steps, vocab['<pad>']) for line in test_tokens])
    train_iter = native.load_array((train_features, torch.tensor(train_data[1])),
                                   batch_size)
    test_iter = native.load_array((test_features, torch.tensor(test_data[1])),
                                   batch_size,
                                   is_train=False)
    return train_iter, test_iter, vocab

# From chapter_15_nlp_applications/sentiment_analysis_cnn.py
def corr1d(X, K):
    w = K.shape[0]
    Y = native.zeros((X.shape[0] - w + 1))
    for i in range(Y.shape[0]):
        Y[i] = (X[i: i + w] * K).sum()
    return Y

```

```

# From chapter_15_nlp_applications/sentiment_analysis_cnn.py
def corr1d_multi_in(X, K):
    # 首先, 遍历'X'和'K'的第0维(通道维)。然后, 把它们加在一起
    return sum(corr1d(x, k) for x, k in zip(X, K))

# From chapter_15_nlp_applications/sentiment_analysis_cnn.py
class TextCNN(nn.Module):
    def __init__(self, vocab_size, embed_size, kernel_sizes, num_channels,
                 **kwargs):
        super(TextCNN, self).__init__(**kwargs)
        self.embedding = nn.Embedding(vocab_size, embed_size)
        # 这个嵌入层不需要训练
        self.constant_embedding = nn.Embedding(vocab_size, embed_size)
        self.dropout = nn.Dropout(0.5)
        self.decoder = nn.Linear(sum(num_channels), 2)
        # 最大时间汇聚层没有参数, 因此可以共享此实例
        self.pool = nn.AdaptiveAvgPool1d(1)
        self.relu = nn.ReLU()
        # 创建多个一维卷积层
        self.convs = nn.ModuleList()
        for c, k in zip(num_channels, kernel_sizes):
            self.convs.append(nn.Conv1d(2 * embed_size, c, k))

    def forward(self, inputs):
        # 沿着向量维度将两个嵌入层连接起来,
        # 每个嵌入层的输出形状都是(批量大小, 词元数量, 词元向量维度)连接起来
        embeddings = torch.cat((
            self.embedding(inputs), self.constant_embedding(inputs)), dim=2)
        # 根据一维卷积层的输入格式, 重新排列张量, 以便通道作为第2维
        embeddings = embeddings.permute(0, 2, 1)
        # 每个一维卷积层在最大时间汇聚层合并后, 获得的张量形状是(批量大小, 通道数, 1)
        # 删除最后一个维度并沿通道维度连接
        encoding = torch.cat([
            torch.squeeze(self.relu(self.pool(conv(embeddings))), dim=-1)
            for conv in self.convs], dim=1)
        outputs = self.decoder(self.dropout(encoding))
        return outputs

# From chapter_15_nlp_applications/sentiment_analysis_cnn.py
def init_weights(m):

```

```

        if type(m) in (nn.Linear, nn.Conv1d):
            nn.init.xavier_uniform_(m.weight)

# From chapter_15_nlp_applications/sentiment_analysis_rnn.py
class BiRNN(nn.Module):
    def __init__(self, vocab_size, embed_size, num_hiddens,
                  num_layers, **kwargs):
        super(BiRNN, self).__init__(**kwargs)
        self.embedding = nn.Embedding(vocab_size, embed_size)
        # 将bidirectional设置为True以获取双向循环神经网络
        self.encoder = nn.LSTM(embed_size, num_hiddens, num_layers=num_layers,
                                bidirectional=True)
        self.decoder = nn.Linear(4 * num_hiddens, 2)

    def forward(self, inputs):
        # inputs的形状是 (批量大小, 时间步数)
        # 因为长短期记忆网络要求其输入的维度是时间维,
        # 所以在获得词元表示之前, 输入会被转置。
        # 输出形状为 (时间步数, 批量大小, 词向量维度)
        embeddings = self.embedding(inputs.T)

        self.encoder.flatten_parameters()
        # 返回上一个隐藏层在不同时间步的隐状态,
        # outputs的形状是 (时间步数, 批量大小, 2*隐藏单元数)
        outputs, _ = self.encoder(embeddings)
        # 连结初始和最终时间步的隐状态, 作为全连接层的输入,
        # 其形状为 (批量大小, 4*隐藏单元数)
        encoding = torch.cat((outputs[0], outputs[-1]), dim=1)
        outs = self.decoder(encoding)
        return outs

# From chapter_15_nlp_applications/sentiment_analysis_rnn.py
def init_weights(m):
    if type(m) == nn.Linear:
        nn.init.xavier_uniform_(m.weight)
    if type(m) == nn.LSTM:
        for param in m._flat_weights_names:
            if "weight" in param:
                nn.init.xavier_uniform_(m._parameters[param])

# From chapter_15_nlp_applications/sentiment_analysis_rnn.py
def predict_sentiment(net, vocab, sequence):
    """预测文本序列的情感"""
    sequence = torch.tensor(vocab[sequence.split()], device=native.try_gpu())
    label = torch.argmax(net(sequence.reshape(1, -1)), dim=1)
    return 'positive' if label == 1 else 'negative'

```

第 03 章：线性神经网络

本章分类：线性模型与分类模型 章节说明：线性回归、Softmax 回归；配套图像分类数据管线。

本章网络/流程总览



第 03 章 · 线性回归 · [chapters/chapter_03_linear_networks/linear_regression.py](#)

章节对应：第 03 章 类别：网络结构 代码块：7 个

结构图：线性回归（网络结构）



单层线性映射；训练目标是最小化均方误差。

阅读注释：阅读顺序：先看输入与输出，再看核心层/模块，最后看损失函数和训练循环。

nn.Linear 把输入特征映射为一个连续值，重点看预测值与均方误差之间的关系。

"""linear-regression: 线性神经网络"""

来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。

改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。

from pathlib import Path

import sys

_ROOT = Path(__file__).resolve().parents[1]

if str(_ROOT) not in sys.path:

sys.path.insert(0, str(_ROOT))

import math

import random

import time

import numpy as np

import pandas as pd

import torch

from torch import nn

from torch.nn import functional as F

from torch.utils import data

import torchvision

from torchvision import transforms

from PIL import Image

from matplotlib import pyplot as plt

from IPython import display

import common as native

import common as native

import math

import torch

import numpy as np

import time

n = 10000

a = native.ones([n])

b = native.ones([n])

class Timer: #@save

"""记录多次运行时间"""

def __init__(self):

self.times = []

```

        self.start()

    def start(self):
        """启动计时器"""
        self.tik = time.time()

    def stop(self):
        """停止计时器并将时间记录在列表中"""
        self.times.append(time.time() - self.tik)
        return self.times[-1]

    def avg(self):
        """返回平均时间"""
        return sum(self.times) / len(self.times)

    def sum(self):
        """返回时间总和"""
        return sum(self.times)

    def cumsum(self):
        """返回累计时间"""
        return np.array(self.times).cumsum().tolist()

c = native.zeros(n)
timer = Timer()
for i in range(n):
    c[i] = a[i] + b[i]
f'{timer.stop():.5f} sec'

timer.start()
d = a + b
f'{timer.stop():.5f} sec'

def normal(x, mu, sigma):
    p = 1 / math.sqrt(2 * math.pi * sigma**2)
    return p * np.exp(-0.5 / sigma**2 * (x - mu)**2)

# 再次使用numpy进行可视化
x = np.arange(-7, 7, 0.01)

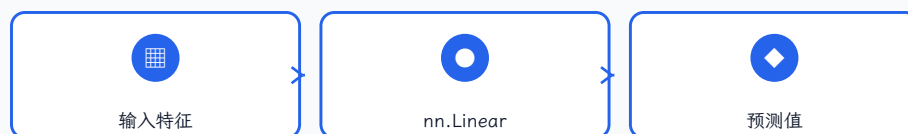
# 均值和标准差对
params = [(0, 1), (0, 2), (3, 1)]
native.plot(x, [normal(x, mu, sigma) for mu, sigma in params], xlabel='x',
             ylabel='p(x)', figsize=(4.5, 2.5),
             legend=[f'mean {mu}, std {sigma}' for mu, sigma in params])

```

第 03 章 · 线性回归 · chapters/chapter_03_linear_networks/linear_regression_scratch.py

章节对应：第 03 章 类别：网络结构 代码块：14 个

结构图：线性回归（网络结构）



单层线性映射；训练目标是最小化均方误差。

阅读注释：阅读顺序：先看输入与输出，再看核心层/模块，最后看损失函数和训练循环。

nn.Linear 把输入特征映射为一个连续值，重点看预测值与均方误差之间的关系。

```

"""linear-regression-scratch: 线性神经网络"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

```

```

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import torch
import random

def synthetic_data(w, b, num_examples): #@save
    """生成y=Xw+b+噪声"""
    X = native.normal(0, 1, (num_examples, len(w)))
    y = native.matmul(X, w) + b
    y += native.normal(0, 0.01, y.shape)
    return X, native.reshape(y, (-1, 1))

true_w = native.tensor([2, -3.4])
true_b = 4.2
features, labels = synthetic_data(true_w, true_b, 1000)

print('features:', features[0], '\nlabel:', labels[0])

native.set_figsize()
native.plt.scatter(native.numpy(features[:, 1]), native.numpy(labels), 1);

def data_iter(batch_size, features, labels):
    num_examples = len(features)
    indices = list(range(num_examples))
    # 这些样本是随机读取的, 没有特定的顺序
    random.shuffle(indices)
    for i in range(0, num_examples, batch_size):
        batch_indices = native.tensor(
            indices[i: min(i + batch_size, num_examples)])
        yield features[batch_indices], labels[batch_indices]

batch_size = 10

for X, y in data_iter(batch_size, features, labels):
    print(X, '\n', y)
    break

w = torch.normal(0, 0.01, size=(2,1), requires_grad=True)
b = torch.zeros(1, requires_grad=True)

def linreg(X, w, b): #@save
    """线性回归模型"""
    return native.matmul(X, w) + b

def squared_loss(y_hat, y): #@save
    """均方损失"""
    return (y_hat - native.reshape(y, y_hat.shape)) ** 2 / 2

def sgd(params, lr, batch_size): #@save
    """小批量随机梯度下降"""
    with torch.no_grad():
        for param in params:
            param -= lr * param.grad / batch_size
            param.grad.zero_()

lr = 0.03
num_epochs = 3
net = linreg
loss = squared_loss

for epoch in range(num_epochs):
    for X, y in data_iter(batch_size, features, labels):

```

```

l = loss(net(X, w, b), y) # X和y的小批量损失
# 因为l形状是(batch_size,1)，而不是一个标量。l中的所有元素被加到一起，
# 并以此计算关于[w,b]的梯度
l.sum().backward()
sgd([w, b], lr, batch_size) # 使用参数的梯度更新参数
with torch.no_grad():
    train_l = loss(net(features, w, b), labels)
    print(f'epoch {epoch + 1}, loss {float(train_l.mean()):f}')

print(f'w的估计误差: {true_w - native.reshape(w, true_w.shape)}')
print(f'b的估计误差: {true_b - b}')

```

第 03 章 · 线性回归 · chapters/chapter_03_linear_networks/linear_regression_concise.py

章节对应：第 03 章 类别：网络结构 代码块：11 个

结构图：线性回归（网络结构）



单层线性映射；训练目标是最小化均方误差。

阅读注释：阅读顺序：先看输入与输出，再看核心层/模块，最后看损失函数和训练循环。
nn.Linear 把输入特征映射为一个连续值，重点看预测值与均方误差之间的关系。

```

"""linear-regression-concise: 线性神经网络"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import numpy as np
import torch
from torch.utils import data

true_w = native.tensor([2, -3.4])
true_b = 4.2
features, labels = native.synthetic_data(true_w, true_b, 1000)

def load_array(data_arrays, batch_size, is_train=True): #@save
    """构造一个PyTorch数据迭代器"""
    dataset = data.TensorDataset(*data_arrays)
    return data.DataLoader(dataset, batch_size, shuffle=is_train)

batch_size = 10
data_iter = load_array((features, labels), batch_size)

next(iter(data_iter))

```

```

# nn是神经网络的缩写
from torch import nn
net = nn.Sequential(nn.Linear(2, 1))

net[0].weight.data.normal_(0, 0.01)
net[0].bias.data.fill_(0)

loss = nn.MSELoss()

trainer = torch.optim.SGD(net.parameters(), lr=0.03)

num_epochs = 3
for epoch in range(num_epochs):
    for X, y in data_iter:
        l = loss(net(X), y)

        trainer.zero_grad()
        l.backward()
        trainer.step()
        l = loss(net(features), labels)
        print(f'epoch {epoch + 1}, loss {l:f}')

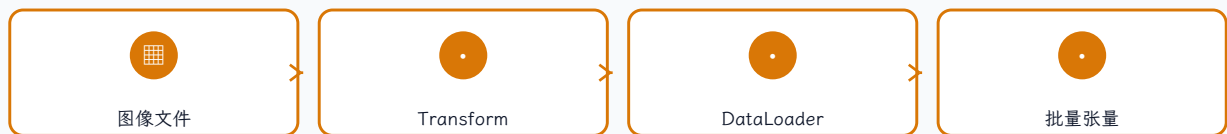
w = net[0].weight.data
print('w的估计误差: ', true_w - native.reshape(w, true_w.shape))
b = net[0].bias.data
print('b的估计误差: ', true_b - b)

```

第 03 章 · 图像分类数据管线 · chapters/chapter_03_linear_networks/image_classification_dataset.py

章节对应：第 03 章 类别：数据流程 代码块：11 个

结构图：图像分类数据管线（数据流程）



这是数据输入流程，不是新的神经网络。

阅读注释：阅读顺序：先看样本格式，再看预处理与 Dataset，最后看 DataLoader 生成的批次。
关键点：沿图中的箭头追踪张量形状，重点定位 nn.Module、forward、损失函数和训练循环。

```

"""image-classification-dataset: 线性神经网络"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import torch
import torchvision

```

```

from torchvision import transforms
from torch.utils import data

native.use_svg_display()

# 通过ToTensor实例将图像数据从PIL类型转换成32位浮点数格式。
# 并除以255使得所有像素的数值均在0~1之间
trans = transforms.ToTensor()
mnist_train = torchvision.datasets.FashionMNIST(
    root="../data", train=True, transform=trans, download=True)
mnist_test = torchvision.datasets.FashionMNIST(
    root="../data", train=False, transform=trans, download=True)

len(mnist_train), len(mnist_test)

mnist_train[0][0].shape

def get_fashion_mnist_labels(labels): #@save
    """返回Fashion-MNIST数据集的文本标签"""
    text_labels = ['t-shirt', 'trouser', 'pullover', 'dress', 'coat',
                   'sandal', 'shirt', 'sneaker', 'bag', 'ankle boot']
    return [text_labels[int(i)] for i in labels]

def show_images(imgs, num_rows, num_cols, titles=None, scale=1.5): #@save
    """绘制图像列表"""
    figsize = (num_cols * scale, num_rows * scale)
    _, axes = native.plt.subplots(num_rows, num_cols, figsize=figsize)
    axes = axes.flatten()
    for i, (ax, img) in enumerate(zip(axes, imgs)):
        if torch.is_tensor(img):
            # 图片张量
            ax.imshow(img.numpy())
        else:
            # PIL图片
            ax.imshow(img)
        ax.axes.get_xaxis().set_visible(False)
        ax.axes.get_yaxis().set_visible(False)
        if titles:
            ax.set_title(titles[i])
    return axes

X, y = next(iter(data.DataLoader(mnist_train, batch_size=18)))
show_images(X.reshape(18, 28, 28), 2, 9, titles=get_fashion_mnist_labels(y));

batch_size = 256

def get_dataloader_workers(): #@save
    """使用4个进程来读取数据"""
    return 4

train_iter = data.DataLoader(mnist_train, batch_size, shuffle=True,
                             num_workers=get_dataloader_workers())

timer = native.Timer()
for X, y in train_iter:
    continue
f'{timer.stop():.2f} sec'

def load_data_fashion_mnist(batch_size, resize=None): #@save
    """下载Fashion-MNIST数据集，然后将其加载到内存中"""
    trans = [transforms.ToTensor()]
    if resize:
        trans.insert(0, transforms.Resize(resize))
    trans = transforms.Compose(trans)
    mnist_train = torchvision.datasets.FashionMNIST(
        root="../data", train=True, transform=trans, download=True)
    mnist_test = torchvision.datasets.FashionMNIST(
        root="../data", train=False, transform=trans, download=True)
    return (data.DataLoader(mnist_train, batch_size, shuffle=True,
                            num_workers=get_dataloader_workers()),
            data.DataLoader(mnist_test, batch_size, shuffle=False,
                            num_workers=get_dataloader_workers()))

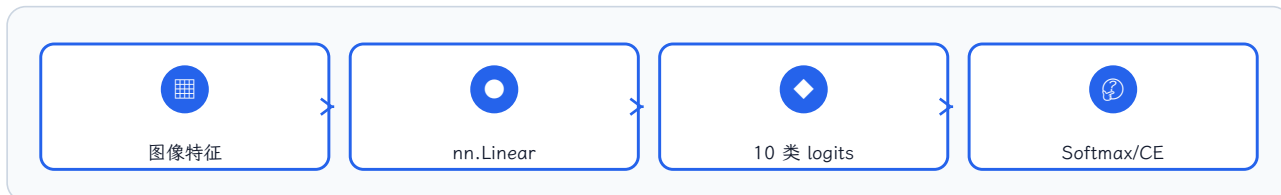
train_iter, test_iter = load_data_fashion_mnist(32, resize=64)
for X, y in train_iter:
    print(X.shape, X.dtype, y.shape, y.dtype)
    break

```

第 03 章 • Softmax 回归 • chapters/chapter_03_linear_networks/softmax_regression_scratch.py

章节对应：第 03 章 类别：网络结构 代码块：20 个

结构图：Softmax 回归（网络结构）



单层多类分类器；输出类别 logits 后交给交叉熵损失。

阅读注释：阅读顺序：先看输入与输出，再看核心层/模块，最后看损失函数和训练循环。

输出是类别 logits；使用 CrossEntropyLoss 时不要再手动对 logits 做 Softmax。

"""softmax-regression-scratch: 线性神经网络"""

来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。

改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。

from pathlib import Path

import sys

_ROOT = Path(__file__).resolve().parents[1]

if str(_ROOT) not in sys.path:

sys.path.insert(0, str(_ROOT))

import math

import random

import time

import numpy as np

import pandas as pd

import torch

from torch import nn

from torch.nn import functional as F

from torch.utils import data

import torchvision

from torchvision import transforms

from PIL import Image

from matplotlib import pyplot as plt

from IPython import display

import common as native

import common as native

import torch

from IPython import display

batch_size = 256

train_iter, test_iter = native.load_data_fashion_mnist(batch_size)

num_inputs = 784

num_outputs = 10

W = torch.normal(0, 0.01, size=(num_inputs, num_outputs), requires_grad=True)

b = torch.zeros(num_outputs, requires_grad=True)

X = native.tensor([[1.0, 2.0, 3.0], [4.0, 5.0, 6.0]])

native.reduce_sum(X, 0, keepdim=True), native.reduce_sum(X, 1, keepdim=True)

def softmax(X):

X_exp = native.exp(X)

partition = native.reduce_sum(X_exp, 1, keepdim=True)

return X_exp / partition # 这里应用了广播机制

X = native.normal(0, 1, (2, 5))

X_prob = softmax(X)

X_prob, native.reduce_sum(X_prob, 1)

def net(X):

return softmax(native.matmul(native.reshape(X, (-1, W.shape[0])), W) + b)

y = native.tensor([0, 2])

```

y_hat = native.tensor([[0.1, 0.3, 0.6], [0.3, 0.2, 0.5]])
y_hat[[0, 1], y]

def cross_entropy(y_hat, y):
    return - native.log(y_hat[range(len(y_hat)), y])

cross_entropy(y_hat, y)

def accuracy(y_hat, y): #@save
    """计算预测正确的数量"""
    if len(y_hat.shape) > 1 and y_hat.shape[1] > 1:
        y_hat = native.argmax(y_hat, axis=1)
        cmp = native.astype(y_hat, y.dtype) == y
        return float(native.reduce_sum(native.astype(cmp, y.dtype)))

accuracy(y_hat, y) / len(y)

def evaluate_accuracy(net, data_iter): #@save
    """计算在指定数据集上模型的精度"""
    if isinstance(net, torch.nn.Module):
        net.eval() # 将模型设置为评估模式
    metric = Accumulator(2) # 正确预测数、预测总数
    with torch.no_grad():
        for X, y in data_iter:
            metric.add(accuracy(net(X), y), native.size(y))
    return metric[0] / metric[1]

class Accumulator: #@save
    """在n个变量上累加"""
    def __init__(self, n):
        self.data = [0.0] * n

    def add(self, *args):
        self.data = [a + float(b) for a, b in zip(self.data, args)]

    def reset(self):
        self.data = [0.0] * len(self.data)

    def __getitem__(self, idx):
        return self.data[idx]

evaluate_accuracy(net, test_iter)

def train_epoch_ch3(net, train_iter, loss, updater): #@save
    """训练模型一个迭代周期（定义见第3章）"""
    # 将模型设置为训练模式
    if isinstance(net, torch.nn.Module):
        net.train()
    # 训练损失总和、训练准确度总和、样本数
    metric = Accumulator(3)
    for X, y in train_iter:
        # 计算梯度并更新参数
        y_hat = net(X)
        l = loss(y_hat, y)
        if isinstance(updater, torch.optim.Optimizer):
            # 使用PyTorch内置的优化器和损失函数
            updater.zero_grad()
            l.mean().backward()
            updater.step()
        else:
            # 使用定制的优化器和损失函数
            l.sum().backward()
            updater(X.shape[0])
        metric.add(float(l.sum()), accuracy(y_hat, y), y.numel())
    # 返回训练损失和训练精度
    return metric[0] / metric[2], metric[1] / metric[2]

class Animator: #@save
    """在动画中绘制数据"""
    def __init__(self, xlabel=None, ylabel=None, legend=None, xlim=None,
                 ylim=None, xscale='linear', yscale='linear',
                 fmts=('-', 'm--', 'g-', 'r:'), nrows=1, ncols=1,
                 figsize=(3.5, 2.5)):
        # 增量地绘制多条线
        if legend is None:
            legend = []

```

```

        native.use_svg_display()
        self.fig, self.axes = native.plt.subplots(nrows, ncols, figsize=figsize)
        if nrows * ncols == 1:
            self.axes = [self.axes, ]
# 使用lambda函数捕获参数
        self.config_axes = lambda: native.set_axes(
            self.axes[0], xlabel, ylabel, xlim, ylim, xscale, yscale, legend)
        self.X, self.Y, self.fmts = None, None, fmts

def add(self, x, y):
# 向图表中添加多个数据点
    if not hasattr(y, "__len__"):
        y = [y]
    n = len(y)
    if not hasattr(x, "__len__"):
        x = [x] * n
    if not self.X:
        self.X = [[] for _ in range(n)]
    if not self.Y:
        self.Y = [[] for _ in range(n)]
    for i, (a, b) in enumerate(zip(x, y)):
        if a is not None and b is not None:
            self.X[i].append(a)
            self.Y[i].append(b)
    self.axes[0].cla()
    for x, y, fmt in zip(self.X, self.Y, self.fmts):
        self.axes[0].plot(x, y, fmt)
    self.config_axes()
    display.display(self.fig)
    display.clear_output(wait=True)

def train_ch3(net, train_iter, test_iter, loss, num_epochs, updater): #@save
    """训练模型（定义见第3章）"""
    animator = Animator(xlabel='epoch', xlim=[1, num_epochs], ylim=[0.3, 0.9],
                        legend=['train loss', 'train acc', 'test acc'])
    for epoch in range(num_epochs):
        train_metrics = train_epoch_ch3(net, train_iter, loss, updater)
        test_acc = evaluate_accuracy(net, test_iter)
        animator.add(epoch + 1, train_metrics + (test_acc,))
    train_loss, train_acc = train_metrics
    assert train_loss < 0.5, train_loss
    assert train_acc <= 1 and train_acc > 0.7, train_acc
    assert test_acc <= 1 and test_acc > 0.7, test_acc

lr = 0.1

def updater(batch_size):
    return native.sgd([W, b], lr, batch_size)

num_epochs = 10
train_ch3(net, train_iter, test_iter, cross_entropy, num_epochs, updater)

def predict_ch3(net, test_iter, n=6): #@save
    """预测标签（定义见第3章）"""
    for X, y in test_iter:
        break
    trues = native.get_fashion_mnist_labels(y)
    preds = native.get_fashion_mnist_labels(native.argmax(net(X), axis=1))
    titles = [true + '\n' + pred for true, pred in zip(trues, preds)]
    native.show_images(
        native.reshape(X[0:n], (n, 28, 28)), 1, n, titles=titles[0:n])

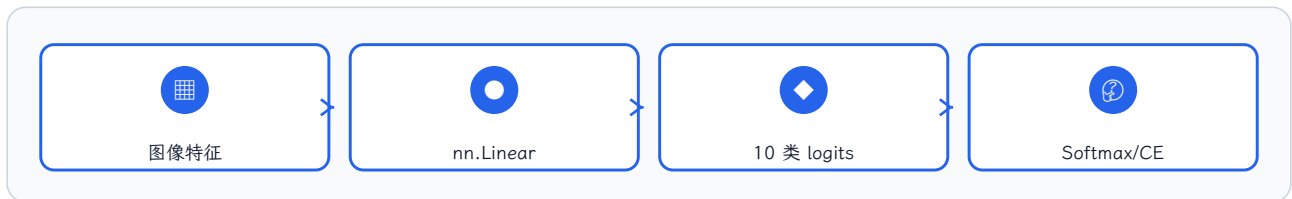
predict_ch3(net, test_iter)

```

第 03 章 • Softmax 回归 • chapters/chapter_03_linear_networks/softmax_regression_concise.py

章节对应：第 03 章 类别：网络结构 代码块：6 个

结构图：Softmax 回归（网络结构）



单层多类分类器；输出类别 logits 后交给交叉熵损失。

阅读注释：阅读顺序：先看输入与输出，再看核心层/模块，最后看损失函数和训练循环。
输出是类别 logits；使用 CrossEntropyLoss 时不要再手动对 logits 做 Softmax。

```
"""softmax-regression-concise: 线性神经网络"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import torch
from torch import nn

batch_size = 256
train_iter, test_iter = native.load_data_fashion_mnist(batch_size)

# PyTorch不会隐式地调整输入的形状。因此，
# 我们在线性层前定义了展平层（flatten），来调整网络输入的形状
net = nn.Sequential(nn.Flatten(), nn.Linear(784, 10))

def init_weights(m):
    if type(m) == nn.Linear:
        nn.init.normal_(m.weight, std=0.01)

net.apply(init_weights);

loss = nn.CrossEntropyLoss(reduction='none')

trainer = torch.optim.SGD(net.parameters(), lr=0.1)

num_epochs = 10
native.train_ch3(net, train_iter, test_iter, loss, num_epochs, trainer)
```

第 04 章：多层感知机

本章分类：多层感知机与正则化 章节说明：MLP、Dropout、权重衰减、初始化和房价回归。

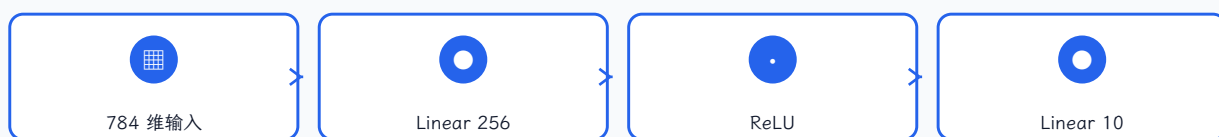
本章网络/流程总览



第 04 章 · 多层感知机 (MLP) · [chapters/chapter_04_multilayer_perceptrons/mlp.py](#)

章节对应：第 04 章 类别：网络结构 代码块：7 个

结构图：多层感知机 (MLP) (网络结构)



全连接分类网络；本章用它从线性模型过渡到深层网络。

阅读注释：阅读顺序：先看输入与输出，再看核心层/模块，最后看损失函数和训练循环。
ReLU 把多个线性层串成非线性网络，阅读时先确认每层的输入和输出维度。

```
"""mlp: 多层感知机"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import torch

x = torch.arange(-8.0, 8.0, 0.1, requires_grad=True)
y = torch.relu(x)
native.plot(x.detach(), y.detach(), 'x', 'relu(x)', figsize=(5, 2.5))

y.backward(torch.ones_like(x), retain_graph=True)
native.plot(x.detach(), x.grad, 'x', 'grad of relu', figsize=(5, 2.5))

y = torch.sigmoid(x)
native.plot(x.detach(), y.detach(), 'x', 'sigmoid(x)', figsize=(5, 2.5))

# 清除以前的梯度
```

```

x.grad.data.zero_()
y.backward(torch.ones_like(x), retain_graph=True)
native.plot(x.detach(), x.grad, 'x', 'grad of sigmoid', figsize=(5, 2.5))

y = torch.tanh(x)
native.plot(x.detach(), y.detach(), 'x', 'tanh(x)', figsize=(5, 2.5))

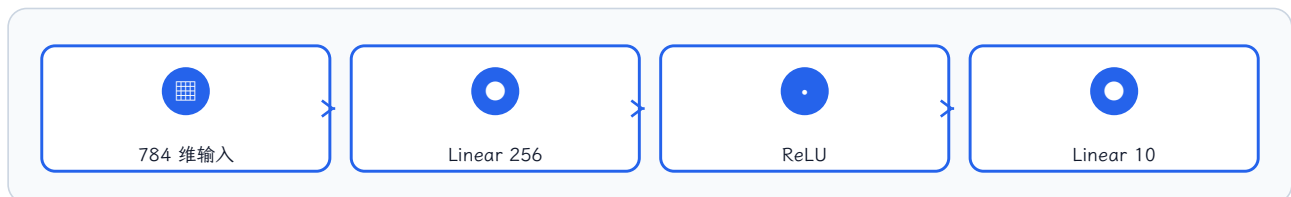
# 清除以前的梯度
x.grad.data.zero_()
y.backward(torch.ones_like(x), retain_graph=True)
native.plot(x.detach(), x.grad, 'x', 'grad of tanh', figsize=(5, 2.5))

```

第 04 章 · 多层感知机 (MLP) · chapters/chapter_04_multilayer_perceptrons/mlp_scratch.py

章节对应: 第 04 章 类别: 网络结构 代码块: 8 个

结构图: 多层感知机 (MLP) (网络结构)



全连接分类网络; 本章用它从线性模型过渡到深层网络。

阅读注释: 阅读顺序: 先看输入与输出, 再看核心层/模块, 最后看损失函数和训练循环。
ReLU 把多个线性层串成非线性网络, 阅读时先确认每层的输入和输出维度。

```

"""mlp-scratch: 多层感知机"""

# 来源: 依据《动手学深度学习 (PyTorch 第二版)》的 D2L 中文 PyTorch 示例整理。
# 改写说明: 已移除 d2l 命名空间, 共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import torch
from torch import nn

batch_size = 256
train_iter, test_iter = native.load_data_fashion_mnist(batch_size)

num_inputs, num_outputs, num_hiddens = 784, 10, 256

W1 = nn.Parameter(torch.randn(
    num_inputs, num_hiddens, requires_grad=True) * 0.01)
b1 = nn.Parameter(torch.zeros(num_hiddens, requires_grad=True))
W2 = nn.Parameter(torch.randn(
    num_hiddens, num_outputs, requires_grad=True) * 0.01)
b2 = nn.Parameter(torch.zeros(num_outputs, requires_grad=True))

params = [W1, b1, W2, b2]

```

```

def relu(X):
    a = torch.zeros_like(X)
    return torch.max(X, a)

def net(X):
    X = native.reshape(X, (-1, num_inputs))
    H = relu(X@W1 + b1) # 这里"@"代表矩阵乘法
    return (H@W2 + b2)

loss = nn.CrossEntropyLoss(reduction='none')

num_epochs, lr = 10, 0.1
updater = torch.optim.SGD(params, lr=lr)
native.train_ch3(net, train_iter, test_iter, loss, num_epochs, updater)

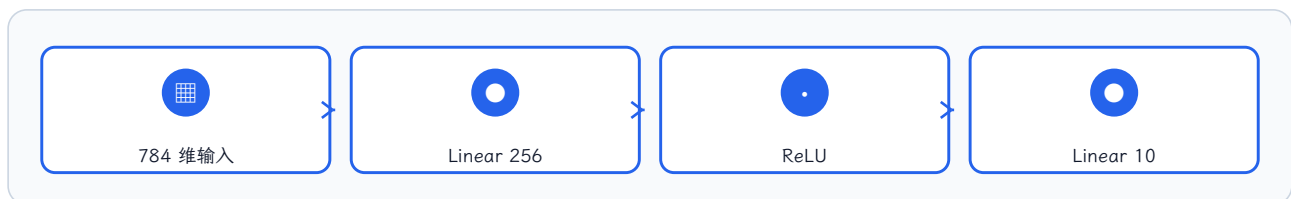
native.predict_ch3(net, test_iter)

```

第 04 章 · 多层感知机 (MLP) · chapters/chapter_04_multilayer_perceptrons/mlp_concise.py

章节对应: 第 04 章 类别: 网络结构 代码块: 4 个

结构图: 多层感知机 (MLP) (网络结构)



全连接分类网络; 本章用它从线性模型过渡到深层网络。

阅读注释: 阅读顺序: 先看输入与输出, 再看核心层/模块, 最后看损失函数和训练循环。

ReLU 把多个线性层串成非线性网络, 阅读时先确认每层的输入和输出维度。

```

"""mlp-concise: 多层感知机"""

# 来源: 依据《动手学深度学习 (PyTorch 第二版)》的 D2L 中文 PyTorch 示例整理。
# 改写说明: 已移除 d2l 命名空间, 共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import torch
from torch import nn

net = nn.Sequential(nn.Flatten(),
                    nn.Linear(784, 256),
                    nn.ReLU(),
                    nn.Linear(256, 10))

def init_weights(m):
    if type(m) == nn.Linear:
        nn.init.normal_(m.weight, std=0.01)

```

```
net.apply(init_weights);

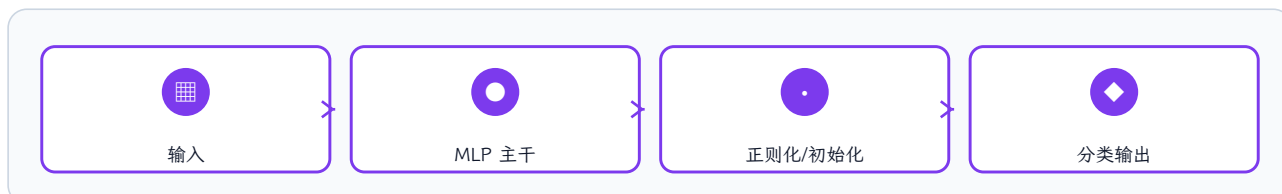
batch_size, lr, num_epochs = 256, 0.1, 10
loss = nn.CrossEntropyLoss(reduction='none')
trainer = torch.optim.SGD(net.parameters(), lr=lr)

train_iter, test_iter = native.load_data_fashion_mnist(batch_size)
native.train_ch3(net, train_iter, test_iter, loss, num_epochs, trainer)
```

第 04 章 · MLP 的泛化与正则化 · chapters/chapter_04_multilayer_perceptrons/underfit_overfit.py

章节对应：第 04 章 类别：训练/正则化 代码块：9 个

结构图：MLP 的泛化与正则化（训练/正则化）



这些小节分析欠拟合、过拟合、权重衰减、Dropout 与初始化。

阅读注释：阅读顺序：先看参数和梯度，再看更新规则，最后看学习率或正则化设置。

关键点：沿图中的箭头追踪张量形状，重点定位 nn.Module、forward、损失函数和训练循环。

"""underfit-overfit: 多层感知机"""

来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。

改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。

from pathlib import Path

import sys

_ROOT = Path(__file__).resolve().parents[1]

if str(_ROOT) not in sys.path:

sys.path.insert(0, str(_ROOT))

import math

import random

import time

import numpy as np

import pandas as pd

import torch

from torch import nn

from torch.nn import functional as F

from torch.utils import data

import torchvision

from torchvision import transforms

from PIL import Image

from matplotlib import pyplot as plt

from IPython import display

import common as native

import common as native

import torch

from torch import nn

import numpy as np

import math

max_degree = 20 # 多项式的最大阶数

n_train, n_test = 100, 100 # 训练和测试数据集大小

true_w = np.zeros(max_degree) # 分配大量的空间

true_w[0:4] = np.array([5, 1.2, -3.4, 5.6])

features = np.random.normal(size=(n_train + n_test, 1))

np.random.shuffle(features)

poly_features = np.power(features, np.arange(max_degree).reshape(1, -1))

for i in range(max_degree):

poly_features[:, i] /= math.gamma(i + 1) # gamma(n)=(n-1)!

labels的维度:(n_train+n_test,)

labels = np.dot(poly_features, true_w)

labels += np.random.normal(scale=0.1, size=labels.shape)

NumPy ndarray转换为tensor

```

true_w, features, poly_features, labels = [native.tensor(x, dtype=
    native.float32) for x in [true_w, features, poly_features, labels]]

features[:2], poly_features[:2, :], labels[:2]

def evaluate_loss(net, data_iter, loss): # @save
    """评估给定数据集上模型的损失"""
    metric = native.Accumulator(2) # 损失的总和,样本数量
    for X, y in data_iter:
        out = net(X)
        y = native.reshape(y, out.shape)
        l = loss(out, y)
        metric.add(native.reduce_sum(l), native.size(l))

    return metric[0] / metric[1]

def train(train_features, test_features, train_labels, test_labels,
    num_epochs=400):
    loss = nn.MSELoss(reduction='none')
    input_shape = train_features.shape[-1]
    # 不设置偏置, 因为我们已经在多项式中实现了它
    net = nn.Sequential(nn.Linear(input_shape, 1, bias=False))
    batch_size = min(10, train_labels.shape[0])
    train_iter = native.load_array((train_features, train_labels.reshape(-1,1)),
        batch_size)
    test_iter = native.load_array((test_features, test_labels.reshape(-1,1)),
        batch_size, is_train=False)
    trainer = torch.optim.SGD(net.parameters(), lr=0.01)
    animator = native.Animator(xlabel='epoch', ylabel='loss', yscale='log',
        xlim=[1, num_epochs], ylim=[1e-3, 1e2],
        legend=['train', 'test'])
    for epoch in range(num_epochs):
        native.train_epoch_ch3(net, train_iter, loss, trainer)
        if epoch == 0 or (epoch + 1) % 20 == 0:
            animator.add(epoch + 1, (evaluate_loss(net, train_iter, loss),
                evaluate_loss(net, test_iter, loss)))
    print('weight:', net[0].weight.data.numpy())

# 从多项式特征中选择前4个维度, 即1,x,x^2/2!,x^3/3!
train(poly_features[:n_train, :4], poly_features[n_train:, :4],
    labels[:n_train], labels[n_train:])

# 从多项式特征中选择前2个维度, 即1和x
train(poly_features[:n_train, :2], poly_features[n_train:, :2],
    labels[:n_train], labels[n_train:])

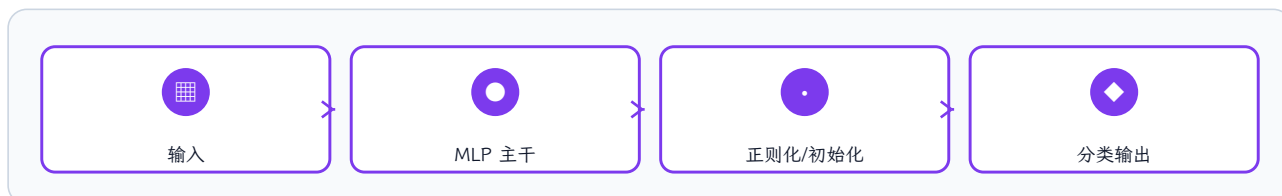
# 从多项式特征中选取所有维度
train(poly_features[:n_train, :], poly_features[n_train:, :],
    labels[:n_train], labels[n_train:], num_epochs=1500)

```

第 04 章 · MLP 的泛化与正则化 · chapters/chapter_04_multilayer_perceptrons/weight_decay.py

章节对应: 第 04 章 类别: 训练/正则化 代码块: 10 个

结构图: MLP 的泛化与正则化 (训练/正则化)



这些小节分析欠拟合、过拟合、权重衰减、Dropout 与初始化。

阅读注释: 阅读顺序: 先看参数和梯度, 再看更新规则, 最后看学习率或正则化设置。

关键点: 沿图中的箭头追踪张量形状, 重点定位 nn.Module、forward、损失函数和训练循环。

```

"""weight-decay: 多层感知机"""

# 来源: 依据《动手学深度学习 (PyTorch 第二版)》的 D2L 中文 PyTorch 示例整理。
# 改写说明: 已移除 d2l 命名空间, 共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]

```

```

if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import torch
from torch import nn

n_train, n_test, num_inputs, batch_size = 20, 100, 200, 5
true_w, true_b = native.ones((num_inputs, 1)) * 0.01, 0.05
train_data = native.synthetic_data(true_w, true_b, n_train)
train_iter = native.load_array(train_data, batch_size)
test_data = native.synthetic_data(true_w, true_b, n_test)
test_iter = native.load_array(test_data, batch_size, is_train=False)

def init_params():
    w = torch.normal(0, 1, size=(num_inputs, 1), requires_grad=True)
    b = torch.zeros(1, requires_grad=True)
    return [w, b]

def l2_penalty(w):
    return torch.sum(w.pow(2)) / 2

def train(lambd):
    w, b = init_params()
    net, loss = lambda X: native.linreg(X, w, b), native.squared_loss
    num_epochs, lr = 100, 0.003
    animator = native.Animator(xlabel='epochs', ylabel='loss', yscale='log',
                               xlim=[5, num_epochs], legend=['train', 'test'])
    for epoch in range(num_epochs):
        for X, y in train_iter:
            # 增加了L2范数惩罚项,
            # 广播机制使l2_penalty(w)成为一个长度为batch_size的向量
            l = loss(net(X), y) + lambd * l2_penalty(w)
            l.sum().backward()
            native.sgd([w, b], lr, batch_size)
        if (epoch + 1) % 5 == 0:
            animator.add(epoch + 1, (native.evaluate_loss(net, train_iter, loss),
                                     native.evaluate_loss(net, test_iter, loss)))

    print('w的L2范数是: ', torch.norm(w).item())

train(lambd=0)

train(lambd=3)

def train_concise(wd):
    net = nn.Sequential(nn.Linear(num_inputs, 1))
    for param in net.parameters():
        param.data.normal_()
    loss = nn.MSELoss(reduction='none')
    num_epochs, lr = 100, 0.003
    # 偏置参数没有衰减
    trainer = torch.optim.SGD([
        {"params": net[0].weight, 'weight_decay': wd},
        {"params": net[0].bias}], lr=lr)
    animator = native.Animator(xlabel='epochs', ylabel='loss', yscale='log',
                               xlim=[5, num_epochs], legend=['train', 'test'])
    for epoch in range(num_epochs):
        for X, y in train_iter:
            trainer.zero_grad()
            l = loss(net(X), y)
            l.mean().backward()

```

```

        trainer.step()
    if (epoch + 1) % 5 == 0:
        animator.add(epoch + 1,
                      (native.evaluate_loss(net, train_iter, loss),
                       native.evaluate_loss(net, test_iter, loss)))
    print('w的L2范数: ', net[0].weight.norm().item())

train_concise(0)

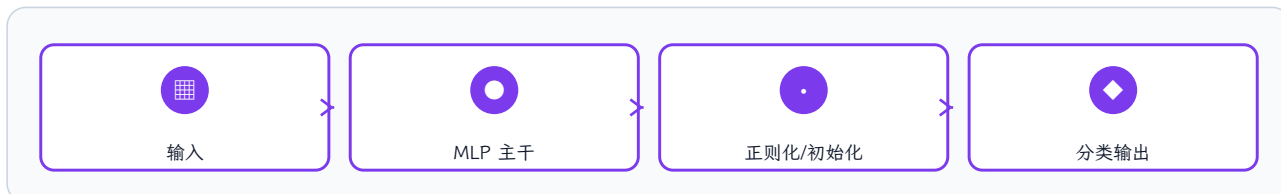
train_concise(3)

```

第 04 章 · MLP 的泛化与正则化 · chapters/chapter_04_multilayer_perceptrons/dropout.py

章节对应：第 04 章 类别：训练/正则化 代码块：7 个

结构图：MLP 的泛化与正则化（训练/正则化）



这些小节分析欠拟合、过拟合、权重衰减、Dropout 与初始化。

阅读注释：阅读顺序：先看参数和梯度，再看更新规则，最后看学习率或正则化设置。

关键点：沿图中的箭头追踪张量形状，重点定位 `nn.Module`、`forward`、损失函数和训练循环。

```

"""dropout: 多层感知机"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import torch
from torch import nn

def dropout_layer(X, dropout):
    assert 0 <= dropout <= 1
    # 在本情况中，所有元素都被丢弃
    if dropout == 1:
        return torch.zeros_like(X)
    # 在本情况中，所有元素都被保留
    if dropout == 0:
        return X
    mask = (torch.rand(X.shape) > dropout).float()
    return mask * X / (1.0 - dropout)

X = torch.arange(16, dtype = torch.float32).reshape((2, 8))
print(X)
print(dropout_layer(X, 0.))
print(dropout_layer(X, 0.5))

```

```

print(dropout_layer(X, 1.))

num_inputs, num_outputs, num_hiddens1, num_hiddens2 = 784, 10, 256, 256

dropout1, dropout2 = 0.2, 0.5

class Net(nn.Module):
    def __init__(self, num_inputs, num_outputs, num_hiddens1, num_hiddens2,
                  is_training = True):
        super(Net, self).__init__()
        self.num_inputs = num_inputs
        self.training = is_training
        self.lin1 = nn.Linear(num_inputs, num_hiddens1)
        self.lin2 = nn.Linear(num_hiddens1, num_hiddens2)
        self.lin3 = nn.Linear(num_hiddens2, num_outputs)
        self.relu = nn.ReLU()

    def forward(self, X):
        H1 = self.relu(self.lin1(X.reshape((-1, self.num_inputs))))
        # 只有在训练模型时才使用dropout
        if self.training == True:
            # 在第一个全连接层之后添加一个dropout层
            H1 = dropout_layer(H1, dropout1)
        H2 = self.relu(self.lin2(H1))
        if self.training == True:
            # 在第二个全连接层之后添加一个dropout层
            H2 = dropout_layer(H2, dropout2)
        out = self.lin3(H2)
        return out

net = Net(num_inputs, num_outputs, num_hiddens1, num_hiddens2)

num_epochs, lr, batch_size = 10, 0.5, 256
loss = nn.CrossEntropyLoss(reduction='none')
train_iter, test_iter = native.load_data_fashion_mnist(batch_size)
trainer = torch.optim.SGD(net.parameters(), lr=lr)
native.train_ch3(net, train_iter, test_iter, loss, num_epochs, trainer)

net = nn.Sequential(nn.Flatten(),
                    nn.Linear(784, 256),
                    nn.ReLU(),
                    # 在第一个全连接层之后添加一个dropout层
                    nn.Dropout(dropout1),
                    nn.Linear(256, 256),
                    nn.ReLU(),
                    # 在第二个全连接层之后添加一个dropout层
                    nn.Dropout(dropout2),
                    nn.Linear(256, 10))

def init_weights(m):
    if type(m) == nn.Linear:
        nn.init.normal_(m.weight, std=0.01)

net.apply(init_weights);

trainer = torch.optim.SGD(net.parameters(), lr=lr)
native.train_ch3(net, train_iter, test_iter, loss, num_epochs, trainer)

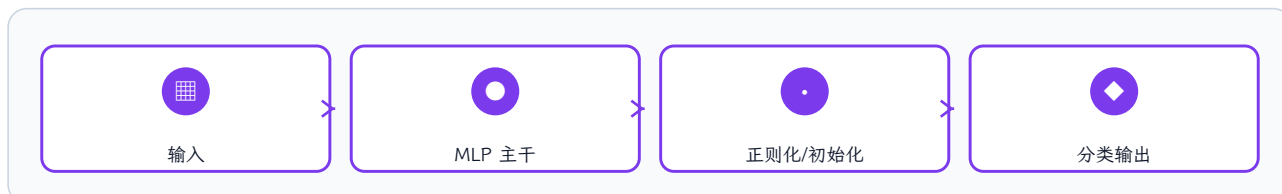
```

第 04 章 · MLP 的泛化与正则化 ·

chapters/chapter_04_multilayer_perceptrons/numerical_stability_and_init.py

章节对应：第 04 章 类别：训练/正则化 代码块：2 个

结构图：MLP 的泛化与正则化（训练/正则化）



这些小节分析欠拟合、过拟合、权重衰减、Dropout 与初始化。

阅读注释：阅读顺序：先看参数和梯度，再看更新规则，最后看学习率或正则化设置。

关键点：沿图中的箭头追踪张量形状，重点定位 nn.Module、forward、损失函数和训练循环。

"""numerical-stability-and-init: 多层感知机"""

来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。

改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。

from pathlib import Path

import sys

_ROOT = Path(__file__).resolve().parents[1]

if str(_ROOT) not in sys.path:

sys.path.insert(0, str(_ROOT))

import math

import random

import time

import numpy as np

import pandas as pd

import torch

from torch import nn

from torch.nn import functional as F

from torch.utils import data

import torchvision

from torchvision import transforms

from PIL import Image

from matplotlib import pyplot as plt

from IPython import display

import common as native

import common as native

import torch

x = torch.arange(-8.0, 8.0, 0.1, requires_grad=True)

y = torch.sigmoid(x)

y.backward(torch.ones_like(x))

native.plot(x.detach().numpy(), [y.detach().numpy(), x.grad.numpy()],
legend=['sigmoid', 'gradient'], figsize=(4.5, 2.5))

M = torch.normal(0, 1, size=(4,4))

print('一个矩阵\n', M)

for i in range(100):

M = torch.mm(M, torch.normal(0, 1, size=(4, 4)))

print('乘以100个矩阵后\n', M)

章节对应：第 04 章 类别：网络结构 代码块：20 个

结构图：MLP 回归（网络结构）



面向连续值房价预测的全连接回归网络。

阅读注释：阅读顺序：先看输入与输出，再看核心层/模块，最后看损失函数和训练循环。

关键点：沿图中的箭头追踪张量形状，重点定位 `nn.Module`、`forward`、损失函数和训练循环。

```
"""kaggle-house-price: 多层感知机"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import os
import requests
import zipfile
import tarfile
import hashlib

DATA_HUB = dict()
DATA_URL = 'http://d2l-data.s3-accelerate.amazonaws.com/'

def download(name, cache_dir=os.path.join '..', 'data'): #@save
    """下载一个DATA_HUB中的文件，返回本地文件名"""
    assert name in DATA_HUB, f'{name} 不存在于 {DATA_HUB}'
    url, sha1_hash = DATA_HUB[name]
    os.makedirs(cache_dir, exist_ok=True)
    fname = os.path.join(cache_dir, url.split('/')[-1])
    if os.path.exists(fname):
        sha1 = hashlib.sha1()
        with open(fname, 'rb') as f:
            while True:
                data = f.read(1048576)
                if not data:
                    break
                sha1.update(data)
        if sha1.hexdigest() == sha1_hash:
            return fname # 命中缓存
    print(f'正在从{url}下载{fname}...')
    r = requests.get(url, stream=True, verify=True)
    with open(fname, 'wb') as f:
        f.write(r.content)
    return fname
```

```

def download_extract(name, folder=None): #@save
    """下载并解压zip/tar文件"""
    fname = download(name)

    base_dir = os.path.dirname(fname)
    data_dir, ext = os.path.splitext(fname)
    if ext == '.zip':
        fp = zipfile.ZipFile(fname, 'r')
    elif ext in ('.tar', '.gz'):
        fp = tarfile.open(fname, 'r')
    else:
        assert False, '只有zip/tar文件可以被解压缩'
    fp.extractall(base_dir)
    return os.path.join(base_dir, folder) if folder else data_dir

def download_all(): #@save
    """下载DATA_HUB中的所有文件"""
    for name in DATA_HUB:
        download(name)

# 如果没有安装pandas, 请取消下一行的注释
# !pip install pandas

import common as native
import torch
from torch import nn
import pandas as pd
import numpy as np

DATA_HUB['kaggle_house_train'] = ( #@save
    DATA_URL + 'kaggle_house_pred_train.csv',
    '585e9cc93e70b39160e7921475f9bcd7d31219ce')

DATA_HUB['kaggle_house_test'] = ( #@save
    DATA_URL + 'kaggle_house_pred_test.csv',
    'fa19780a7b011d9b009e8bff8e99922a8ee2eb90')

train_data = pd.read_csv(download('kaggle_house_train'))
test_data = pd.read_csv(download('kaggle_house_test'))

print(train_data.shape)
print(test_data.shape)

print(train_data.iloc[0:4, [0, 1, 2, 3, -3, -2, -1]])

all_features = pd.concat((train_data.iloc[:, 1:-1], test_data.iloc[:, 1:]))

# 若无法获得测试数据, 则可根据训练数据计算均值和标准差
numeric_features = all_features.dtypes[all_features.dtypes != 'object'].index
all_features[numeric_features] = all_features[numeric_features].apply(
    lambda x: (x - x.mean()) / (x.std()))
# 在标准化数据之后, 所有均值消失, 因此我们可以将缺失值设置为0
all_features[numeric_features] = all_features[numeric_features].fillna(0)

# "Dummy_na=True"将"na" (缺失值) 视为有效的特征值, 并为其创建指示符特征
all_features = pd.get_dummies(all_features, dummy_na=True)
all_features.shape

n_train = train_data.shape[0]
train_features = native.tensor(all_features[:n_train].values, dtype=native.float32)
test_features = native.tensor(all_features[n_train:].values, dtype=native.float32)
train_labels = native.tensor(
    train_data.SalePrice.values.reshape(-1, 1), dtype=native.float32)

loss = nn.MSELoss()
in_features = train_features.shape[1]

def get_net():
    net = nn.Sequential(nn.Linear(in_features, 1))
    return net

def log_rmse(net, features, labels):
    # 为了在取对数时进一步稳定该值, 将小于1的值设置为1
    clipped_preds = torch.clamp(net(features), 1, float('inf'))
    rmse = torch.sqrt(loss(torch.log(clipped_preds),
                           torch.log(labels)))

```

```

    return rmse.item()

def train(net, train_features, train_labels, test_features, test_labels,
         num_epochs, learning_rate, weight_decay, batch_size):
    train_ls, test_ls = [], []
    train_iter = native.load_array((train_features, train_labels), batch_size)
    # 这里使用的是Adam优化算法
    optimizer = torch.optim.Adam(net.parameters(),
                                  lr = learning_rate,
                                  weight_decay = weight_decay)
    for epoch in range(num_epochs):
        for X, y in train_iter:
            optimizer.zero_grad()
            l = loss(net(X), y)
            l.backward()
            optimizer.step()
            train_ls.append(log_rmse(net, train_features, train_labels))
            if test_labels is not None:
                test_ls.append(log_rmse(net, test_features, test_labels))
    return train_ls, test_ls

def get_k_fold_data(k, i, X, y):
    assert k > 1
    fold_size = X.shape[0] // k
    X_train, y_train = None, None
    for j in range(k):
        idx = slice(j * fold_size, (j + 1) * fold_size)
        X_part, y_part = X[idx, :], y[idx]
        if j == i:
            X_valid, y_valid = X_part, y_part
        elif X_train is None:
            X_train, y_train = X_part, y_part
        else:
            X_train = native.concat([X_train, X_part], 0)
            y_train = native.concat([y_train, y_part], 0)
    return X_train, y_train, X_valid, y_valid

def k_fold(k, X_train, y_train, num_epochs, learning_rate, weight_decay,
         batch_size):
    train_l_sum, valid_l_sum = 0, 0
    for i in range(k):
        data = get_k_fold_data(k, i, X_train, y_train)
        net = get_net()
        train_ls, valid_ls = train(net, *data, num_epochs, learning_rate,
                                   weight_decay, batch_size)
        train_l_sum += train_ls[-1]
        valid_l_sum += valid_ls[-1]
        if i == 0:
            native.plot(list(range(1, num_epochs + 1)), [train_ls, valid_ls],
                        xlabel='epoch', ylabel='rmse', xlim=[1, num_epochs],
                        legend=['train', 'valid'],yscale='log')
    print(f'折{i + 1}, 训练log rmse{float(train_ls[-1]):f}, '
          f'验证log rmse{float(valid_ls[-1]):f}')
    return train_l_sum / k, valid_l_sum / k

k, num_epochs, lr, weight_decay, batch_size = 5, 100, 5, 0, 64
train_l, valid_l = k_fold(k, train_features, train_labels, num_epochs, lr,
                          weight_decay, batch_size)
print(f'{k}-折验证: 平均训练log rmse: {float(train_l):f}, '
      f'平均验证log rmse: {float(valid_l):f}')

def train_and_pred(train_features, test_features, train_labels, test_data,
                  num_epochs, lr, weight_decay, batch_size):
    net = get_net()
    train_ls, _ = train(net, train_features, train_labels, None, None,
                        num_epochs, lr, weight_decay, batch_size)
    native.plot(np.arange(1, num_epochs + 1), [train_ls], xlabel='epoch',
               ylabel='log rmse', xlim=[1, num_epochs], yscale='log')
    print(f'训练log rmse: {float(train_ls[-1]):f}')
    # 将网络应用于测试集。
    preds = native.numpy(net(test_features))
    # 将其重新格式化以导出到Kaggle
    test_data['SalePrice'] = pd.Series(preds.reshape(1, -1)[0])
    submission = pd.concat([test_data['Id'], test_data['SalePrice']], axis=1)
    submission.to_csv('submission.csv', index=False)

train_and_pred(train_features, test_features, train_labels, test_data,

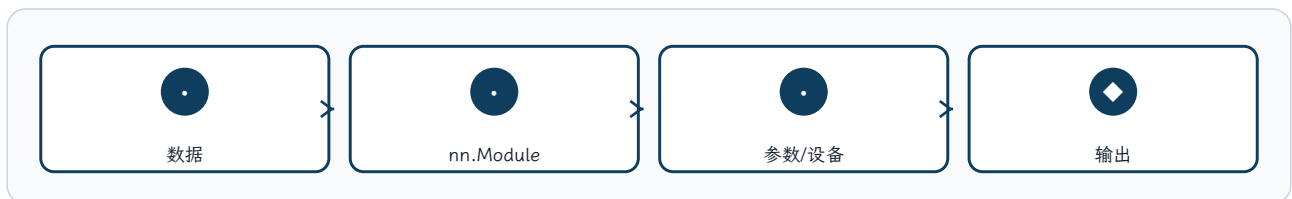
```

```
num_epochs, lr, weight_decay, batch_size)
```

第 05 章：深度学习计算

本章分类：深度学习计算 章节说明：模块组合、参数管理、设备迁移和 LazyLinear；重点是 PyTorch 工程表达。

本章网络/流程总览

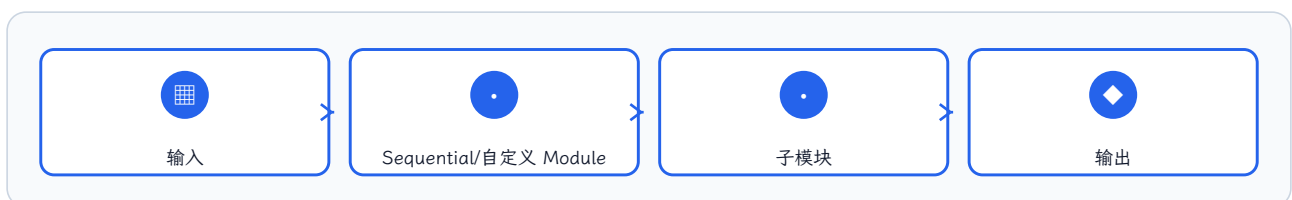


第 05 章 · nn.Module 组合模型 ·

[chapters/chapter_05_deep_learning_computation/model_construction.py](#)

章节对应：第 05 章 类别：网络结构 代码块：8 个

结构图：nn.Module 组合模型（网络结构）



展示 PyTorch 如何用模块嵌套表达网络结构。

阅读注释：阅读顺序：先看输入与输出，再看核心层/模块，最后看损失函数和训练循环。

关键点：沿图中的箭头追踪张量形状，重点定位 nn.Module、forward、损失函数和训练循环。

"""model-construction: 深度学习计算"""

来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。

改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。

from pathlib import Path

import sys

_ROOT = Path(__file__).resolve().parents[1]

if str(_ROOT) not in sys.path:
 sys.path.insert(0, str(_ROOT))

import math

import random

import time

import numpy as np

import pandas as pd

import torch

from torch import nn

from torch.nn import functional as F

from torch.utils import data

import torchvision

from torchvision import transforms

from PIL import Image

from matplotlib import pyplot as plt

from IPython import display

import common as native

import torch

from torch import nn

from torch.nn import functional as F

net = nn.Sequential(nn.Linear(20, 256), nn.ReLU(), nn.Linear(256, 10))

X = torch.rand(2, 20)

net(X)

class MLP(nn.Module):

用模型参数声明层。这里，我们声明两个全连接的层

def __init__(self):

```

# 调用MLP的父类Module的构造函数来执行必要的初始化。
# 这样，在类实例化时也可以指定其他函数参数，例如模型参数params（稍后将介绍）
    super().__init__()
self.hidden = nn.Linear(20, 256) # 隐藏层
self.out = nn.Linear(256, 10) # 输出层

# 定义模型的前向传播，即如何根据输入X返回所需的模型输出
def forward(self, X):
    # 注意，这里我们使用ReLU的函数版本，其在nn.functional模块中定义。
    return self.out(F.relu(self.hidden(X)))

net = MLP()
net(X)

class MySequential(nn.Module):
    def __init__(self, *args):
        super().__init__()
        for idx, module in enumerate(args):
            # 这里，module是Module子类的一个实例。我们把它保存在'Module'类的成员
            # 变量_modules中。_module的类型是OrderedDict
            self._modules[str(idx)] = module

    def forward(self, X):
        # OrderedDict保证了按照成员添加的顺序遍历它们
        for block in self._modules.values():
            X = block(X)
        return X

net = MySequential(nn.Linear(20, 256), nn.ReLU(), nn.Linear(256, 10))
net(X)

class FixedHiddenMLP(nn.Module):
    def __init__(self):
        super().__init__()
        # 不计算梯度的随机权重参数。因此其在训练期间保持不变
        self.rand_weight = torch.rand((20, 20), requires_grad=False)
        self.linear = nn.Linear(20, 20)

    def forward(self, X):
        X = self.linear(X)
        # 使用创建的常量参数以及relu和mm函数
        X = F.relu(torch.mm(X, self.rand_weight) + 1)
        # 复用全连接层。这相当于两个全连接层共享参数
        X = self.linear(X)
        # 控制流
        while X.abs().sum() > 1:
            X /= 2
        return X.sum()

net = FixedHiddenMLP()
net(X)

class NestMLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(20, 64), nn.ReLU(),
                                nn.Linear(64, 32), nn.ReLU())
        self.linear = nn.Linear(32, 16)

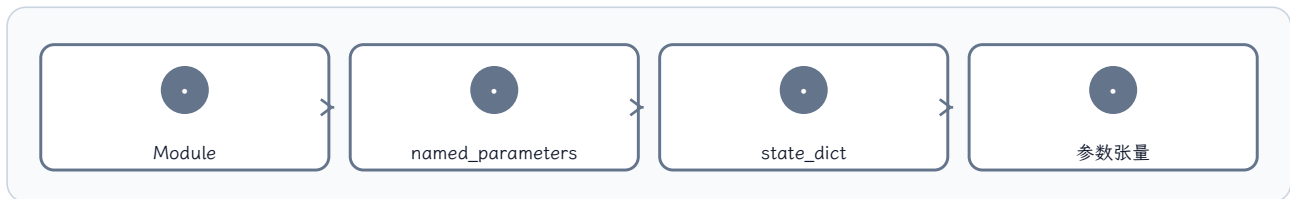
    def forward(self, X):
        return self.linear(self.net(X))

chimera = nn.Sequential(NestMLP(), nn.Linear(16, 20), FixedHiddenMLP())
chimera(X)

```

章节对应：第 05 章 类别：模型流程 代码块：15 个

结构图：参数管理（模型流程）



这是参数注册与访问流程，不对应单独网络。

阅读注释：阅读顺序：先看数据/模型所在设备，再看执行或保存边界，最后看输出结果。

关键点：沿图中的箭头追踪张量形状，重点定位 `nn.Module`、`forward`、损失函数和训练循环。

```

"""parameters: 深度学习计算"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import torch
from torch import nn

net = nn.Sequential(nn.Linear(4, 8), nn.ReLU(), nn.Linear(8, 1))
X = torch.rand(size=(2, 4))
net(X)

print(net[2].state_dict())

print(type(net[2].bias))
print(net[2].bias)
print(net[2].bias.data)

net[2].weight.grad == None

print(*[(name, param.shape) for name, param in net[0].named_parameters()])
print(*[(name, param.shape) for name, param in net.named_parameters()])

net.state_dict()['2.bias'].data

def block1():
    return nn.Sequential(nn.Linear(4, 8), nn.ReLU(),
                        nn.Linear(8, 4), nn.ReLU())

def block2():
    net = nn.Sequential()
    for i in range(4):
        # 在这里嵌套
        net.add_module(f'block {i}', block1())
    return net
  
```

```

rgnet = nn.Sequential(block2(), nn.Linear(4, 1))
rgnet(X)

print(rgnet)

rgnet[0][1][0].bias.data

def init_normal(m):
    if type(m) == nn.Linear:
        nn.init.normal_(m.weight, mean=0, std=0.01)
        nn.init.zeros_(m.bias)
net.apply(init_normal)
net[0].weight.data[0], net[0].bias.data[0]

def init_constant(m):
    if type(m) == nn.Linear:
        nn.init.constant_(m.weight, 1)
        nn.init.zeros_(m.bias)
net.apply(init_constant)
net[0].weight.data[0], net[0].bias.data[0]

def init_xavier(m):
    if type(m) == nn.Linear:
        nn.init.xavier_uniform_(m.weight)
def init_42(m):
    if type(m) == nn.Linear:
        nn.init.constant_(m.weight, 42)

net[0].apply(init_xavier)
net[2].apply(init_42)
print(net[0].weight.data[0])
print(net[2].weight.data)

def my_init(m):
    if type(m) == nn.Linear:
        print("Init", *[(name, param.shape)
                        for name, param in m.named_parameters()][0])
        nn.init.uniform_(m.weight, -10, 10)
        m.weight.data *= m.weight.data.abs() >= 5

net.apply(my_init)
net[0].weight[:2]

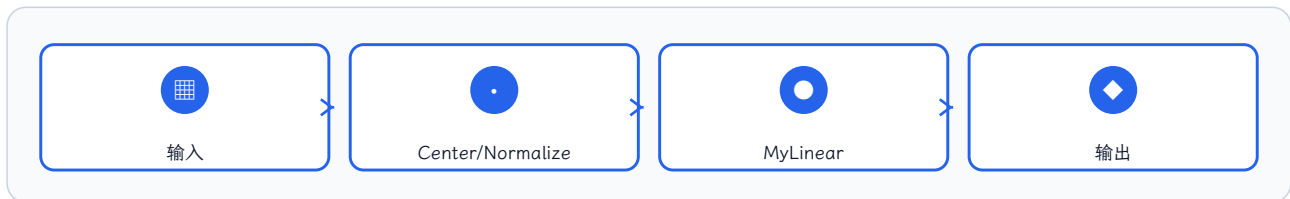
net[0].weight.data[:] += 1
net[0].weight.data[0, 0] = 42
net[0].weight.data[0]

# 我们需要给共享层一个名称，以便可以引用它的参数
shared = nn.Linear(8, 8)
net = nn.Sequential(nn.Linear(4, 8), nn.ReLU(),
                    shared, nn.ReLU(),
                    shared, nn.ReLU(),
                    nn.Linear(8, 1))
net(X)
# 检查参数是否相同
print(net[2].weight.data[0] == net[4].weight.data[0])
net[2].weight.data[0, 0] = 100
# 确保它们实际上是同一个对象，而不只是有相同的值
print(net[2].weight.data[0] == net[4].weight.data[0])

```

章节对应：第 05 章 类别：网络结构 代码块：8 个

结构图：自定义层 MLP（网络结构）



把自定义层组合到 PyTorch 网络中。

阅读注释：阅读顺序：先看输入与输出，再看核心层/模块，最后看损失函数和训练循环。

关键点：沿图中的箭头追踪张量形状，重点定位 nn.Module、forward、损失函数和训练循环。

```
"""custom-layer: 深度学习计算"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import torch
from torch import nn
import torch.nn.functional as F

class CenteredLayer(nn.Module):
    def __init__(self):
        super().__init__()

    def forward(self, X):
        return X - X.mean()

layer = CenteredLayer()
layer(torch.FloatTensor([1, 2, 3, 4, 5]))

net = nn.Sequential(nn.Linear(8, 128), CenteredLayer())

Y = net(torch.rand(4, 8))
Y.mean()

class MyLinear(nn.Module):
    def __init__(self, in_units, units):
        super().__init__()
        self.weight = nn.Parameter(torch.randn(in_units, units))
        self.bias = nn.Parameter(torch.randn(units,))
    def forward(self, X):
        linear = torch.matmul(X, self.weight.data) + self.bias.data
        return F.relu(linear)

linear = MyLinear(5, 3)
linear.weight
```

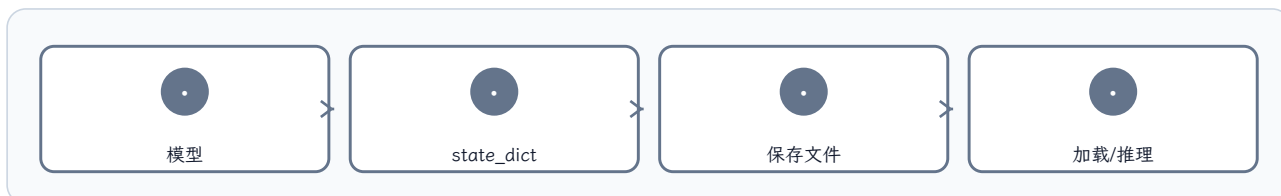
```
linear(torch.rand(2, 5))

net = nn.Sequential(MyLinear(64, 8), MyLinear(8, 1))
net(torch.rand(2, 64))
```

第 05 章 · 模型保存与加载 · chapters/chapter_05_deep_learning_computation/read_write.py

章节对应：第 05 章 类别：工程流程 代码块：8 个

结构图：模型保存与加载（工程流程）



这是模型持久化流程，不对应单独网络。

阅读注释：阅读顺序：先看数据/模型所在设备，再看执行或保存边界，最后看输出结果。

关键点：沿图中的箭头追踪张量形状，重点定位 `nn.Module`、`forward`、损失函数和训练循环。

```
"""read-write: 深度学习计算"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOTT = Path(__file__).resolve().parents[1]
if str(_ROOTT) not in sys.path:
    sys.path.insert(0, str(_ROOTT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import torch
from torch import nn
from torch.nn import functional as F

x = torch.arange(4)
torch.save(x, 'x-file')

x2 = torch.load('x-file')
x2

y = torch.zeros(4)
torch.save([x, y], 'x-files')
x2, y2 = torch.load('x-files')
(x2, y2)

mydict = {'x': x, 'y': y}
torch.save(mydict, 'mydict')
mydict2 = torch.load('mydict')
mydict2

class MLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.hidden = nn.Linear(20, 256)
        self.output = nn.Linear(256, 10)
```

```

def forward(self, x):
    return self.output(F.relu(self.hidden(x)))

net = MLP()
X = torch.randn(size=(2, 20))
Y = net(X)

torch.save(net.state_dict(), 'mlp.params')

clone = MLP()
clone.load_state_dict(torch.load('mlp.params'))
clone.eval()

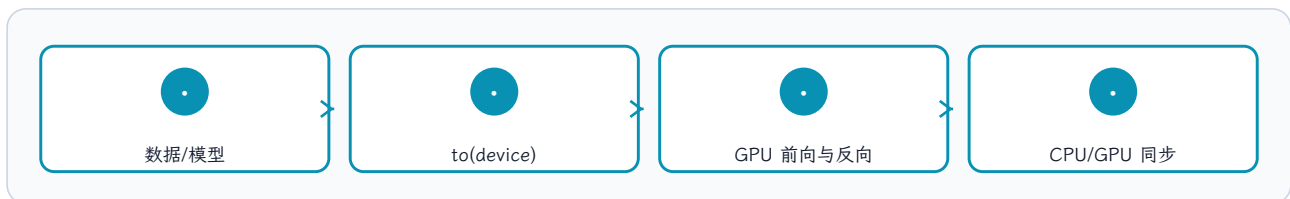
Y_clone = clone(X)
Y_clone == Y

```

第 05 章 · GPU 训练流程 · chapters/chapter_05_deep_learning_computation/use_gpu.py

章节对应：第 05 章 类别：性能流程 代码块：13 个

结构图：GPU 训练流程（性能流程）



展示设备迁移与 GPU 训练，不对应单独网络。

阅读注释：阅读顺序：先看数据/模型所在设备，再看执行或保存边界，最后看输出结果。

关键点：沿图中的箭头追踪张量形状，重点定位 `nn.Module`、`forward`、损失函数和训练循环。

```

"""use-gpu: 深度学习计算"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

# Run `nvidia-smi` in a terminal to inspect the available NVIDIA GPU.

import torch
from torch import nn

torch.device('cpu'), torch.device('cuda'), torch.device('cuda:1')

torch.cuda.device_count()

def try_gpu(i=0): #@save
    """如果存在，则返回gpu(i)，否则返回cpu()"""
    if torch.cuda.device_count() >= i + 1:
        return torch.device(f'cuda:{i}')

```

```

    return torch.device('cpu')

def try_all_gpus(): #@save
    """返回所有可用的GPU, 如果没有GPU, 则返回[cpu().]"""
    devices = [torch.device(f'cuda:{i}')
                for i in range(torch.cuda.device_count())]
    return devices if devices else [torch.device('cpu')]

try_gpu(), try_gpu(10), try_all_gpus()

x = torch.tensor([1, 2, 3])
x.device

X = torch.ones(2, 3, device=try_gpu())
X

Y = torch.rand(2, 3, device=try_gpu(1))
Y

Z = X.cuda(1)
print(X)
print(Z)

Y + Z

Z.cuda(1) is Z

net = nn.Sequential(nn.Linear(3, 1))
net = net.to(device=try_gpu())

net(X)

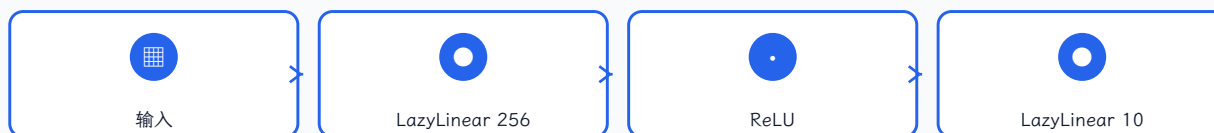
net[0].weight.data.device

```

第 05 章 • LazyLinear MLP • chapters/chapter_05_deep_learning_computation/deferred_init.py

章节对应: 第 05 章 类别: 网络结构 代码块: 1 个; 延后初始化采用 nn.LazyLinear。

结构图: LazyLinear MLP (网络结构)



延后推断输入维度的 MLP。

阅读注释: 阅读顺序: 先看输入与输出, 再看核心层/模块, 最后看损失函数和训练循环。
 关键点: 沿图中的箭头追踪张量形状, 重点定位 nn.Module、forward、损失函数和训练循环。

```

"""deferred-init: 深度学习计算"""

# 来源: 依据《动手学深度学习 (PyTorch 第二版)》的 D2L 中文 PyTorch 示例整理。
# 改写说明: 已移除 d2l 命名空间, 共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image

```

```
from matplotlib import pyplot as plt
from IPython import display
import common as native

# PyTorch's LazyLinear is the native equivalent of deferred initialization.

def get_net():
    return nn.Sequential(nn.LazyLinear(256), nn.ReLU(), nn.LazyLinear(10))

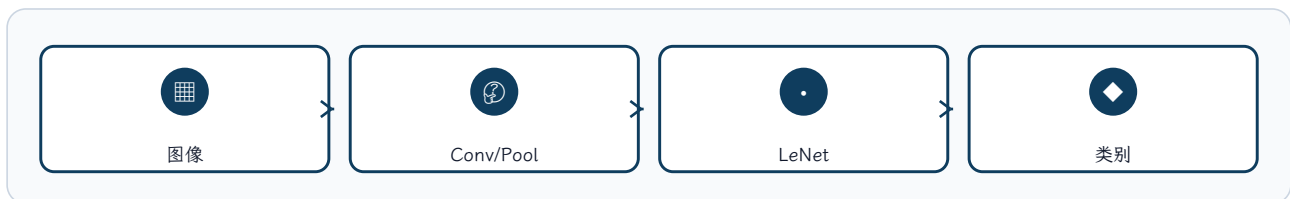
net = get_net()
print(net)

# Parameters become materialized after the first forward pass.
X = torch.rand(2, 20)
Y = net(X)
print(net)
print(Y.shape)
```

第 06 章：卷积神经网络

本章分类：卷积基本运算与 LeNet 章节说明：卷积、通道、池化等基本算子，最终组成 LeNet。

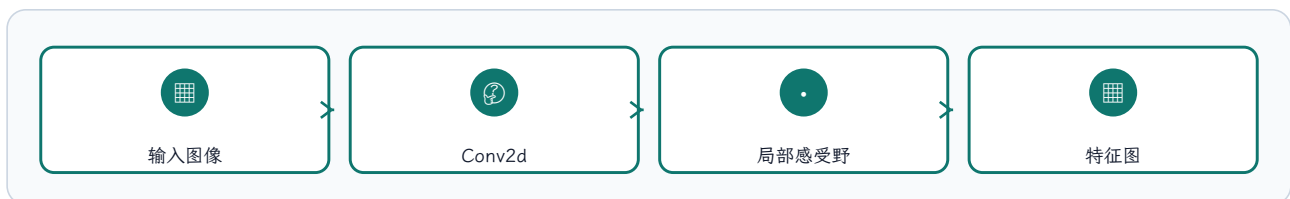
本章网络/流程总览



第 06 章 · 二维卷积层 · [chapters/chapter_06_convolutional_neural_networks/conv_layer.py](#)

章节对应：第 06 章 类别：运算原理 代码块：10 个

结构图：二维卷积层（运算原理）



卷积基本运算，不是完整分类网络。

阅读注释：阅读顺序：沿结构图从左到右追踪张量，再回到代码寻找对应的 Module 和 forward。
关键点：沿图中的箭头追踪张量形状，重点定位 nn.Module、forward、损失函数和训练循环。

"""conv-layer: 卷积神经网络"""

来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。

改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。

from pathlib import Path

import sys

_ROOT = Path(__file__).resolve().parents[1]

if str(_ROOT) not in sys.path:

sys.path.insert(0, str(_ROOT))

import math

import random

import time

import numpy as np

import pandas as pd

import torch

from torch import nn

from torch.nn import functional as F

from torch.utils import data

import torchvision

from torchvision import transforms

from PIL import Image

from matplotlib import pyplot as plt

from IPython import display

import common as native

import common as native

import torch

from torch import nn

def corr2d(X, K): #@save

"""计算二维互相关运算"""

h, w = K.shape

Y = native.zeros((X.shape[0] - h + 1, X.shape[1] - w + 1))

for i in range(Y.shape[0]):

for j in range(Y.shape[1]):

Y[i, j] = native.reduce_sum((X[i: i + h, j: j + w] * K))

return Y

X = native.tensor([[0.0, 1.0, 2.0], [3.0, 4.0, 5.0], [6.0, 7.0, 8.0]])

```

K = native.tensor([[0.0, 1.0], [2.0, 3.0]])
corr2d(X, K)

class Conv2D(nn.Module):
    def __init__(self, kernel_size):
        super().__init__()
        self.weight = nn.Parameter(torch.rand(kernel_size))
        self.bias = nn.Parameter(torch.zeros(1))

    def forward(self, x):
        return corr2d(x, self.weight) + self.bias

X = native.ones((6, 8))
X[:, 2:6] = 0
X

K = native.tensor([[1.0, -1.0]])

Y = corr2d(X, K)
Y

corr2d(native.transpose(X), K)

# 构造一个二维卷积层，它具有1个输出通道和形状为 (1, 2) 的卷积核
conv2d = nn.Conv2d(1,1, kernel_size=(1, 2), bias=False)

# 这个二维卷积层使用四维输入和输出格式 (批量大小、通道、高度、宽度)，
# 其中批量大小和通道数都为1
X = X.reshape((1, 1, 6, 8))
Y = Y.reshape((1, 1, 6, 7))
lr = 3e-2 # 学习率

for i in range(10):
    Y_hat = conv2d(X)
    l = (Y_hat - Y) ** 2
    conv2d.zero_grad()
    l.sum().backward()
    # 迭代卷积核
    conv2d.weight.data[:] -= lr * conv2d.weight.grad
    if (i + 1) % 2 == 0:
        print(f'epoch {i+1}, loss {l.sum():.3f}')

native.reshape(conv2d.weight.data, (1, 2))

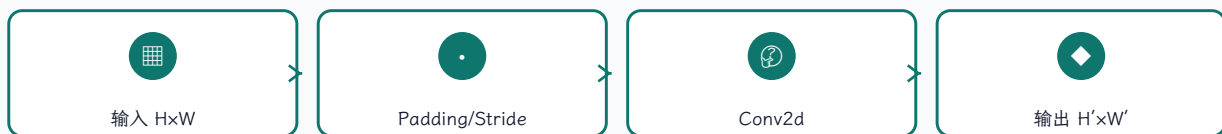
```

第 06 章 · 卷积尺寸变换 ·

[chapters/chapter_06_convolutional_neural_networks/padding_and_strides.py](#)

章节对应：第 06 章 类别：运算原理 代码块：4 个

结构图：卷积尺寸变换（运算原理）



说明填充和步幅如何改变空间尺寸。

阅读注释：阅读顺序：沿结构图从左到右追踪张量，再回到代码寻找对应的 Module 和 forward。

关键点：沿图中的箭头追踪张量形状，重点定位 nn.Module、forward、损失函数和训练循环。

```

"""padding-and-strides: 卷积神经网络"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOTT = Path(__file__).resolve().parents[1]
if str(_ROOTT) not in sys.path:
    sys.path.insert(0, str(_ROOTT))

import math

```

```

import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import torch
from torch import nn

# 为了方便起见，我们定义了一个计算卷积层的函数。
# 此函数初始化卷积层权重，并对输入和输出提高和缩减相应的维数
def comp_conv2d(conv2d, X):
    # 这里的 (1, 1) 表示批量大小和通道数都是1
    X = X.reshape((1, 1) + X.shape)
    Y = conv2d(X)
    # 省略前两个维度：批量大小和通道
    return Y.reshape(Y.shape[2:])

# 请注意，这里每边都填充了1行或1列，因此总共添加了2行或2列
conv2d = nn.Conv2d(1, 1, kernel_size=3, padding=1)
X = torch.rand(size=(8, 8))
comp_conv2d(conv2d, X).shape

conv2d = nn.Conv2d(1, 1, kernel_size=(5, 3), padding=(2, 1))
comp_conv2d(conv2d, X).shape

conv2d = nn.Conv2d(1, 1, kernel_size=3, padding=1, stride=2)
comp_conv2d(conv2d, X).shape

conv2d = nn.Conv2d(1, 1, kernel_size=(3, 5), padding=(0, 1), stride=(3, 4))
comp_conv2d(conv2d, X).shape

```

第 06 章 · 多通道卷积 · [chapters/chapter_06_convolutional_neural_networks/channels.py](#)

章节对应：第 06 章 类别：运算原理 代码块：9 个

结构图：多通道卷积（运算原理）



说明输入/输出通道与卷积核的关系。

阅读注释：阅读顺序：沿结构图从左到右追踪张量，再回到代码寻找对应的 Module 和 forward。

关键点：沿图中的箭头追踪张量形状，重点定位 nn.Module、forward、损失函数和训练循环。

```

"""channels: 卷积神经网络"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd

```

```

import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import torch

def corr2d_multi_in(X, K):
    # 先遍历“X”和“K”的第0个维度（通道维度），再把它们加在一起
    return sum(native.corr2d(x, k) for x, k in zip(X, K))

X = native.tensor([[[[0.0, 1.0, 2.0], [3.0, 4.0, 5.0], [6.0, 7.0, 8.0]],
                    [[1.0, 2.0, 3.0], [4.0, 5.0, 6.0], [7.0, 8.0, 9.0]]]])
K = native.tensor([[[[0.0, 1.0], [2.0, 3.0]], [[1.0, 2.0], [3.0, 4.0]]]])

corr2d_multi_in(X, K)

def corr2d_multi_in_out(X, K):
    # 迭代“K”的第0个维度，每次都对输入“X”执行互相关运算。
    # 最后将所有结果都叠加在一起
    return native.stack([corr2d_multi_in(X, k) for k in K], 0)

K = native.stack((K, K + 1, K + 2), 0)
K.shape

corr2d_multi_in_out(X, K)

def corr2d_multi_in_out_1x1(X, K):
    c_i, h, w = X.shape
    c_o = K.shape[0]
    X = native.reshape(X, (c_i, h * w))
    K = native.reshape(K, (c_o, c_i))
    # 全连接层中的矩阵乘法
    Y = native.matmul(K, X)
    return native.reshape(Y, (c_o, h, w))

X = native.normal(0, 1, (3, 3, 3))
K = native.normal(0, 1, (2, 3, 1, 1))

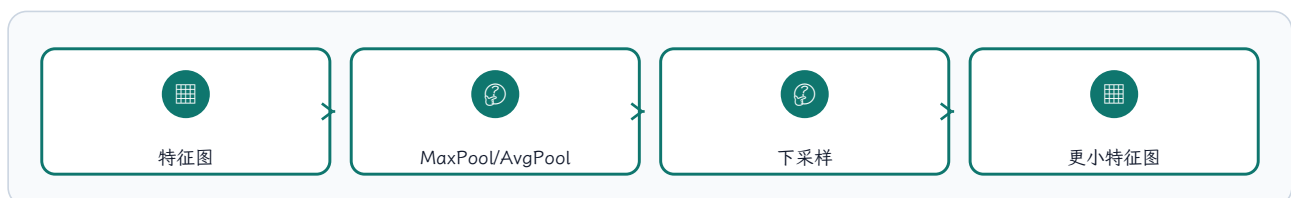
Y1 = corr2d_multi_in_out_1x1(X, K)
Y2 = corr2d_multi_in_out(X, K)
assert float(native.reduce_sum(native.abs(Y1 - Y2))) < 1e-6

```

第 06 章 · 池化层 · [chapters/chapter_06_convolutional_neural_networks/pooling.py](https://github.com/d2l-ai/d2l-pytorch/blob/master/chapters/chapter_06_convolutional_neural_networks/pooling.py)

章节对应：第 06 章 类别：运算原理 代码块：10 个

结构图：池化层（运算原理）



池化是下采样运算，不是独立分类网络。

阅读注释：阅读顺序：沿结构图从左到右追踪张量，再回到代码寻找对应的 Module 和 forward。

关键点：沿图中的箭头追踪张量形状，重点定位 nn.Module、forward、损失函数和训练循环。

```

"""pooling: 卷积神经网络"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys

```

```

_RROOT = Path(__file__).resolve().parents[1]
if str(_RROOT) not in sys.path:
    sys.path.insert(0, str(_RROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import torch
from torch import nn

def pool2d(X, pool_size, mode='max'):
    p_h, p_w = pool_size
    Y = native.zeros((X.shape[0] - p_h + 1, X.shape[1] - p_w + 1))
    for i in range(Y.shape[0]):
        for j in range(Y.shape[1]):
            if mode == 'max':
                Y[i, j] = X[i: i + p_h, j: j + p_w].max()
            elif mode == 'avg':
                Y[i, j] = X[i: i + p_h, j: j + p_w].mean()
    return Y

X = native.tensor([[0.0, 1.0, 2.0], [3.0, 4.0, 5.0], [6.0, 7.0, 8.0]])
pool2d(X, (2, 2))

pool2d(X, (2, 2), 'avg')

X = native.reshape(native.arange(16, dtype=native.float32), (1, 1, 4, 4))
X

pool2d = nn.MaxPool2d(3)
pool2d(X)

pool2d = nn.MaxPool2d(3, padding=1, stride=2)
pool2d(X)

pool2d = nn.MaxPool2d((2, 3), stride=(2, 3), padding=(0, 1))
pool2d(X)

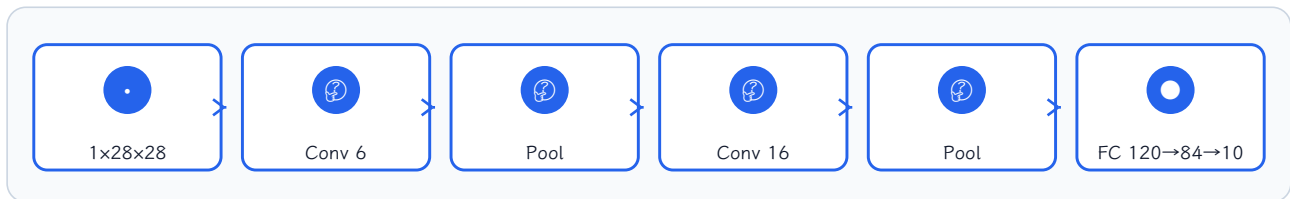
X = native.concat((X, X + 1), 1)
X

pool2d = nn.MaxPool2d(3, padding=1, stride=2)
pool2d(X)

```

章节对应：第 06 章 类别：网络结构 代码块：6 个

结构图：LeNet（网络结构）



第 6 章经典卷积神经网络。

阅读注释：阅读顺序：先看输入与输出，再看核心层/模块，最后看损失函数和训练循环。
卷积和池化逐步压缩空间尺寸、提取局部特征，最后由全连接层完成分类。

```
"""lenet: 卷积神经网络"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import torch
from torch import nn

net = nn.Sequential(
    nn.Conv2d(1, 6, kernel_size=5, padding=2), nn.Sigmoid(),
    nn.AvgPool2d(kernel_size=2, stride=2),
    nn.Conv2d(6, 16, kernel_size=5), nn.Sigmoid(),
    nn.AvgPool2d(kernel_size=2, stride=2),
    nn.Flatten(),
    nn.Linear(16 * 5 * 5, 120), nn.Sigmoid(),
    nn.Linear(120, 84), nn.Sigmoid(),
    nn.Linear(84, 10))

X = torch.rand(size=(1, 1, 28, 28), dtype=torch.float32)
for layer in net:
    X = layer(X)
    print(layer.__class__.__name__, 'output shape: \t', X.shape)

batch_size = 256
train_iter, test_iter = native.load_data_fashion_mnist(batch_size=batch_size)

def evaluate_accuracy_gpu(net, data_iter, device=None): #@save
    """使用GPU计算模型在数据集上的精度"""
    if isinstance(net, nn.Module):
        net.eval() # 设置为评估模式
        if not device:
            device = next(iter(net.parameters())).device
    # 正确预测的数量，总预测的数量
    metric = native.Accumulator(2)
    with torch.no_grad():
```

```

    for X, y in data_iter:
        if isinstance(X, list):
            # BERT微调所需的 (之后将介绍)
            X = [x.to(device) for x in X]
        else:
            X = X.to(device)
        y = y.to(device)
        metric.add(native.accuracy(net(X), y), native.size(y))
    return metric[0] / metric[1]

def train_ch6(net, train_iter, test_iter, num_epochs, lr, device):
    """用GPU训练模型(在第六章定义)"""
    def init_weights(m):
        if type(m) == nn.Linear or type(m) == nn.Conv2d:
            nn.init.xavier_uniform_(m.weight)
    net.apply(init_weights)
    print('training on', device)
    net.to(device)
    optimizer = torch.optim.SGD(net.parameters(), lr=lr)
    loss = nn.CrossEntropyLoss()
    animator = native.Animator(xlabel='epoch', xlim=[1, num_epochs],
                              legend=['train loss', 'train acc', 'test acc'])
    timer, num_batches = native.Timer(), len(train_iter)
    for epoch in range(num_epochs):
        # 训练损失之和, 训练准确率之和, 样本数
        metric = native.Accumulator(3)
        net.train()
        for i, (X, y) in enumerate(train_iter):
            timer.start()
            optimizer.zero_grad()
            X, y = X.to(device), y.to(device)
            y_hat = net(X)
            l = loss(y_hat, y)
            l.backward()
            optimizer.step()
            with torch.no_grad():
                metric.add(l * X.shape[0], native.accuracy(y_hat, y), X.shape[0])
            timer.stop()
            train_l = metric[0] / metric[2]
            train_acc = metric[1] / metric[2]
            if (i + 1) % (num_batches // 5) == 0 or i == num_batches - 1:
                animator.add(epoch + (i + 1) / num_batches,
                             (train_l, train_acc, None))
            test_acc = evaluate_accuracy_gpu(net, test_iter)
            animator.add(epoch + 1, (None, None, test_acc))
        print(f'loss {train_l:.3f}, train acc {train_acc:.3f}, '
              f'test acc {test_acc:.3f}')
        print(f'{metric[2] * num_epochs / timer.sum():.1f} examples/sec '
              f'on {str(device)}')

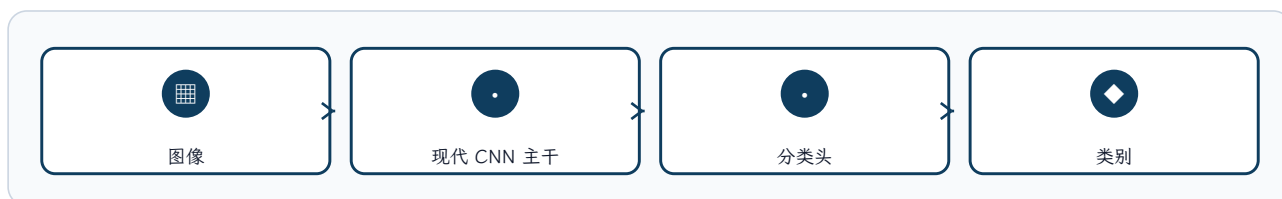
    lr, num_epochs = 0.9, 10
    train_ch6(net, train_iter, test_iter, num_epochs, lr, native.try_gpu())

```

第 07 章：现代卷积神经网络

本章分类：现代卷积网络 章节说明：AlexNet、VGG、NiN、GoogLeNet、ResNet、DenseNet 和 BatchNorm。

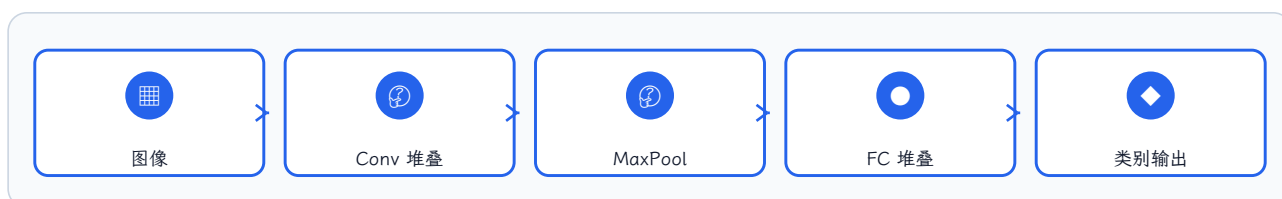
本章网络/流程总览



第 07 章 · AlexNet · [chapters/chapter_07_modern_convolutional_networks/alexnet.py](#)

章节对应：第 07 章 类别：网络结构 代码块：4 个

结构图：AlexNet（网络结构）



第 7 章深层卷积网络代表。

阅读注释：阅读顺序：先看输入与输出，再看核心层/模块，最后看损失函数和训练循环。
先看卷积主干如何提取图像特征，再看全连接分类头和 Dropout。

"""alexnet: 现代卷积神经网络"""

来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。

改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。

from pathlib import Path

import sys

_ROOT = Path(__file__).resolve().parents[1]

if str(_ROOT) not in sys.path:

sys.path.insert(0, str(_ROOT))

import math

import random

import time

import numpy as np

import pandas as pd

import torch

from torch import nn

from torch.nn import functional as F

from torch.utils import data

import torchvision

from torchvision import transforms

from PIL import Image

from matplotlib import pyplot as plt

from IPython import display

import common as native

import common as native

import torch

from torch import nn

net = nn.Sequential()

这里使用一个11*11的更大窗口来捕捉对象。

同时，步幅为4，以减少输出的高度和宽度。

另外，输出通道的数目远大于LeNet

nn.Conv2d(1, 96, kernel_size=11, stride=4, padding=1), nn.ReLU(),

nn.MaxPool2d(kernel_size=3, stride=2),

减小卷积窗口，使用填充为2来使得输入与输出的高和宽一致，且增大输出通道数

nn.Conv2d(96, 256, kernel_size=5, padding=2), nn.ReLU(),

nn.MaxPool2d(kernel_size=3, stride=2),

使用三个连续的卷积层和较小的卷积窗口。

```
# 除了最后的卷积层，输出通道的数量进一步增加。
# 在前两个卷积层之后，汇聚层不用于减少输入的高度和宽度
nn.Conv2d(256, 384, kernel_size=3, padding=1), nn.ReLU(),
nn.Conv2d(384, 384, kernel_size=3, padding=1), nn.ReLU(),
nn.Conv2d(384, 256, kernel_size=3, padding=1), nn.ReLU(),
nn.MaxPool2d(kernel_size=3, stride=2),
nn.Flatten(),
# 这里，全连接层的输出数量是LeNet中的好几倍。使用dropout层来减轻过拟合
nn.Linear(6400, 4096), nn.ReLU(),
nn.Dropout(p=0.5),
nn.Linear(4096, 4096), nn.ReLU(),
nn.Dropout(p=0.5),
# 最后是输出层。由于这里使用Fashion-MNIST，所以用类别数为10，而非论文中的1000
nn.Linear(4096, 10))
```

```
X = torch.randn(1, 1, 224, 224)
for layer in net:
    X=layer(X)
    print(layer.__class__.__name__, 'output shape:\t',X.shape)
```

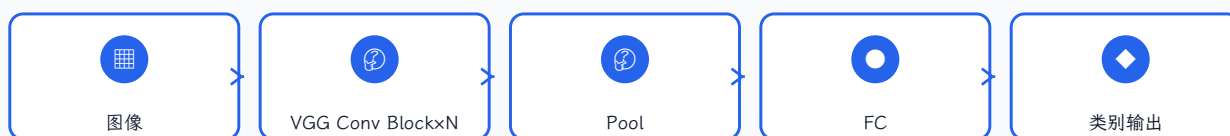
```
batch_size = 128
train_iter, test_iter = native.load_data_fashion_mnist(batch_size, resize=224)

lr, num_epochs = 0.01, 10
native.train_ch6(net, train_iter, test_iter, num_epochs, lr, native.try_gpu())
```

第 07 章 · VGG · [chapters/chapter_07_modern_convolutional_networks/vgg.py](#)

章节对应：第 07 章 类别：网络结构 代码块：6 个

结构图：VGG（网络结构）



第 7 章使用重复卷积块构建深度网络。

阅读注释：阅读顺序：先看输入与输出，再看核心层/模块，最后看损失函数和训练循环。
VGG 用重复的卷积块堆叠深度；每个池化层通常会减小空间尺寸。

```
"""vgg: 现代卷积神经网络"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import torch
from torch import nn
```

```
def vgg_block(num_convs, in_channels, out_channels):
    layers = []
    for _ in range(num_convs):
        layers.append(nn.Conv2d(in_channels, out_channels,
                                kernel_size=3, padding=1))
        layers.append(nn.ReLU())
        in_channels = out_channels
    layers.append(nn.MaxPool2d(kernel_size=2, stride=2))
    return nn.Sequential(*layers)

conv_arch = ((1, 64), (1, 128), (2, 256), (2, 512), (2, 512))

def vgg(conv_arch):
    conv_blks = []
    in_channels = 1
    # 卷积层部分
    for (num_convs, out_channels) in conv_arch:
        conv_blks.append(vgg_block(num_convs, in_channels, out_channels))
        in_channels = out_channels

    return nn.Sequential(
        *conv_blks, nn.Flatten(),
        # 全连接层部分
        nn.Linear(out_channels * 7 * 7, 4096), nn.ReLU(), nn.Dropout(0.5),
        nn.Linear(4096, 4096), nn.ReLU(), nn.Dropout(0.5),
        nn.Linear(4096, 10))

net = vgg(conv_arch)

X = torch.randn(size=(1, 1, 224, 224))

for blk in net:
    X = blk(X)
    print(blk.__class__.__name__, 'output shape:\t', X.shape)

ratio = 4
small_conv_arch = [(pair[0], pair[1] // ratio) for pair in conv_arch]
net = vgg(small_conv_arch)

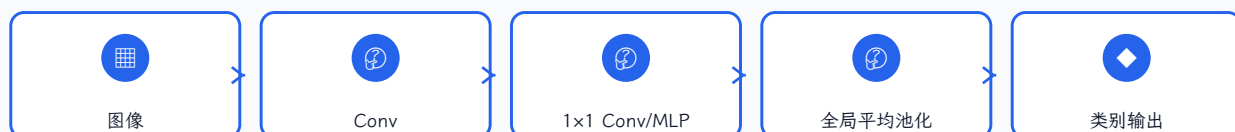
lr, num_epochs, batch_size = 0.05, 10, 128
train_iter, test_iter = native.load_data_fashion_mnist(batch_size, resize=224)
native.train_ch6(net, train_iter, test_iter, num_epochs, lr, native.try_gpu())
```

第 07 章 · NiN (Network in Network) ·

chapters/chapter_07_modern_convolutional_networks/nin.py

章节对应：第 07 章 类别：网络结构 代码块：4 个

结构图：NiN (Network in Network) (网络结构)



第 7 章用 1×1 卷积替代部分全连接表达。

阅读注释：阅读顺序：先看输入与输出，再看核心层/模块，最后看损失函数和训练循环。

1×1 卷积在通道维度上完成局部 MLP，最后用全局平均池化得到类别分数。

```
"""nin: 现代卷积神经网络"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
```

```

import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import torch
from torch import nn

def nin_block(in_channels, out_channels, kernel_size, strides, padding):
    return nn.Sequential(
        nn.Conv2d(in_channels, out_channels, kernel_size, strides, padding),
        nn.ReLU(),
        nn.Conv2d(out_channels, out_channels, kernel_size=1, nn.ReLU(),
        nn.Conv2d(out_channels, out_channels, kernel_size=1, nn.ReLU())

net = nn.Sequential(
    nin_block(1, 96, kernel_size=11, strides=4, padding=0),
    nn.MaxPool2d(3, stride=2),
    nin_block(96, 256, kernel_size=5, strides=1, padding=2),
    nn.MaxPool2d(3, stride=2),
    nin_block(256, 384, kernel_size=3, strides=1, padding=1),
    nn.MaxPool2d(3, stride=2),
    nn.Dropout(0.5),
    # 标签类别数是10
    nin_block(384, 10, kernel_size=3, strides=1, padding=1),
    nn.AdaptiveAvgPool2d((1, 1)),
    # 将四维的输出转成二维的输出，其形状为(批量大小,10)
    nn.Flatten())

X = torch.rand(size=(1, 1, 224, 224))
for layer in net:
    X = layer(X)
    print(layer.__class__.__name__, 'output shape:\t', X.shape)

lr, num_epochs, batch_size = 0.1, 10, 128
train_iter, test_iter = native.load_data_fashion_mnist(batch_size, resize=224)
native.train_ch6(net, train_iter, test_iter, num_epochs, lr, native.try_gpu())

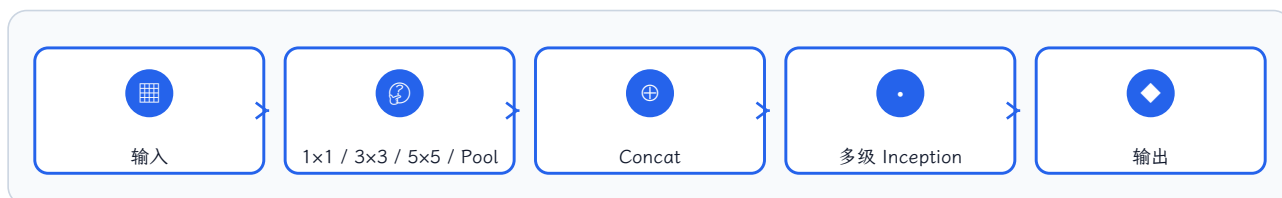
```

第 07 章 • GoogLeNet / Inception •

[chapters/chapter_07_modern_convolutional_networks/googlenet.py](#)

章节对应：第 07 章 类别：网络结构 代码块：8 个

结构图：GoogLeNet / Inception（网络结构）



第 7 章并行多尺度卷积结构。

阅读注释：阅读顺序：先看输入与输出，再看核心层/模块，最后看损失函数和训练循环。

Inception 分支并行提取不同尺度特征，Concat 会在通道维度合并分支结果。

"""googlenet: 现代卷积神经网络"""

来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。

改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。

from pathlib import Path

import sys

_ROOT = Path(__file__).resolve().parents[1]

if str(_ROOT) not in sys.path:

```

sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import torch
from torch import nn
from torch.nn import functional as F

class Inception(nn.Module):
    # c1--c4是每条路径的输出通道数
    def __init__(self, in_channels, c1, c2, c3, c4, **kwargs):
        super(Inception, self).__init__(**kwargs)
        # 线路1, 单1x1卷积层
        self.p1_1 = nn.Conv2d(in_channels, c1, kernel_size=1)
        # 线路2, 1x1卷积层后接3x3卷积层
        self.p2_1 = nn.Conv2d(in_channels, c2[0], kernel_size=1)
        self.p2_2 = nn.Conv2d(c2[0], c2[1], kernel_size=3, padding=1)
        # 线路3, 1x1卷积层后接5x5卷积层
        self.p3_1 = nn.Conv2d(in_channels, c3[0], kernel_size=1)
        self.p3_2 = nn.Conv2d(c3[0], c3[1], kernel_size=5, padding=2)
        # 线路4, 3x3最大汇聚层后接1x1卷积层
        self.p4_1 = nn.MaxPool2d(kernel_size=3, stride=1, padding=1)
        self.p4_2 = nn.Conv2d(in_channels, c4, kernel_size=1)

    def forward(self, x):
        p1 = F.relu(self.p1_1(x))
        p2 = F.relu(self.p2_2(F.relu(self.p2_1(x))))
        p3 = F.relu(self.p3_2(F.relu(self.p3_1(x))))
        p4 = F.relu(self.p4_2(self.p4_1(x)))
        # 在通道维度上连结输出
        return torch.cat((p1, p2, p3, p4), dim=1)

b1 = nn.Sequential(nn.Conv2d(1, 64, kernel_size=7, stride=2, padding=3),
                  nn.ReLU(),
                  nn.MaxPool2d(kernel_size=3, stride=2, padding=1))

b2 = nn.Sequential(nn.Conv2d(64, 64, kernel_size=1),
                  nn.ReLU(),
                  nn.Conv2d(64, 192, kernel_size=3, padding=1),
                  nn.ReLU(),
                  nn.MaxPool2d(kernel_size=3, stride=2, padding=1))

b3 = nn.Sequential(Inception(192, 64, (96, 128), (16, 32), 32),
                  Inception(256, 128, (128, 192), (32, 96), 64),
                  nn.MaxPool2d(kernel_size=3, stride=2, padding=1))

b4 = nn.Sequential(Inception(480, 192, (96, 208), (16, 48), 64),
                  Inception(512, 160, (112, 224), (24, 64), 64),
                  Inception(512, 128, (128, 256), (24, 64), 64),
                  Inception(512, 112, (144, 288), (32, 64), 64),
                  Inception(528, 256, (160, 320), (32, 128), 128),
                  nn.MaxPool2d(kernel_size=3, stride=2, padding=1))

b5 = nn.Sequential(Inception(832, 256, (160, 320), (32, 128), 128),
                  Inception(832, 384, (192, 384), (48, 128), 128),
                  nn.AdaptiveAvgPool2d((1,1)),
                  nn.Flatten())

net = nn.Sequential(b1, b2, b3, b4, b5, nn.Linear(1024, 10))

X = torch.rand(size=(1, 1, 96, 96))
for layer in net:

```

```

X = layer(X)
print(layer.__class__.__name__, 'output shape:\t', X.shape)

lr, num_epochs, batch_size = 0.1, 10, 128
train_iter, test_iter = native.load_data_fashion_mnist(batch_size, resize=96)
native.train_ch6(net, train_iter, test_iter, num_epochs, lr, native.try_gpu())

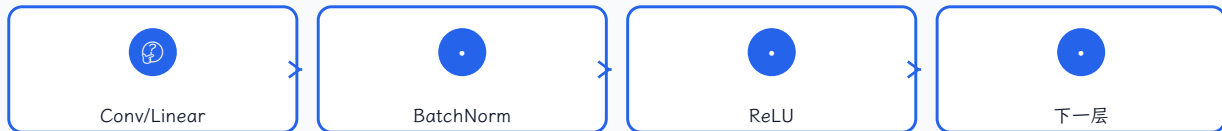
```

第 07 章 · 带 BatchNorm 的 CNN ·

chapters/chapter_07_modern_convolutional_networks/batch_norm.py

章节对应：第 07 章 类别：网络结构 代码块：7 个

结构图：带 BatchNorm 的 CNN（网络结构）



第 7 章把归一化层插入卷积或全连接块。

阅读注释：阅读顺序：先看输入与输出，再看核心层/模块，最后看损失函数和训练循环。

关键点：沿图中的箭头追踪张量形状，重点定位 `nn.Module`、`forward`、损失函数和训练循环。

```

"""batch-norm: 现代卷积神经网络"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import torch
from torch import nn

def batch_norm(X, gamma, beta, moving_mean, moving_var, eps, momentum):
    # 通过is_grad_enabled来判断当前模式是训练模式还是预测模式
    if not torch.is_grad_enabled():
        # 如果是在预测模式下，直接使用传入的移动平均所得的均值和方差
        X_hat = (X - moving_mean) / torch.sqrt(moving_var + eps)
    else:
        assert len(X.shape) in (2, 4)
        if len(X.shape) == 2:
            # 使用全连接层的情况，计算特征维上的均值和方差
            mean = X.mean(dim=0)
            var = ((X - mean) ** 2).mean(dim=0)
        else:
            # 使用二维卷积层的情况，计算通道维上（axis=1）的均值和方差。
            # 这里我们需要保持X的形状以便后面可以做广播运算
            mean = X.mean(dim=(0, 2, 3), keepdim=True)
            var = ((X - mean) ** 2).mean(dim=(0, 2, 3), keepdim=True)
        # 训练模式下，用当前的均值和方差做标准化
        X_hat = (X - mean) / torch.sqrt(var + eps)

```

```

    # 更新移动平均的均值和方差
    moving_mean = momentum * moving_mean + (1.0 - momentum) * mean
    moving_var = momentum * moving_var + (1.0 - momentum) * var
    Y = gamma * X_hat + beta # 缩放和移位
    return Y, moving_mean.data, moving_var.data

class BatchNorm(nn.Module):
    # num_features: 完全连接层的输出数量或卷积层的输出通道数。
    # num_dims: 2表示完全连接层, 4表示卷积层
    def __init__(self, num_features, num_dims):
        super().__init__()
        if num_dims == 2:
            shape = (1, num_features)
        else:
            shape = (1, num_features, 1, 1)
        # 参与求梯度和迭代的拉伸和偏移参数, 分别初始化成1和0
        self.gamma = nn.Parameter(torch.ones(shape))
        self.beta = nn.Parameter(torch.zeros(shape))
        # 非模型参数的变量初始化为0和1
        self.moving_mean = torch.zeros(shape)
        self.moving_var = torch.ones(shape)

    def forward(self, X):
        # 如果X不在内存上, 将moving_mean和moving_var
        # 复制到X所在显存上
        if self.moving_mean.device != X.device:
            self.moving_mean = self.moving_mean.to(X.device)
            self.moving_var = self.moving_var.to(X.device)
        # 保存更新过的moving_mean和moving_var
        Y, self.moving_mean, self.moving_var = batch_norm(
            X, self.gamma, self.beta, self.moving_mean,
            self.moving_var, eps=1e-5, momentum=0.9)
        return Y

net = nn.Sequential(
    nn.Conv2d(1, 6, kernel_size=5), BatchNorm(6, num_dims=4), nn.Sigmoid(),
    nn.AvgPool2d(kernel_size=2, stride=2),
    nn.Conv2d(6, 16, kernel_size=5), BatchNorm(16, num_dims=4), nn.Sigmoid(),
    nn.AvgPool2d(kernel_size=2, stride=2), nn.Flatten(),
    nn.Linear(16*4*4, 120), BatchNorm(120, num_dims=2), nn.Sigmoid(),
    nn.Linear(120, 84), BatchNorm(84, num_dims=2), nn.Sigmoid(),
    nn.Linear(84, 10))

lr, num_epochs, batch_size = 1.0, 10, 256
train_iter, test_iter = native.load_data_fashion_mnist(batch_size)
native.train_ch6(net, train_iter, test_iter, num_epochs, lr, native.try_gpu())

net[1].gamma.reshape((-1,)), net[1].beta.reshape((-1,))

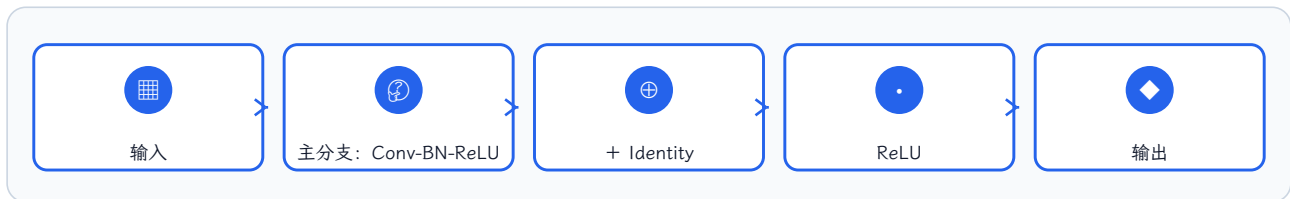
net = nn.Sequential(
    nn.Conv2d(1, 6, kernel_size=5), nn.BatchNorm2d(6), nn.Sigmoid(),
    nn.AvgPool2d(kernel_size=2, stride=2),
    nn.Conv2d(6, 16, kernel_size=5), nn.BatchNorm2d(16), nn.Sigmoid(),
    nn.AvgPool2d(kernel_size=2, stride=2), nn.Flatten(),
    nn.Linear(256, 120), nn.BatchNorm1d(120), nn.Sigmoid(),
    nn.Linear(120, 84), nn.BatchNorm1d(84), nn.Sigmoid(),
    nn.Linear(84, 10))

native.train_ch6(net, train_iter, test_iter, num_epochs, lr, native.try_gpu())

```

章节对应：第 07 章 类别：网络结构 代码块：9 个

结构图：ResNet（网络结构）



第 7 章残差连接网络。

阅读注释：阅读顺序：先看输入与输出，再看核心层/模块，最后看损失函数和训练循环。
残差分支与主分支相加，二者形状必须一致；形状变化时需要投影捷径。

"""resnet: 现代卷积神经网络"""

来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。

改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。

```

from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import torch
from torch import nn
from torch.nn import functional as F

class Residual(nn.Module):  #@save
    def __init__(self, input_channels, num_channels,
                 use_1x1conv=False, strides=1):
        super().__init__()
        self.conv1 = nn.Conv2d(input_channels, num_channels,
                                kernel_size=3, padding=1, stride=strides)
        self.conv2 = nn.Conv2d(num_channels, num_channels,
                                kernel_size=3, padding=1)
        if use_1x1conv:
            self.conv3 = nn.Conv2d(input_channels, num_channels,
                                    kernel_size=1, stride=strides)
        else:
            self.conv3 = None
        self.bn1 = nn.BatchNorm2d(num_channels)
        self.bn2 = nn.BatchNorm2d(num_channels)

    def forward(self, X):
        Y = F.relu(self.bn1(self.conv1(X)))
        Y = self.bn2(self.conv2(Y))
        if self.conv3:
            X = self.conv3(X)
        Y += X
        return F.relu(Y)

blk = Residual(3,3)
X = torch.rand(4, 3, 6, 6)

```

```

Y = blk(X)
Y.shape

blk = Residual(3,6, use_1x1conv=True, strides=2)
blk(X).shape

b1 = nn.Sequential(nn.Conv2d(1, 64, kernel_size=7, stride=2, padding=3),
                    nn.BatchNorm2d(64), nn.ReLU(),
                    nn.MaxPool2d(kernel_size=3, stride=2, padding=1))

def resnet_block(input_channels, num_channels, num_residuals,
                  first_block=False):
    blk = []
    for i in range(num_residuals):
        if i == 0 and not first_block:
            blk.append(Residual(input_channels, num_channels,
                                use_1x1conv=True, strides=2))
        else:
            blk.append(Residual(num_channels, num_channels))
    return blk

b2 = nn.Sequential(*resnet_block(64, 64, 2, first_block=True))
b3 = nn.Sequential(*resnet_block(64, 128, 2))
b4 = nn.Sequential(*resnet_block(128, 256, 2))
b5 = nn.Sequential(*resnet_block(256, 512, 2))

net = nn.Sequential(b1, b2, b3, b4, b5,
                    nn.AdaptiveAvgPool2d((1,1)),
                    nn.Flatten(), nn.Linear(512, 10))

X = torch.rand(size=(1, 1, 224, 224))
for layer in net:
    X = layer(X)
    print(layer.__class__.__name__, 'output shape:\t', X.shape)

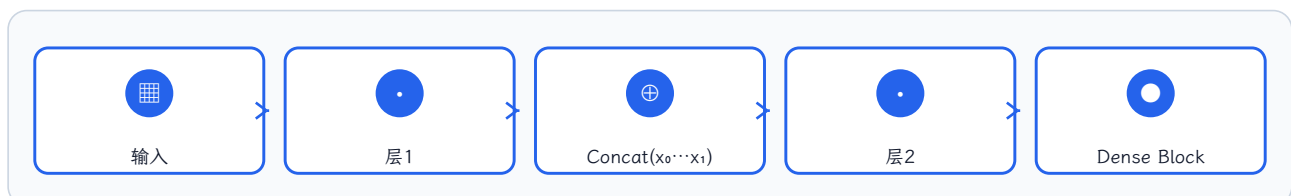
lr, num_epochs, batch_size = 0.05, 10, 256
train_iter, test_iter = native.load_data_fashion_mnist(batch_size, resize=96)
native.train_ch6(net, train_iter, test_iter, num_epochs, lr, native.try_gpu())

```

第 07 章 · DenseNet · chapters/chapter_07_modern_convolutional_networks/densenet.py

章节对应：第 07 章 类别：网络结构 代码块：9 个

结构图：DenseNet（网络结构）



第 7 章密集连接网络。

阅读注释：阅读顺序：先看输入与输出，再看核心层/模块，最后看损失函数和训练循环。
每一层都接收之前层的特征，Concat 会不断增加通道数，注意显存和维度变化。

```

"""densenet: 现代卷积神经网络"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn

```

```

from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import torch
from torch import nn

def conv_block(input_channels, num_channels):
    return nn.Sequential(
        nn.BatchNorm2d(input_channels), nn.ReLU(),
        nn.Conv2d(input_channels, num_channels, kernel_size=3, padding=1))

class DenseBlock(nn.Module):
    def __init__(self, num_convs, input_channels, num_channels):
        super(DenseBlock, self).__init__()
        layer = []
        for i in range(num_convs):
            layer.append(conv_block(
                num_channels * i + input_channels, num_channels))
        self.net = nn.Sequential(*layer)

    def forward(self, X):
        for blk in self.net:
            Y = blk(X)
        # 连接通道维度上每个块的输入和输出
        X = torch.cat((X, Y), dim=1)
        return X

blk = DenseBlock(2, 3, 10)
X = torch.randn(4, 3, 8, 8)
Y = blk(X)
Y.shape

def transition_block(input_channels, num_channels):
    return nn.Sequential(
        nn.BatchNorm2d(input_channels), nn.ReLU(),
        nn.Conv2d(input_channels, num_channels, kernel_size=1),
        nn.AvgPool2d(kernel_size=2, stride=2))

blk = transition_block(23, 10)
blk(Y).shape

b1 = nn.Sequential(
    nn.Conv2d(1, 64, kernel_size=7, stride=2, padding=3),
    nn.BatchNorm2d(64), nn.ReLU(),
    nn.MaxPool2d(kernel_size=3, stride=2, padding=1))

# num_channels为当前的通道数
num_channels, growth_rate = 64, 32
num_convs_in_dense_blocks = [4, 4, 4, 4]
blks = []
for i, num_convs in enumerate(num_convs_in_dense_blocks):
    blks.append(DenseBlock(num_convs, num_channels, growth_rate))
    # 上一个稠密块的输出通道数
    num_channels += num_convs * growth_rate
    # 在稠密块之间添加一个转换层, 使通道数量减半
    if i != len(num_convs_in_dense_blocks) - 1:
        blks.append(transition_block(num_channels, num_channels // 2))
        num_channels = num_channels // 2

net = nn.Sequential(
    b1, *blks,
    nn.BatchNorm2d(num_channels), nn.ReLU(),
    nn.AdaptiveAvgPool2d((1, 1)),
    nn.Flatten(),
    nn.Linear(num_channels, 10))

lr, num_epochs, batch_size = 0.1, 10, 256
train_iter, test_iter = native.load_data_fashion_mnist(batch_size, resize=96)
native.train_ch6(net, train_iter, test_iter, num_epochs, lr, native.try_gpu())

```

第 08 章：循环神经网络

本章分类：循环神经网络 章节说明：序列数据预处理与基础 RNN 语言模型。

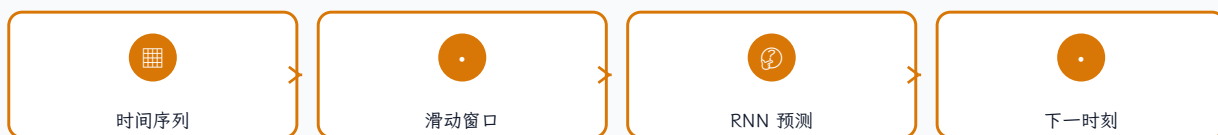
本章网络/流程总览



第 08 章 · 序列预测流程 · [chapters/chapter_08_recurrent_neural_networks/sequence.py](#)

章节对应：第 08 章 类别：数据/训练流程 代码块：12 个

结构图：序列预测流程（数据/训练流程）



这是序列预测任务流程。

阅读注释：阅读顺序：先看样本格式，再看预处理与 Dataset，最后看 DataLoader 生成的批次。
关键点：沿图中的箭头追踪张量形状，重点定位 nn.Module、forward、损失函数和训练循环。

"""sequence: 循环神经网络"""

```
# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import torch
from torch import nn

T = 1000 # 总共产生1000个点
time = native.arange(1, T + 1, dtype=native.float32)
x = native.sin(0.01 * time) + native.normal(0, 0.2, (T,))
native.plot(time, [x], 'time', 'x', xlim=[1, 1000], figsize=(6, 3))

tau = 4
features = native.zeros((T - tau, tau))
for i in range(tau):
    features[:, i] = x[i: T - tau + i]
labels = native.reshape(x[tau:], (-1, 1))
```

```

batch_size, n_train = 16, 600
# 只有前n_train个样本用于训练
train_iter = native.load_array((features[:n_train], labels[:n_train]),
                               batch_size, is_train=True)

# 初始化网络权重的函数
def init_weights(m):
    if type(m) == nn.Linear:
        nn.init.xavier_uniform_(m.weight)

# 一个简单的多层感知机
def get_net():
    net = nn.Sequential(nn.Linear(4, 10),
                        nn.ReLU(),
                        nn.Linear(10, 1))
    net.apply(init_weights)
    return net

# 平方损失。注意: MSELoss计算平方误差时不带系数1/2
loss = nn.MSELoss(reduction='none')

def train(net, train_iter, loss, epochs, lr):
    trainer = torch.optim.Adam(net.parameters(), lr)
    for epoch in range(epochs):
        for X, y in train_iter:
            trainer.zero_grad()
            l = loss(net(X), y)
            l.sum().backward()
            trainer.step()
        print(f'epoch {epoch + 1}, '
              f'loss: {native.evaluate_loss(net, train_iter, loss):f}')

net = get_net()
train(net, train_iter, loss, 5, 0.01)

onestep_preds = net(features)
native.plot([time, time[tau:]],
            [native.numpy(x), native.numpy(onestep_preds)], 'time',
            'x', legend=['data', '1-step preds'], xlim=[1, 1000],
            figsize=(6, 3))

multistep_preds = native.zeros(T)
multistep_preds[: n_train + tau] = x[: n_train + tau]
for i in range(n_train + tau, T):
    multistep_preds[i] = net(
        native.reshape(multistep_preds[i - tau: i], (1, -1)))

native.plot([time, time[tau:], time[n_train + tau:]],
            [native.numpy(x), native.numpy(onestep_preds),
             native.numpy(multistep_preds[n_train + tau:])], 'time',
            'x', legend=['data', '1-step preds', 'multistep preds'],
            xlim=[1, 1000], figsize=(6, 3))

max_steps = 64

features = native.zeros((T - tau - max_steps + 1, tau + max_steps))
# 列i (i<tau) 是来自x的观测, 其时间步从 (i) 到 (i+T-tau-max_steps+1)
for i in range(tau):
    features[:, i] = x[i: i + T - tau - max_steps + 1]

# 列i (i>=tau) 是来自 (i-tau+1) 步的预测, 其时间步从 (i) 到 (i+T-tau-max_steps+1)
for i in range(tau, tau + max_steps):
    features[:, i] = native.reshape(net(features[:, i - tau: i]), -1)

steps = (1, 4, 16, 64)
native.plot([time[tau + i - 1: T - max_steps + i] for i in steps],
            [native.numpy(features[:, tau + i - 1]) for i in steps], 'time', 'x',
            legend=[f'{i}-step preds' for i in steps], xlim=[5, 1000],
            figsize=(6, 3))

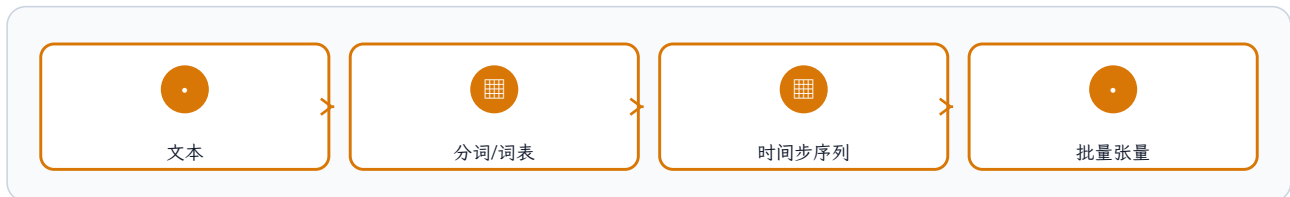
```

第 08 章 · 语言模型数据管线 ·

chapters/chapter_08_recurrent_neural_networks/text_preprocessing.py

章节对应：第 08 章 类别：数据流程 代码块：7 个

结构图：语言模型数据管线（数据流程）



为循环网络准备文本序列，不是单独网络。

阅读注释：阅读顺序：先看样本格式，再看预处理与 Dataset，最后看 DataLoader 生成的批次。

关键点：沿图中的箭头追踪张量形状，重点定位 nn.Module、forward、损失函数和训练循环。

```
"""text-preprocessing: 循环神经网络"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import collections
import common as native
import re

native.DATA_HUB['time_machine'] = (native.DATA_URL + 'timemachine.txt',
                                   '090b5e7e70c295757f55df93cb0a180b9691891a')

def read_time_machine(): #@save
    """将时间机器数据集加载到文本行的列表中"""
    with open(native.download('time_machine'), 'r') as f:
        lines = f.readlines()
    return [re.sub('[^A-Za-z]+', ' ', line).strip().lower() for line in lines]

lines = read_time_machine()
print(f'# 文本总行数: {len(lines)}')
print(lines[0])
print(lines[10])

def tokenize(lines, token='word'): #@save
    """将文本行拆分为单词或字符词元"""
    if token == 'word':
        return [line.split() for line in lines]
    elif token == 'char':
        return [list(line) for line in lines]
    else:
        print('错误：未知词元类型: ' + token)

tokens = tokenize(lines)
for i in range(11):
```

```

print(tokens[i])

class Vocab:  #@save
    """文本词表"""
    def __init__(self, tokens=None, min_freq=0, reserved_tokens=None):
        if tokens is None:
            tokens = []
        if reserved_tokens is None:
            reserved_tokens = []
        # 按出现频率排序
        counter = count_corpus(tokens)
        self._token_freqs = sorted(counter.items(), key=lambda x: x[1],
                                   reverse=True)
        # 未知词元的索引为0
        self.idx_to_token = ['<unk>'] + reserved_tokens
        self.token_to_idx = {token: idx
                              for idx, token in enumerate(self.idx_to_token)}
        for token, freq in self._token_freqs:
            if freq < min_freq:
                break
            if token not in self.token_to_idx:
                self.idx_to_token.append(token)
                self.token_to_idx[token] = len(self.idx_to_token) - 1

    def __len__(self):
        return len(self.idx_to_token)

    def __getitem__(self, tokens):
        if not isinstance(tokens, (list, tuple)):
            return self.token_to_idx.get(tokens, self.unk)
        return [self.__getitem__(token) for token in tokens]

    def to_tokens(self, indices):
        if not isinstance(indices, (list, tuple)):
            return self.idx_to_token[indices]
        return [self.idx_to_token[index] for index in indices]

    @property
    def unk(self): # 未知词元的索引为0
        return 0

    @property
    def token_freqs(self):
        return self._token_freqs

def count_corpus(tokens):  #@save
    """统计词元的频率"""
    # 这里的tokens是1D列表或2D列表
    if len(tokens) == 0 or isinstance(tokens[0], list):
        # 将词元列表展平成一个列表
        tokens = [token for line in tokens for token in line]
    return collections.Counter(tokens)

vocab = Vocab(tokens)
print(list(vocab.token_to_idx.items())[:10])

for i in [0, 10]:
    print('文本:', tokens[i])
    print('索引:', vocab[tokens[i]])

def load_corpus_time_machine(max_tokens=-1):  #@save
    """返回时光机器数据集的词元索引列表和词表"""
    lines = read_time_machine()
    tokens = tokenize(lines, 'char')
    vocab = Vocab(tokens)

    # 因为时光机器数据集中的每个文本行不一定是句子或段落,
    # 所以将所有文本行展平到一个列表中
    corpus = [vocab[token] for line in tokens for token in line]
    if max_tokens > 0:
        corpus = corpus[:max_tokens]
    return corpus, vocab

corpus, vocab = load_corpus_time_machine()
len(corpus), len(vocab)

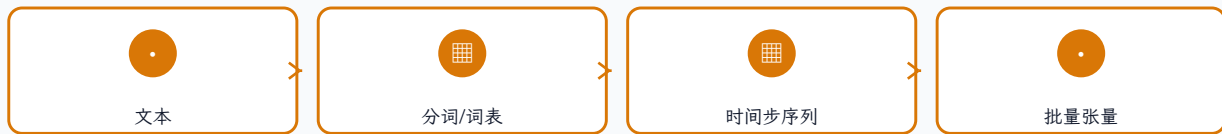
```

第 08 章 · 语言模型数据管线 ·

chapters/chapter_08_recurrent_neural_networks/language_models_and_dataset.py

章节对应：第 08 章 类别：数据流程 代码块：12 个

结构图：语言模型数据管线（数据流程）



为循环网络准备文本序列，不是单独网络。

阅读注释：阅读顺序：先看样本格式，再看预处理与 Dataset，最后看 DataLoader 生成的批次。

关键点：沿图中的箭头追踪张量形状，重点定位 nn.Module、forward、损失函数和训练循环。

"""language-models-and-dataset: 循环神经网络"""

来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。

改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。

from pathlib import Path

import sys

_ROOT = Path(__file__).resolve().parents[1]

if str(_ROOT) not in sys.path:

sys.path.insert(0, str(_ROOT))

import math

import random

import time

import numpy as np

import pandas as pd

import torch

from torch import nn

from torch.nn import functional as F

from torch.utils import data

import torchvision

from torchvision import transforms

from PIL import Image

from matplotlib import pyplot as plt

from IPython import display

import common as native

import common as native

import torch

import random

tokens = native.tokenize(native.read_time_machine())

因为每个文本行不一定是一个句子或一个段落，因此我们把所有文本行拼接到一起

corpus = [token for line in tokens for token in line]

vocab = native.Vocab(corpus)

vocab.token_freqs[:10]

freqs = [freq for token, freq in vocab.token_freqs]

native.plot(freqs, xlabel='token: x', ylabel='frequency: n(x)',
xscale='log', yscale='log')

bigram_tokens = [pair for pair in zip(corpus[:-1], corpus[1:])]

bigram_vocab = native.Vocab(bigram_tokens)

bigram_vocab.token_freqs[:10]

trigram_tokens = [triple for triple in zip(
corpus[:-2], corpus[1:-1], corpus[2:])]

trigram_vocab = native.Vocab(trigram_tokens)

trigram_vocab.token_freqs[:10]

bigram_freqs = [freq for token, freq in bigram_vocab.token_freqs]

trigram_freqs = [freq for token, freq in trigram_vocab.token_freqs]

native.plot([freqs, bigram_freqs, trigram_freqs], xlabel='token: x',
ylabel='frequency: n(x)', xscale='log', yscale='log',
legend=['unigram', 'bigram', 'trigram'])

```

def seq_data_iter_random(corpus, batch_size, num_steps): #@save
    """使用随机抽样生成一个小批量子序列"""
    # 从随机偏移量开始对序列进行分区，随机范围包括num_steps-1
    corpus = corpus[random.randint(0, num_steps - 1):]
    # 减去1，是因为我们需要考虑标签

    num_subseqs = (len(corpus) - 1) // num_steps
    # 长度为num_steps的子序列的起始索引
    initial_indices = list(range(0, num_subseqs * num_steps, num_steps))
    # 在随机抽样的迭代过程中，
    # 来自两个相邻的、随机的、小批量中的子序列不一定在原始序列上相邻
    random.shuffle(initial_indices)

    def data(pos):
        # 返回从pos位置开始的长度为num_steps的序列
        return corpus[pos: pos + num_steps]

    num_batches = num_subseqs // batch_size
    for i in range(0, batch_size * num_batches, batch_size):
        # 在这里，initial_indices包含子序列的随机起始索引
        initial_indices_per_batch = initial_indices[i: i + batch_size]
        X = [data(j) for j in initial_indices_per_batch]
        Y = [data(j + 1) for j in initial_indices_per_batch]
        yield native.tensor(X), native.tensor(Y)

my_seq = list(range(35))
for X, Y in seq_data_iter_random(my_seq, batch_size=2, num_steps=5):
    print('X: ', X, '\nY:', Y)

def seq_data_iter_sequential(corpus, batch_size, num_steps): #@save
    """使用顺序分区生成一个小批量子序列"""
    # 从随机偏移量开始划分序列
    offset = random.randint(0, num_steps)
    num_tokens = ((len(corpus) - offset - 1) // batch_size) * batch_size
    Xs = native.tensor(corpus[offset: offset + num_tokens])
    Ys = native.tensor(corpus[offset + 1: offset + 1 + num_tokens])
    Xs, Ys = Xs.reshape(batch_size, -1), Ys.reshape(batch_size, -1)
    num_batches = Xs.shape[1] // num_steps
    for i in range(0, num_steps * num_batches, num_steps):
        X = Xs[:, i: i + num_steps]
        Y = Ys[:, i: i + num_steps]
        yield X, Y

for X, Y in seq_data_iter_sequential(my_seq, batch_size=2, num_steps=5):
    print('X: ', X, '\nY:', Y)

class SeqDataLoader: #@save
    """加载序列数据的迭代器"""
    def __init__(self, batch_size, num_steps, use_random_iter, max_tokens):
        if use_random_iter:
            self.data_iter_fn = native.seq_data_iter_random
        else:
            self.data_iter_fn = native.seq_data_iter_sequential
        self.corpus, self.vocab = native.load_corpus_time_machine(max_tokens)
        self.batch_size, self.num_steps = batch_size, num_steps

    def __iter__(self):
        return self.data_iter_fn(self.corpus, self.batch_size, self.num_steps)

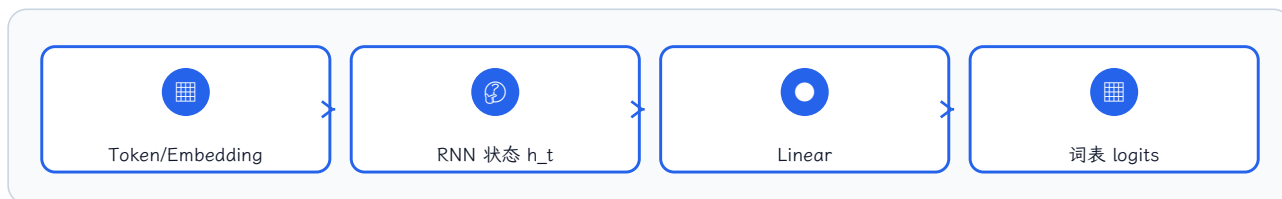
def load_data_time_machine(batch_size, num_steps, #@save
                           use_random_iter=False, max_tokens=10000):
    """返回时光机器数据集的迭代器和词表"""
    data_iter = SeqDataLoader(
        batch_size, num_steps, use_random_iter, max_tokens)
    return data_iter, data_iter.vocab

```

第 08 章 · RNN 语言模型 · chapters/chapter_08_recurrent_neural_networks/rnn.py

章节对应：第 08 章 类别：网络结构 代码块：3 个

结构图：RNN 语言模型（网络结构）



第 8 章基础循环神经网络语言模型。

阅读注释：阅读顺序：先看输入与输出，再看核心层/模块，最后看损失函数和训练循环。
隐藏状态 h_t 携带历史信息；重点检查时间步、批量和词表维度的排列。

```
"""rnn: 循环神经网络"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import import display
import common as native

import common as native
import torch

X, W_xh = native.normal(0, 1, (3, 1)), native.normal(0, 1, (1, 4))
H, W_hh = native.normal(0, 1, (3, 4)), native.normal(0, 1, (4, 4))
native.matmul(X, W_xh) + native.matmul(H, W_hh)

native.matmul(native.concat((X, H), 1), native.concat((W_xh, W_hh), 0))
```

第 08 章 · RNN 语言模型 · chapters/chapter_08_recurrent_neural_networks/rnn_scratch.py

章节对应：第 08 章 类别：网络结构 代码块：16 个

结构图：RNN 语言模型（网络结构）



第 8 章基础循环神经网络语言模型。

阅读注释：阅读顺序：先看输入与输出，再看核心层/模块，最后看损失函数和训练循环。
隐藏状态 h_t 携带历史信息；重点检查时间步、批量和词表维度的排列。

```
"""rnn-scratch: 循环神经网络"""
```

```

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_RROOT = Path(__file__).resolve().parents[1]
if str(_RROOT) not in sys.path:
    sys.path.insert(0, str(_RROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import math
import torch
from torch import nn
from torch.nn import functional as F

batch_size, num_steps = 32, 35
train_iter, vocab = native.load_data_time_machine(batch_size, num_steps)

F.one_hot(torch.tensor([0, 2]), len(vocab))

X = native.reshape(native.arange(10), (2, 5))
F.one_hot(X.T, 28).shape

def get_params(vocab_size, num_hiddens, device):
    num_inputs = num_outputs = vocab_size

    def normal(shape):
        return torch.randn(size=shape, device=device) * 0.01

    # 隐藏层参数
    W_xh = normal((num_inputs, num_hiddens))
    W_hh = normal((num_hiddens, num_hiddens))
    b_h = native.zeros(num_hiddens, device=device)
    # 输出层参数
    W_hq = normal((num_hiddens, num_outputs))
    b_q = native.zeros(num_outputs, device=device)
    # 附加梯度
    params = [W_xh, W_hh, b_h, W_hq, b_q]
    for param in params:
        param.requires_grad_(True)
    return params

def init_rnn_state(batch_size, num_hiddens, device):
    return (native.zeros((batch_size, num_hiddens), device=device), )

def rnn(inputs, state, params):
    # inputs的形状：(时间步数量, 批量大小, 词表大小)
    W_xh, W_hh, b_h, W_hq, b_q = params
    H, = state
    outputs = []
    # X的形状：(批量大小, 词表大小)
    for X in inputs:
        H = torch.tanh(torch.mm(X, W_xh) + torch.mm(H, W_hh) + b_h)
        Y = torch.mm(H, W_hq) + b_q
        outputs.append(Y)
    return torch.cat(outputs, dim=0), (H, )

class RNNModelScratch: #@save
    """从零开始实现的循环神经网络模型"""
    def __init__(self, vocab_size, num_hiddens, device,
                  get_params, init_state, forward_fn):

```

```

        self.vocab_size, self.num_hiddens = vocab_size, num_hiddens
        self.params = get_params(vocab_size, num_hiddens, device)
        self.init_state, self.forward_fn = init_state, forward_fn

    def __call__(self, X, state):
        X = F.one_hot(X.T, self.vocab_size).type(torch.float32)
        return self.forward_fn(X, state, self.params)

    def begin_state(self, batch_size, device):
        return self.init_state(batch_size, self.num_hiddens, device)

num_hiddens = 512
net = RNNModelScratch(len(vocab), num_hiddens, native.try_gpu(), get_params,
                      init_rnn_state, rnn)
state = net.begin_state(X.shape[0], native.try_gpu())
Y, new_state = net(X.to(native.try_gpu()), state)
Y.shape, len(new_state), new_state[0].shape

def predict_ch8(prefix, num_preds, net, vocab, device): #@save
    """在prefix后面生成新字符"""
    state = net.begin_state(batch_size=1, device=device)
    outputs = [vocab[prefix[0]]]
    get_input = lambda: native.reshape(native.tensor(
        [outputs[-1]], device=device), (1, 1))
    for y in prefix[1:]: # 预热期
        _, state = net(get_input(), state)
        outputs.append(vocab[y])
    for _ in range(num_preds): # 预测num_preds步
        y, state = net(get_input(), state)
        outputs.append(int(y.argmax(dim=1).reshape(1)))
    return ''.join([vocab.idx_to_token[i] for i in outputs])

predict_ch8('time traveller ', 10, net, vocab, native.try_gpu())

def grad_clipping(net, theta): #@save
    """裁剪梯度"""
    if isinstance(net, nn.Module):
        params = [p for p in net.parameters() if p.requires_grad]
    else:
        params = net.params
    norm = torch.sqrt(sum(torch.sum((p.grad ** 2)) for p in params))
    if norm > theta:
        for param in params:
            param.grad[:] *= theta / norm

def train_epoch_ch8(net, train_iter, loss, updater, device, use_random_iter):
    """训练网络一个迭代周期（定义见第8章）"""
    state, timer = None, native.Timer()
    metric = native.Accumulator(2) # 训练损失之和,词元数量
    for X, Y in train_iter:
        if state is None or use_random_iter:
            # 在第一次迭代或使用随机抽样时初始化state
            state = net.begin_state(batch_size=X.shape[0], device=device)
        else:
            if isinstance(net, nn.Module) and not isinstance(state, tuple):
                # state对于nn.GRU是个张量
                state.detach_()
            else:
                # state对于nn.LSTM或对于我们从零开始实现的模型是个张量
                for s in state:
                    s.detach_()
        y = Y.T.reshape(-1)
        X, y = X.to(device), y.to(device)
        y_hat, state = net(X, state)
        l = loss(y_hat, y.long()).mean()
        if isinstance(updater, torch.optim.Optimizer):
            updater.zero_grad()
            l.backward()
            grad_clipping(net, 1)
            updater.step()
        else:
            l.backward()
            grad_clipping(net, 1)
        # 因为已经调用了mean函数
        updater(batch_size=1)
        metric.add(l * native.size(y), native.size(y))
    return math.exp(metric[0] / metric[1]), metric[1] / timer.stop()

```

```
def train_ch8(net, train_iter, vocab, lr, num_epochs, device,
             use_random_iter=False):
    """训练模型（定义见第8章）"""
    loss = nn.CrossEntropyLoss()
    animator = native.Animator(xlabel='epoch', ylabel='perplexity',
                               legend=['train'], xlim=[10, num_epochs])

    # 初始化
    if isinstance(net, nn.Module):
        updater = torch.optim.SGD(net.parameters(), lr)
    else:
        updater = lambda batch_size: native.sgd(net.params, lr, batch_size)
    predict = lambda prefix: predict_ch8(prefix, 50, net, vocab, device)

    # 训练和预测
    for epoch in range(num_epochs):
        ppl, speed = train_epoch_ch8(
            net, train_iter, loss, updater, device, use_random_iter)
        if (epoch + 1) % 10 == 0:
            print(predict('time traveller'))
            animator.add(epoch + 1, [ppl])
    print(f'困惑度 {ppl:.1f}, {speed:.1f} 词元/秒 {str(device)}')
    print(predict('time traveller'))
    print(predict('traveller'))

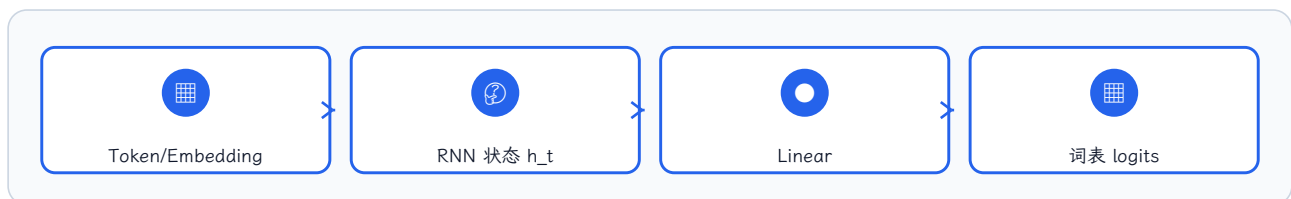
num_epochs, lr = 500, 1
train_ch8(net, train_iter, vocab, lr, num_epochs, native.try_gpu())

net = RNNModelScratch(len(vocab), num_hiddens, native.try_gpu(), get_params,
                      init_rnn_state, rnn)
train_ch8(net, train_iter, vocab, lr, num_epochs, native.try_gpu(),
          use_random_iter=True)
```

第 08 章 · RNN 语言模型 · chapters/chapter_08_recurrent_neural_networks/rnn_concise.py

章节对应：第 08 章 类别：网络结构 代码块：7 个

结构图：RNN 语言模型（网络结构）



第 8 章基础循环神经网络语言模型。

阅读注释：阅读顺序：先看输入与输出，再看核心层/模块，最后看损失函数和训练循环。

隐藏状态 h_t 携带历史信息；重点检查时间步、批量和词表维度的排列。

```
"""rnn-concise: 循环神经网络"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
```

```

import common as native

import common as native
import torch
from torch import nn
from torch.nn import functional as F

batch_size, num_steps = 32, 35
train_iter, vocab = native.load_data_time_machine(batch_size, num_steps)

num_hiddens = 256
rnn_layer = nn.RNN(len(vocab), num_hiddens)

state = torch.zeros((1, batch_size, num_hiddens))
state.shape

X = torch.rand(size=(num_steps, batch_size, len(vocab)))
Y, state_new = rnn_layer(X, state)
Y.shape, state_new.shape

class RNNModel(nn.Module):
    """循环神经网络模型"""
    def __init__(self, rnn_layer, vocab_size, **kwargs):
        super(RNNModel, self).__init__(**kwargs)
        self.rnn = rnn_layer
        self.vocab_size = vocab_size
        self.num_hiddens = self.rnn.hidden_size
        # 如果RNN是双向的（之后将介绍），num_directions应该是2，否则应该是1
        if not self.rnn.bidirectional:
            self.num_directions = 1
            self.linear = nn.Linear(self.num_hiddens, self.vocab_size)
        else:
            self.num_directions = 2
            self.linear = nn.Linear(self.num_hiddens * 2, self.vocab_size)

    def forward(self, inputs, state):
        X = F.one_hot(inputs.T.long(), self.vocab_size)
        X = X.to(torch.float32)
        Y, state = self.rnn(X, state)
        # 全连接层首先将Y的形状改为(时间步数*批量大小,隐藏单元数)
        # 它的输出形状是(时间步数*批量大小,词表大小)。
        output = self.linear(Y.reshape((-1, Y.shape[-1])))
        return output, state

    def begin_state(self, device, batch_size=1):
        if not isinstance(self.rnn, nn.LSTM):
            # nn.GRU以张量作为隐状态
            return torch.zeros((self.num_directions * self.rnn.num_layers,
                                batch_size, self.num_hiddens),
                                device=device)
        else:
            # nn.LSTM以元组作为隐状态
            return (torch.zeros((
                self.num_directions * self.rnn.num_layers,
                batch_size, self.num_hiddens), device=device),
                    torch.zeros((
                        self.num_directions * self.rnn.num_layers,
                        batch_size, self.num_hiddens), device=device))

device = native.try_gpu()
net = RNNModel(rnn_layer, vocab_size=len(vocab))
net = net.to(device)
native.predict_ch8('time traveller', 10, net, vocab, device)

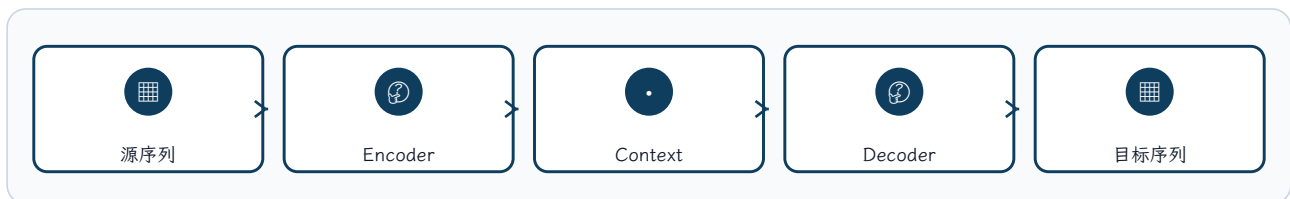
num_epochs, lr = 500, 1
native.train_ch8(net, train_iter, vocab, lr, num_epochs, device)

```

第 09 章：现代循环神经网络

本章分类：现代循环网络与 Seq2Seq 章节说明：GRU、LSTM、深层/双向 RNN 以及编码器-解码器。

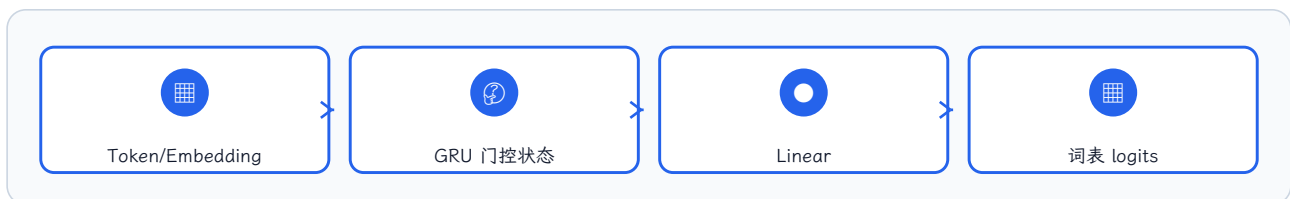
本章网络/流程总览



第 09 章 · GRU 语言模型 · [chapters/chapter_09_modern_recurrent_neural_networks/gru.py](#)

章节对应：第 09 章 类别：网络结构 代码块：6 个

结构图：GRU 语言模型（网络结构）



第 9 章门控循环单元。

阅读注释：阅读顺序：先看输入与输出，再看核心层/模块，最后看损失函数和训练循环。
更新门和重置门控制历史信息保留比例，阅读时关注门控状态如何进入下一时间步。

```
"""gru: 现代循环神经网络"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import torch
from torch import nn

batch_size, num_steps = 32, 35
train_iter, vocab = native.load_data_time_machine(batch_size, num_steps)

def get_params(vocab_size, num_hiddens, device):
    num_inputs = num_outputs = vocab_size

    def normal(shape):
        return torch.randn(size=shape, device=device)*0.01

    def three():
```

```

        return (normal((num_inputs, num_hiddens)),
                normal((num_hiddens, num_hiddens)),
                native.zeros(num_hiddens, device=device))

W_xz, W_hz, b_z = three() # 更新门参数
W_xr, W_hr, b_r = three() # 重置门参数
W_xh, W_hh, b_h = three() # 候选隐状态参数
# 输出层参数
W_hq = normal((num_hiddens, num_outputs))
b_q = native.zeros(num_outputs, device=device)
# 附加梯度
params = [W_xz, W_hz, b_z, W_xr, W_hr, b_r, W_xh, W_hh, b_h, W_hq, b_q]
for param in params:
    param.requires_grad_(True)
return params

def init_gru_state(batch_size, num_hiddens, device):
    return (torch.zeros((batch_size, num_hiddens), device=device), )

def gru(inputs, state, params):
    W_xz, W_hz, b_z, W_xr, W_hr, b_r, W_xh, W_hh, b_h, W_hq, b_q = params
    H, = state
    outputs = []
    for X in inputs:
        Z = torch.sigmoid((X @ W_xz) + (H @ W_hz) + b_z)
        R = torch.sigmoid((X @ W_xr) + (H @ W_hr) + b_r)
        H_tilda = torch.tanh((X @ W_xh) + ((R * H) @ W_hh) + b_h)
        H = Z * H + (1 - Z) * H_tilda
        Y = H @ W_hq + b_q
        outputs.append(Y)
    return torch.cat(outputs, dim=0), (H,)

vocab_size, num_hiddens, device = len(vocab), 256, native.try_gpu()
num_epochs, lr = 500, 1
model = native.RNNModelScratch(len(vocab), num_hiddens, device, get_params,
                               init_gru_state, gru)
native.train_ch8(model, train_iter, vocab, lr, num_epochs, device)

num_inputs = vocab_size
gru_layer = nn.GRU(num_inputs, num_hiddens)
model = native.RNNModel(gru_layer, len(vocab))
model = model.to(device)
native.train_ch8(model, train_iter, vocab, lr, num_epochs, device)

```

第 09 章 · LSTM 语言模型 · chapters/chapter_09_modern_recurrent_neural_networks/lstm.py

章节对应: 第 09 章 类别: 网络结构 代码块: 6 个

结构图: LSTM 语言模型 (网络结构)



第 9 章长短期记忆网络。

阅读注释: 阅读顺序: 先看输入与输出, 再看核心层/模块, 最后看损失函数和训练循环。

LSTM 同时维护隐藏状态 h 和记忆单元 c , 两个状态共同传递长期信息。

```

"""lstm: 现代循环神经网络"""

# 来源: 依据《动手学深度学习 (PyTorch 第二版)》的 D2L 中文 PyTorch 示例整理。
# 改写说明: 已移除 d2l 命名空间, 共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random

```

```

import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import torch
from torch import nn

batch_size, num_steps = 32, 35
train_iter, vocab = native.load_data_time_machine(batch_size, num_steps)

def get_lstm_params(vocab_size, num_hiddens, device):
    num_inputs = num_outputs = vocab_size

    def normal(shape):
        return torch.randn(size=shape, device=device)*0.01

    def three():
        return (normal((num_inputs, num_hiddens)),
                normal((num_hiddens, num_hiddens)),
                native.zeros(num_hiddens, device=device))

    W_xi, W_hi, b_i = three() # 输入门参数
    W_xf, W_hf, b_f = three() # 遗忘门参数
    W_xo, W_ho, b_o = three() # 输出门参数
    W_xc, W_hc, b_c = three() # 候选记忆元参数
    # 输出层参数
    W_hq = normal((num_hiddens, num_outputs))
    b_q = native.zeros(num_outputs, device=device)
    # 附加梯度
    params = [W_xi, W_hi, b_i, W_xf, W_hf, b_f, W_xo, W_ho, b_o, W_xc, W_hc,
              b_c, W_hq, b_q]
    for param in params:
        param.requires_grad_(True)
    return params

def init_lstm_state(batch_size, num_hiddens, device):
    return (torch.zeros((batch_size, num_hiddens), device=device),
            torch.zeros((batch_size, num_hiddens), device=device))

def lstm(inputs, state, params):
    [W_xi, W_hi, b_i, W_xf, W_hf, b_f, W_xo, W_ho, b_o, W_xc, W_hc, b_c,
     W_hq, b_q] = params
    (H, C) = state
    outputs = []
    for X in inputs:
        I = torch.sigmoid((X @ W_xi) + (H @ W_hi) + b_i)
        F = torch.sigmoid((X @ W_xf) + (H @ W_hf) + b_f)
        O = torch.sigmoid((X @ W_xo) + (H @ W_ho) + b_o)
        C_tilda = torch.tanh((X @ W_xc) + (H @ W_hc) + b_c)
        C = F * C + I * C_tilda
        H = O * torch.tanh(C)
        Y = (H @ W_hq) + b_q
        outputs.append(Y)
    return torch.cat(outputs, dim=0), (H, C)

vocab_size, num_hiddens, device = len(vocab), 256, native.try_gpu()
num_epochs, lr = 500, 1
model = native.RNNModelScratch(len(vocab), num_hiddens, device, get_lstm_params,
                               init_lstm_state, lstm)
native.train_ch8(model, train_iter, vocab, lr, num_epochs, device)

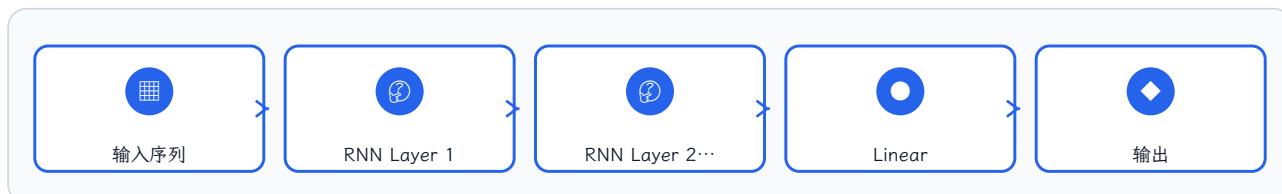
num_inputs = vocab_size
lstm_layer = nn.LSTM(num_inputs, num_hiddens)
model = native.RNNModel(lstm_layer, len(vocab))
model = model.to(device)
native.train_ch8(model, train_iter, vocab, lr, num_epochs, device)

```

第 09 章 · 深层 RNN · chapters/chapter_09_modern_recurrent_neural_networks/deep_rnn.py

章节对应：第 09 章 类别：网络结构 代码块：3 个

结构图：深层 RNN（网络结构）



第 9 章堆叠多层循环网络。

阅读注释：阅读顺序：先看输入与输出，再看核心层/模块，最后看损失函数和训练循环。

关键点：沿图中的箭头追踪张量形状，重点定位 nn.Module、forward、损失函数和训练循环。

```
"""deep-rnn: 现代循环神经网络"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import torch
from torch import nn

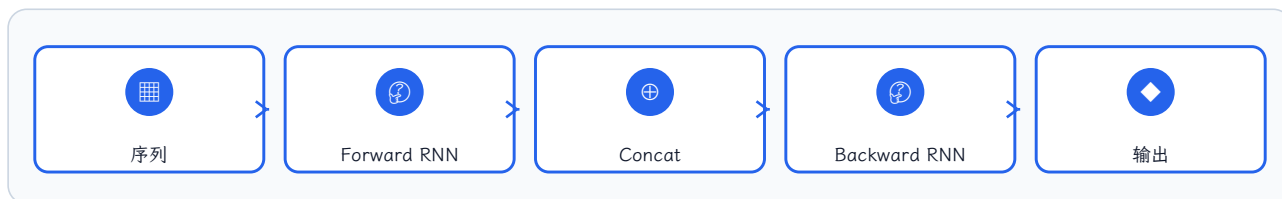
batch_size, num_steps = 32, 35
train_iter, vocab = native.load_data_time_machine(batch_size, num_steps)

vocab_size, num_hiddens, num_layers = len(vocab), 256, 2
num_inputs = vocab_size
device = native.try_gpu()
lstm_layer = nn.LSTM(num_inputs, num_hiddens, num_layers)
model = native.RNNModel(lstm_layer, len(vocab))
model = model.to(device)

num_epochs, lr = 500, 2
native.train_ch8(model, train_iter, vocab, lr*1.0, num_epochs, device)
```

章节对应：第 09 章 类别：网络结构 代码块：1 个

结构图：双向 RNN（网络结构）



第 9 章双向循环网络。

阅读注释：阅读顺序：先看输入与输出，再看核心层/模块，最后看损失函数和训练循环。
正向和反向序列分别编码，再拼接两端信息；注意输出维度会随方向数增加。

"""bi-rnn: 现代循环神经网络"""

```
# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import torch
from torch import nn

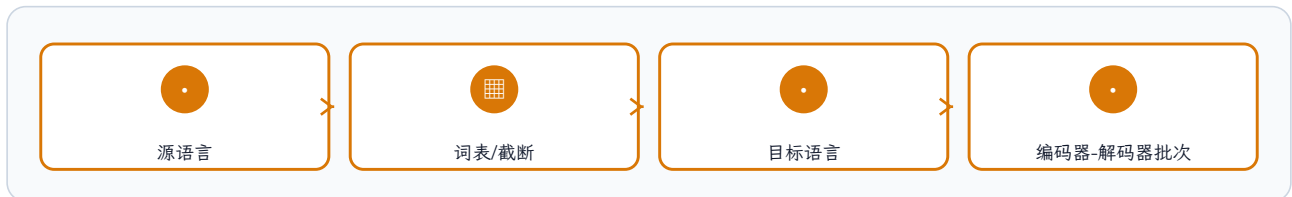
# 加载数据
batch_size, num_steps, device = 32, 35, native.try_gpu()
train_iter, vocab = native.load_data_time_machine(batch_size, num_steps)
# 通过设置“bidirective=True”来定义双向LSTM模型
vocab_size, num_hiddens, num_layers = len(vocab), 256, 2
num_inputs = vocab_size
lstm_layer = nn.LSTM(num_inputs, num_hiddens, num_layers, bidirectional=True)
model = native.RNNModel(lstm_layer, len(vocab))
model = model.to(device)
# 训练模型
num_epochs, lr = 500, 1
native.train_ch8(model, train_iter, vocab, lr, num_epochs, device)
```

第 09 章 · 机器翻译数据管线 ·

chapters/chapter_09_modern_recurrent_neural_networks/machine_translation_and_dataset.py

章节对应：第 09 章 类别：数据流程 代码块：10 个

结构图：机器翻译数据管线（数据流程）



为序列到序列模型准备平行语料。

阅读注释：阅读顺序：先看样本格式，再看预处理与 Dataset，最后看 DataLoader 生成的批次。

关键点：沿图中的箭头追踪张量形状，重点定位 nn.Module、forward、损失函数和训练循环。

```
"""machine-translation-and-dataset: 现代循环神经网络"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import torch
import os

native.DATA_HUB['fra-eng'] = (native.DATA_URL + 'fra-eng.zip',
                              '94646ad1522d915e7b0f9296181140edcf86a4f5')

def read_data_nmt():
    """载入“英语—法语”数据集"""
    data_dir = native.download_extract('fra-eng')
    with open(os.path.join(data_dir, 'fra.txt'), 'r',
              encoding='utf-8') as f:
        return f.read()

raw_text = read_data_nmt()
print(raw_text[:75])

def preprocess_nmt(text):
    """预处理“英语—法语”数据集"""
    def no_space(char, prev_char):
        return char in set(',.!?') and prev_char != ' '

    # 使用空格替换不间断空格
    # 使用小写字母替换大写字母
    text = text.replace('\u202f', ' ').replace('\xa0', ' ').lower()
    # 在单词和标点符号之间插入空格
    out = [' ' + char if i > 0 and no_space(char, text[i - 1]) else char
            for i, char in enumerate(text)]
    return ''.join(out)
```

```

text = preprocess_nmt(raw_text)
print(text[:80])

def tokenize_nmt(text, num_examples=None):
    """词元化“英语-法语”数据集"""
    source, target = [], []
    for i, line in enumerate(text.split('\n')):
        if num_examples and i > num_examples:
            break
        parts = line.split('\t')
        if len(parts) == 2:
            source.append(parts[0].split(' '))
            target.append(parts[1].split(' '))
    return source, target

source, target = tokenize_nmt(text)
source[:6], target[:6]

def show_list_len_pair_hist(legend, xlabel, ylabel, xlist, ylist):
    """绘制列表长度对的直方图"""
    native.set_figsize()
    _, _, patches = native.plt.hist(
        [[len(l) for l in xlist], [len(l) for l in ylist]])
    native.plt.xlabel(xlabel)
    native.plt.ylabel(ylabel)
    for patch in patches[1].patches:
        patch.set_hatch('/')
    native.plt.legend(legend)

show_list_len_pair_hist(['source', 'target'], '# tokens per sequence',
                        'count', source, target);

src_vocab = native.Vocab(source, min_freq=2,
                        reserved_tokens=['<pad>', '<bos>', '<eos>'])
len(src_vocab)

def truncate_pad(line, num_steps, padding_token):
    """截断或填充文本序列"""
    if len(line) > num_steps:
        return line[:num_steps] # 截断
    return line + [padding_token] * (num_steps - len(line)) # 填充

truncate_pad(src_vocab[source[0]], 10, src_vocab['<pad>'])

def build_array_nmt(lines, vocab, num_steps):
    """将机器翻译的文本序列转换成小批量"""
    lines = [vocab[l] for l in lines]
    lines = [l + [vocab['<eos>']] for l in lines]
    array = native.tensor([truncate_pad(
        l, num_steps, vocab['<pad>']) for l in lines])
    valid_len = native.reduce_sum(
        native.astype(array != vocab['<pad>'], native.int32), 1)
    return array, valid_len

def load_data_nmt(batch_size, num_steps, num_examples=600):
    """返回翻译数据集的迭代器和词表"""
    text = preprocess_nmt(read_data_nmt())
    source, target = tokenize_nmt(text, num_examples)
    src_vocab = native.Vocab(source, min_freq=2,
                            reserved_tokens=['<pad>', '<bos>', '<eos>'])
    tgt_vocab = native.Vocab(target, min_freq=2,
                            reserved_tokens=['<pad>', '<bos>', '<eos>'])
    src_array, src_valid_len = build_array_nmt(source, src_vocab, num_steps)
    tgt_array, tgt_valid_len = build_array_nmt(target, tgt_vocab, num_steps)

    data_arrays = (src_array, src_valid_len, tgt_array, tgt_valid_len)
    data_iter = native.load_array(data_arrays, batch_size)
    return data_iter, src_vocab, tgt_vocab

train_iter, src_vocab, tgt_vocab = load_data_nmt(batch_size=2, num_steps=8)
for X, X_valid_len, Y, Y_valid_len in train_iter:
    print('X:', native.astype(X, native.int32))
    print('X的有效长度:', X_valid_len)
    print('Y:', native.astype(Y, native.int32))
    print('Y的有效长度:', Y_valid_len)
    break

```

第 09 章 • Encoder-Decoder •

chapters/chapter_09_modern_recurrent_neural_networks/encoder_decoder.py

章节对应：第 09 章 类别：网络结构 代码块：3 个

结构图：Encoder-Decoder（网络结构）



第 9 章序列到序列框架。

阅读注释：阅读顺序：先看输入与输出，再看核心层/模块，最后看损失函数和训练循环。

关键点：沿图中的箭头追踪张量形状，重点定位 nn.Module、forward、损失函数和训练循环。

"""encoder-decoder: 现代循环神经网络"""

来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。

改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。

from pathlib import Path

import sys

_ROOT = Path(__file__).resolve().parents[1]

if str(_ROOT) not in sys.path:

sys.path.insert(0, str(_ROOT))

import math

import random

import time

import numpy as np

import pandas as pd

import torch

from torch import nn

from torch.nn import functional as F

from torch.utils import data

import torchvision

from torchvision import transforms

from PIL import Image

from matplotlib import pyplot as plt

from IPython import display

import common as native

from torch import nn

class Encoder(nn.Module):

"""编码器-解码器架构的基本编码器接口"""

def __init__(self, **kwargs):

super(Encoder, self).__init__(**kwargs)

def forward(self, X, *args):

raise NotImplementedError

class Decoder(nn.Module):

"""编码器-解码器架构的基本解码器接口"""

def __init__(self, **kwargs):

super(Decoder, self).__init__(**kwargs)

def init_state(self, enc_outputs, *args):

raise NotImplementedError

def forward(self, X, state):

raise NotImplementedError

class EncoderDecoder(nn.Module):

"""编码器-解码器架构的基类"""

def __init__(self, encoder, decoder, **kwargs):

super(EncoderDecoder, self).__init__(**kwargs)

self.encoder = encoder

self.decoder = decoder

def forward(self, enc_X, dec_X, *args):

```

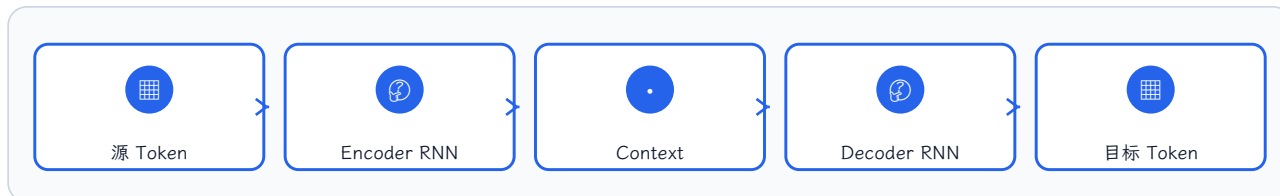
enc_outputs = self.encoder(enc_X, *args)
dec_state = self.decoder.init_state(enc_outputs, *args)
return self.decoder(dec_X, dec_state)

```

第 09 章 · Seq2Seq · chapters/chapter_09_modern_recurrent_neural_networks/seq2seq.py

章节对应：第 09 章 类别：网络结构 代码块：15 个

结构图：Seq2Seq（网络结构）



第 9 章机器翻译基础网络。

阅读注释：阅读顺序：先看输入与输出，再看核心层/模块，最后看损失函数和训练循环。

Encoder 压缩源序列信息，Decoder 自回归地产生目标序列；训练和推理的输入方式可能不同。

"""seq2seq: 现代循环神经网络"""

来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。

改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。

from pathlib import Path

import sys

_ROOT = Path(__file__).resolve().parents[1]

if str(_ROOT) not in sys.path:

sys.path.insert(0, str(_ROOT))

import math

import random

import time

import numpy as np

import pandas as pd

import torch

from torch import nn

from torch.nn import functional as F

from torch.utils import data

import torchvision

from torchvision import transforms

from PIL import Image

from matplotlib import pyplot as plt

from IPython import display

import common as native

import collections

import common as native

import math

import torch

from torch import nn

class Seq2SeqEncoder(native.Encoder):

"""用于序列到序列学习的循环神经网络编码器"""

def __init__(self, vocab_size, embed_size, num_hiddens, num_layers,
dropout=0, **kwargs):

super(Seq2SeqEncoder, self).__init__(**kwargs)

嵌入层

self.embedding = nn.Embedding(vocab_size, embed_size)

self.rnn = nn.GRU(embed_size, num_hiddens, num_layers,
dropout=dropout)

def forward(self, X, *args):

输出'X'的形状: (batch_size,num_steps,embed_size)

X = self.embedding(X)

在循环神经网络模型中，第一个轴对应于时间步

X = X.permute(1, 0, 2)

如果未提及状态，则默认为0

output, state = self.rnn(X)

output的形状:(num_steps,batch_size,num_hiddens)

state的形状:(num_layers,batch_size,num_hiddens)

return output, state

```

encoder = Seq2SeqEncoder(vocab_size=10, embed_size=8, num_hiddens=16,
                        num_layers=2)
encoder.eval()
X = native.zeros((4, 7), dtype=torch.long)
output, state = encoder(X)
output.shape

state.shape

class Seq2SeqDecoder(native.Decoder):
    """用于序列到序列学习的循环神经网络解码器"""
    def __init__(self, vocab_size, embed_size, num_hiddens, num_layers,
                dropout=0, **kwargs):
        super(Seq2SeqDecoder, self).__init__(**kwargs)
        self.embedding = nn.Embedding(vocab_size, embed_size)
        self.rnn = nn.GRU(embed_size + num_hiddens, num_hiddens, num_layers,
                        dropout=dropout)
        self.dense = nn.Linear(num_hiddens, vocab_size)

    def init_state(self, enc_outputs, *args):
        return enc_outputs[1]

    def forward(self, X, state):
        # 输出'X'的形状: (batch_size,num_steps,embed_size)
        X = self.embedding(X).permute(1, 0, 2)
        # 广播context, 使其具有与X相同的num_steps
        context = state[-1].repeat(X.shape[0], 1, 1)
        X_and_context = native.concat((X, context), 2)
        output, state = self.rnn(X_and_context, state)
        output = self.dense(output).permute(1, 0, 2)
        # output的形状:(batch_size,num_steps,vocab_size)
        # state的形状:(num_layers,batch_size,num_hiddens)
        return output, state

decoder = Seq2SeqDecoder(vocab_size=10, embed_size=8, num_hiddens=16,
                        num_layers=2)
decoder.eval()
state = decoder.init_state(encoder(X))
output, state = decoder(X, state)
output.shape, state.shape

def sequence_mask(X, valid_len, value=0):
    """在序列中屏蔽不相关的项"""
    maxlen = X.size(1)
    mask = torch.arange((maxlen), dtype=torch.float32,
                        device=X.device)[None, :] < valid_len[:, None]
    X[~mask] = value
    return X

X = torch.tensor([[1, 2, 3], [4, 5, 6]])
sequence_mask(X, torch.tensor([1, 2]))

X = native.ones(2, 3, 4)
sequence_mask(X, torch.tensor([1, 2]), value=-1)

class MaskedSoftmaxCELoss(nn.CrossEntropyLoss):
    """带遮蔽的softmax交叉熵损失函数"""
    # pred的形状: (batch_size,num_steps,vocab_size)
    # label的形状: (batch_size,num_steps)
    # valid_len的形状: (batch_size,)
    def forward(self, pred, label, valid_len):
        weights = torch.ones_like(label)
        weights = sequence_mask(weights, valid_len)
        self.reduction='none'
        unweighted_loss = super(MaskedSoftmaxCELoss, self).forward(
            pred.permute(0, 2, 1), label)
        weighted_loss = (unweighted_loss * weights).mean(dim=1)
        return weighted_loss

loss = MaskedSoftmaxCELoss()
loss(native.ones(3, 4, 10), native.ones((3, 4), dtype=torch.long),
    torch.tensor([4, 2, 0]))

def train_seq2seq(net, data_iter, lr, num_epochs, tgt_vocab, device):
    """训练序列到序列模型"""

```

```

def xavier_init_weights(m):
    if type(m) == nn.Linear:
        nn.init.xavier_uniform_(m.weight)
    if type(m) == nn.GRU:
        for param in m._flat_weights_names:
            if "weight" in param:
                nn.init.xavier_uniform_(m._parameters[param])

net.apply(xavier_init_weights)
net.to(device)
optimizer = torch.optim.Adam(net.parameters(), lr=lr)
loss = MaskedSoftmaxCELoss()
net.train()
animator = native.Animator(xlabel='epoch', ylabel='loss',
                           xlim=[10, num_epochs])
for epoch in range(num_epochs):
    timer = native.Timer()
    metric = native.Accumulator(2) # 训练损失总和, 词元数量
    for batch in data_iter:
        optimizer.zero_grad()
        X, X_valid_len, Y, Y_valid_len = [x.to(device) for x in batch]
        bos = torch.tensor([tgt_vocab['<bos>']] * Y.shape[0],
                           device=device).reshape(-1, 1)
        dec_input = torch.cat([bos, Y[:, :-1]], 1) # 强制教学
        Y_hat, _ = net(X, dec_input, X_valid_len)
        l = loss(Y_hat, Y, Y_valid_len)
        l.sum().backward() # 损失函数的标量进行“反向传播”
        native.grad_clipping(net, 1)
        num_tokens = Y_valid_len.sum()
        optimizer.step()
        with torch.no_grad():
            metric.add(l.sum(), num_tokens)
    if (epoch + 1) % 10 == 0:
        animator.add(epoch + 1, (metric[0] / metric[1],))
    print(f'loss {metric[0] / metric[1]:.3f}, {metric[1] / timer.stop():.1f} '
          f'tokens/sec on {str(device)}')

```

```

embed_size, num_hiddens, num_layers, dropout = 32, 32, 2, 0.1
batch_size, num_steps = 64, 10
lr, num_epochs, device = 0.005, 300, native.try_gpu()

```

```

train_iter, src_vocab, tgt_vocab = native.load_data_nmt(batch_size, num_steps)
encoder = Seq2SeqEncoder(len(src_vocab), embed_size, num_hiddens, num_layers,
                        dropout)
decoder = Seq2SeqDecoder(len(tgt_vocab), embed_size, num_hiddens, num_layers,
                        dropout)
net = native.EncoderDecoder(encoder, decoder)
train_seq2seq(net, train_iter, lr, num_epochs, tgt_vocab, device)

```

```

def predict_seq2seq(net, src_sentence, src_vocab, tgt_vocab, num_steps,
                    device, save_attention_weights=False):

```

"""序列到序列模型的预测"""

```

# 在预测时将net设置为评估模式
net.eval()
src_tokens = src_vocab[src_sentence.lower().split(' ')] + [
    src_vocab['<eos>']]
enc_valid_len = torch.tensor([len(src_tokens)], device=device)
src_tokens = native.truncate_pad(src_tokens, num_steps, src_vocab['<pad>'])
# 添加批量轴
enc_X = torch.unsqueeze(
    torch.tensor(src_tokens, dtype=torch.long, device=device), dim=0)
enc_outputs = net.encoder(enc_X, enc_valid_len)
dec_state = net.decoder.init_state(enc_outputs, enc_valid_len)
# 添加批量轴
dec_X = torch.unsqueeze(torch.tensor(
    [tgt_vocab['<bos>']], dtype=torch.long, device=device), dim=0)
output_seq, attention_weight_seq = [], []
for _ in range(num_steps):
    Y, dec_state = net.decoder(dec_X, dec_state)
    # 我们使用具有预测最高可能性的词元, 作为解码器在下一时间步的输入
    dec_X = Y.argmax(dim=2)
    pred = dec_X.squeeze(dim=0).type(torch.int32).item()
    # 保存注意力权重 (稍后讨论)
    if save_attention_weights:
        attention_weight_seq.append(net.decoder.attention_weights)
    # 一旦序列结束词元被预测, 输出序列的生成就完成了
    if pred == tgt_vocab['<eos>']:

```

```

        break
    output_seq.append(pred)
    return ' '.join(tgt_vocab.to_tokens(output_seq)), attention_weight_seq

def bleu(pred_seq, label_seq, k): #@save
    """计算BLEU"""
    pred_tokens, label_tokens = pred_seq.split(' '), label_seq.split(' ')
    len_pred, len_label = len(pred_tokens), len(label_tokens)
    score = math.exp(min(0, 1 - len_label / len_pred))
    for n in range(1, k + 1):
        num_matches, label_subs = 0, collections.defaultdict(int)
        for i in range(len_label - n + 1):
            label_subs[' '.join(label_tokens[i: i + n])] += 1
        for i in range(len_pred - n + 1):
            if label_subs[' '.join(pred_tokens[i: i + n])] > 0:
                num_matches += 1
                label_subs[' '.join(pred_tokens[i: i + n])] -= 1
        score *= math.pow(num_matches / (len_pred - n + 1), math.pow(0.5, n))
    return score

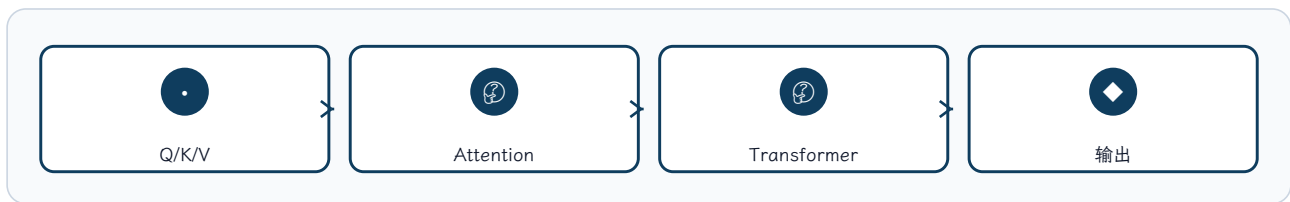
engs = ['go .', 'i lost .', 'he\'s calm .', 'i\'m home .']
fras = ['va !', 'j\'ai perdu .', 'il est calme .', 'je suis chez moi .']
for eng, fra in zip(engs, fras):
    translation, attention_weight_seq = predict_seq2seq(
        net, eng, src_vocab, tgt_vocab, num_steps, device)
    print(f'{eng} => {translation}, bleu {bleu(translation, fra, k=2):.3f}')

```

第 10 章：注意力机制

本章分类：注意力机制与 Transformer 章节说明：评分函数、加性/多头/自注意力和 Transformer。

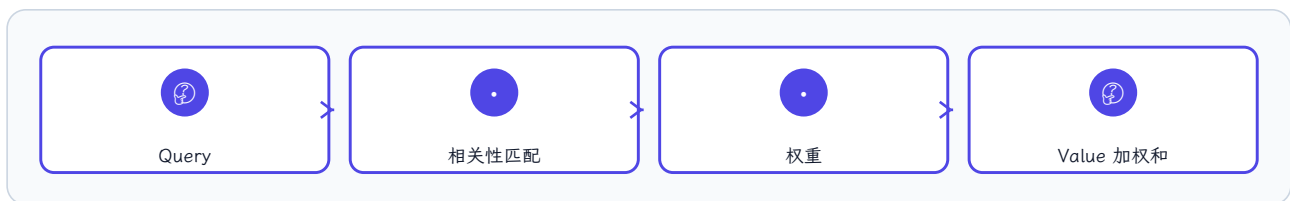
本章网络/流程总览



第 10 章 · 注意力机制 · [chapters/chapter_10_attention_mechanisms/attention_cues.py](#)

章节对应：第 10 章 类别：网络结构 代码块：3 个

结构图：注意力机制（网络结构）



第 10 章注意力的基本抽象。

阅读注释：阅读顺序：先看输入与输出，再看核心层/模块，最后看损失函数和训练循环。

Query 用来提问，Key 用来匹配相关性，Value 提供被加权汇总的内容。

"""attention-cues: 注意力机制"""

来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。

改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。

from pathlib import Path

import sys

_ROOT = Path(__file__).resolve().parents[1]

if str(_ROOT) not in sys.path:

sys.path.insert(0, str(_ROOT))

import math

import random

import time

import numpy as np

import pandas as pd

import torch

from torch import nn

from torch.nn import functional as F

from torch.utils import data

import torchvision

from torchvision import transforms

from PIL import Image

from matplotlib import pyplot as plt

from IPython import display

import common as native

import common as native

import torch

def show_heatmaps(matrices, xlabel, ylabel, titles=None, figsize=(2.5, 2.5),
cmap='Reds'):

"""显示矩阵热图"""

native.use_svg_display()

num_rows, num_cols = matrices.shape[0], matrices.shape[1]

fig, axes = native.plt.subplots(num_rows, num_cols, figsize=figsize,
sharex=True, sharey=True, squeeze=False)

for i, (row_axes, row_matrices) in enumerate(zip(axes, matrices)):

for j, (ax, matrix) in enumerate(zip(row_axes, row_matrices)):

pcm = ax.imshow(native.numpy(matrix), cmap=cmap)

if i == num_rows - 1:

```

        ax.set_xlabel(xlabel)
    if j == 0:
        ax.set_ylabel(ylabel)
    if titles:
        ax.set_title(titles[j])
    fig.colorbar(pcm, ax=axes, shrink=0.6);

attention_weights = native.reshape(native.eye(10), (1, 1, 10, 10))
show_heatmaps(attention_weights, xlabel='Keys', ylabel='Queries')

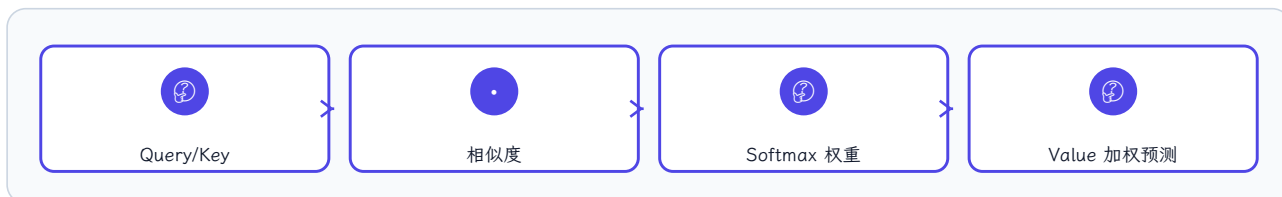
```

第 10 章 • Nadaraya–Watson 注意力 •

chapters/chapter_10_attention_mechanisms/nadaraya_waston.py

章节对应：第 10 章 类别：网络结构 代码块：14 个

结构图：Nadaraya–Watson 注意力（网络结构）



第 10 章注意力加权回归。

阅读注释：阅读顺序：先看输入与输出，再看核心层/模块，最后看损失函数和训练循环。

关键点：沿图中的箭头追踪张量形状，重点定位 nn.Module、forward、损失函数和训练循环。

"""nadaraya-waston: 注意力机制"""

来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。

改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。

from pathlib import Path

import sys

_ROOT = Path(__file__).resolve().parents[1]

if str(_ROOT) not in sys.path:

sys.path.insert(0, str(_ROOT))

import math

import random

import time

import numpy as np

import pandas as pd

import torch

from torch import nn

from torch.nn import functional as F

from torch.utils import data

import torchvision

from torchvision import transforms

from PIL import Image

from matplotlib import pyplot as plt

from IPython import display

import common as native

import common as native

import torch

from torch import nn

n_train = 50 # 训练样本数

x_train, _ = torch.sort(native.rand(n_train) * 5) # 排序后的训练样本

def f(x):

return 2 * native.sin(x) + x**0.8

y_train = f(x_train) + native.normal(0.0, 0.5, (n_train,)) # 训练样本的输出

x_test = native.arange(0, 5, 0.1) # 测试样本

y_truth = f(x_test) # 测试样本的真实输出

n_test = len(x_test) # 测试样本数

n_test

def plot_kernel_reg(y_hat):

native.plot(x_test, [y_truth, y_hat], 'x', 'y', legend=['Truth', 'Pred'],
 xlim=[0, 5], ylim=[-1, 5])

```

    native.plt.plot(x_train, y_train, 'o', alpha=0.5);

y_hat = torch.repeat_interleave(y_train.mean(), n_test)
plot_kernel_reg(y_hat)

# X_repeat的形状:(n_test,n_train),
# 每一行都包含着相同的测试输入 (例如: 同样的查询)
X_repeat = native.reshape(x_test.repeat_interleave(n_train), (-1, n_train))
# x_train包含着键. attention_weights的形状: (n_test,n_train),
# 每一行都包含着要在给定的每个查询的值 (y_train) 之间分配的注意力权重
attention_weights = nn.functional.softmax(-(X_repeat - x_train)**2 / 2, dim=1)
# y_hat的每个元素都是值的加权平均值, 其中的权重是注意力权重
y_hat = native.matmul(attention_weights, y_train)
plot_kernel_reg(y_hat)

native.show_heatmaps(attention_weights.unsqueeze(0).unsqueeze(0),
                    xlabel='Sorted training inputs',
                    ylabel='Sorted testing inputs')

X = native.ones((2, 1, 4))
Y = native.ones((2, 4, 6))
torch.bmm(X, Y).shape

weights = native.ones((2, 10)) * 0.1
values = native.reshape(native.arange(20.0), (2, 10))
torch.bmm(weights.unsqueeze(1), values.unsqueeze(-1))

class NWKernelRegression(nn.Module):
    def __init__(self, **kwargs):
        super().__init__(**kwargs)
        self.w = nn.Parameter(torch.rand((1,), requires_grad=True))

    def forward(self, queries, keys, values):
        # queries和attention_weights的形状为(查询个数, "键-值"对个数)
        queries = native.reshape(
            queries.repeat_interleave(keys.shape[1]), (-1, keys.shape[1]))
        self.attention_weights = nn.functional.softmax(
            -((queries - keys) * self.w)**2 / 2, dim=1)
        # values的形状为(查询个数, "键-值"对个数)
        return torch.bmm(self.attention_weights.unsqueeze(1),
                        values.unsqueeze(-1)).reshape(-1)

# X_tile的形状:(n_train, n_train), 每一行都包含着相同的训练输入
X_tile = x_train.repeat((n_train, 1))
# Y_tile的形状:(n_train, n_train), 每一行都包含着相同的训练输出
Y_tile = y_train.repeat((n_train, 1))
# keys的形状:('n_train', 'n_train'-1)
keys = native.reshape(X_tile[(1 - native.eye(n_train)).type(torch.bool)],
                    (n_train, -1))
# values的形状:('n_train', 'n_train'-1)
values = native.reshape(Y_tile[(1 - native.eye(n_train)).type(torch.bool)],
                    (n_train, -1))

net = NWKernelRegression()
loss = nn.MSELoss(reduction='none')
trainer = torch.optim.SGD(net.parameters(), lr=0.5)
animator = native.Animator(xlabel='epoch', ylabel='loss', xlim=[1, 5])

for epoch in range(5):
    trainer.zero_grad()
    l = loss(net(x_train, keys, values), y_train)
    l.sum().backward()
    trainer.step()
    print(f'epoch {epoch + 1}, loss {float(l.sum()):.6f}')
    animator.add(epoch + 1, float(l.sum()))

# keys的形状:(n_test, n_train), 每一行包含着相同的训练输入 (例如, 相同的键)
keys = x_train.repeat((n_test, 1))
# value的形状:(n_test, n_train)
values = y_train.repeat((n_test, 1))
y_hat = net(x_test, keys, values).unsqueeze(1).detach()
plot_kernel_reg(y_hat)

native.show_heatmaps(net.attention_weights.unsqueeze(0).unsqueeze(0),
                    xlabel='Sorted training inputs',
                    ylabel='Sorted testing inputs')

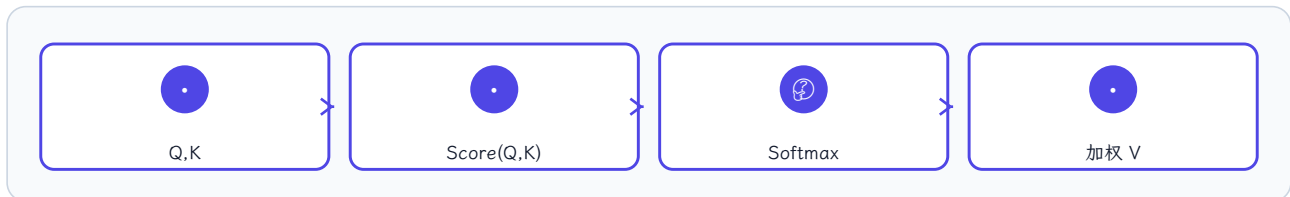
```

第 10 章 · 注意力评分函数 ·

chapters/chapter_10_attention_mechanisms/attention_scoring_functions.py

章节对应：第 10 章 类别：网络结构 代码块：10 个

结构图：注意力评分函数（网络结构）



第 10 章点积与加性评分函数。

阅读注释：阅读顺序：先看输入与输出，再看核心层/模块，最后看损失函数和训练循环。

关键点：沿图中的箭头追踪张量形状，重点定位 nn.Module、forward、损失函数和训练循环。

```
"""attention-scoring-functions: 注意力机制"""
```

```
# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
```

```
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
```

```
from pathlib import Path
```

```
import sys
```

```
_ROOT = Path(__file__).resolve().parents[1]
```

```
if str(_ROOT) not in sys.path:
```

```
    sys.path.insert(0, str(_ROOT))
```

```
import math
```

```
import random
```

```
import time
```

```
import numpy as np
```

```
import pandas as pd
```

```
import torch
```

```
from torch import nn
```

```
from torch.nn import functional as F
```

```
from torch.utils import data
```

```
import torchvision
```

```
from torchvision import transforms
```

```
from PIL import Image
```

```
from matplotlib import pyplot as plt
```

```
from IPython import display
```

```
import common as native
```

```
import common as native
```

```
import math
```

```
import torch
```

```
from torch import nn
```

```
def masked_softmax(X, valid_lens):
```

```
    """通过在最后一个轴上掩蔽元素来执行softmax操作"""
```

```
    # X:3D张量, valid_lens:1D或2D张量
```

```
    if valid_lens is None:
```

```
        return nn.functional.softmax(X, dim=-1)
```

```
    else:
```

```
        shape = X.shape
```

```
        if valid_lens.dim() == 1:
```

```
            valid_lens = torch.repeat_interleave(valid_lens, shape[1])
```

```
        else:
```

```
            valid_lens = valid_lens.reshape(-1)
```

```
    # 最后一轴上被掩蔽的元素使用一个非常大的负值替换，从而其softmax输出为0
```

```
    X = native.sequence_mask(X.reshape(-1, shape[-1]), valid_lens,
```

```
                             value=-1e6)
```

```
    return nn.functional.softmax(X.reshape(shape), dim=-1)
```

```
masked_softmax(torch.rand(2, 2, 4), torch.tensor([2, 3]))
```

```
masked_softmax(torch.rand(2, 2, 4), native.tensor([[1, 3], [2, 4]]))
```

```
class AdditiveAttention(nn.Module):
```

```
    """加性注意力"""
```

```
    def __init__(self, key_size, query_size, num_hiddens, dropout, **kwargs):
```

```
        super(AdditiveAttention, self).__init__(**kwargs)
```

```

self.W_k = nn.Linear(key_size, num_hiddens, bias=False)
self.W_q = nn.Linear(query_size, num_hiddens, bias=False)
self.w_v = nn.Linear(num_hiddens, 1, bias=False)
self.dropout = nn.Dropout(dropout)

def forward(self, queries, keys, values, valid_lens):
    queries, keys = self.W_q(queries), self.W_k(keys)
    # 在维度扩展后,
    # queries的形状: (batch_size, 查询的个数, 1, num_hidden)
    # key的形状: (batch_size, 1, "键-值"对的个数, num_hiddens)
    # 使用广播方式进行求和
    features = queries.unsqueeze(2) + keys.unsqueeze(1)
    features = torch.tanh(features)
    # self.w_v仅有一个输出, 因此从形状中移除最后那个维度。
    # scores的形状: (batch_size, 查询的个数, "键-值"对的个数)
    scores = self.w_v(features).squeeze(-1)
    self.attention_weights = masked_softmax(scores, valid_lens)
    # values的形状: (batch_size, "键-值"对的个数, 值的维度)
    return torch.bmm(self.dropout(self.attention_weights), values)

queries, keys = native.normal(0, 1, (2, 1, 20)), native.ones((2, 10, 2))
# values的小批量, 两个值矩阵是相同的
values = torch.arange(40, dtype=torch.float32).reshape(1, 10, 4).repeat(
    2, 1, 1)
valid_lens = native.tensor([2, 6])

attention = AdditiveAttention(key_size=2, query_size=20, num_hiddens=8,
                             dropout=0.1)
attention.eval()
attention(queries, keys, values, valid_lens)

native.show_heatmaps(native.reshape(attention.attention_weights, (1, 1, 2, 10)),
                     xlabel='Keys', ylabel='Queries')

class DotProductAttention(nn.Module):
    """缩放点积注意力"""
    def __init__(self, dropout, **kwargs):
        super(DotProductAttention, self).__init__(**kwargs)
        self.dropout = nn.Dropout(dropout)

    # queries的形状: (batch_size, 查询的个数, d)
    # keys的形状: (batch_size, "键-值"对的个数, d)
    # values的形状: (batch_size, "键-值"对的个数, 值的维度)
    # valid_lens的形状:(batch_size, )或者(batch_size, 查询的个数)
    def forward(self, queries, keys, values, valid_lens=None):
        d = queries.shape[-1]
        # 设置transpose_b=True为了交换keys的最后两个维度
        scores = torch.bmm(queries, keys.transpose(1,2)) / math.sqrt(d)
        self.attention_weights = masked_softmax(scores, valid_lens)
        return torch.bmm(self.dropout(self.attention_weights), values)

queries = native.normal(0, 1, (2, 1, 2))
attention = DotProductAttention(dropout=0.5)
attention.eval()
attention(queries, keys, values, valid_lens)

native.show_heatmaps(native.reshape(attention.attention_weights, (1, 1, 2, 10)),
                     xlabel='Keys', ylabel='Queries')

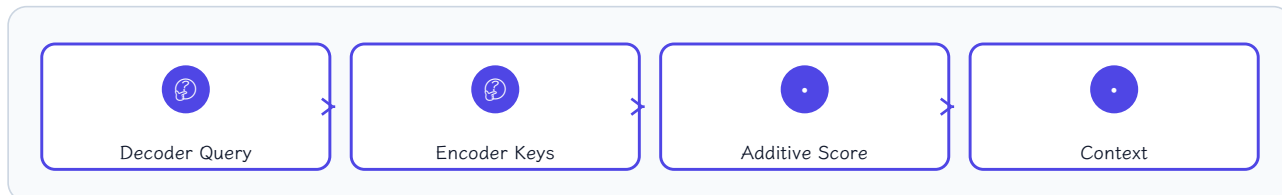
```

第 10 章 · Bahdanau 加性注意力 ·

chapters/chapter_10_attention_mechanisms/bahdanau_attention.py

章节对应：第 10 章 类别：网络结构 代码块：8 个

结构图：Bahdanau 加性注意力（网络结构）



第 10 章序列到序列中的加性注意力。

阅读注释：阅读顺序：先看输入与输出，再看核心层/模块，最后看损失函数和训练循环。

关键点：沿图中的箭头追踪张量形状，重点定位 nn.Module、forward、损失函数和训练循环。

"""bahdanau-attention: 注意力机制"""

来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。

改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。

from pathlib import Path

import sys

_ROOT = Path(__file__).resolve().parents[1]

if str(_ROOT) not in sys.path:

sys.path.insert(0, str(_ROOT))

import math

import random

import time

import numpy as np

import pandas as pd

import torch

from torch import nn

from torch.nn import functional as F

from torch.utils import data

import torchvision

from torchvision import transforms

from PIL import Image

from matplotlib import pyplot as plt

from IPython import display

import common as native

import common as native

import torch

from torch import nn

class AttentionDecoder(native.Decoder):

"""带有注意力机制解码器的基本接口"""

def __init__(self, **kwargs):

super(AttentionDecoder, self).__init__(**kwargs)

@property

def attention_weights(self):

raise NotImplementedError

class Seq2SeqAttentionDecoder(AttentionDecoder):

def __init__(self, vocab_size, embed_size, num_hiddens, num_layers,

dropout=0, **kwargs):

super(Seq2SeqAttentionDecoder, self).__init__(**kwargs)

self.attention = native.AdditiveAttention(
num_hiddens, num_hiddens, num_hiddens, dropout)

self.embedding = nn.Embedding(vocab_size, embed_size)

self.rnn = nn.GRU(
embed_size + num_hiddens, num_hiddens, num_layers,

dropout=dropout)

self.dense = nn.Linear(num_hiddens, vocab_size)

def init_state(self, enc_outputs, enc_valid_lens, *args):

outputs的形状为(batch_size, num_steps, num_hiddens).

hidden_state的形状为(num_layers, batch_size, num_hiddens)

outputs, hidden_state = enc_outputs

```

        return (outputs.permute(1, 0, 2), hidden_state, enc_valid_lens)

def forward(self, X, state):
    # enc_outputs的形状为(batch_size,num_steps,num_hiddens).
    # hidden_state的形状为(num_layers,batch_size,
        # num_hiddens)
    enc_outputs, hidden_state, enc_valid_lens = state
    # 输出X的形状为(num_steps,batch_size,embed_size)
    X = self.embedding(X).permute(1, 0, 2)
    outputs, self._attention_weights = [], []
    for x in X:
        # query的形状为(batch_size,1,num_hiddens)
        query = torch.unsqueeze(hidden_state[-1], dim=1)
        # context的形状为(batch_size,1,num_hiddens)
        context = self.attention(
            query, enc_outputs, enc_outputs, enc_valid_lens)
        # 在特征维度上连结
        x = torch.cat((context, torch.unsqueeze(x, dim=1)), dim=-1)
        # 将x变形为(1,batch_size,embed_size+num_hiddens)
        out, hidden_state = self.rnn(x.permute(1, 0, 2), hidden_state)
        outputs.append(out)
        self._attention_weights.append(self.attention.attention_weights)
    # 全连接层变换后, outputs的形状为
    # (num_steps,batch_size,vocab_size)
    outputs = self.dense(torch.cat(outputs, dim=0))
    return outputs.permute(1, 0, 2), [enc_outputs, hidden_state,
                                       enc_valid_lens]

@property
def attention_weights(self):
    return self._attention_weights

encoder = native.Seq2SeqEncoder(vocab_size=10, embed_size=8, num_hiddens=16,
                               num_layers=2)
encoder.eval()
decoder = Seq2SeqAttentionDecoder(vocab_size=10, embed_size=8, num_hiddens=16,
                                  num_layers=2)
decoder.eval()
X = native.zeros((4, 7), dtype=torch.long) # (batch_size,num_steps)
state = decoder.init_state(encoder(X), None)
output, state = decoder(X, state)
output.shape, len(state), state[0].shape, len(state[1]), state[1][0].shape

embed_size, num_hiddens, num_layers, dropout = 32, 32, 2, 0.1
batch_size, num_steps = 64, 10
lr, num_epochs, device = 0.005, 250, native.try_gpu()

train_iter, src_vocab, tgt_vocab = native.load_data_nmt(batch_size, num_steps)
encoder = native.Seq2SeqEncoder(
    len(src_vocab), embed_size, num_hiddens, num_layers, dropout)
decoder = Seq2SeqAttentionDecoder(
    len(tgt_vocab), embed_size, num_hiddens, num_layers, dropout)
net = native.EncoderDecoder(encoder, decoder)
native.train_seq2seq(net, train_iter, lr, num_epochs, tgt_vocab, device)

engs = ['go .', 'i lost .', 'he\'s calm .', 'i\'m home .']
fras = ['va !', 'j\'ai perdu .', 'il est calme .', 'je suis chez moi .']
for eng, fra in zip(engs, fras):
    translation, dec_attention_weight_seq = native.predict_seq2seq(
        net, eng, src_vocab, tgt_vocab, num_steps, device, True)
    print(f'{eng} => {translation}, ',
          f'bleu {native.bleu(translation, fra, k=2):.3f}')

attention_weights = native.reshape(
    native.concat([step[0][0][0] for step in dec_attention_weight_seq], 0),
    (1, 1, -1, num_steps))

# 加上一个包含序列结束词元
native.show_heatmaps(
    attention_weights[:, :, :, :len(engs[-1].split()) + 1].cpu(),
    xlabel='Key positions', ylabel='Query positions')

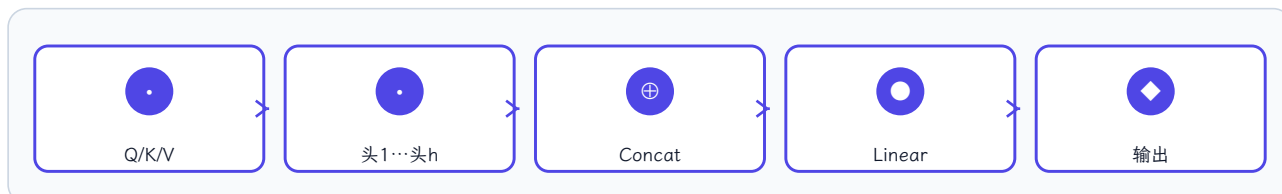
```

第 10 章 • Multi-Head Attention •

chapters/chapter_10_attention_mechanisms/multihead_attention.py

章节对应：第 10 章 类别：网络结构 代码块：5 个

结构图：Multi-Head Attention（网络结构）



第 10 章多头注意力。

阅读注释：阅读顺序：先看输入与输出，再看核心层/模块，最后看损失函数和训练循环。

各注意力头在不同表示子空间中工作，Concat 后再通过 Linear 混合。

"""multihead-attention: 注意力机制"""

来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。

改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。

from pathlib import Path

import sys

_ROOT = Path(__file__).resolve().parents[1]

if str(_ROOT) not in sys.path:

sys.path.insert(0, str(_ROOT))

import math

import random

import time

import numpy as np

import pandas as pd

import torch

from torch import nn

from torch.nn import functional as F

from torch.utils import data

import torchvision

from torchvision import transforms

from PIL import Image

from matplotlib import pyplot as plt

from IPython import display

import common as native

import common as native

import math

import torch

from torch import nn

class MultiHeadAttention(nn.Module):

"""多头注意力"""

def __init__(self, key_size, query_size, value_size, num_hiddens,

num_heads, dropout, bias=False, **kwargs):

super(MultiHeadAttention, self).__init__(**kwargs)

self.num_heads = num_heads

self.attention = native.DotProductAttention(dropout)

self.W_q = nn.Linear(query_size, num_hiddens, bias=bias)

self.W_k = nn.Linear(key_size, num_hiddens, bias=bias)

self.W_v = nn.Linear(value_size, num_hiddens, bias=bias)

self.W_o = nn.Linear(num_hiddens, num_hiddens, bias=bias)

def forward(self, queries, keys, values, valid_lens):

queries, keys, values 的形状:

(batch_size, 查询或者“键-值”对的个数, num_hiddens)

valid_lens 的形状:

(batch_size,) 或 (batch_size, 查询的个数)

经过变换后, 输出的 queries, keys, values 的形状:

(batch_size*num_heads, 查询或者“键-值”对的个数,

num_hiddens/num_heads)

queries = transpose_qkv(self.W_q(queries), self.num_heads)

keys = transpose_qkv(self.W_k(keys), self.num_heads)

values = transpose_qkv(self.W_v(values), self.num_heads)

```

        if valid_lens is not None:
            # 在轴0, 将第一项 (标量或者向量) 复制num_heads次,
            # 然后如此复制第二项, 然后诸如此类。
            valid_lens = torch.repeat_interleave(
                valid_lens, repeats=self.num_heads, dim=0)

        # output的形状:(batch_size*num_heads, 查询的个数,
        # num_hiddens/num_heads)
        output = self.attention(queries, keys, values, valid_lens)

        # output_concat的形状:(batch_size, 查询的个数, num_hiddens)
        output_concat = transpose_output(output, self.num_heads)
        return self.W_o(output_concat)

def transpose_qkv(X, num_heads):
    """为了多注意力头的并行计算而变换形状"""
    # 输入X的形状:(batch_size, 查询或者“键-值”对的个数, num_hiddens)
    # 输出X的形状:(batch_size, 查询或者“键-值”对的个数, num_heads,
    # num_hiddens/num_heads)
    X = X.reshape(X.shape[0], X.shape[1], num_heads, -1)

    # 输出X的形状:(batch_size, num_heads, 查询或者“键-值”对的个数,
    # num_hiddens/num_heads)
    X = X.permute(0, 2, 1, 3)

    # 最终输出的形状:(batch_size*num_heads,查询或者“键-值”对的个数,
    # num_hiddens/num_heads)
    return X.reshape(-1, X.shape[2], X.shape[3])

def transpose_output(X, num_heads):
    """逆转transpose_qkv函数的操作"""
    X = X.reshape(-1, num_heads, X.shape[1], X.shape[2])
    X = X.permute(0, 2, 1, 3)
    return X.reshape(X.shape[0], X.shape[1], -1)

num_hiddens, num_heads = 100, 5
attention = MultiHeadAttention(num_hiddens, num_hiddens, num_hiddens,
                               num_hiddens, num_heads, 0.5)
attention.eval()

batch_size, num_queries = 2, 4
num_kvpairs, valid_lens = 6, native.tensor([3, 2])
X = native.ones((batch_size, num_queries, num_hiddens))
Y = native.ones((batch_size, num_kvpairs, num_hiddens))
attention(X, Y, Y, valid_lens).shape

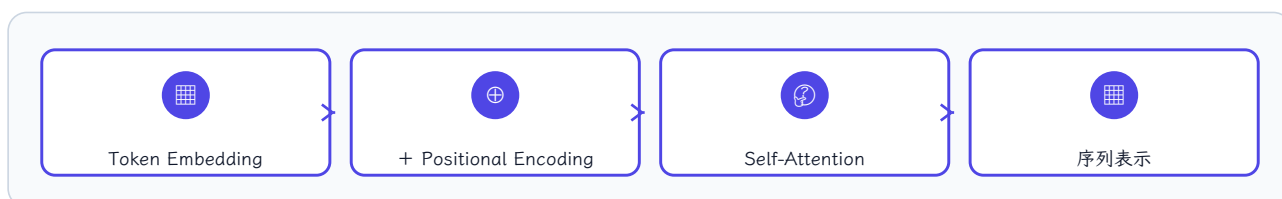
```

第 10 章 · Self-Attention + 位置编码 ·

chapters/chapter_10_attention_mechanisms/self_attention_and_positional_encoding.py

章节对应: 第 10 章 类别: 网络结构 代码块: 7 个

结构图: Self-Attention + 位置编码 (网络结构)



第 10 章 Transformer 的核心输入表示。

阅读注释: 阅读顺序: 先看输入与输出, 再看核心层/模块, 最后看损失函数和训练循环。

关键点: 沿图中的箭头追踪张量形状, 重点定位 nn.Module、forward、损失函数和训练循环。

```

"""self-attention-and-positional-encoding: 注意力机制"""

# 来源: 依据《动手学深度学习 (PyTorch 第二版)》的 D2L 中文 PyTorch 示例整理。
# 改写说明: 已移除 d2l 命名空间, 共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

```

```

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import math
import torch
from torch import nn

num_hiddens, num_heads = 100, 5
attention = native.MultiHeadAttention(num_hiddens, num_hiddens, num_hiddens,
                                     num_hiddens, num_heads, 0.5)
attention.eval()

batch_size, num_queries, valid_lens = 2, 4, native.tensor([3, 2])
X = native.ones((batch_size, num_queries, num_hiddens))
attention(X, X, X, valid_lens).shape

class PositionalEncoding(nn.Module):
    """位置编码"""
    def __init__(self, num_hiddens, dropout, max_len=1000):
        super(PositionalEncoding, self).__init__()
        self.dropout = nn.Dropout(dropout)
        # 创建一个足够长的P
        self.P = native.zeros((1, max_len, num_hiddens))
        X = native.arange(max_len, dtype=torch.float32).reshape(
            -1, 1) / torch.pow(10000, torch.arange(
                0, num_hiddens, 2, dtype=torch.float32) / num_hiddens)
        self.P[:, :, 0::2] = torch.sin(X)
        self.P[:, :, 1::2] = torch.cos(X)

    def forward(self, X):
        X = X + self.P[:, :X.shape[1], :].to(X.device)
        return self.dropout(X)

encoding_dim, num_steps = 32, 60
pos_encoding = PositionalEncoding(encoding_dim, 0)
pos_encoding.eval()

X = pos_encoding(native.zeros((1, num_steps, encoding_dim)))
P = pos_encoding.P[:, :X.shape[1], :]
native.plot(native.arange(num_steps), P[0, :, 6:10].T, xlabel='Row (position)',
           figsize=(6, 2.5), legend=["Col %d" % d for d in native.arange(6, 10)])

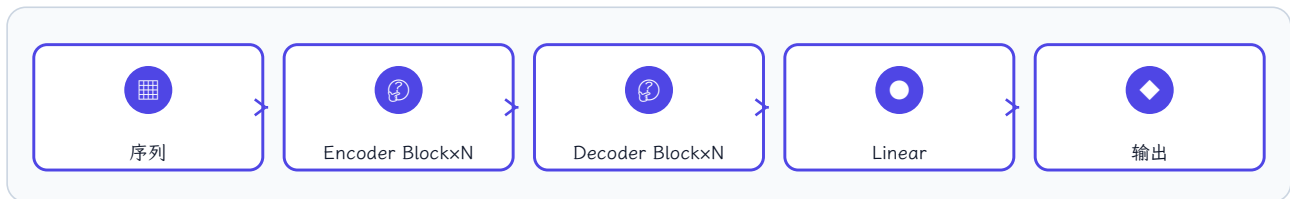
for i in range(8):
    print(f'{i}的二进制是: {i:>03b}')

P = P[0, :, :].unsqueeze(0).unsqueeze(0)
native.show_heatmaps(P, xlabel='Column (encoding dimension)',
                    ylabel='Row (position)', figsize=(3.5, 4), cmap='Blues')

```

章节对应：第 10 章 类别：网络结构 代码块：20 个

结构图：Transformer（网络结构）



第 10 章基于注意力的编码器-解码器网络。

阅读注释：阅读顺序：先看输入与输出，再看核心层/模块，最后看损失函数和训练循环。
注意力负责 token 间交互，位置编码补充顺序信息，Encoder/Decoder block 可重复堆叠。

```

"""transformer: 注意力机制"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import math
import pandas as pd
import torch
from torch import nn

class PositionWiseFFN(nn.Module):
    """基于位置的前馈网络"""
    def __init__(self, ffn_num_input, ffn_num_hiddens, ffn_num_outputs,
                 **kwargs):
        super(PositionWiseFFN, self).__init__(**kwargs)
        self.dense1 = nn.Linear(ffn_num_input, ffn_num_hiddens)
        self.relu = nn.ReLU()
        self.dense2 = nn.Linear(ffn_num_hiddens, ffn_num_outputs)

    def forward(self, X):
        return self.dense2(self.relu(self.dense1(X)))

ffn = PositionWiseFFN(4, 4, 8)
ffn.eval()
ffn(native.ones((2, 3, 4)))

ln = nn.LayerNorm(2)
bn = nn.BatchNorm1d(2)
X = native.tensor([[1, 2], [2, 3]], dtype=torch.float32)
# 在训练模式下计算X的均值和方差
print('layer norm:', ln(X), '\nbatch norm:', bn(X))

class AddNorm(nn.Module):
    """残差连接后进行层规范化"""
    def __init__(self, normalized_shape, dropout, **kwargs):

```

```

        super(AddNorm, self).__init__(**kwargs)
        self.dropout = nn.Dropout(dropout)
        self.ln = nn.LayerNorm(normalized_shape)

    def forward(self, X, Y):
        return self.ln(self.dropout(Y) + X)

add_norm = AddNorm([3, 4], 0.5)
add_norm.eval()
add_norm(native.ones((2, 3, 4)), native.ones((2, 3, 4))).shape

class EncoderBlock(nn.Module):
    """Transformer编码器块"""
    def __init__(self, key_size, query_size, value_size, num_hiddens,
                  norm_shape, ffn_num_input, ffn_num_hiddens, num_heads,
                  dropout, use_bias=False, **kwargs):
        super(EncoderBlock, self).__init__(**kwargs)
        self.attention = native.MultiHeadAttention(
            key_size, query_size, value_size, num_hiddens, num_heads, dropout,
            use_bias)
        self.addnorm1 = AddNorm(norm_shape, dropout)
        self.ffn = PositionWiseFFN(
            ffn_num_input, ffn_num_hiddens, num_hiddens)
        self.addnorm2 = AddNorm(norm_shape, dropout)

    def forward(self, X, valid_lens):
        Y = self.addnorm1(X, self.attention(X, X, X, valid_lens))
        return self.addnorm2(Y, self.ffn(Y))

X = native.ones((2, 100, 24))
valid_lens = native.tensor([3, 2])
encoder_blk = EncoderBlock(24, 24, 24, 24, [100, 24], 24, 48, 8, 0.5)
encoder_blk.eval()
encoder_blk(X, valid_lens).shape

class TransformerEncoder(native.Encoder):
    """Transformer编码器"""
    def __init__(self, vocab_size, key_size, query_size, value_size,
                  num_hiddens, norm_shape, ffn_num_input, ffn_num_hiddens,
                  num_heads, num_layers, dropout, use_bias=False, **kwargs):
        super(TransformerEncoder, self).__init__(**kwargs)
        self.num_hiddens = num_hiddens
        self.embedding = nn.Embedding(vocab_size, num_hiddens)
        self.pos_encoding = native.PositionalEncoding(num_hiddens, dropout)
        self.blks = nn.Sequential()
        for i in range(num_layers):
            self.blks.add_module("block"+str(i),
                                  EncoderBlock(key_size, query_size, value_size, num_hiddens,
                                                  norm_shape, ffn_num_input, ffn_num_hiddens,
                                                  num_heads, dropout, use_bias))

    def forward(self, X, valid_lens, *args):
        # 因为位置编码值在-1和1之间,
        # 因此嵌入值乘以嵌入维度的平方根进行缩放,
        # 然后再与位置编码相加。
        X = self.pos_encoding(self.embedding(X) * math.sqrt(self.num_hiddens))
        self.attention_weights = [None] * len(self.blks)
        for i, blk in enumerate(self.blks):
            X = blk(X, valid_lens)
            self.attention_weights[
                i] = blk.attention.attention.attention_weights
        return X

encoder = TransformerEncoder(
    200, 24, 24, 24, 24, [100, 24], 24, 48, 8, 2, 0.5)
encoder.eval()
encoder(native.ones((2, 100), dtype=torch.long), valid_lens).shape

class DecoderBlock(nn.Module):
    """解码器中第i个块"""
    def __init__(self, key_size, query_size, value_size, num_hiddens,
                  norm_shape, ffn_num_input, ffn_num_hiddens, num_heads,
                  dropout, i, **kwargs):
        super(DecoderBlock, self).__init__(**kwargs)
        self.i = i
        self.attention1 = native.MultiHeadAttention(

```

```

        key_size, query_size, value_size, num_hiddens, num_heads, dropout)
self.addnorm1 = AddNorm(norm_shape, dropout)
self.attention2 = native.MultiHeadAttention(
    key_size, query_size, value_size, num_hiddens, num_heads, dropout)
self.addnorm2 = AddNorm(norm_shape, dropout)
self.ffn = PositionWiseFFN(ffn_num_input, ffn_num_hiddens,
                           num_hiddens)
self.addnorm3 = AddNorm(norm_shape, dropout)

def forward(self, X, state):
    enc_outputs, enc_valid_lens = state[0], state[1]
    # 训练阶段，输出序列的所有词元都在同一时间处理。
    # 因此state[2][self.i]初始化为None。
    # 预测阶段，输出序列是通过词元一个接着一个解码的。
    # 因此state[2][self.i]包含着直到当前时间步第i块解码的输出表示
    if state[2][self.i] is None:
        key_values = X
    else:
        key_values = torch.cat((state[2][self.i], X), axis=1)
    state[2][self.i] = key_values
    if self.training:
        batch_size, num_steps, _ = X.shape
        # dec_valid_lens的开头:(batch_size,num_steps).
        # 其中每一行是[1,2,...,num_steps]
        dec_valid_lens = torch.arange(
            1, num_steps + 1, device=X.device).repeat(batch_size, 1)
    else:
        dec_valid_lens = None

    # 自注意力
    X2 = self.attention1(X, key_values, key_values, dec_valid_lens)
    Y = self.addnorm1(X, X2)
    # 编码器-解码器注意力。
    # enc_outputs的开头:(batch_size,num_steps,num_hiddens)
    Y2 = self.attention2(Y, enc_outputs, enc_outputs, enc_valid_lens)
    Z = self.addnorm2(Y, Y2)
    return self.addnorm3(Z, self.ffn(Z)), state

decoder_blk = DecoderBlock(24, 24, 24, 24, [100, 24], 24, 48, 8, 0.5, 0)
decoder_blk.eval()
X = native.ones((2, 100, 24))
state = [encoder_blk(X, valid_lens), valid_lens, [None]]
decoder_blk(X, state)[0].shape

class TransformerDecoder(native.AttentionDecoder):
    def __init__(self, vocab_size, key_size, query_size, value_size,
                 num_hiddens, norm_shape, ffn_num_input, ffn_num_hiddens,
                 num_heads, num_layers, dropout, **kwargs):
        super(TransformerDecoder, self).__init__(**kwargs)
        self.num_hiddens = num_hiddens
        self.num_layers = num_layers
        self.embedding = nn.Embedding(vocab_size, num_hiddens)
        self.pos_encoding = native.PositionalEncoding(num_hiddens, dropout)
        self.blks = nn.Sequential()
        for i in range(num_layers):
            self.blks.add_module("block"+str(i),
                DecoderBlock(key_size, query_size, value_size, num_hiddens,
                             norm_shape, ffn_num_input, ffn_num_hiddens,
                             num_heads, dropout, i))
        self.dense = nn.Linear(num_hiddens, vocab_size)

    def init_state(self, enc_outputs, enc_valid_lens, *args):
        return [enc_outputs, enc_valid_lens, [None] * self.num_layers]

    def forward(self, X, state):
        X = self.pos_encoding(self.embedding(X) * math.sqrt(self.num_hiddens))
        self._attention_weights = [[None] * len(self.blks) for _ in range(2)]
        for i, blk in enumerate(self.blks):
            X, state = blk(X, state)
        # 解码器自注意力权重
        self._attention_weights[0][
            i] = blk.attention1.attention.attention_weights
        # “编码器-解码器”自注意力权重
        self._attention_weights[1][
            i] = blk.attention2.attention.attention_weights
        return self.dense(X), state

```

```

@property
def attention_weights(self):
    return self._attention_weights

num_hiddens, num_layers, dropout, batch_size, num_steps = 32, 2, 0.1, 64, 10
lr, num_epochs, device = 0.005, 200, native.try_gpu()
ffn_num_input, ffn_num_hiddens, num_heads = 32, 64, 4
key_size, query_size, value_size = 32, 32, 32
norm_shape = [32]

train_iter, src_vocab, tgt_vocab = native.load_data_nmt(batch_size, num_steps)

encoder = TransformerEncoder(
    len(src_vocab), key_size, query_size, value_size, num_hiddens,
    norm_shape, ffn_num_input, ffn_num_hiddens, num_heads,
    num_layers, dropout)
decoder = TransformerDecoder(
    len(tgt_vocab), key_size, query_size, value_size, num_hiddens,
    norm_shape, ffn_num_input, ffn_num_hiddens, num_heads,
    num_layers, dropout)
net = native.EncoderDecoder(encoder, decoder)
native.train_seq2seq(net, train_iter, lr, num_epochs, tgt_vocab, device)

engs = ['go .', 'i lost .', 'he\'s calm .', 'i\'m home .']
fras = ['va !', 'j\'ai perdu .', 'il est calme .', 'je suis chez moi .']
for eng, fra in zip(engs, fras):
    translation, dec_attention_weight_seq = native.predict_seq2seq(
        net, eng, src_vocab, tgt_vocab, num_steps, device, True)
    print(f'{eng} => {translation}, '
          f'bleu {native.bleu(translation, fra, k=2):.3f}')

enc_attention_weights = native.reshape(
    native.concat(net.encoder.attention_weights, 0),
    (num_layers, num_heads, -1, num_steps))
enc_attention_weights.shape

native.show_heatmaps(
    enc_attention_weights.cpu(), xlabel='Key positions',
    ylabel='Query positions', titles=['Head %d' % i for i in range(1, 5)],
    figsize=(7, 3.5))

dec_attention_weights_2d = [head[0].tolist()
                           for step in dec_attention_weight_seq
                           for attn in step for blk in attn for head in blk]
dec_attention_weights_filled = native.tensor(
    pd.DataFrame(dec_attention_weights_2d).fillna(0.0).values)
dec_attention_weights = native.reshape(dec_attention_weights_filled,
    (-1, 2, num_layers, num_heads, num_steps))
dec_self_attention_weights, dec_inter_attention_weights = \
    dec_attention_weights.permute(1, 2, 3, 0, 4)
dec_self_attention_weights.shape, dec_inter_attention_weights.shape

# Plusonetoincludethebeginning-of-sequencetoken
native.show_heatmaps(
    dec_self_attention_weights[:, :, :, :len(translation.split()) + 1],
    xlabel='Key positions', ylabel='Query positions',
    titles=['Head %d' % i for i in range(1, 5)], figsize=(7, 3.5))

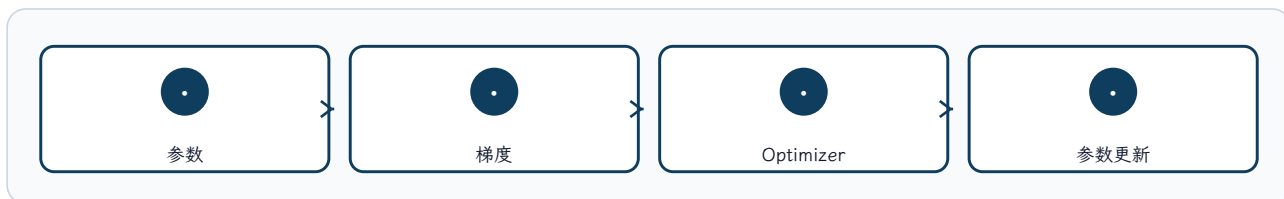
native.show_heatmaps(
    dec_inter_attention_weights, xlabel='Key positions',
    ylabel='Query positions', titles=['Head %d' % i for i in range(1, 5)],
    figsize=(7, 3.5))

```

第 11 章：优化算法

本章分类：优化算法 章节说明：梯度下降、动量、自适应方法和学习率调度；这里标注为训练流程。

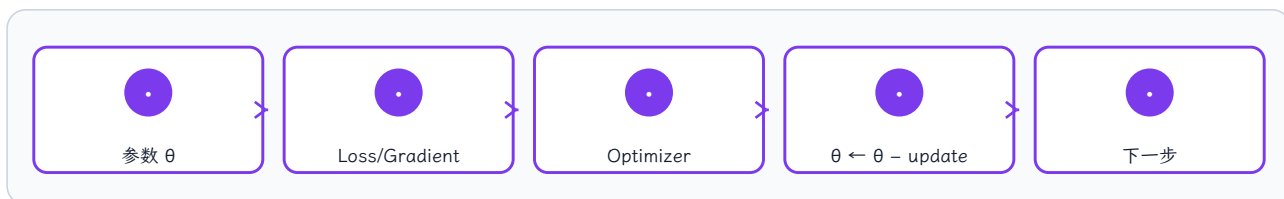
本章网络/流程总览



第 11 章 · 优化算法训练流程 · `chapters/chapter_11_optimization/optimization_intro.py`

章节对应：第 11 章 类别：训练流程 代码块：7 个

结构图：优化算法训练流程（训练流程）



第 11 章内容是训练算法或学习率调度，不对应新的网络结构。

阅读注释：阅读顺序：先看参数和梯度，再看更新规则，最后看学习率或正则化设置。

关键点：沿图中的箭头追踪张量形状，重点定位 `nn.Module`、`forward`、损失函数和训练循环。

"""optimization-intro: 优化算法"""

来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。

改写说明：已移除 `d2l` 命名空间，共享工具集中在 `common.py`。

from pathlib import Path

import sys

_ROOT = Path(__file__).resolve().parents[1]

if str(_ROOT) not in sys.path:

sys.path.insert(0, str(_ROOT))

import math

import random

import time

import numpy as np

import pandas as pd

import torch

from torch import nn

from torch.nn import functional as F

from torch.utils import data

import torchvision

from torchvision import transforms

from PIL import Image

from matplotlib import pyplot as plt

from IPython import display

import common as native

import common as native

import numpy as np

from mpl_toolkits import mplot3d

import torch

def f(x):

return x * native.cos(np.pi * x)

def g(x):

return f(x) + 0.2 * native.cos(5 * np.pi * x)

def annotate(text, xy, xytext): #@save

native.plt.gca().annotate(text, xy=xy, xytext=xytext,
arrowprops=dict(arrowstyle='->'))

```

x = native.arange(0.5, 1.5, 0.01)
native.set_figsize((4.5, 2.5))
native.plot(x, [f(x), g(x)], 'x', 'risk')
annotate('min of\nempirical risk', (1.0, -1.2), (0.5, -1.1))
annotate('min of risk', (1.1, -1.05), (0.95, -0.5))

x = native.arange(-1.0, 2.0, 0.01)
native.plot(x, [f(x), ], 'x', 'f(x)')
annotate('local minimum', (-0.3, -0.25), (-0.77, -1.0))
annotate('global minimum', (1.1, -0.95), (0.6, 0.8))

x = native.arange(-2.0, 2.0, 0.01)
native.plot(x, [x**3], 'x', 'f(x)')
annotate('saddle point', (0, -0.2), (-0.52, -5.0))

x, y = native.meshgrid(
    native.linspace(-1.0, 1.0, 101), native.linspace(-1.0, 1.0, 101))
z = x**2 - y**2

ax = native.plt.figure().add_subplot(111, projection='3d')
ax.plot_wireframe(x, y, z, **{'rstride': 10, 'cstride': 10})
ax.plot([0], [0], [0], 'rx')
ticks = [-1, 0, 1]
native.plt.xticks(ticks)
native.plt.yticks(ticks)
ax.set_zticks(ticks)
native.plt.xlabel('x')
native.plt.ylabel('y');

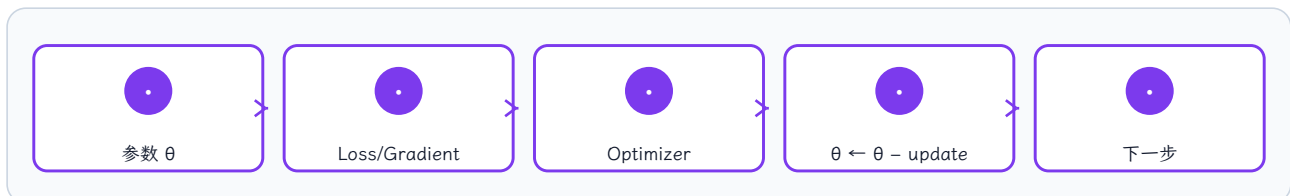
x = native.arange(-2.0, 5.0, 0.01)
native.plot(x, [native.tanh(x)], 'x', 'f(x)')
annotate('vanishing gradient', (4, 1), (2, 0.0))

```

第 11 章 · 优化算法训练流程 · chapters/chapter_11_optimization/convexity.py

章节对应: 第 11 章 类别: 训练流程 代码块: 3 个

结构图: 优化算法训练流程 (训练流程)



第 11 章内容是训练算法或学习率调度, 不对应新的网络结构。

阅读注释: 阅读顺序: 先看参数和梯度, 再看更新规则, 最后看学习率或正则化设置。

关键点: 沿图中的箭头追踪跟踪量形状, 重点定位 nn.Module、forward、损失函数和训练循环。

```

"""convexity: 优化算法"""

# 来源: 依据《动手学深度学习 (PyTorch 第二版)》的 D2L 中文 PyTorch 示例整理。
# 改写说明: 已移除 d2l 命名空间, 共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image

```

```

from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import numpy as np
from mpl_toolkits import mplot3d
import torch

f = lambda x: 0.5 * x**2 # 凸函数
g = lambda x: native.cos(np.pi * x) # 非凸函数
h = lambda x: native.exp(0.5 * x) # 凸函数

x, segment = native.arange(-2, 2, 0.01), native.tensor([-1.5, 1])
native.use_svg_display()
_, axes = native.plt.subplots(1, 3, figsize=(9, 3))
for ax, func in zip(axes, [f, g, h]):
    native.plot([x, segment], [func(x), func(segment)], axes=ax)

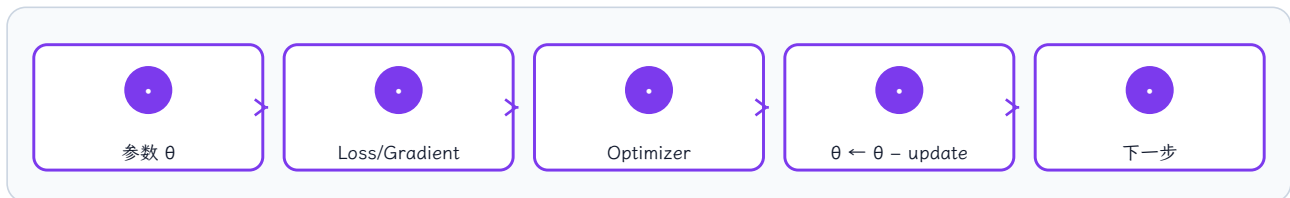
f = lambda x: (x - 1) ** 2
native.set_figsize()
native.plot([x, segment], [f(x), f(segment)], 'x', 'f(x)')

```

第 11 章 · 优化算法训练流程 · chapters/chapter_11_optimization/gd.py

章节对应：第 11 章 类别：训练流程 代码块：12 个

结构图：优化算法训练流程（训练流程）



第 11 章内容是训练算法或学习率调度，不对应新的网络结构。

阅读注释：阅读顺序：先看参数和梯度，再看更新规则，最后看学习率或正则化设置。

关键点：沿图中的箭头追踪张量形状，重点定位 nn.Module、forward、损失函数和训练循环。

```

"""gd: 优化算法"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import numpy as np
import torch

def f(x): # 目标函数
    return x ** 2

```

```

def f_grad(x): # 目标函数的梯度(导数)
    return 2 * x

def gd(eta, f_grad):
    x = 10.0
    results = [x]
    for i in range(10):
        x -= eta * f_grad(x)
        results.append(float(x))
    print(f'epoch 10, x: {x:f}')
    return results

results = gd(0.2, f_grad)

def show_trace(results, f):
    n = max(abs(min(results)), abs(max(results)))
    f_line = native.arange(-n, n, 0.01)
    native.set_figsize()
    native.plot([f_line, results], [[f(x) for x in f_line], [
        f(x) for x in results]], 'x', 'f(x)', fmts=['-', '-o'])

show_trace(results, f)

show_trace(gd(0.05, f_grad), f)

show_trace(gd(1.1, f_grad), f)

c = native.tensor(0.15 * np.pi)

def f(x): # 目标函数
    return x * native.cos(c * x)

def f_grad(x): # 目标函数的梯度
    return native.cos(c * x) - c * x * native.sin(c * x)

show_trace(gd(2, f_grad), f)

def train_2d(trainer, steps=20, f_grad=None): #@save
    """用定制的训练机优化2D目标函数"""
    # s1和s2是稍后将使用的内部状态变量
    x1, x2, s1, s2 = -5, -2, 0, 0
    results = [(x1, x2)]
    for i in range(steps):
        if f_grad:
            x1, x2, s1, s2 = trainer(x1, x2, s1, s2, f_grad)
        else:
            x1, x2, s1, s2 = trainer(x1, x2, s1, s2)
        results.append((x1, x2))
    print(f'epoch {i + 1}, x1: {float(x1):f}, x2: {float(x2):f}')
    return results

def show_trace_2d(f, results): #@save
    """显示优化过程中2D变量的轨迹"""
    native.set_figsize()
    native.plt.plot(*zip(*results), '-o', color='#ff7f0e')
    x1, x2 = native.meshgrid(native.arange(-5.5, 1.0, 0.1),
                             native.arange(-3.0, 1.0, 0.1))
    native.plt.contour(x1, x2, f(x1, x2), colors='#1f77b4')
    native.plt.xlabel('x1')
    native.plt.ylabel('x2')

def f_2d(x1, x2): # 目标函数
    return x1 ** 2 + 2 * x2 ** 2

def f_2d_grad(x1, x2): # 目标函数的梯度
    return (2 * x1, 4 * x2)

def gd_2d(x1, x2, s1, s2, f_grad):
    g1, g2 = f_grad(x1, x2)
    return (x1 - eta * g1, x2 - eta * g2, 0, 0)

eta = 0.1
show_trace_2d(f_2d, train_2d(gd_2d, f_grad=f_2d_grad))

c = native.tensor(0.5)

def f(x): # 0目标函数

```

```

        return native.cosh(c * x)

def f_grad(x): # 目标函数的梯度
    return c * native.sinh(c * x)

def f_hess(x): # 目标函数的Hessian
    return c**2 * native.cosh(c * x)

def newton(eta=1):
    x = 10.0
    results = [x]
    for i in range(10):
        x -= eta * f_grad(x) / f_hess(x)
        results.append(float(x))
    print('epoch 10, x:', x)
    return results

show_trace(newton(), f)

c = native.tensor(0.15 * np.pi)

def f(x): # 目标函数
    return x * native.cos(c * x)

def f_grad(x): # 目标函数的梯度
    return native.cos(c * x) - c * x * native.sin(c * x)

def f_hess(x): # 目标函数的Hessian
    return - 2 * c * native.sin(c * x) - x * c**2 * native.cos(c * x)

show_trace(newton(), f)

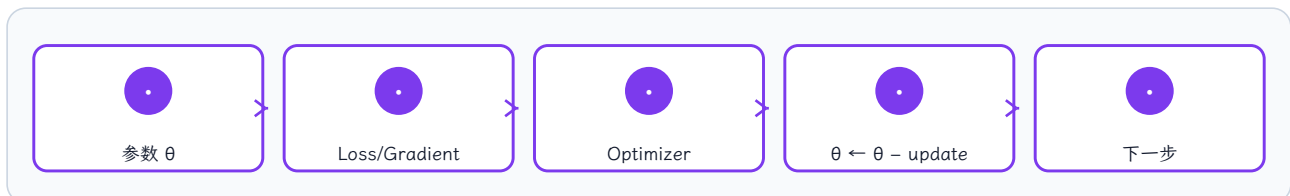
show_trace(newton(0.5), f)

```

第 11 章 · 优化算法训练流程 · chapters/chapter_11_optimization/sgd.py

章节对应：第 11 章 类别：训练流程 代码块：6 个

结构图：优化算法训练流程（训练流程）



第 11 章内容是训练算法或学习率调度，不对应新的网络结构。

阅读注释：阅读顺序：先看参数和梯度，再看更新规则，最后看学习率或正则化设置。

关键点：沿图中的箭头追踪流量形状，重点定位 `nn.Module`、`forward`、损失函数和训练循环。

```

"""sgd: 优化算法"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image

```

```

from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import math
import torch

def f(x1, x2): # 目标函数
    return x1 ** 2 + 2 * x2 ** 2

def f_grad(x1, x2): # 目标函数的梯度
    return 2 * x1, 4 * x2

def sgd(x1, x2, s1, s2, f_grad):
    g1, g2 = f_grad(x1, x2)
    # 模拟有噪声的梯度
    g1 += native.normal(0.0, 1, (1,))
    g2 += native.normal(0.0, 1, (1,))
    eta_t = eta * lr()
    return (x1 - eta_t * g1, x2 - eta_t * g2, 0, 0)

def constant_lr():
    return 1

eta = 0.1
lr = constant_lr # 常数学习速度
native.show_trace_2d(f, native.train_2d(sgd, steps=50, f_grad=f_grad))

def exponential_lr():
    # 在函数外部定义，而在内部更新的全局变量
    global t
    t += 1
    return math.exp(-0.1 * t)

t = 1
lr = exponential_lr
native.show_trace_2d(f, native.train_2d(sgd, steps=1000, f_grad=f_grad))

def polynomial_lr():
    # 在函数外部定义，而在内部更新的全局变量
    global t
    t += 1
    return (1 + 0.1 * t) ** (-0.5)

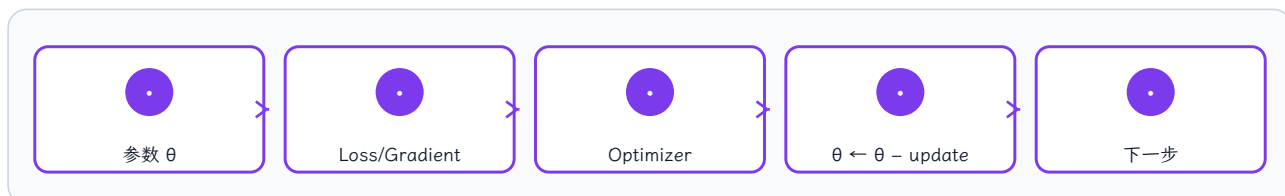
t = 1
lr = polynomial_lr
native.show_trace_2d(f, native.train_2d(sgd, steps=50, f_grad=f_grad))

```

第 11 章 · 优化算法训练流程 · chapters/chapter_11_optimization/minibatch_sgd.py

章节对应：第 11 章 类别：训练流程 代码块：15 个

结构图：优化算法训练流程（训练流程）



第 11 章内容是训练算法或学习率调度，不对应新的网络结构。

阅读注释：阅读顺序：先看参数和梯度，再看更新规则，最后看学习率或正则化设置。

关键点：沿图中的箭头追踪张量形状，重点定位 `nn.Module`、`forward`、损失函数和训练循环。

```

"""minibatch-sgd: 优化算法"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]

```

```

if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import torch
from torch import nn
import numpy as np

timer = native.Timer()
A = torch.zeros(256, 256)
B = torch.randn(256, 256)
C = torch.randn(256, 256)

# 逐元素计算A=BC
timer.start()
for i in range(256):
    for j in range(256):
        A[i, j] = torch.dot(B[i, :], C[:, j])
timer.stop()

# 逐列计算A=BC
timer.start()
for j in range(256):
    A[:, j] = torch.mv(B, C[:, j])
timer.stop()

# 一次性计算A=BC
timer.start()
A = torch.mm(B, C)
timer.stop()

# 乘法和加法作为单独的操作（在实践中融合）
gigaflops = [2/i for i in timer.times]
print(f'performance in Gigaflops: element {gigaflops[0]:.3f}, '
      f'column {gigaflops[1]:.3f}, full {gigaflops[2]:.3f}')

timer.start()
for j in range(0, 256, 64):
    A[:, j:j+64] = torch.mm(B, C[:, j:j+64])
timer.stop()
print(f'performance in Gigaflops: block {2 / timer.times[3]:.3f}')

native.DATA_HUB['airfoil'] = (native.DATA_URL + 'airfoil_self_noise.dat',
                             '76e5be1548fd8222e5074cf0faae75edff8cf93f')

def get_data_ch11(batch_size=10, n=1500):
    data = np.genfromtxt(native.download('airfoil'),
                        dtype=np.float32, delimiter='\t')
    data = torch.from_numpy((data - data.mean(axis=0)) / data.std(axis=0))
    data_iter = native.load_array((data[:n, :-1], data[:n, -1]),
                                  batch_size, is_train=True)
    return data_iter, data.shape[1]-1

def sgd(params, states, hyperparams):
    for p in params:
        p.data.sub_(hyperparams['lr'] * p.grad)
        p.grad.data.zero_()

def train_ch11(trainer_fn, states, hyperparams, data_iter,
               feature_dim, num_epochs=2):
    # 初始化模型

```

```

w = torch.normal(mean=0.0, std=0.01, size=(feature_dim, 1),
                  requires_grad=True)
b = torch.zeros(1, requires_grad=True)
net, loss = lambda X: native.linreg(X, w, b), native.squared_loss
# 训练模型
animator = native.Animator(xlabel='epoch', ylabel='loss',
                           xlim=[0, num_epochs], ylim=[0.22, 0.35])
n, timer = 0, native.Timer()
for _ in range(num_epochs):
    for X, y in data_iter:
        l = loss(net(X), y).mean()
        l.backward()
        trainer_fn([w, b], states, hyperparams)
        n += X.shape[0]
        if n % 200 == 0:
            timer.stop()
            animator.add(n/X.shape[0]/len(data_iter),
                        (native.evaluate_loss(net, data_iter, loss),))
            timer.start()
    print(f'loss: {animator.Y[0][-1]:.3f}, {timer.avg():.3f} sec/epoch')
    return timer.cumsum(), animator.Y[0]

def train_sgd(lr, batch_size, num_epochs=2):
    data_iter, feature_dim = get_data_ch11(batch_size)
    return train_ch11(
        sgd, None, {'lr': lr}, data_iter, feature_dim, num_epochs)

gd_res = train_sgd(1, 1500, 10)
sgd_res = train_sgd(0.005, 1)
mini1_res = train_sgd(.4, 100)
mini2_res = train_sgd(.05, 10)

native.set_figsize([6, 3])
native.plot(*list(map(list, zip(gd_res, sgd_res, mini1_res, mini2_res))),
            'time (sec)', 'loss', xlim=[1e-2, 10],
            legend=['gd', 'sgd', 'batch size=100', 'batch size=10'])
native.plt.gca().set_xscale('log')

def train_concise_ch11(trainer_fn, hyperparams, data_iter, num_epochs=4):
    # 初始化模型
    net = nn.Sequential(nn.Linear(5, 1))
    def init_weights(m):
        if type(m) == nn.Linear:
            torch.nn.init.normal_(m.weight, std=0.01)
    net.apply(init_weights)

    optimizer = trainer_fn(net.parameters(), **hyperparams)
    loss = nn.MSELoss(reduction='none')
    animator = native.Animator(xlabel='epoch', ylabel='loss',
                              xlim=[0, num_epochs], ylim=[0.22, 0.35])
    n, timer = 0, native.Timer()
    for _ in range(num_epochs):
        for X, y in data_iter:
            optimizer.zero_grad()
            out = net(X)
            y = y.reshape(out.shape)
            l = loss(out, y)
            l.mean().backward()
            optimizer.step()
            n += X.shape[0]
            if n % 200 == 0:
                timer.stop()
                # MSELoss计算平方误差时不带系数1/2
                animator.add(n/X.shape[0]/len(data_iter),
                            (native.evaluate_loss(net, data_iter, loss) / 2,))
                timer.start()
        print(f'loss: {animator.Y[0][-1]:.3f}, {timer.avg():.3f} sec/epoch')

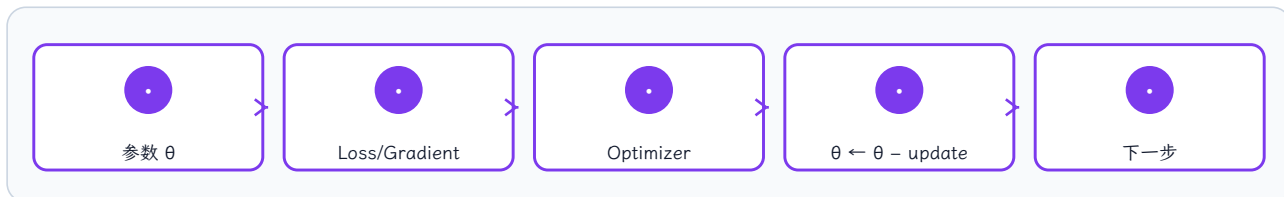
data_iter, _ = get_data_ch11(10)
trainer = torch.optim.SGD
train_concise_ch11(trainer, {'lr': 0.01}, data_iter)

```

第 11 章 · 优化算法训练流程 · chapters/chapter_11_optimization/momentum.py

章节对应：第 11 章 类别：训练流程 代码块：12 个

结构图：优化算法训练流程（训练流程）



第 11 章内容是训练算法或学习率调度，不对应新的网络结构。

阅读注释：阅读顺序：先看参数和梯度，再看更新规则，最后看学习率或正则化设置。

关键点：沿图中的箭头追踪张量形状，重点定位 `nn.Module`、`forward`、损失函数和训练循环。

"""momentum: 优化算法"""

来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。

改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。

from pathlib import Path

import sys

_ROOT = Path(__file__).resolve().parents[1]

if str(_ROOT) not in sys.path:

sys.path.insert(0, str(_ROOT))

import math

import random

import time

import numpy as np

import pandas as pd

import torch

from torch import nn

from torch.nn import functional as F

from torch.utils import data

import torchvision

from torchvision import transforms

from PIL import Image

from matplotlib import pyplot as plt

from IPython import display

import common as native

import common as native

import torch

eta = 0.4

def f_2d(x1, x2):

return 0.1 * x1 ** 2 + 2 * x2 ** 2

def gd_2d(x1, x2, s1, s2):

return (x1 - eta * 0.2 * x1, x2 - eta * 4 * x2, 0, 0)

native.show_trace_2d(f_2d, native.train_2d(gd_2d))

eta = 0.6

native.show_trace_2d(f_2d, native.train_2d(gd_2d))

def momentum_2d(x1, x2, v1, v2):

v1 = beta * v1 + 0.2 * x1

v2 = beta * v2 + 4 * x2

return x1 - eta * v1, x2 - eta * v2, v1, v2

eta, beta = 0.6, 0.5

native.show_trace_2d(f_2d, native.train_2d(momentum_2d))

eta, beta = 0.6, 0.25

native.show_trace_2d(f_2d, native.train_2d(momentum_2d))

native.set_figsize()

betas = [0.95, 0.9, 0.6, 0]

for beta in betas:

x = native.numpy(native.arange(40))

native.plt.plot(x, beta ** x, label=f'beta = {beta:.2f}')

native.plt.xlabel('time')

```

native=plt.legend();

def init_momentum_states(feature_dim):
    v_w = native.zeros((feature_dim, 1))
    v_b = native.zeros(1)
    return (v_w, v_b)

def sgd_momentum(params, states, hyperparams):
    for p, v in zip(params, states):
        with torch.no_grad():
            v[:] = hyperparams['momentum'] * v + p.grad
            p[:] -= hyperparams['lr'] * v
        p.grad.data.zero_()

def train_momentum(lr, momentum, num_epochs=2):
    native.train_ch11(sgd_momentum, init_momentum_states(feature_dim),
                     {'lr': lr, 'momentum': momentum}, data_iter,
                     feature_dim, num_epochs)

data_iter, feature_dim = native.get_data_ch11(batch_size=10)
train_momentum(0.02, 0.5)

train_momentum(0.01, 0.9)

train_momentum(0.005, 0.9)

trainer = torch.optim.SGD
native.train_concise_ch11(trainer, {'lr': 0.005, 'momentum': 0.9}, data_iter)

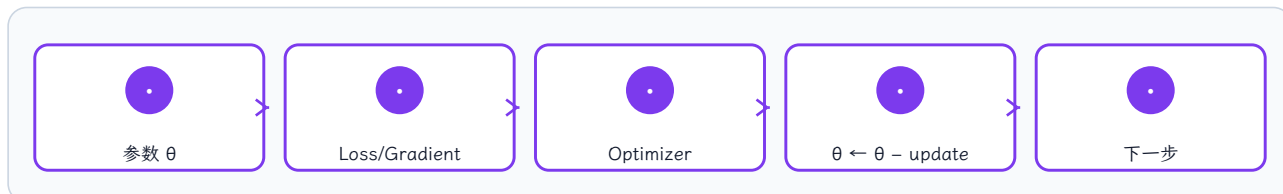
lambdas = [0.1, 1, 10, 19]
eta = 0.1
native.set_figsize((6, 4))
for lam in lambdas:
    t = native.numpy(native.arange(20))
    native=plt.plot(t, (1 - eta * lam) ** t, label=f'lambda = {lam:.2f}')
native=plt.xlabel('time')
native=plt.legend();

```

第 11 章 · 优化算法训练流程 · chapters/chapter_11_optimization/adagrad.py

章节对应：第 11 章 类别：训练流程 代码块：6 个

结构图：优化算法训练流程（训练流程）



第 11 章内容是训练算法或学习率调度，不对应新的网络结构。

阅读注释：阅读顺序：先看参数和梯度，再看更新规则，最后看学习率或正则化设置。

关键点：沿图中的箭头追踪张量形状，重点定位 nn.Module、forward、损失函数和训练循环。

```

"""adagrad: 优化算法"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F

```

```

from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import math
import torch

def adagrad_2d(x1, x2, s1, s2):
    eps = 1e-6
    g1, g2 = 0.2 * x1, 4 * x2
    s1 += g1 ** 2
    s2 += g2 ** 2
    x1 -= eta / math.sqrt(s1 + eps) * g1
    x2 -= eta / math.sqrt(s2 + eps) * g2
    return x1, x2, s1, s2

def f_2d(x1, x2):
    return 0.1 * x1 ** 2 + 2 * x2 ** 2

eta = 0.4
native.show_trace_2d(f_2d, native.train_2d(adagrad_2d))

eta = 2
native.show_trace_2d(f_2d, native.train_2d(adagrad_2d))

def init_adagrad_states(feature_dim):
    s_w = native.zeros((feature_dim, 1))
    s_b = native.zeros(1)
    return (s_w, s_b)

def adagrad(params, states, hyperparams):
    eps = 1e-6
    for p, s in zip(params, states):
        with torch.no_grad():
            s[:] += torch.square(p.grad)
            p[:] -= hyperparams['lr'] * p.grad / torch.sqrt(s + eps)
        p.grad.data.zero_()

data_iter, feature_dim = native.get_data_ch11(batch_size=10)
native.train_ch11(adagrad, init_adagrad_states(feature_dim),
                  {'lr': 0.1}, data_iter, feature_dim);

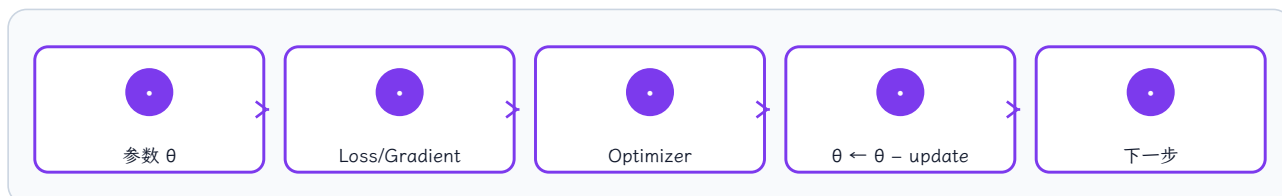
trainer = torch.optim.Adagrad
native.train_concise_ch11(trainer, {'lr': 0.1}, data_iter)

```

第 11 章 · 优化算法训练流程 · chapters/chapter_11_optimization/rmsprop.py

章节对应：第 11 章 类别：训练流程 代码块：7 个

结构图：优化算法训练流程（训练流程）



第 11 章内容是训练算法或学习率调度，不对应新的网络结构。

阅读注释：阅读顺序：先看参数和梯度，再看更新规则，最后看学习率或正则化设置。

关键点：沿图中的箭头追踪张量形状，重点定位 nn.Module、forward、损失函数和训练循环。

```

"""rmsprop: 优化算法"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys

```

```

_ROOTT = Path(__file__).resolve().parents[1]
if str(_ROOTT) not in sys.path:
    sys.path.insert(0, str(_ROOTT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import torch
import math

native.set_figsize()
gammas = [0.95, 0.9, 0.8, 0.7]
for gamma in gammas:
    x = native.numpy(native.arange(40))
    native.plt.plot(x, (1-gamma) * gamma ** x, label=f'gamma = {gamma:.2f}')
native.plt.xlabel('time');

def rmsprop_2d(x1, x2, s1, s2):
    g1, g2, eps = 0.2 * x1, 4 * x2, 1e-6
    s1 = gamma * s1 + (1 - gamma) * g1 ** 2
    s2 = gamma * s2 + (1 - gamma) * g2 ** 2
    x1 -= eta / math.sqrt(s1 + eps) * g1
    x2 -= eta / math.sqrt(s2 + eps) * g2
    return x1, x2, s1, s2

def f_2d(x1, x2):
    return 0.1 * x1 ** 2 + 2 * x2 ** 2

eta, gamma = 0.4, 0.9
native.show_trace_2d(f_2d, native.train_2d(rmsprop_2d))

def init_rmsprop_states(feature_dim):
    s_w = native.zeros((feature_dim, 1))
    s_b = native.zeros(1)
    return (s_w, s_b)

def rmsprop(params, states, hyperparams):
    gamma, eps = hyperparams['gamma'], 1e-6
    for p, s in zip(params, states):
        with torch.no_grad():
            s[:] = gamma * s + (1 - gamma) * torch.square(p.grad)
            p[:] -= hyperparams['lr'] * p.grad / torch.sqrt(s + eps)
            p.grad.data.zero_()

data_iter, feature_dim = native.get_data_ch11(batch_size=10)
native.train_ch11(rmsprop, init_rmsprop_states(feature_dim),
                  {'lr': 0.01, 'gamma': 0.9}, data_iter, feature_dim);

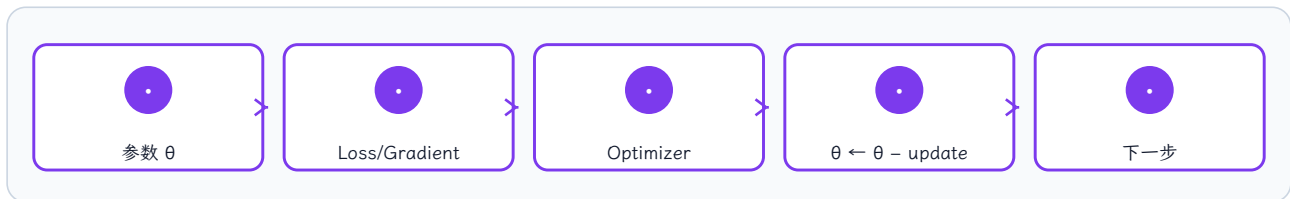
trainer = torch.optim.RMSprop
native.train_concise_ch11(trainer, {'lr': 0.01, 'alpha': 0.9},
                          data_iter)

```

第 11 章 · 优化算法训练流程 · chapters/chapter_11_optimization/adadelta.py

章节对应：第 11 章 类别：训练流程 代码块：3 个

结构图：优化算法训练流程（训练流程）



第 11 章内容是训练算法或学习率调度，不对应新的网络结构。

阅读注释：阅读顺序：先看参数和梯度，再看更新规则，最后看学习率或正则化设置。

关键点：沿图中的箭头追踪张量形状，重点定位 `nn.Module`、`forward`、损失函数和训练循环。

"""adadelta: 优化算法"""

来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。

改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。

from pathlib import Path

import sys

_ROOT = Path(__file__).resolve().parents[1]

if str(_ROOT) not in sys.path:

sys.path.insert(0, str(_ROOT))

import math

import random

import time

import numpy as np

import pandas as pd

import torch

from torch import nn

from torch.nn import functional as F

from torch.utils import data

import torchvision

from torchvision import transforms

from PIL import Image

from matplotlib import pyplot as plt

from IPython import display

import common as native

import common as native

import torch

def init_adadelta_states(feature_dim):

s_w, s_b = native.zeros((feature_dim, 1)), native.zeros(1)

delta_w, delta_b = native.zeros((feature_dim, 1)), native.zeros(1)

return ((s_w, delta_w), (s_b, delta_b))

def adadelta(params, states, hyperparams):

rho, eps = hyperparams['rho'], 1e-5

for p, (s, delta) in zip(params, states):

with torch.no_grad():

In-place updates via [:]

s[:] = rho * s + (1 - rho) * torch.square(p.grad)

g = (torch.sqrt(delta + eps) / torch.sqrt(s + eps)) * p.grad

p[:] -= g

delta[:] = rho * delta + (1 - rho) * g * g

p.grad.data.zero_()

data_iter, feature_dim = native.get_data_ch11(batch_size=10)

native.train_ch11(adadelta, init_adadelta_states(feature_dim),

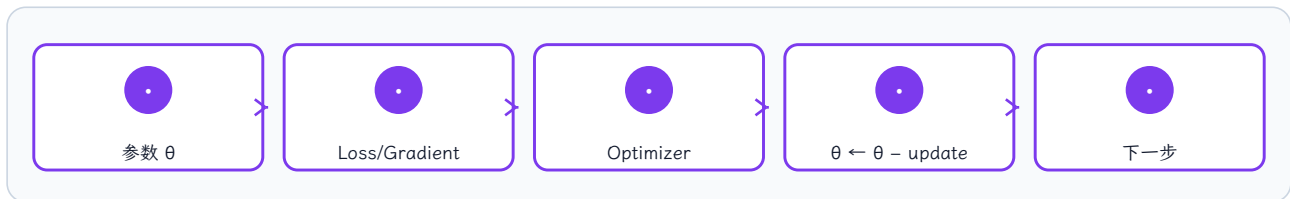
{'rho': 0.9}, data_iter, feature_dim);

trainer = torch.optim.Adadelta

native.train_concise_ch11(trainer, {'rho': 0.9}, data_iter)

章节对应：第 11 章 类别：训练流程 代码块：4 个

结构图：优化算法训练流程（训练流程）



第 11 章内容是训练算法或学习率调度，不对应新的网络结构。

阅读注释：阅读顺序：先看参数和梯度，再看更新规则，最后看学习率或正则化设置。

关键点：沿图中的箭头追踪张量形状，重点定位 nn.Module、forward、损失函数和训练循环。

"""adam: 优化算法"""

来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。

改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。

from pathlib import Path

import sys

_ROOT = Path(__file__).resolve().parents[1]

if str(_ROOT) not in sys.path:

sys.path.insert(0, str(_ROOT))

import math

import random

import time

import numpy as np

import pandas as pd

import torch

from torch import nn

from torch.nn import functional as F

from torch.utils import data

import torchvision

from torchvision import transforms

from PIL import Image

from matplotlib import pyplot as plt

from IPython import display

import common as native

import common as native

import torch

def init_adam_states(feature_dim):

v_w, v_b = native.zeros((feature_dim, 1)), native.zeros(1)

s_w, s_b = native.zeros((feature_dim, 1)), native.zeros(1)

return ((v_w, s_w), (v_b, s_b))

def adam(params, states, hyperparams):

beta1, beta2, eps = 0.9, 0.999, 1e-6

for p, (v, s) in zip(params, states):

with torch.no_grad():

v[:] = beta1 * v + (1 - beta1) * p.grad

s[:] = beta2 * s + (1 - beta2) * torch.square(p.grad)

v_bias_corr = v / (1 - beta1 ** hyperparams['t'])

s_bias_corr = s / (1 - beta2 ** hyperparams['t'])

p[:] -= hyperparams['lr'] * v_bias_corr / (torch.sqrt(s_bias_corr) + eps)

p.grad.data.zero_()

hyperparams['t'] += 1

data_iter, feature_dim = native.get_data_ch11(batch_size=10)

native.train_ch11(adam, init_adam_states(feature_dim),

{'lr': 0.01, 't': 1}, data_iter, feature_dim);

trainer = torch.optim.Adam

native.train_concise_ch11(trainer, {'lr': 0.01}, data_iter)

def yogi(params, states, hyperparams):

beta1, beta2, eps = 0.9, 0.999, 1e-3

for p, (v, s) in zip(params, states):

```

with torch.no_grad():
    v[:] = beta1 * v + (1 - beta1) * p.grad
    s[:] = s + (1 - beta2) * torch.sign(
        torch.square(p.grad) - s) * torch.square(p.grad)
    v_bias_corr = v / (1 - beta1 ** hyperparams['t'])
    s_bias_corr = s / (1 - beta2 ** hyperparams['t'])
    p[:] -= hyperparams['lr'] * v_bias_corr / (torch.sqrt(s_bias_corr)
                                              + eps)

p.grad.data.zero_()
hyperparams['t'] += 1

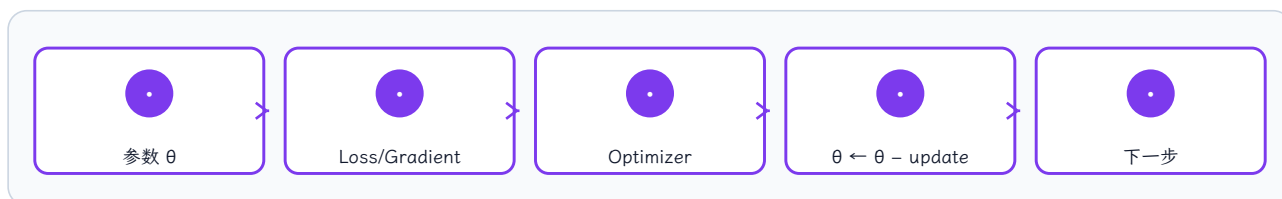
data_iter, feature_dim = native.get_data_ch11(batch_size=10)
native.train_ch11(yogi, init_adam_states(feature_dim),
                  {'lr': 0.01, 't': 1}, data_iter, feature_dim);

```

第 11 章 · 优化算法训练流程 · chapters/chapter_11_optimization/lr_scheduler.py

章节对应：第 11 章 类别：训练流程 代码块：13 个

结构图：优化算法训练流程（训练流程）



第 11 章内容是训练算法或学习率调度，不对应新的网络结构。

阅读注释：阅读顺序：先看参数和梯度，再看更新规则，最后看学习率或正则化设置。

关键点：沿图中的箭头追踪张量形状，重点定位 nn.Module、forward、损失函数和训练循环。

"""lr-scheduler: 优化算法"""

来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。

改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。

```

from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

```

```

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

```

```

import common as native
import math
import torch
from torch import nn
from torch.optim import lr_scheduler

```

```

def net_fn():
    model = nn.Sequential(
        nn.Conv2d(1, 6, kernel_size=5, padding=2), nn.ReLU(),
        nn.MaxPool2d(kernel_size=2, stride=2),
        nn.Conv2d(6, 16, kernel_size=5), nn.ReLU(),
        nn.MaxPool2d(kernel_size=2, stride=2),
        nn.Flatten(),
        nn.Linear(16 * 5 * 5, 120), nn.ReLU(),
        nn.Linear(120, 84), nn.ReLU(),

```

```

        nn.Linear(84, 10))

    return model

loss = nn.CrossEntropyLoss()
device = native.try_gpu()

batch_size = 256
train_iter, test_iter = native.load_data_fashion_mnist(batch_size=batch_size)

# 代码几乎与native.train_ch6定义在卷积神经网络一章LeNet一节中的相同
def train(net, train_iter, test_iter, num_epochs, loss, trainer, device,
          scheduler=None):
    net.to(device)
    animator = native.Animator(xlabel='epoch', xlim=[0, num_epochs],
                               legend=['train loss', 'train acc', 'test acc'])

    for epoch in range(num_epochs):
        metric = native.Accumulator(3) # train loss, train acc, num examples
        for i, (X, y) in enumerate(train_iter):
            net.train()
            trainer.zero_grad()
            X, y = X.to(device), y.to(device)
            y_hat = net(X)
            l = loss(y_hat, y)
            l.backward()
            trainer.step()
            with torch.no_grad():
                metric.add(l * X.shape[0], native.accuracy(y_hat, y), X.shape[0])
            train_loss = metric[0] / metric[2]
            train_acc = metric[1] / metric[2]
            if (i + 1) % 50 == 0:
                animator.add(epoch + i / len(train_iter),
                             (train_loss, train_acc, None))

        test_acc = native.evaluate_accuracy_gpu(net, test_iter)
        animator.add(epoch+1, (None, None, test_acc))

        if scheduler:
            if scheduler.__module__ == lr_scheduler.__name__:
                # UsingPyTorchIn-Built scheduler
                scheduler.step()
            else:
                # Using custom defined scheduler
                for param_group in trainer.param_groups:
                    param_group['lr'] = scheduler(epoch)

        print(f'train loss {train_loss:.3f}, train acc {train_acc:.3f}, '
              f'test acc {test_acc:.3f}')

    lr, num_epochs = 0.3, 30
    net = net_fn()
    trainer = torch.optim.SGD(net.parameters(), lr=lr)
    train(net, train_iter, test_iter, num_epochs, loss, trainer, device)

    lr = 0.1
    trainer.param_groups[0]['lr'] = lr
    print(f'learning rate is now {trainer.param_groups[0]["lr"]:.2f}')

    class SquareRootScheduler:
        def __init__(self, lr=0.1):
            self.lr = lr

        def __call__(self, num_update):
            return self.lr * pow(num_update + 1.0, -0.5)

    scheduler = SquareRootScheduler(lr=0.1)
    native.plot(native.arange(num_epochs), [scheduler(t) for t in range(num_epochs)])

    net = net_fn()
    trainer = torch.optim.SGD(net.parameters(), lr)
    train(net, train_iter, test_iter, num_epochs, loss, trainer, device,
          scheduler)

    class FactorScheduler:
        def __init__(self, factor=1, stop_factor_lr=1e-7, base_lr=0.1):
            self.factor = factor

```

```

self.stop_factor_lr = stop_factor_lr
self.base_lr = base_lr

def __call__(self, num_update):
    self.base_lr = max(self.stop_factor_lr, self.base_lr * self.factor)
    return self.base_lr

scheduler = FactorScheduler(factor=0.9, stop_factor_lr=1e-2, base_lr=2.0)
native.plot(native.arange(50), [scheduler(t) for t in range(50)])

net = net_fn()
trainer = torch.optim.SGD(net.parameters(), lr=0.5)
scheduler = lr_scheduler.MultiStepLR(trainer, milestones=[15, 30], gamma=0.5)

def get_lr(trainer, scheduler):
    lr = scheduler.get_last_lr()[0]
    trainer.step()
    scheduler.step()
    return lr

native.plot(native.arange(num_epochs), [get_lr(trainer, scheduler)
                                         for t in range(num_epochs)])

train(net, train_iter, test_iter, num_epochs, loss, trainer, device,
      scheduler)

class CosineScheduler:
    def __init__(self, max_update, base_lr=0.01, final_lr=0,
                 warmup_steps=0, warmup_begin_lr=0):
        self.base_lr_orig = base_lr
        self.max_update = max_update
        self.final_lr = final_lr
        self.warmup_steps = warmup_steps
        self.warmup_begin_lr = warmup_begin_lr
        self.max_steps = self.max_update - self.warmup_steps

    def get_warmup_lr(self, epoch):
        increase = (self.base_lr_orig - self.warmup_begin_lr) \
            * float(epoch) / float(self.warmup_steps)
        return self.warmup_begin_lr + increase

    def __call__(self, epoch):
        if epoch < self.warmup_steps:
            return self.get_warmup_lr(epoch)
        if epoch <= self.max_update:
            self.base_lr = self.final_lr + (
                self.base_lr_orig - self.final_lr) * (1 + math.cos(
                    math.pi * (epoch - self.warmup_steps) / self.max_steps)) / 2
            return self.base_lr

scheduler = CosineScheduler(max_update=20, base_lr=0.3, final_lr=0.01)
native.plot(native.arange(num_epochs), [scheduler(t) for t in range(num_epochs)])

net = net_fn()
trainer = torch.optim.SGD(net.parameters(), lr=0.3)
train(net, train_iter, test_iter, num_epochs, loss, trainer, device,
      scheduler)

scheduler = CosineScheduler(20, warmup_steps=5, base_lr=0.3, final_lr=0.01)
native.plot(native.arange(num_epochs), [scheduler(t) for t in range(num_epochs)])

net = net_fn()
trainer = torch.optim.SGD(net.parameters(), lr=0.3)
train(net, train_iter, test_iter, num_epochs, loss, trainer, device,
      scheduler)

```

第 12 章：计算性能

本章分类：计算性能 章节说明：异步计算、计算图、自动并行和多 GPU；这里标注为性能流程。

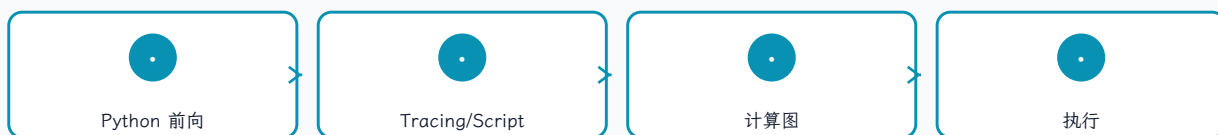
本章网络/流程总览



第 12 章 · 计算图/执行流程 · [chapters/chapter_12_computational_performance/hybridize.py](#)

章节对应：第 12 章 类别：性能流程 代码块：7 个

结构图：计算图/执行流程（性能流程）



第 12 章性能优化流程，不对应新的网络。

阅读注释：阅读顺序：先看数据/模型所在设备，再看执行或保存边界，最后看输出结果。

关键点：沿图中的箭头追踪张量形状，重点定位 `nn.Module`、`forward`、损失函数和训练循环。

"""hybridize: 计算性能"""

来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。

改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。

```
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))
```

```
import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native
```

```
def add(a, b):
    return a + b
```

```
def fancy_func(a, b, c, d):
    e = add(a, b)
    f = add(c, d)
    g = add(e, f)
    return g
```

```
print(fancy_func(1, 2, 3, 4))
```

```
def add_():
    return ''
def add(a, b):
```

```

    return a + b
...

def fancy_func_():
    return ''
def fancy_func(a, b, c, d):
    e = add(a, b)
    f = add(c, d)
    g = add(e, f)
    return g
...

def evoke_():
    return add_() + fancy_func_() + 'print(fancy_func(1, 2, 3, 4))'

prog = evoke_()
print(prog)
y = compile(prog, '', 'exec')
exec(y)

```

```

import common as native
import torch
from torch import nn

# 生产网络的工厂模式
def get_net():
    net = nn.Sequential(nn.Linear(512, 256),
                        nn.ReLU(),
                        nn.Linear(256, 128),
                        nn.ReLU(),
                        nn.Linear(128, 2))
    return net

x = torch.randn(size=(1, 512))
net = get_net()
net(x)

net = torch.jit.script(net)
net(x)

class Benchmark:
    """用于测量运行时间"""
    def __init__(self, description='Done'):
        self.description = description

    def __enter__(self):
        self.timer = native.Timer()
        return self

    def __exit__(self, *args):
        print(f'{self.description}: {self.timer.stop():.4f} sec')

net = get_net()
with Benchmark('无torchscript'):
    for i in range(1000): net(x)

net = torch.jit.script(net)
with Benchmark('有torchscript'):
    for i in range(1000): net(x)

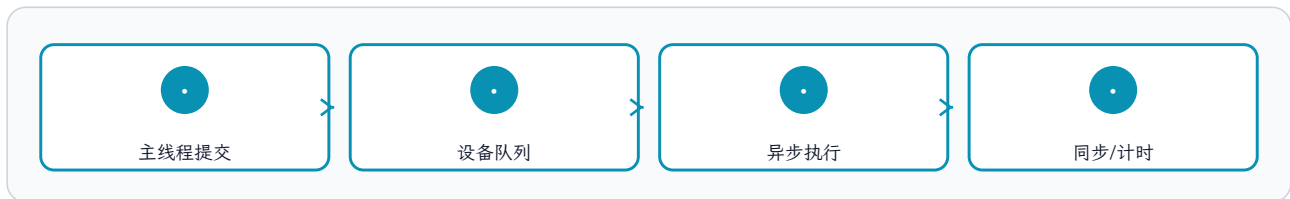
net.save('my_mlp')
# In a terminal, `ls -lh my_mlp*` shows the generated TorchScript files.

```

第 12 章 · 异步计算流程 · chapters/chapter_12_computational_performance/async_computation.py

章节对应：第 12 章 类别：性能流程 代码块：4 个

结构图：异步计算流程（性能流程）



第 12 章异步执行流程。

阅读注释：阅读顺序：先看数据/模型所在设备，再看执行或保存边界，最后看输出结果。

关键点：沿图中的箭头追踪张量形状，重点定位 `nn.Module`、`forward`、损失函数和训练循环。

"""async-computation: 计算性能"""

来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。

改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。

```
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import numpy, os, subprocess
import torch
from torch import nn

# GPU计算热身
device = native.try_gpu()
a = torch.randn(size=(1000, 1000), device=device)
b = torch.mm(a, a)

with native.Benchmark('numpy'):
    for _ in range(10):
        a = numpy.random.normal(size=(1000, 1000))
        b = numpy.dot(a, a)

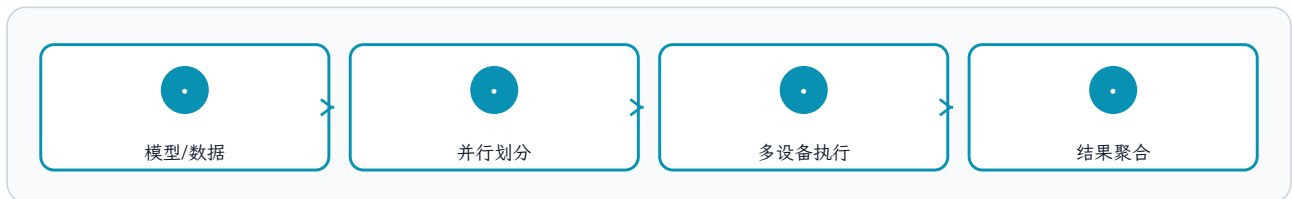
with native.Benchmark('torch'):
    for _ in range(10):
        a = torch.randn(size=(1000, 1000), device=device)
        b = torch.mm(a, a)

with native.Benchmark():
    for _ in range(10):
        a = torch.randn(size=(1000, 1000), device=device)
        b = torch.mm(a, a)
        torch.cuda.synchronize(device)

x = torch.ones((1, 2), device=device)
y = torch.ones((1, 2), device=device)
z = x * y + 2
z
```

章节对应：第 12 章 类别：性能流程 代码块：6 个

结构图：自动并行流程（性能流程）



第 12 章并行执行流程。

阅读注释：阅读顺序：先看数据/模型所在设备，再看执行或保存边界，最后看输出结果。

关键点：沿图中的箭头追踪张量形状，重点定位 `nn.Module`、`forward`、损失函数和训练循环。

```

"""auto-parallelism: 计算性能"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import torch

devices = native.try_all_gpus()
def run(x):
    return [x.mm(x) for _ in range(50)]

x_gpu1 = torch.rand(size=(4000, 4000), device=devices[0])
x_gpu2 = torch.rand(size=(4000, 4000), device=devices[1])

run(x_gpu1)
run(x_gpu2) # 预热设备
torch.cuda.synchronize(devices[0])
torch.cuda.synchronize(devices[1])

with native.Benchmark('GPU1 time'):
    run(x_gpu1)
    torch.cuda.synchronize(devices[0])

with native.Benchmark('GPU2 time'):
    run(x_gpu2)
    torch.cuda.synchronize(devices[1])

with native.Benchmark('GPU1 & GPU2'):
    run(x_gpu1)
    run(x_gpu2)
    torch.cuda.synchronize()

def copy_to_cpu(x, non_blocking=False):
    return [y.to('cpu', non_blocking=non_blocking) for y in x]
  
```

```
with native.Benchmark('在GPU1上运行'):
    y = run(x_gpu1)
    torch.cuda.synchronize()
```

```
with native.Benchmark('复制到CPU'):
    y_cpu = copy_to_cpu(y)
    torch.cuda.synchronize()
```

```
with native.Benchmark('在GPU1上运行并复制到CPU'):
    y = run(x_gpu1)
    y_cpu = copy_to_cpu(y, True)
    torch.cuda.synchronize()
```

第 12 章 · 多 GPU 数据并行 · chapters/chapter_12_computational_performance/multiple_gpus.py

章节对应：第 12 章 类别：性能流程 代码块：12 个

结构图：多 GPU 数据并行（性能流程）



第 12 章多 GPU 训练流程，不对应新的网络。

阅读注释：阅读顺序：先看数据/模型所在设备，再看执行或保存边界，最后看输出结果。

关键点：沿图中的箭头追踪张量形状，重点定位 `nn.Module`、`forward`、损失函数和训练循环。

```
"""multiple-gpus: 计算性能"""
```

```
# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。  
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
```

```
from pathlib import Path  
import sys  
_ROOT = Path(__file__).resolve().parents[1]  
if str(_ROOT) not in sys.path:  
    sys.path.insert(0, str(_ROOT))
```

```
import math  
import random  
import time  
import numpy as np  
import pandas as pd  
import torch  
from torch import nn  
from torch.nn import functional as F  
from torch.utils import data  
import torchvision  
from torchvision import transforms  
from PIL import Image  
from matplotlib import pyplot as plt  
from IPython import display  
import common as native
```

```
import common as native  
import torch  
from torch import nn  
from torch.nn import functional as F
```

```
# 初始化模型参数  
scale = 0.01  
W1 = torch.randn(size=(20, 1, 3, 3)) * scale  
b1 = torch.zeros(20)  
W2 = torch.randn(size=(50, 20, 5, 5)) * scale  
b2 = torch.zeros(50)  
W3 = torch.randn(size=(800, 128)) * scale  
b3 = torch.zeros(128)  
W4 = torch.randn(size=(128, 10)) * scale  
b4 = torch.zeros(10)  
params = [W1, b1, W2, b2, W3, b3, W4, b4]
```

```

# 定义模型
def lenet(X, params):
    h1_conv = F.conv2d(input=X, weight=params[0], bias=params[1])
    h1_activation = F.relu(h1_conv)
    h1 = F.avg_pool2d(input=h1_activation, kernel_size=(2, 2), stride=(2, 2))
    h2_conv = F.conv2d(input=h1, weight=params[2], bias=params[3])
    h2_activation = F.relu(h2_conv)
    h2 = F.avg_pool2d(input=h2_activation, kernel_size=(2, 2), stride=(2, 2))
    h2 = h2.reshape(h2.shape[0], -1)
    h3_linear = torch.mm(h2, params[4]) + params[5]
    h3 = F.relu(h3_linear)
    y_hat = torch.mm(h3, params[6]) + params[7]
    return y_hat

# 交叉熵损失函数
loss = nn.CrossEntropyLoss(reduction='none')

def get_params(params, device):
    new_params = [p.to(device) for p in params]
    for p in new_params:
        p.requires_grad_()
    return new_params

new_params = get_params(params, native.try_gpu(0))
print('b1 权重:', new_params[1])
print('b1 梯度:', new_params[1].grad)

def allreduce(data):
    for i in range(1, len(data)):
        data[0][:] += data[i].to(data[0].device)
    for i in range(1, len(data)):
        data[i][:] = data[0].to(data[i].device)

data = [torch.ones((1, 2), device=native.try_gpu(i)) * (i + 1) for i in range(2)]
print('allreduce之前: \n', data[0], '\n', data[1])
allreduce(data)
print('allreduce之后: \n', data[0], '\n', data[1])

data = torch.arange(20).reshape(4, 5)
devices = [torch.device('cuda:0'), torch.device('cuda:1')]
split = nn.parallel.scatter(data, devices)
print('input :', data)
print('load into', devices)
print('output:', split)

def split_batch(X, y, devices):
    """将X和y拆分到多个设备上"""
    assert X.shape[0] == y.shape[0]
    return (nn.parallel.scatter(X, devices),
            nn.parallel.scatter(y, devices))

def train_batch(X, y, device_params, devices, lr):
    X_shards, y_shards = split_batch(X, y, devices)
    # 在每个GPU上分别计算损失
    ls = [loss(lenet(X_shard, device_W), y_shard).sum()
           for X_shard, y_shard, device_W in zip(
               X_shards, y_shards, device_params)]
    for l in ls: # 反向传播在每个GPU上分别执行
        l.backward()
    # 将每个GPU的所有梯度相加, 并将其广播到所有GPU
    with torch.no_grad():
        for i in range(len(device_params[0])):
            allreduce(
                [device_params[c][i].grad for c in range(len(devices))])
    # 在每个GPU上分别更新模型参数
    for param in device_params:
        native.sgd(param, lr, X.shape[0]) # 在这里, 我们使用全尺寸的小批量

def train(num_gpus, batch_size, lr):
    train_iter, test_iter = native.load_data_fashion_mnist(batch_size)
    devices = [native.try_gpu(i) for i in range(num_gpus)]
    # 将模型参数复制到num_gpus个GPU
    device_params = [get_params(params, d) for d in devices]
    num_epochs = 10
    animator = native.Animator('epoch', 'test acc', xlim=[1, num_epochs])
    timer = native.Timer()
    for epoch in range(num_epochs):

```

```

        timer.start()
        for X, y in train_iter:
            # 为单个小批量执行多GPU训练
            train_batch(X, y, device_params, devices, lr)
            torch.cuda.synchronize()
        timer.stop()
    # 在GPU0上评估模型
    animator.add(epoch + 1, (native.evaluate_accuracy_gpu(
        lambda x: lenet(x, device_params[0]), test_iter, devices[0]),))
print(f'测试精度: {animator.Y[0][-1]:.2f}, {timer.avg():.1f}秒/轮, '
      f'在{str(devices)}')

train(num_gpus=1, batch_size=256, lr=0.2)

train(num_gpus=2, batch_size=256, lr=0.2)

```

第 12 章 · 多 GPU 数据并行 ·

chapters/chapter_12_computational_performance/multiple_gpus_concise.py

章节对应: 第 12 章 类别: 性能流程 代码块: 6 个

结构图: 多 GPU 数据并行 (性能流程)



第 12 章多 GPU 训练流程, 不对应新的网络。

阅读注释: 阅读顺序: 先看数据/模型所在设备, 再看执行或保存边界, 最后看输出结果。

关键点: 沿图中的箭头追踪张量形状, 重点定位 nn.Module、forward、损失函数和训练循环。

```

"""multiple-gpus-concise: 计算性能"""

# 来源: 依据《动手学深度学习 (PyTorch 第二版)》的 D2L 中文 PyTorch 示例整理。
# 改写说明: 已移除 d2l 命名空间, 共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import torch
from torch import nn

def resnet18(num_classes, in_channels=1):
    """稍加修改的ResNet-18模型"""
    def resnet_block(in_channels, out_channels, num_residuals,
                     first_block=False):
        blk = []
        for i in range(num_residuals):
            if i == 0 and not first_block:
                blk.append(native.Residual(in_channels, out_channels,
                                           use_1x1conv=True, strides=2))

```

```

        else:
            blk.append(native.Residual(out_channels, out_channels))
        return nn.Sequential(*blk)

# 该模型使用了更小的卷积核、步长和填充, 而且删除了最大汇聚层
net = nn.Sequential(
    nn.Conv2d(in_channels, 64, kernel_size=3, stride=1, padding=1),
    nn.BatchNorm2d(64),
    nn.ReLU())
net.add_module("resnet_block1", resnet_block(
    64, 64, 2, first_block=True))
net.add_module("resnet_block2", resnet_block(64, 128, 2))
net.add_module("resnet_block3", resnet_block(128, 256, 2))
net.add_module("resnet_block4", resnet_block(256, 512, 2))
net.add_module("global_avg_pool", nn.AdaptiveAvgPool2d((1,1)))
net.add_module("fc", nn.Sequential(nn.Flatten(),
                                    nn.Linear(512, num_classes)))

return net

net = resnet18(10)
# 获取GPU列表
devices = native.try_all_gpus()
# 我们将在训练代码实现中初始化网络

def train(net, num_gpus, batch_size, lr):
    train_iter, test_iter = native.load_data_fashion_mnist(batch_size)
    devices = [native.try_gpu(i) for i in range(num_gpus)]
    def init_weights(m):
        if type(m) in [nn.Linear, nn.Conv2d]:
            nn.init.normal_(m.weight, std=0.01)
    net.apply(init_weights)
# 在多个GPU上设置模型
net = nn.DataParallel(net, device_ids=devices)
trainer = torch.optim.SGD(net.parameters(), lr)
loss = nn.CrossEntropyLoss()
timer, num_epochs = native.Timer(), 10
animator = native.Animator('epoch', 'test acc', xlim=[1, num_epochs])
for epoch in range(num_epochs):
    net.train()
    timer.start()
    for X, y in train_iter:
        trainer.zero_grad()
        X, y = X.to(devices[0]), y.to(devices[0])
        l = loss(net(X), y)
        l.backward()
        trainer.step()
    timer.stop()
    animator.add(epoch + 1, (native.evaluate_accuracy_gpu(net, test_iter),))
print(f'测试精度: {animator.Y[0][-1]:.2f}, {timer.avg():.1f}秒/轮, '
      f'在{str(devices)}')

train(net, num_gpus=1, batch_size=256, lr=0.1)
train(net, num_gpus=2, batch_size=512, lr=0.2)

```

第 13 章：计算机视觉

本章分类：计算机视觉 章节说明：数据增强、迁移学习、目标检测、语义分割、风格迁移和实战。

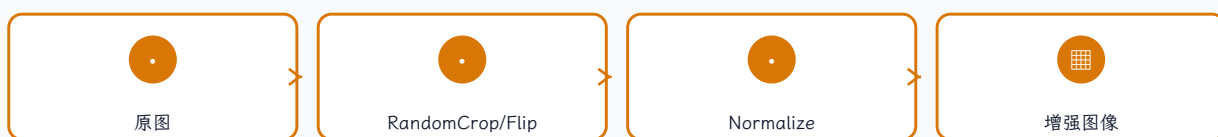
本章网络/流程总览



第 13 章 · 图像增强管线 · `chapters/chapter_13_computer_vision/image_augmentation.py`

章节对应：第 13 章 类别：数据流程 代码块：17 个

结构图：图像增强管线（数据流程）



第 13 章视觉数据预处理。

阅读注释：阅读顺序：先看样本格式，再看预处理与 Dataset，最后看 DataLoader 生成的批次。
关键点：沿图中的箭头追踪张量形状，重点定位 `nn.Module`、`forward`、损失函数和训练循环。

```
"""image-augmentation: 计算机视觉"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import torch
import torchvision
from torch import nn

native.set_figsize()
img = native.Image.open('../img/cat1.jpg')
native.plt.imshow(img);

def apply(img, aug, num_rows=2, num_cols=4, scale=1.5):
    Y = [aug(img) for _ in range(num_rows * num_cols)]
    native.show_images(Y, num_rows, num_cols, scale=scale)

apply(img, torchvision.transforms.RandomHorizontalFlip())
```

```

apply(img, torchvision.transforms.RandomVerticalFlip())

shape_aug = torchvision.transforms.RandomResizedCrop(
    (200, 200), scale=(0.1, 1), ratio=(0.5, 2))
apply(img, shape_aug)

apply(img, torchvision.transforms.ColorJitter(
    brightness=0.5, contrast=0, saturation=0, hue=0))

apply(img, torchvision.transforms.ColorJitter(
    brightness=0, contrast=0, saturation=0, hue=0.5))

color_aug = torchvision.transforms.ColorJitter(
    brightness=0.5, contrast=0.5, saturation=0.5, hue=0.5)
apply(img, color_aug)

augs = torchvision.transforms.Compose([
    torchvision.transforms.RandomHorizontalFlip(), color_aug, shape_aug])
apply(img, augs)

all_images = torchvision.datasets.CIFAR10(train=True, root="../data",
                                         download=True)
native.show_images([all_images[i][0] for i in range(32)], 4, 8, scale=0.8);

train_augs = torchvision.transforms.Compose([
    torchvision.transforms.RandomHorizontalFlip(),
    torchvision.transforms.ToTensor()])

test_augs = torchvision.transforms.Compose([
    torchvision.transforms.ToTensor()])

def load_cifar10(is_train, augs, batch_size):
    dataset = torchvision.datasets.CIFAR10(root="../data", train=is_train,
                                           transform=augs, download=True)
    dataloader = torch.utils.data.DataLoader(dataset, batch_size=batch_size,
                                             shuffle=is_train, num_workers=native.get_dataloader_workers())
    return dataloader

def train_batch_ch13(net, X, y, loss, trainer, devices):
    """用多GPU进行小批量训练"""
    if isinstance(X, list):
        # 微调BERT中所需
        X = [x.to(devices[0]) for x in X]
    else:
        X = X.to(devices[0])
    y = y.to(devices[0])
    net.train()
    trainer.zero_grad()
    pred = net(X)
    l = loss(pred, y)
    l.sum().backward()
    trainer.step()
    train_loss_sum = l.sum()
    train_acc_sum = native.accuracy(pred, y)
    return train_loss_sum, train_acc_sum

def train_ch13(net, train_iter, test_iter, loss, trainer, num_epochs,
               devices=native.try_all_gpus()):
    """用多GPU进行模型训练"""
    timer, num_batches = native.Timer(), len(train_iter)
    animator = native.Animator(xlabel='epoch', xlim=[1, num_epochs], ylim=[0, 1],
                              legend=['train loss', 'train acc', 'test acc'])
    net = nn.DataParallel(net, device_ids=devices).to(devices[0])
    for epoch in range(num_epochs):
        # 4个维度: 储存训练损失, 训练准确度, 实例数, 特点数
        metric = native.Accumulator(4)
        for i, (features, labels) in enumerate(train_iter):
            timer.start()
            l, acc = train_batch_ch13(
                net, features, labels, loss, trainer, devices)
            metric.add(l, acc, labels.shape[0], labels.numel())
            timer.stop()
            if (i + 1) % (num_batches // 5) == 0 or i == num_batches - 1:
                animator.add(epoch + (i + 1) / num_batches,
                             (metric[0] / metric[2], metric[1] / metric[3],
                              None))

```

```

        test_acc = native.evaluate_accuracy_gpu(net, test_iter)
        animator.add(epoch + 1, (None, None, test_acc))
    print(f'loss {metric[0] / metric[2]:.3f}, train acc '
          f'{metric[1] / metric[3]:.3f}, test acc {test_acc:.3f}')
    print(f'{metric[2] * num_epochs / timer.sum():.1f} examples/sec on '
          f'{str(devices)}')

batch_size, devices, net = 256, native.try_all_gpus(), native.resnet18(10, 3)

def init_weights(m):
    if type(m) in [nn.Linear, nn.Conv2d]:
        nn.init.xavier_uniform_(m.weight)

net.apply(init_weights)

def train_with_data_aug(train_augs, test_augs, net, lr=0.001):
    train_iter = load_cifar10(True, train_augs, batch_size)
    test_iter = load_cifar10(False, test_augs, batch_size)
    loss = nn.CrossEntropyLoss(reduction="none")
    trainer = torch.optim.Adam(net.parameters(), lr=lr)
    train_ch13(net, train_iter, test_iter, loss, trainer, 10, devices)

train_with_data_aug(train_augs, test_augs, net)

```

第 13 章 · 预训练模型微调 · chapters/chapter_13_computer_vision/fine_tuning.py

章节对应：第 13 章 类别：网络结构 代码块：13 个

结构图：预训练模型微调（网络结构）



第 13 章迁移学习/微调结构。

阅读注释：阅读顺序：先看输入与输出，再看核心层/模块，最后看损失函数和训练循环。

关键点：沿图中的箭头追踪张量形状，重点定位 nn.Module、forward、损失函数和训练循环。

```

"""fine-tuning: 计算机视觉"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
from torch import nn
import torch
import torchvision
import os

```

```

native.DATA_HUB['hotdog'] = (native.DATA_URL + 'hotdog.zip',
                             'fba480ffa8aa7e0febbb511d181409f899b9baa5')

data_dir = native.download_extract('hotdog')

train_imgs = torchvision.datasets.ImageFolder(os.path.join(data_dir, 'train'))
test_imgs = torchvision.datasets.ImageFolder(os.path.join(data_dir, 'test'))

hotdogs = [train_imgs[i][0] for i in range(8)]
not_hotdogs = [train_imgs[-i - 1][0] for i in range(8)]
native.show_images(hotdogs + not_hotdogs, 2, 8, scale=1.4);

# 使用RGB通道的均值和标准差，以标准化每个通道
normalize = torchvision.transforms.Normalize(
    [0.485, 0.456, 0.406], [0.229, 0.224, 0.225])

train_augs = torchvision.transforms.Compose([
    torchvision.transforms.RandomResizedCrop(224),
    torchvision.transforms.RandomHorizontalFlip(),
    torchvision.transforms.ToTensor(),
    normalize])

test_augs = torchvision.transforms.Compose([
    torchvision.transforms.Resize([256, 256]),
    torchvision.transforms.CenterCrop(224),
    torchvision.transforms.ToTensor(),
    normalize])

pretrained_net = torchvision.models.resnet18(pretrained=True)

pretrained_net.fc

finetune_net = torchvision.models.resnet18(pretrained=True)
finetune_net.fc = nn.Linear(finetune_net.fc.in_features, 2)
nn.init.xavier_uniform_(finetune_net.fc.weight);

# 如果param_group=True，输出层中的模型参数将使用十倍的学习率
def train_fine_tuning(net, learning_rate, batch_size=128, num_epochs=5,
                      param_group=True):
    train_iter = torch.utils.data.DataLoader(torchvision.datasets.ImageFolder(
        os.path.join(data_dir, 'train'), transform=train_augs),
        batch_size=batch_size, shuffle=True)
    test_iter = torch.utils.data.DataLoader(torchvision.datasets.ImageFolder(
        os.path.join(data_dir, 'test'), transform=test_augs),
        batch_size=batch_size)
    devices = native.try_all_gpus()
    loss = nn.CrossEntropyLoss(reduction="none")
    if param_group:
        params_1x = [param for name, param in net.named_parameters()
                     if name not in ["fc.weight", "fc.bias"]]
        trainer = torch.optim.SGD([{'params': params_1x},
                                    {'params': net.fc.parameters(),
                                     'lr': learning_rate * 10}],
                                   lr=learning_rate, weight_decay=0.001)
    else:
        trainer = torch.optim.SGD(net.parameters(), lr=learning_rate,
                                   weight_decay=0.001)
    native.train_ch13(net, train_iter, test_iter, loss, trainer, num_epochs,
                      devices)

train_fine_tuning(finetune_net, 5e-5)

scratch_net = torchvision.models.resnet18()
scratch_net.fc = nn.Linear(scratch_net.fc.in_features, 2)
train_fine_tuning(scratch_net, 5e-4, param_group=False)

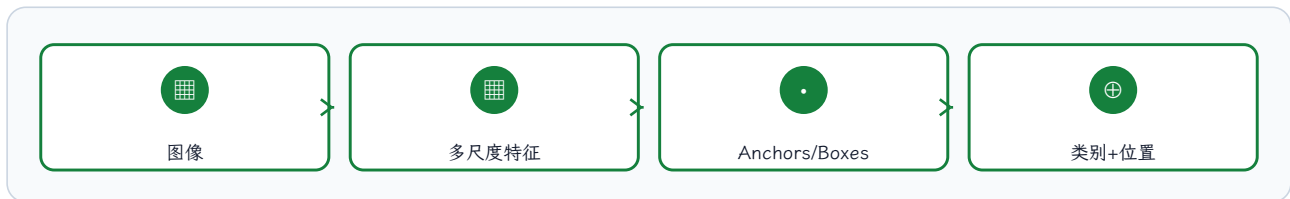
for param in finetune_net.parameters():
    param.requires_grad = False

weight = pretrained_net.fc.weight
hotdog_w = torch.split(weight.data, 1, dim=0)[934]
hotdog_w.shape

```

章节对应：第 13 章 类别：视觉流程 代码块：7 个

结构图：目标检测数据与锚框（视觉流程）



第 13 章目标检测的框、锚框和数据组织流程。

阅读注释：阅读顺序：沿结构图从左到右追踪张量形状，再回到代码寻找对应的 Module 和 forward。

关键点：沿图中的箭头追踪张量形状，重点定位 nn.Module、forward、损失函数和训练循环。

```
"""bounding-box: 计算机视觉"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import torch

native.set_figsize()
img = native.plt.imread('../img/catdog.jpg')
native.plt.imshow(img);

def box_corner_to_center(boxes):
    """从（左上，右下）转换到（中间，宽度，高度）"""
    x1, y1, x2, y2 = boxes[:, 0], boxes[:, 1], boxes[:, 2], boxes[:, 3]
    cx = (x1 + x2) / 2
    cy = (y1 + y2) / 2
    w = x2 - x1
    h = y2 - y1
    boxes = native.stack((cx, cy, w, h), axis=-1)
    return boxes

def box_center_to_corner(boxes):
    """从（中间，宽度，高度）转换到（左上，右下）"""
    cx, cy, w, h = boxes[:, 0], boxes[:, 1], boxes[:, 2], boxes[:, 3]
    x1 = cx - 0.5 * w
    y1 = cy - 0.5 * h
    x2 = cx + 0.5 * w
    y2 = cy + 0.5 * h
    boxes = native.stack((x1, y1, x2, y2), axis=-1)
    return boxes

# bbox是边界框的英文缩写
dog_bbox, cat_bbox = [60.0, 45.0, 378.0, 516.0], [400.0, 112.0, 655.0, 493.0]

boxes = native.tensor((dog_bbox, cat_bbox))
```

```

box_center_to_corner(box_corner_to_center(boxes)) == boxes

def bbox_to_rect(bbox, color):
    # 将边界框(左上x,左上y,右下x,右下y)格式转换成matplotlib格式:
    # ((左上x,左上y),宽,高)
    return native.plt.Rectangle(
        xy=(bbox[0], bbox[1]), width=bbox[2]-bbox[0], height=bbox[3]-bbox[1],
        fill=False, edgecolor=color, linewidth=2)

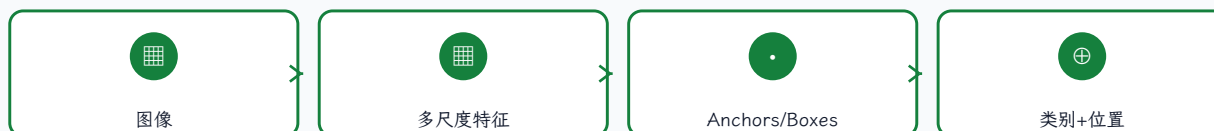
fig = native.plt.imshow(img)
fig.axes.add_patch(bbox_to_rect(dog_bbox, 'blue'))
fig.axes.add_patch(bbox_to_rect(cat_bbox, 'red'));

```

第 13 章 · 目标检测数据与锚框 · chapters/chapter_13_computer_vision/anchor.py

章节对应：第 13 章 类别：视觉流程 代码块：22 个

结构图：目标检测数据与锚框（视觉流程）



第 13 章目标检测的框、锚框和数据组织流程。

阅读注释：阅读顺序：沿结构图从左到右追踪张量，再回到代码寻找对应的 Module 和 forward。

关键点：沿图中的箭头追踪张量形状，重点定位 nn.Module、forward、损失函数和训练循环。

```

"""anchor: 计算机视觉"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import torch

torch.set_printoptions(2) # 精简输出精度

def multibox_prior(data, sizes, ratios):
    """生成以每个像素为中心具有不同形状的锚框"""
    in_height, in_width = data.shape[-2:]
    device, num_sizes, num_ratios = data.device, len(sizes), len(ratios)
    boxes_per_pixel = (num_sizes + num_ratios - 1)
    size_tensor = native.tensor(sizes, device=device)
    ratio_tensor = native.tensor(ratios, device=device)

    # 为了将锚点移动到像素的中心，需要设置偏移量。
    # 因为一个像素的高为1且宽为1，我们选择偏移我们的中心0.5
    offset_h, offset_w = 0.5, 0.5
    steps_h = 1.0 / in_height # 在y轴上缩放步长

```

```

steps_w = 1.0 / in_width # 在x轴上缩放步长

# 生成锚框的所有中心点
center_h = (torch.arange(in_height, device=device) + offset_h) * steps_h
center_w = (torch.arange(in_width, device=device) + offset_w) * steps_w
shift_y, shift_x = torch.meshgrid(center_h, center_w, indexing='ij')
shift_y, shift_x = shift_y.reshape(-1), shift_x.reshape(-1)

# 生成“boxes_per_pixel”个高和宽,
# 之后用于创建锚框的四角坐标(xmin,xmax,ymin,ymax)
w = torch.cat((size_tensor * torch.sqrt(ratio_tensor[0]),
               sizes[0] * torch.sqrt(ratio_tensor[1:])))\
              * in_height / in_width # 处理矩形输入
h = torch.cat((size_tensor / torch.sqrt(ratio_tensor[0]),
               sizes[0] / torch.sqrt(ratio_tensor[1:]))\
              * in_width / in_height # 处理矩形输入)

# 除以2来获得半高和半宽
anchor_manipulations = torch.stack((-w, -h, w, h)).T.repeat(
    in_height * in_width, 1) / 2

# 每个中心点都将有“boxes_per_pixel”个锚框,
# 所以生成含所有锚框中心的网格, 重复了“boxes_per_pixel”次
out_grid = torch.stack([shift_x, shift_y, shift_x, shift_y],
                       dim=1).repeat_interleave(boxes_per_pixel, dim=0)
output = out_grid + anchor_manipulations
return output.unsqueeze(0)

img = native.plt.imread('../img/catdog.jpg')
h, w = img.shape[:2]

print(h, w)
X = torch.rand(size=(1, 3, h, w))
Y = multibox_prior(X, sizes=[0.75, 0.5, 0.25], ratios=[1, 2, 0.5])
Y.shape

boxes = Y.reshape(h, w, 5, 4)
boxes[250, 250, 0, :]

def show_bboxes(axes, bboxes, labels=None, colors=None):
    """显示所有边界框"""
    def _make_list(obj, default_values=None):
        if obj is None:
            obj = default_values
        elif not isinstance(obj, (list, tuple)):
            obj = [obj]
        return obj

    labels = _make_list(labels)
    colors = _make_list(colors, ['b', 'g', 'r', 'm', 'c'])
    for i, bbox in enumerate(bboxes):
        color = colors[i % len(colors)]
        rect = native.bbox_to_rect(native.numpy(bbox), color)
        axes.add_patch(rect)
        if labels and len(labels) > i:
            text_color = 'k' if color == 'w' else 'w'
            axes.text(rect.xy[0], rect.xy[1], labels[i],
                      va='center', ha='center', fontsize=9, color=text_color,
                      bbox=dict(facecolor=color, lw=0))

native.set_figsize()
bbox_scale = native.tensor((w, h, w, h))
fig = native.plt.imshow(img)
show_bboxes(fig.axes, boxes[250, 250, :, :] * bbox_scale,
            ['s=0.75, r=1', 's=0.5, r=1', 's=0.25, r=1', 's=0.75, r=2',
             's=0.75, r=0.5'])

def box_iou(boxes1, boxes2):
    """计算两个锚框或边界框列表中成对的交并比"""
    box_area = lambda boxes: ((boxes[:, 2] - boxes[:, 0]) *
                               (boxes[:, 3] - boxes[:, 1]))

    # boxes1, boxes2, areas1, areas2的形状:
    # boxes1: (boxes1的数量, 4),
    # boxes2: (boxes2的数量, 4),
    # areas1: (boxes1的数量, ),
    # areas2: (boxes2的数量, )
    areas1 = box_area(boxes1)
    areas2 = box_area(boxes2)

    # inter_upperlefts, inter_lowerrights, inters的形状:

```

```

# (boxes1的数量,boxes2的数量,2)
inter_upperlefts = torch.max(boxes1[:, None, :2], boxes2[:, :2])
inter_lowerrights = torch.min(boxes1[:, None, 2:], boxes2[:, :2])
inters = (inter_lowerrights - inter_upperlefts).clamp(min=0)
# inter_areas和union_areas的形状:(boxes1的数量,boxes2的数量)
inter_areas = inters[:, :, 0] * inters[:, :, 1]
union_areas = areas1[:, None] + areas2 - inter_areas
return inter_areas / union_areas

def assign_anchor_to_bbox(ground_truth, anchors, device, iou_threshold=0.5):
    """将最近的真实边界框分配给锚框"""
    num_anchors, num_gt_boxes = anchors.shape[0], ground_truth.shape[0]
    # 位于第i行和第j列的元素x_ij是锚框i和真实边界框j的IoU
    jaccard = box_iou(anchors, ground_truth)
    # 对于每个锚框, 分配的真实边界框的张量
    anchors_bbox_map = torch.full((num_anchors,), -1, dtype=torch.long,
                                   device=device)
    # 根据阈值, 决定是否分配真实边界框
    max_iou, indices = torch.max(jaccard, dim=1)
    anc_i = torch.nonzero(max_iou >= iou_threshold).reshape(-1)
    box_j = indices[max_iou >= iou_threshold]
    anchors_bbox_map[anc_i] = box_j
    col_discard = torch.full((num_anchors,), -1)
    row_discard = torch.full((num_gt_boxes,), -1)
    for _ in range(num_gt_boxes):
        max_idx = torch.argmax(jaccard)
        box_idx = (max_idx % num_gt_boxes).long()
        anc_idx = (max_idx / num_gt_boxes).long()
        anchors_bbox_map[anc_idx] = box_idx
        jaccard[:, box_idx] = col_discard
        jaccard[anc_idx, :] = row_discard
    return anchors_bbox_map

def offset_boxes(anchors, assigned_bb, eps=1e-6):
    """对锚框偏移量的转换"""
    c_anc = native.box_corner_to_center(anchors)
    c_assigned_bb = native.box_corner_to_center(assigned_bb)
    offset_xy = 10 * (c_assigned_bb[:, :2] - c_anc[:, :2]) / c_anc[:, 2:]
    offset_wh = 5 * native.log(eps + c_assigned_bb[:, 2:] / c_anc[:, 2:])
    offset = native.concat([offset_xy, offset_wh], axis=1)
    return offset

def multibox_target(anchors, labels):
    """使用真实边界框标记锚框"""
    batch_size, anchors = labels.shape[0], anchors.squeeze(0)
    batch_offset, batch_mask, batch_class_labels = [], [], []
    device, num_anchors = anchors.device, anchors.shape[0]
    for i in range(batch_size):
        label = labels[i, :, :]
        anchors_bbox_map = assign_anchor_to_bbox(
            label[:, 1:], anchors, device)
        bbox_mask = ((anchors_bbox_map >= 0).float().unsqueeze(-1)).repeat(
            1, 4)
        # 将类标签和分配的边界框坐标初始化为零
        class_labels = torch.zeros(num_anchors, dtype=torch.long,
                                    device=device)
        assigned_bb = torch.zeros((num_anchors, 4), dtype=torch.float32,
                                   device=device)
        # 使用真实边界框来标记锚框的类别。
        # 如果一个锚框没有被分配, 标记其为背景 (值为零)
        indices_true = torch.nonzero(anchors_bbox_map >= 0)
        bb_idx = anchors_bbox_map[indices_true]
        class_labels[indices_true] = label[bb_idx, 0].long() + 1
        assigned_bb[indices_true] = label[bb_idx, 1:]
        # 偏移量转换
        offset = offset_boxes(anchors, assigned_bb) * bbox_mask
        batch_offset.append(offset.reshape(-1))
        batch_mask.append(bbox_mask.reshape(-1))
        batch_class_labels.append(class_labels)
    bbox_offset = torch.stack(batch_offset)
    bbox_mask = torch.stack(batch_mask)
    class_labels = torch.stack(batch_class_labels)
    return (bbox_offset, bbox_mask, class_labels)

ground_truth = native.tensor([[0, 0.1, 0.08, 0.52, 0.92],
                               [1, 0.55, 0.2, 0.9, 0.88]])

```

```

anchors = native.tensor([[0, 0.1, 0.2, 0.3], [0.15, 0.2, 0.4, 0.4],
                        [0.63, 0.05, 0.88, 0.98], [0.66, 0.45, 0.8, 0.8],
                        [0.57, 0.3, 0.92, 0.9]])

fig = native.plt.imshow(img)
show_bboxes(fig.axes, ground_truth[:, 1:] * bbox_scale, ['dog', 'cat'], 'k')
show_bboxes(fig.axes, anchors * bbox_scale, ['0', '1', '2', '3', '4']);

labels = multibox_target(anchors.unsqueeze(dim=0),
                        ground_truth.unsqueeze(dim=0))

labels[2]

labels[1]

labels[0]

def offset_inverse(anchors, offset_preds):
    """根据带有预测偏移量的锚框来预测边界框"""
    anc = native.box_corner_to_center(anchors)
    pred_bbox_xy = (offset_preds[:, :2] * anc[:, 2:] / 10) + anc[:, :2]
    pred_bbox_wh = native.exp(offset_preds[:, 2:] / 5) * anc[:, 2:]
    pred_bbox = native.concat((pred_bbox_xy, pred_bbox_wh), axis=1)
    predicted_bbox = native.box_center_to_corner(pred_bbox)
    return predicted_bbox

def nms(bboxes, scores, iou_threshold):
    """对预测边界框的置信度进行排序"""
    B = torch.argsort(scores, dim=-1, descending=True)
    keep = [] # 保留预测边界框的指标
    while B.numel() > 0:
        i = B[0]
        keep.append(i)
        if B.numel() == 1: break
        iou = box_iou(bboxes[i, :].reshape(-1, 4),
                     bboxes[B[1:], :].reshape(-1, 4)).reshape(-1)
        inds = torch.nonzero(iou <= iou_threshold).reshape(-1)
        B = B[inds + 1]
    return native.tensor(keep, device=bboxes.device)

def multibox_detection(cls_probs, offset_preds, anchors, nms_threshold=0.5,
                      pos_threshold=0.009999999):
    """使用非极大值抑制来预测边界框"""
    device, batch_size = cls_probs.device, cls_probs.shape[0]

    anchors = anchors.squeeze(0)
    num_classes, num_anchors = cls_probs.shape[1], cls_probs.shape[2]
    out = []
    for i in range(batch_size):
        cls_prob, offset_pred = cls_probs[i], offset_preds[i].reshape(-1, 4)
        conf, class_id = torch.max(cls_prob[1:], 0)
        predicted_bb = offset_inverse(anchors, offset_pred)
        keep = nms(predicted_bb, conf, nms_threshold)

        # 找到所有的non_keep索引。并将类设置为背景
        all_idx = torch.arange(num_anchors, dtype=torch.long, device=device)
        combined = torch.cat((keep, all_idx))
        uniques, counts = combined.unique(return_counts=True)
        non_keep = uniques[counts == 1]
        all_id_sorted = torch.cat((keep, non_keep))
        class_id[non_keep] = -1
        class_id = class_id[all_id_sorted]
        conf, predicted_bb = conf[all_id_sorted], predicted_bb[all_id_sorted]

        # pos_threshold是一个用于非背景预测的阈值
        below_min_idx = (conf < pos_threshold)
        class_id[below_min_idx] = -1
        conf[below_min_idx] = 1 - conf[below_min_idx]
        pred_info = torch.cat((class_id.unsqueeze(1),
                              conf.unsqueeze(1),
                              predicted_bb), dim=1)

        out.append(pred_info)
    return native.stack(out)

anchors = native.tensor([[0.1, 0.08, 0.52, 0.92], [0.08, 0.2, 0.56, 0.95],
                        [0.15, 0.3, 0.62, 0.91], [0.55, 0.2, 0.9, 0.88]])
offset_preds = native.tensor([0] * native.size(anchors))
cls_probs = native.tensor([0] * 4, # 背景的概率
                          [0.9, 0.8, 0.7, 0.1], # 狗的概率)

```

```

[0.1, 0.2, 0.3, 0.9]]) # 猫的检测概率

fig = native.plt.imshow(img)
show_bboxes(fig.axes, anchors * bbox_scale,
            ['dog=0.9', 'dog=0.8', 'dog=0.7', 'cat=0.9'])

output = multibox_detection(cls_probs.unsqueeze(dim=0),
                           offset_preds.unsqueeze(dim=0),
                           anchors.unsqueeze(dim=0),
                           nms_threshold=0.5)

output

fig = native.plt.imshow(img)
for i in native.numpy(output[0]):
    if i[0] == -1:
        continue
    label = ('dog=', 'cat=')[int(i[0])] + str(i[1])
    show_bboxes(fig.axes, [native.tensor(i[2:]) * bbox_scale], label)

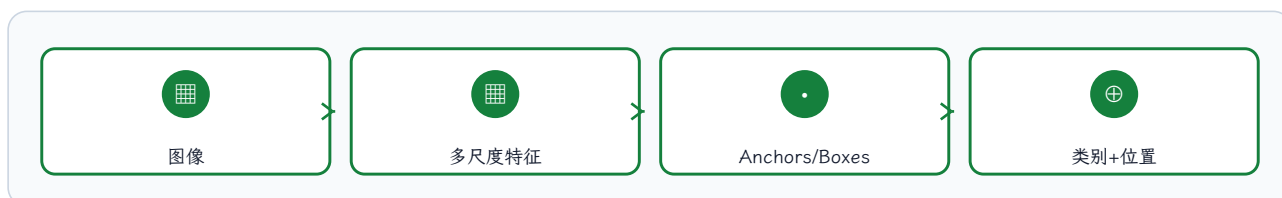
```

第 13 章 · 目标检测数据与锚框 ·

chapters/chapter_13_computer_vision/multiscale_object_detection.py

章节对应：第 13 章 类别：视觉流程 代码块：5 个

结构图：目标检测数据与锚框（视觉流程）



第 13 章目标检测的框、锚框和数据组织流程。

阅读注释：阅读顺序：沿结构图从左到右追踪张量，再回到代码寻找对应的 Module 和 forward。

关键点：沿图中的箭头追踪张量形状，重点定位 nn.Module、forward、损失函数和训练循环。

```

"""multiscale-object-detection: 计算机视觉"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import torch

img = native.plt.imread('../img/catdog.jpg')
h, w = img.shape[:2]
h, w

def display_anchors(fmap_w, fmap_h, s):
    native.set_figsize()
    # 前两个维度上的值不影响输出

```

```

fmap = native.zeros((1, 10, fmap_h, fmap_w))
anchors = native.multibox_prior(fmap, sizes=s, ratios=[1, 2, 0.5])
bbox_scale = native.tensor((w, h, w, h))
native.show_bboxes(native.plt.imshow(img).axes,
                    anchors[0] * bbox_scale)

display_anchors(fmap_w=4, fmap_h=4, s=[0.15])

display_anchors(fmap_w=2, fmap_h=2, s=[0.4])

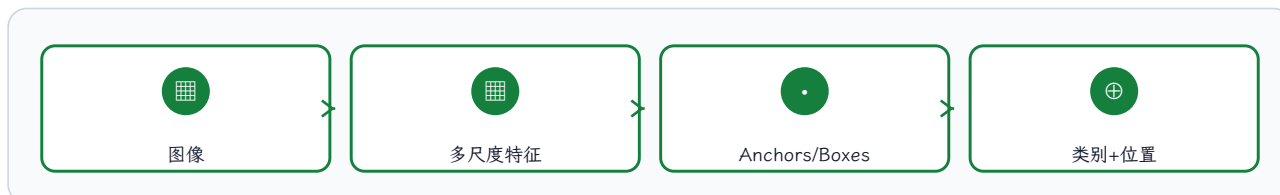
display_anchors(fmap_w=1, fmap_h=1, s=[0.8])

```

第 13 章 · 目标检测数据与锚框 · chapters/chapter_13_computer_vision/object_detection_dataset.py

章节对应：第 13 章 类别：视觉流程 代码块：7 个

结构图：目标检测数据与锚框（视觉流程）



第 13 章目标检测的框、锚框和数据组织流程。

阅读注释：阅读顺序：沿结构图从左到右追踪张量，再回到代码寻找对应的 Module 和 forward。

关键点：沿图中的箭头追踪张量形状，重点定位 nn.Module、forward、损失函数和训练循环。

```

"""object-detection-dataset: 计算机视觉"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import torch
import torchvision
import os
import pandas as pd

native.DATA_HUB['banana-detection'] = (
    native.DATA_URL + 'banana-detection.zip',
    '5de26c8fce5ccdea9f91267273464dc968d20d72')

def read_data_bananas(is_train=True):
    """读取香蕉检测数据集中的图像和标签"""
    data_dir = native.download_extract('banana-detection')
    csv_fname = os.path.join(data_dir, 'bananas_train' if is_train
                              else 'bananas_val', 'label.csv')
    csv_data = pd.read_csv(csv_fname)
    csv_data = csv_data.set_index('img_name')
    images, targets = [], []

```

```

for img_name, target in csv_data.iterrows():
    images.append(torchvision.io.read_image(
        os.path.join(data_dir, 'bananas_train' if is_train else
            'bananas_val', 'images', f'{img_name}'))
    # 这里的target包含 (类别, 左上角x, 左上角y, 右下角x, 右下角y),
    # 其中所有图像都具有相同的香蕉类 (索引为0)
    targets.append(list(target))
return images, torch.tensor(targets).unsqueeze(1) / 256

class BananasDataset(torch.utils.data.Dataset):
    """一个用于加载香蕉检测数据集的自定义数据集"""
    def __init__(self, is_train):
        self.features, self.labels = read_data_bananas(is_train)
        print('read ' + str(len(self.features)) + (f' training examples' if
            is_train else f' validation examples'))

    def __getitem__(self, idx):
        return (self.features[idx].float(), self.labels[idx])

    def __len__(self):
        return len(self.features)

def load_data_bananas(batch_size):
    """加载香蕉检测数据集"""
    train_iter = torch.utils.data.DataLoader(BananasDataset(is_train=True),
        batch_size, shuffle=True)
    val_iter = torch.utils.data.DataLoader(BananasDataset(is_train=False),
        batch_size)

    return train_iter, val_iter

batch_size, edge_size = 32, 256
train_iter, _ = load_data_bananas(batch_size)
batch = next(iter(train_iter))
batch[0].shape, batch[1].shape

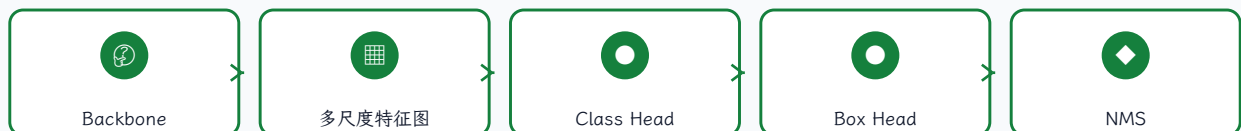
imgs = (batch[0][0:10].permute(0, 2, 3, 1)) / 255
axes = native.show_images(imgs, 2, 5, scale=2)
for ax, label in zip(axes, batch[1][0:10]):
    native.show_bboxes(ax, [label[0][1:5] * edge_size], colors=['w'])

```

第 13 章 · SSD · [chapters/chapter_13_computer_vision/ssd.py](#)

章节对应：第 13 章 类别：网络结构 代码块：23 个

结构图：SSD（网络结构）



第 13 章单阶段多尺度目标检测网络。

阅读注释：阅读顺序：先看输入与输出，再看核心层/模块，最后看损失函数和训练循环。
SSD 在多个尺度的特征图上同时预测类别和边界框，最后通过 NMS 去除重复候选框。

```

"""ssd: 计算机视觉"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch

```

```

from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import torch
import torchvision
from torch import nn
from torch.nn import functional as F

def cls_predictor(num_inputs, num_anchors, num_classes):
    return nn.Conv2d(num_inputs, num_anchors * (num_classes + 1),
                     kernel_size=3, padding=1)

def bbox_predictor(num_inputs, num_anchors):
    return nn.Conv2d(num_inputs, num_anchors * 4, kernel_size=3, padding=1)

def forward(x, block):
    return block(x)

Y1 = forward(torch.zeros((2, 8, 20, 20)), cls_predictor(8, 5, 10))
Y2 = forward(torch.zeros((2, 16, 10, 10)), cls_predictor(16, 3, 10))
Y1.shape, Y2.shape

def flatten_pred(pred):
    return torch.flatten(pred.permute(0, 2, 3, 1), start_dim=1)

def concat_preds(preds):
    return torch.cat([flatten_pred(p) for p in preds], dim=1)

concat_preds([Y1, Y2]).shape

def down_sample_blk(in_channels, out_channels):
    blk = []
    for _ in range(2):
        blk.append(nn.Conv2d(in_channels, out_channels,
                             kernel_size=3, padding=1))
        blk.append(nn.BatchNorm2d(out_channels))

    blk.append(nn.ReLU())
    in_channels = out_channels
    blk.append(nn.MaxPool2d(2))
    return nn.Sequential(*blk)

forward(torch.zeros((2, 3, 20, 20)), down_sample_blk(3, 10)).shape

def base_net():
    blk = []
    num_filters = [3, 16, 32, 64]
    for i in range(len(num_filters) - 1):
        blk.append(down_sample_blk(num_filters[i], num_filters[i+1]))
    return nn.Sequential(*blk)

forward(torch.zeros((2, 3, 256, 256)), base_net()).shape

def get_blk(i):
    if i == 0:
        blk = base_net()
    elif i == 1:
        blk = down_sample_blk(64, 128)
    elif i == 4:
        blk = nn.AdaptiveMaxPool2d((1,1))
    else:
        blk = down_sample_blk(128, 128)
    return blk

def blk_forward(X, blk, size, ratio, cls_predictor, bbox_predictor):
    Y = blk(X)
    anchors = native.multibox_prior(Y, sizes=size, ratios=ratio)
    cls_preds = cls_predictor(Y)
    bbox_preds = bbox_predictor(Y)
    return (Y, anchors, cls_preds, bbox_preds)

```

```

sizes = [[0.2, 0.272], [0.37, 0.447], [0.54, 0.619], [0.71, 0.79],
         [0.88, 0.961]]
ratios = [[1, 2, 0.5]] * 5
num_anchors = len(sizes[0]) + len(ratios[0]) - 1

class TinySSD(nn.Module):
    def __init__(self, num_classes, **kwargs):
        super(TinySSD, self).__init__(**kwargs)
        self.num_classes = num_classes
        idx_to_in_channels = [64, 128, 128, 128, 128]
        for i in range(5):
            # 即赋值语句self.blk_i=get_blk(i)
            setattr(self, f'blk_{i}', get_blk(i))
            setattr(self, f'cls_{i}', cls_predictor(idx_to_in_channels[i],
                                                    num_anchors, num_classes))
            setattr(self, f'bbox_{i}', bbox_predictor(idx_to_in_channels[i],
                                                    num_anchors))

    def forward(self, X):
        anchors, cls_preds, bbox_preds = [None] * 5, [None] * 5, [None] * 5
        for i in range(5):
            # getattr(self, 'blk_%d'%i)即访问self.blk_i
            X, anchors[i], cls_preds[i], bbox_preds[i] = blk_forward(
                X, getattr(self, f'blk_{i}'), sizes[i], ratios[i],
                getattr(self, f'cls_{i}'), getattr(self, f'bbox_{i}'))
            anchors = torch.cat(anchors, dim=1)

            cls_preds = concat_preds(cls_preds)
            cls_preds = cls_preds.reshape(
                cls_preds.shape[0], -1, self.num_classes + 1)
            bbox_preds = concat_preds(bbox_preds)
            return anchors, cls_preds, bbox_preds

net = TinySSD(num_classes=1)
X = torch.zeros((32, 3, 256, 256))
anchors, cls_preds, bbox_preds = net(X)

print('output anchors:', anchors.shape)
print('output class preds:', cls_preds.shape)
print('output bbox preds:', bbox_preds.shape)

batch_size = 32
train_iter, _ = native.load_data_bananas(batch_size)

device, net = native.try_gpu(), TinySSD(num_classes=1)
trainer = torch.optim.SGD(net.parameters(), lr=0.2, weight_decay=5e-4)

cls_loss = nn.CrossEntropyLoss(reduction='none')
bbox_loss = nn.L1Loss(reduction='none')

def calc_loss(cls_preds, cls_labels, bbox_preds, bbox_labels, bbox_masks):
    batch_size, num_classes = cls_preds.shape[0], cls_preds.shape[2]
    cls = cls_loss(cls_preds.reshape(-1, num_classes),
                   cls_labels.reshape(-1)).reshape(batch_size, -1).mean(dim=1)
    bbox = bbox_loss(bbox_preds * bbox_masks,
                     bbox_labels * bbox_masks).mean(dim=1)
    return cls + bbox

def cls_eval(cls_preds, cls_labels):
    # 由于类别预测结果放在最后一维, argmax需要指定最后一维。
    return float((cls_preds.argmax(dim=-1).type(
        cls_labels.dtype) == cls_labels).sum())

def bbox_eval(bbox_preds, bbox_labels, bbox_masks):
    return float((torch.abs((bbox_labels - bbox_preds) * bbox_masks)).sum())

num_epochs, timer = 20, native.Timer()
animator = native.Animator(xlabel='epoch', xlim=[1, num_epochs],
                           legend=['class error', 'bbox mae'])
net = net.to(device)
for epoch in range(num_epochs):
    # 训练精确度的和, 训练精确度的和中的示例数
    # 绝对误差的和, 绝对误差的和中的示例数
    metric = native.Accumulator(4)
    net.train()
    for features, target in train_iter:
        timer.start()

```

```

        trainer.zero_grad()
        X, Y = features.to(device), target.to(device)
        # 生成多尺度的锚框, 为每个锚框预测类别和偏移量
        anchors, cls_preds, bbox_preds = net(X)
        # 为每个锚框标注类别和偏移量
        bbox_labels, bbox_masks = native.multibox_target(anchors, Y)
        # 根据类别和偏移量的预测和标注值计算损失函数
        l = calc_loss(cls_preds, cls_labels, bbox_preds, bbox_labels,
                      bbox_masks)
        l.mean().backward()

        trainer.step()
        metric.add(cls_eval(cls_preds, cls_labels), cls_labels.numel(),
                  bbox_eval(bbox_preds, bbox_labels, bbox_masks),
                  bbox_labels.numel())
        cls_err, bbox_mae = 1 - metric[0] / metric[1], metric[2] / metric[3]
        animator.add(epoch + 1, (cls_err, bbox_mae))
    print(f'class err {cls_err:.2e}, bbox mae {bbox_mae:.2e}')
    print(f'{len(train_iter.dataset) / timer.stop():.1f} examples/sec on '
          f'{str(device)}')

X = torchvision.io.read_image('../img/banana.jpg').unsqueeze(0).float()
img = X.squeeze(0).permute(1, 2, 0).long()

def predict(X):
    net.eval()
    anchors, cls_preds, bbox_preds = net(X.to(device))
    cls_probs = F.softmax(cls_preds, dim=2).permute(0, 2, 1)
    output = native.multibox_detection(cls_probs, bbox_preds, anchors)
    idx = [i for i, row in enumerate(output[0]) if row[0] != -1]
    return output[0, idx]

output = predict(X)

def display(img, output, threshold):
    native.set_figsize((5, 5))
    fig = native.plt.imshow(img)
    for row in output:
        score = float(row[1])
        if score < threshold:
            continue
        h, w = img.shape[0:2]
        bbox = [row[2:6] * torch.tensor((w, h, w, h), device=row.device)]
        native.show_bboxes(fig.axes, bbox, '%.2f' % score, 'w')

display(img, output.cpu(), threshold=0.9)

def smooth_l1(data, scalar):
    out = []
    for i in data:
        if abs(i) < 1 / (scalar ** 2):
            out.append(((scalar * i) ** 2) / 2)
        else:
            out.append(abs(i) - 0.5 / (scalar ** 2))
    return torch.tensor(out)

sigmas = [10, 1, 0.5]
lines = ['-', '--', '-.']
x = torch.arange(-2, 2, 0.1)
native.set_figsize()

for l, s in zip(lines, sigmas):
    y = smooth_l1(x, scalar=s)
    native.plt.plot(x, y, l, label='sigma=%.1f' % s)
native.plt.legend();

def focal_loss(gamma, x):
    return -(1 - x) ** gamma * torch.log(x)

x = torch.arange(0.01, 1, 0.01)
for l, gamma in zip(lines, [0, 1, 5]):
    y = native.plt.plot(x, focal_loss(gamma, x), l, label='gamma=%.1f' % gamma)
    native.plt.legend();

```

第 13 章 · R-CNN · chapters/chapter_13_computer_vision/rcnn.py

章节对应：第 13 章 类别：网络结构 代码块：3 个

结构图：R-CNN（网络结构）



第 13 章候选区域式目标检测流程。

阅读注释：阅读顺序：先看输入与输出，再看核心层/模块，最后看损失函数和训练循环。
先产生候选区域，再对区域提取 CNN 特征并进行分类和位置回归。

```
"""rcnn: 计算机视觉"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import torch
import torchvision

X = torch.arange(16.).reshape(1, 1, 4, 4)
X

rois = torch.Tensor([[0, 0, 0, 20, 20], [0, 0, 10, 30, 30]])

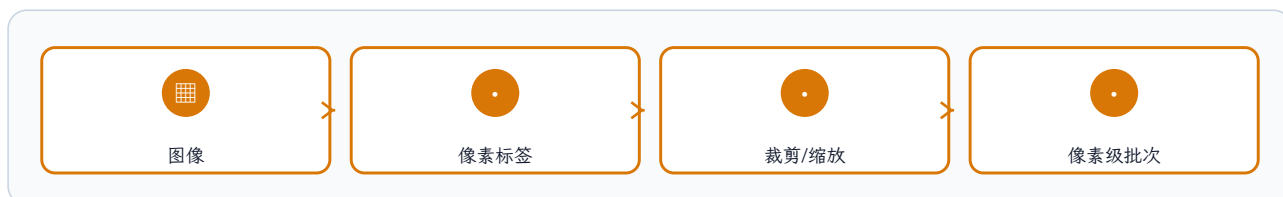
torchvision.ops.roi_pool(X, rois, output_size=(2, 2), spatial_scale=0.1)
```

第 13 章 · 语义分割数据管线 ·

chapters/chapter_13_computer_vision/semantic_segmentation_and_dataset.py

章节对应：第 13 章 类别：数据流程 代码块：13 个

结构图：语义分割数据管线（数据流程）



第 13 章语义分割数据准备。

阅读注释：阅读顺序：先看样本格式，再看预处理与 Dataset，最后看 DataLoader 生成的批次。
关键点：沿图中的箭头追踪张量形状，重点定位 nn.Module、forward、损失函数和训练循环。

```

"""semantic-segmentation-and-dataset: 计算机视觉"""

# 来源: 依据《动手学深度学习 (PyTorch 第二版)》的 D2L 中文 PyTorch 示例整理。
# 改写说明: 已移除 d2l 命名空间, 共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import torch
import torchvision
import os

native.DATA_HUB['voc2012'] = (native.DATA_URL + 'VOCtrainval_11-May-2012.tar',
                             '4e443f8a2eca6b1dac8a6c57641b67dd40621a49')

voc_dir = native.download_extract('voc2012', 'VOCdevkit/VOC2012')

def read_voc_images(voc_dir, is_train=True):
    """读取所有VOC图像并标注"""
    txt_fname = os.path.join(voc_dir, 'ImageSets', 'Segmentation',
                             'train.txt' if is_train else 'val.txt')
    mode = torchvision.io.image.ImageReadMode.RGB
    with open(txt_fname, 'r') as f:
        images = f.read().split()
        features, labels = [], []
        for i, fname in enumerate(images):
            features.append(torchvision.io.read_image(os.path.join(
                voc_dir, 'JPEGImages', f'{fname}.jpg')))
            labels.append(torchvision.io.read_image(os.path.join(
                voc_dir, 'SegmentationClass', f'{fname}.png'), mode))
        return features, labels

train_features, train_labels = read_voc_images(voc_dir, True)

n = 5
imgs = train_features[0:n] + train_labels[0:n]
imgs = [img.permute(1,2,0) for img in imgs]
native.show_images(imgs, 2, n);

VOC_COLORMAP = [[0, 0, 0], [128, 0, 0], [0, 128, 0], [128, 128, 0],
                 [0, 0, 128], [128, 0, 128], [0, 128, 128], [128, 128, 128],
                 [64, 0, 0], [192, 0, 0], [64, 128, 0], [192, 128, 0],
                 [64, 0, 128], [192, 0, 128], [64, 128, 128], [192, 128, 128],
                 [0, 64, 0], [128, 64, 0], [0, 192, 0], [128, 192, 0],
                 [0, 64, 128]]

VOC_CLASSES = ['background', 'aeroplane', 'bicycle', 'bird', 'boat',
               'bottle', 'bus', 'car', 'cat', 'chair', 'cow',
               'diningtable', 'dog', 'horse', 'motorbike', 'person',
               'potted plant', 'sheep', 'sofa', 'train', 'tv/monitor']

def voc_colormap2label():
    """构建从RGB到VOC类别索引的映射"""
    colormap2label = torch.zeros(256 ** 3, dtype=torch.long)
    for i, colormap in enumerate(VOC_COLORMAP):
        colormap2label[
            (colormap[0] * 256 + colormap[1]) * 256 + colormap[2]] = i
    return colormap2label

```

```

def voc_label_indices(colormap, colormap2label):
    """将VOC标签中的RGB值映射到它们的类别索引"""
    colormap = colormap.permute(1, 2, 0).numpy().astype('int32')
    idx = ((colormap[:, :, 0] * 256 + colormap[:, :, 1]) * 256
           + colormap[:, :, 2])
    return colormap2label[idx]

y = voc_label_indices(train_labels[0], voc_colormap2label())
y[105:115, 130:140], VOC_CLASSES[1]

def voc_rand_crop(feature, label, height, width):
    """随机裁剪特征和标签图像"""
    rect = torchvision.transforms.RandomCrop.get_params(
        feature, (height, width))
    feature = torchvision.transforms.functional.crop(feature, *rect)
    label = torchvision.transforms.functional.crop(label, *rect)
    return feature, label

imgs = []
for _ in range(n):
    imgs += voc_rand_crop(train_features[0], train_labels[0], 200, 300)

imgs = [img.permute(1, 2, 0) for img in imgs]
native.show_images(imgs[::2] + imgs[1::2], 2, n);

class VOCSegDataset(torch.utils.data.Dataset):
    """一个用于加载VOC数据集的自定义数据集"""

    def __init__(self, is_train, crop_size, voc_dir):
        self.transform = torchvision.transforms.Normalize(
            mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
        self.crop_size = crop_size
        features, labels = read_voc_images(voc_dir, is_train=is_train)
        self.features = [self.normalize_image(feature)
                        for feature in self.filter(features)]
        self.labels = self.filter(labels)
        self.colormap2label = voc_colormap2label()
        print('read ' + str(len(self.features)) + ' examples')

    def normalize_image(self, img):
        return self.transform(img.float() / 255)

    def filter(self, imgs):
        return [img for img in imgs if (
            img.shape[1] >= self.crop_size[0] and
            img.shape[2] >= self.crop_size[1])]

    def __getitem__(self, idx):
        feature, label = voc_rand_crop(self.features[idx], self.labels[idx],
                                       *self.crop_size)
        return (feature, voc_label_indices(label, self.colormap2label))

    def __len__(self):
        return len(self.features)

crop_size = (320, 480)
voc_train = VOCSegDataset(True, crop_size, voc_dir)
voc_test = VOCSegDataset(False, crop_size, voc_dir)

batch_size = 64
train_iter = torch.utils.data.DataLoader(voc_train, batch_size, shuffle=True,
                                         drop_last=True,
                                         num_workers=native.get_data_loader_workers())

for X, Y in train_iter:
    print(X.shape)
    print(Y.shape)
    break

def load_data_voc(batch_size, crop_size):
    """加载VOC语义分割数据集"""
    voc_dir = native.download_extract('voc2012', os.path.join(
        'VOCdevkit', 'VOC2012'))
    num_workers = native.get_data_loader_workers()
    train_iter = torch.utils.data.DataLoader(
        VOCSegDataset(True, crop_size, voc_dir), batch_size,
        shuffle=True, drop_last=True, num_workers=num_workers)

```

```
test_iter = torch.utils.data.DataLoader(
    VOCSegDataset(False, crop_size, voc_dir), batch_size,
    drop_last=True, num_workers=num_workers)
return train_iter, test_iter
```

第 13 章 · 转置卷积 · chapters/chapter_13_computer_vision/transposed_conv.py

章节对应：第 13 章 类别：运算原理 代码块：11 个

结构图：转置卷积（运算原理）



第 13 章学习式上采样运算。

阅读注释：阅读顺序：沿结构图从左到右追踪张量，再回到代码寻找对应的 Module 和 forward。
关键点：沿图中的箭头追踪张量形状，重点定位 nn.Module、forward、损失函数和训练循环。

```
"""transposed-conv: 计算机视觉"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import torch
from torch import nn
import common as native

def trans_conv(X, K):
    h, w = K.shape
    Y = native.zeros((X.shape[0] + h - 1, X.shape[1] + w - 1))
    for i in range(X.shape[0]):
        for j in range(X.shape[1]):
            Y[i: i + h, j: j + w] += X[i, j] * K
    return Y

X = native.tensor([[0.0, 1.0], [2.0, 3.0]])
K = native.tensor([[0.0, 1.0], [2.0, 3.0]])
trans_conv(X, K)

X, K = X.reshape(1, 1, 2, 2), K.reshape(1, 1, 2, 2)
tconv = nn.ConvTranspose2d(1, 1, kernel_size=2, bias=False)
tconv.weight.data = K
tconv(X)

tconv = nn.ConvTranspose2d(1, 1, kernel_size=2, padding=1, bias=False)
tconv.weight.data = K
tconv(X)
```

```

tconv = nn.ConvTranspose2d(1, 1, kernel_size=2, stride=2, bias=False)
tconv.weight.data = K
tconv(X)

X = torch.rand(size=(1, 10, 16, 16))
conv = nn.Conv2d(10, 20, kernel_size=5, padding=2, stride=3)
tconv = nn.ConvTranspose2d(20, 10, kernel_size=5, padding=2, stride=3)
tconv(conv(X)).shape == X.shape

X = native.arange(9.0).reshape(3, 3)
K = native.tensor([[1.0, 2.0], [3.0, 4.0]])
Y = native.corr2d(X, K)
Y

def kernel2matrix(K):
    k, W = native.zeros(5), native.zeros((4, 9))
    k[:2], k[3:5] = K[0, :], K[1, :]
    W[0, :5], W[1, 1:6], W[2, 3:8], W[3, 4:] = k, k, k, k
    return W

W = kernel2matrix(K)
W

Y == native.matmul(W, native.reshape(X, -1)).reshape(2, 2)

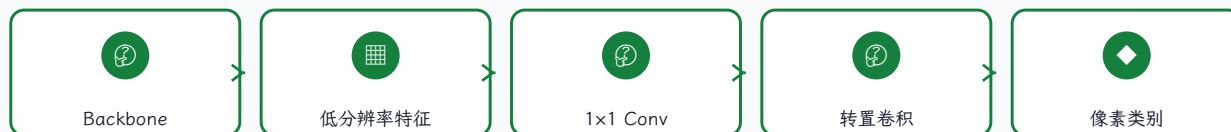
Z = trans_conv(Y, K)
Z == native.matmul(W.T, native.reshape(Y, -1)).reshape(3, 3)

```

第 13 章 · FCN（全卷积网络） · chapters/chapter_13_computer_vision/fcn.py

章节对应：第 13 章 类别：网络结构 代码块：15 个

结构图：FCN（全卷积网络）（网络结构）



第 13 章像素级语义分割网络。

阅读注释：阅读顺序：先看输入与输出，再看核心层/模块，最后看损失函数和训练循环。
网络不使用固定尺寸的全连接层，输出是每个像素的类别 logits。

```

"""fcn: 计算机视觉"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_R00T = Path(__file__).resolve().parents[1]
if str(_R00T) not in sys.path:
    sys.path.insert(0, str(_R00T))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import torch

```

```

import torchvision
from torch import nn
from torch.nn import functional as F

pretrained_net = torchvision.models.resnet18(pretrained=True)
list(pretrained_net.children())[-3:]

net = nn.Sequential(*list(pretrained_net.children())[:-2])

X = torch.rand(size=(1, 3, 320, 480))
net(X).shape

num_classes = 21
net.add_module('final_conv', nn.Conv2d(512, num_classes, kernel_size=1))
net.add_module('transpose_conv', nn.ConvTranspose2d(num_classes, num_classes,
                                                    kernel_size=64, padding=16, stride=32))

def bilinear_kernel(in_channels, out_channels, kernel_size):
    factor = (kernel_size + 1) // 2
    if kernel_size % 2 == 1:
        center = factor - 1
    else:
        center = factor - 0.5
    og = (torch.arange(kernel_size).reshape(-1, 1),
          torch.arange(kernel_size).reshape(1, -1))
    filt = (1 - torch.abs(og[0] - center) / factor) * \
           (1 - torch.abs(og[1] - center) / factor)
    weight = torch.zeros((in_channels, out_channels,
                           kernel_size, kernel_size))
    weight[range(in_channels), range(out_channels), :, :] = filt
    return weight

conv_trans = nn.ConvTranspose2d(3, 3, kernel_size=4, padding=1, stride=2,
                                bias=False)
conv_trans.weight.data.copy_(bilinear_kernel(3, 3, 4));

img = torchvision.transforms.ToTensor()(native.Image.open('../img/catdog.jpg'))
X = img.unsqueeze(0)
Y = conv_trans(X)
out_img = Y[0].permute(1, 2, 0).detach()

native.set_figsize()
print('input image shape:', img.permute(1, 2, 0).shape)
native.plt.imshow(img.permute(1, 2, 0));
print('output image shape:', out_img.shape)
native.plt.imshow(out_img);

W = bilinear_kernel(num_classes, num_classes, 64)
net.transpose_conv.weight.data.copy_(W);

batch_size, crop_size = 32, (320, 480)
train_iter, test_iter = native.load_data_voc(batch_size, crop_size)

def loss(inputs, targets):
    return F.cross_entropy(inputs, targets, reduction='none').mean(1).mean(1)

num_epochs, lr, wd, devices = 5, 0.001, 1e-3, native.try_all_gpus()
trainer = torch.optim.SGD(net.parameters(), lr=lr, weight_decay=wd)
native.train_ch13(net, train_iter, test_iter, loss, trainer, num_epochs, devices)

def predict(img):
    X = test_iter.dataset.normalize_image(img).unsqueeze(0)
    pred = net(X.to(devices[0])).argmax(dim=1)
    return pred.reshape(pred.shape[1], pred.shape[2])

def label2image(pred):
    colormap = torch.tensor(native.VOC_COLORMAP, device=devices[0])
    X = pred.long()
    return colormap[X, :]

voc_dir = native.download_extract('voc2012', 'VOCdevkit/VOC2012')
test_images, test_labels = native.read_voc_images(voc_dir, False)
n, imgs = 4, []
for i in range(n):
    crop_rect = (0, 0, 320, 480)
    X = torchvision.transforms.functional.crop(test_images[i], *crop_rect)
    pred = label2image(predict(X))

```

```

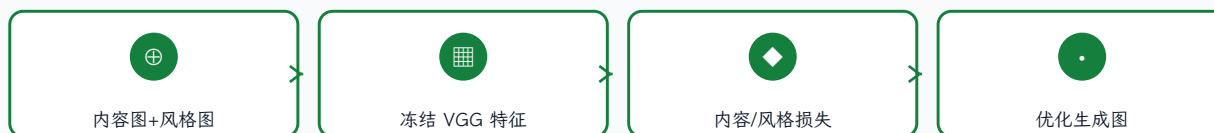
imgs += [X.permute(1,2,0), pred.cpu(),
         torchvision.transforms.functional.crop(
             test_labels[i], *crop_rect).permute(1,2,0)]
native.show_images(imgs[::3] + imgs[1::3] + imgs[2::3], 3, n, scale=2);

```

第 13 章 · 神经风格迁移 · chapters/chapter_13_computer_vision/neural_style.py

章节对应：第 13 章 类别：网络/损失流程 代码块：17 个

结构图：神经风格迁移（网络/损失流程）



第 13 章以特征提取器和损失函数驱动图像生成。

阅读注释：阅读顺序：先看输入与输出，再看核心层/模块，最后看损失函数和训练循环。

关键点：沿图中的箭头追踪张量形状，重点定位 nn.Module、forward、损失函数和训练循环。

```

"""neural-style: 计算机视觉"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import torch
import torchvision
from torch import nn

native.set_figsize()
content_img = native.Image.open('../img/rainier.jpg')
native.plt.imshow(content_img);

style_img = native.Image.open('../img/autumn-oak.jpg')
native.plt.imshow(style_img);

rgb_mean = torch.tensor([0.485, 0.456, 0.406])
rgb_std = torch.tensor([0.229, 0.224, 0.225])

def preprocess(img, image_shape):
    transforms = torchvision.transforms.Compose([
        torchvision.transforms.Resize(image_shape),
        torchvision.transforms.ToTensor(),
        torchvision.transforms.Normalize(mean=rgb_mean, std=rgb_std)])
    return transforms(img).unsqueeze(0)

def postprocess(img):
    img = img[0].to(rgb_std.device)
    img = torch.clamp(img.permute(1, 2, 0) * rgb_std + rgb_mean, 0, 1)

```

```

        return torchvision.transforms.ToPILImage()(img.permute(2, 0, 1))

pretrained_net = torchvision.models.vgg19(pretrained=True)

style_layers, content_layers = [0, 5, 10, 19, 28], [25]

net = nn.Sequential(*[pretrained_net.features[i] for i in
                      range(max(content_layers + style_layers) + 1)])

def extract_features(X, content_layers, style_layers):
    contents = []
    styles = []
    for i in range(len(net)):
        X = net[i](X)
        if i in style_layers:
            styles.append(X)
        if i in content_layers:
            contents.append(X)
    return contents, styles

def get_contents(image_shape, device):
    content_X = preprocess(content_img, image_shape).to(device)
    contents_Y, _ = extract_features(content_X, content_layers, style_layers)
    return content_X, contents_Y

def get_styles(image_shape, device):
    style_X = preprocess(style_img, image_shape).to(device)
    _, styles_Y = extract_features(style_X, content_layers, style_layers)
    return style_X, styles_Y

def content_loss(Y_hat, Y):
    # 我们从动态计算梯度的树中分离目标:
    # 这是一个规定的值, 而不是一个变量。
    return torch.square(Y_hat - Y.detach()).mean()

def gram(X):
    num_channels, n = X.shape[1], native.size(X) // X.shape[1]
    X = native.reshape(X, (num_channels, n))
    return native.matmul(X, X.T) / (num_channels * n)

def style_loss(Y_hat, gram_Y):
    return torch.square(gram(Y_hat) - gram_Y.detach()).mean()

def tv_loss(Y_hat):
    return 0.5 * (native.abs(Y_hat[:, :, 1:, :] - Y_hat[:, :, :-1, :]).mean() +
                  native.abs(Y_hat[:, :, :, 1:] - Y_hat[:, :, :, :-1]).mean())

content_weight, style_weight, tv_weight = 1, 1e3, 10

def compute_loss(X, contents_Y_hat, styles_Y_hat, contents_Y, styles_Y_gram):
    # 分别计算内容损失、风格损失和全变分损失
    contents_l = [content_loss(Y_hat, Y) * content_weight for Y_hat, Y in zip(
        contents_Y_hat, contents_Y)]
    styles_l = [style_loss(Y_hat, Y) * style_weight for Y_hat, Y in zip(
        styles_Y_hat, styles_Y_gram)]
    tv_l = tv_loss(X) * tv_weight
    # 对所有损失求和
    l = sum(10 * styles_l + contents_l + [tv_l])
    return contents_l, styles_l, tv_l, l

class SynthesizedImage(nn.Module):
    def __init__(self, img_shape, **kwargs):
        super(SynthesizedImage, self).__init__(**kwargs)
        self.weight = nn.Parameter(torch.rand(*img_shape))

    def forward(self):
        return self.weight

def get_inits(X, device, lr, styles_Y):
    gen_img = SynthesizedImage(X.shape).to(device)
    gen_img.weight.data.copy_(X.data)
    trainer = torch.optim.Adam(gen_img.parameters(), lr=lr)
    styles_Y_gram = [gram(Y) for Y in styles_Y]
    return gen_img(), styles_Y_gram, trainer

def train(X, contents_Y, styles_Y, device, lr, num_epochs, lr_decay_epoch):

```

```

X, styles_Y_gram, trainer = get_inits(X, device, lr, styles_Y)
scheduler = torch.optim.lr_scheduler.StepLR(trainer, lr_decay_epoch, 0.8)
animator = native.Animator(xlabel='epoch', ylabel='loss',
                           xlim=[10, num_epochs],
                           legend=['content', 'style', 'TV'],
                           ncols=2, figsize=(7, 2.5))
for epoch in range(num_epochs):
    trainer.zero_grad()
    contents_Y_hat, styles_Y_hat = extract_features(
        X, content_layers, style_layers)
    contents_l, styles_l, tv_l, l = compute_loss(
        X, contents_Y_hat, styles_Y_hat, contents_Y, styles_Y_gram)
    l.backward()
    trainer.step()
    scheduler.step()
    if (epoch + 1) % 10 == 0:
        animator.axes[1].imshow(postprocess(X))
        animator.add(epoch + 1, [float(sum(contents_l)),
                                float(sum(styles_l)), float(tv_l)])
return X

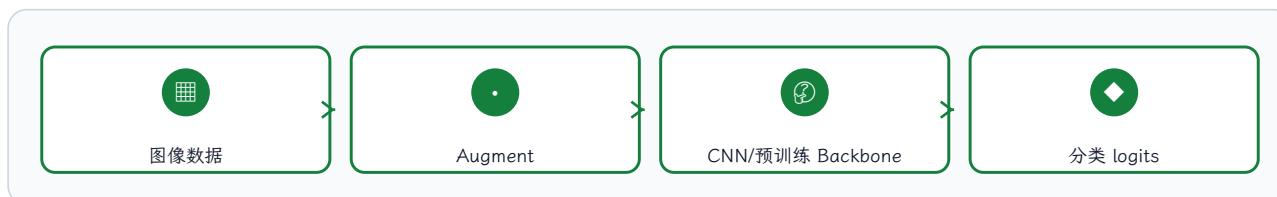
device, image_shape = native.try_gpu(), (300, 450)
net = net.to(device)
content_X, contents_Y = get_contents(image_shape, device)
_, styles_Y = get_styles(image_shape, device)
output = train(content_X, contents_Y, styles_Y, device, 0.3, 500, 50)

```

第 13 章 · 图像分类训练管线 · chapters/chapter_13_computer_vision/kaggle_cifar10.py

章节对应：第 13 章 类别：视觉流程 代码块：15 个

结构图：图像分类训练管线（视觉流程）



第 13 章图像分类实战流程。

阅读注释：阅读顺序：沿结构图从左到右追踪张量，再回到代码寻找对应的 Module 和 forward。
关键点：沿图中的箭头追踪张量形状，重点定位 nn.Module、forward、损失函数和训练循环。

```

"""kaggle-cifar10: 计算机视觉"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import collections
import common as native
import math

```

```

import torch
import torchvision
from torch import nn
import os
import pandas as pd
import shutil

native.DATA_HUB['cifar10_tiny'] = (native.DATA_URL + 'kaggle_cifar10_tiny.zip',
                                   '2068874e4b9a9f0fb07ebe0ad2b29754449ccacd')

# 如果使用完整的Kaggle竞赛的数据集, 设置demo为False
demo = True

if demo:
    data_dir = native.download_extract('cifar10_tiny')
else:
    data_dir = '../data/cifar-10/'

def read_csv_labels(fname):
    """读取fname来给标签字典返回一个文件名"""
    with open(fname, 'r') as f:
        # 跳过文件头行(列名)
        lines = f.readlines()[1:]
        tokens = [l.rstrip().split(',') for l in lines]
        return dict(((name, label) for name, label in tokens))

labels = read_csv_labels(os.path.join(data_dir, 'trainLabels.csv'))
print('# 训练样本:', len(labels))
print('# 类别:', len(set(labels.values())))

def copyfile(filename, target_dir):
    """将文件复制到目标目录"""
    os.makedirs(target_dir, exist_ok=True)
    shutil.copy(filename, target_dir)

def reorg_train_valid(data_dir, labels, valid_ratio):
    """将验证集从原始的训练集中拆分出来"""
    # 训练数据集中样本最少的类别中的样本数
    n = collections.Counter(labels.values()).most_common()[-1][1]
    # 验证集中每个类别的样本数
    n_valid_per_label = max(1, math.floor(n * valid_ratio))
    label_count = {}
    for train_file in os.listdir(os.path.join(data_dir, 'train')):
        label = labels[train_file.split('.')[0]]
        fname = os.path.join(data_dir, 'train', train_file)
        copyfile(fname, os.path.join(data_dir, 'train_valid_test',
                                     'train_valid', label))
        if label not in label_count or label_count[label] < n_valid_per_label:
            copyfile(fname, os.path.join(data_dir, 'train_valid_test',
                                         'valid', label))
            label_count[label] = label_count.get(label, 0) + 1
        else:
            copyfile(fname, os.path.join(data_dir, 'train_valid_test',
                                         'train', label))
    return n_valid_per_label

def reorg_test(data_dir):
    """在预测期间整理测试集, 以方便读取"""
    for test_file in os.listdir(os.path.join(data_dir, 'test')):
        copyfile(os.path.join(data_dir, 'test', test_file),
                 os.path.join(data_dir, 'train_valid_test', 'test',
                              'unknown'))

def reorg_cifar10_data(data_dir, valid_ratio):
    labels = read_csv_labels(os.path.join(data_dir, 'trainLabels.csv'))
    reorg_train_valid(data_dir, labels, valid_ratio)
    reorg_test(data_dir)

batch_size = 32 if demo else 128
valid_ratio = 0.1
reorg_cifar10_data(data_dir, valid_ratio)

transform_train = torchvision.transforms.Compose([
    # 在高度和宽度上将图像放大到40像素的正方形
    torchvision.transforms.Resize(40),
    # 随机裁剪出一个高度和宽度均为40像素的正方形图像,
    # 生成一个面积为原始图像面积0.64~1倍的小正方形,

```

```

# 然后将其缩放为高度和宽度均为32像素的正方形
torchvision.transforms.RandomResizedCrop(32, scale=(0.64, 1.0),
                                          ratio=(1.0, 1.0)),

torchvision.transforms.RandomHorizontalFlip(),
torchvision.transforms.ToTensor(),
# 标准化图像的每个通道
torchvision.transforms.Normalize([0.4914, 0.4822, 0.4465],
                                 [0.2023, 0.1994, 0.2010]))

transform_test = torchvision.transforms.Compose([
    torchvision.transforms.ToTensor(),
    torchvision.transforms.Normalize([0.4914, 0.4822, 0.4465],
                                     [0.2023, 0.1994, 0.2010]))

train_ds, train_valid_ds = [torchvision.datasets.ImageFolder(
    os.path.join(data_dir, 'train_valid_test', folder),
    transform=transform_train) for folder in ['train', 'train_valid']]

valid_ds, test_ds = [torchvision.datasets.ImageFolder(
    os.path.join(data_dir, 'train_valid_test', folder),
    transform=transform_test) for folder in ['valid', 'test']]

train_iter, train_valid_iter = [torch.utils.data.DataLoader(
    dataset, batch_size, shuffle=True, drop_last=True)
    for dataset in (train_ds, train_valid_ds)]

valid_iter = torch.utils.data.DataLoader(valid_ds, batch_size, shuffle=False,
                                         drop_last=True)

test_iter = torch.utils.data.DataLoader(test_ds, batch_size, shuffle=False,
                                         drop_last=False)

def get_net():
    num_classes = 10
    net = native.resnet18(num_classes, 3)
    return net

loss = nn.CrossEntropyLoss(reduction="none")

def train(net, train_iter, valid_iter, num_epochs, lr, wd, devices, lr_period,
          lr_decay):
    trainer = torch.optim.SGD(net.parameters(), lr=lr, momentum=0.9,
                              weight_decay=wd)
    scheduler = torch.optim.lr_scheduler.StepLR(trainer, lr_period, lr_decay)
    num_batches, timer = len(train_iter), native.Timer()
    legend = ['train loss', 'train acc']
    if valid_iter is not None:
        legend.append('valid acc')
    animator = native.Animator(xlabel='epoch', xlim=[1, num_epochs],
                              legend=legend)
    net = nn.DataParallel(net, device_ids=devices).to(devices[0])
    for epoch in range(num_epochs):
        net.train()
        metric = native.Accumulator(3)
        for i, (features, labels) in enumerate(train_iter):
            timer.start()
            l, acc = native.train_batch_ch13(net, features, labels,
                                             loss, trainer, devices)
            metric.add(l, acc, labels.shape[0])
            timer.stop()
            if (i + 1) % (num_batches // 5) == 0 or i == num_batches - 1:
                animator.add(epoch + (i + 1) / num_batches,
                             (metric[0] / metric[2], metric[1] / metric[2],
                              None))
            if valid_iter is not None:
                valid_acc = native.evaluate_accuracy_gpu(net, valid_iter)
                animator.add(epoch + 1, (None, None, valid_acc))
        scheduler.step()
    measures = (f'train loss {metric[0] / metric[2]:.3f}',
                f'train acc {metric[1] / metric[2]:.3f}')
    if valid_iter is not None:
        measures += f', valid acc {valid_acc:.3f}'
    print(measures + f'\n{metric[2] * num_epochs / timer.sum():.1f}'
          f' examples/sec on {str(devices)}')

devices, num_epochs, lr, wd = native.try_all_gpus(), 20, 2e-4, 5e-4

```

```

lr_period, lr_decay, net = 4, 0.9, get_net()
train(net, train_iter, valid_iter, num_epochs, lr, wd, devices, lr_period,
      lr_decay)

net, preds = get_net(), []
train(net, train_valid_iter, None, num_epochs, lr, wd, devices, lr_period,
      lr_decay)

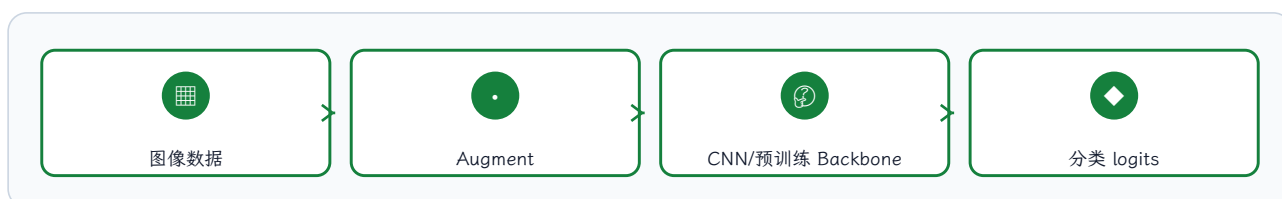
for X, _ in test_iter:
    y_hat = net(X.to(devices[0]))
    preds.extend(y_hat.argmax(dim=1).type(torch.int32).cpu().numpy())
sorted_ids = list(range(1, len(test_ds) + 1))
sorted_ids.sort(key=lambda x: str(x))
df = pd.DataFrame({'id': sorted_ids, 'label': preds})
df['label'] = df['label'].apply(lambda x: train_valid_ds.classes[x])
df.to_csv('submission.csv', index=False)

```

第 13 章 · 图像分类训练管线 · chapters/chapter_13_computer_vision/kaggle_dog.py

章节对应：第 13 章 类别：视觉流程 代码块：12 个

结构图：图像分类训练管线（视觉流程）



第 13 章图像分类实战流程。

阅读注释：阅读顺序：沿结构图从左到右追踪张量，再回到代码寻找对应的 Module 和 forward。

关键点：沿图中的箭头追踪张量形状，重点定位 nn.Module、forward、损失函数和训练循环。

```

"""kaggle-dog: 计算机视觉"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import torch
import torchvision
from torch import nn
import os

native.DATA_HUB['dog_tiny'] = (native.DATA_URL + 'kaggle_dog_tiny.zip',
                              '0cb91d09b814ecd07b50f31f8dcad3e81d6a86d')

# 如果使用Kaggle比赛的完整数据集，请将下面的变量更改为False
demo = True
if demo:
    data_dir = native.download_extract('dog_tiny')

```

```

else:
    data_dir = os.path.join '..', 'data', 'dog-breed-identification')

def reorg_dog_data(data_dir, valid_ratio):
    labels = native.read_csv_labels(os.path.join(data_dir, 'labels.csv'))
    native.reorg_train_valid(data_dir, labels, valid_ratio)
    native.reorg_test(data_dir)

batch_size = 32 if demo else 128
valid_ratio = 0.1
reorg_dog_data(data_dir, valid_ratio)

transform_train = torchvision.transforms.Compose([
    # 随机裁剪图像, 所得图像为原始面积的0.08~1之间, 高宽比在3/4和4/3之间。
    # 然后, 缩放图像以创建224x224的新图像
    torchvision.transforms.RandomResizedCrop(224, scale=(0.08, 1.0),
                                              ratio=(3.0/4.0, 4.0/3.0)),
    torchvision.transforms.RandomHorizontalFlip(),
    # 随机更改亮度, 对比度和饱和度
    torchvision.transforms.ColorJitter(brightness=0.4,
                                       contrast=0.4,
                                       saturation=0.4),

    # 添加随机噪声
    torchvision.transforms.ToTensor(),
    # 标准化图像的每个通道
    torchvision.transforms.Normalize([0.485, 0.456, 0.406],
                                     [0.229, 0.224, 0.225]))

transform_test = torchvision.transforms.Compose([
    torchvision.transforms.Resize(256),
    # 从图像中心裁剪224x224大小的图片
    torchvision.transforms.CenterCrop(224),
    torchvision.transforms.ToTensor(),
    torchvision.transforms.Normalize([0.485, 0.456, 0.406],
                                     [0.229, 0.224, 0.225]))

train_ds, train_valid_ds = [torchvision.datasets.ImageFolder(
    os.path.join(data_dir, 'train_valid_test', folder),
    transform=transform_train) for folder in ['train', 'train_valid']]

valid_ds, test_ds = [torchvision.datasets.ImageFolder(
    os.path.join(data_dir, 'train_valid_test', folder),
    transform=transform_test) for folder in ['valid', 'test']]

train_iter, train_valid_iter = [torch.utils.data.DataLoader(
    dataset, batch_size, shuffle=True, drop_last=True)
    for dataset in (train_ds, train_valid_ds)]

valid_iter = torch.utils.data.DataLoader(valid_ds, batch_size, shuffle=False,
                                         drop_last=True)

test_iter = torch.utils.data.DataLoader(test_ds, batch_size, shuffle=False,
                                         drop_last=False)

def get_net(devices):
    finetune_net = nn.Sequential()
    finetune_net.features = torchvision.models.resnet34(pretrained=True)
    # 定义一个新的输出网络, 共有120个输出类别
    finetune_net.output_new = nn.Sequential(nn.Linear(1000, 256),
                                           nn.ReLU(),
                                           nn.Linear(256, 120))

    # 将模型参数分配给用于计算的CPU或GPU
    finetune_net = finetune_net.to(devices[0])
    # 冻结参数
    for param in finetune_net.features.parameters():
        param.requires_grad = False
    return finetune_net

loss = nn.CrossEntropyLoss(reduction='none')

def evaluate_loss(data_iter, net, devices):
    l_sum, n = 0.0, 0
    for features, labels in data_iter:
        features, labels = features.to(devices[0]), labels.to(devices[0])
        outputs = net(features)
        l = loss(outputs, labels)

```

```

    l_sum += l.sum()
    n += labels.numel()
return (l_sum / n).to('cpu')

```

```

def train(net, train_iter, valid_iter, num_epochs, lr, wd, devices, lr_period,
        lr_decay):
    # 只训练小型自定义输出网络
    net = nn.DataParallel(net, device_ids=devices).to(devices[0])
    trainer = torch.optim.SGD([param for param in net.parameters()
                                if param.requires_grad], lr=lr,
                                momentum=0.9, weight_decay=wd)
    scheduler = torch.optim.lr_scheduler.StepLR(trainer, lr_period, lr_decay)
    num_batches, timer = len(train_iter), native.Timer()
    legend = ['train loss']
    if valid_iter is not None:
        legend.append('valid loss')
    animator = native.Animator(xlabel='epoch', xlim=[1, num_epochs],
                                legend=legend)
    for epoch in range(num_epochs):
        metric = native.Accumulator(2)
        for i, (features, labels) in enumerate(train_iter):
            timer.start()
            features, labels = features.to(devices[0]), labels.to(devices[0])
            trainer.zero_grad()
            output = net(features)
            l = loss(output, labels).sum()
            l.backward()
            trainer.step()
            metric.add(l, labels.shape[0])
            timer.stop()
            if (i + 1) % (num_batches // 5) == 0 or i == num_batches - 1:
                animator.add(epoch + (i + 1) / num_batches,
                              (metric[0] / metric[1], None))
        measures = f'train loss {metric[0] / metric[1]:.3f}'
        if valid_iter is not None:
            valid_loss = evaluate_loss(valid_iter, net, devices)
            animator.add(epoch + 1, (None, valid_loss.detach().cpu()))
        scheduler.step()
        if valid_iter is not None:
            measures += f', valid loss {valid_loss:.3f}'
        print(measures + f'\n{metric[1] * num_epochs / timer.sum():.1f}'
              f' examples/sec on {str(devices)}')

devices, num_epochs, lr, wd = native.try_all_gpus(), 10, 1e-4, 1e-4
lr_period, lr_decay, net = 2, 0.9, get_net(devices)
train(net, train_iter, valid_iter, num_epochs, lr, wd, devices, lr_period,
      lr_decay)

net = get_net(devices)
train(net, train_valid_iter, None, num_epochs, lr, wd, devices, lr_period,
      lr_decay)

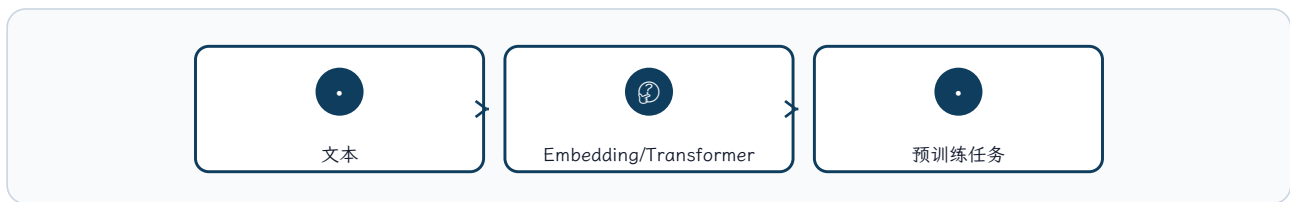
preds = []
for data, label in test_iter:
    output = torch.nn.functional.softmax(net(data.to(devices[0])), dim=1)
    preds.extend(output.cpu().detach().numpy())
ids = sorted(os.listdir(
    os.path.join(data_dir, 'train_valid_test', 'test', 'unknown')))
with open('submission.csv', 'w') as f:
    f.write('id,' + ','.join(train_valid_ds.classes) + '\n')
    for i, output in zip(ids, preds):
        f.write(i.split('.')[0] + ',' + ','.join(
            [str(num) for num in output]) + '\n')

```

第 14 章：自然语言处理：预训练

本章分类：自然语言处理：预训练 章节说明：Word2Vec、子词嵌入和 BERT 预训练。

本章网络/流程总览



第 14 章 · 词嵌入数据管线 · `chapters/chapter_14_nlp_pretraining/word_embedding_dataset.py`

章节对应：第 14 章 类别：数据流程 代码块：18 个

结构图：词嵌入数据管线（数据流程）



第 14 章词向量预训练数据构造。

阅读注释：阅读顺序：先看样本格式，再看预处理与 Dataset，最后看 DataLoader 生成的批次。
关键点：沿图中的箭头追踪张量形状，重点定位 `nn.Module`、`forward`、损失函数和训练循环。

```
"""word-embedding-dataset: 自然语言处理：预训练"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import math
import torch
import os
import random

native.DATA_HUB['ptb'] = (native.DATA_URL + 'ptb.zip',
                          '319d85e578af0cdc590547f26231e4e31cdf1e42')

def read_ptb():
    """将PTB数据集加载到文本行的列表中"""
    data_dir = native.download_extract('ptb')
    # Read the training set.
    with open(os.path.join(data_dir, 'ptb.train.txt')) as f:
```

```

        raw_text = f.read()
        return [line.split() for line in raw_text.split('\n')]

sentences = read_ptb()
f'# sentences数: {len(sentences)}'

vocab = native.Vocab(sentences, min_freq=10)
f'vocab size: {len(vocab)}'

def subsample(sentences, vocab):
    """下采样高频词"""
    # 排除未知词元<unk>
    sentences = [[token for token in line if vocab[token] != vocab.unk]
                  for line in sentences]
    counter = native.count_corpus(sentences)
    num_tokens = sum(counter.values())

    # 如果在下采样期间保留词元, 则返回True
    def keep(token):
        return (random.uniform(0, 1) <
                math.sqrt(1e-4 / counter[token] * num_tokens))

    return ([[token for token in line if keep(token)] for line in sentences],
            counter)

subsampled, counter = subsample(sentences, vocab)

native.show_list_len_pair_hist(
    ['origin', 'subsampled'], '# tokens per sentence',
    'count', sentences, subsampled);

def compare_counts(token):
    return (f'"{token}"的数量: '
            f'之前={sum([l.count(token) for l in sentences])}, '
            f'之后={sum([l.count(token) for l in subsampled])}')

compare_counts('the')

compare_counts('join')

corpus = [vocab[line] for line in subsampled]
corpus[:3]

def get_centers_and_contexts(corpus, max_window_size):
    """返回跳元模型中的中心词和上下文词"""
    centers, contexts = [], []
    for line in corpus:
        # 要形成"中心词-上下文词"对, 每个句子至少需要有2个词
        if len(line) < 2:
            continue
        centers += line
        for i in range(len(line)): # 上下文窗口中间
            window_size = random.randint(1, max_window_size)
            indices = list(range(max(0, i - window_size),
                                   min(len(line), i + 1 + window_size)))

            # 从上下文词中排除中心词
            indices.remove(i)
            contexts.append([line[idx] for idx in indices])
    return centers, contexts

tiny_dataset = [list(range(7)), list(range(7, 10))]
print('数据集', tiny_dataset)
for center, context in zip(*get_centers_and_contexts(tiny_dataset, 2)):
    print('中心词', center, '的上下文词是', context)

all_centers, all_contexts = get_centers_and_contexts(corpus, 5)
f'# "中心词-上下文词对"的数量: {sum([len(contexts) for contexts in all_contexts])}'

class RandomGenerator:
    """根据n个采样权重在{1,...,n}中随机抽取"""
    def __init__(self, sampling_weights):
        # Exclude
        self.population = list(range(1, len(sampling_weights) + 1))
        self.sampling_weights = sampling_weights
        self.candidates = []
        self.i = 0

```

```

def draw(self):
    if self.i == len(self.candidates):
        # 缓存k个随机采样结果
        self.candidates = random.choices(
            self.population, self.sampling_weights, k=10000)
        self.i = 0
    self.i += 1
    return self.candidates[self.i - 1]

generator = RandomGenerator([2, 3, 4])
[generator.draw() for _ in range(10)]

def get_negatives(all_contexts, vocab, counter, K):
    """返回负采样中的噪声词"""
    # 索引为1、2、... (索引0是词表中排除的未知标记)
    sampling_weights = [counter[vocab.to_tokens(i)]**0.75
                        for i in range(1, len(vocab))]
    all_negatives, generator = [], RandomGenerator(sampling_weights)
    for contexts in all_contexts:
        negatives = []
        while len(negatives) < len(contexts) * K:
            neg = generator.draw()
            # 噪声词不能是上下文词
            if neg not in contexts:
                negatives.append(neg)
        all_negatives.append(negatives)
    return all_negatives

all_negatives = get_negatives(all_contexts, vocab, counter, 5)

def batchify(data):
    """返回带有负采样的跳元模型的小批量样本"""
    max_len = max(len(c) + len(n) for _, c, n in data)
    centers, contexts_negatives, masks, labels = [], [], [], []
    for center, context, negative in data:
        cur_len = len(context) + len(negative)
        centers += [center]
        contexts_negatives += \
            [context + negative + [0] * (max_len - cur_len)]
        masks += [[1] * cur_len + [0] * (max_len - cur_len)]
        labels += [[1] * len(context) + [0] * (max_len - len(context))]
    return (native.reshape(native.tensor(centers), (-1, 1)), native.tensor(
        contexts_negatives), native.tensor(masks), native.tensor(labels))

x_1 = (1, [2, 2], [3, 3, 3, 3])
x_2 = (1, [2, 2, 2], [3, 3])
batch = batchify((x_1, x_2))

names = ['centers', 'contexts_negatives', 'masks', 'labels']
for name, data in zip(names, batch):
    print(name, '=', data)

def load_data_ptb(batch_size, max_window_size, num_noise_words):
    """下载PTB数据集，然后将其加载到内存中"""
    num_workers = native.get_data_loader_workers()
    sentences = read_ptb()
    vocab = native.Vocab(sentences, min_freq=10)
    subsampled, counter = subsample(sentences, vocab)
    corpus = [vocab[line] for line in subsampled]
    all_centers, all_contexts = get_centers_and_contexts(
        corpus, max_window_size)
    all_negatives = get_negatives(
        all_contexts, vocab, counter, num_noise_words)

class PTBDataset(torch.utils.data.Dataset):
    def __init__(self, centers, contexts, negatives):
        assert len(centers) == len(contexts) == len(negatives)
        self.centers = centers
        self.contexts = contexts
        self.negatives = negatives

    def __getitem__(self, index):
        return (self.centers[index], self.contexts[index],
                self.negatives[index])

    def __len__(self):

```

```

        return len(self.centers)

dataset = PTBDataset(all_centers, all_contexts, all_negatives)

data_iter = torch.utils.data.DataLoader(
    dataset, batch_size, shuffle=True,
    collate_fn=batchify, num_workers=num_workers)
return data_iter, vocab

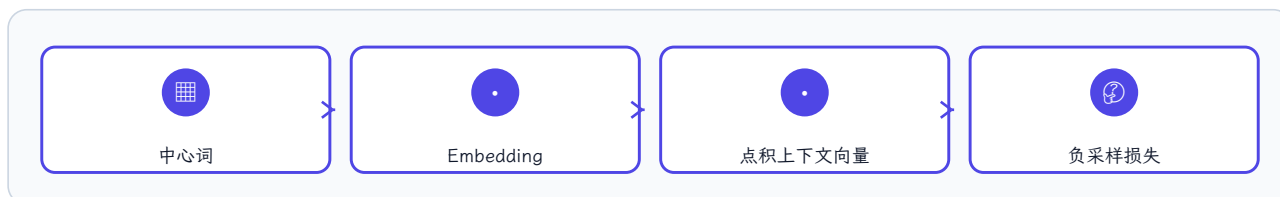
data_iter, vocab = load_data_ptb(512, 5, 5)
for batch in data_iter:
    for name, data in zip(names, batch):
        print(name, 'shape:', data.shape)
    break

```

第 14 章 • Word2Vec / Skip-gram • chapters/chapter_14_nlp_pretraining/word2vec_pretraining.py

章节对应: 第 14 章 类别: 网络结构 代码块: 12 个

结构图: Word2Vec / Skip-gram (网络结构)



第 14 章经典词向量预训练模型。

阅读注释: 阅读顺序: 先看输入与输出, 再看核心层/模块, 最后看损失函数和训练循环。
中心词和上下文词分别查表得到向量, 通过点积与负采样损失学习表示。

```

"""word2vec-pretraining: 自然语言处理: 预训练"""

# 来源: 依据《动手学深度学习 (PyTorch 第二版)》的 D2L 中文 PyTorch 示例整理。
# 改写说明: 已移除 d2l 命名空间, 共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import math
import torch
from torch import nn

batch_size, max_window_size, num_noise_words = 512, 5, 5
data_iter, vocab = native.load_data_ptb(batch_size, max_window_size,
                                       num_noise_words)

embed = nn.Embedding(num_embeddings=vocab.get_vocab_size('token'), embedding_dim=EMBEDDING_DIM)
print(f'Parameter embedding_weight ({embed.weight.shape}, '
      f'dtype={embed.weight.dtype})')

x = native.tensor([1, 2, 3], [4, 5, 6])
embed(x)

```

```

def skip_gram(center, contexts_and_negatives, embed_v, embed_u):
    v = embed_v(center)
    u = embed_u(contexts_and_negatives)
    pred = torch.bmm(v, u.permute(0, 2, 1))
    return pred

skip_gram(torch.ones((2, 1), dtype=torch.long),
          torch.ones((2, 4), dtype=torch.long), embed, embed).shape

class SigmoidBCELoss(nn.Module):
    # 带掩码的二元交叉熵损失
    def __init__(self):
        super().__init__()

    def forward(self, inputs, target, mask=None):
        out = nn.functional.binary_cross_entropy_with_logits(
            inputs, target, weight=mask, reduction="none")
        return out.mean(dim=1)

loss = SigmoidBCELoss()

pred = native.tensor([[1.1, -2.2, 3.3, -4.4]] * 2)
label = native.tensor([[1.0, 0.0, 0.0, 0.0], [0.0, 1.0, 0.0, 0.0]])
mask = native.tensor([[1, 1, 1, 1], [1, 1, 0, 0]])
loss(pred, label, mask) * mask.shape[1] / mask.sum(axis=1)

def sigmd(x):
    return -math.log(1 / (1 + math.exp(-x)))

print(f'{{sigmd(1.1) + sigmd(2.2) + sigmd(-3.3) + sigmd(4.4)) / 4:.4f}}')
print(f'{{sigmd(-1.1) + sigmd(-2.2)) / 2:.4f}}')

embed_size = 100
net = nn.Sequential(nn.Embedding(num_embeddings=len(vocab),
                                embedding_dim=embed_size),
                    nn.Embedding(num_embeddings=len(vocab),
                                embedding_dim=embed_size))

def train(net, data_iter, lr, num_epochs, device=native.try_gpu()):
    def init_weights(m):
        if type(m) == nn.Embedding:
            nn.init.xavier_uniform_(m.weight)
    net.apply(init_weights)
    net = net.to(device)
    optimizer = torch.optim.Adam(net.parameters(), lr=lr)
    animator = native.Animator(xlabel='epoch', ylabel='loss',
                              xlim=[1, num_epochs])
    # 规范化的损失之和, 规范化的损失数
    metric = native.Accumulator(2)
    for epoch in range(num_epochs):
        timer, num_batches = native.Timer(), len(data_iter)
        for i, batch in enumerate(data_iter):
            optimizer.zero_grad()
            center, context_negative, mask, label = [
                data.to(device) for data in batch]

            pred = skip_gram(center, context_negative, net[0], net[1])
            l = (loss(pred.reshape(label.shape).float(), label.float(), mask)
                 / mask.sum(axis=1) * mask.shape[1])
            l.sum().backward()
            optimizer.step()
            metric.add(l.sum(), l.numel())
            if (i + 1) % (num_batches // 5) == 0 or i == num_batches - 1:
                animator.add(epoch + (i + 1) / num_batches,
                             (metric[0] / metric[1],))
        print(f'loss {metric[0] / metric[1]:.3f}, '
              f'{metric[1] / timer.stop():.1f} tokens/sec on {str(device)}')

lr, num_epochs = 0.002, 5
train(net, data_iter, lr, num_epochs)

def get_similar_tokens(query_token, k, embed):
    W = embed.weight.data
    x = W[vocab[query_token]]
    # 计算余弦相似性。增加1e-9以获得数值稳定性
    cos = torch.mv(W, x) / torch.sqrt(torch.sum(W * W, dim=1) *

```

```

        torch.sum(x * x) + 1e-9)
    topk = torch.topk(cos, k=k+1)[1].cpu().numpy().astype('int32')
    for i in topk[1:]: # 删除输入词
        print(f'cosine sim={float(cos[i]):.3f}: {vocab.to_tokens(i)}')

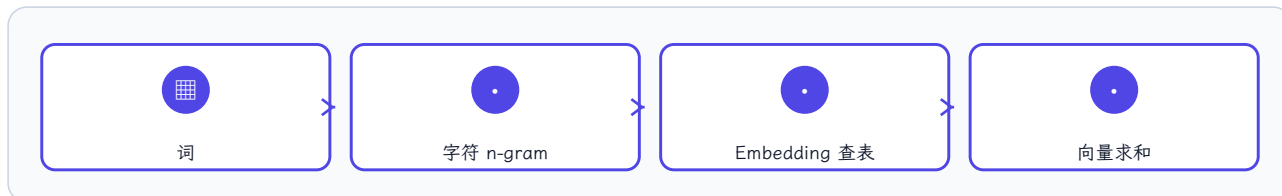
get_similar_tokens('chip', 3, net[0])

```

第 14 章 · 子词嵌入 · chapters/chapter_14_nlp_pretraining/subword_embedding.py

章节对应：第 14 章 类别：网络结构 代码块：9 个

结构图：子词嵌入（网络结构）



第 14 章利用子词信息构造词表示。

阅读注释：阅读顺序：先看输入与输出，再看核心层/模块，最后看损失函数和训练循环。

关键点：沿图中的箭头追踪张量形状，重点定位 nn.Module、forward、损失函数和训练循环。

```

"""subword-embedding: 自然语言处理：预训练"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import collections

symbols = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm',
          'n', 'o', 'p', 'q', 'r', 's', 't', 'u', 'v', 'w', 'x', 'y', 'z',
          '_', '[UNK]']

raw_token_freqs = {'fast_': 4, 'faster_': 3, 'tall_': 5, 'taller_': 4}
token_freqs = {}
for token, freq in raw_token_freqs.items():
    token_freqs[' '.join(list(token))] = raw_token_freqs[token]
token_freqs

def get_max_freq_pair(token_freqs):
    pairs = collections.defaultdict(int)
    for token, freq in token_freqs.items():
        symbols = token.split()
        for i in range(len(symbols) - 1):
            # "pairs"的键是两个连续符号的元组
            pairs[symbols[i], symbols[i + 1]] += freq
    return max(pairs, key=pairs.get) # 具有最大值的"pairs"键

def merge_symbols(max_freq_pair, token_freqs, symbols):
    symbols.append(' '.join(max_freq_pair))

```

```

new_token_freqs = dict()
for token, freq in token_freqs.items():
    new_token = token.replace(' '.join(max_freq_pair),
                              ' '.join(max_freq_pair))
    new_token_freqs[new_token] = token_freqs[token]
return new_token_freqs

num_merges = 10
for i in range(num_merges):
    max_freq_pair = get_max_freq_pair(token_freqs)
    token_freqs = merge_symbols(max_freq_pair, token_freqs, symbols)

print(f'合并# {i+1}: ', max_freq_pair)

print(symbols)

print(list(token_freqs.keys()))

def segment_BPE(tokens, symbols):
    outputs = []
    for token in tokens:
        start, end = 0, len(token)
        cur_output = []
        # 具有符号中可能最长子字的词元段
        while start < len(token) and start < end:
            if token[start: end] in symbols:
                cur_output.append(token[start: end])
                start = end
                end = len(token)
            else:
                end -= 1
        if start < len(token):
            cur_output.append('[UNK]')
        outputs.append(' '.join(cur_output))
    return outputs

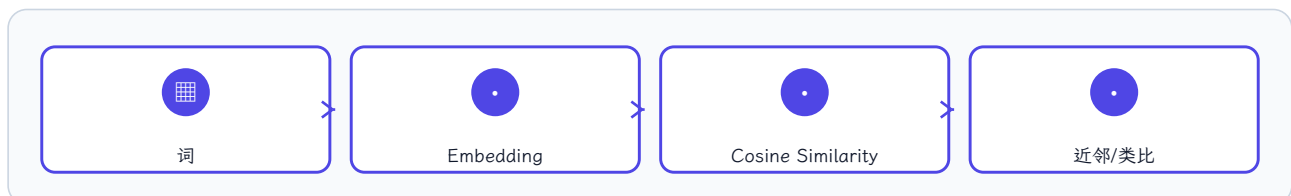
tokens = ['tallest_', 'fatter_']
print(segment_BPE(tokens, symbols))

```

第 14 章 · 词向量相似度/类比 · chapters/chapter_14_nlp_pretraining/similarity_analogy.py

章节对应：第 14 章 类别：推理流程 代码块：16 个

结构图：词向量相似度/类比（推理流程）



第 14 章词向量的推理与可视化。

阅读注释：阅读顺序：沿结构图从左到右追踪张量，再回到代码寻找对应的 Module 和 forward。

关键点：沿图中的箭头追踪张量形状，重点定位 nn.Module、forward、损失函数和训练循环。

```

"""similarity-analogy: 自然语言处理：预训练"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F

```

```

from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import torch
from torch import nn
import os

native.DATA_HUB['glove.6b.50d'] = (native.DATA_URL + 'glove.6B.50d.zip',
                                  '0b8703943ccdb6eb788e6f091b8946e82231bc4d')

native.DATA_HUB['glove.6b.100d'] = (native.DATA_URL + 'glove.6B.100d.zip',
                                   'cd43bfb07e44e6f27cbcc7bc9ae3d80284fdaf5a')

native.DATA_HUB['glove.42b.300d'] = (native.DATA_URL + 'glove.42B.300d.zip',
                                    'b5116e234e9eb9076672cfeabf5469f3eec904fa')

native.DATA_HUB['wiki.en'] = (native.DATA_URL + 'wiki.en.zip',
                              'c1816da3821ae9f43899be655002f6c723e91b88')

class TokenEmbedding:
    """GloVe嵌入"""
    def __init__(self, embedding_name):
        self.idx_to_token, self.idx_to_vec = self._load_embedding(
            embedding_name)
        self.unknown_idx = 0
        self.token_to_idx = {token: idx for idx, token in
                              enumerate(self.idx_to_token)}

    def _load_embedding(self, embedding_name):
        idx_to_token, idx_to_vec = ['<unk>'], []
        data_dir = native.download_extract(embedding_name)
        # GloVe网站: https://nlp.stanford.edu/projects/glove/
        # fastText网站: https://fasttext.cc/
        with open(os.path.join(data_dir, 'vec.txt'), 'r') as f:
            for line in f:
                elems = line.rstrip().split(' ')
                token, elems = elems[0], [float(elem) for elem in elems[1:]]
                # 跳过标题信息, 例如fastText中的首行
                if len(elems) > 1:
                    idx_to_token.append(token)
                    idx_to_vec.append(elems)
        idx_to_vec = [[0] * len(idx_to_vec[0])] + idx_to_vec
        return idx_to_token, native.tensor(idx_to_vec)

    def __getitem__(self, tokens):
        indices = [self.token_to_idx.get(token, self.unknown_idx)
                   for token in tokens]
        vecs = self.idx_to_vec[native.tensor(indices)]
        return vecs

    def __len__(self):
        return len(self.idx_to_token)

glove_6b50d = TokenEmbedding('glove.6b.50d')

len(glove_6b50d)

glove_6b50d.token_to_idx['beautiful'], glove_6b50d.idx_to_token[3367]

def knn(W, x, k):
    # 增加1e-9以获得数值稳定性
    cos = torch.mv(W, x.reshape(-1,)) / (
        torch.sqrt(torch.sum(W * W, axis=1) + 1e-9) *
        torch.sqrt((x * x).sum()))
    _, topk = torch.topk(cos, k=k)
    return topk, [cos[int(i)] for i in topk]

def get_similar_tokens(query_token, k, embed):
    topk, cos = knn(embed.idx_to_vec, embed[[query_token]], k + 1)
    for i, c in zip(topk[1:], cos[1:]): # 排除输入词
        print(f'{embed.idx_to_token[int(i)]}: cosine相似度={float(c):.3f}')

```

```

get_similar_tokens('chip', 3, glove_6b50d)

get_similar_tokens('baby', 3, glove_6b50d)

get_similar_tokens('beautiful', 3, glove_6b50d)

def get_analogy(token_a, token_b, token_c, embed):
    vecs = embed[[token_a, token_b, token_c]]
    x = vecs[1] - vecs[0] + vecs[2]
    topk, cos = knn(embed.idx_to_vec, x, 1)
    return embed.idx_to_token[int(topk[0])] # 删除未知词

get_analogy('man', 'woman', 'son', glove_6b50d)

get_analogy('beijing', 'china', 'tokyo', glove_6b50d)

get_analogy('bad', 'worst', 'big', glove_6b50d)

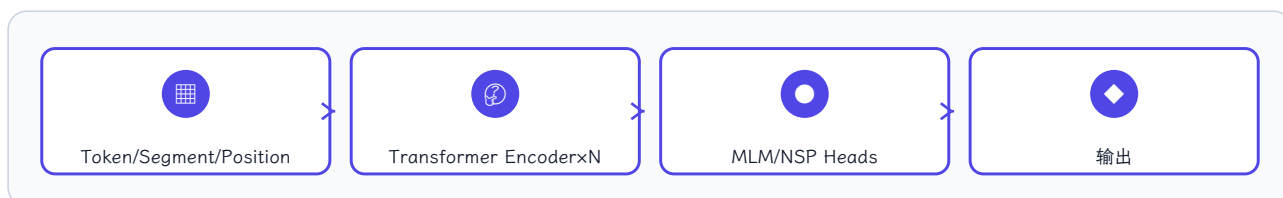
get_analogy('do', 'did', 'go', glove_6b50d)

```

第 14 章 · BERT Encoder · chapters/chapter_14_nlp_pretraining/bert.py

章节对应：第 14 章 类别：网络结构 代码块：12 个

结构图：BERT Encoder（网络结构）



第 14 章双向 Transformer 预训练编码器。

阅读注释：阅读顺序：先看输入与输出，再看核心层/模块，最后看损失函数和训练循环。
BERT 用双向 Transformer Encoder 建模上下文，MLM/NSP 是附加的预训练任务头。

```

"""bert: 自然语言处理：预训练"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import torch
from torch import nn

def get_tokens_and_segments(tokens_a, tokens_b=None):
    """获取输入序列的词元及其片段索引"""
    tokens = ['<cls>'] + tokens_a + ['<sep>']
    # 0和1分别标记片段A和B
    segments = [0] * (len(tokens_a) + 2)

```

```

if tokens_b is not None:
    tokens += tokens_b + ['<sep>']
    segments += [1] * (len(tokens_b) + 1)
return tokens, segments

class BERTEncoder(nn.Module):
    """BERT编码器"""
    def __init__(self, vocab_size, num_hiddens, norm_shape, ffn_num_input,
                  ffn_num_hiddens, num_heads, num_layers, dropout,
                  max_len=1000, key_size=768, query_size=768, value_size=768,
                  **kwargs):
        super(BERTEncoder, self).__init__(**kwargs)
        self.token_embedding = nn.Embedding(vocab_size, num_hiddens)
        self.segment_embedding = nn.Embedding(2, num_hiddens)
        self.blks = nn.Sequential()
        for i in range(num_layers):
            self.blks.add_module(f"{i}", native.EncoderBlock(
                key_size, query_size, value_size, num_hiddens, norm_shape,
                ffn_num_input, ffn_num_hiddens, num_heads, dropout, True))
        # 在BERT中, 位置嵌入是可学习的, 因此我们创建一个足够长的位置嵌入参数
        self.pos_embedding = nn.Parameter(torch.randn(1, max_len,
                                                         num_hiddens))

    def forward(self, tokens, segments, valid_lens):
        # 在以下代码段中, X的形状保持不变: (批量大小, 最大序列长度, num_hiddens)
        X = self.token_embedding(tokens) + self.segment_embedding(segments)
        X = X + self.pos_embedding.data[:, :X.shape[1], :]
        for blk in self.blks:
            X = blk(X, valid_lens)
        return X

vocab_size, num_hiddens, ffn_num_hiddens, num_heads = 10000, 768, 1024, 4
norm_shape, ffn_num_input, num_layers, dropout = [768], 768, 2, 0.2
encoder = BERTEncoder(vocab_size, num_hiddens, norm_shape, ffn_num_input,
                      ffn_num_hiddens, num_heads, num_layers, dropout)

tokens = torch.randint(0, vocab_size, (2, 8))
segments = torch.tensor([[0, 0, 0, 0, 1, 1, 1, 1], [0, 0, 0, 1, 1, 1, 1, 1]])
encoded_X = encoder(tokens, segments, None)
encoded_X.shape

class MaskLM(nn.Module):
    """BERT的掩蔽语言模型任务"""
    def __init__(self, vocab_size, num_hiddens, num_inputs=768, **kwargs):
        super(MaskLM, self).__init__(**kwargs)
        self.mlp = nn.Sequential(nn.Linear(num_inputs, num_hiddens),
                                  nn.ReLU(),
                                  nn.LayerNorm(num_hiddens),
                                  nn.Linear(num_hiddens, vocab_size))

    def forward(self, X, pred_positions):
        num_pred_positions = pred_positions.shape[1]
        pred_positions = pred_positions.reshape(-1)
        batch_size = X.shape[0]
        batch_idx = torch.arange(0, batch_size)
        # 假设batch_size=2, num_pred_positions=3
        # 那么batch_idx是np.array ([0,0,0,1,1,1])
        batch_idx = torch.repeat_interleave(batch_idx, num_pred_positions)
        masked_X = X[batch_idx, pred_positions]
        masked_X = masked_X.reshape((batch_size, num_pred_positions, -1))
        mlm_Y_hat = self.mlp(masked_X)
        return mlm_Y_hat

mlm = MaskLM(vocab_size, num_hiddens)
mlm_positions = torch.tensor([[1, 5, 2], [6, 1, 5]])
mlm_Y_hat = mlm(encoded_X, mlm_positions)
mlm_Y_hat.shape

mlm_Y = torch.tensor([[7, 8, 9], [10, 20, 30]])
loss = nn.CrossEntropyLoss(reduction='none')
mlm_l = loss(mlm_Y_hat.reshape((-1, vocab_size)), mlm_Y.reshape(-1))
mlm_l.shape

class NextSentencePred(nn.Module):
    """BERT的下一句预测任务"""
    def __init__(self, num_inputs, **kwargs):
        super(NextSentencePred, self).__init__(**kwargs)

```

```

        self.output = nn.Linear(num_inputs, 2)

    def forward(self, X):
        # X的形状: (batchsize,num_hiddens)
        return self.output(X)

encoded_X = torch.flatten(encoded_X, start_dim=1)
# NSP的输入形状:(batchsize, num_hiddens)
nsp = NextSentencePred(encoded_X.shape[-1])
nsp_Y_hat = nsp(encoded_X)
nsp_Y_hat.shape

nsp_y = torch.tensor([0, 1])
nsp_l = loss(nsp_Y_hat, nsp_y)
nsp_l.shape

class BERTModel(nn.Module):
    """BERT模型"""
    def __init__(self, vocab_size, num_hiddens, norm_shape, ffn_num_input,
                  ffn_num_hiddens, num_heads, num_layers, dropout,
                  max_len=1000, key_size=768, query_size=768, value_size=768,
                  hid_in_features=768, mlm_in_features=768,
                  nsp_in_features=768):
        super(BERTModel, self).__init__()
        self.encoder = BERTEncoder(vocab_size, num_hiddens, norm_shape,
                                   ffn_num_input, ffn_num_hiddens, num_heads, num_layers,
                                   dropout, max_len=max_len, key_size=key_size,
                                   query_size=query_size, value_size=value_size)
        self.hidden = nn.Sequential(nn.Linear(hid_in_features, num_hiddens),
                                    nn.Tanh())
        self.mlm = MaskLM(vocab_size, num_hiddens, mlm_in_features)
        self.nsp = NextSentencePred(nsp_in_features)

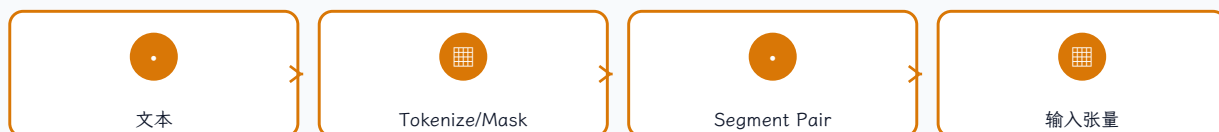
    def forward(self, tokens, segments, valid_lens=None,
                pred_positions=None):
        encoded_X = self.encoder(tokens, segments, valid_lens)
        if pred_positions is not None:
            mlm_Y_hat = self.mlm(encoded_X, pred_positions)
        else:
            mlm_Y_hat = None
        # 用于下一句预测的多层感知机分类器的隐藏层, 0是"<cls>"标记的索引
        nsp_Y_hat = self.nsp(self.hidden(encoded_X[:, 0, :]))
        return encoded_X, mlm_Y_hat, nsp_Y_hat

```

第 14 章 · BERT 预训练数据管线 · [chapters/chapter_14_nlp_pretraining/bert_dataset.py](#)

章节对应: 第 14 章 类别: 数据流程 代码块: 11 个

结构图: BERT 预训练数据管线 (数据流程)



第 14 章构造遮蔽语言模型和下一句预测样本。

阅读注释: 阅读顺序: 先看样本格式, 再看预处理与 Dataset, 最后看 DataLoader 生成的批次。
 关键点: 沿图中的箭头追踪张量形状, 重点定位 nn.Module、forward、损失函数和训练循环。

```

"""bert-dataset: 自然语言处理: 预训练"""

# 来源: 依据《动手学深度学习 (PyTorch 第二版)》的 D2L 中文 PyTorch 示例整理。
# 改写说明: 已移除 d2l 命名空间, 共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random

```

```

import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import os
import random
import torch

native.DATA_HUB['wikitext-2'] = (
    'https://s3.amazonaws.com/research.metamind.io/wikitext/'
    'wikitext-2-v1.zip', '3c914d17d80b1459be871a5039ac23e752a53cbe')

def _read_wiki(data_dir):
    file_name = os.path.join(data_dir, 'wiki.train.tokens')
    with open(file_name, 'r') as f:
        lines = f.readlines()
    # 大写字母转换为小写字母
    paragraphs = [line.strip().lower().split(' . ')]
        for line in lines if len(line.split(' . ')) >= 2]
    random.shuffle(paragraphs)
    return paragraphs

def _get_next_sentence(sentence, next_sentence, paragraphs):
    if random.random() < 0.5:
        is_next = True
    else:
        # paragraphs是三重列表的嵌套
        next_sentence = random.choice(random.choice(paragraphs))
        is_next = False
    return sentence, next_sentence, is_next

def _get_nsp_data_from_paragraph(paragraph, paragraphs, vocab, max_len):
    nsp_data_from_paragraph = []
    for i in range(len(paragraph) - 1):
        tokens_a, tokens_b, is_next = _get_next_sentence(
            paragraph[i], paragraph[i + 1], paragraphs)
        # 考虑1个'<cls>'词元和2个'<sep>'词元
        if len(tokens_a) + len(tokens_b) + 3 > max_len:
            continue
        tokens, segments = native.get_tokens_and_segments(tokens_a, tokens_b)
        nsp_data_from_paragraph.append((tokens, segments, is_next))
    return nsp_data_from_paragraph

def _replace_mlm_tokens(tokens, candidate_pred_positions, num_mlm_preds,
                        vocab):
    # 为遮蔽语言模型的输入创建新的词元副本，其中输入可能包含替换的"<mask>"或随机词元
    mlm_input_tokens = [token for token in tokens]
    pred_positions_and_labels = []
    # 打乱后用于在遮蔽语言模型任务中获取15%的随机词元进行预测
    random.shuffle(candidate_pred_positions)
    for mlm_pred_position in candidate_pred_positions:
        if len(pred_positions_and_labels) >= num_mlm_preds:
            break
        masked_token = None
        # 80%的时间：将词替换为"<mask>"词元
        if random.random() < 0.8:
            masked_token = '<mask>'
        else:
            # 10%的时间：保持词不变
            if random.random() < 0.5:
                masked_token = tokens[mlm_pred_position]
            # 10%的时间：用随机词替换该词
            else:
                masked_token = random.choice(vocab.idx_to_token)
        mlm_input_tokens[mlm_pred_position] = masked_token
        pred_positions_and_labels.append(

```

```

        (mlm_pred_position, tokens[mlm_pred_position]))
    return mlm_input_tokens, pred_positions_and_labels

def _get_mlm_data_from_tokens(tokens, vocab):
    candidate_pred_positions = []
    # tokens是一个字符串列表
    for i, token in enumerate(tokens):
        # 在遮蔽语言模型任务中不会预测特殊词元
        if token in ['<cls>', '<sep>']:
            continue
        candidate_pred_positions.append(i)
    # 遮蔽语言模型任务中预测15%的随机词元
    num_mlm_preds = max(1, round(len(tokens) * 0.15))
    mlm_input_tokens, pred_positions_and_labels = _replace_mlm_tokens(
        tokens, candidate_pred_positions, num_mlm_preds, vocab)
    pred_positions_and_labels = sorted(pred_positions_and_labels,
                                       key=lambda x: x[0])
    pred_positions = [v[0] for v in pred_positions_and_labels]
    mlm_pred_labels = [v[1] for v in pred_positions_and_labels]
    return vocab[mlm_input_tokens], pred_positions, vocab[mlm_pred_labels]

def _pad_bert_inputs(examples, max_len, vocab):
    max_num_mlm_preds = round(max_len * 0.15)
    all_token_ids, all_segments, valid_lens, = [], [], []
    all_pred_positions, all_mlm_weights, all_mlm_labels = [], [], []
    nsp_labels = []
    for (token_ids, pred_positions, mlm_pred_label_ids, segments,
         is_next) in examples:
        all_token_ids.append(torch.tensor(token_ids + [vocab['<pad>']] * (
            max_len - len(token_ids))), dtype=torch.long))
        all_segments.append(torch.tensor(segments + [0] * (
            max_len - len(segments))), dtype=torch.long))
    # valid_lens不包括'<pad>'的计数
    valid_lens.append(torch.tensor(len(token_ids), dtype=torch.float32))
    all_pred_positions.append(torch.tensor(pred_positions + [0] * (
        max_num_mlm_preds - len(pred_positions))), dtype=torch.long))
    # 填充词元的预测将通过乘以0权重在损失中过滤掉
    all_mlm_weights.append(
        torch.tensor([1.0] * len(mlm_pred_label_ids) + [0.0] * (
            max_num_mlm_preds - len(pred_positions)),
            dtype=torch.float32))
    all_mlm_labels.append(torch.tensor(mlm_pred_label_ids + [0] * (
        max_num_mlm_preds - len(mlm_pred_label_ids)), dtype=torch.long))
    nsp_labels.append(torch.tensor(is_next, dtype=torch.long))
    return (all_token_ids, all_segments, valid_lens, all_pred_positions,
            all_mlm_weights, all_mlm_labels, nsp_labels)

class _WikiTextDataset(torch.utils.data.Dataset):
    def __init__(self, paragraphs, max_len):
        # 输入paragraphs[i]是代表段落的句子字符串列表;
        # 而输出paragraphs[i]是代表段落的句子列表, 其中每个句子都是词元列表
        paragraphs = [native.tokenize(
            paragraph, token='word') for paragraph in paragraphs]
        sentences = [sentence for paragraph in paragraphs
                     for sentence in paragraph]
        self.vocab = native.Vocab(sentences, min_freq=5, reserved_tokens=[
            '<pad>', '<mask>', '<cls>', '<sep>'])
    # 获取下一句子预测任务的数据
    examples = []
    for paragraph in paragraphs:
        examples.extend(_get_nsp_data_from_paragraph(
            paragraph, paragraphs, self.vocab, max_len))
    # 获取遮蔽语言模型任务的数据
    examples = [(_get_mlm_data_from_tokens(tokens, self.vocab)
                 + (segments, is_next))
                for tokens, segments, is_next in examples]
    # 填充输入
    (self.all_token_ids, self.all_segments, self.valid_lens,
     self.all_pred_positions, self.all_mlm_weights,
     self.all_mlm_labels, self.nsp_labels) = _pad_bert_inputs(
        examples, max_len, self.vocab)

    def __getitem__(self, idx):
        return (self.all_token_ids[idx], self.all_segments[idx],
                self.valid_lens[idx], self.all_pred_positions[idx],
                self.all_mlm_weights[idx], self.all_mlm_labels[idx],
                self.nsp_labels[idx])

```

```

def __len__(self):
    return len(self.all_token_ids)

def load_data_wiki(batch_size, max_len):
    """加载WikiText-2数据集"""
    num_workers = native.get_dataloader_workers()
    data_dir = native.download_extract('wikitext-2', 'wikitext-2')
    paragraphs = _read_wiki(data_dir)
    train_set = _WikiTextDataset(paragraphs, max_len)
    train_iter = torch.utils.data.DataLoader(train_set, batch_size,
                                             shuffle=True, num_workers=num_workers)
    return train_iter, train_set.vocab

batch_size, max_len = 512, 64
train_iter, vocab = load_data_wiki(batch_size, max_len)

for (tokens_X, segments_X, valid_lens_x, pred_positions_X, mlm_weights_X,
     mlm_Y, nsp_y) in train_iter:
    print(tokens_X.shape, segments_X.shape, valid_lens_x.shape,
          pred_positions_X.shape, mlm_weights_X.shape, mlm_Y.shape,
          nsp_y.shape)
    break

len(vocab)

```

第 14 章 · BERT 预训练 · chapters/chapter_14_nlp_pretraining/bert_pretraining.py

章节对应：第 14 章 类别：网络结构 代码块：9 个

结构图：BERT 预训练（网络结构）



第 14 章 BERT 的预训练任务与损失。

阅读注释：阅读顺序：先看输入与输出，再看核心层/模块，最后看损失函数和训练循环。
先构造带 Mask 的输入，再由 MLM 和 NSP 两个头产生联合损失。

```

"""bert-pretraining: 自然语言处理：预训练"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import torch
from torch import nn

```

```

batch_size, max_len = 512, 64
train_iter, vocab = native.load_data_wiki(batch_size, max_len)

net = native.BERTModel(len(vocab), num_hiddens=128, norm_shape=[128],
                        ffn_num_input=128, ffn_num_hiddens=256, num_heads=2,
                        num_layers=2, dropout=0.2, key_size=128, query_size=128,
                        value_size=128, hid_in_features=128, mlm_in_features=128,
                        nsp_in_features=128)
devices = native.try_all_gpus()
loss = nn.CrossEntropyLoss()

def _get_batch_loss_bert(net, loss, vocab_size, tokens_X,
                        segments_X, valid_lens_x,
                        pred_positions_X, mlm_weights_X,
                        mlm_Y, nsp_y):
    # 前向传播
    _, mlm_Y_hat, nsp_Y_hat = net(tokens_X, segments_X,
                                   valid_lens_x.reshape(-1),
                                   pred_positions_X)

    # 计算遮蔽语言模型损失
    mlm_l = loss(mlm_Y_hat.reshape(-1, vocab_size), mlm_Y.reshape(-1)) * \
        mlm_weights_X.reshape(-1, 1)
    mlm_l = mlm_l.sum() / (mlm_weights_X.sum() + 1e-8)
    # 计算下一句子预测任务的损失
    nsp_l = loss(nsp_Y_hat, nsp_y)
    l = mlm_l + nsp_l
    return mlm_l, nsp_l, l

def train_bert(train_iter, net, loss, vocab_size, devices, num_steps):
    net = nn.DataParallel(net, device_ids=devices).to(devices[0])

    trainer = torch.optim.Adam(net.parameters(), lr=0.01)
    step, timer = 0, native.Timer()
    animator = native.Animator(xlabel='step', ylabel='loss',
                               xlim=[1, num_steps], legend=['mlm', 'nsp'])
    # 遮蔽语言模型损失的和, 下一句预测任务损失的和, 句子对的数量, 计数
    metric = native.Accumulator(4)
    num_steps_reached = False
    while step < num_steps and not num_steps_reached:
        for tokens_X, segments_X, valid_lens_x, pred_positions_X, \
            mlm_weights_X, mlm_Y, nsp_y in train_iter:
            tokens_X = tokens_X.to(devices[0])
            segments_X = segments_X.to(devices[0])
            valid_lens_x = valid_lens_x.to(devices[0])
            pred_positions_X = pred_positions_X.to(devices[0])
            mlm_weights_X = mlm_weights_X.to(devices[0])
            mlm_Y, nsp_y = mlm_Y.to(devices[0]), nsp_y.to(devices[0])
            trainer.zero_grad()
            timer.start()
            mlm_l, nsp_l, l = _get_batch_loss_bert(
                net, loss, vocab_size, tokens_X, segments_X, valid_lens_x,
                pred_positions_X, mlm_weights_X, mlm_Y, nsp_y)
            l.backward()
            trainer.step()
            metric.add(mlm_l, nsp_l, tokens_X.shape[0], 1)
            timer.stop()
            animator.add(step + 1,
                         (metric[0] / metric[3], metric[1] / metric[3]))
            step += 1
        if step == num_steps:
            num_steps_reached = True
            break

    print(f'MLM loss {metric[0] / metric[3]:.3f}, '
          f'NSP loss {metric[1] / metric[3]:.3f}')
    print(f'{metric[2] / timer.sum():.1f} sentence pairs/sec on '
          f'{str(devices)}')

train_bert(train_iter, net, loss, len(vocab), devices, 50)

def get_bert_encoding(net, tokens_a, tokens_b=None):
    tokens, segments = native.get_tokens_and_segments(tokens_a, tokens_b)
    token_ids = torch.tensor(vocab[tokens], device=devices[0]).unsqueeze(0)
    segments = torch.tensor(segments, device=devices[0]).unsqueeze(0)
    valid_len = torch.tensor(len(tokens), device=devices[0]).unsqueeze(0)
    encoded_X, _, _ = net(token_ids, segments, valid_len)
    return encoded_X

```

```

tokens_a = ['a', 'crane', 'is', 'flying']
encoded_text = get_bert_encoding(net, tokens_a)
# 词元: '<cls>', 'a', 'crane', 'is', 'flying', '<sep>'
encoded_text_cls = encoded_text[:, 0, :]
encoded_text_crane = encoded_text[:, 2, :]
encoded_text.shape, encoded_text_cls.shape, encoded_text_crane[0][:3]

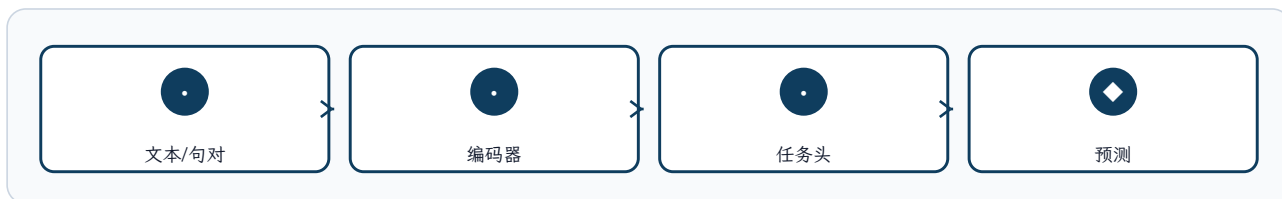
tokens_a, tokens_b = ['a', 'crane', 'driver', 'came'], ['he', 'just', 'left']
encoded_pair = get_bert_encoding(net, tokens_a, tokens_b)
# 词元: '<cls>', 'a', 'crane', 'driver', 'came', '<sep>', 'he', 'just',
# 'left', '<sep>'
encoded_pair_cls = encoded_pair[:, 0, :]
encoded_pair_crane = encoded_pair[:, 2, :]
encoded_pair.shape, encoded_pair_cls.shape, encoded_pair_crane[0][:3]

```

第 15 章：自然语言处理：应用

本章分类：自然语言处理：应用 章节说明：情感分析和自然语言推断的 RNN、CNN、Attention、BERT 实现。

本章网络/流程总览

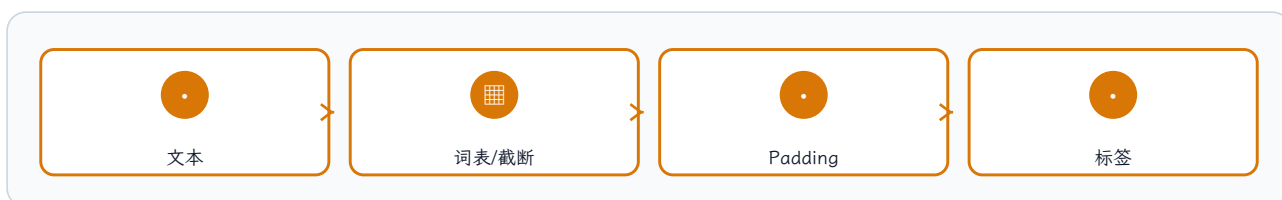


第 15 章 · 情感分析数据管线 ·

[chapters/chapter_15_nlp_applications/sentiment_analysis_and_dataset.py](#)

章节对应：第 15 章 类别：数据流程 代码块：8 个

结构图：情感分析数据管线（数据流程）



第 15 章情感分类数据准备。

阅读注释：阅读顺序：先看样本格式，再看预处理与 Dataset，最后看 DataLoader 生成的批次。

关键点：沿图中的箭头追踪张量形状，重点定位 nn.Module、forward、损失函数和训练循环。

```
"""sentiment-analysis-and-dataset: 自然语言处理：应用"""
```

```
# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
```

```
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
```

```
from pathlib import Path
```

```
import sys
```

```
_ROOT = Path(__file__).resolve().parents[1]
```

```
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))
```

```
import math
```

```
import random
```

```
import time
```

```
import numpy as np
```

```
import pandas as pd
```

```
import torch
```

```
from torch import nn
```

```
from torch.nn import functional as F
```

```
from torch.utils import data
```

```
import torchvision
```

```
from torchvision import transforms
```

```
from PIL import Image
```

```
from matplotlib import pyplot as plt
```

```
from IPython import display
```

```
import common as native
```

```
import common as native
```

```
import torch
```

```
from torch import nn
```

```
import os
```

```
native.DATA_HUB['aclImdb'] = (
```

```
    'http://ai.stanford.edu/~amaas/data/sentiment/aclImdb_v1.tar.gz',
```

```
    '01ada507287d82875905620988597833ad4e0903')
```

```
data_dir = native.download_extract('aclImdb', 'aclImdb')
```

```
def read_imdb(data_dir, is_train):
```

```

"""读取IMDb评论数据集文本序列和标签"""
data, labels = [], []
for label in ('pos', 'neg'):
    folder_name = os.path.join(data_dir, 'train' if is_train else 'test',
                                label)
    for file in os.listdir(folder_name):
        with open(os.path.join(folder_name, file), 'rb') as f:
            review = f.read().decode('utf-8').replace('\n', '')
            data.append(review)
            labels.append(1 if label == 'pos' else 0)
return data, labels

train_data = read_imdb(data_dir, is_train=True)
print('训练集数目: ', len(train_data[0]))
for x, y in zip(train_data[0][:3], train_data[1][:3]):
    print('标签: ', y, 'review:', x[0:60])

train_tokens = native.tokenize(train_data[0], token='word')
vocab = native.Vocab(train_tokens, min_freq=5, reserved_tokens=['<pad>'])

native.set_figsize()
native.plt.xlabel('# tokens per review')
native.plt.ylabel('count')
native.plt.hist([len(line) for line in train_tokens], bins=range(0, 1000, 50));

num_steps = 500 # 序列长度
train_features = native.tensor([native.truncate_pad(
    vocab[line], num_steps, vocab['<pad>']) for line in train_tokens])
print(train_features.shape)

train_iter = native.load_array((train_features,
    torch.tensor(train_data[1])), 64)

for X, y in train_iter:
    print('X:', X.shape, 'y:', y.shape)
    break
print('小批量数目: ', len(train_iter))

def load_data_imdb(batch_size, num_steps=500):
    """返回数据迭代器和IMDb评论数据集的词汇"""
    data_dir = native.download_extract('aclImdb', 'aclImdb')
    train_data = read_imdb(data_dir, True)
    test_data = read_imdb(data_dir, False)
    train_tokens = native.tokenize(train_data[0], token='word')
    test_tokens = native.tokenize(test_data[0], token='word')
    vocab = native.Vocab(train_tokens, min_freq=5)
    train_features = torch.tensor([native.truncate_pad(
        vocab[line], num_steps, vocab['<pad>']) for line in train_tokens])
    test_features = torch.tensor([native.truncate_pad(
        vocab[line], num_steps, vocab['<pad>']) for line in test_tokens])
    train_iter = native.load_array((train_features, torch.tensor(train_data[1])),
        batch_size)
    test_iter = native.load_array((test_features, torch.tensor(test_data[1])),
        batch_size,
        is_train=False)
    return train_iter, test_iter, vocab

```

第 15 章 · BiRNN 情感分类器 · chapters/chapter_15_nlp_applications/sentiment_analysis_rnn.py

章节对应: 第 15 章 类别: 网络结构 代码块: 11 个

结构图: BiRNN 情感分类器 (网络结构)



第 15 章基于双向循环网络的情感分类。

阅读注释: 阅读顺序: 先看输入与输出, 再看核心层/模块, 最后看损失函数和训练循环。

双向编码后拼接首尾状态, 最后由线性层把句子表示映射到情感类别。

```

"""sentiment-analysis-rnn: 自然语言处理: 应用"""

# 来源: 依据《动手学深度学习 (PyTorch 第二版)》的 D2L 中文 PyTorch 示例整理。
# 改写说明: 已移除 d2l 命名空间, 共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import torch
from torch import nn

batch_size = 64
train_iter, test_iter, vocab = native.load_data_imdb(batch_size)

class BiRNN(nn.Module):
    def __init__(self, vocab_size, embed_size, num_hiddens,
                 num_layers, **kwargs):
        super(BiRNN, self).__init__(**kwargs)
        self.embedding = nn.Embedding(vocab_size, embed_size)
        # 将bidirectional设置为True以获取双向循环神经网络
        self.encoder = nn.LSTM(embed_size, num_hiddens, num_layers=num_layers,
                               bidirectional=True)
        self.decoder = nn.Linear(4 * num_hiddens, 2)

    def forward(self, inputs):
        # inputs的形状是 (批量大小, 时间步数)
        # 因为长短期记忆网络要求其输入的维度是时间维,
        # 所以在获得词元表示之前, 输入会被转置。
        # 输出形状为 (时间步数, 批量大小, 词向量维度)
        embeddings = self.embedding(inputs.T)
        self.encoder.flatten_parameters()
        # 返回上一个隐藏层在不同时间步的隐状态,
        # outputs的形状是 (时间步数, 批量大小, 2*隐藏单元数)
        outputs, _ = self.encoder(embeddings)
        # 连结初始和最终时间步的隐状态, 作为全连接层的输入,
        # 其形状为 (批量大小, 4*隐藏单元数)
        encoding = torch.cat((outputs[0], outputs[-1]), dim=1)
        outs = self.decoder(encoding)
        return outs

embed_size, num_hiddens, num_layers = 100, 100, 2

devices = native.try_all_gpus()
net = BiRNN(len(vocab), embed_size, num_hiddens, num_layers)

def init_weights(m):
    if type(m) == nn.Linear:
        nn.init.xavier_uniform_(m.weight)
    if type(m) == nn.LSTM:
        for param in m._flat_weights_names:
            if "weight" in param:
                nn.init.xavier_uniform_(m._parameters[param])
net.apply(init_weights);

glove_embedding = native.TokenEmbedding('glove.6b.100d')

embeds = glove_embedding[vocab.idx_to_token]
embeds.shape

```

```

net.embedding.weight.data.copy_(embeds)
net.embedding.weight.requires_grad = False

lr, num_epochs = 0.01, 5
trainer = torch.optim.Adam(net.parameters(), lr=lr)
loss = nn.CrossEntropyLoss(reduction="none")
native.train_ch13(net, train_iter, test_iter, loss, trainer, num_epochs,
                  devices)

def predict_sentiment(net, vocab, sequence):
    """预测文本序列的情感"""
    sequence = torch.tensor(vocab[sequence.split()], device=native.try_gpu())
    label = torch.argmax(net(sequence.reshape(1, -1)), dim=1)
    return 'positive' if label == 1 else 'negative'

predict_sentiment(net, vocab, 'this movie is so great')
predict_sentiment(net, vocab, 'this movie is so bad')

```

第 15 章 • TextCNN 情感分类器 • chapters/chapter_15_nlp_applications/sentiment_analysis_cnn.py

章节对应：第 15 章 类别：网络结构 代码块：10 个

结构图：TextCNN 情感分类器（网络结构）



第 15 章基于卷积的文本分类。

阅读注释：阅读顺序：先看输入与输出，再看核心层/模块，最后看损失函数和训练循环。
不同宽度的 Conv1d 捕捉不同长度的 n-gram，MaxPool 后汇总成句子表示。

```

"""sentiment-analysis-cnn: 自然语言处理：应用"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import torch
from torch import nn

batch_size = 64
train_iter, test_iter, vocab = native.load_data_imdb(batch_size)

def corr1d(X, K):
    w = K.shape[0]
    Y = native.zeros((X.shape[0] - w + 1))
    for i in range(Y.shape[0]):

```

```

        Y[i] = (X[i: i + w] * K).sum()
    return Y

X, K = native.tensor([0, 1, 2, 3, 4, 5, 6]), native.tensor([1, 2])
corr1d(X, K)

def corr1d_multi_in(X, K):
    # 首先, 遍历'X'和'K'的第0维 (通道维)。然后, 把它们加在一起
    return sum(corr1d(x, k) for x, k in zip(X, K))

X = native.tensor([[0, 1, 2, 3, 4, 5, 6],
                   [1, 2, 3, 4, 5, 6, 7],
                   [2, 3, 4, 5, 6, 7, 8]])
K = native.tensor([1, 2], [3, 4], [-1, -3])
corr1d_multi_in(X, K)

class TextCNN(nn.Module):
    def __init__(self, vocab_size, embed_size, kernel_sizes, num_channels,
                 **kwargs):
        super(TextCNN, self).__init__(**kwargs)
        self.embedding = nn.Embedding(vocab_size, embed_size)
        # 这个嵌入层不需要训练
        self.constant_embedding = nn.Embedding(vocab_size, embed_size)

        self.dropout = nn.Dropout(0.5)
        self.decoder = nn.Linear(sum(num_channels), 2)
        # 最大时间汇聚层没有参数, 因此可以共享此实例
        self.pool = nn.AdaptiveAvgPool1d(1)
        self.relu = nn.ReLU()
        # 创建多个一维卷积层
        self.convs = nn.ModuleList()
        for c, k in zip(num_channels, kernel_sizes):
            self.convs.append(nn.Conv1d(2 * embed_size, c, k))

    def forward(self, inputs):
        # 沿着向量维度将两个嵌入层连结起来,
        # 每个嵌入层的输出形状都是 (批量大小, 词元数量, 词元向量维度) 连结起来
        embeddings = torch.cat((
            self.embedding(inputs), self.constant_embedding(inputs)), dim=2)
        # 根据一维卷积层的输入格式, 重新排列张量, 以便通道作为第2维
        embeddings = embeddings.permute(0, 2, 1)
        # 每个一维卷积层在最大时间汇聚层合并后, 获得的张量形状是 (批量大小, 通道数, 1)
        # 删除最后一个维度并沿通道维度连结
        encoding = torch.cat([
            torch.squeeze(self.relu(self.pool(conv(embeddings))), dim=-1)
            for conv in self.convs], dim=1)
        outputs = self.decoder(self.dropout(encoding))
        return outputs

embed_size, kernel_sizes, num_channels = 100, [3, 4, 5], [100, 100, 100]
devices = native.try_all_gpus()
net = TextCNN(len(vocab), embed_size, kernel_sizes, num_channels)

def init_weights(m):
    if type(m) in (nn.Linear, nn.Conv1d):
        nn.init.xavier_uniform_(m.weight)

net.apply(init_weights);

glove_embedding = native.TokenEmbedding('glove.6b.100d')
embeds = glove_embedding[vocab.idx_to_token]
net.embedding.weight.data.copy_(embeds)
net.constant_embedding.weight.data.copy_(embeds)
net.constant_embedding.weight.requires_grad = False

lr, num_epochs = 0.001, 5
trainer = torch.optim.Adam(net.parameters(), lr=lr)
loss = nn.CrossEntropyLoss(reduction="none")
native.train_ch13(net, train_iter, test_iter, loss, trainer, num_epochs, devices)

native.predict_sentiment(net, vocab, 'this movie is so great')

native.predict_sentiment(net, vocab, 'this movie is so bad')

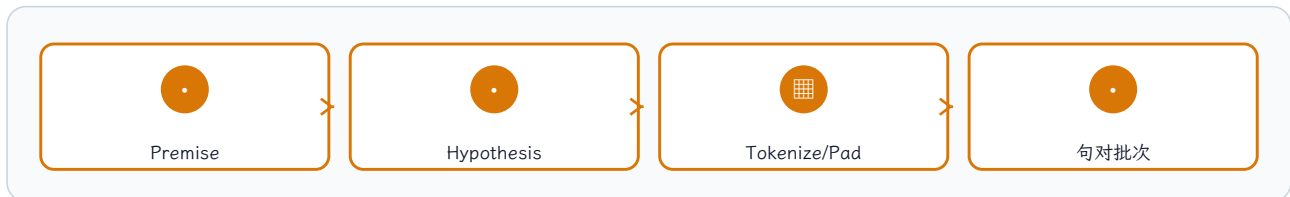
```

第 15 章 · 自然语言推断数据管线 ·

chapters/chapter_15_nlp_applications/natural_language_inference_and_dataset.py

章节对应：第 15 章 类别：数据流程 代码块：8 个

结构图：自然语言推断数据管线（数据流程）



第 15 章自然语言推断句对数据准备。

阅读注释：阅读顺序：先看样本格式，再看预处理与 Dataset，最后看 DataLoader 生成的批次。

关键点：沿图中的箭头追踪流量形状，重点定位 nn.Module、forward、损失函数和训练循环。

```
"""natural-language-inference-and-dataset: 自然语言处理：应用"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import torch
from torch import nn
import os
import re

native.DATA_HUB['SNLI'] = (
    'https://nlp.stanford.edu/projects/snli/snli_1.0.zip',
    '9fcde07509c7e87ec61c640c1b2753d9041758e4')

data_dir = native.download_extract('SNLI')

def read_snli(data_dir, is_train):
    """将SNLI数据集解析为前提、假设和标签"""
    def extract_text(s):
        # 删除我们不会使用的信息
        s = re.sub('\\(', '', s)
        s = re.sub('\\)', '', s)
        # 用一个空格替换两个或多个连续的空格
        s = re.sub('\\s{2,}', ' ', s)
        return s.strip()
    label_set = {'entailment': 0, 'contradiction': 1, 'neutral': 2}
    file_name = os.path.join(data_dir, 'snli_1.0_train.txt'
                              if is_train else 'snli_1.0_test.txt')
    with open(file_name, 'r') as f:
        rows = [row.split('\t') for row in f.readlines()[1:]]
        premises = [extract_text(row[1]) for row in rows if row[0] in label_set]
        hypotheses = [extract_text(row[2]) for row in rows if row[0] \
                       in label_set]
```

```

        labels = [label_set[row[0]] for row in rows if row[0] in label_set]
        return premises, hypotheses, labels

train_data = read_snli(data_dir, is_train=True)
for x0, x1, y in zip(train_data[0][:3], train_data[1][:3], train_data[2][:3]):
    print('前提: ', x0)
    print('假设: ', x1)
    print('标签: ', y)

test_data = read_snli(data_dir, is_train=False)
for data in [train_data, test_data]:
    print([row for row in data[2]].count(i) for i in range(3)])

class SNLIDataset(torch.utils.data.Dataset):
    """用于加载SNLI数据集的自定义数据集"""
    def __init__(self, dataset, num_steps, vocab=None):
        self.num_steps = num_steps
        all_premise_tokens = native.tokenize(dataset[0])
        all_hypothesis_tokens = native.tokenize(dataset[1])
        if vocab is None:
            self.vocab = native.Vocab(all_premise_tokens + \
                                     all_hypothesis_tokens, min_freq=5, reserved_tokens=['<pad>'])
        else:
            self.vocab = vocab
        self.premises = self._pad(all_premise_tokens)
        self.hypotheses = self._pad(all_hypothesis_tokens)
        self.labels = torch.tensor(dataset[2])
        print('read ' + str(len(self.premises)) + ' examples')

    def _pad(self, lines):
        return torch.tensor([native.truncate_pad(
            self.vocab[line], self.num_steps, self.vocab['<pad>'])
            for line in lines])

    def __getitem__(self, idx):
        return (self.premises[idx], self.hypotheses[idx]), self.labels[idx]

    def __len__(self):
        return len(self.premises)

def load_data_snli(batch_size, num_steps=50):
    """下载SNLI数据集并返回数据迭代器和词表"""
    num_workers = native.get_data_loader_workers()
    data_dir = native.download_extract('SNLI')
    train_data = read_snli(data_dir, True)
    test_data = read_snli(data_dir, False)
    train_set = SNLIDataset(train_data, num_steps)
    test_set = SNLIDataset(test_data, num_steps, train_set.vocab)
    train_iter = torch.utils.data.DataLoader(train_set, batch_size,
                                             shuffle=True,
                                             num_workers=num_workers)
    test_iter = torch.utils.data.DataLoader(test_set, batch_size,
                                             shuffle=False,
                                             num_workers=num_workers)
    return train_iter, test_iter, train_set.vocab

train_iter, test_iter, vocab = load_data_snli(128, 50)
len(vocab)

for X, Y in train_iter:
    print(X[0].shape)
    print(X[1].shape)
    print(Y.shape)
    break

```

第 15 章 • Attention NLI •

chapters/chapter_15_nlp_applications/natural_language_inference_attention.py

章节对应：第 15 章 类别：网络结构 代码块：11 个

结构图：Attention NLI（网络结构）



第 15 章基于注意力的自然语言推断。

阅读注释：阅读顺序：先看输入与输出，再看核心层/模块，最后看损失函数和训练循环。
先分别编码前提和假设，再用跨句注意力对齐信息，最后比较和聚合。

```
"""natural-language-inference-attention: 自然语言处理：应用"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import torch
from torch import nn
from torch.nn import functional as F

def mlp(num_inputs, num_hiddens, flatten):
    net = []
    net.append(nn.Dropout(0.2))
    net.append(nn.Linear(num_inputs, num_hiddens))
    net.append(nn.ReLU())
    if flatten:
        net.append(nn.Flatten(start_dim=1))
    net.append(nn.Dropout(0.2))
    net.append(nn.Linear(num_hiddens, num_hiddens))
    net.append(nn.ReLU())
    if flatten:
        net.append(nn.Flatten(start_dim=1))
    return nn.Sequential(*net)

class Attend(nn.Module):
    def __init__(self, num_inputs, num_hiddens, **kwargs):
        super(Attend, self).__init__(**kwargs)
        self.f = mlp(num_inputs, num_hiddens, flatten=False)

    def forward(self, A, B):
        # A/B的形状：（批量大小，序列A/B的词元数，embed_size）
        # f_A/f_B的形状：（批量大小，序列A/B的词元数，num_hiddens）
        f_A = self.f(A)
        f_B = self.f(B)
```

```

# e的形状: (批量大小, 序列A的词元数, 序列B的词元数)
e = torch.bmm(f_A, f_B.permute(0, 2, 1))
# beta的形状: (批量大小, 序列A的词元数, embed_size),
# 意味着序列B被软对齐到序列A的每个词元(beta的第1个维度)
beta = torch.bmm(F.softmax(e, dim=-1), B)

# beta的形状: (批量大小, 序列B的词元数, embed_size),
# 意味着序列A被软对齐到序列B的每个词元(alpha的第1个维度)
alpha = torch.bmm(F.softmax(e.permute(0, 2, 1), dim=-1), A)
return beta, alpha

class Compare(nn.Module):
    def __init__(self, num_inputs, num_hiddens, **kwargs):
        super(Compare, self).__init__(**kwargs)
        self.g = mlp(num_inputs, num_hiddens, flatten=False)

    def forward(self, A, B, beta, alpha):
        V_A = self.g(torch.cat([A, beta], dim=2))
        V_B = self.g(torch.cat([B, alpha], dim=2))
        return V_A, V_B

class Aggregate(nn.Module):
    def __init__(self, num_inputs, num_hiddens, num_outputs, **kwargs):
        super(Aggregate, self).__init__(**kwargs)
        self.h = mlp(num_inputs, num_hiddens, flatten=True)
        self.linear = nn.Linear(num_hiddens, num_outputs)

    def forward(self, V_A, V_B):
        # 对两组比较向量分别求和
        V_A = V_A.sum(dim=1)
        V_B = V_B.sum(dim=1)
        # 将两个求和结果的连接送到多层感知机中
        Y_hat = self.linear(self.h(torch.cat([V_A, V_B], dim=1)))
        return Y_hat

class DecomposableAttention(nn.Module):
    def __init__(self, vocab, embed_size, num_hiddens, num_inputs_attend=100,
                 num_inputs_compare=200, num_inputs_agg=400, **kwargs):
        super(DecomposableAttention, self).__init__(**kwargs)
        self.embedding = nn.Embedding(len(vocab), embed_size)
        self.attend = Attend(num_inputs_attend, num_hiddens)
        self.compare = Compare(num_inputs_compare, num_hiddens)
        # 有3种可能的输出: 蕴涵、矛盾和中性
        self.aggregate = Aggregate(num_inputs_agg, num_hiddens, num_outputs=3)

    def forward(self, X):
        premises, hypotheses = X
        A = self.embedding(premises)
        B = self.embedding(hypotheses)
        beta, alpha = self.attend(A, B)
        V_A, V_B = self.compare(A, B, beta, alpha)
        Y_hat = self.aggregate(V_A, V_B)
        return Y_hat

batch_size, num_steps = 256, 50
train_iter, test_iter, vocab = native.load_data_snli(batch_size, num_steps)

embed_size, num_hiddens, devices = 100, 200, native.try_all_gpus()
net = DecomposableAttention(vocab, embed_size, num_hiddens)
glove_embedding = native.TokenEmbedding('glove.6b.100d')
embeds = glove_embedding[vocab.idx_to_token]
net.embedding.weight.data.copy_(embeds);

lr, num_epochs = 0.001, 4
trainer = torch.optim.Adam(net.parameters(), lr=lr)
loss = nn.CrossEntropyLoss(reduction="none")

native.train_ch13(net, train_iter, test_iter, loss, trainer, num_epochs,
                 devices)

def predict_snli(net, vocab, premise, hypothesis):
    """预测前提和假设之间的逻辑关系"""
    net.eval()
    premise = torch.tensor(vocab[premise], device=native.try_gpu())
    hypothesis = torch.tensor(vocab[hypothesis], device=native.try_gpu())
    label = torch.argmax(net([premise.reshape((1, -1)),
                             hypothesis.reshape((1, -1))]), dim=1)
    return 'entailment' if label == 0 else 'contradiction' if label == 1 \

```

```

else 'neutral'

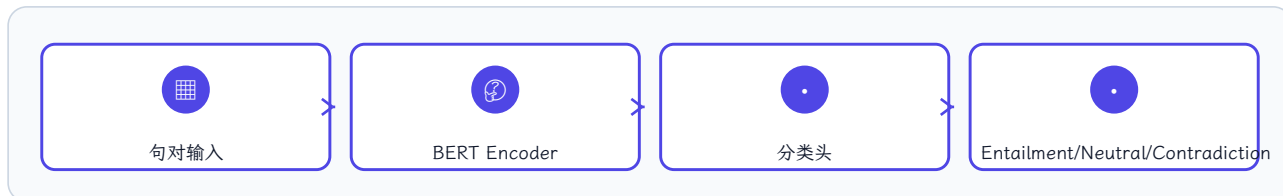
predict_snli(net, vocab, ['he', 'is', 'good', '.'], ['he', 'is', 'bad', '.'])

```

第 15 章 · BERT NLI · chapters/chapter_15_nlp_applications/natural_language_inference_bert.py

章节对应：第 15 章 类别：网络结构 代码块：9 个

结构图：BERT NLI（网络结构）



第 15 章用预训练 BERT 完成自然语言推断。

阅读注释：阅读顺序：先看输入与输出，再看核心层/模块，最后看损失函数和训练循环。
把句对作为一个带分隔符的输入交给 BERT，分类头读取句对级表示。

```

"""natural-language-inference-bert: 自然语言处理：应用"""

# 来源：依据《动手学深度学习（PyTorch 第二版）》的 D2L 中文 PyTorch 示例整理。
# 改写说明：已移除 d2l 命名空间，共享工具集中在 common.py。
from pathlib import Path
import sys
_ROOT = Path(__file__).resolve().parents[1]
if str(_ROOT) not in sys.path:
    sys.path.insert(0, str(_ROOT))

import math
import random
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils import data
import torchvision
from torchvision import transforms
from PIL import Image
from matplotlib import pyplot as plt
from IPython import display
import common as native

import common as native
import json
import multiprocessing
import torch
from torch import nn
import os

native.DATA_HUB['bert.base'] = (native.DATA_URL + 'bert.base.torch.zip',
                                '225d66f04cae318b841a13d32af3acc165f253ac')
native.DATA_HUB['bert.small'] = (native.DATA_URL + 'bert.small.torch.zip',
                                  'c72329e68a732bef0452e4b96a1c341c8910f81f')

def load_pretrained_model(pretrained_model, num_hiddens, ffn_num_hiddens,
                          num_heads, num_layers, dropout, max_len, devices):
    data_dir = native.download_extract(pretrained_model)
    # 定义空词表以加载预定义词表
    vocab = native.Vocab()
    vocab.idx_to_token = json.load(open(os.path.join(data_dir,
                                                    'vocab.json')))
    vocab.token_to_idx = {token: idx for idx, token in enumerate(
        vocab.idx_to_token)}
    bert = native.BERTModel(len(vocab), num_hiddens, norm_shape=[256],
                            ffn_num_input=256, ffn_num_hiddens=ffn_num_hiddens,
                            num_heads=4, num_layers=2, dropout=0.2,
                            max_len=max_len, key_size=256, query_size=256,
                            value_size=256, hid_in_features=256,

```

```

        mlm_in_features=256, nsp_in_features=256)
# 加载预训练BERT参数
bert.load_state_dict(torch.load(os.path.join(data_dir,
                                              'pretrained.params')))

return bert, vocab

devices = native.try_all_gpus()
bert, vocab = load_pretrained_model(
    'bert.small', num_hiddens=256, ffn_num_hiddens=512, num_heads=4,
    num_layers=2, dropout=0.1, max_len=512, devices=devices)

class SNLIBERTDataset(torch.utils.data.Dataset):
    def __init__(self, dataset, max_len, vocab=None):
        all_premise_hypothesis_tokens = [[
            p_tokens, h_tokens] for p_tokens, h_tokens in zip(
                *[native.tokenize([s.lower() for s in sentences])
                  for sentences in dataset[:2]])]

        self.labels = torch.tensor(dataset[2])
        self.vocab = vocab
        self.max_len = max_len
        (self.all_token_ids, self.all_segments,
         self.valid_lens) = self._preprocess(all_premise_hypothesis_tokens)
        print('read ' + str(len(self.all_token_ids)) + ' examples')

    def _preprocess(self, all_premise_hypothesis_tokens):
        pool = multiprocessing.Pool(4) # 使用4个进程
        out = pool.map(self._mp_worker, all_premise_hypothesis_tokens)
        all_token_ids = [
            token_ids for token_ids, segments, valid_len in out]
        all_segments = [segments for token_ids, segments, valid_len in out]
        valid_lens = [valid_len for token_ids, segments, valid_len in out]
        return (torch.tensor(all_token_ids, dtype=torch.long),
                torch.tensor(all_segments, dtype=torch.long),
                torch.tensor(valid_lens))

    def _mp_worker(self, premise_hypothesis_tokens):
        p_tokens, h_tokens = premise_hypothesis_tokens
        self._truncate_pair_of_tokens(p_tokens, h_tokens)
        tokens, segments = native.get_tokens_and_segments(p_tokens, h_tokens)
        token_ids = self.vocab[tokens] + [self.vocab['<pad>']] \
            * (self.max_len - len(tokens))
        segments = segments + [0] * (self.max_len - len(segments))
        valid_len = len(tokens)
        return token_ids, segments, valid_len

    def _truncate_pair_of_tokens(self, p_tokens, h_tokens):
        # 为BERT输入中的'<CLS>','<SEP>'和'<SEP>'词元保留位置
        while len(p_tokens) + len(h_tokens) > self.max_len - 3:
            if len(p_tokens) > len(h_tokens):
                p_tokens.pop()
            else:
                h_tokens.pop()

    def __getitem__(self, idx):
        return (self.all_token_ids[idx], self.all_segments[idx],
                self.valid_lens[idx]), self.labels[idx]

    def __len__(self):
        return len(self.all_token_ids)

# 如果出现显存不足错误, 请减少"batch_size"。在原始的BERT模型中, max_len=512
batch_size, max_len, num_workers = 512, 128, native.get_data_loader_workers()
data_dir = native.download_extract('SNLI')
train_set = SNLIBERTDataset(native.read_snli(data_dir, True), max_len, vocab)
test_set = SNLIBERTDataset(native.read_snli(data_dir, False), max_len, vocab)
train_iter = torch.utils.data.DataLoader(train_set, batch_size, shuffle=True,
                                         num_workers=num_workers)

test_iter = torch.utils.data.DataLoader(test_set, batch_size,
                                         num_workers=num_workers)

class BERTClassifier(nn.Module):
    def __init__(self, bert):
        super(BERTClassifier, self).__init__()
        self.encoder = bert.encoder
        self.hidden = bert.hidden

```

```
self.output = nn.Linear(256, 3)

def forward(self, inputs):
    tokens_X, segments_X, valid_lens_x = inputs
    encoded_X = self.encoder(tokens_X, segments_X, valid_lens_x)
    return self.output(self.hidden(encoded_X[:, 0, :]))

net = BERTClassifier(bert)

lr, num_epochs = 1e-4, 5
trainer = torch.optim.Adam(net.parameters(), lr=lr)
loss = nn.CrossEntropyLoss(reduction='none')
native.train_ch13(net, train_iter, test_iter, loss, trainer, num_epochs,
                  devices)
```